abLEMON — A User Manual

For abLEMON 1.9.0 (November 2026)

Donato Meoli

Contents

abLEMON — A User Manual **5**
Abstract . . . 5
Conventions . . . 5
Contents . . . 5

1. Introduction **7**
1.1 What abLEMON changes . . . 7
1.2 How the manual is organized . . . 8
1.3 Conventions . . . 9

2. Installation **10**
2.1 Packages . . . 10
2.2 Building from the sources . . . 10
2.3 Using the installed library . . . 11
2.4 Optional backends . . . 11
2.5 Exercises . . . 11

3. Graphs, items and maps **12**
3.1 A first example . . . 12
3.2 Items, identifiers and INVALID . . . 12
3.3 Iterators and ranges . . . 14
3.4 Maps . . . 14
3.5 Concepts and their checking . . . 15
3.6 The algorithm classes . . . 15
3.7 Tolerances and numeric traits . . . 17
3.8 Exercises . . . 18

4. Graph structures **19**
4.1 The graph concepts . . . 19
4.2 List graphs . . . 20
4.3 Smart graphs . . . 22
4.4 Static graphs . . . 22
4.5 Full graphs . . . 24
4.6 Grid and hypercube graphs . . . 25
4.7 Arc sets and edge sets . . . 26
4.8 Counting and finding arcs . . . 27
4.9 Copying graphs . . . 29
4.10 Exercises . . . 30

5. Graph adaptors **31**
5.1 Views of a graph . . . 31
5.2 Reversing and filtering . . . 32
5.3 Changing the orientation . . . 33
5.4 The residual digraph . . . 34
5.5 Splitting the nodes . . . 36
5.6 Exercises . . . 37

6. Maps **39**
6.1 The map concepts . . . 39
6.2 Graph maps . . . 39
6.3 Basic maps . . . 40
6.4 Map adaptors and arithmetic . . . 41
6.5 Maps of the items of a graph . . . 42
6.6 Iterable maps . . . 43
6.7 Map utilities . . . 45
6.8 Exercises . . . 46

7. Heaps **47**
7.1 The heap concept . . . 47
7.2 Implicit d-ary heaps . . . 48
7.3 Linked heaps . . . 50
7.4 Heaps of integer priorities . . . 52
7.5 Choosing a heap . . . 55
7.6 Exercises . . . 55

8. Paths and auxiliary structures **57**
8.1 Paths . . . 57
8.2 Union-find structures . . . 58
8.3 The elevator . . . 61
8.4 Two-dimensional geometry . . . 62
8.5 Radix sort . . . 63
8.6 Random numbers . . . 64
8.7 Counting and timing . . . 65
8.8 Colors . . . 65
8.9 Errors, assertions and the command line . . . 66
8.10 Exercises . . . 66

9. Graph search **68**
9.1 The common interface . . . 68
9.2 Breadth-first search . . . 69
9.3 Depth-first search . . . 70
9.4 Function-type interfaces . . . 71
9.5 Visitors . . . 71
9.6 Maximum cardinality search . . . 72
9.7 Exercises . . . 73

10. Shortest paths **75**
10.1 The problem and its optimality conditions . . . 75
10.2 Dijkstra's algorithm . . . 76
10.3 Shortest paths between two nodes . . . 78
10.4 The Bellman-Ford algorithm . . . 80
10.5 Arc-disjoint paths of least total length . . . 81
10.6 Exercises . . . 83

11. Spanning trees and arborescences **84**
11.1 Minimum spanning trees . . . 84
11.2 Minimum cost arborescences . . . 86
11.3 Exercises . . . 90

12. Maximum flows and circulations **92**
12.1 The problem and its dual . . . 92
12.2 The push-relabel algorithm: Preflow . . . 93
12.3 Augmenting paths: EdmondsKarp . . . 96
12.4 Feasible circulations: Circulation . . . 97
12.5 Tolerances and the common interface . . . 99
12.6 Exercises . . . 99

13. Minimum cost flows **101**
13.1 The problem, its forms and its dual . . . 101
13.2 The common interface . . . 103
13.3 Tolerances and floating-point data . . . 104
13.4 The network simplex method . . . 104
13.5 The starting flow and the infinite bounds . . . 107
13.6 Cost scaling . . . 108
13.7 Capacity scaling . . . 110
13.8 Cycle canceling . . . 111
13.9 Choosing an algorithm . . . 112
13.10 Exercises . . . 112

14. Certificates and the k best flows **114**
14.1 Certificates of infeasibility . . . 114
14.2 Certificates of unboundedness . . . 117
14.3 The k best integer flows: KBestFlows . . . 118
14.4 The k best extreme flows: KBestExtremeFlows . . . 120
14.5 Alternative optimal potentials . . . 123
14.6 Exercises . . . 124

15. Minimum cuts **125**
15.1 The problems . . . 125
15.2 The Hao-Orlin algorithm . . . 126
15.3 The Nagamochi-Ibaraki algorithm . . . 129
15.4 Gomory-Hu trees . . . 131
15.5 Choosing an algorithm . . . 133
15.6 Exercises . . . 134

16. Minimum mean cycles **135**
16.1 The problem . . . 135
16.2 Karp's algorithm . . . 136
16.3 The Hartmann-Orlin algorithm . . . 137
16.4 Howard's policy iteration . . . 139
16.5 The common interface . . . 140
16.6 Exercises . . . 142

17. Matchings **143**
17.1 Problems, duality and the choice of an algorithm . . . 143
17.2 Maximum cardinality matching in bipartite graphs . . . 145
17.3 Minimum cost assignment . . . 147
17.4 Maximum cardinality matching in general graphs . . . 149
17.5 Maximum weight matchings in general graphs . . . 150
17.6 Fractional matchings . . . 153
17.7 Exercises . . . 155

18. Connectivity, planarity, isomorphism **157**
18.1 Connected components, acyclicity and simple graphs . . . 157
18.2 Biconnectivity . . . 158
18.3 Strong components and topological order . . . 161
18.4 Euler tours . . . 162
18.5 Planarity, embedding and drawing . . . 163
18.6 Coloring planar graphs . . . 165
18.7 Subgraph isomorphism: VF2 and VF2++ . . . 167
18.8 Exercises . . . 169

19. Travelling salesman and maximum clique **170**
19.1 The symmetric travelling salesman problem . . . 170
19.2 Tour construction heuristics . . . 171
19.3 Improvement by 2-opt . . . 175
19.4 Choosing a heuristic, and an example . . . 176

19.5 The maximum clique problem . . . 177
19.6 The iterated local search of Grosso, Locatelli and Pullan . . . 177
19.7 The exact algorithm of Tomita and Seki . . . 179
19.8 Exercises . . . 180

20. The LP and MIP interface **182**
20.1 The structure of the interface . . . 182
20.2 Columns, rows and expressions . . . 183
20.3 Building and modifying a problem . . . 184
20.4 Solving an LP and reading the solution . . . 185
20.5 Mixed integer programs . . . 186
20.6 The backends . . . 187
20.7 The skeletons . . . 188
20.8 An example . . . 188
20.9 Exercises . . . 188

21. Input and output **190**
21.1 The LEMON Graph Format . . . 190
21.2 Reading . . . 191
21.3 Writing . . . 192
21.4 Extra sections and the contents of a file . . . 193
21.5 The DIMACS formats . . . 194
21.6 The graph6 format of nauty . . . 195
21.7 Drawing a graph in EPS . . . 195
21.8 Examples . . . 196
21.9 Exercises . . . 196

22. The tools **198**
22.1 Building and calling the tools . . . 198
22.2 dimacs-solver . . . 198
22.3 dimacs-to-lgf . . . 199
22.4 lgf-gen . . . 200
22.5 lemon-0.x-to-1.x.sh and the demos . . . 201
22.6 An example . . . 201
22.7 Exercises . . . 201

A. Writing a graph, a map or a heap **203**
A.1 The concepts as contracts . . . 203
A.2 The base class and the extenders . . . 203
A.3 The alteration notifier . . . 204
A.4 Writing a map . . . 206
A.5 Writing a heap . . . 206
A.6 Checking a new class . . . 207

Y. Glossary **208**

Z. Bibliography **215**

abLEMON — A User Manual

For abLEMON 1.9.0 (November 2026).

Abstract

abLEMON is a C++ template library of graphs, maps, heaps and network optimization algorithms, maintained as a fork of LEMON 1.3.1, whose headers and namespace it keeps. This manual describes the library as it is implemented, family by family: the graph structures and the adaptors that present a graph as another, the maps through which data are attached to nodes, arcs and edges, the heaps and the other auxiliary structures, and the algorithms for graph search, shortest paths, spanning trees, maximum and minimum cost flows (with the certificates of infeasibility and unboundedness and the k best flows that abLEMON adds), minimum cuts, minimum mean cycles, matchings, connectivity, planarity and isomorphism, the heuristics for the travelling salesman and the maximum clique, and the interface to LP and MIP solvers. For each algorithm the manual states the problem it solves, the conditions it maintains, its complexity as implemented and the choices it leaves to the user, gives its pseudocode, and proves its correctness and its complexity; programs that show each class at work are distributed with the manual and run as tests of the library. Yet, it does not replace the reference documentation, which lists every member with its exact signature, and to which the chapters refer when a detail is not worth repeating.

Conventions

The headings carry the numbers of the manual, and sections, figures, tables, algorithms, theorems, lemmas and exercises are numbered by chapter. The programs named by the chapters are the files of `manual/examples/`, each of which is a test of the library (`ctest -R manual_`). A note headed "abLEMON" marks what abLEMON adds to LEMON 1.3.1, or where its behavior differs from it; the rest of the library behaves as in LEMON. Citations refer to the bibliography (Appendix Z), and the terms defined in the chapters are collected in the glossary (Appendix Y).

Contents

Part I: Foundations

- 1. Introduction
- 2. Installation
- 3. Graphs, items and maps

Part II: Data structures

- 4. Graph structures
- 5. Graph adaptors
- 6. Maps
- 7. Heaps
- 8. Paths and auxiliary structures

Part III: Algorithms

- 9. Graph search
- 10. Shortest paths
- 11. Spanning trees and arborescences

- 12. Maximum flows and circulations
- 13. Minimum cost flows
- 14. Certificates and the k best flows
- 15. Minimum cuts
- 16. Minimum mean cycles
- 17. Matchings
- 18. Connectivity, planarity, isomorphism
- 19. Travelling salesman and maximum clique
- 20. The LP and MIP interface

Part IV: Input, output and tools

- 21. Input and output
- 22. The tools

Part V: Appendices

- A. Writing a graph, a map or a heap
- Y. Glossary
- Z. Bibliography

1. Introduction

We are concerned with abLEMON, a C++ template library of graphs and of the algorithms of network optimization built on them. It provides the structures that represent directed, undirected and bipartite graphs, the maps that attach data to their nodes, arcs and edges, the heaps and the other auxiliary structures that the algorithms need, and the algorithms themselves (searches, shortest paths, spanning trees, maximum and minimum cost flows, minimum cuts and mean cycles, matchings, connectivity, planarity and isomorphism, heuristics for the travelling salesman and the maximum clique, and an interface to LP and MIP solvers). abLEMON is a maintained fork of LEMON, the Library for Efficient Modeling and Optimization in Networks of the Egerváry Research Group of the Eötvös Loránd University of Budapest (Dezső, Jüttner and Kovács 2011), which has had no release since 1.3.1 (2014). It starts from the last revision of the Mercurial repository of LEMON, whose history it keeps, and it keeps its headers (`lemon/`) and its namespace (`lemon`); hence, a program written for LEMON usually compiles with abLEMON unchanged.

This manual describes the library as it is implemented in abLEMON 1.9.0. Each public class of `lemon/` is described in the chapter of its family, with the theory that one needs to choose among the algorithms and to read their results, and the statements on costs, defaults and behavior have been checked against the code. The manual is a companion to the reference documentation that Doxygen produces from the headers (https://dmeoli.github.io/abLEMON/), which gives each member with its exact signature. We give instead the problem that each class solves, the conditions it maintains, how the classes are meant to be combined, and the pseudocode of each algorithm with the proofs of its properties; the programs that use the classes are not printed in the chapters, but kept in `manual/examples/`, where each is compiled and run as a test of the library (cf. chapter 2) that checks what the chapter states. We assume that the reader writes C++ and knows the basic notions of graph theory and of linear programming. The network optimization that a chapter needs is recalled in it, with references to the textbook of Ahuja, Magnanti and Orlin (1993) and to the papers of each algorithm.

1.1 What abLEMON changes

The differences from LEMON 1.3.1 fall into four groups, which NEWS lists release by release and which the chapters mark with a note "abLEMON" where they apply. The first group is portability: the library compiles without warnings from C++11 to C++23 with GCC, Clang, AppleClang and MSVC. For instance, the allocators go through `std::allocator_traits`, since C++20 removed the members of `std::allocator` that `ArrayMap` and `ListPath` called; also, each header is self-contained, and the headers survive the macros `min` and `max` of `<windows.h>`. The second group is correctness. Several defects of the algorithms and of the data structures are fixed, most of them with a test that failed before the fix (e.g., `RadixHeap` mixed the items of adjacent boxes, `BinomialHeap` truncated fractional priorities, and `Circulation` read past its storage on an empty digraph), and the items of the graphs are `INVALID` when they are default-constructed.

The third group concerns the minimum cost flow algorithms (chapter 13 and chapter 14). These solve problems with floating-point data, under tolerances relative to the data, and they certify an infeasible or unbounded answer (with a set of nodes whose supply exceeds what can leave it, or with a cycle of negative cost and infinite capacity). Furthermore, they give the k most violated such sets, and two further classes give the k best integer flows and the k best extreme flows; alternative optimal potentials are also available. The fourth group is a set of new algorithms and structures, i.e., `HopcroftKarp` and `MinCostAssignment` for bipartite matching (chapter 17), `BidirDijkstra` and `AStar` for the shortest path between two nodes (chapter 10), `TomitaSekiMc` for the exact maximum clique (chapter 19), and `StaticGraph`, the undirected counterpart of `StaticDigraph` (chapter 4). Also, with CMake 3.28 and Clang 16 or MSVC 19.34, the library may be built as the C++20 module `lemon` (whose interface is `lemon/lemon.cppm`, cf. §2.3).

1.2 How the manual is organized

Part I gives what one needs before anything else: the installation (chapter 2) and the model of the library (chapter 3). In this model graphs have nodes and arcs that are handles, data are attached to them by maps, concepts state what an algorithm requires of the types it is given, and algorithms are classes that are configured, run and then queried. Part II describes the data structures, i.e., the graphs, the adaptors that present a graph as another one without copying it, the maps, the heaps, the paths and the auxiliary structures. Part III describes the algorithms, family by family: each chapter states the problem, the optimality conditions and the complexity of each algorithm, gives its pseudocode with the theorems on its correctness and its complexity, compares the variants that the library offers, names the programs and the tests that check it, and ends with exercises. Part IV covers the formats of input and output (LGF, DIMACS, nauty, PostScript) and the tools of `tools/`. Appendix A explains how a new graph, map or heap is written so that the algorithms accept it, while Appendices Y and Z hold the glossary and the bibliography.

A reader who only needs one algorithm may read chapter 3 and then the chapter of that algorithm, since the chapters of Part III are meant to be read independently of each other. They refer to each other where one algorithm uses another one; e.g., the certificates of chapter 14 rest on the maximum flows of chapter 12. Table 1.1 gives the chapter where each header of `lemon/` is described (the internal headers of `lemon/bits/` only where a user meets them). In it, `core.h` holds the items of the graphs, the basic utilities and the common definitions, and is included by all the other headers; `config.h`, instead, is written by CMake when the library is configured (cf. §2.4), and `lemon.cppm` is the interface of the C++20 module (cf. §2.3).

Table 1.1. The headers of `lemon/` by chapter.

Chapter	Headers
2	`config.h`, `lemon.cppm`
3	`core.h` (items), `concept_check.h`, `tolerance.h`, `math.h`, `bits/traits.h`
4	`list_graph.h`, `smart_graph.h`, `static_graph.h`, `compact_graph.h`, `full_graph.h`, `grid_graph.h`, `hypercube_graph.h`, `edge_set.h`, `core.h` (utilities), `concepts/digraph.h`, `concepts/graph.h`, `concepts/bpgraph.h`, `concepts/graph_components.h`
5	`adaptors.h`
6	`maps.h`, `concepts/maps.h`
7	`bin_heap.h`, `quad_heap.h`, `dheap.h`, `fib_heap.h`, `pairing_heap.h`, `binomial_heap.h`, `radix_heap.h`, `bucket_heap.h`, `concepts/heap.h`
8	`path.h`, `concepts/path.h`, `unionfind.h`, `elevator.h`, `dim2.h`, `radix_sort.h`, `random.h`, `counter.h`, `time_measure.h`, `color.h`, `error.h`, `assert.h`, `arg_parser.h`
9	`bfs.h`, `dfs.h`, `max_cardinality_search.h`
10	`dijkstra.h`, `bidir_dijkstra.h`, `astar.h`, `bellman_ford.h`, `suurballe.h`
11	`kruskal.h`, `min_cost_arborescence.h`
12	`preflow.h`, `edmonds_karp.h`, `circulation.h`, `concepts/max_flow.h`
13	`network_simplex.h`, `cost_scaling.h`, `capacity_scaling.h`, `cycle_canceling.h`, `concepts/min_cost_flow.h`
14	`k_best_flows.h`, `k_best_extreme_flows.h`
15	`hao_orlin.h`, `nagamochi_ibaraki.h`, `gomory_hu.h`
16	`karp_mmc.h`, `hartmann_orlin_mmc.h`, `howard_mmc.h`, `concepts/min_mean_cycle.h`
17	`matching.h`, `bipartite_matching.h`, `fractional_matching.h`
18	`connectivity.h`, `euler.h`, `planarity.h`, `vf2.h`, `vf2pp.h`
19	`nearest_neighbor_tsp.h`, `greedy_tsp.h`, `insertion_tsp.h`, `christofides_tsp.h`, `opt2_tsp.h`, `grosso_locatelli_pullan_mc.h`, `tomita_seki_mc.h`
20	`lp_base.h`, `lp.h`, `lp_skeleton.h`, `glpk.h`, `clp.h`, `cbc.h`, `cplex.h`, `soplex.h`
21	`lgf_reader.h`, `lgf_writer.h`, `dimacs.h`, `nauty_reader.h`, `graph_to_eps.h`

Chapter	Headers
A	`bits/graph_extender.h`, `bits/alteration_notifier.h`

1.3 Conventions

Class names, members and code are set in `monospace`. A class is named without the namespace `lemon`, and a member is written as `Class::member()` when the class is not clear from the context. A graph has n nodes and m arcs (or edges), while C and U denote the largest absolute value of a cost and of a capacity when the data are integer. The complexities are those of the implementation, and the chapters say so when they differ from the bounds of the literature. Sections, figures, tables, algorithms, lemmas, theorems and exercises are numbered by chapter (lemmas and theorems with separate counters), and a proof ends with □. A program is named by the file of `manual/examples/` that holds it, and therefore by the test `manual_<name>` that runs it (the hyphens of the name becoming underscores).

2. Installation

We now describe how abLEMON is obtained, either as a package or from the sources, how a program finds it once it is installed, and which of its parts are optional; a first program, which only needs a compiler and the headers, is described in chapter 3. Most of abLEMON is made of templates, and therefore lives in its headers. The compiled library (`libemon`, a name kept from LEMON) holds the few parts that are not templates, i.e., the random number generator, the parser of the command line, the colors of `graph_to_eps.h`, the default tolerances of `tolerance.h`, the locking on Windows and, when they are enabled, the interfaces to the LP and MIP solvers of chapter 20.

2.1 Packages

The recipes of the packages are in `packaging/` of the repository, and each release brings them to the archive of its tag (the workflow `release.yml` does so, after taking the notes of the release from NEWS). The Homebrew tap of the project works on macOS and Linux:

```
brew install dmeoli/ablemon/ablemon
```

Also, the recipes for conda-forge (package `ablemon`), vcpkg (port `ablemon`) and Conan have been submitted to the respective indices; at the time of writing (abLEMON 1.9.0) they are still under review. Until they are accepted, the recipes in `packaging/` can be used locally, e.g., as an overlay of vcpkg (`vcpkg install ablemon --overlay-ports=packaging/vcpkg`). Each package installs the headers, the library and the CMake configuration of §2.3. None of them enables the LP and MIP backends of §2.4, which one builds from the sources when they are needed.

2.2 Building from the sources

Building needs CMake 3.19 or later and a compiler that supports C++11. In fact, the library compiles without warnings with GCC, Clang, AppleClang and MSVC, from C++11 to C++23 (the continuous integration builds each change on Linux, macOS and Windows with these compilers, and also under the address and undefined behavior sanitizers). The usual sequence configures the build directory by `cmake -S . -B build -DCMAKE_BUILD_TYPE=Release`, compiles by `cmake --build build`, runs the tests by `ctest --test-dir build`, and installs by `cmake --install build --prefix /usr/local`; the tests are those of `test/` and the programs of this manual (the files of `manual/examples/`, which ctest names `manual_*`), and the installation follows `GNUInstallDirs` under the given prefix (e.g., the library goes to `lib64` or to `lib/x86_64-linux-gnu` when the system wants it there).

Among the options, `CMAKE_CXX_STANDARD` is honored when it is set, and is 11 otherwise; `BUILD_SHARED_LIBS` builds a shared library (on Windows, a DLL). `LEMON_BUILD_TESTS`, `LEMON_BUILD_DEMOS`, `LEMON_BUILD_TOOLS` and `LEMON_BUILD_DOC` are `ON` by default, and each of them leaves out a part of the tree when it is `OFF` (the tests and the programs of the manual, the demos and the contributed programs, the tools, and the documentation, respectively). `LEMON_BUILD_MODULE` is `OFF` by default (cf. §2.3), and the four options of the LP and MIP backends are described in §2.4. The variable `LEMON_THREADING` (one of `Pthread`, `Win32` and `None`) chooses the lock that the alteration notifiers of appendix A use, and it defaults to the threads that CMake finds. The version is the value given by `-DLEMON_VERSION=...`, if any (as the packages do), else the one of the environment variable `LEMON_VERSION`, else the tag that `git describe` gives for the sources. It is `unknown` when none of these exists. The build types `Debug` and `Maintainer` define `LEMON_ENABLE_DEBUG`, which turns on the checks of `assert.h` (cf. chapter 8); furthermore, `Maintainer` runs the tests at each build. A project may also

take the library in by `add_subdirectory()` or by `FetchContent`, in which case only the library is configured, whatever the options say.

For a project built with GNU make, the makefiles of the repository build the library and the tests without a configuration step:

```
make -f makefile-lib            # lib/libemon.a
make -C test check              # the tests, built and run
```

Also, `makefile-c`, which the makefile of a project includes, defines the objects of the library (`LEMONOBJ`), the external libraries it needs (`LEMONLIB`), its headers (`LEMONH`) and the `-I` directive (`LEMONINC`); it takes as input the compiler, its options and the directory of the sources.

2.3 Using the installed library

A CMake project finds the installed library by its configuration, which `find_package()` accepts both as `LEMON` and as `lemon`:

```
find_package(LEMON CONFIG REQUIRED)
target_link_libraries(main PRIVATE LEMON::lemon)
```

Note that the target `LEMON::lemon` carries the include directories, the compile definitions and, when the library was built with them, the LP and MIP backends; the variables `LEMON_INCLUDE_DIRS` and `LEMON_LIBRARIES` are also set, for the projects that use variables instead of targets. When the version is a number, a version file is installed as well, so that `find_package(LEMON 1.9)` accepts any later version with the same major number. Also, the installation may be moved to another prefix, since the configuration only uses paths relative to it, and other build systems use pkg-config (`pkg-config --cflags --libs lemon`). The headers are included as `<lemon/...>`, and the library is in the namespace `lemon`, as in LEMON 1.3.1.

With `-DLEMON_BUILD_MODULE=ON`, the library is also built and installed as the C++20 module `lemon`, whose interface is `lemon/lemon.cppm`; a translation unit linked to `LEMON::lemon_module` may then write `import lemon;` in place of the includes. This needs CMake 3.28 or later, the Ninja generator (or one of Visual Studio), and Clang 16 (with its `clang-scan-deps`) or MSVC 19.34 or later. When one of these conditions does not hold, the option is turned off with a warning that names it. GCC (up to version 15, at least) builds the module, but a unit that imports it does not find the operators of the standard library that the templates use; hence, GCC is not accepted for it.

2.4 Optional backends

The LP and MIP interface of chapter 20 needs at least one external solver: GLPK (`LEMON_ENABLE_GLPK`), CPLEX (`LEMON_ENABLE_ILOG`), CLP and CBC of COIN-OR (`LEMON_ENABLE_COIN`), or SoPlex (`LEMON_ENABLE_SOPLEX`). Each option is `YES` by default, i.e., the solver is used if CMake finds it, through its own CMake configuration where it has one. Those found are named by their file names in the installed configuration, so that a project that links `LEMON::lemon` also links them. The variables `LEMON_DEFAULT_LP` and `LEMON_DEFAULT_MIP` choose which of the solvers found the type aliases `Lp` and `Mip` of `lemon/lp.h` refer to; when they are not set, or name a solver that was not found, CMake takes the first one found in the order CPLEX, CLP (CBC for `Mip`), GLPK and SoPlex (which has no MIP). This choice is recorded in `lemon/config.h`, which CMake writes in the build directory, and whose macros `LEMON_HAVE_LP` and `LEMON_HAVE_MIP` tell a program whether there is a solver at all; when there is none, the two aliases are not defined, while the rest of the library is unaffected.

2.5 Exercises

Exercise 2.1. Build the library with `-DCMAKE_CXX_STANDARD=20`, run `ctest --test-dir build -R manual_`, and check that the programs of this manual pass. Then install the library to a temporary prefix, and build the program `manual/examples/03-first-program.cc` of chapter 3 against the installation, once with `find_package(LEMON)` and once with pkg-config.

Exercise 2.2. Configure the library on a machine where GLPK is installed, and check in the output of CMake and in `lemon/config.h` of the build directory which solver `Lp` and `Mip` refer to; the test `lp_test` exercises them.

3. Graphs, items and maps

We are concerned in this chapter with the few ideas on which all the headers of the library are built, and which a reader needs before the data structures of Part II and the algorithms of Part III. A graph is a set of items (nodes, arcs and, if it is undirected, edges), which are handled by small value types, visited by iterators and compared with the constant `INVALID`. Data are attached to the items only through maps, i.e., objects that associate a value with each item. The requirements on a graph, a map or a heap are written as concept classes, against which a type can be checked at compile time, and the algorithms are classes that follow (with few exceptions) one pattern of construction, execution and query. These ideas come from LEMON, whose design is described in Dezső, Jüttner and Kovács 2011, and abLEMON keeps them unchanged. Section 3.1 gives a first example; sections 3.2 to 3.4 describe the items, the iterators and the maps, section 3.5 the concepts, section 3.6 the algorithm classes, and section 3.7 the tolerances with which the library compares numbers that may carry a rounding error, together with the numeric constants and the traits of the types, while section 3.8 collects the exercises. In all the chapters we write $D = (V, A)$ for a digraph and $G = (V, E)$ for an undirected graph, with $n = |V|$ and $m = |A|$ (or $m = |E|$), and $\delta^+(v)$ and $\delta^-(v)$ for the sets of the arcs that leave and enter a node v.

3.1 A first example

We begin with the digraph of Figure 3.1, with four nodes s, a, b, t and five arcs, each with a cost, on which we want the shortest path from s to t. A program that computes it goes through the four steps that every use of the library repeats. (i) It builds the structure, here a `ListDigraph` (chapter 4), which is empty when constructed and grows by `addNode()` and `addArc(s, t)`, each of which returns a handle to the new item (section 3.2). (ii) It attaches the costs to the arcs by a map, here a `ListDigraph::ArcMap< int >`, which holds one `int` for each arc and is read and written by `cost[a]` (section 3.4), since an arc has no field that a user could set. (iii) It constructs the algorithm, the class template `Dijkstra` of chapter 10, on the digraph and on the map, and runs it from the source by `run(s)`. (iv) It reads the results by queries such as `dist(t)` and `predNode(v)` (section 3.6), the second of which gives the node that precedes v on the shortest path found, so that the path is followed backwards from t to s. The program `manual/examples/03-first-program.cc` (the test `manual_03_first_program`) follows these steps, prints the identifiers of the nodes of the path, and checks that the distance of t is 5. As all the programs of the manual, it is built and run by the tests of the library, while how a user program is built against an installed abLEMON is explained in chapter 2.

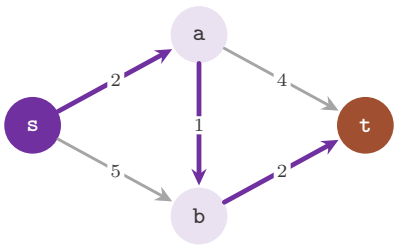


Figure 3.1. The digraph of the first example, with the cost of each arc; the arcs of the shortest path from s to t, of cost 5, are in the primary colour.

3.2 Items, identifiers and INVALID

A digraph has two kinds of items, of the types `Node` and `Arc` nested in its class. An undirected graph also has the type `Edge`: each edge e has two end nodes, `u(e)` and `v(e)`, and it is represented by two arcs of opposite directions,

which the graph creates and erases together with the edge. A bipartite graph (`BpGraph`) is an undirected graph whose nodes are either red or blue, and whose edges join a red node to a blue one; its types `RedNode` and `BlueNode` derive from `Node`. All these types are handles. An object of type `Node` does not contain a node but identifies one, typically by an integer index, and it is "cheap" to copy, to compare and to store. We remark that a handle is meaningful only together with its graph (the same index may identify another node in another graph), and the graph is therefore passed explicitly to each function that reads the structure, as in `g.source(a)` or `g.id(v)`.

The constant INVALID. In `lemon/core.h` one finds the empty type `Invalid` (whose operators `==`, `!=` and `<` find any two of its objects equal) and its only object, `INVALID`. Indeed, each handle and each iterator converts from `INVALID`, and comparing an item with `INVALID` reveals whether it identifies an item at all. In this way an iteration ends (section 3.3), a function such as `findArc()` of chapter 4 returns the absence of a result, and an algorithm marks a node without a predecessor (e.g., the source of a search has `predArc(s) == INVALID`). Also, the handles have the operators `==`, `!=` and `<`, the last of which is an artificial order (unrelated to the order of iteration, and, in the structures of chapter 4, the order of the identifiers) that lets the handles be the keys of ordered containers.

abLEMON. A default-constructed handle (`ListDigraph::Node v;`) is `INVALID` in all the graph structures and adaptors of abLEMON, while in LEMON 1.3.1 its index was left indeterminate in most of them (and the maps copied that indeterminate value when they built their default values). Yet, the concepts still say that the default constructor leaves the value undefined; hence, a generic algorithm should assign `INVALID` explicitly instead of relying on it.

Identifiers. Each item has an integer identifier, `g.id(item)`, which is unique among the items of its kind in the graph and does not change while the item exists, while `g.nodeFromId(i)`, `g.arcFromId(i)` and `g.edgeFromId(i)` give the item back, and `g.maxNodeId()`, `g.maxArcId()` and `g.maxEdgeId()` give an upper bound on the identifiers in use. Identifiers are the indices of the vectors that store the items and the values of the maps, and they are not necessarily contiguous: a `ListDigraph`, e.g., keeps the identifiers of the erased nodes on a free list and gives them to the next nodes added. An erased node may thus become valid again, and the structures that allow erasing (the `List` ones, and the `Smart` ones through their snapshots), as well as `StaticGraph` and `FullBpGraph`, have the function `valid(item)`, which returns whether a handle identifies an item of the graph at present. In fact, a bipartite graph also numbers the red and the blue nodes separately. Thus, `g.id(RedNode)` is the index of a node among the red ones (bounded by `g.maxRedId()`), and it usually differs from `g.id(Node)` of the same node. Finally, the generic forms `g.maxId(Node())` and `g.fromId(i, Node())`, with a dummy item that selects the kind, serve the templates that work on any kind of item.

Edges and arcs. Each edge has an inherent orientation, from `u(e)` to `v(e)`, given by the order in which its end nodes were passed to `addEdge(u, v)`. Of the two arcs of an edge, the forward one goes from `u(e)` to `v(e)` and the backward one from `v(e)` to `u(e)`. These arcs are `g.direct(e, true)` and `g.direct(e, false)`, while `g.direct(e, w)` is the arc of e whose source is the node `w`. Also, `g.direction(a)` is `true` for a forward arc and `g.oppositeArc(a)` is the other arc of the same edge. An `Arc` converts to the `Edge` it represents (so that an edge map can be read with an arc as its key), and `g.oppositeNode(w, e)` is the end of e other than `w`. In the undirected structures of chapter 4 (the list, smart, static and implicit graphs, and the edge sets) the edge of identifier i is represented by the arcs of identifiers $2i+1$ (forward) and $2i$ (backward), as Figure 3.2 shows. Since the adaptors of chapter 5 number their arcs in other ways, a program should usually use `direct()` and `direction()` instead of this arithmetic. A loop, i.e., an edge with $u(e) = v(e)$, also has two arcs, each of which leaves and enters the same node.

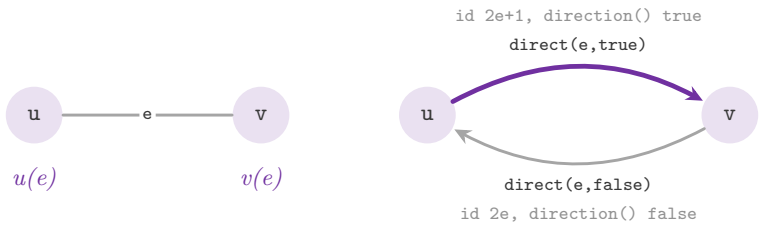


Figure 3.2. An edge e of a ListGraph, with its inherent orientation from u(e) to v(e), and the two arcs that represent it, with their identifiers.

In a bipartite graph the inherent orientation of each edge goes from its red end to its blue end, which `g.redNode(e)` and `g.blueNode(e)` return, while `g.red(v)` and `g.blue(v)` give the colour of a node and `g.asRedNode(v)` converts a `Node` to a `RedNode` (returning `INVALID` if the node is blue). Its variant `g.asRedNodeUnsafe(v`

`)` omits the check, which makes it somewhat faster, but it is correct only on red nodes and on `INVALID`; the functions of the blue nodes are symmetric. Since an undirected graph is also a digraph (with two arcs for each edge), any algorithm written for digraphs runs on it. Furthermore, the function `undirected(g)` of `lemon/core.h` returns whether the type of `g` is undirected; its value is selected at compile time from the tag `UndirectedTag` that the undirected structures define.

3.3 Iterators and ranges

Items are visited by iterators, which are nested classes of the graph. `NodeIt`, `ArcIt` and `EdgeIt` visit all the nodes, arcs and edges; `OutArcIt` and `InArcIt` visit the arcs that leave and enter a node, `IncEdgeIt` the edges incident to a node, and, in a bipartite graph, `RedNodeIt` and `BlueNodeIt` the nodes of one colour. An iterator derives from the handle of the items it visits; hence, it is used wherever a handle is expected, with no dereferencing, and the usual loop is

```
for( ListDigraph::OutArcIt a( g , v ) ; a != INVALID ; ++a )
 total += cost[ a ];
```

in which the iterator is constructed on the graph (and on the node, for the iterators of incident items), advanced by `++`, and compared with `INVALID`, which it becomes after the last item. Each iterator also has a constructor from the graph and an item, which continues the iteration from that item.

We call base node of an incidence iterator `it` the node on which it was constructed. Then `g.baseNode(it)` is the base node and `g.runningNode(it)` the other end of the current item: for `OutArcIt` they are the source and the target of the arc, for `InArcIt` the target and the source, and for `IncEdgeIt` the node of the iterator and the opposite end of the edge. In an undirected graph `OutArcIt` and `InArcIt` visit the arcs of the incident edges in the two directions, whereas `IncEdgeIt` visits the edges. A loop is visited twice by the first two, once for each of its arcs, and also twice by `IncEdgeIt` in `ListGraph` and `SmartGraph`. Note that the order of iteration is not specified by the concepts, and it differs between the structures (e.g., `ListDigraph`, `SmartDigraph` and the static structures visit their nodes from the last added, or the one of largest index, to the first); a program should not depend on it.

For the range-based `for` of C++11, the graphs also have functions that return a range of the same items, since a range is easier to pass to a loop or to an algorithm than a pair of iterators. These are `g.nodes()`, `g.arcs()`, `g.edges()`, `g.outArcs(v)`, `g.inArcs(v)`, `g.incEdges(v)` and, in a bipartite graph, `g.redNodes()` and `g.blueNodes()`, so that one may write `for (auto a : g.outArcs(v))`. A range holds a pair of wrapped iterators (`LemonRangeWrapper1` and `LemonRangeWrapper2` of `lemon/bits/stl_iterators.h`), which define the member types of an input iterator, a dereference that returns the item, and a postfix `++`; hence, the algorithms of the standard library that read an input range (e.g., `std::count_if` or `std::copy`) work on them too.

abLEMON. The ranges come from the repository of LEMON after its release 1.3.1, and they are distributed for the first time in abLEMON. Their iterators define the member types of an input iterator directly, instead of deriving them from `std::iterator`, which C++17 deprecated.

Writing the names of the nested types becomes tedious in a function that uses several of them, and `lemon/core.h` has macros that define them in the current scope. `DIGRAPH_TYPEDEFS(Digraph)` defines `Node`, `NodeIt`, `Arc`, `ArcIt`, `InArcIt` and `OutArcIt`, with the maps `BoolNodeMap`, `IntNodeMap`, `DoubleNodeMap`, `BoolArcMap`, `IntArcMap` and `DoubleArcMap` of that type. `GRAPH_TYPEDEFS(Graph)` adds `Edge`, `EdgeIt`, `IncEdgeIt` and the three edge maps `BoolEdgeMap`, `IntEdgeMap` and `DoubleEdgeMap`, and `BPGRAPH_TYPEDEFS(BpGraph)` adds `RedNode`, `RedNodeIt`, `BlueNode`, `BlueNodeIt` and their maps (`BoolRedNodeMap` and so on). Inside a template, where the graph type is a dependent name, one writes `TEMPLATE_DIGRAPH_TYPEDEFS(GR)` (and its counterparts `TEMPLATE_GRAPH_TYPEDEFS` and `TEMPLATE_BPGRAPH_TYPEDEFS`), which inserts the keywords `typename` and `template` that the compiler then requires. In abLEMON the types defined by these macros are marked as possibly unused by the macro `LEMON_UNUSED_TYPEDEF` (with GCC and Clang, the attribute `unused`; empty with the other compilers); hence, a function that uses only some of them compiles without warnings. LEMON 1.3.1 turned the warning on unused local typedefs off by a pragma, for the whole compilation of the user.

3.4 Maps

A graph stores its structure and no other datum. A cost, a capacity, a flow, a label or the distance computed by an algorithm is kept in a map. This is an object that has the types `Key` and `Value` and that offers `operator[](key)` to read a value, `set(key, value)` to write one, or both (the concepts that make this precise are given

in chapter 6, together with the maps of `lemon/maps.h`). Each graph structure provides the standard maps of its items, namely `NodeMap< V >`, `ArcMap< V >`, `EdgeMap< V >` and, in a bipartite graph, `RedNodeMap< V >` and `BlueNodeMap< V >`. Such a map stores a value of type `V` for each item of the graph on which it is constructed, in a vector indexed by the identifiers, and `map[item]` reads and writes it in $O(1)$ time. Furthermore, the map follows the changes of the graph, since the graph informs its maps of each item added or erased (cf. appendix A): a map constructed on `g` gets a slot for each item added to `g` afterwards, whose value is `V()` (not the value given to the constructor), and it drops the slot of each item erased. Of the two constructors, `NodeMap( g )` value-initializes the values and `NodeMap( g , value )` sets them all to `value`; we remark that the standard maps can be neither copied nor assigned, and the values of one map are copied into another by `mapCopy()` of chapter 6.

An algorithm reads and writes its data through maps whose types are template parameters; hence, any object with the right interface may be passed in place of a standard map, which is often (but not always) the most convenient choice. A length equal for all the arcs may be a `ConstMap`, and a cost may be computed on the fly from two other maps by an arithmetic adaptor (e.g., their difference). Also, a result may go to a map of the caller or to a `NullMap`, which discards what is written to it. The program `manual/examples/03-items-maps.cc` (the test `manual_03_items_maps`) checks the statements of sections 3.2 to 3.4 on a `ListGraph`. These are the two arcs of an edge and their directions, the default-constructed `INVALID`, the iterators and the ranges, the value `V()` that a map gives to an edge added after it, the reuse of an identifier after an erasure, and a function template whose graph type is checked against the concept of undirected graphs (section 3.5).

3.5 Concepts and their checking

A function template of the library (an algorithm, a map adaptor, a copy function) accepts any type that offers the operations it uses. These operations are written down, once for each kind of object, as a concept class in `lemon/concepts/`. A concept class is an archetype, which declares with empty bodies the types and the functions that the concept requires (e.g., `concepts::Digraph` declares `Node`, `Arc`, the iterators, `source()`, `target()`, `id()` and the maps); hence, a template that compiles with the archetype uses only what the concept guarantees. Each concept class also contains a nested template `Constraints< T >`, whose member function `constraints()` exercises those operations on a candidate type `T`. Those of the graphs (`concepts::Digraph`, `concepts::Graph` and `concepts::BpGraph`, with the components of which they are made) are described in chapter 4. Those of the maps are in chapter 6, those of the heaps in chapter 7, and the others (paths, maximum flows, minimum cost flows, minimum mean cycles) in the chapters of their families.

Checking is done by two function templates of `lemon/concept_check.h`. A call `checkConcept< Concept , T >()` takes the address of `Concept::Constraints< T >::constraints`, which forces the compiler to instantiate that function. If `T` lacks one of the operations the concept requires, the compilation fails with an error that names it; at run time the call does nothing. Its older form `function_requires< C >()` does the same for a class `C` whose member `constraints()` is to be instantiated (a form that is now rarely used). Both are empty when `NDEBUG` is defined (i.e., in an optimized build without assertions, where they check nothing), and in a build without `NDEBUG` they cost only compilation time. In the library, the tests check each graph structure, map and heap against its concept (e.g., `test/digraph_test.cc` calls `checkConcept< Digraph , ListDigraph >()`), and a user who writes a new structure checks it in the same way (appendix A). A generic function may also call `checkConcept` on its template parameters, as the function `incidentWeight()` of `manual/examples/03-items-maps.cc` does, in which case a wrong argument is reported where the function is called instead of deep inside its body. Finally, the object `ignore_unused_variable_warning`, called with any arguments, silences the warnings on the variables that a function `constraints()` declares and does not use.

> **abLEMON.** `ignore_unused_variable_warning` is an object of a class with a variadic `operator()`, instead of a set of overloaded functions as in LEMON 1.3.1. Hence, an unqualified call to a function of the same name of another library is no longer made ambiguous by argument-dependent lookup on the types of `lemon`.

3.6 The algorithm classes

Most algorithms of the library are class templates that share one pattern, which we describe here on `Dijkstra` (of `lemon/dijkstra.h`, cf. chapter 10), while the chapters of Part III give the template parameters, the defaults and the queries of each class, and they say where a class departs from the pattern.

Template parameters and traits. The first parameters of the class are the types of its input; thus, `Dijkstra< GR , LEN , TR >` takes the type `GR` of the digraph (by default `ListDigraph`) and the type `LEN` of the map of the

lengths (by default `GR::ArcMap< int >`). Its last parameter `TR` is a traits class, by default `DijkstraDefaultTraits< GR , LEN >`. It defines the other types used by the algorithm (the heap with its cross reference map, the maps of the predecessors, of the distances and of the processed nodes, the operations on the lengths), each with a static function that creates an object of that type; the lengths have the type `LEN::Value`, and the default heap is a `BinHeap` on them.

Named template parameters. We remark that a traits class is rarely written by hand, since each of its types can be replaced by a named template parameter. This is a nested template of the algorithm class, whose member `Create` is the class with that type replaced. For instance, `Dijkstra< GR , LEN >::SetDistMap< M >::Create` is a `Dijkstra` that writes the distances into a map of type `M`, which the caller passes by `distMap(m)`, and `SetHeap< H , CR >` replaces the heap and its cross reference map (cf. chapter 7), while `SetOperationTraits< T >` replaces the operations on the lengths. Named parameters can be chained, as in `SetPredMap< P >::Create::SetDistMap< M >::Create`. A type replaced in this way is not created by the algorithm (its creating function only asserts), and the caller must give the object before the first run; the variants `SetStandardHeap< H >` and `SetStandardProcessedMap` instead let the algorithm create the new type itself.

Execution. A constructor takes the input objects (here the digraph and the length map) by reference, and they must outlive the algorithm, while the maps that the algorithm writes are created by it at the first run, unless the caller has given them by the setter functions `predMap()`, `distMap()`, `processedMap()` and `heap()` (which takes the heap and its cross reference map). Each setter returns a reference to the algorithm, so that the calls can be chained, and `lengthMap()` and `operationTraits()` replace the input in the same way. The simplest way to execute the algorithm is `run(s)`, which computes the shortest paths from `s` to all the nodes reachable from it, while `run(s, t)` stops as soon as `t` is processed and returns whether `t` is reachable. For a finer control, `run(s)` is the sequence `init()`, `addSource(s)`, `start()` of Algorithm 3.1, and these functions may also be called one by one. Thus, `init()` resets the data of the nodes, and `addSource(s, d)` adds a source with the initial distance `d` (zero by default); one may give several sources, a node already in the heap keeps the smaller of its two distances, and a node already processed is left alone. Then `start()` runs the search until the heap is empty, `start(t)` stops when `t` is processed, and `start(nm)` stops at the first node `v` with `nm[v]` true that reaches the top of the heap, which it processes and returns (or `INVALID` if there is none). Between these calls, `processNextNode()` performs one step and returns the node it processed, `nextNode()` gives the node that the next step will process (`INVALID` if the heap is empty), and `emptyQueue()` and `queueSize()` describe the heap.

Algorithm 3.1 (The run of an algorithm class, on Dijkstra).

```
run(s):
 1  init()                      pred[v] = INVALID, processed[v] = false and
                                the state of v PRE_HEAP for every node v
 2  addSource(s, 0)             push s with priority 0
 3  start():
 4    while not emptyQueue()
 5      v = processNextNode():   pop the node v of least priority p,
 6                               dist[v] = p, processed[v] = true;
 7                               for each arc a = (v, w) of delta+(v):
 8                                 push w with p + len[a] if w is PRE_HEAP,
 9                                 or decrease it to p + len[a] if w is in
10                                 the heap with a larger priority;
11                                 then pred[w] = a
```

With nonnegative lengths, the priority of each node when it leaves the heap is its distance from the sources, and the arcs `pred` form a forest of shortest paths; this correctness of Algorithm 3.1 is Theorem 10.2 of chapter 10; the same theorem bounds the run by n pushes and pops and m decreases of a priority, whose cost depends on the heap (chapter 7). Since the nodes leave the heap in nondecreasing order of distance, `run(s, t)` and `start(t)` give the distance of `t` as soon as `t` has left it.

Queries. After a run the results are read by query functions, named after the objects of the problem: `dist(v)`, `predArc(v)`, `predNode(v)` and `path(t)` for the shortest paths, `currentDist(v)` for the priority of a node still in the heap, `reached(v)` and `processed(v)` for the state of a node, and `distMap()` and `predMap()` for the maps. A query is meaningful only after a run that reached the node (and, for `dist(v)`, that processed it): this is its precondition, stated in the documentation of each function, and its value remains available until the next `init()`.

Function-type interface. Some algorithms (among them `bfs()`, `dfs()` and `dijkstra()`) also have a function

that returns a "wizard" object, on which the optional outputs are given by name before the run, as in

```
bool found = dijkstra( g , length ).distMap( d ).path( p ).run( s , t );
```

which writes the distances into `d` and the path from `s` to `t` into `p`. Such a wizard constructs the algorithm for the run, together with the maps that were not given, and destroys them afterwards. It is rather convenient in simple cases, whereas the class gives access to the intermediate state and to repeated runs. The program `manual/examples/03-algorithm-pattern.cc` (the test `manual_03_algorithm_pattern`) goes through the pattern on a `SmartDigraph`. It uses the named parameter `SetDistMap`, the separate steps `init()`, `addSource()` and `start(t)`, a run with two sources at different initial distances, the queries on a node that is not reachable, and the function-type interface.

3.7 Tolerances and numeric traits

An algorithm that works on floating-point data compares numbers that may carry a rounding error, and a test such as `x == 0` or `a < b` then gives an answer that depends on the last bits of the operands. In `lemon/tolerance.h` the class template `Tolerance< T >`, whose type `Value` is `T`, collects the comparisons that the algorithms need. Each of them returns `true` only if the relation holds surely, i.e., even allowing for an error up to a threshold $\varepsilon \geq 0$, as follows:

$$\begin{array}{ll} \texttt{less(a, b)}: & a + \varepsilon < b, \\ \texttt{different(a, b)}: & \texttt{less(a, b)} \vee \texttt{less(b, a)}, \\ \texttt{positive(a)}: & \varepsilon < a, \\ \texttt{negative(a)}: & a < -\varepsilon, \\ \texttt{nonZero(a)}: & \texttt{positive(a)} \vee \texttt{negative(a)}, \end{array} \tag{3.1}$$

where the sum $a + \varepsilon$ is the one computed in the type `T`, while `zero()` returns 0. Its general version is meant for exact types (the integers, or a rational type), and its functions are static and compare exactly, i.e., as in (3.1) with $\varepsilon = 0$. Instead, the specializations for `float`, `double` and `long double` hold an ε in each object. It is set by the constructor (as in `Tolerance< double > tol( 1e-8 )`) or by `tol.epsilon( e )`, and read by `tol.epsilon()`; its default is 10^{-4}, 10^{-10} and 10^{-14}, respectively (the last one being the `double` value 10^{-14}, widened, as `lemon/base.cc` defines it). The static functions `defaultEpsilon()` and `defaultEpsilon( e )` read and change the default for the objects constructed afterwards, everywhere in the program. Any other type gets the general template, and a user who works with another inexact type writes a specialization with the same functions. The comparisons (3.1) have the following properties, which the algorithms rely on when they sort or compare values under a tolerance.

Lemma 3.1. *Let $\varepsilon \geq 0$, let the operands be numbers of `T` other than NaN, and let $a \approx b$ mean that `different( a, b)` is false. Then (i) `less` is irreflexive and transitive, i.e., a strict partial order, also when $a + \varepsilon$ is rounded; (ii) $\approx$ is reflexive and symmetric; (iii) for $\varepsilon = 0$, $a \approx b$ holds if and only if $a = b$; (iv) for $\varepsilon > 0$ and in exact arithmetic, $\approx$ is not transitive, and $a \approx b$ holds if and only if $|a - b| \leq \varepsilon$.*

Proof. Let $\mathrm{fl}(x)$ be the rounding of x to `T`, which is monotone and leaves the numbers of `T` unchanged. (i) Since $a + \varepsilon \geq a$, one has $\mathrm{fl}(a + \varepsilon) \geq \mathrm{fl}(a) = a$, and therefore $\mathrm{fl}(a + \varepsilon) < a$ never holds. If $\mathrm{fl}(a + \varepsilon) < b$ and $\mathrm{fl}(b + \varepsilon) < c$, then $\mathrm{fl}(a + \varepsilon) < b \leq \mathrm{fl}(b + \varepsilon) < c$ by the same argument on b, i.e., `less(a, c)`. (ii) `different(a, a)` is `less(a, a)`, which is false by (i), and `different` is symmetric by its definition. (iii) With $\varepsilon = 0$, `different(a, b)` is $a \neq b$ in the general template, and $a < b$ or $b < a$ in the specializations (since then $\mathrm{fl}(a + 0) = a$), which for numbers other than NaN is again $a \neq b$. (iv) In exact arithmetic `different(a, b)` is $a + \varepsilon < b$ or $b + \varepsilon < a$, i.e., $|a - b| > \varepsilon$. Hence, $a = 0$, $b = \varepsilon$ and $c = 2\varepsilon$ give $a \approx b$ and $b \approx c$, but not $a \approx c$. □

We stress that the threshold is absolute, i.e., ε in (3.1) does not scale with the operands; hence, `Tolerance < double >` finds 10^{20} and $10^{20} + 10^5$ surely different (although their relative gap is 10^{-15}), and 10^{-12} and $2 \cdot 10^{-11}$ equal (although one is 20 times the other). Its default suits, therefore, data of "moderate" magnitude, and the algorithms that take a `Tolerance` (the maximum flow and minimum cut algorithms, among others) have a function `tolerance()` by which the caller sets one that fits the scale of the data. On the other hand, the minimum cost flow algorithms keep tolerances of a different kind, which are relative to the data unless the caller sets them (cf. chapter 13). The program `manual/examples/03-tolerance.cc` (the test `manual_03_tolerance`) checks the exact comparisons of the general template, the default thresholds, the two examples above of the absolute threshold, and the function `countNodes()` discussed at the end of the section.

Numeric constants. The "small" header `lemon/math.h` includes `<cmath>` and defines, in the namespace `lemon`, the constants `E`, `LOG2E`, `LOG10E`, `LN2`, `LN10`, `PI`, `PI_2`, `PI_4`, `SQRT2` and `SQRT1_2`, as `long double` values written with 35 significant digits. It also defines `isNaN(v)`, which tests `v != v` on a `double`, and `round(r)`, which rounds a `double` to the nearest integer with the halves away from zero, i.e., $\lfloor r + 1/2 \rfloor$ for $r > 0$ and $\lceil r - 1/2 \rceil$ otherwise. In abLEMON the constants (and the object `ignore_unused_variable_warning` of section 3.5) are declared by the macro `LEMON_CONSTANT` of `lemon/config.h`, i.e., `inline constexpr` from C++17 on and `cons t` before, so that the C++20 module `lemon` can export them.

Traits of graphs and maps. A user rarely includes `lemon/bits/traits.h` directly; it holds the templates by which the library selects an implementation at compile time from the properties of a type. Thus, `ItemSetTrait s< GR , Item >` gives, for a graph type and a kind of item, the iterator (`ItemIt`), the map template (`Map< V > ::Type`) and the notifier of the items, and a template can be written once for nodes, arcs and edges. Also, `M apTraits< M >` gives whether a map is a reference map and which types its `operator[]` returns (chapter 6), and `MatrixMapTraits< M >` does the same for the maps with two keys. Other templates detect the tags that a structure may define (`NodeNumTagIndicator< GR >` and its siblings, whose member `value` is `true` when the tag is there). Among these tags, `NodeNumTag`, `ArcNumTag` and `EdgeNumTag` mean that the structure counts its items in $O(1)$ time, `FindArcTag` and `FindEdgeTag` that it finds the arc between two nodes directly, `BuildTag` that it is built in one call from another graph, and `UndirectedTag` that it is undirected. Typically, a counting function of `lemon/core.h` is selected by these tags; indeed, `countNodes(g)` calls `g.nodeNum()` on a structure with `No deNumTag`, in $O(1)$ time, and scans the nodes with `NodeIt` otherwise, in $O(n)$ time, and the other utilities of chapter 4 (`countArcs()`, `findArc()`, the copies) are selected in the same way, by `enable_if` on the tag.

3.8 Exercises

Exercise 3.1. Write a variant of the first example that prints the nodes of the shortest path from s to t in the order in which the path visits them, by `dijkstra.path(t)` and the path utilities of chapter 8 instead of a loop on `predNode()`. (The program `manual/examples/03-first-program.cc` gives the distance against which to check the result.)

Exercise 3.2. Write a function template that, for any undirected graph `g` and edge map `w`, returns the edge of largest weight incident to a node (or `INVALID` if the node is isolated), and check its graph type by `checkConcep t`; call it on a `ListGraph` and on a `SmartGraph`. (The function `incidentWeight()` of `manual/examples/03-it ems-maps.cc` is a model.)

Exercise 3.3. Build a `ListGraph` with a loop and two parallel edges, and count, for one of their end nodes, the items visited by `OutArcIt`, `InArcIt` and `IncEdgeIt`. Explain the counts by the representation of Figure 3.2. (The test `test/graph_test.cc` checks the degrees of such graphs.)

Exercise 3.4. By Lemma 3.1, the equality under a tolerance is not transitive. Discuss what this implies for an algorithm that groups values by equality, and show that grouping the sorted values by cutting where two consecutive ones are `different()` gives groups that do not depend on the order of the input, although two values of one group may be different. (The program `manual/examples/03-tolerance.cc` checks the threshold.)

Exercise 3.5. Replace the length map of the program of section 3.6 by a map of type `ConstMap< SmartDigrap h::Arc , double >` with value 1 (cf. chapter 6), so that `Dijkstra` computes the number of arcs of the shortest paths, and check the result against a breadth-first search (chapter 9). (The test `test/bfs_test.cc` shows the breadth-first search.)

4. Graph structures

We are concerned in this chapter with the classes that store a graph, and with the utilities of `lemon/core.h` that count, find and copy its items. All of them implement the concepts of chapter 3; hence, an algorithm written for a digraph runs on any of them. They differ in what they allow after the construction (adding items, erasing them, nothing at all), in the time of their operations and in the memory they take. One usually chooses between a list-based structure, which allows any modification, a vector-based one, which only grows, a static one, which is built once and is the most compact, and an implicit one, which stores no item and computes the structure of a full, grid or hypercube graph from the indices. Section 4.1 describes the concepts and compares the structures in Table 4.1, and sections 4.2 to 4.6 give the structures family by family. Section 4.7 deals with the arc and edge sets that share the nodes of another graph, section 4.8 with the functions and classes that count the items and find the arcs between two nodes, and section 4.9 with the copies of a graph into another structure; the exercises are in section 4.10. Their representations follow the textbook ones (cf. Ahuja, Magnanti and Orlin 1993, §2.3 and Cormen et al. 2001, §22.1): adjacency lists, linked in the list-based and vector-based structures, and stored contiguously (the "forward star" representation) in the static ones. Besides the notation of chapter 3, we write $d^+(v) = |\delta^+(v)|$ and $d^-(v) = |\delta^-(v)|$ for the out-degree and the in-degree of a node v of a digraph. Also, $d(v)$ is the number of arcs that leave v in an undirected graph (i.e., its degree, a loop counting twice), and $\Delta = \max_v d^+(v)$.

4.1 The graph concepts

In `lemon/concepts/digraph.h` the class `concepts::Digraph` declares the items `Node` and `Arc` and the iterators `NodeIt`, `ArcIt`, `OutArcIt` and `InArcIt`, with their ranges `nodes()`, `arcs()`, `outArcs(v)` and `inArcs(v)`. It also declares the functions `source()`, `target()`, `id()`, `nodeFromId()`, `arcFromId()`, `maxNodeId()`, `maxArcId()`, their generic forms `maxId()` and `fromId()`, `oppositeNode()`, `baseNode()` and `runningNode()`, the low-level enumeration `first()`, `next()`, `firstOut()`, `nextOut()`, `firstIn()` and `nextIn()` on which the iterators are built, and the map templates `NodeMap< T >` and `ArcMap< T >`. Note that a digraph cannot be copied or assigned, since its copy constructor and assignment are private; it is duplicated by the copies of section 4.9. The concept `concepts::Graph`, in `lemon/concepts/graph.h`, adds the item `Edge`, the iterators `EdgeIt` and `IncEdgeIt` with their ranges, the functions `u()`, `v()`, `direction()`, the two forms of `direct()`, `oppositeArc()`, `edgeFromId()`, `maxEdgeId()`, `firstInc()` and `nextInc()`, the template `EdgeMap< T >`, and the tag `UndirectedTag`. An undirected graph conforms to `concepts::Digraph` too, with two arcs for each edge (cf. chapter 3). In `lemon/concepts/bpgraph.h`, `concepts::BpGraph` adds the items `RedNode` and `BlueNode`, the iterators `RedNodeIt` and `BlueNodeIt`, the maps `RedNodeMap< T >` and `BlueNodeMap< T >`, and the functions `red()`, `blue()`, `asRedNode()`, `asBlueNode()`, their unsafe variants, `redNode(e)`, `blueNode(e)`, `maxRedId()`, `maxBlueId()`, `firstRed()`, `nextRed()`, `firstBlue()` and `nextBlue()`. All three concepts describe only the queries; adding, erasing and clearing are not part of them, since some structures do not offer them.

These concepts are assembled from components, the classes of `lemon/concepts/graph_components.h`, which a structure or an algorithm may also require separately. Each component except the first four below exists in three versions, for digraphs, graphs and bipartite graphs, and it takes as a template parameter `BAS` the component on which it is built (by default the base one):

- `GraphItem< sel >`, the concept of a handle (default and copy construction, assignment, conversion from `INVALID`, `==`, `!=`, `<`), with a character `sel` that keeps the types of nodes, arcs and edges distinct.
- `GraphItemIt< GR , Item >` and `GraphIncIt< GR , Item , Base , sel >`, the concepts of `NodeIt`, `ArcIt` and `EdgeIt` and of `OutArcIt`, `InArcIt` and `IncEdgeIt`, respectively.
- `GraphMap< GR , K , V >`, the concept of a standard map, i.e., a reference map constructed on the graph, with or without an initial value.

- `BaseDigraphComponent`, `BaseGraphComponent` and `BaseBpGraphComponent`, the items with `source()` and `target()` (and `u()`, `v()`, `direction()`, `direct()`, `oppositeArc()`, or the colours).
- `IDableDigraphComponent` (and its graph and bipartite versions), the identifiers and the functions that invert them, which must be unique and immutable.
- `IterableDigraphComponent`, the iterators, built on `GraphItemIt` and `GraphIncIt`, with the functions `first()`, `next()`, `firstOut()` and the others of the base classes.
- `AlterableDigraphComponent`, the alteration notifiers, by which the structure informs its maps and other observers of each change (cf. appendix A).
- `MappableDigraphComponent`, the map templates, each of which must conform to `GraphMap`.
- `ExtendableDigraphComponent`, `ErasableDigraphComponent` and `ClearableDigraphComponent`, the functions `addNode()` and `addArc()` (or `addEdge()`, `addRedNode()`, `addBlueNode()`), `erase()`, and `clear()`, each of which requires (in its documentation) the alterable component.

Thus, `concepts::Digraph` is made of the base, idable, iterable and mappable components, as its `Constraints` check. Besides it, the tests of the library check the components that each structure offers in addition: e.g., `test/digraph_test.cc` checks `ListDigraph` against the extendable, erasable, clearable and alterable components, and `StaticDigraph` against the clearable one only. Table 4.1 lists the structures of the chapter with what they offer and what they cost. Its memory column gives the number of `int` values that each node and each arc take in the arrays of the structure (from the private structs `NodeT` and `ArcT` of the base classes of `lemon/list_graph.h`, `lemon/smart_graph.h` and `lemon/edge_set.h`, and from the arrays of the static structures), and the last column the time of `findArc()` of section 4.8.

Table 4.1. The graph structures.

Structure	Concept	Changes	Counting	Memory per node / arc	`findArc(u, v)`
`ListDigraph`	Digraph	add, erase, others	$O(n)$, $O(m)$	4 / 6	$O(d^+(u))$
`ListGraph`	Graph	add, erase, others	$O(n)$, $O(m)$	3 / 3	$O(d(u))$
`ListBpGraph`	BpGraph	add, erase, others	$O(n)$, $O(m)$	6 and a `bool` / 3	$O(d(u))$
`SmartDigraph`	Digraph	add, snapshot	$O(1)$	2 / 4	$O(d^+(u))$
`SmartGraph`	Graph	add, snapshot	$O(1)$	1 / 2	$O(d(u))$
`SmartBpGraph`	BpGraph	add, snapshot	$O(1)$	3 and a `bool` / 2	$O(d(u))$
`StaticDigraph`	Digraph	build	$O(1)$	2 / 4	$O(d^+(u))$
`StaticGraph`	Graph	build	$O(1)$	1 / 3	$O(d(u))$
`CompactDigraph`	Digraph	build	$O(1)$	1 / 1	$O(d^+(u))$
`FullDigraph`, `FullGraph`, `FullBpGraph`	Digraph, Graph, BpGraph	resize	$O(1)$	none	$O(1)$
`GridGraph`, `HypercubeGraph`	Graph	resize	$O(1)$	none	$O(1)$
`ListArcSet`, `ListEdgeSet`	Digraph, Graph	add, erase	$O(n)$, $O(m)$	2 / 6, 1 / 3	$O(d^+(u))$
`SmartArcSet`, `SmartEdgeSet`	Digraph, Graph	add	$O(n)$, $O(m)$	2 / 4, 1 / 2	$O(d^+(u))$

In the table, "others" are the changes of the end nodes, the contraction and the splitting of section 4.2, and "snapshot" means that the items added after a snapshot can be removed (section 4.3). Also, "build" means that the structure is built in one call and can be rebuilt (section 4.4), and "resize" that the implicit structure changes its size, which clears its maps. Counting is that of `countNodes()` and `countArcs()` (section 4.8), and the memory of an undirected structure is given per arc, an edge having two. For the arc and edge sets, the memory per node is that of the node map in which they keep the heads of their lists (the nodes belong to the underlying graph). Their arcs store the end nodes as handles of the underlying graph, which are counted here as one `int` each.

4.2 List graphs

In `lemon/list_graph.h` we find `ListDigraph`, `ListGraph` and `ListBpGraph`, the structures that allow any modification. A `ListDigraph` keeps its nodes and its arcs in two `std::vectors`. The nodes form a doubly linked

list, and so do the arcs out of each node and the arcs into each node, through indices stored in the arcs. An erased node or arc is unlinked and put on a free list, whose items are reused by the next additions; hence, the identifiers are the positions in the vectors, bounded by the largest number of items that existed at the same time. The nodes are visited from the last added to the first, and `ArcIt` follows the lists of the arcs out of each node. In `ListGraph` each edge is stored as two arcs with consecutive indices (cf. Figure 3.2), and each node keeps one doubly linked list of the arcs that leave it, the arcs that enter a node being the opposites of those that leave it. The class `ListBpGraph` adds to the nodes of `ListGraph` a colour, an index within the colour (`id(RedNode)`, `id(BlueNode)`), and a doubly linked list for each colour. Besides the functions of the concepts, the three classes have the following ones, which keep the iterators valid unless it is said otherwise.

- `addNode()` and `addArc(s, t)` (`addEdge(u, v)` in `ListGraph`, and `addRedNode()`, `addBlueNode()` and `addEdge(r, b)` in `ListBpGraph`, which also accepts the blue node first), `erase(item)` for a node, an arc or an edge (which invalidates the iterators on the erased items), and `clear()`.
- `valid(item)`, which returns whether a handle identifies an item at present; an erased item may become valid again after an addition that reuses its index.
- `changeTarget(a, n)`, `changeSource(a, n)` and `reverseArc(a)` in `ListDigraph`, which move the ends of an arc and keep its identifier and its values in the maps. In the same way, `ListGraph` has `changeU(e, n)` and `changeV(e, n)`, and `ListBpGraph` has `changeRed(e, n)` and `changeBlue(e, n)`. The iterators along the lists that the arc leaves are invalidated (e.g., an `InArcIt` on `a` after `changeTarget()`, and all three arc iterators on `a` after `reverseArc()`).
- `contract(u, v, r = true)` in `ListDigraph` and `ListGraph`, which moves the arcs (or edges) of `v` to `u` by the changes above and erases `v`. If `r` is `true`, the arcs that would become new loops at `u` are erased instead (in `ListDigraph`, those between `u` and `v` and the loops at `v`). The loops that `u` already had are kept.
- `split(n, connect = true)` in `ListDigraph`, which adds a node, moves to it the arcs out of `n`, and, if `connect` is `true`, adds an arc from `n` to it, keeping all the iterators valid. Also, `split(a)` adds a node `w`, makes it the target of `a`, and adds an arc from `w` to the old target of `a`; both return the new node.
- `reserveNode(n)` and `reserveArc(m)` (`reserveEdge(m)` in the undirected classes), which reserve the memory of the vectors before a large graph is built.

Lemma 4.1. *In `ListDigraph`, `addNode()` and `addArc()` take $O(1)$ amortized time, `erase(a)`, `changeSource()`, `changeTarget()` and `reverseArc()` take $O(1)$ time, `erase(v)` takes $O(d^+(v)+d^-(v))$ time, `contract(u, v)` takes $O(d^+(v)+d^-(v))$ time, `split(n)` takes $O(d^+(n))$ time, and `countNodes()` and `countArcs()` take $O(n)$ and $O(m)$ time. The same bounds hold in `ListGraph` and `ListBpGraph` with $d(v)$ in place of $d^+(v)+d^-(v)$.*

Proof. An addition takes an item from the free list, or appends one to its vector, whose growth costs $O(1)$ amortized per element, and links the item at the head of its lists. Unlinking an arc from a doubly linked list, or linking it at the head of another, takes $O(1)$ time, since its predecessor and its successor are stored in it; `erase(a)`, `changeSource()` and `changeTarget()` do one or two of these operations, and `reverseArc()` is a `changeTarget()` followed by a `changeSource()`. The extender (cf. appendix A) erases a node by erasing first each arc of its two lists, and then unlinks the node. `contract(u, v)` visits the same arcs once each, and either erases or moves each of them in $O(1)$, before erasing `v`, which has then no arc. `split(n)` moves the whole list of the arcs out of `n` to the new node in $O(1)$ and rewrites the source stored in each of its $d^+(n)$ arcs. Finally, the counting functions iterate on the items (section 4.8), since the structure keeps no count. The undirected classes keep one list per node, of $d(v)$ arcs, and the same arguments apply. □

Snapshots. A nested class `Snapshot` saves the state of the graph and restores it later, removing the items added in between. A snapshot is made by the constructor `Snapshot(g)` or by `save(g)` (which replaces a previous one, and is needed after the default constructor `Snapshot()`), and it is used by `restore()`. It observes the notifiers of the graph (cf. appendix A) and records the items added, while `valid()` returns whether it is still attached to the graph. When one of the recorded items is erased, the snapshot removes it from its record. An item that existed before the snapshot, instead, cannot be erased without consequences: its erasure detaches the snapshot from the graph, after which `valid()` returns `false`. Also `restore()` detaches the snapshot, and a state cannot be restored twice unless `save()` is called again. Furthermore, the other modifications (`changeTarget()`, `reverseArc()`, `contract()`, `split()` and so on) are not notified, and therefore not recorded, and they must not be used together with a snapshot.

Lemma 4.2. *Let k be the number of items in the record of a `Snapshot` of a list graph. If, between `save(g)` and `restore()`, no item that existed at `save(g)` is erased and no modification other than additions and erasures is made, then `restore()` leaves in `g` exactly the items that existed at `save(g)`, with their identifiers, in $O(k)$ time; each erasure of a recorded item costs $O(k)$ time more.*

Proof. The snapshot puts each item added after `save(g)` at the front of its record, and removes each erased one after a linear search. Since no older item is erased (an erasure that would detach the snapshot), the record holds, at `restore()`, exactly the items present that did not exist at `save(g)`. `restore()` detaches the snapshot, erases the recorded arcs (or edges), each in $O(1)$ time by Lemma 4.1, and then the recorded nodes. A remaining arc at a recorded node would have been added after the node, since the ends of an arc are not changed under the hypothesis; hence, it is recorded and already erased, and each node is erased in $O(1)$ time. The items that existed at `save(g)` are not touched, and they keep their identifiers. □

The program `manual/examples/04-list-graph.cc` (the test `manual_04_list_graph`) goes through these functions on a `ListDigraph`: the reversal and the change of the target of an arc, which keep its value in a map, the contraction of two nodes and the splitting of one, a snapshot that removes the items added after it and is no longer valid after `restore()`, and the erasure of a node with its arcs.

4.3 Smart graphs

In `lemon/smart_graph.h` we find `SmartDigraph`, `SmartGraph` and `SmartBpGraph`, which store their items in vectors and link the arcs of each node in singly linked lists, each new arc at the head of the lists of its ends. They cannot erase an item, and in exchange they are typically faster to build and somewhat smaller than the list-based ones (Table 4.1). Their identifiers are the positions in the vectors, i.e., the nodes and the arcs are numbered from 0 in the order of addition. The items are counted in $O(1)$ time, since the structures have the tags `NodeNumTag` and `ArcNumTag` (and `EdgeNumTag` in the undirected ones) and the functions `nodeNum()`, `arcNum()` and `edgeNum()` (and `redNum()`, `blueNum()` in `SmartBpGraph`). Besides the functions of the concepts, they have `addNode()`, `addArc()` (or `addEdge()`, `addRedNode()` and `addBlueNode()`), `valid()`, `clear()`, `reserveNode()` and `reserveArc()` (or `reserveEdge()`). Also, `SmartDigraph` has `split(n, connect = true)`, as in `ListDigraph`.

Snapshots. Items can be removed from a smart graph only by its `Snapshot`, which records the numbers of nodes and arcs at `save(g)` (or at the constructor `Snapshot(g)`; the default constructor `Snapshot()` makes an empty snapshot, to be set by `save()`). Its `restore()` erases the items with larger indices, the last first, and notifies the maps of each of them. Unlike the snapshot of a list graph, this one does not observe the graph, and its validity is not checked. Hence, a `restore()` after a `split()` of `SmartDigraph`, or after a restore to an older snapshot, leaves the graph in an inconsistent state, as the header warns.

Lemma 4.3. *If only additions are made to a smart graph between `save(g)` and `restore()`, then `restore()` leaves in `g` exactly the items that existed at `save(g)`, with their lists of arcs as they were; `save(g)` takes $O(1)$ time, and `restore()` time linear in the number of items it removes.*

Proof. The indices are the positions of the items in their vectors, and the items added after `save(g)` are those of index at least the numbers recorded. Each new arc is put at the head of the list of the arcs out of its source and of the list of the arcs into its target (one list per node in the undirected structures), and an arc is never moved. Hence, the lists are stacks, and the arc of largest index is at the head of the lists of its ends. `restore()` pops the arcs from the largest index, and resets the head of each of their lists to the next arc stored in the popped one, which undoes the additions in the reverse order, in $O(1)$ each. Then it pops the nodes, which have no arc left. Recording two numbers takes $O(1)$ time. □

4.4 Static graphs

The headers `lemon/static_graph.h` and `lemon/compact_graph.h` define `StaticDigraph`, `StaticGraph` and `CompactDigraph`, which are built in one call and cannot be modified afterwards (but can be built again). They store the arcs out of each node one after the other, in arrays with no pointer to follow. Hence, they take less memory than the other structures, and they scan the arcs out of a node contiguously, which is often (but not always) faster in practice. Their nodes and arcs are numbered from 0, and `node(i)`, `arc(i)` (`edge(i)` in `StaticGraph`) and `index(item)` convert between items and indices, the index of an item being its identifier. Note that each class has `nodeNum()` and `arcNum()` (and `edgeNum()`), in $O(1)$ time, `clear()`, and two functions `build()`; `StaticGraph` also has `valid(item)`.

- `build(g, nodeRef, arcRef)` copies any digraph `g` (any graph, for `StaticGraph`, with `edgeRef` in place of `arcRef`). It writes into the map `nodeRef` the image of each node, which it also reads, and into `arcRef` (or `edgeRef`) the image of each arc. For the latter a `NullMap` may be given if it is not needed. The copies of section 4.9 call it, by the tag `BuildTag`.

- `build(n, begin, end)` builds the structure with `n` nodes from a range of pairs of node indices (e.g., `std::pair< int , int >`), the k-th pair (i, j) giving the arc (or edge) of index $k-1$ from the node of index i to that of index j.

Rebuilding a structure, or clearing it, clears the values of its maps.

StaticDigraph. Its arrays are those of Figure 4.1: `node_first_out`, of $n+1$ entries, gives the first arc out of each node (the arcs out of node v being those from `node_first_out[v]` to `node_first_out[v+1]-1`), and `arc_target` the target of each arc. Two more arrays, `node_first_in` and `arc_next_in`, link the arcs into each node in a list, and `arc_source` and `arc_next_out` complete the four `int` values of each arc. Its `OutArcIt` is specialized: it walks the indices from the first arc of the node to the last, with no array access per step. When the structure is built from a list, the pairs must come in nondecreasing order of their first element, and the arcs keep the order of the list. Two `LEMON_ASSERT`s (active unless the assertions are disabled, cf. chapter 8) check that each target is a valid index and that the whole list was consumed, which fails when the pairs are not sorted or a source is out of range. When built from a digraph, the nodes take the indices in the order of the `NodeIt` of `g`, and the arcs out of each node are sorted (by the private comparator `ArcLess` of the class) by the index of their target; the order of parallel arcs is not specified.

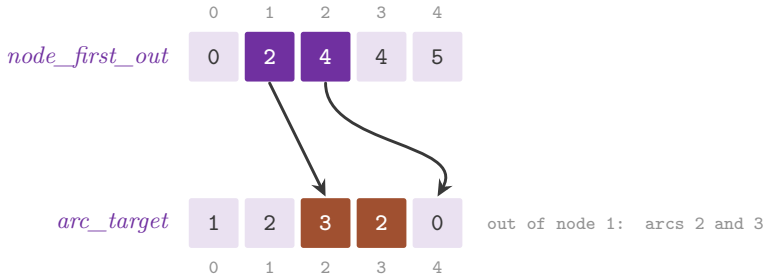


Figure 4.1. The arrays of the StaticDigraph built from the arcs (0,1), (0,2), (1,3), (1,2), (3,0): the arcs out of node 1 are those of indices 2 and 3, whose targets are 3 and 2.

Lemma 4.4. *Built from a list of m pairs sorted by their first element, `StaticDigraph` has the arc $k-1$ from i to j for the k-th pair (i, j), its `OutArcIt` at v visits the arcs of indices `node_first_out[v]` to `node_first_out[v+1]-1`, and its `InArcIt` the arcs into v; the build takes $O(n+m)$ time. Built from a digraph, it takes $O(n + \sum_v d^+(v) \log d^+(v))$ time.*

Proof. The build scans the nodes $i = 0, \ldots, n-1$ and, for each, consumes the pairs whose first element is i, which are consecutive in a sorted list. It sets `node_first_out[i]` to the index of the first of them and gives the consumed pairs the consecutive indices, so that the arcs out of i are the indices between `node_first_out[i]` and `node_first_out[i+1]-1` (an empty range if i has none). Each arc is also put at the head of the list `node_first_in`, `arc_next_in` of its target, which then holds the arcs into each node. Every pair and every node is handled once, in $O(1)$ time. From a digraph, the nodes are numbered in one scan, and the arcs out of each node v are collected and sorted in $O(d^+(v) \log d^+(v))$ time before they are stored in the same way. □

StaticGraph. This undirected counterpart of `StaticDigraph` represents the edge of index i by the arcs $2i+1$, from u to v, and $2i$, from v to u, as `SmartGraph` does. Since the arcs out of each node are stored one after the other, and the arcs into a node are the opposites of those out of it, one array of positions serves both directions. It takes three `int` values per arc (its target, its position, and the arc at each position) and one per node, the first position of its arcs. Here the pairs of `build(n, begin, end)` may come in any order; the edge $k-1$ has u equal to the first element of the k-th pair and v to the second, and the arcs are placed by a counting sort on their sources (Algorithm 4.1). A copy from a graph numbers the nodes and the edges in the order of their iterators and then builds from the list of the end nodes. A `LEMON_ASSERT` checks that the indices of the pairs are valid.

Algorithm 4.1 (Building a StaticGraph from a list of edges).

```
build(n, ends[0..m-1]):
 1  for each edge i:  target[2i] = u_i ; target[2i+1] = v_i
 2  first[0..n] = 0
 3  for each edge i:  first[u_i + 1]++ ; first[v_i + 1]++
 4  for v = 0 .. n-1: first[v+1] += first[v]      (prefix sums)
 5  fill[v] = first[v] for each node v
 6  for each arc a = 0 .. 2m-1:
```

```
7    p = fill[source(a)]++ ,  source(a) = target[a xor 1]
8    out[p] = a ;  pos[a] = p
```

Here the arcs out of node v are `out[first[v]]` to `out[first[v+1]-1]`, `nextOut(a)` is `out[pos[a]+1]` while it stays below `first[v+1]`, and `nextIn(a)` is the opposite of `nextOut` on the opposite arc (`lemon/static_graph.h`, `StaticGraphBase::build()`).

Theorem 4.1. *After Algorithm 4.1, for every node v the entries `out[first[v]]`, ..., `out[first[v+1]-1]` are exactly the arcs whose source is v, in increasing order of index, and `nextIn()` visits the arcs into v as the opposites of these, in the same order. The algorithm takes $O(n+m)$ time, and so does the copy from a graph.*

Proof. The arc $2i+1$ has target v_i and source target$[2i] = u_i$, and the arc $2i$ has target u_i and source v_i, i.e., each edge i gives one arc out of u_i and one out of v_i (two out of u_i if it is a loop). Hence, after line 3 `first[v+1]` is $d(v)$, and after line 4 `first[v]` is $\sum_{w<v} d(w)$, so that the intervals [`first[v]`, `first[v+1]`) partition $\{0,\dots,2m-1\}$ into blocks of sizes $d(v)$. Line 7 gives the arcs of source v the positions `first[v]`, `first[v]`$+1$, and so on, in the order in which line 6 meets them, i.e., in increasing order of index; since there are $d(v)$ of them, they fill the block of v. An arc a enters v if and only if its opposite $a \oplus 1$ leaves v, and `nextIn()` applies `nextOut()` to the opposite and takes the opposite again; hence, it visits the arcs into v in the order of the arcs out of v. Each line costs $O(n)$ or $O(m)$, and the copy from a graph adds one scan of the nodes and one of the edges. □

abLEMON. `StaticGraph` is new in abLEMON (version 1.5.0): LEMON 1.3.1 had only the static digraph. Also, the copy constructor and the assignment of `StaticDigraph` are private, as those of the other structures, while in LEMON 1.3.1 the implicit ones copied the pointers to its arrays, which were then deleted twice.

CompactDigraph. It is a `StaticDigraph` without the arrays of the incoming arcs and of the sources: it stores only `node_first_out` and `arc_target`, i.e., one `int` per node and one per arc, and an `Arc` handle carries its source together with its index. It is built as a `StaticDigraph` is (also sorting the arcs by its own `ArcLess` when it copies a digraph). Its `InArcIt` exists only to conform to `concepts::Digraph`, since with no index of the incoming arcs it scans all the arcs. Also, `arc(i)` (which is not static here) finds the source of the arc by a binary search on `node_first_out`. An algorithm that visits the incoming arcs (e.g., one run on the reverse digraph of chapter 5) should use `StaticDigraph` instead.

Lemma 4.5. *In `CompactDigraph`, `arc(i)` (and `arcFromId(i)`) returns the arc of index i with its source in $O(\log n)$ time, visiting the arcs into one node takes $O(m)$ time, and into all the nodes $O(nm)$.*

Proof. The array `node_first_out` is nondecreasing, and the arc i leaves the node v with `node_first_out[v]` $\le i <$ `node_first_out[v+1]`; this v is the last index whose entry is at most i, since the nodes with no arc between v and the next node with arcs share the entry of the latter, which is larger than i. The binary search `std::upper_bound` finds it in $O(\log n)$ comparisons. `firstIn()` and `nextIn()` move along all the arcs in the order of `ArcIt` and stop at those with the given target. This takes $O(n+m)$ time for one node, since the nodes with no arc are skipped by `nextSource()`, i.e., $O(m)$ when there is no isolated node, and n times as much for all. □

abLEMON. `CompactDigraph` comes from the repository of LEMON after its release 1.3.1, and it is distributed for the first time in abLEMON, whose documentation also gives the cost of its `InArcIt`.

The program `manual/examples/04-static-graphs.cc` (the test `manual_04_static_graphs`) builds `StaticDigraph` and `CompactDigraph` from a sorted list of arcs and `StaticGraph` from a list of edges in arbitrary order. It also copies a `ListDigraph` into a `StaticDigraph` with one of its maps and checks that the arcs out of a node come sorted by target, and checks the counts and the indices of the implicit graphs of the next two sections.

4.5 Full graphs

In `lemon/full_graph.h` we find `FullDigraph`, `FullGraph` and `FullBpGraph`, the complete graphs, which store only their sizes and compute all the rest from the indices; they take $O(1)$ memory, and each of their functions takes $O(1)$ time. `FullDigraph(n)` has the nodes $0,\dots,n-1$ and an arc from each node to each node, loops included, i.e., n^2 arcs, the arc from s to t having the identifier $ns+t$. `FullGraph(n)` has an edge between each pair of distinct nodes, i.e., $n(n-1)/2$ edges and no loop, numbered as Lemma 4.6 states. `FullBpGraph(r, b)` has r red nodes (the nodes $0,\dots,r-1$), b blue ones (the nodes $r,\dots,r+b-1$), and the rb edges between them, the edge between the red node of index i and the blue node of index j having the identifier $i+rj$; the default constructors give the empty graphs. Nodes are given by their index, as `g(i)` (or `g.redNode(i)` and `g.blueNode(i)` in `FullBpGraph`), and `index(v)` is the inverse. Also, `g.arc(s, t)` and `g.edge(u, v)` give the arc and the edge between two nodes, which are `INVALID` for $u = v$ in `FullGraph` (and for two nodes of one colour in `FullBpGraph`), and `findArc()` and `findEdge()` of section 4.8 use them, since the classes have the tags `FindArcTa`

`g` and `FindEdgeTag`. `FullBpGraph` also has `valid(item)`. Finally, `resize(n)` (or `resize(r, b)`) changes the number of nodes; it notifies the maps that all the items were erased and then that the new ones were built, and hence the values of the maps are lost.

Lemma 4.6. *Let $n \geq 2$ and $k = \lfloor (n-1)/2 \rfloor$. The map that `FullGraph(n)` uses from a pair $u < v$ to the identifier of its edge,*

$$e(u,v) = \begin{cases} un + v & \text{if } u < k, \\ (n-2-u)\,n + (n-1-v) & \text{if } u \geq k, \end{cases} \tag{4.1}$$

is a bijection onto $\{0, \ldots, n(n-1)/2 - 1\}$, whose inverse writes $e = qn + r$ with $0 \leq r < n$ and returns (q, r) if $q < r$ and $(n-2-q, n-1-r)$ otherwise; the arcs of the edge e are $2e+1$, from u to v, and $2e$.

Proof. Read $e = qn + r$ as the cell (q, r) of an array with n columns. The first case of (4.1) takes the cells of the rows $q < k$ above the diagonal ($r > q$). The second, with $q = n-2-u$ and $r = n-1-v$, the cells with $r \leq q$ (since $v > u$ gives $n-1-v \leq n-2-u$) of the rows $q \leq n-2-k$. Each case is injective, and the two sets of cells are disjoint, one being above and one on or below the diagonal. If $n = 2k+1$, then $n-2-k = k-1$, and the rows $0, \ldots, k-1$ are covered entirely, which gives the $kn = n(n-1)/2$ smallest identifiers. If $n = 2k+2$, then $n-2-k = k$, the rows $0, \ldots, k-1$ are covered entirely and the row k in its cells $r \leq k$, which gives the $kn + k + 1 = n(n-1)/2$ smallest identifiers. In both cases the image is an initial segment of the right size; the inverse follows by reading the case from the position of the cell with respect to the diagonal. The numbering of the arcs is that of the code of `arc()`, `source()` and `target()`. □

4.6 Grid and hypercube graphs

The headers `lemon/grid_graph.h` and `lemon/hypercube_graph.h` define two more implicit undirected graphs, with $O(1)$ memory, constant time counting and the tags `FindArcTag` and `FindEdgeTag`.

GridGraph. `GridGraph(w, h)` has wh nodes, the node (i, j) in column $i \in \{0, \ldots, w-1\}$ and row $j \in \{0, \ldots, h-1\}$ having the index $i + jw$, and an edge between two nodes whose positions differ by 1 in one coordinate, the vertical ones first (Figure 4.2), with the origin at the bottom left corner. The node at a position is `g(i, j)` (whose range is checked by a `LEMON_DEBUG`, i.e., only when `LEMON_ENABLE_DEBUG` is defined), and `col(v)`, `row(v)` and `pos(v)`, a `dim2::Point< int >` (cf. chapter 8), give the position of a node, while `width()` and `height()` give the size. Also, the arcs `right(v)`, `left(v)`, `up(v)` and `down(v)` leave v in the four directions, and they are `INVALID` on the border. Its read maps `IndexMap`, `ColMap` and `RowMap`, returned by `indexMap()`, `colMap()` and `rowMap()`, give the position (a `dim2::Point< int >`), the column and the row of each node (e.g., for the coordinates of a drawing), and `resize(w, h)` changes the size as in the full graphs (we remark that the class has no default constructor).

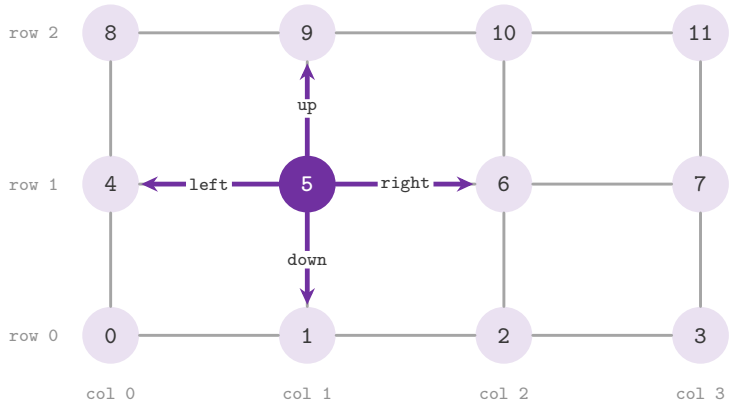


Figure 4.2. GridGraph(4, 3): the index of each node, and the arcs that right(), left(), up() and down() give for node 5, i.e., g(1, 1).

Lemma 4.7. *`GridGraph(w, h)` has $2wh - w - h$ edges. For $e < w(h-1)$ the edge e joins the nodes e and $e + w$ (vertical), and for $e = w(h-1) + e'$ with $0 \leq e' < h(w-1)$ it joins the nodes $i + jw$ and $i + 1 + jw$, where $j = \lfloor e'/(w-1) \rfloor$ and $i = e' \bmod (w-1)$ (horizontal); this numbering is a bijection between the identifiers and the pairs of adjacent positions.*

Proof. A vertical pair is $\{(i,j),(i,j+1)\}$ with $j < h-1$, whose lower node has index $i+jw < w(h-1)$, and every index below $w(h-1)$ is the lower node of exactly one such pair. Hence, $e \mapsto \{e, e+w\}$ numbers the $w(h-1)$ vertical pairs. A horizontal pair is $\{(i,j),(i+1,j)\}$ with $i < w-1$, and $(i,j) \mapsto i+j(w-1)$ is a bijection from these $h(w-1)$ pairs onto $\{0,\dots,h(w-1)-1\}$, whose inverse is the one in the statement. The total is $w(h-1)+h(w-1) = 2wh-w-h$. □

HypercubeGraph. `HypercubeGraph(d)`, for $d \geq 1$ (which a `LEMON_ASSERT` checks), has the 2^d nodes whose indices are the integers of d bits, and an edge between two nodes whose indices differ in exactly one bit. Since the numbers of edges and arcs are of type `int`, the largest dimension is 26 on a platform with 32-bit `int` (since $26 \cdot 2^{26} < 2^{31} \leq 27 \cdot 2^{27}$). Thus, the node of index i is `g(i)` and `index(v)` the inverse; `dimension()` is d, `dimension(e)` and `dimension(a)` give the bit in which the end nodes of an edge or an arc differ, `projection(v, k)` returns whether bit k of the index of v is one, and `resize(d)` changes the dimension. Its nested read map `HyperMap< T , BF = std::plus< T > >` is constructed on the graph, a range of d values $x_0,\dots,x_{d-1}$ (whose length a `LEMON_ASSERT` checks), an initial value `fv` (default 0) and a binary function `bf`. It gives for each node the accumulation of `bf` over the values of the bits that are one in its index, starting from `fv`, in $O(d)$ time; with the default function it is $\texttt{fv} + \sum_{k:\,\text{bit } k=1} x_k$, which gives, e.g., the position of each node in a drawing of the hypercube from d base vectors.

Lemma 4.8. *`HypercubeGraph(d)` has $d\,2^{d-1}$ edges, the edge of identifier $e = k\,2^{d-1}+b$, with $0 \leq k < d$ and $0 \leq b < 2^{d-1}$, joining the node u obtained by inserting a zero bit at position k into b and the node $v = u+2^k$; this numbering is a bijection, and `findEdge()` computes its inverse in $O(d)$ time.*

Proof. Two indices differ only in the bit k if and only if they are u and $u+2^k$ with bit k of u zero. Deleting bit k from u gives a $(d-1)$-bit integer b, and inserting a zero bit at position k into any such b gives back a u with bit k zero. Hence, for each k the edges of dimension k correspond one to one to the 2^{d-1} values of b, and $(k,b) \mapsto k\,2^{d-1}+b$ is a bijection onto $\{0,\dots,d\,2^{d-1}-1\}$. Finally, `findEdge(u, v)` takes $u \oplus v$, rejects it unless it is a single bit 2^k, which it finds by at most d shifts, and returns $k\,2^{d-1}$ plus the index u with bit k deleted. □

4.7 Arc sets and edge sets

The header `lemon/edge_set.h` defines four structures that have their arcs (or edges) on the nodes of another graph: `ListArcSet< GR >` and `SmartArcSet< GR >`, which conform to `concepts::Digraph`, and `ListEdgeSet< GR >` and `SmartEdgeSet< GR >`, which conform to `concepts::Graph`. Their parameter `GR` is the type of the graph whose nodes are shared, which may be a digraph or a graph. A set has the `Node` type of `GR` and the nodes of `GR` (with their identifiers, iterators and notifier), and only the arcs or edges belong to it. Hence, several sets may be built on one graph (e.g., the arcs of several commodities, or a sparse subgraph whose arcs are not in the graph), and the node maps of `GR` serve all of them. A set is constructed on the graph, `ListArcSet< GR > s( g )`, and it has `addArc(s, t)` (or `addEdge(u, v)`), `clear()`, and the functions of its concept; its counting is linear.

Both `List` sets link the arcs of each node in doubly linked lists, as `ListDigraph` and `ListGraph` do, and they also have `erase(a)` (or `erase(e)`). They observe the nodes of the underlying graph, through a private node map of the nested class `NodesImpl`; when a node is erased from it, they erase its arcs first, and when the graph is cleared they clear themselves. The two `Smart` sets store the arcs in a vector with singly linked lists, which is slightly faster, and cannot erase them. They observe the nodes too, but an erased node that has an arc in the set invalidates the whole set (by the exception `ImmediateDetach` of appendix A), which can no longer be used; their function `valid()` returns whether this has not happened. An erased node with no arc in the set leaves it valid.

Lemma 4.9. *In the `List` sets, `addArc()` (or `addEdge()`) takes $O(1)$ amortized time, `erase(a)` (or `erase(e)`) $O(1)$ time, and the erasure of a node of the underlying graph $O(1)$ time plus $O(1)$ for each arc of the set at the node; in the `Smart` sets, `addArc()` (or `addEdge()`) takes $O(1)$ amortized time, and the erasure of a node of the underlying graph $O(1)$ time.*

Proof. The arcs are kept in a vector with a free list (the `List` sets) or appended to it (the `Smart` sets), and linked at the head of the lists of their ends, as in Lemma 4.1. The `List` sets unlink an arc in $O(1)$ time from its doubly linked lists, and erase the arcs at an erased node one by one from the heads of its lists. The `Smart` sets only test whether the lists of the erased node are empty, in $O(1)$ time, and detach themselves otherwise. □

The program of section 4.9 builds a `ListEdgeSet` and a `SmartArcSet` on a `ListDigraph`, and checks both behaviors.

4.8 Counting and finding arcs

Counting. In `lemon/core.h`, the functions `countNodes(g)`, `countArcs(g)`, `countEdges(g)`, `countRedNodes(g)` and `countBlueNodes(g)` return the number of items of a kind. They call `nodeNum()`, `arcNum()`, `edgeNum()`, `redNum()` or `blueNum()` if the structure defines the corresponding tag (`NodeNumTag`, `ArcNumTag`, `EdgeNumTag`; `NodeNumTag` also selects `redNum()` and `blueNum()`), and count the items with an iterator otherwise, whereas `countItems< GR , Item >(g)` always iterates. The choice is made at compile time by the helper classes `CountNodesSelector`, `CountArcsSelector`, `CountEdgesSelector`, `CountRedNodesSelector` and `CountBlueNodesSelector` of the internal namespace `_core_bits` of `lemon/core.h`, specialized by `enable_if` on the tag, which a user does not call directly. Similarly, `countOutArcs(g, v)`, `countInArcs(g, v)` and `countIncEdges(g, v)` iterate on the arcs or edges at a node, through the template `countNodeDegree< GR , DegIt >(g, v)`, which counts the items of any incidence iterator `DegIt`.

Finding the arcs between two nodes. A call `findArc(g, u, v, prev = INVALID)` returns an arc from `u` to `v`, the first one if `prev` is `INVALID` and the one after `prev` otherwise, or `INVALID` if there is none; repeated calls, each with the previous result, enumerate the parallel arcs. On a structure with the tag `FindArcTag` (the full, grid and hypercube graphs) it calls `g.findArc()`, and on the others it scans the arcs out of `u`. An iterator `ConArcIt< GR >`, constructed as `ConArcIt< GR > a( g , u , v )`, wraps the same enumeration, and `ConArcIt< GR >( g , a )` continues it from the arc `a`. For undirected graphs, `findEdge(g, u, v, prev = INVALID)` and `ConEdgeIt< GR >` find the edges between `u` and `v` (in either orientation) by scanning the edges at `u`, or by `g.findEdge()` with `FindEdgeTag`. Each loop at `u` is returned once for $u = v$, and `ConEdgeIt< GR >( g , e )` continues the enumeration from the edge `e`, on the edges between its two ends. The two functions dispatch on the tags through the internal classes `FindArcSelector` and `FindEdgeSelector`.

> **abLEMON.** `ConEdgeIt` constructed from an edge continues on the edges between the two ends of that edge; in LEMON 1.3.1 it left its two nodes `INVALID`, and its `operator++` searched the edges between two `INVALID` nodes. The test `graph_utils_test` counts the parallel edges after one.

Lemma 4.10. *(i) Without the tags, the counting functions take $O(n)$, $O(m)$ (or the number of edges) and $O(d^+(v))$, $O(d^-(v))$, $O(d(v))$ time, and with them $O(1)$ time. (ii) Without `FindArcTag`, the calls a_1 = `findArc(g, u, v)`, a_{i+1} = `findArc(g, u, v, a_i)` return each arc from `u` to `v` exactly once, in the order of `OutArcIt` at `u`, and then `INVALID`, in $O(d^+(u))$ time in all. (iii) Without `FindEdgeTag`, `findEdge()` does the same for the edges between `u` and `v`, each loop once, in $O(d(u))$ time in all.*

Proof. (i) Each iteration visits each item once (it is the contract of the iterators, cf. appendix A), and a counter is increased in $O(1)$ per item; the tagged functions return a stored number. (ii) The first call starts at `firstOut(e, u)`, and each call resumes from the successor of `prev` in the list of the arcs out of `u`. Each call moves along that list until an arc with target `v` or the end, so that the arcs out of `u` are visited once over all the calls, and every arc from `u` to `v` is returned when it is met. (iii) For $u \neq v$ the scan follows the incidences of `u` by `firstInc()` and `nextInc()` and stops at the edges whose other end is `v`. In it, the direction flag of `prev` is recomputed as whether `u` is its first end; for $u = v$ it stops only at the incidences with the flag `true` and the other end `u`, which a loop has once of its two. The incidences of `u` are $d(u)$, each visited once. □

Look-up structures. When many arcs are looked for in one digraph, three classes of `lemon/core.h` answer faster than the scan, at the price of some memory and of a preprocessing. They are queried by `operator()`, as in `ArcLookUp< GR > look( g )` followed by `look( s , t )`.

- `ArcLookUp< GR >` builds, for each node, a binary search tree of the arcs out of it, keyed by their targets under the order `<` of the nodes (the order of the identifiers, in the structures of this chapter; Figure 4.3). Its constructor calls `refresh()`, which builds all the trees, and `refresh(v)` rebuilds the tree of one node. A query `look(s, t)` returns an arc from `s` to `t`, or `INVALID`. It finds one arc only. The structure is static: after a change of the digraph `refresh()` must be called (or `refresh(v)`, if only the arcs out of `v` changed) before the next query.
- `AllArcLookUp< GR >` is an `ArcLookUp` that also links the arcs with the same source and target, and `look(s, t, prev = INVALID)` enumerates them: the first if `prev` is `INVALID`, the one after `prev` otherwise. It is static as well; its `refresh(v)` and `refresh()` rebuild the trees and the links.
- `DynArcLookUp< GR >` keeps a self-adjusting binary search tree, the splay tree of Sleator and Tarjan 1985, for the arcs out of each node, and it observes the arcs of the digraph, so that it follows each addition and erasure with no refresh (a build or a clear of the digraph rebuilds or empties it). A query `look(s, t, p = INVALID)` enumerates the arcs from `s` to `t` as the one of `AllArcLookUp` does. However, as the header remarks, its constant factors are often larger than those of the two static classes, which are therefore preferable on a digraph that changes rarely.

The trees of `ArcLookUp` are built by Algorithm 4.2: `refresh(v)` sorts the arcs out of `v` (by its private comparator `ArcLess`) and makes the middle one the root of each subtree.

Algorithm 4.2 (The search tree of ArcLookUp).

```
refresh(v):
 1  L = the arcs out of v, sorted by their target
 2  head[v] = build(L, 0, |L|-1)          (INVALID if L is empty)
build(L, a, b):
 3  m = (a + b) / 2
 4  left[L[m]]  = build(L, a, m-1) if a < m, else INVALID
 5  right[L[m]] = build(L, m+1, b) if m < b, else INVALID
 6  return L[m]
lookup(s, t):                              (operator()(s, t))
 7  e = head[s]
 8  while e != INVALID and target(e) != t:
 9    e = left[e] if t < target(e) else right[e]
10  return e
```

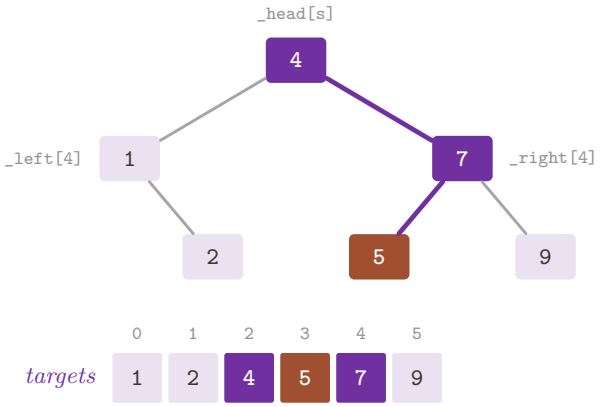


Figure 4.3. The tree of ArcLookUp for a node whose arcs go to the nodes 1, 2, 4, 5, 7 and 9; the look-up of target 5 visits the arcs to 4, 7 and 5.

Theorem 4.2. *The tree that Algorithm 4.2 builds on k arcs is a binary search tree on their targets of height $\lceil \log_2(k+1) \rceil$, whose in-order is the sorted order L. Hence, a query* ***`look(s, t)`*** *returns an arc from* `s` *to* `t` *if there is one, and* ***`INVALID`*** *otherwise, after following at most $\lceil \log_2(d^+(s)+1) \rceil$ arcs of the tree;* ***`refresh(v)`*** *takes $O(d^+(v) \log d^+(v))$ time and* ***`refresh()`*** *$O(n + m \log \Delta)$ time.*

Proof. By induction on $k = b - a + 1$, `build(L, a, b)` returns a tree whose in-order is $L[a..b]$: the root is $L[m]$, its left subtree has the in-order $L[a..m-1]$ and its right one $L[m+1..b]$. Since L is sorted, every target in the left subtree is not larger, and every target in the right one is not smaller, than the target of the root, which is the property of a binary search tree. The subtrees have $\lfloor (k-1)/2 \rfloor$ and $\lfloor k/2 \rfloor$ arcs, so that the height satisfies $h(0) = 0$ and $h(k) = 1 + h(\lfloor k/2 \rfloor)$; if $2^{j-1} < k+1 \le 2^j$, then $2^{j-2} < \lfloor k/2 \rfloor + 1 \le 2^{j-1}$, and $h(k) = \lceil \log_2(k+1) \rceil$ follows by induction on j. The query of lines 7 to 10 follows one path from the root, going left when t is smaller than the target of the current arc and right otherwise; by the search tree property, an arc with target t, if any, lies on that path, and the path has at most $h(d^+(s))$ arcs. The sort dominates the cost of `refresh(v)`, and `refresh()` calls it for every node, with $\sum_v d^+(v) \log d^+(v) \le m \log \Delta$. □

Lemma 4.11. *In* ***`AllArcLookUp`****, the calls $a_1 =$* ***`look(s, t)`****, $a_{i+1} =$* ***`look(s, t, a_i)`*** *return each arc from* `s` *to* `t` *exactly once and then* ***`INVALID`****; the first call takes $O(\log d^+(s))$ time and each of the others $O(1)$.*

Proof. By Theorem 4.2 the arcs from `s` to `t` are consecutive in the in-order of the tree of `s`. The first call descends as in Algorithm 4.2 until an arc with target t. Then it keeps descending, to the left from each arc with target t (recording it) and to the right from the others; the last arc recorded is the first of the consecutive ones in the in-order, and the path has at most $\lceil \log_2(d^+(s)+1) \rceil$ arcs. After the trees are built, `refresh` visits each tree in reverse in-order and sets the link `next` of each arc to its in-order successor if this has the same target, and to `INVALID` otherwise; the next calls follow these links, in $O(1)$ each, through the consecutive arcs with target t. □

Theorem 4.3. *In* ***`DynArcLookUp`****, consider a sequence of q operations on the tree of one node s with at most d*

arcs, each of which is an addition or an erasure of an arc of the digraph, or a first query `look(s, t)`*. Its total time is* $O(d \log d + q \log d)$*, i.e.,* $O(\log d)$ *amortized per operation. Furthermore, a first query* `look(s, t)` *splays the arc* x_t *that ends its search path, which is the leftmost arc with target at least* t *or the rightmost one with target smaller than* t*; for a sequence of first queries in which the arc* a *is splayed* $q_a \geq 1$ *times, the total time is* $O(d \log d + \sum_a q_a(1 + \log(q/q_a)))$*, i.e., within a constant factor of the best static search tree for these frequencies.*

Proof sketch. Each operation either costs $O(1)$ or follows a path from the root and then splays the last node of the path, i.e., moves it to the root by the rotations zig, zig-zig and zig-zag of the function `splay()`. An addition inserts the arc as a leaf and splays it. An erasure costs $O(1)$ when the arc has at most one child, or when its left child has no right one, and otherwise replaces the arc by its in-order predecessor and splays the old parent of the predecessor. A first query descends to the left at the arcs with target at least t and to the right at the others, and splays the last arc visited, which is therefore one of the two in-order neighbours of the boundary between the targets smaller than t and the others. With the potential $\Phi = \sum_a \log_2 s(a)$, where $s(a)$ is the size of the subtree of a, the access lemma of Sleator and Tarjan 1985, Lemma 1 bounds the amortized cost of splaying a node at the end of a path by $3 \log_2 d + 1$ (the path being paid by the rotations), and an erasure does not increase Φ, since every subtree size it changes decreases. The initial potential of the balanced trees of `refresh()` is at most $d \log_2 d$, which gives the first bound (the balance theorem, ibid., Theorem 1); with the weights q_a/q in place of the unit ones, the same lemma gives the second (the static optimality theorem, ibid., Theorem 2). □

We remark that Theorem 4.3 does not cover the next queries `look(s, t, p)`. Such a query takes the in-order successor of p and splays it; when p has a right subtree, the successor is the leftmost arc of that subtree, and the descent to it is paid by the splay, while otherwise the query climbs from p to the successor, an ancestor of p, and this climb is not paid by the splay of the ancestor. Hence, the statement of the header that every look-up takes $O(\log d)$ amortized time is proved here only for the first queries. The program `manual/examples/04-arc-lookup.cc` (the test `manual_04_arc_lookup`) compares the four ways of finding an arc on a digraph with parallel arcs, enumerates the three parallel arcs by `findArc()`, `ConArcIt`, `AllArcLookUp` and `DynArcLookUp`, and shows that only `DynArcLookUp` sees a new arc before a refresh.

> **abLEMON.** `AllArcLookUp::refresh()`, the variant without arguments, calls `refresh(n)` for each node; in LEMON 1.3.1 it passed an arc to `refresh(Node)` and did not compile. The test `graph_utils_test` and the program `manual/examples/04-arc-lookup.cc` call it.

4.9 Copying graphs

Since a graph structure cannot be copied by its copy constructor, its duplicate, possibly into a structure of another type, is made by the classes `DigraphCopy< From , To >`, `GraphCopy< From , To >` and `BpGraphCopy< From , To >` of `lemon/core.h`, or by the functions `digraphCopy(from, to)`, `graphCopy(from, to)` and `bpGraphCopy(from, to)`, which return such an object. Before running it one can request the following data, each by a function that returns the object (so that the requests can be chained).

- `nodeRef(map)`, a map with the nodes of `from` as keys, into which the copy writes the image of each node, and `nodeCrossRef(map)`, a map with the nodes of `to` as keys, into which it writes the original of each node; `arcRef()`, `arcCrossRef()`, `edgeRef()`, `edgeCrossRef()`, `redRef()`, `redCrossRef()`, `blueRef()` and `blueCrossRef()` do the same for the other kinds of items;
- `nodeMap(fmap, tmap)`, which copies the values of the map `fmap` of `from` into the map `tmap` of `to`, item by item; `arcMap()`, `edgeMap()`, `redNodeMap()` and `blueNodeMap()` do the same for the other kinds of items;
- `node(n, tn)`, which writes into `tn` the image of the node `n`, and `arc()`, `edge()`, `redNode()` and `blueNode()` likewise.

Then `run()` clears `to` and copies the structure, followed by the data that were requested (Algorithm 4.3). Internally, each request is an object of one of the classes `MapCopy`, `ItemCopy`, `RefCopy` and `CrossRefCopy` (derived from `MapCopyBase`), and the structure is copied by `DigraphCopySelector`, `GraphCopySelector` or `BpGraphCopySelector`, which call `build()` on a target with `BuildTag`. Also, `GraphCopy` and `BpGraphCopy` compute the images of the arcs (and `BpGraphCopy` those of the nodes, from the red and the blue ones) on demand, by their private maps `ArcRefMap` and `NodeRefMap`. In `GraphCopy` the image of an arc is the arc of the image of its edge whose source is the image of its source (for a loop, the arc with the same `direction()`), so that the orientation of each arc is kept even if the target structure orders the end nodes of its edges differently.

Algorithm 4.3 (DigraphCopy::run()).

```
run():
 1  nr = node map of from, ar = arc map of from      (local references)
```

```
2  if To has BuildTag:  to.build(from, nr, ar)
3  else:
4    to.clear()
5    for each node v of from:  nr[v] = to.addNode()
6    for each arc a of from:   ar[a] = to.addArc(nr[source(a)], nr[target(a)])
7  for each requested node datum:  copy it through nr
8  for each requested arc datum:   copy it through ar
```

Theorem 4.4. *After* `DigraphCopy::run()`, `nr` *is a bijection from the nodes of* `from` *onto those of* `to` *and* `ar` *one from the arcs, with* $\mathrm{source}(\mathrm{ar}[a]) = \mathrm{nr}[\mathrm{source}(a)]$ *and* $\mathrm{target}(\mathrm{ar}[a]) = \mathrm{nr}[\mathrm{target}(a)]$ *for every arc* a*; the requested data are the images of those of* `from` *under these bijections. The copy takes* $O(n+m)$ *time plus the time of the data requested, except when the target is built by* `build()`*, whose cost is that of section 4.4 (e.g.,* $O(n + \sum_v d^+(v) \log d^+(v))$ *for a* `StaticDigraph`*). The same holds for* `GraphCopy` *and* `BpGraphCopy` *with the edges, and with the red and blue nodes.*

Proof. Without `BuildTag`, `to` is cleared, and each node of `from` gets a new node (distinct, since they are added one by one). Each arc a new arc between the images of its ends, which gives the two bijections and the relation of the ends; each addition takes $O(1)$ amortized time on the structures of this chapter. With `BuildTag`, `build()` writes the same maps by Lemma 4.4 and Theorem 4.1 (whose copies from a graph keep the end nodes of each edge). A reference, a cross reference, a map or an item is then copied by one pass over the items of its kind (or one access) through `nr` or `ar`. In `GraphCopy`, the image of an arc is computed in $O(1)$ from the image of its edge, as described above. The node references are always computed, because the arcs need them. □

abLEMON. `DigraphCopy`, `GraphCopy` and `BpGraphCopy` hold the objects that copy the requested data, which their destructor deletes; in abLEMON they have a move constructor and no copy constructor, which `digraphCopy()` and the other functions use to return them by value. LEMON 1.3.1 copied them, and a copy that was not elided deleted those objects twice.

The program `manual/examples/04-copy-and-sets.cc` (the test `manual_04_copy_and_sets`) copies a `ListGraph` with a loop into a nonempty `SmartGraph`. It requests the node references, the cross references of the edges, an edge map and the image of one backward arc; then it builds a `ListEdgeSet` and a `SmartArcSet` on a `ListDigraph`, and checks that erasing a node of the underlying digraph erases its edges from the first and invalidates the second.

4.10 Exercises

Exercise 4.1. Build a `ListDigraph` with 10^5 nodes and 10^6 random arcs, and measure the time of a loop that visits the arcs out of each node, on it and on the `StaticDigraph` obtained by `digraphCopy()`. Explain the difference by the storage of Figure 4.1. (The test `test/digraph_test.cc` checks that the two structures have the same arcs.)

Exercise 4.2. Find, from the code of `ListDigraph::contract()`, which arcs `contract(u, v)` with `r = true` erases and which loops at `u` it keeps, and write the sequence of calls that contracts `u` and `v` and erases all the loops at `u`. (The program `manual/examples/04-list-graph.cc` contracts two nodes.)

Exercise 4.3. Modify Algorithm 4.1 so that the arcs out of each node come sorted by the index of their target, still in $O(n+m)$ time, and prove it as Theorem 4.1 is proved. (The test `test/graph_test.cc` checks a `StaticGraph` built from a random graph with loops and parallel edges.)

Exercise 4.4. Prove that `FullBpGraph(r, b)` numbers its edges bijectively by $i + rj$, as Lemma 4.6 does for `FullGraph`, and check the formulae of Lemmas 4.6 to 4.8 against `nodeNum()`, `edgeNum()` and `arcNum()` and against `findEdge()` for small sizes. (The program `manual/examples/04-static-graphs.cc` checks some of them.)

Exercise 4.5. On a digraph with n nodes and m arcs, compare the total time of q queries by `findArc()`, by `ArcLookUp` (preprocessing included) and by `DynArcLookUp`, and find, by Lemma 4.10 and Theorems 4.2 and 4.3, the values of q above which each look-up structure becomes the faster choice. (The program `manual/examples/04-arc-lookup.cc` and the test `test/arc_look_up_test.cc` use the three structures.)

Exercise 4.6. Copy a `ListBpGraph` into a `SmartBpGraph` with `bpGraphCopy()`, requesting the references of the red nodes and a blue node map, and check that each edge keeps its red and its blue end. (The test `test/graph_copy_test.cc` copies bipartite graphs.)

5. Graph adaptors

We are concerned in this chapter with the classes of `lemon/adaptors.h`, which present a digraph or a graph to an algorithm as if it were another one, without copying it. The other structure may have fewer nodes or arcs, reversed arcs or no orientation at all, split nodes, or only the arcs of positive residual capacity of a flow. An adaptor stores a pointer to the adapted structure (and, possibly, to some maps), and it answers each query of the graph concepts of chapter 4 by one or more queries on that structure; hence, building it costs $O(1)$ time and memory, while the price is paid at each step of an iteration. Section 5.1 describes what the adaptors have in common, sections 5.2 to 5.5 describe them one family at a time (filters, orientation, residual digraph, node splitting), with the results on the flows and the paths that the residual digraph and the split digraph serve, and section 5.6 collects the exercises. Throughout the chapter, n_0 and m_0 are the numbers of nodes and of arcs (or edges) of the adapted structure.

5.1 Views of a graph

Each adaptor of the header conforms to `concepts::Digraph` or to `concepts::Graph` (cf. chapter 3). Therefore, it can be passed to the algorithms of the library and to the functions of `core.h`, and it can be adapted again: the function `checkCombiningAdaptors()` of `test/adaptors_test.cc` builds, e.g., an `Undirector` of a `SplitNodes` of an `Orienter` of a `GridGraph`, and checks it against `concepts::Graph`. Three properties are shared, in some form, by all the classes of the chapter, and we state them once here.

Constness and modification. The first template parameter is the type of the adapted structure, and it may be given as `const` (e.g., `ReverseDigraph< const ListDigraph >`). In that case the adaptor can be built from a constant reference, and it offers no modifying function. Otherwise, the adaptors that keep the item set of the adapted structure (`ReverseDigraph`, `SubDigraph`, `SubGraph`, `FilterNodes`, `FilterArcs`, `FilterEdges`, `Undirector` and `Orienter`) forward `addNode()`, `addArc()` (or `addEdge()`), `erase()` and `clear()` to it, and one changes the adapted structure through the adaptor. `ResidualDigraph` and `SplitNodes`, instead, take their digraph as implicitly `const`, since an arc added to a residual digraph (or a node added to a split digraph) has no meaning for the underlying one. Each adaptor names the adapted structure by the type `Digraph` (or `Graph`, for the adaptors of a graph), and the filtering adaptors also name their filters by `NodeFilterMap`, `ArcFilterMap` and `EdgeFilterMap`. Each adaptor has a creator function, i.e., `reverseDigraph()`, `subDigraph()`, `subGraph()`, `filterNodes()`, `filterArcs()`, `filterEdges()`, `undirector()`, `orienter()`, `residualDigraph()` and `splitNodes()`, which returns the read-only version (the one on the `const` structure), with the filter maps `const` or not as they are passed.

Items. The `Node` and `Arc` (and `Edge`) types of an adaptor are usually convertible to and from those of the adapted structure, as the notes of the header state class by class. For instance, a node of a `SubDigraph` is a node of the digraph, and an edge of an `Undirector` is an arc of the digraph (the exceptions are given where they occur). Hence, a map of the adapted structure can be read with the items of the adaptor.

Maps and counting. For the adaptors that keep the item set, the `NodeMap`, `ArcMap` and `EdgeMap` of the adaptor are maps of the adapted structure, and they store a value for each of its items (the hidden ones included). `Undirector`, `ResidualDigraph` and `SplitNodes`, whose arcs are not arcs of the digraph, build their arc maps from two maps of the digraph. Counting by `countNodes()`, `countArcs()` and `countEdges()` takes constant time on `ReverseDigraph`, `Undirector`, `Orienter` and `SplitNodes` when the adapted structure counts in constant time (`SplitNodes` counts by `countNodes()` and `countArcs()` of the digraph, whatever the digraph does). The filtering adaptors (and `ResidualDigraph`, which is one of them) count by iterating, in linear time; in fact, their tags `NodeNumTag` and `ArcNumTag` are `False`.

In general, the cost of an iteration on an adaptor is that of the iteration on the adapted structure plus a constant

amount of work per item visited (which is rather small). There is one exception, which may matter in practice: a filtering adaptor skips the hidden items one by one, since it has no list of the visible ones. The following lemma makes this precise.

Lemma 5.1. *Let F be one of `SubDigraph`, `SubGraph`, `FilterNodes`, `FilterArcs`, `FilterEdges` and `ResidualDigraph`, and let v be a node of the adapted structure with $h(v)$ outgoing arcs in it. A complete iteration of `OutArcIt` over v on F visits each of the $h(v)$ arcs once and costs $O(1 + h(v))$ time, a single step of it may cost $\Theta(h(v))$, and a complete iteration of `NodeIt` or `ArcIt` costs $O(1 + n_0)$ or $O(1 + n_0 + m_0)$ time, provided that a read of a filter, a call of `firstOut()`, `nextOut()`, `first()` and `next()` of the adapted structure cost $O(1)$. The same holds for `InArcIt` and, on the graphs, for `IncEdgeIt` and `EdgeIt`.*

Proof. The functions `first()`, `firstOut()`, `next()` and `nextOut()` of the bases `SubDigraphBase` and `SubGraphBase` call the function of the same name of the adapted structure, and then its `next()` (or `nextOut()`) while the current item is hidden. Thus, the adapted iteration is advanced exactly once per item, and each item is tested by at most three reads of the filters (its own value and those of its two ends, or only its own for `FilterArcs` and `FilterEdges`, whose template flag `ch` is `false`). A single step from a visible arc to the next visible one passes over all the hidden arcs in between, which may be all the others. For `ResidualDigraph` the adapted structure is an `Undirector`, whose outgoing iteration over v visits the arcs entering and leaving v in the digraph once each (section 5.3), and the filter is read in $O(1)$ (section 5.4). □

Hence, an algorithm run on a filter that hides most of the items may be (somewhat, or much) slower than the same algorithm run on a copy made with `digraphCopy()` (cf. chapter 4). Of course, the copy costs time and memory, and it does not follow the later changes of the filter.

We remark that `FilterNodes`, `FilterArcs`, `FilterEdges` and `ResidualDigraph` hold, as members, objects to which their base class points (the constant filter that replaces the one they lack and, for the residual digraph, the inner `Undirector` and the filters of the residual capacities). The implicit copy of such an adaptor copies the pointers, which still refer to the members of the original; hence, a copy must not outlive the object it was copied from, and one should pass these adaptors by reference.

The classes of the header are built on base classes that a user does not instantiate, i.e., `DigraphAdaptorBase` and `GraphAdaptorBase` (which forward each query to the adapted structure), `ReverseDigraphBase`, `SubDigraphBase`, `SubGraphBase`, `UndirectorBase`, `OrienterBase` and `SplitNodesBase` (which change the queries), completed by the extenders of `lemon/bits/` (cf. appendix A). The maps of `Undirector` and `SplitNodes` are built on their private classes `ArcMapBase` and `NodeMapBase`, and the arc filters of `ResidualDigraph` are the functors `ResForwardFilter` and `ResBackwardFilter` of the namespace `_adaptor_bits`.

5.2 Reversing and filtering

`ReverseDigraph< DGR >` presents the digraph `DGR` with all its arcs reversed. Its `source()` and `target()` are exchanged, the outgoing arcs of a node are its incoming arcs in the digraph (and vice versa), `findArc(u, v)` looks for an arc from `v` to `u` in the digraph, and `addArc(u, v)` adds the arc (v, u) to it. Its constructor takes the digraph (`explicit ReverseDigraph( DGR & digraph )`), and the adaptor counts in the time of the digraph. One typically uses it to run a search backwards, e.g., to find the nodes from which a target is reachable, or to compute the distances to a node by `Dijkstra` with the same length map.

The filtering adaptors hide some of the items by means of `bool` maps, called filters: an item is visible if and only if its filter value is `true`. `SubDigraph< DGR, NF, AF >` filters both the nodes and the arcs of a digraph, with a node map `NF` and an arc map `AF`, whose defaults are `DGR::NodeMap< bool >` and `DGR::ArcMap< bool >`. An arc is visible when it is enabled and both its ends are enabled. `SubGraph< GR, NF, EF >` does the same for a graph, with an edge filter `EF` (by default `GR::EdgeMap< bool >`), and an edge is visible when it and both its ends are enabled. Three specialized filters keep a constant map, i.e., a `ConstMap` with the inlined value `true` (cf. §6.3), in place of the filter they lack. `FilterNodes< GR, NF >` works on a digraph or on a graph (it chooses the concept from the `UndirectedTag` of `GR`), and it hides the nodes together with the arcs or edges incident to them. `FilterArcs< DGR, AF >` hides arcs of a digraph, and `FilterEdges< GR, EF >` hides edges of a graph (and both their arcs); neither of the two hides a node.

Their constructors take the structure and references to the filters, which the adaptor keeps by pointer, e.g., `SubDigraph( DGR & digraph, NF & node_filter, AF & arc_filter )` and `FilterArcs( DGR & digraph, AF & arc_filter )`. Thus, the filters must outlive the adaptor, and a value written directly into a filter is seen at once through the adaptor. The same can be done through the adaptor itself: `status(item, v)` sets the filter value of the item to `v` and `status(item)` reads it, while `enable(item)` and `disable(item)` are `status(item,`

`true)` and `status(item, false)`. Each of them costs one access to the filter map (a `FilterNodes` has them for the nodes only, a `FilterArcs` for the arcs, a `FilterEdges` for the edges). Building the filters (e.g., `DGR::ArcMap< bool > f( g, true )`, which shows the whole digraph) is left to the caller, since only the caller knows which items are to be seen.

A `SubDigraph` with an all-true node filter and a `FilterArcs` show the same digraph; the difference is one of efficiency. The former checks the filter values of the two ends of each arc it visits (the template flag `ch = true` of `SubDigraphBase`), while the latter, whose node filter is the constant `true`, checks only the arc filter. The search of an arc between two nodes behaves as follows.

Lemma 5.2. *On a filtering adaptor, `findArc(u, v, prev)` (and, on the graphs, `findEdge(u, v, prev)`) returns `INVALID` at once if `u` or `v` is hidden, and otherwise the first visible arc from `u` to `v` that the adapted structure returns after `prev`, or `INVALID` if there is none. It calls `findArc()` (or `findEdge()`) of the adapted structure $k+1$ times, where k is the number of hidden arcs that it skips.*

Proof. The function reads the node filter at `u` and `v`; then it calls `findArc(u, v, prev)` of the adapted structure and, while the arc returned is hidden, calls it again from that arc. Each call either returns `INVALID`, which ends the loop, or returns the next arc from `u` to `v`, which is either the result or one of the k hidden arcs skipped. □

5.3 Changing the orientation

`Undirector< DGR >` presents a digraph as an undirected graph. Each arc of the digraph is an edge of the adaptor, with `u(e)` its source and `v(e)` its target, and, as for the other graphs of the library, each edge also gives two arcs: the forward one (`direction(a) == true`), oriented as the arc of the digraph, and the backward one. The `Edge` type of the adaptor is the `Arc` type of the digraph, while its `Arc` is a pair (arc of the digraph, direction), which converts to the arc of the digraph. The constructor takes the digraph, `addEdge(u, v)` adds the arc (u, v) to it, and the `ArcMap` of an `Undirector` keeps two arc maps of the digraph, one per direction. The outgoing arcs of a node v are the backward arcs of the arcs that enter v in the digraph, followed by the forward arcs of those that leave it; thus, an outgoing iteration over v costs the in-degree plus the out-degree of v in the digraph. Also, `edgeNum()` is `arcNum()` of the digraph and `arcNum()` is twice as much, both in the time of the digraph, and `findArc(s, t)` returns first the forward arcs of the arcs from s to t and then the backward arcs of those from t to s. The identifiers obey the following rule.

Lemma 5.3. *Let i be the identifier of an arc of the digraph. In an `Undirector`, the edge of that arc has identifier i, its forward arc $2i+1$ and its backward arc $2i$; these are distinct for distinct arcs of the adaptor, `arcFromId()` inverts `id()`, and they do not exceed `maxArcId()`, which is $2\,$`maxArcId()`$+1$ of the digraph.*

Proof. The code computes `id(a)` as $(i \ll 1) \mid d$, with d the direction bit, and `arcFromId(x)` as the arc of identifier $x \gg 1$ with the direction $x \,\&\, 1$; the two maps are inverse to each other, and $2i + d \le 2 \max_i i + 1$. □

When an algorithm on the undirected graph needs an arc map and the data are on the digraph, the nested class `Undirector< DGR >::CombinedArcMap< FW, BK >` reads the forward arcs from the arc map `FW` and the backward ones from `BK`, and its `set()` writes to the one of the two that the direction selects. It is created by the static function `combinedArcMap(forward, backward)`, its value type is that of `FW`, and it is a reference map when `FW` is one. For instance, `Undirector< const G >::combinedArcMap( cost, cost )` gives an undirected length (or capacity) taken from one arc map of the digraph.

`Orienter< GR, DM >` goes the other way, and it presents a graph as a digraph in which each edge is an arc. The arc goes from `u(e)` to `v(e)` when the direction map `DM` (a `bool` edge map, by default `GR::EdgeMap< bool >`, named by the type `DirectionMap`) is `true` on the edge, and from `v(e)` to `u(e)` otherwise. Its constructor takes the graph and the direction map, which is kept by pointer, and `reverseArc(a)` negates the direction of an arc by one write to the map. Furthermore, `addArc(u, v)` adds an edge to the graph and sets its direction, so that the new arc goes from `u` to `v`. Since the arcs of the adaptor are the edges of the graph, `arcNum()` is `edgeNum()` of the graph, and the identifier of an arc is that of its edge.

Lemma 5.4. *On an `Orienter`, a complete iteration of `OutArcIt` (or `InArcIt`) over a node v costs $O(1+\deg(v))$, where $\deg(v)$ is the number of edges incident to v in the graph, and `findArc(u, v)` calls `findEdge(u, v)` of the graph once for each edge between u and v that it skips, plus one.*

Proof. `firstOut()` and `nextOut()` iterate on the incident edges of v by `firstInc()` and `nextInc()` of the graph and skip each edge whose direction does not make v its source, i.e., they pass once over each incident edge. Then `findArc()` calls `findEdge()` of the graph and repeats while the source of the edge found is not `u`. □

The function `checkCombiningAdaptors()` of `test/adaptors_test.cc` orients a grid by an `Orienter`, splits its nodes and views the result through an `Undirector`. The program `manual/examples/05-filter-undirector.cc` (the test `manual_05_filter_undirector`) keeps by a `FilterArcs` the arcs of a digraph whose cost is at most a threshold, and it counts the connected components of the result through an `Undirector` before and after enabling one more arc. It also finds by `Bfs` on a `ReverseDigraph` the nodes from which a node is reached.

5.4 The residual digraph

Let $G = (V, A)$ be a digraph, $u : A \to \mathbb{R}_+$ a capacity, and $s \neq t$ two nodes. A flow from s to t is a vector x with $0 \leq x_a \leq u_a$ for each arc a and with

$$\sum_{(i,j)\in A} x_{ij} - \sum_{(j,i)\in A} x_{ji} = 0 \quad \text{for each } i \in V \setminus \{s, t\}, \tag{5.1}$$

and its value $v(x)$ is the left-hand side of (5.1) at $i = s$, i.e., the net flow out of s. The residual digraph of x (cf. Ahuja, Magnanti and Orlin 1993, §2.4) has node set V and arc set $A_f \cup A_b$, where

$$A_f = \{(i, j) : (i, j) \in A,\ x_{ij} < u_{ij}\}, \qquad A_b = \{(j, i) : (i, j) \in A,\ x_{ij} > 0\}. \tag{5.2}$$

Each forward arc $(i, j) \in A_f$ has residual capacity $r_{ij} = u_{ij} - x_{ij}$, and each backward arc $(j, i) \in A_b$ has residual capacity $r_{ji} = x_{ij}$. The union is taken with multiplicity; thus, an arc of G may give two arcs of the residual digraph, and parallel arcs may appear (Figure 5.1). A directed path from s to t in the residual digraph is called an augmenting path, and its bottleneck is the least residual capacity of its arcs. For a set S of nodes with $s \in S$ and $t \notin S$ (a cut), its capacity is $u(S) = \sum_{(i,j)\in A:\, i\in S,\, j\notin S} u_{ij}$.

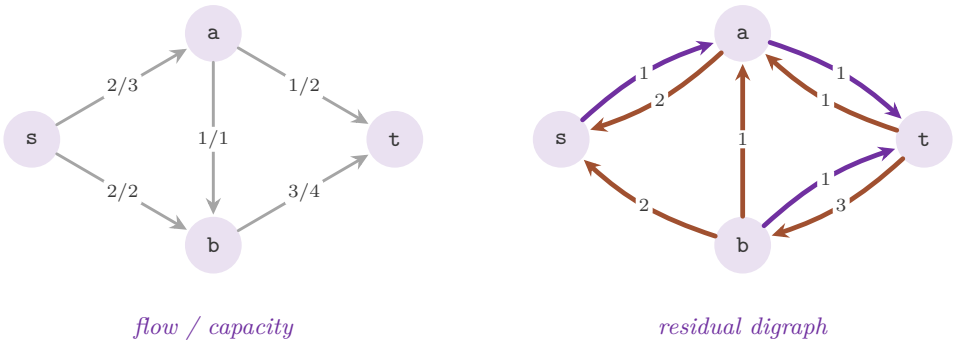


Figure 5.1. A flow x (left, flow/capacity on each arc) and its residual digraph (right): the forward arcs, in the primary colour, where x is below the capacity, and the backward arcs, in the second colour, where x is positive, each with its residual capacity.

`ResidualDigraph< DGR, CM, FM, TL >` implements (5.2) as an adaptor. `DGR` is the digraph (implicitly `const`), `CM` the capacity map (by default `DGR::ArcMap< int >`, implicitly `const`), `FM` the flow map (by default `CM`), which the adaptor may write, and `TL` the tolerance (by default `Tolerance< CM::Value >`, cf. chapter 3). The class names them by the types `Digraph`, `CapacityMap`, `FlowMap` and `Tolerance`, and the value type of the capacities by `Value`. The tolerance decides which arcs exist. A forward arc is there when `tolerance.positive( cap[a] - flow[a] )` holds, and a backward arc when `tolerance.positive( flow[a] )` holds; on floating-point data, an arc whose residual capacity is below the epsilon of the tolerance is therefore hidden. The class is a `SubDigraph` of an `Undirector< const DGR >` of the digraph, with the constant `true` as node filter and, as arc filter, a `CombinedArcMap` of these two tests (the note of the header calls it a combination of `Undirector` and `FilterArcs`, which describes the effect). Hence, an arc of the adaptor is an arc of the `Undirector` (convertible to the arc of the digraph it comes from), the adaptor counts in linear time, and its iterations skip the arcs of zero residual capacity one by one (Lemma 5.1). The constructor `ResidualDigraph( digraph, capacity, flow, tolerance )` takes the digraph and the capacity map by constant reference, the flow map by reference and the tolerance by constant reference, by default `Tolerance()`, and the members that are specific to the residual digraph are the following.

- `residualCapacity(a)` returns $u_a - x_a$ for a forward arc and x_a for a backward one, computed without the tolerance.

- `augment(a, v)` adds `v` to the flow of the original arc if `a` is forward, and it subtracts `v` if `a` is backward. It writes the flow map by `set()`, and it does not check that the result stays within the bounds.
- `forward(a)` and `backward(a)` (static), on an arc of the adaptor, give its direction. On an arc of the digraph, they return the forward or the backward residual arc that corresponds to it, whether or not that arc is currently visible.
- `ResidualCapacity` is a read-only arc map of the adaptor whose value at an arc is `residualCapacity(a)`. It is created by `residualCapacity()`, and its value type is that of the capacity map.

The function `residualDigraph(digraph, capacity, flow)` returns the adaptor with the default tolerance.

The augmenting path method of Ford and Fulkerson 1956 needs only the residual digraph, and Algorithm 5.1 writes it with the classes of the library; the choice of a shortest path, i.e., of a path found by a breadth-first search, is the rule of Edmonds and Karp 1972.

Algorithm 5.1 (Augmenting paths on the residual digraph).

```
input:  digraph G, capacity u, flow x from s to t (e.g., x = 0)
R := ResidualDigraph(G, u, x)
loop
  run Bfs on R from s, stopped when t is reached     (bfs.run(s, t))
  if t is not reached then stop
  P := the path from s to t of the search tree        (bfs.path(t))
  delta := min { residualCapacity(a) : a in P }
  for each arc a of P: augment(a, delta)
end loop
output: x, a maximum flow; S, the nodes reached by the last search,
        the source side of a minimum cut
```

The correctness of the algorithm rests on three facts, which we prove in turn: an augmentation keeps x a flow and increases its value, a flow is maximum exactly when no augmenting path exists, and the search by shortest paths bounds the number of augmentations.

Lemma 5.5. *Let x be a flow from s to t, P an augmenting path of its residual digraph with no repeated node, and $0 < \delta \le \min_{a\in P} r_a$. Augmenting x by δ along P (adding δ to the flow of the original arc of each forward arc of P, and subtracting it from that of each backward arc) gives a flow x' from s to t with $v(x') = v(x) + \delta$.*

Proof. On the original arc of a forward arc the flow grows by $\delta \le u_a - x_a$, and on that of a backward arc it drops by $\delta \le x_a$; hence, $0 \le x' \le u$. An inner node i of P is entered by one arc of P and left by one. Each of the two changes the net flow out of i by δ in opposite directions (a forward arc entering i adds δ to the flow into i, a backward arc entering i removes δ from the flow out of i, and symmetrically for the arc leaving i); thus, (5.1) still holds. At s, the first arc of P either adds δ to an arc leaving s or removes it from an arc entering s, and no arc of P enters s; hence, $v(x') = v(x) + \delta$. □

Theorem 5.1. *(Maximum flow and minimum cut.) For every flow x from s to t and every cut S, $v(x) \le u(S)$. A flow x is maximum if and only if its residual digraph has no path from s to t; in that case, the set S of the nodes reachable from s in the residual digraph is a cut with $u(S) = v(x)$, i.e., a minimum cut.*

Proof. Summing (5.1) over the nodes of $S \setminus \{s\}$ and adding the definition of $v(x)$ gives $v(x) = \sum_{i\in S,\, j\notin S} x_{ij} - \sum_{j\notin S,\, i\in S} x_{ji}$, where the sums run over the arcs of G that leave and that enter S, since the arcs with both ends in S cancel. Then $0 \le x \le u$ gives $v(x) \le u(S)$. If the residual digraph has an augmenting path, it also has one with no repeated node, and Lemma 5.5 gives a flow of larger value; hence, x is not maximum. Conversely, let no path exist and let S be the set of the nodes reachable from s, so that $s \in S$ and $t \notin S$. No residual arc leaves S; by (5.2), each arc of G from S to $V \setminus S$ has $x_{ij} = u_{ij}$, and each arc from $V \setminus S$ to S has $x_{ji} = 0$. The identity above then gives $v(x) = u(S)$, and the first part shows that no flow exceeds $u(S)$. □

Theorem 5.2. *If the capacities and the initial flow are integers, Algorithm 5.1 (with the default tolerance, under which `positive(y)` is $y > 0$) terminates; its final flow is maximum and integral, and the set S is a minimum cut. Hence, on integral capacities a maximum flow exists whose values are integers.*

Proof. By induction, all the flows and residual capacities stay integral, since δ is a minimum of integers; each augmentation increases the value by $\delta \ge 1$ (Lemma 5.5, where the path of the search tree has no repeated node), and the value is bounded by $u(\{s\})$ (Theorem 5.1). Therefore, the algorithm stops, which happens when the complete search from s does not reach t (`run(s, t)` stops early only when it reaches t); the reached nodes are then all the nodes reachable from s, and Theorem 5.1 applies. □

On floating-point data the arcs of residual capacity below the epsilon of the tolerance are hidden; hence, at the end no augmenting path of bottleneck larger than that epsilon remains, and the flow is maximum only up to a quantity of that order per arc of the final cut. The number of augmentations is bounded independently of the capacities when the paths are shortest.

Theorem 5.3. *(Edmonds and Karp.) Let $n = |V|$ and $m = |A|$. Algorithm 5.1 makes at most nm augmentations, whatever the capacities, and it runs in $O(nm(n+m))$ time.*

Proof. Let $d_x(i)$ be the number of arcs of a shortest path from s to i in the residual digraph $R(x)$ ($+\infty$ if none). Let x' be obtained from x by an augmentation along a shortest path P. Every arc of $R(x')$ that is not in $R(x)$ is the reverse of an arc (i,j) of P, for which $d_x(j) = d_x(i) + 1$. Hence, every arc (a,b) of $R(x')$ satisfies $d_x(b) \le d_x(a) + 1$, and, by induction along a shortest path of $R(x')$, $d_x(i) \le d_{x'}(i)$ for every node i: the distances never decrease. An arc of P whose residual capacity equals δ (one exists) is called critical, and it leaves the residual digraph. Let (i,j) be critical at x, so that $d_x(j) = d_x(i) + 1$. Before (i,j) is critical again it must reappear, which requires an augmentation, at some flow z, along a shortest path that contains (j,i); then $d_z(i) = d_z(j) + 1 \ge d_x(j) + 1 = d_x(i) + 2$. Since a node on an augmenting path other than t has distance at most $n-2$, the residual arc (i,j) is critical at most $n/2$ times. The residual arcs are the forward and the backward arc of each arc of G, i.e., $2m$ of them, and each augmentation has a critical arc; hence, there are at most nm augmentations. Each iteration costs one breadth-first search on the adaptor, i.e., $O(n+m)$ time by Lemma 5.1 (an outgoing iteration over i on the `Undirector` visits the arcs entering and leaving i in G), plus $O(n)$ for the path and the augmentations. □

The library has faster maximum flow algorithms, `Preflow` and `EdmondsKarp` (cf. chapter 12). In our view, the adaptor is meant, rather, for the algorithms that search the residual digraph of a flow already found, e.g., for a minimum cut, for the cycles of a circulation, or for a shortest path of reduced costs. The program `manual/examples/05-residual-augment.cc` (the test `manual_05_residual_augment`) runs Algorithm 5.1 on a digraph of five nodes, and it checks the value of the flow and the capacity of the cut of Theorem 5.1 against `Preflow`. The function `checkResidualDigraph()` of `test/adaptors_test.cc` checks the members of the adaptor and augments along shortest paths on another digraph.

5.5 Splitting the nodes

Several network problems (often, but not always, routing problems) put a bound or a cost on the nodes (the number of paths through a node, the capacity of a switch, the cost of a visit), while the algorithms of the library take them on the arcs. The standard transformation (cf. Ahuja, Magnanti and Orlin 1993, §2.4) replaces each node w of G by two nodes w_{in} and w_{out}, joined by the bind arc (w_{in}, w_{out}), which carries the data of the node; each arc (v,w) of G becomes the arc (v_{out}, w_{in}), called an original arc (Figure 5.2). We denote by G' the digraph so obtained, which has $2n$ nodes and $m+n$ arcs.

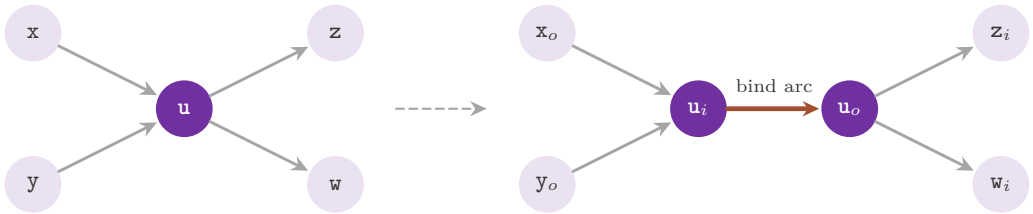


Figure 5.2. The transformation of SplitNodes: the node u becomes the in-node u$_i$, entered by the arcs that entered u, and the out-node u$_o$, left by the arcs that left u, joined by the bind arc.

Lemma 5.6. *Let $s \ne t$. The map that sends a path $(s = w_0, a_1, w_1, \dots, a_k, w_k = t)$ of G with no repeated node to the path of G' that starts at s_{out}, takes the original arc of each a_h and, between two of them, the bind arc of w_h $(0 < h < k)$, is a bijection between the s-t paths of G with no repeated node and the s_{out}-t_{in} paths of G' with no repeated node. These never use the bind arcs of s and t, and they use the bind arc of w exactly when the path of G passes through w as an inner node.*

Proof. In G' the only arc leaving an in-node is its bind arc, and the only arc entering an out-node is its bind arc. Hence, a path of G' from s_{out} alternates an original arc, which ends at an in-node, and the bind arc of that node, until it ends at t_{in}; the bind arc of s would enter s_{out} again, and that of t would leave t_{in}, so a path with no repeated node uses neither. Reading the original arcs in order gives a path of G from s to t, which has no

repeated node since each node w of it gives the distinct nodes w_{in} and w_{out}; this is the inverse of the map of the statement. □

`SplitNodes< DGR >` implements the transformation on the digraph `DGR`, which is implicitly `const`. A node of the adaptor is a node of the digraph with a flag (in or out), and it converts to the node of the digraph. An arc, instead, is either an arc or a node of the digraph (a `BiVariant`); it has conversions to both types, of which only the one that matches its kind is meaningful (that to the arc for an original arc, that to the node for a bind arc). The identifiers are $2i$ for the in-node and $2i + 1$ for the out-node, and $2i$ for an original arc and $2i + 1$ for the bind arc of a node, where i is the identifier in the digraph. `nodeNum()` and `arcNum()` are computed by `countNodes()` and `countArcs()` of the digraph, i.e., in the time of the digraph. Also, `findArc(u, v)` returns the bind arc when `u` is the in-node and `v` the out-node of the same node, the original arcs found by `findArc()` of the digraph when `u` is an out-node and `v` an in-node, and `INVALID` otherwise. Besides the types `Digraph`, `DigraphNode` and `DigraphArc`, the static members translate between the two structures, as follows.

- `inNode(n)` and `outNode(n)`, on a node of the adaptor, return which of the two it is; on a node of the digraph, they return its in-node or its out-node.
- `origArc(a)` and `bindArc(a)` return whether an arc of the adaptor is an original arc or a bind arc.
- `arc(n)` returns the bind arc of a node of the digraph, and `arc(a)` the arc of the adaptor that corresponds to an arc of the digraph.

One gives the data by two combining maps, nested in the class. `CombinedNodeMap< In, Out >`, created by `combinedNodeMap(in_map, out_map)`, reads the in-nodes from the node map `In` and the out-nodes from `Out`. `CombinedArcMap< AM, NM >`, created by `combinedArcMap(arc_map, node_map)`, reads the original arcs from the arc map `AM` and the bind arcs from the node map `NM`. Their value types are those of `In` and `AM`, respectively, and their `set()` writes to the map that the kind of the item selects. In this way, one may turn a node capacity into the capacity of a bind arc, and the following form of the theorem of Menger 1927 (cf. Ahuja, Magnanti and Orlin 1993, §6.6) computes the largest number of paths with no inner node in common as a maximum flow.

Theorem 5.4. *Let $s \neq t$, and give G' integral capacities: 1 on the bind arc of each node other than s and t, 1 on the original arcs of the arcs from s to t, at least 1 on the other original arcs, and any value on the bind arcs of s and t. The value of a maximum flow from s_{out} to t_{in} in G' equals the largest number of distinct s-t paths of G with no repeated node and no inner node in common.*

Proof. Let $P_1, \dots, P_k$ be such paths. By Lemma 5.6 their images in G' are paths from s_{out} to t_{in}; two of them share no bind arc, since they have no inner node in common, and no original arc, since an arc of G with an inner end belongs to one path only and an arc from s to t is a path by itself. Sending one unit along each image gives a flow of value k within the capacities. Conversely, by Theorem 5.2 there is a maximum flow y of integral values, say of value k. An integral flow of value k is the sum of k unit flows along paths from s_{out} to t_{in} with no repeated node and of a circulation. To see this, start from s_{out} and follow arcs of positive flow, which exist by (5.1) at each node other than t_{in}, until t_{in} is reached or a node repeats; in the first case subtract one unit along the path, in the second along the cycle, and repeat (each step lowers the sum of the flows, cf. the flow decomposition of Ahuja, Magnanti and Orlin 1993, §3.5). The k unit paths map to s-t paths of G by Lemma 5.6. Two of them that shared an inner node w would carry two units on the bind arc of w, whose capacity is 1, and two of them that were the same arc from s to t would carry two units on it. Hence, they are k distinct paths with no inner node in common. □

The program `manual/examples/05-split-nodes.cc` (the test `manual_05_split_nodes`) computes in this way, by `Preflow` on a `SplitNodes`, the largest number of paths with no inner node in common in a digraph of six nodes, and it checks that the split digraph has 12 nodes and 14 arcs.

5.6 Exercises

Exercise 5.1. Write the type of the adaptor that presents the subgraph of a `ListDigraph` induced by a set of nodes, with the arcs reversed, to an algorithm that takes a `const` digraph. Say which of its operations cost more than the corresponding operation of the digraph (Lemmas 5.1 and 5.2). The functions `checkReverseDigraph()` and `checkSubDigraph()` of `test/adaptors_test.cc` check the two adaptors separately.

Exercise 5.2. Modify `manual/examples/05-residual-augment.cc` so that each augmentation uses a path of largest bottleneck capacity (a "widest" path) instead of a shortest one, and compare the number of augmentations on a digraph with "large" capacities. The value must still agree with `Preflow`.

Exercise 5.3. Build a digraph with two parallel arcs and an arc in each direction between two nodes, give it

a flow that is positive and below the capacity on each arc. Then check with `countArcs()` that the residual digraph has $|A_f| + |A_b|$ arcs (cf. (5.2)), and with `findArc()` that it returns all of them between the two nodes (Lemma 5.2).

Exercise 5.4. Starting from `manual/examples/05-split-nodes.cc`, use `SplitNodes` and a minimum cost flow algorithm (cf. chapter 13) to find k paths from s to t with no inner node in common and of least total length, and prove, along the lines of Theorem 5.4, that the paths found have this property.

Exercise 5.5. Given an undirected graph and an orientation of its edges in a `bool` edge map, use `Orienter` and an algorithm for the strongly connected components (cf. chapter 18) to decide whether the orientation is strongly connected. Then reverse one arc by `reverseArc()` and decide again, without rebuilding anything; the function `checkOrienter()` of `test/adaptors_test.cc` checks `reverseArc()`.

6. Maps

We are concerned in this chapter with the maps of the library, i.e., with the objects that associate a value with a key. Since the graphs store no data of their own, maps are the only way in which data are attached to the nodes, arcs and edges of a graph (cf. chapter 3). A map is any type that offers the types `Key` and `Value` together with an `operator[]` (to read) or a `set()` (to write). Hence, besides the standard graph maps, which store a value for each item, one can pass to an algorithm a map that computes its values on demand, from other maps, from a function or from the graph. Section 6.1 gives the concepts and section 6.2 the standard graph maps, as a user sees them. Sections 6.3 to 6.6 describe the classes of `lemon/maps.h` (the basic maps, the map adaptors, the maps tied to a graph and the iterable maps, with an algorithm that the iterable maps make linear), section 6.7 the functions that scan a map, and section 6.8 collects the exercises. For a map on the items of one kind of a graph, n_K denotes the number of the items of that kind (nodes, arcs or edges), and N_K the largest identifier of that kind plus one.

6.1 The map concepts

The header `lemon/concepts/maps.h` defines four concepts, as archetype classes with a nested `Constraints` template that `checkConcept()` instantiates on a candidate type (cf. chapter 3). `concepts::ReadMap< K, T >` requires the types `Key` and `Value` and the member `Value operator[]( const Key & ) const`. `concepts::WriteMap< K, T >` requires the same types and `void set( const Key &, const Value & )`, and `concepts::ReadWriteMap< K, T >` is both. Finally, `concepts::ReferenceMap< K, T, R, CR >` is a read-write map whose `operator[]` returns a `Reference` (`R`), and a `ConstReference` (`CR`) on a constant map, through which values can be read and assigned. It declares the tag `typedef True ReferenceMapTag`, by which the library recognizes it, and its constraints are enabled only for a type that declares the tag. Note that an algorithm reads a map only through `operator[]` and `set()`, and the result of `operator[]` need not be a reference (in fact, the adaptors of section 6.4 return their values by copy).

Two helpers make writing a map short: `MapBase< K, V >` is an empty base class that only defines `Key` and `Value`, and the classes of `lemon/maps.h` derive from it. `MapTraits< M >`, in `lemon/bits/traits.h`, gives the `ReferenceMapTag` of any map type and the types `ReturnValue` and `ConstReturnValue` that the map returns. These are references for a reference map and values otherwise, and the adaptors use them to forward the reference of the map they wrap.

> **abLEMON.** The archetypes of `lemon/concepts/maps.h` return a static buffer of the size and alignment of the value, instead of dereferencing the address `sizeof(Value)` as LEMON 1.3.1 did (which is undefined behavior, and which the optimizer flagged as an access out of bounds), while the concepts are unchanged.

6.2 Graph maps

Each graph structure of chapter 4 has the nested templates `NodeMap< V >`, `ArcMap< V >` and, if it is undirected, `EdgeMap< V >` (a bipartite graph also has `RedNodeMap< V >` and `BlueNodeMap< V >`). They are reference maps, built by `NodeMap( const Graph & g )`, which value-initializes the values, or by `NodeMap( const Graph & g, const V & value )`. The graph keeps them up to date, since each map registers itself as an observer of the alteration notifier of the graph (cf. appendix A), and it is told of each item added or erased. A key added after the construction of the map receives the value `V()`, not the value given to the constructor, and the value of an erased key is lost. The assignment operators of the standard maps are private; one copies the values of a map into another by `mapCopy()` (section 6.7).

The storage is chosen by `DefaultMap`, in `lemon/bits/default_map.h`: for `bool`, the character types, the integer types, the floating-point types and the pointers, it is a `VectorMap`, i.e., a `std::vector< V >` indexed by the

identifier of the item. For any other value type it is an `ArrayMap`, an array allocated through `std::allocator`, whose capacity is the first power of two above the largest identifier when the map is built, and which doubles until it exceeds the identifier of an item added beyond it.

Lemma 6.1. *A read or a write of a standard graph map costs $O(1)$; the notification of an added item costs $O(1)$ amortized over a sequence of additions, and the memory of the map is $O(N_K)$ values, whatever the number n_K of the items present.*

Proof. Both storages index an array by the identifier of the item, and a `VectorMap` grows by `resize()` of its vector to the new identifier plus one, of amortized constant cost per element added. An `ArrayMap` that grows from capacity c to $2^j c$ moves its $O(n_K) \leq O(c)$ values once, and since the capacities are powers of two the total cost of the moves up to capacity c is $O(c)$, i.e., $O(1)$ per identifier added. The sizes are bounded by N_K (rounded up to a power of two for the `ArrayMap`), and they do not shrink when items are erased. □

Each map also has three iterators, inherited from `MapExtender` (in `lemon/bits/map_extender.h`). `MapIt` iterates on the items and dereferences to a reference to their value, `ConstMapIt` dereferences to a constant reference, and `ItemIt` only iterates on the items. The test `test/maps_test.cc` uses `MapIt` and `ConstMapIt` on the maps of a `SmartDigraph`.

abLEMON. `ArrayMap` constructs and destroys its values through `std::allocator_traits`, since C++20 removed the members `construct()` and `destroy()` of `std::allocator`. When it grows, it moves the values into the larger buffer by `std::move_if_noexcept()`, rather than copying them.

6.3 Basic maps

The basic maps of `lemon/maps.h` need no graph; `NullMap< K, V >` reads `V()` at each key and absorbs whatever is written to it. It is the map to pass (in place of a real one) when an algorithm requires a map whose content is not wanted (e.g., a predecessor map that the caller does not read). `ConstMap< K, V >` reads one value at each key, given to the constructor (`ConstMap( const V & v )`) or default-constructed, and it also absorbs the writes. Its `setAll(v)` changes the value, and a further constructor `ConstMap( const ConstMap< K, V1 > &, const V & v )` builds the map of value `v` from one of another value type. Its specialization `ConstMap< K, Const< V, v > >` carries the value in its type, as the template argument of the empty tag `Const< T, v >`, so that the compiler may fold the read. The filtering adaptors of chapter 5, e.g., use `ConstMap< Node, Const< bool, true > >` in place of the filter they lack. `TrueMap< K >` and `FalseMap< K >` are the read-only constant maps of `true` and `false`, and `IdentityMap< T >` reads the key (its key and value types are both `T`).

`RangeMap< V >` is a `std::vector< V >` with integer keys in $[0, \text{size})$. It is built from a size and a value (`RangeMap( int size = 0, const V & value = V() )`), from a `std::vector`, or from another `RangeMap` whose value type converts to `V`; `size()` returns the size, and `resize(size, value)` changes the key set, giving `value` to the new keys. It is a reference map, whose types `Reference` and `ConstReference` are those of the vector, and, unlike the graph maps, it can be copied and assigned; it is typically used as the cross-reference map of a heap whose items are small integers (cf. chapter 7).

`SparseMap< K, V, Comp >` is a `std::map< K, V, Comp >`, with `Comp = std::less< K >` by default, together with a default value for the keys that are not stored. It is built from the default value (by default `V()`), from a `std::map` and a default value, or from another `SparseMap` with the same key type. One remark is in order here: the non-constant `operator[]`, which returns a reference, inserts an absent key with the default value, while the constant one does not. `setAll(v)` changes the default value and erases the stored keys, and the class can be copied but not assigned (its copy assignment is private).

Lemma 6.2. *The reads and the writes of `NullMap`, `ConstMap`, `TrueMap`, `FalseMap`, `IdentityMap` and `RangeMap` cost $O(1)$; those of `SparseMap` cost $O(\log s)$ comparisons by `Comp`, where s is the number of stored keys, and `setAll()` of `SparseMap` costs $O(s)$.*

Proof. The first five maps return a stored value, a constant or the key, and `RangeMap` indexes a vector. `SparseMap` finds the key by `lower_bound()` or `find()` of the balanced search tree of `std::map`, and inserts it, if absent, with the position found as a hint; `setAll()` calls `clear()` of the tree. □

Each of these classes has a creator function with the name of the class in lower case: `nullMap< K, V >()`, `constMap< K >( v )` (and `constMap< K, V, v >()` for the inlined value), `identityMap< T >()`, `rangeMap( size, value )` and `rangeMap( vector )`, `sparseMap< K >( value )` (and `sparseMap( map, value )` from a `std::map`), `trueMap< K >()` and `falseMap< K >()`.

`LoggerBoolMap< IT, KEY >` is a write-only `bool` map, which copies each key set to `true` to the output iterator `IT` (named by the type `Iterator`), in the order of the writes, and it ignores the keys set to `false`. Its `begin()` and `end()` return the first iterator and the one after the last key written. The map is meant for the algorithms that report a set by writing `true` once per item. For instance, `dfs( g ).processedMap( loggerBoolMap( std::back_inserter( v ) ) ).run( s )` stores the nodes in the order in which the search processes them. The key type `KEY` is deduced from the iterator (from its `value_type`, or from that of its container for an insert iterator, by the traits class `IteratorTraits` of the namespace `_maps_bits`), and the iterator must either be an inserter or point to enough room. Being only writable, a `LoggerBoolMap` cannot serve where a readable map is required, e.g., as the reached map of `Bfs`, `Dfs` or `Dijkstra`.

6.4 Map adaptors and arithmetic

A map adaptor is a map whose values are computed, at each read, from one or two other maps. It stores references to the maps it combines (or copies of the constants and functors it uses) and nothing else.

Lemma 6.3. *Building a map adaptor costs $O(1)$, and a read (or, for the writable ones, a write) of it costs one read (or write) of each map it adapts, plus one evaluation of its operator or functor.*

Proof. Each class of Table 6.1 stores references (or a copy of a functor or of a constant), and its `operator[]` and `set()` are one expression in the values of the adapted maps at the key, as the table gives them. □

Two remarks apply to all the adaptors: first, they keep the maps by reference, and the maps must therefore outlive them. For instance, a temporary map passed to a creator function (e.g., `addMap( m, constMap< Node > ( 1 ) )`) is destroyed at the end of the full expression, and an adaptor that refers to it cannot be stored. Second, most adaptors are read-only, since a write to a computed value has, in general, no meaning; the exceptions are said explicitly. Table 6.1 lists them, with `m`, `m1` and `m2` the adapted maps, `x` a key, `v` a constant and `f` a functor.

Table 6.1. The map adaptors of `lemon/maps.h`: the value at a key and, when the adaptor is writable, the effect of `set(x, y)`.

Class	Value at `x`	Writes	Key and value types
`ComposeMap< M1, M2 >`	`m1[m2[x]]`	no	key of `M2`, value of `M1`
`CombineMap< M1, M2, F, V >`	`f(m1[x], m2[x])`	no	key of `M1`, value `V`
`FunctorToMap< F, K, V >`	`f(x)`	no	`K`, `V`
`MapToFunctor< M >`	`m[x]`, also as `operator()`	no	those of `M`
`ConvertMap< M, V >`	`m[x]` converted to `V`	no	key of `M`, `V`
`ForkMap< M1, M2 >`	`m1[x]`	to `m1` and `m2`	those of `M1`
`AddMap`, `SubMap`, `MulMap`, `DivMap< M1, M2 >`	`m1[x]` ∘ `m2[x]`	no	those of `M1`
`ShiftMap< M, C >`	`m[x] + v`	no	those of `M`
`ShiftWriteMap< M, C >`	`m[x] + v`	`m.set(x, y - v)`	those of `M`
`ScaleMap< M, C >`	`v * m[x]`	no	those of `M`
`ScaleWriteMap< M, C >`	`v * m[x]`	`m.set(x, y / v)`	those of `M`
`NegMap< M >`	`-m[x]`	no	those of `M`
`NegWriteMap< M >`	`-m[x]`	`m.set(x, -y)`	those of `M`
`AbsMap< M >`	`m[x]` if `m[x] >= 0`, else `-m[x]`	no	those of `M`
`AndMap`, `OrMap< M1, M2 >`	`m1[x] && m2[x]`, `m1[x] || m2[x]`	no	key of `M1`, `bool`
`NotMap< M >`	`!m[x]`	no	key of `M`, `bool`
`NotWriteMap< M >`	`!m[x]`	`m.set(x, !y)`	key of `M`, `bool`
`EqualMap`, `LessMap< M1, M2 >`	`m1[x] == m2[x]`, `m1[x] < m2[x]`	no	key of `M1`, `bool`

In the four arithmetic adaptors ∘ is `+`, `-`, `*` and `/`, and the key and value types of `M2` must convert to those of `M1`. The constant type `C` of the shift and scale adaptors defaults to the value type of `M`. `ScaleWriteMap` can be written only when the constant is not zero and `/` is defined between the value and `C`. `ComposeMap` returns its value with the `ConstReturnValue` of `M1` (by reference, when `M1` is a reference map), while the others return by value. Each class has a creator function with the name of the class in lower case, e.g., `composeMap( m1, m2 )`, `addMap( m1, m2 )`, `shiftMap( m, v )` and `notWriteMap( m )`. The creators of the writable adaptors take the adapted maps by non-constant reference, and `convertMap< V >( m )` takes the value type as its explicit template argument.

`FunctorToMap` takes its key and value types from the members `argument_type` and `result_type` of the functor when they are not given. Since lambdas (and most modern functors) lack these members, `K` and `V` must then be given explicitly, as in `functorToMap< int, double >( f )`; for a plain function pointer `V (*)(K)`, an overload of `functorToMap()` deduces them. Conversely, `MapToFunctor` defines `argument_type` and `result_type`, for the code that still uses them. Likewise, `combineMap( m1, m2, f )` deduces the value type from the functor, and an overload takes a function pointer `V (*)(K1, K2)`.

One passes the adaptors to the algorithms like any other map, but when the algorithm is a class, its map type must be named, which the creator functions do not do; in C++11 and later, `decltype` names it. The program `manual/examples/06-map-adaptors.cc` (the test `manual_06_map_adaptors`) runs `Dijkstra` on the travel time of each arc as the quotient of a length map and a speed map, through a `DivMap` that is never stored, and on that time plus a delay through a `CombineMap` of a lambda. It checks the two distances and the results of five utilities of section 6.7.

> **abLEMON.** The value type of `CombineMap` defaults to the type that the functor returns on the values of the two maps, deduced by `decltype` (the helper `CombineValue` of the namespace `_maps_bits`). LEMON 1.3.1 read the member `result_type` of the functor instead, which C++20 removed from the function objects of `<functional>` and which lambdas never had; thus, `combineMap( m1, m2, f )` now accepts a lambda.

6.5 Maps of the items of a graph

The maps of this section are tied to a graph, whose type `GR` is their first template parameter, and most of them take the item type `K` (`GR::Node`, `GR::Arc` or `GR::Edge`) as the second. They are of two kinds: views compute their value from the graph (and possibly from a map) at each read, while look-up maps store a value per item together with an inverse structure that finds the items by their value. All of them name the graph by the type `Graph`, and the classes with an item type also name it by `Item`.

Views. `IdMap< GR, K >` reads the identifier `id()` of an item, which is unique among the items of its kind, and it does not change while the item exists (even if other items are erased). Its `operator()(id)`, and its nested `InverseMap` (returned by `inverse()`, or built from the graph or from the `IdMap`), give the item with a given identifier, through `fromId()` of the graph. `SourceMap< GR >` and `TargetMap< GR >` read the source and the target of an arc, while `ForwardMap< GR >` and `BackwardMap< GR >` read the arcs `direct(e, true)` and `direct(e, false)` of an edge of an undirected graph. `PotentialDifferenceMap< GR, POT >` reads, at an arc (i, j), the difference $\pi_j - \pi_i$ of a node map of potentials π; for a zero cost, it is the reduced cost of the arc up to the sign convention (cf. chapter 13). Each view reads in $O(1)$ calls of the graph and of the potential map, and each has a creator function, i.e., `idMap< K >( g )`, `sourceMap( g )`, `targetMap( g )`, `forwardMap( g )`, `backwardMap( g )` and `potentialDifferenceMap( g, pot )`.

Degree maps. `InDegMap< GR >` and `OutDegMap< GR >` read the in-degree and the out-degree of a node of a digraph. They keep the degrees in a private node map, `AutoNodeMap`, which gives the degree zero to a new node, and they observe the arcs of the digraph.

Lemma 6.4. *An `InDegMap` (or `OutDegMap`) is built in $O(n + m)$ time, it reads a degree in $O(1)$, and it costs $O(1)$ per arc added or erased through the notifier of the digraph; its values are the in-degrees (out-degrees) as long as the digraph is changed only by additions and erasures of nodes and arcs, and by `clear()` and `build()`.*

Proof. The constructor attaches the map to the notifier of the arcs and counts the incoming arcs of each node by `countInArcs()`, i.e., it visits each arc once. Thereafter, the functions `add()` and `erase()` of the observer increase or decrease the degree of the target of each arc added or erased, and a node added gets degree zero from `AutoNodeMap`; the erasure of a node is preceded by the erasure of its arcs, which the notifier reports. `build()` recounts and `clear()` sets all degrees to zero, while a change of an arc that is not reported as an erasure and an addition breaks the invariant. □

As the header warns, the functions that change an arc without adding or erasing it (e.g., `changeSource()`, `changeTarget()` and `reverseArc()` of `ListDigraph`) leave the degrees wrong.

Look-up maps. `RangeIdMap< GR, K >` gives each item of its kind an identifier in $[0, n_K)$, called its range identifier, and it keeps the inverse in a `std::vector`. Hence, `operator()(id)` (and `InverseMap::operator[]`, of the constant map returned by `inverse()`) return the item with a given range identifier, and `size()` (also a member of the `InverseMap`) returns n_K as an `unsigned int`. The identifiers follow the order of the items at construction, and a new item takes the next one. When an item is erased, the item with the largest identifier takes its identifier; the range thus stays contiguous, but the identifiers may change (and they are usually not those of `IdM`

ap). Furthermore, `swap(p, q)` exchanges the identifiers of two items, by which one can impose any order; the creator function is `rangeIdMap< K >( g )`.

Lemma 6.5. *At any time the range identifiers of the items of a* ***RangeIdMap*** *are a bijection onto* $[0, n_K)$*, whose inverse is the vector of the map. A read in either direction, an addition, an erasure and* ***swap()*** *cost* $O(1)$*, and the construction* $O(n_K)$*.*

Proof. The invariant holds after the construction, which numbers the items in the order of the notifier and pushes them on the vector. An addition gives the new item the identifier n_K and pushes it, so that the range grows by one. An erasure of the item of identifier r moves the last item of the vector, of identifier $n_K - 1$, to position r, gives it the identifier r and pops the vector, so that the range shrinks by one and both directions stay consistent. Finally, `swap()` exchanges two identifiers and the two entries of the vector. Each of these steps is a constant number of map and vector operations. □

`CrossRefMap< GR, K, V >` stores a value for each item, and it keeps a `std::multimap` from the values to the items. Its `operator()(val)`, or the `InverseMap`, returns one item with the value (or `INVALID`), `count(val)` returns the number of items with a value, and `beginValue()` and `endValue()` delimit a forward iterator, `ValueIt` (also named `ValueIterator`), on the values, with multiplicity and in increasing order. We remark that the inverse structure holds only the values written by `set()`: on a newly built map, which reads `V()` at each item, `count( V() )` is zero. The map is meant for values that are all different, or nearly so; when many items share a value, `IterableValueMap` (section 6.6) is usually the better choice. Neither `CrossRefMap` nor `IterableValueMap` is a reference map, i.e., their values are written only by `set()`.

Lemma 6.6. *Let* c *be the number of items whose value equals the old value of the key. In a* ***CrossRefMap****,* ***set(key, val)*** *costs* $O(\log n_K + c)$ *time,* ***operator()(val)*** *costs* $O(\log n_K)$*,* ***count(val)*** *costs* $O(\log n_K + c')$ *with* c' *the result, and the erasure of an item from the graph costs* $O(c \log n_K)$*, with* c *the number of items of its value.*

Proof. The multimap holds at most n_K pairs, and `set()` computes the range of the old value by one `equal_range()`, scans its at most c pairs for the key, erases the pair found, and inserts the new one, each in $O(\log n_K)$; the other searches are one `find()` and one `count()`. The function `erase()` called by the notifier scans the range of the value of the item as `set()` does, but it recomputes the end of the range by `equal_range()` at each step of the scan. □

abLEMON. `CrossRefMap::set()` computes the range of the old value of the key once, where LEMON 1.3.1 recomputed it at each step of the search of the key among the items of that value, for $O(c \log n_K)$; the iterators `ValueIt` of `CrossRefMap` and `IterableValueMap` define their member types, instead of deriving from `std::iterator`, which C++17 deprecated.

6.6 Iterable maps

An iterable map stores a value for each item of a graph and, besides the reads and writes of a map, it lists the items that have a given value, in time proportional to their number. `IterableBoolMap` and `IterableIntMap` are reference maps, whose `operator[]` returns a proxy object (the nested class `Reference`) that converts to the value and writes through `set()`; `IterableValueMap` is a read-write map, and the three classes are kept up to date when items are added to or erased from the graph.

`IterableBoolMap< GR, K >` keeps the items in one vector, together with the position of each item, and the items mapped to `true` are in the positions before a separator, and those mapped to `false` after it (Figure 6.1). `set(key, value)` moves the item across the separator by swapping it with the item next to the separator on the other side, `setAll(value)` moves only the separator, and `trueNum()` and `falseNum()` count the two sides. The iterators `TrueIt`, `FalseIt` and `ItemIt(map, value)` list the items of one side, from the separator outwards, and so do the ranges `trueKeys()`, `falseKeys()` and `items(value)` of the range-based `for` loops. The constructor `IterableBoolMap( const GR & g, bool def = false )` maps all the items to `def`, and an item added later is mapped to `false`; the proxy also offers `&=`, `|=` and `^=`.

Lemma 6.7. *In an* ***IterableBoolMap*** *with vector* $a_0, \ldots, a_{n_K-1}$ *and separator* σ*, an item is mapped to* ***true*** *if and only if its position is below* σ*, and the position stored for each item is its index in the vector.* ***set(), setAll(), trueNum(), falseNum()****, the reads, the additions and the erasures of items cost* $O(1)$*, and an iterator lists the items of one value in* $O(1)$ *per item.*

Proof. The constructor pushes the items in the order of the notifier and puts σ at n_K or at 0. To set an item at position $p \geq \sigma$ to `true`, `set()` exchanges it with a_σ, updates both positions and increases σ; to set an item at $p < \sigma$ to `false`, it decreases σ and exchanges the item with a_σ. In both cases only the two items exchanged

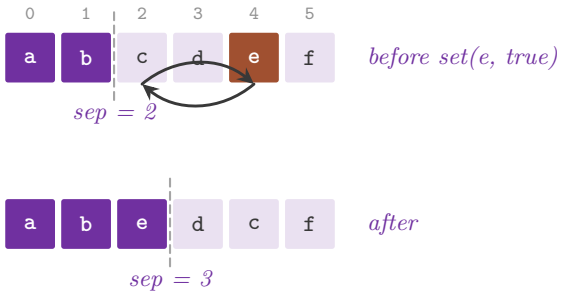


Figure 6.1. IterableBoolMap: the items in one vector, the true ones before the separator; set(e, true) swaps e with the first false item and moves the separator.

change position, and only the moved item changes side. A new item is pushed at the end, i.e., on the `false` side; an erased item at $p < \sigma$ is replaced by $a_{\sigma-1}$, which is replaced by the last item, and σ is decreased, while one at $p \geq \sigma$ is replaced by the last item. `trueNum()` is σ, `falseNum()` is $n_K - \sigma$, and an iterator steps from a position to the previous one until it crosses σ (or 0). □

`IterableIntMap< GR, K >` stores an integer per item, and it keeps a doubly linked list of the items for each non-negative value, with the heads in a vector indexed by the value. Thus, `set(key, value)` unlinks the item from the list of its old value and links it at the head of the list of the new one. `size()` returns the largest value with a non-empty list plus one, and `ItemIt(map, value)` and `items(value)` list the items with a value, while an item with a negative value is in no list and cannot be listed. The constructor without a value sets all the values to −1, the one with a value sets them to it, and an item added later reads −1. The proxy returned by `operator[]` offers the integer operators (`++`, `--`, `+=`, `-=`, `*=`, `/=`, `%=`, `&=`, `|=`, `^=`, `<<=`, `>>=`), each of which goes through `set()`.

`IterableValueMap< GR, K, V >` does the same for any value type with an `operator<`, and it keeps a linked list per value, with the heads in a `std::map< V, K >` that holds only the values in use. Its `ItemIt(map, value)` and `items(value)` list the items with a value, and `beginValue()` and `endValue()` delimit a forward iterator, `ValueIt`, on the distinct values, in increasing order. The constructor maps all the items to a value (by default `V()`), and an item added later reads `V()`. The records of the lists are the structures `IterableIntMapNode` and `IterableValueMapNode` of the namespace `_maps_bits`, which are not meant to be used directly.

Lemma 6.8. *Let w be the largest value of an `IterableIntMap`, and d the number of distinct values of an `IterableValueMap`. A `set()` of an `IterableIntMap` costs $O(1)$ plus the change of length of the vector of heads, which is a growth to the new value plus one, or a shrink by the number of empty lists at its end; the memory is $O(N_K + w)$. A `set()` and the construction of an `ItemIt` of an `IterableValueMap` cost $O(\log d)$ comparisons. In both maps an `ItemIt` lists the items of a value in $O(1)$ per item.*

Proof. `set()` of an `IterableIntMap` unlinks the item (one update of its two neighbours or of the head), pops the empty heads at the end of the vector, sets the value and links the item at the head of its new list, after resizing the vector if the value exceeds it. The steps other than the pops and the resize are $O(1)$. In an `IterableValueMap` the head of a value is found by `find()` on `std::map`, and a value whose list becomes empty is erased from it, in $O(\log d)$; an `ItemIt` follows the links `next` of the records. □

abLEMON. The default constructor of `IterableIntMapNode`, by which a graph map creates the record of a new item, and the constructor of `IterableValueMapNode` initialize the links to `INVALID`, which LEMON 1.3.1 left indeterminate.

The iterable integer map is the natural data structure for the buckets of degrees of the smallest-last ordering of Matula and Beck 1983. Let $G = (V, E)$ be a simple undirected graph (no loops, no parallel edges), and, for H a subgraph, let $\delta(H)$ be its least degree. The k-core of G is the union of the subgraphs H with $\delta(H) \geq k$ (itself such a subgraph, since a node of the union has at least k neighbours in the subgraph that contains it). The core number $c(v)$ of a node is the largest k such that v is in the k-core, and the degeneracy of G is $\delta^*(G) = \max_{H \subseteq G} \delta(H)$. Algorithm 6.1 removes, one at a time, a node of least degree in the graph that remains, and records the running maximum of the degrees at removal.

Algorithm 6.1 (Smallest-last ordering and core numbers by an iterable integer map).

```
input:  simple graph G = (V, E)
deg := IterableIntMap on V, deg[v] := degree of v      (deg.set(v, .))
```

```
k := 0;  d := 0
repeat |V| times
  while ItemIt(deg, d) == INVALID: d := d + 1
  u := ItemIt(deg, d)                  (a node of least current degree)
  k := max(k, d);  core[u] := k;  append u to the order
  deg.set(u, -1)                       (u leaves all the lists)
  for each edge (u, w) with deg[w] > 0: deg.set(w, deg[w] - 1)
  d := max(d - 1, 0)
output: the order (reversed, the smallest-last order), core, k
```

Theorem 6.1. *Algorithm 6.1 removes at each iteration a node of least degree in the graph that remains, and on output* $\mathrm{core}[v] = c(v)$ *for each node and* $k = \delta^*(G)$*; also, each node has at most* k *neighbours that are removed after it.*

Proof. Let $u_1, \dots, u_n$ be the nodes in the order of removal, G_t the subgraph induced by $u_t, \dots, u_n$, and d_t the value of d when u_t is chosen. We first show that $d \le \delta(G_t)$ at the start of iteration t, which holds for $t = 1$, since $d = 0$. If it holds at t, the buckets below $\delta(G_t)$ are empty and that of $\delta(G_t)$ is not, so that the loop stops at $d_t = \delta(G_t)$ and u_t has least degree in G_t. The neighbours of u_t that remain (exactly those with `deg` positive, since a remaining neighbour has degree at least 1 and a removed node has -1) lose one unit of degree each, as G is simple, so that $\delta(G_{t+1}) \ge d_t - 1$, and the new d is at most $\delta(G_{t+1})$. Hence, $\mathrm{core}[u_t] = k_t = \max_{s \le t} \delta(G_s)$. Let $r \le t$ attain the maximum; G_r contains u_t and has least degree k_t, so that $c(u_t) \ge k_t$. Conversely, let H be the $c(u_t)$-core and s the first index with $u_s \in H$; then $s \le t$, $H \subseteq G_s$, and $\delta(G_s) = \deg_{G_s}(u_s) \ge \deg_H(u_s) \ge c(u_t)$, so that $k_t \ge c(u_t)$. Since every subgraph H has $\delta(H) \le c(v)$ for its nodes v, $\delta^*(G) = \max_v c(v) = \max_t k_t = k$; finally, the neighbours of u_t removed after it are its neighbours in G_t, i.e., $\delta(G_t) \le k$ of them. □

Theorem 6.2. *Algorithm 6.1 runs in* $O(|V| + |E|)$ *time.*

Proof. The initial degrees cost $O(|V| + |E|)$, and there are $|V|$ initial `set()`, $|V|$ removals and at most $|E|$ decrements (an edge decrements one end, when its other end is removed). The vector of heads grows only during the initialization, to the largest degree $\Delta + 1$, and the shrinks after it remove at most $\Delta + 1$ heads in all; by Lemma 6.8, all the `set()` cost $O(|V| + |E| + \Delta)$. The value d decreases by at most one per iteration and never exceeds Δ, so that the tests of the inner loop are at most $|V| + \Delta$ plus one per iteration; since $\Delta \le |E|$, the total is $O(|V| + |E|)$. □

On a graph with parallel edges, a removal may lower a degree by more than one, and the last step of Algorithm 6.1 would have to lower d accordingly. The program `manual/examples/06-degeneracy.cc` (the test `manual_06_degeneracy`) implements Algorithm 6.1 on a graph of seven nodes, checks the degeneracy, the core numbers and the first node removed, and marks the nodes of the largest core in an `IterableBoolMap`, whose `trueNum()` gives their number and whose `TrueIt` lists them.

6.7 Map utilities

The functions at the end of `lemon/maps.h` scan the items of one kind of a graph, the kind given by the key type of the map, and use the values of a map.

- `mapCopy(g, from, to)` writes `from[x]` into `to` for each item `x` of the kind of `to`. An edge map can thus be copied to an arc map of the same graph, since an arc converts to its edge, but not vice versa. `mapFill(g, map, val)` writes `val` at each item.
- `mapCompare(g, m1, m2)` returns `true` if the two maps agree, by `operator==`, at each item of the kind of `m2`.
- `mapMin(g, map)` and `mapMax(g, map)` return an item of least and of largest value, or `INVALID` on an empty item set. When there are ties (which may well happen on integer data), they return the first one met in the order of the item iterator. The variants with a third argument take a comparison functor in place of `std::less< V >`. `mapMinValue()` and `mapMaxValue()` (with or without the functor) return the value, and they require a non-empty item set, since they read the map at the item that `mapMin()` or `mapMax()` returns.
- `mapFind(g, map, val)` and `mapFindIf(g, map, pred)` return the first item whose value equals `val` (or satisfies `pred`), and `INVALID` if there is none, while `mapCount(g, map, val)` and `mapCountIf(g, map, pred)` return the number of such items.

Lemma 6.9. *Each utility makes one pass over the items of its kind, with* $O(n_K)$ *reads or writes of the maps and* $O(n_K)$ *evaluations of* `==`*, of the comparison or of the predicate (the searches stop at the first item found).*

Proof. Each function is one loop of the item iterator of the kind, with one read (or write) of each map and one evaluation per item; `mapMin()` keeps the current minimum and replaces it only on a strict comparison, which gives the first item of least value. □

The program `manual/examples/06-map-adaptors.cc` uses `mapMin()`, `mapCountIf()`, `mapCopy()`, `mapCompare()` and `mapMaxValue()`, and the test `test/maps_test.cc` checks all the utilities.

6.8 Exercises

Exercise 6.1. Write a map class, derived from `MapBase`, whose value at an arc is the Euclidean length of the arc, computed from a node map of coordinates. Check that it conforms to `concepts::ReadMap` by `checkConcept()`, and then obtain the same map without writing a class, from `SourceMap`, `TargetMap`, `ComposeMap` and `CombineMap`.

Exercise 6.2. In `manual/examples/06-map-adaptors.cc`, replace the `DivMap` by a `CombineMap` with a lambda, and the delay by a `ShiftMap` of the `DivMap`; check that the distances do not change.

Exercise 6.3. Explain why the declaration `AddMap< M, ConstMap< K, int > > m = addMap( length, constMap< K >( 1 ) );` compiles, while a later read of `m` has undefined behavior, and correct it.

Exercise 6.4. Modify `manual/examples/06-degeneracy.cc` so that it colors the graph greedily in the smallest-last order, with at most $k + 1$ colors (by the last statement of Theorem 6.1), and check the coloring. Then extend Algorithm 6.1 to graphs with parallel edges, and prove that it still runs in $O(|V| + |E|)$ time.

Exercise 6.5. Use a `RangeIdMap` and its `swap()` to number the nodes of an acyclic digraph in a topological order, and check that each arc goes from a smaller to a larger range identifier (the test `test/maps_test.cc` checks `swap()` after an erasure).

7. Heaps

We are concerned in this chapter with the heaps of the library, i.e., with the implementations of the abstract data type priority queue, which the algorithms of Part III use to choose the next node to scan. A heap stores a set of items, each with a priority. It gives the item of least priority, removes it, and changes the priority of an item in the heap. The cost of these operations differs (sometimes rather markedly) from one structure to another; hence, the best choice depends on the mix of operations of the algorithm and on the type of the priorities. Section 7.1 gives the interface common to all the heaps (the concept `concepts::Heap`). Sections 7.2 to 7.4 describe the three families of implementations, i.e., the implicit d-ary heaps, the linked heaps and the heaps of integer priorities, each with its algorithms, its invariants and its costs. Section 7.5 compares them and shows how an algorithm is given a heap, and section 7.6 collects the exercises. Throughout the chapter, n is the number of items in the heap, "$p < q$" between priorities means `comp(p, q)` for the comparison functor of the heap, and an amortized cost is a bound on the total cost of any sequence of operations from the empty heap, divided among its operations.

7.1 The heap concept

The concept `concepts::Heap< PR, IM, CMP >` of `lemon/concepts/heap.h` describes the interface. `PR` is the type of the priorities (named `Prio`), `IM` a reference map from the items to `int`, called the cross-reference map (named `ItemIntMap`), and `CMP` a functor that compares two priorities, by default `std::less< PR >`. The type of the items, `Item`, is `IM::Key`; thus, the items are whatever the map takes as keys (nodes, arcs, or integers with a `RangeMap`). The heap keeps a reference to the map, which must therefore outlive it, and it uses the map to find an item in its storage. The states of an item are the values of the enum `State`: `PRE_HEAP` (-1), if the item has never been in the heap, `IN_HEAP` (0), if it is in the heap, and `POST_HEAP` (-2), if it has been in the heap and has left it. The map must hold `PRE_HEAP` for each item before the first insertion, which is the precondition of the constructors `explicit Heap( IM & map )` and `Heap( IM & map, const CMP & comp )`. The concept also requires the heap to be copy-constructible.

The operations are the following; Table 7.1 gives their costs.

- `size()` returns the number of items, and `empty()` returns whether it is zero.
- `push(i, p)` inserts the item `i`, which must not be in the heap, with priority `p`.
- `top()` and `prio()` return the item of least priority and its priority (the heap must not be empty). With `std::greater` as `CMP`, they return the item of largest priority.
- `pop()` removes the item of least priority (the heap must not be empty), whose state becomes `POST_HEAP`, and `erase(i)` removes the item `i`, which must be in the heap.
- `operator[](i)` returns the priority of the item `i`, which must be in the heap.
- `decrease(i, p)` and `increase(i, p)` change the priority of an item in the heap to `p`, which must not be larger (respectively, smaller) than the current one under `CMP`. The heaps do not check it.
- `set(i, p)` changes the priority of `i` if it is in the heap, by `decrease()` or `increase()` as the case may be, and it inserts `i` otherwise (also when it is `POST_HEAP`).
- `state(i)` returns the state of an item. `state(i, st)`, with `st` equal to `PRE_HEAP` or `POST_HEAP`, erases the item if it is in the heap and then writes `st` into the map (`IN_HEAP` is ignored).
- `clear()` empties the heap without touching the map. To reuse the heap on the same items, the caller must also reset the map to `PRE_HEAP`.

What the map holds for an item depends on the family. In the implicit heaps of section 7.2 and in the heaps of section 7.4 it holds the position of the item in the storage while the item is in the heap, and `POST_HEAP` after the item has left it. In the linked heaps of section 7.3 it holds the index of the record of the item from its first insertion until `clear()` or `state(i, st)`, and `state(i)` reads, in the record, whether the item is in the heap.

In both cases `state(i)` costs $O(1)$, and it lets an algorithm such as `Dijkstra` distinguish a scanned node from an unreached one without a further map. Furthermore, the heaps write the map through `operator[]` (e.g., `state(i, st)` in all of them, `decrease()` and `erase()` of `BinomialHeap`, and `push()`, `pop()` and `erase()` of `RadixHeap` and `BucketHeap`), and therefore need a reference map, as their documentation says; the maps used in practice (a graph map of `int` or a `RangeMap< int >`) are reference maps.

Besides the concept, `BinHeap`, `QuadHeap`, `DHeap`, `BucketHeap` and `SimpleBucketHeap` have `push(pair)`, which takes a `std::pair< Item, Prio >` (the type `Pair`, which `FibHeap` also names), and the comparison heaps name their functor by `Compare`. Also, `BinHeap`, `QuadHeap` and `DHeap` have `replace(i, j)`, which gives the place and the priority of the item `i` (in the heap) to the item `j` (not in the heap), in $O(1)$; after it, the map holds for `i` the state that it held for `j`. The test `test/heap_test.cc` checks each heap against the concept, by `checkConcept()`, and then by sorting, by increasing priorities, by priorities with many ties, by erasures under `std::less` and `std::greater`, and inside `Dijkstra`.

7.2 Implicit d-ary heaps

A d-ary heap (Johnson 1975) is a complete d-ary tree, stored level by level in a vector, whose items satisfy the heap order: the priority of an item is not smaller than that of its parent. With positions from 0, the parent of position $i > 0$ is $\lfloor (i-1)/d \rfloor$, and its children are $di+1, \ldots, di+d$ (those below n). The binary heap of Williams 1964 is the case $d = 2$ (Figure 7.1). The cross-reference map holds the position of each item, and each move of an item updates it.

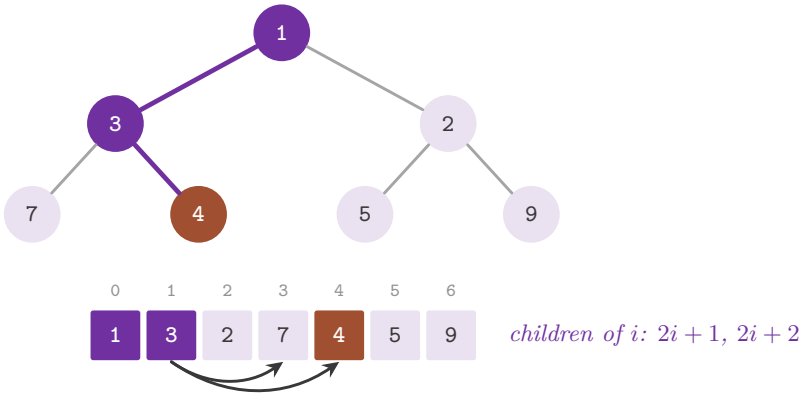


Figure 7.1. A binary heap as a tree and as the vector that stores it; a decrease of the item in position 4 moves it along the path to the root, here through the positions 1 and 0.

All the operations reduce to the two procedures of Algorithm 7.1, which move an item along a path of the tree until the heap order holds again.

Algorithm 7.1 (Sifting in a d-ary heap).

```
bubbleUp(hole, (i, p)):           (the item i with priority p goes up)
  while hole > 0 and p < prio(parent(hole)):
    move the item of parent(hole) to hole;  hole := parent(hole)
  put (i, p) at hole
bubbleDown(hole, (i, p), n):      (the item i with priority p goes down)
  while hole has children among the positions 0 .. n-1:
    c := the child of least priority
    if not prio(c) < p: break
    move the item of c to hole;  hole := c
  put (i, p) at hole
push(i, p):    append a position n; bubbleUp(n, (i, p))
pop():         mark top POST_HEAP; bubbleDown(0, last item, n - 1)
decrease(i,p): bubbleUp(position of i, (i, p))
increase(i,p): bubbleDown(position of i, (i, p), n)
set(i, p):     push if i is not in the heap, else decrease if p < prio(i),
               else increase
```

```
erase(i):       mark i POST_HEAP; put the last item in the hole of i by
                bubbleUp and, if it did not move, by bubbleDown
```

Each `move` and each `put` writes the new position into the cross-reference map. `BinHeap` implements `erase()` as written; `QuadHeap` and `DHeap` compare the last item with the parent of the hole first, and call `bubbleDown()` if the parent is smaller and `bubbleUp()` otherwise, which has the same effect.

Lemma 7.1. *A d-ary heap of $n \geq 1$ items has $L = \lceil \log_d(n(d-1)+1) \rceil$ levels, i.e., a path from the root to a leaf has at most L positions.*

Proof. The levels $0, \ldots, L-2$ are full, and level $L-1$ holds at least one item; since level h holds d^h positions, $\sum_{h<L-1} d^h < n \leq \sum_{h<L} d^h$, i.e., $d^{L-1} < n(d-1)+1 \leq d^L$, which is the statement. □

Theorem 7.1. *Each operation of Algorithm 7.1 keeps the heap order and keeps in the cross-reference map the position of each item in the heap; hence, `top()` returns an item of least priority. `push()` and `decrease()` cost $O(\log_d n)$ comparisons, `pop()`, `increase()` and `erase()` cost $O(d \log_d n)$, and `top()`, `prio()`, `operator[]`, `state()` and `replace()` cost $O(1)$.*

Proof. Call a tree with a hole at position h ordered if the heap order holds between all the pairs of positions other than h and, moreover, between the parent of h and the children of h. `bubbleUp()` is called on an ordered hole whose children are not smaller than p (for `push()`, the hole has no child; for `decrease()`, the children were not smaller than the old priority, which is not smaller than p); each step moves the parent into the hole, which keeps the tree ordered with the hole one level up and keeps the children of the new hole not smaller than p; it stops when p is not smaller than the parent, and then putting p restores the heap order. Symmetrically, `bubbleDown()` is called on an ordered hole whose parent is not larger than p, moves the least child up while it is smaller than p, and stops when no child is smaller. For `erase()`, the last item is put into the hole of i: `bubbleUp()` succeeds when it is smaller than the parent of the hole, and otherwise the parent is not larger than it and `bubbleDown()` applies. Every move writes the new position into the map. By the heap order, the root has least priority. As for the costs, `bubbleUp()` makes one comparison per level, and `bubbleDown()` makes d per level (at most $d-1$ to find the least child and one with p); by Lemma 7.1 there are $O(\log_d n)$ levels. `top()` and `prio()` read position 0, `operator[]` and `state()` read the map and the vector, and `replace()` writes two values of the map and one item of the vector. □

The degree d thus trades the cost of `push()` and `decrease()` for that of `pop()`. In an algorithm with many decreases per removal, as `Dijkstra` on a dense digraph, the choice of d balances the two, as follows.

Theorem 7.2. *(Johnson.) `Dijkstra` on a digraph of n nodes and m arcs with a d-ary heap makes $O(nd \log_d n + m \log_d n)$ comparisons in the heap; with $d = \max(2, \lceil m/n \rceil)$, this is $O(m \log_{2+m/n} n)$.*

Proof. `Dijkstra` pushes and pops each node at most once and decreases a priority at most once per arc, which by Theorem 7.1 gives the first bound. With $d = \max(2, \lceil m/n \rceil)$ one has $nd = O(n+m)$ and $\log_d n = O(\log n / \log(2+m/n))$, and $n \leq m$ may be assumed for the nodes that are reached from the source. □

Of course, a larger degree also means fewer levels and fewer cache misses, which in practice often (but not always) favours $d = 4$ over $d = 2$, even when the counts of comparisons do not.

The library has three classes of this family, all with the template parameters `PR`, `IM` and `CMP` (by default `std::less< PR >`) and with both constructors of the concept.

- `BinHeap< PR, IM, CMP >`, in `lemon/bin_heap.h`, is the binary heap. It is the default heap of `Dijkstra`, `BidirDijkstra`, `AStar`, `Suurballe` and `MinCostArborescence`, and the heap (with `std::greater`) of `NagamochiIbaraki` and of `MaxCardinalitySearch` on general capacities; the matching algorithms also use it.
- `QuadHeap< PR, IM, CMP >`, in `lemon/quad_heap.h`, is the 4-ary heap (the header calls it "fourary"), whose four children are compared by unrolled code.
- `DHeap< PR, IM, D, CMP >`, in `lemon/dheap.h`, is the d-ary heap, with the degree as the template parameter `D`, by default 16. The header suggests a power of two, which lets the compiler turn the divisions and multiplications of the positions into shifts.

abLEMON. `CapacityScaling` (cf. chapter 13) keeps the nodes of its shortest path searches in a `QuadHeap`, where LEMON 1.3.1 used a `BinHeap`.

7.3 Linked heaps

The heaps of this section store the items in heap-ordered trees, linked by indices. Two trees are joined in $O(1)$ by making the root of larger priority a child of the other (a link). Each of the three classes keeps one record per item in a vector, indexed by the cross-reference map. The record stays after the item leaves the heap (a flag records whether it is in), and it is reused when the item comes back; hence, the memory is proportional to the number N of distinct items pushed since the last `clear()`, and $n \le N$. The three classes have the template parameters `PR`, `IM` and `CMP` and both constructors of the concept, and in all three `increase()` is `erase()` followed by `push()`.

Fibonacci heaps. `FibHeap< PR, IM, CMP >`, in `lemon/fib_heap.h`, is the heap of Fredman and Tarjan 1987. The roots of its trees form a circular list, with a pointer to a root of least priority, and the children of a node form a circular list as well; a node may be marked. `push()` adds a root, and `decrease()` cuts the item from its parent when the heap order breaks; the parent, if it was marked, is cut in turn (the cascading cut), and so on upwards, while an unmarked parent that is not a root is marked. `pop()` removes the minimum, moves its children to the list of the roots, and then consolidates the roots, linking two roots of equal degree until all the degrees differ. Furthermore, `erase(i)` cuts the item (with the cascading cut), makes it the minimum (in the words of a comment of the code, "as if its prio would be -infinity"), and pops it.

Algorithm 7.2 (Decrease and pop in a Fibonacci heap).

```
decrease(i, p):
  prio(i) := p;  q := parent(i)
  if q exists and p < prio(q):
    cut(i, q)                        (i becomes a root, unmarked)
    while q is not a root:           (cascading cut, cascade())
      if q is unmarked: mark q; stop
      r := parent(q);  cut(q, r);  q := r
  if p < prio(min): min := i
pop():
  remove min from the roots and add its children to them
  A[0 .. maxdeg - 1] := none,  maxdeg = floor(2.08 ln N) + 1   (balance())
  for each root r:                    (consolidation)
    while A[degree(r)] is a root s:
      link the one of larger priority under the other, unmarked,
        call the result r             (on a tie, s stays the root)
      A[degree(r) - 1] := none
    A[degree(r)] := r
  min := the root of least priority
erase(i):
  if i has a parent q: cut(i, q) and the cascading cut from q
  min := i;  pop()
```

Lemma 7.2. *Let $\phi = (1+\sqrt{5})/2$. In a Fibonacci heap, a node of degree k has at least $F_{k+2} \ge \phi^k$ nodes in its subtree, where F_j is the j-th Fibonacci number; hence, every degree is at most $\log_\phi n$, and the array of `balance()`, of size $\lfloor 2.08 \ln N \rfloor + 1$, is large enough.*

Proof. Let $y_1, \dots, y_k$ be the children of a node x in the order in which they were linked to it. When y_h was linked, x had at least the children $y_1, \dots, y_{h-1}$, and links join roots of equal degree; hence, y_h had degree at least $h-1$, and since then it has lost at most one child, because a node that is not a root is marked at its first loss and cut at its second (marks are cleared by links and cuts). Thus y_h has degree at least $h-2$. Let s_k be the least size of a subtree whose root has degree k; then $s_0 = 1$, $s_1 = 2$, and $s_k \ge 2 + \sum_{h=2}^{k} s_{h-2}$, which by induction gives $s_k \ge F_{k+2}$, since $F_{k+2} = 1 + \sum_{j=0}^{k} F_j$; the bound $F_{k+2} \ge \phi^k$ is the classical one. Hence $k \le \log_\phi n = \ln n / \ln \phi$ with $1/\ln\phi \approx 2.078$, and the degrees met during the consolidation, at most $\log_\phi n \le 2.078 \ln N$, are indices of the array. □

Theorem 7.3. *In a `FibHeap`, `top()`, `prio()`, `operator[]` and `state()` cost $O(1)$, `push()` and `decrease()` cost $O(1)$ amortized time, and `pop()`, `erase()` and `increase()` cost $O(\log N)$ amortized time; every operation keeps the heap order, and `pop()` and `erase()` remove exactly the item they should, whatever the type of the priorities and the comparison.*

Proof. (Sketch.) With the potential $\Phi = t + 2\mu$, where t is the number of roots and μ that of the marked nodes, `push()` raises Φ by one; a `decrease()` that makes c cuts costs $O(c)$, adds c roots and unmarks at least $c-1$

nodes, so that its amortized cost is $O(1)$; a `pop()` costs $O(D+t)$, where $D = O(\log N)$ bounds the degrees and the array (Lemma 7.2), and leaves at most $D+1$ roots, so that its amortized cost is $O(D)$. An `erase()` is a `decrease()` followed by a `pop()`, and `increase()` adds a `push()`. The details are in Fredman and Tarjan 1987, §2 and in Cormen et al. 2001, §20.2 to §20.4. As for the correctness, a cut makes a subtree a root and links put the larger root under the smaller, so that every tree stays heap-ordered; `pop()` ends by scanning all the roots for the least priority. `erase()` never compares the priority of the item: it makes the item a root, and the `pop()` that follows removes that root and finds the new minimum by the scan. □

Accordingly, the header warns that a heap with many increases and erasures is better a binary one.

Pairing heaps. `PairingHeap< PR, IM, CMP >`, in `lemon/pairing_heap.h`, is the self-adjusting heap of Fredman, Sedgewick, Sleator and Tarjan 1986, i.e., a single heap-ordered tree in which each node keeps its children in a list (stored in the records as a binary tree of first children and right siblings). `push()` links the new item with the root, and `decrease()` cuts the subtree of the item, when the heap order with its parent breaks, and links it with the root. `pop()` removes the root and joins its subtrees in two passes: it first links them in pairs from left to right, and then it links the results from right to left into one tree (the two-pass pairing). `erase(i)` cuts the subtree of the item and makes the item the root over the rest, whatever the priorities, and then pops it.

Algorithm 7.3 (Pairing heap).

```
link(a, b):   make b the first child of a                    (fuse(a, b))
push(i, p):   if the heap is empty: root := i
              else if p < prio(root): link(i, root); root := i
              else link(root, i)
decrease(i, p):
  prio(i) := p
  if i is not the root and p < prio(parent(i)):
    cut the subtree of i                                      (cut())
    if prio(root) < p: link(root, i) else link(i, root); root := i
pop():
  T[0 .. k-1] := the subtrees of the children of the root, left to right
  for h = 0, 2, 4, ... while h + 1 < k:          (first pass)
    link the larger of T[h], T[h+1] under the other, into T[h]
  h := the last even index;  for h, h - 2, ..., 2:   (second pass)
    link the larger of T[h], T[h-2] under the other, into T[h-2]
  root := T[0], or none if k = 0
erase(i):
  if i is not the root: cut the subtree of i; link(i, root); root := i
  pop()
```

Theorem 7.4. *Every operation of a* ***PairingHeap*** *keeps a single heap-ordered tree whose root is an item of least priority, and* ***erase(i)*** *removes exactly* i*, whatever the type of the priorities and the comparison.* ***push()****,* ***decrease()****,* ***top()*** *and* ***prio()*** *cost* $O(1)$ *actual time, and* ***pop()*** *costs* $O(1+k)$*, with* k *the number of children of the root. In amortized terms, every operation costs* $O(\log n)$ *(Fredman et al. 1986), and* ***push()*** *and* ***decrease()*** *cost* $O(2^{2\sqrt{\log\log n}})$ *while* ***pop()*** *costs* $O(\log n)$ *(Pettie 2005).*

Proof. A link of two heap-ordered trees, with the root of larger priority under the other, is heap-ordered; a cut subtree is heap-ordered, and so is the rest. Hence, `push()`, `decrease()` and `pop()` keep the order, and the root of an ordered tree has least priority. In `erase(i)`, the tree made by linking the rest under i may violate the order only at its root i; `pop()` removes that root and joins its subtrees, which are heap-ordered, by links that compare only their roots, so that the priority of i is never compared and the result is an ordered tree of the other items. The actual costs follow from the algorithm: one link, one cut, or one pass over the k subtrees. The amortized bounds are the analyses cited, which are proved there for the two-pass pairing heap that cuts at each decrease (the class cuts only when the heap order breaks), and whose proofs take several pages. □

The amortized cost of `decrease()` is thus $o(\log n)$, and it cannot be $O(1)$ for this kind of structure (Fredman 1999). Yet, the pairing heap is often reported to be as fast as a binary heap (or faster) on the decreases of the shortest path algorithms.

Binomial heaps. `BinomialHeap< PR, IM, CMP >`, in `lemon/binomial_heap.h`, is the heap of Vuillemin 1978. A binomial tree of degree 0 is a single node, and one of degree k is the link of two binomial trees of degree $k-1$; the heap is a list of heap-ordered binomial trees of distinct degrees, in increasing order of degree, like the binary digits of n. Its item of least priority is kept, so that `top()` and `prio()` cost $O(1)$.

Algorithm 7.4 (Binomial heap).

```
merge(list L):                                              (merge())
  interleave L with the roots, in increasing order of degree
  scan the roots: when two consecutive roots x, y have the same degree
    and the root after y has a different one, link the larger under
    the smaller (as in a binary addition)
push(i, p):    merge the tree of the single node i;  min := findMin()
pop():         remove min from the roots;
               merge the list of its children, reversed;  min := findMin()
decrease(i, p):
  prio(i) := p
  while i has a parent q and p < prio(q):
    exchange the items (and priorities) of the records of i and q,
    updating the cross-reference map;  i := q
  if p < prio(min): min := i
erase(i):
  move each item of the path from the root of the tree of i to i one
  record down, and put i in the root, updating the map;  min := that root
  pop()
```

Here `findMin()` scans the roots for one of least priority, comparing the priorities by `CMP`.

Lemma 7.3. *A binomial tree of degree k has 2^k nodes and height k, and its root has degree k; hence, a binomial heap of n items has at most $\lfloor \log_2 n \rfloor + 1$ trees, and every path from a root to a leaf has at most $\lfloor \log_2 n \rfloor + 1$ nodes.*

Proof. By induction on k, from the definition: the link of two trees of 2^{k-1} nodes, height $k-1$ and root degree $k-1$ has 2^k nodes, height k and root degree k. The trees of a heap have distinct degrees, so that $n = \sum_k b_k 2^k$ with $b_k \in \{0, 1\}$ is the binary representation of n, which has at most $\lfloor \log_2 n \rfloor + 1$ digits, and no tree has degree above $\log_2 n$. □

Theorem 7.5. *Every operation of a `BinomialHeap` keeps a list of heap-ordered binomial trees of distinct degrees and keeps `min` on a root of least priority. `push()`, `pop()`, `decrease()`, `erase()` and `increase()` cost $O(\log n)$, and `top()`, `prio()`, `operator[]` and `state()` cost $O(1)$. `erase(i)` removes exactly i, whatever the type of the priorities and the comparison.*

Proof. A merge of two lists of binomial trees of distinct degrees links only trees of equal degree, which gives a binomial tree of the next degree, and it ends with distinct degrees, as the carry of a binary addition; each link puts the larger root under the smaller, so the trees stay heap-ordered. `pop()` merges the children of the minimum, which are binomial trees of degrees $0, \ldots, k-1$. `decrease()` moves the decreased item up by exchanges with its parent while it is smaller, which restores the heap order as in `bubbleUp()` (Theorem 7.1). In `erase()`, the items on the path from the root r to i move one record down; each of them is then above records that held its own descendants, whose priorities are not smaller, so that the order can fail only at r, which holds i; `pop()` removes r without comparing its priority, and `findMin()` recomputes the minimum. The costs follow from Lemma 7.3: a merge and `findMin()` scan $O(\log n)$ roots, and `decrease()` and `erase()` climb $O(\log n)$ records. □

abLEMON. `BinomialHeap` compares the priorities of the roots as priorities, where LEMON 1.3.1 compared them as `int` and truncated the fractional ones, taking for the minimum a root that was not; and `push()` looks for the minimum among the roots after the merge, since the merge may make the former minimum the child of a root of equal priority, after which `pop()` of LEMON 1.3.1 looped for ever (trac #646). The test `test/heap_test.cc` gives every comparison heap floating-point priorities with many ties, and the case of the ticket.

abLEMON. `PairingHeap` and `BinomialHeap` erase an item by making it the root (as in Algorithms 7.3 and 7.4) and popping it. LEMON 1.3.1 decreased its priority to `prio() - 1` and popped it, which needs a numeric priority type and removes the minimum in place of the item whenever `prio() - 1` does not compare smaller than `prio()` under `CMP`, i.e., with `std::greater`, or with priorities so large that $x - 1 = x$ (e.g., above 2^{53} in `double`); the test `heap_test` checks both cases for all the heaps.

7.4 Heaps of integer priorities

When the priorities are integers and the sequence of the minima removed does not decrease (as in `Dijkstra` with non-negative integer lengths), an array indexed by the priority may do better than a comparison heap. The

library has, indeed, three such heaps. Their priority type is `int`, they have no comparison functor, and their only constructor takes the map (plus the initial minimum and capacity, for `RadixHeap`).

Radix heaps. `RadixHeap< IM >`, in `lemon/radix_heap.h`, is the one-level radix heap of Ahuja, Mehlhorn, Orlin and Tarjan 1990. The items are kept in boxes $0, \dots, K-1$, each a doubly linked list, and box k covers the priorities in $[b_k, b_{k+1})$, where $b_0 \le b_1 \le \cdots \le b_K$ (Figure 7.2). Box k has a nominal width w_k, i.e., $1, 1, 2, 4, 8, \dots$, and its range is never wider than w_k. Each box is a private record `RadixBox` (the start b_k of its range, its width and the head of its list), and each item a record with its links, its box and its priority.

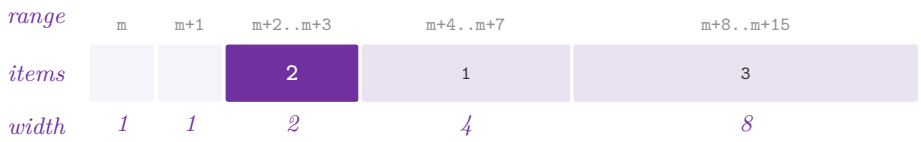


Figure 7.2. The boxes of a radix heap from the minimum m, of widths 1, 1, 2, 4, 8; the first non-empty box (here box 2) is the one that is redistributed when box 0 is empty.

Algorithm 7.5 (Radix heap).

```
push(i, p):
  while p >= b_K: add a box K of width 2 w_{K-1} after the last  (extend())
  put i in the box of p, searching downwards from box K-1;
  if p < b_0, throw PriorityUnderflowError                    (findDown())
decrease(i, p):  prio(i) := p; if p < b_(box of i): move i to the box of
                 p, searching downwards, or throw as in push()
increase(i, p):  prio(i) := p; if p >= b_(box of i + 1): move i to the
                 box of p, searching upwards and extending   (findUp())
moveDown():                          (called by top(), prio() and pop())
  b := the first non-empty box                                (findFirst())
  if b > 0:
    m' := the least priority in box b;  e := b_(b+1)
    for k = 0 .. b: b_k := min(m' + w_0 + ... + w_(k-1), e)
    move each item of box b down to the box of its priority
pop():           moveDown(); remove the first item of box 0
erase(i):        remove i from its box
```

The record of an item removed is replaced by the last record of the vector, whose position is then updated in the cross-reference map.

Lemma 7.4. *The operations of a* ***`RadixHeap`*** *keep $b_0 \le b_1 \le \cdots \le b_K$, keep each range $[b_k, b_{k+1})$ no wider than w_k, and keep each item in the box whose range contains its priority. After* ***`moveDown()`****, box 0 is not empty and all its items have the least priority of the heap; every item of the box b redistributed moves to a box of smaller index.*

Proof. The constructor creates the boxes $[m, m+1)$, $[m+1, m+2)$ and then boxes of doubling width, each starting where the previous one ends, and `extend()` does the same; `push()`, `decrease()` and `increase()` place an item by searching for the box whose range contains its priority. In `moveDown()`, the least priority m' of the first non-empty box b lies in $[b_b, e)$, whose width is at most $w_b = 2^{b-1} = w_0 + \cdots + w_{b-1}$ for $b \ge 1$. Hence, the new starts $b_k = \min(m' + w_0 + \cdots + w_{k-1}, e)$ for $k \le b$ are non-decreasing, they begin at $b_0 = m'$, the range of each box $k < b$ is no wider than w_k, the boxes $0, \dots, b-1$ cover $[m', e)$, and box b gets the empty range $[e, e)$; the boxes above b are unchanged. Every item of box b has its priority in $[m', e)$, so that it moves to a box of index below b, and the item of priority m' moves to box 0, whose range is $[m', m'+1)$. Since the boxes below b were empty and those above hold larger priorities, m' is the least priority of the heap. □

Hence, the priorities must not go below b_0. This is `minimum` at first and, once `top()`, `prio()` or `pop()` has found box 0 empty, the least priority in the heap at that moment. A `push()`, `set()` or `decrease()` below it throws the exception `RadixHeap< IM >::PriorityUnderflowError`, even if no item has been removed yet. The constructor `RadixHeap( IM & map, int minimum = 0, int capacity = 0 )` creates the boxes for the priorities from `minimum` to `minimum + capacity - 1`, the heap adds boxes when a larger priority arrives, and `clear(minimum, capacity)` empties it and creates the boxes again.

Theorem 7.6. *Let K be the number of boxes, i.e., $O(\log U)$ with U the largest priority pushed minus the initial*

minimum (or the capacity, if larger). In a ***RadixHeap****,* ***push()*** *and* ***increase()*** *cost* $O(K)$*,* ***erase()*** *costs* $O(1)$*, and, in amortized terms,* ***decrease()*** *costs* $O(1)$ *and* ***top()****,* ***prio()*** *and* ***pop()*** *cost* $O(K)$*.*

Proof. Let Φ be the sum, over the items in the heap, of the index of their box. A search downwards (or upwards) over j boxes costs $O(1+j)$ and lowers (raises) Φ by j. `push()` costs $O(K)$ for the extensions and the search, and raises Φ by at most K; `increase()` likewise. `decrease()` costs $O(1+j)$ and lowers Φ by j, i.e., $O(1)$ amortized. `moveDown()` costs $O(b)$ to find box b, plus $O(s)$ to scan its s items for m' and to move them, and each of these moves lowers Φ by at least one (Lemma 7.4); hence, its amortized cost is $O(K)$. `erase()` and the removal of `pop()` unlink one item and move one record. □

In `Dijkstra` with integer lengths in $[0, C]$, the priorities in the heap lie in $[d, d+C]$, where d is the last distance removed, so that the priorities never go below b_0; for this case, Ahuja et al. 1990 (cf. Ahuja, Magnanti and Orlin 1993, §4.8) give the bound $O(m + n\log(nC))$ on the running time, m and n being the numbers of arcs and nodes, which follows from Theorem 7.6 with $U \le nC$.

abLEMON. The boxes of a `RadixHeap` stay contiguous: in LEMON 1.3.1, `moveDown()` gave the boxes before the first non-empty one ranges that could reach past the start of the next box, which still held its items, so that after an `increase()` an item could sit in a box before one with a smaller priority and be popped first (trac #681); `test/heap_test.cc` has the case of the ticket and a long monotone sequence of operations checked against a sorted set.

Bucket heaps. `BucketHeap< IM, MIN >`, in `lemon/bucket_heap.h`, keeps a doubly linked list of the items of each priority, with the heads in a vector indexed by the priority (the buckets of Dial 1969), together with a pointer μ to a bucket at or below that of the minimum. The priorities must be non-negative, since they index the vector, which grows up to the largest priority plus one. The items are private records `BucketItem` (the item, its priority and its two links), kept contiguous in a vector: an erased record is replaced by the last one, whose position is then updated in the cross-reference map.

Algorithm 7.6 (Bucket heap, `MIN = true`).

```
push(i, p):      link i at the head of bucket p (growing the vector);
                 if p < mu: mu := p
decrease(i, p):  unlink i; if p < mu: mu := p; link i in bucket p
increase(i, p):  unlink i; link i in bucket p
top(), prio(), pop():
                 while bucket mu is empty: mu := mu + 1
                 return (or unlink) the head of bucket mu
erase(i):        unlink i
```

Theorem 7.7. *A* ***BucketHeap*** *keeps every non-empty bucket at or above* μ*, so that* ***top()*** *returns an item of least priority.* ***push()****,* ***erase()****,* ***decrease()*** *and* ***increase()*** *cost* $O(1)$ *apart from the growth of the vector, and* ***top()****,* ***prio()*** *and* ***pop()*** *cost* $O(1+j)$*, where* j *is the number of buckets that* μ *passes. On a sequence of operations with* r *removals in which no priority pushed or decreased is below the last minimum removed, the advances of* μ *add up to at most* C*, the largest priority, and all the removals cost* $O(r+C)$*.*

Proof. The invariant holds when the heap is built ($\mu = 0$); `push()` and `decrease()` lower μ to the new priority when it is below, `increase()` and `erase()` empty no bucket below μ, and the loop of `top()` advances μ over empty buckets only. The costs of the list operations are $O(1)$. After a removal μ equals the priority removed, and under the hypothesis no later operation lowers it; hence, μ never decreases and never exceeds C. □

With `MIN = false` the heap gives the item of largest priority, and the pointer moves downwards (Theorem 7.7 holds with the order reversed); e.g., `MaxCardinalitySearch` uses `BucketHeap< IM, false >` when its capacity map is the constant map `ConstMap< Arc, Const< int, 1 > >`. The header states that the class "is not intended to use as a Dijkstra heap". It works there (the test runs `Dijkstra` with it), but its time and memory grow with the largest distance, rather than with the number of nodes.

`SimpleBucketHeap< IM, MIN >` is a reduced bucket heap, with singly linked lists and a free list of records. It supports only `push()`, `top()`, `prio()`, `pop()`, `size()`, `empty()`, `clear()` and `state(i)`, with the costs of Theorem 7.7; it has no `erase()`, `decrease()`, `increase()`, `set()` or `state(i, st)`, and it does not conform to the concept. Its `operator[](i)` scans all the buckets, i.e., it costs $O(n+C)$, and it returns -1 for an item that is not in the heap.

7.5 Choosing a heap

Table 7.1 summarizes the costs proved in sections 7.2 to 7.4; n is the number of items in the heap, $N \geq n$ the number of records of a linked heap, d the degree of a `DHeap`, K the number of boxes of a `RadixHeap` (Theorem 7.6), and C the largest priority of a bucket heap.

Table 7.1. The costs of the heaps (am. = amortized; "−" = not available).

Heap	`push`	`top`	`pop`	`decrease`	`increase`, `erase`
`BinHeap`	$O(\log n)$	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
`QuadHeap`	$O(\log_4 n)$	$O(1)$	$O(4 \log_4 n)$	$O(\log_4 n)$	$O(4 \log_4 n)$
`DHeap`	$O(\log_d n)$	$O(1)$	$O(d \log_d n)$	$O(\log_d n)$	$O(d \log_d n)$
`FibHeap`	$O(1)$	$O(1)$	$O(\log N)$ am.	$O(1)$ am.	$O(\log N)$ am.
`PairingHeap`	$O(1)$	$O(1)$	$O(\log n)$ am.	$o(\log n)$ am.	$O(\log n)$ am.
`BinomialHeap`	$O(\log n)$	$O(1)$	$O(\log n)$	$O(\log n)$	$O(\log n)$
`RadixHeap`	$O(K)$	$O(K)$ am.	$O(K)$ am.	$O(1)$ am.	$O(K)$, $O(1)$
`BucketHeap`	$O(1)$	$O(C)$	$O(C)$	$O(1)$	$O(1)$
`SimpleBucketHeap`	$O(1)$	$O(C)$	$O(C)$	−	−

The costs of `push()` and `decrease()` of the pairing heap are its actual costs, and the amortized ones are those of Theorem 7.4. For the bucket heaps, the costs of `top()` and `pop()` are worst case; on a monotone sequence of operations, they add up to $O(r + C)$ in all (Theorem 7.7).

These bounds suggest a choice, which measurements on the instances of interest should confirm. A binary or 4-ary heap is the safe default, since it is simple, it has small constants and good locality, and it supports each operation in logarithmic time with any priority type and comparison. A Fibonacci or pairing heap may pay off when the decreases far outnumber the removals and the heap is large; the counts of comparisons then favour it, but the constants often do not. A radix heap or a bucket heap fits integer priorities with a monotone sequence of minima and a "small" span of priorities.

The algorithms of the library take the type of their heap from their traits class, and they accept another one through a named template parameter (cf. chapter 3 and chapter 10). `SetStandardHeap< H, CR >` gives the heap type `H` and the cross-reference type `CR` (by default `Digraph::NodeMap< int >`), and the algorithm then creates both objects, as `CR( digraph )` and `H( cr )`. `SetHeap< H, CR >` gives only the types, and the objects are passed by the function `heap(h, cr)` before `run()` or `init()`. For instance, the following declaration runs `Dijkstra` on a radix heap.

```
Dijkstra< ListDigraph, ListDigraph::ArcMap< int > >
  ::SetStandardHeap< RadixHeap< ListDigraph::NodeMap< int > > >
  ::Create dijkstra( g, length );
```

The program `manual/examples/07-dijkstra-heaps.cc` (the test `manual_07_dijkstra_heaps`) solves one shortest path problem on a grid digraph with each heap of the library in this way, and checks the distances against those of the default binary heap.

abLEMON. `BidirDijkstra` and `AStar` (which LEMON 1.3.1 did not have) and `MinCostArborescence` take the type of their heap from their traits, and they accept another one through the named parameter `SetHeap`.

The program `manual/examples/07-heap-sort.cc` (the test `manual_07_heap_sort`) uses the heaps directly: each of them sorts one sequence of priorities after a decrease, an increase and a `set()`, with the states checked after each removal; then a `BinHeap` with `std::greater` and a `BucketHeap` with `MIN = false` give the largest priority first, a `SimpleBucketHeap` sorts by push and pop only, and a `RadixHeap` throws on a priority below the last one removed.

7.6 Exercises

Exercise 7.1. From Lemma 7.1, count the comparisons of `pop()` in a `DHeap` of n items as a function of d, and find the degree that minimizes the bound of Theorem 7.2 when $m = n^{1+\epsilon}$ for a fixed $\epsilon > 0$.

Exercise 7.2. Extend `manual/examples/07-heap-sort.cc` with `DHeap` of degrees 2, 8 and 32, and with `std::greater` as the comparison for each comparison heap; then call `erase()` on an item that is not the top and

check that the others still come out in order (Theorems 7.3 to 7.5). The function `heapEraseTest()` of `test/heap_test.cc` does it for the comparison heaps.

Exercise 7.3. On grid digraphs of growing size, time `manual/examples/07-dijkstra-heaps.cc` with each heap and with lengths in $[1, 9]$ and in $[1, 10^6]$, and explain the ranking of `BucketHeap` and `RadixHeap` in the two cases by Theorems 7.6 and 7.7. The test `test/heap_test.cc` checks the distances of each heap (function `dijkstraHeapTest()`).

Exercise 7.4. Show by an example that a `RadixHeap` throws `PriorityUnderflowError` on a `push()` below the current minimum after a call of `top()` with no removal (Lemma 7.4), and explain why `Dijkstra` never does it.

Exercise 7.5. Write the cross-reference map of a heap whose items are the integers $0, \ldots, k-1$ as a `RangeMap< int >`, and reuse one heap for several sorts, resetting the map between them with `state(i, PRE_HEAP)` instead of a new map. The function `heapSortTest()` of `test/heap_test.cc` sorts with a `RangeMap`.

8. Paths and auxiliary structures

This chapter collects the data structures and the tools that the algorithms of Part III use internally, and that a program often needs around them. They are the paths of `lemon/path.h` (with the concept of `lemon/concepts/path.h`), the union-find structures of `lemon/unionfind.h`, the elevator of `lemon/elevator.h`, the plane geometry of `lemon/dim2.h` and the radix sorts of `lemon/radix_sort.h`; then come the random numbers of `lemon/random.h`, the counters and timers of `lemon/counter.h` and `lemon/time_measure.h`, the colours of `lemon/color.h`, and the exceptions, assertions and parser of the command line of `lemon/error.h`, `lemon/assert.h` and `lemon/arg_parser.h`. The first three are data structures whose costs matter for the algorithms built on them, and we give them most of the chapter. We describe the others rather more briefly, but with all their members, since the reader of a program that uses them should find each call here.

8.1 Paths

A path of a digraph $G = (N, A)$ is a sequence of arcs $(a_1, \ldots, a_k)$ such that the target of a_i is the source of a_{i+1} for $i = 1, \ldots, k-1$. Its length, in the sense of the path classes, is the number k of its arcs (the length under a cost map is computed by the user, cf. §10.1). The four path classes of abLEMON store the sequence of arcs and no other data, i.e., they do not store the nodes, they cannot give the source of a path of no arcs, and they do not check that consecutive arcs are joined (which is left to the function `checkPath()`). All four take the digraph type as their only template parameter `GR`, and define the types `Digraph` and `Arc`.

Their common interface is made of a default constructor, a constructor and an assignment from a path of any type (both implemented by `pathCopy()`), `length()`, `empty()` and `clear()`. The arcs are read by `nth(n)` and by its alias `operator[]`, by `front()` and `back()`, and by the iterator `ArcIt`, which converts to `Arc` and runs from the first arc to `INVALID`; `nthIt(n)` places an `ArcIt` on the n-th arc, and `arcs()` wraps `ArcIt` in a range for the range-based `for` of C++11. The classes differ in how the arcs are stored, which decides what can be done and at which cost (Table 8.1).

Table 8.1. The path classes of `lemon/path.h` and the cost of their operations, for a path of k arcs ("am." stands for amortized, "no" for an operation that the class does not have).

class	storage	`nth(i)`	`addFront`, `eraseFront`	`addBack`, `eraseBack`	`length()`	other
`Path`	two vectors	$O(1)$	$O(1)$ am.	$O(1)$ am.	$O(1)$	
`SimplePath`	one vector	$O(1)$	no	$O(1)$ am.	$O(1)$	
`ListPath`	doubly linked list	$O(i)$	$O(1)$	$O(1)$	$O(k)$	`splice`, `split` in $O(1)$
`StaticPath`	one array	$O(1)$	no	no	$O(1)$	read only

In `Path` the arcs added at the front are kept in one vector, in reverse order, and those added at the back in another. When one of the two vectors is empty and an arc must be erased from its end, half of the other vector is moved into it. Hence, `eraseFront()` and `eraseBack()` cost $O(1)$ amortized over any sequence of operations. `SimplePath` keeps one vector and grows only at the back, which makes it somewhat faster than `Path` when the arcs are only added at the back. A `ListPath` allocates one node per arc; hence, `nth()` walks i nodes and `length()` walks the whole list; however, it is the only class that can be cut and joined in constant time, which is indeed the reason to prefer it. Its member `spliceBack(tpath)` moves all the arcs of `tpath` to the back of the path, and `spliceFront(tpath)` to its front, leaving `tpath` empty; `splice(it, tpath)` inserts them before the arc of the iterator `it` (at the back if `it` is `INVALID`). Conversely, `split(it, tpath)` moves the arcs from `it` on into `tpath`, after clearing it (the clearing being the only part that may cost more than $O(1)$). Finally, a `Sta`

`ticPath` is an array of exactly k arcs, which can only be built from another path or assigned; it is the most compact of the four, and usually the one to choose for storing many paths. The bounds of Table 8.1 follow from the storage, except the amortized one of `Path`, which needs an argument.

Theorem 8.1. *Starting from an empty* `Path`*, any sequence of q calls of* `addFront()`*,* `addBack()`*,* `eraseFront()` *and* `eraseBack()` *takes $O(q)$ time.*

Proof. Let f and b be the numbers of arcs in the vector of the front and in that of the back, and let $\Phi = |f - b|$, which is 0 at the start and never negative. An addition, or an erasure from a vector that is not empty, takes $O(1)$ time (amortized over the growth of the vector) and changes Φ by at most 1. An erasure from an empty vector, say the front one, with $b = r \geq 1$ arcs in the other, erases the first arc of the back vector, moves the next $\lfloor r/2 \rfloor$ arcs into the front vector and shifts the remaining $r - 1 - \lfloor r/2 \rfloor$ down, in $O(r)$ time; afterwards $\Phi \leq 1$, while it was r before. Hence, with a suitable constant c, the amortized cost (the actual cost plus c times the change of Φ) of each call is $O(1)$. The total actual cost is at most the total amortized cost, since $\Phi \geq 0$ at the end. □

We remark that the copy assignment of each path class calls `pathCopy()`, which clears the target before reading the source; hence, assigning a path to itself (`p = p`) leaves it empty.

The concept `concepts::Path` of `lemon/concepts/path.h` states this common interface (a path type, the copy from another path, `length()`, `empty()`, `clear()`, `ArcIt` and `arcs()`), and the four classes conform to it. A second concept, `concepts::PathDumper`, describes the objects that the search algorithms of chapters 9 and 10 return from their queries `path()`. A path dumper has `length()`, `empty()` and either an `ArcIt` or, when its type `RevPathTag` is `True`, a `RevArcIt` that lists the arcs from the last to the first. In the library, the dumpers are `PredMapPath` and `PredMatrixMapPath` of `lemon/bits/path_dump.h`, which follow a map of predecessor arcs back from a node, and therefore list the arcs in reverse. They are not meant to be kept, but to be assigned to a path, as in `Path<ListDigraph> p = dijkstra.path(t);` (note that an `ArcIt` built on the temporary that such a conversion creates would point to a destroyed object).

All constructors and assignments of a path call `pathCopy(from, to)`. This function checks at compile time that `from` is a path dumper, and then chooses among four ways of copying. If `from` lists its arcs backwards (`RevPathTag`), the arcs are added to the front of `to`, else to its back; and if `to` defines the type `BuildTag` (as the four classes do), its member `build()` or `buildRev()` fills it in one pass, after reserving the memory, in place of a sequence of `addBack()` or `addFront()`. This choice is made by the helpers `RevPathTagIndicator`, `BuildTagIndicator`, `PathCopySelector`, `PathCopySelectorForward` and `PathCopySelectorBackward` of the namespace `_path_bits`, which are not part of the interface. Also, `copyPath(to, from)` is the same copy with the arguments in the opposite order, kept only for compatibility with older programs.

Four other utilities read a path together with its digraph. The predicate `checkPath(g, p)` is true if the target of each arc of `p` is the source of the next one (an empty path being consistent), and `pathSource(g, p)` and `pathTarget(g, p)` return the source of the first arc and the target of the last one, or `INVALID` for an empty path. Next, the iterator `PathNodeIt<Path>` lists the $k + 1$ nodes of a consistent path from its source to its target, given the digraph and the path in its constructor; a third argument gives the node to start from, which is needed only for a path of no arcs. Also, `pathNodes(g, p)` wraps this iterator in a range, and `for (auto u : pathNodes(g, p))` then visits the nodes in order.

> **abLEMON.** The allocator of `ListPath` constructs and destroys its nodes through `std::allocator_traits`, since C++20 removed the members `construct()` and `destroy()` of `std::allocator` that LEMON 1.3.1 called; the paths therefore compile from C++11 to C++23. The alias `operator[]` of `nth()` and the ranges of `arcs()` and `pathNodes()` come from the development of LEMON after its release 1.3.1, which did not distribute them.

The program `manual/examples/08-paths.cc` (the test `manual_08_paths`) builds a path at both ends and copies it into the other three classes. Then it splits and splices a `ListPath`, lists the nodes of a path, and copies the path dumper of a search of Dijkstra (cf. §10.2) into a `Path`.

8.2 Union-find structures

A union-find structure maintains a partition of a finite set of items into classes under two operations: the union of the classes of two items, and the find of the class of an item, whose answer is a representative of the class, the same for all its items. It is the structure behind Kruskal's algorithm (§11.1), where the classes are the components of the forest grown so far, and behind the blossoms of the weighted matching algorithms of chapter 17. Each class is stored as a tree whose root is the representative and whose other items point to their parents. A find walks from the item to the root, and a union makes one root the child of the other. Two rules keep the

trees shallow: the union by size puts the root of the tree with fewer items under the root of the other, and the path compression makes the items met during a find point to the root (Figure 8.1).

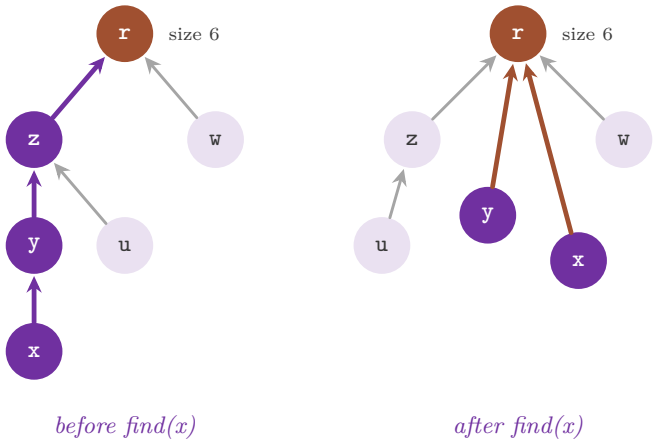


Figure 8.1. A class of a union-find structure as a tree, the root r carrying the size of the class: find(x) follows the path x, y, z, r (left), and the compression of that path makes x and y children of r (right).

Lemma 8.1. *With the union by size, a tree of s items has depth at most $\log_2 s$, whatever finds (with or without compression) are made.*

Proof. The compression only shortens paths, and we may ignore it. An item gets deeper only when the root of its tree is put under another root, by a union with a tree at least as large; the tree that contains it then at least doubles. Since a tree has at most s items, this happens at most $\log_2 s$ times to each item. □

Theorem 8.2. *With the union by size and the path compression, any sequence of n insertions and $m \geq n$ finds and unions takes $O(m\,\alpha(m,n))$ time, where α is the inverse of Ackermann's function, which is at most 4 for any n that can be stored.*

Proof sketch. Each union costs $O(1)$ after its two finds, and a find costs $O(1)$ plus the number of items whose parent it changes. One groups the items by the logarithm of the size of their subtree, which by Lemma 8.1 grows along each path to the root, and then the levels into blocks of exponentially growing ranges, as many as the inverse of Ackermann's function allows. A compression changes the parent of an item either for the last time in its block, which is charged to the find (at most $\alpha(m,n)$ such items per find), or to a parent in a strictly higher level of the same block, which is charged to the item. The charges to the items of each block add up to $O(m)$ over the blocks. The bound was proved in Tarjan 1975 for the union by rank, and in Tarjan and van Leeuwen 1984, section 3, for the union by size and the other linking rules. A textbook proof, for the union by rank, is in Cormen et al. 2001, §21.4. □

In practice, the cost of an operation is usually a small constant, and one may take it as such when choosing an algorithm.

All the classes of `lemon/unionfind.h` take as template parameter the type `IM` of a map from items to integers. The items are the keys of `IM` (typically the nodes, arcs or edges of a graph), and the map, given to the constructor by reference, stores the index of each item in the internal vectors. From then on the map belongs to the structure: writing into it corrupts the structure, and each item must be added with `insert()` before it is used in another operation.

`UnionFind<IM>` is the plain structure, with four operations and `clear()`. Here `insert(a)` makes `a` a singleton class and returns its index, and `find(a)` returns the index of the representative of the class of `a`, an integer between 0 and the number of the items inserted, minus one (the index that `insert()` returned for that item). A call `join(a, b)` returns `false` if `a` and `b` are already in the same class, and otherwise unites the two classes and returns `true`; `size(a)` is the number of items of the class of `a`. Algorithm 8.1 gives `find()` and `join()` as the code has them, *parent* and *size* being stored in one vector of integers (a root holds its size, negated, and any other item the index of its parent); Lemma 8.1 and Theorem 8.2 apply to it. Note that `clear()` empties the vector but leaves the map, whose old values must not be used before the items are inserted again.

Algorithm 8.1 (Find and union in `UnionFind`).

`find(x):`

```
1  r := x
2  while r is not a root: r := parent(r)
3  while x != r: next := parent(x); parent(x) := r; x := next
4  return r

join(a, b):
1  ra := find(a); rb := find(b)
2  if ra = rb: return false
3  if size(ra) > size(rb): parent(rb) := ra; size(ra) := size(ra) + size(rb)
4  else:                   parent(ra) := rb; size(rb) := size(rb) + size(ra)
5  return true
```

Next, `UnionFindEnum<IM>` adds the enumeration of the classes and of the items of each class, at the price of some memory and time per operation. Each class keeps a circular doubly linked list of its items (its private records `ItemT` and `ClassT`), and the classes are linked in a second list; it joins by size and compresses the paths as `UnionFind` does. Its classes are identified by integer ids rather than by representatives. Thus, `insert(item)` makes a new class and returns its id, `insert(item, cls)` adds the item to the class `cls`, `find(item)` returns the id of the class of the item, and `join(a, b)` returns the id of the united class, or -1 if the two items were already together. Also, `size(cls)` is the number of items of a class, and `item(cls)` returns one of its items. This structure can also undo: `split(cls)` breaks a class into singletons, and `erase(item)` removes an item from its class, deleting the class if it becomes empty. Finally, `eraseClass(cls)` removes a whole class, and `clear()` all of them. Its iterator `ClassIt` lists the ids of the classes, and `ItemIt`, constructed from the structure and a class id, lists the items of that class; both are compared with `INVALID` to stop.

Theorem 8.3. *In a `UnionFindEnum`, `insert()`, `size()`, `item()` and `eraseClass()` take $O(1)$ time, `find()` and `join()` the time of the finds of Theorem 8.2 plus $O(1)$, and `split(cls)` and `erase(item)` $O(s)$, where s is the size of the class. In an `ExtendFindEnum`, all the operations take $O(1)$ time.*

Proof. A join links the two roots and splices the two circular lists, in $O(1)$ after the finds. The split walks the list of the class once, making each item a root of its own class, and the erasure makes the next item of the list the root and walks the list once to point every other item to it, which keeps the trees of depth one. The other operations of `UnionFindEnum` read or relink a constant number of records. An `ExtendFindEnum` stores in each item its class, which never changes; its other operations relink lists. □

We remark that `clear()` of `UnionFindEnum` empties the vector of the items and the list of the classes, but neither the vector of the classes nor its list of free records; hence, the records of the classes alive before the call are never reused, which costs memory but no correctness.

`ExtendFindEnum<IM>` is the simplification of `UnionFindEnum` in which the classes are never united. It has the same two forms of `insert()`, `find()`, `item()`, `erase()`, `eraseClass()`, `clear()`, `ClassIt` and `ItemIt`, and no `join()`, a class therefore growing only by insertion; in exchange, `find()` reads the class of an item in $O(1)$ (Theorem 8.3). All three classes define the types `ItemIntMap` (that is, `IM`) and `Item`.

`HeapUnionFind<V, IM, Comp>` attaches a priority of type `V` to each item and keeps the item of least priority of each class, where `Comp` (by default `std::less<V>`) decides what least means. It is stored as a $(2, 16)$-tree whose leaves are the items, with a tournament for the minimum on the internal nodes, i.e., each internal node has between 2 and 16 children (the root excepted) and stores the least priority of its subtree and its item. Besides the union, it supports the split of a class into the classes it was made of, which the blossoms of the weighted matching algorithms need. A call `insert(item, prio)` makes a class of one item and returns its id, and `find(item)` returns the id of the alive class of the item. A call `join(begin, end)` unites the k classes whose ids are in the range (at least two) and returns the id of the new class, and `split(cls, out)` restores the k classes of which `cls` was made, and writes their ids into the output iterator `out`.

Theorem 8.4. *In a `HeapUnionFind`, `insert()` takes $O(1)$ time, `find()` and the changes of a priority $O(\log n)$, where n is the number of the items of the class, and `join()` and `split()` of k classes $O(k \log n)$, where n is the number of the items involved.*

Proof sketch. A tree whose internal nodes have at least two children has height at most $\log_2 n$. A find walks from the leaf of the item to the root; a decrease walks up from the leaf while the new priority beats the minimum of the node, and an increase recomputes the minimum of the at most 16 children of each ancestor whose minimum was the item; each of them is therefore $O(\log n)$. A join hangs the trees of the k classes on one another, in the order of the range, under nodes of the right height, and splits a node into two when it exceeds 16 children. This costs $O(\log n)$ per class joined; a split cuts the tree along the paths of the k subtrees that the join recorded, and

repairs the degrees and the minima along them (the private members `repairLeft()` and `repairRight()`), with the same cost. The bounds are those of the documentation of the class. The argument is the usual one for the B-trees and their variants (Cormen et al. 2001, chapter 18). □

Priorities are read by `operator[](item)` and written by `set(item, prio)`, `decrease(item, prio)` and `increase(item, prio)`, the last two requiring a priority that is not larger, or not smaller, than the present one (and `set()` choosing between them); the class defines `Value`, `Item`, `ItemIntMap` and `Compare`. The least priority of a class and its item are given by `classPrio(cls)` and `classTop(cls)`, and `classRep(cls)` gives an item of the class that does not depend on the priorities. Furthermore, `alive(cls)` is true if a class is not nested into another one (only alive classes are returned by `find()`), and `trivial(cls)` if it holds a single item directly. Its iterators are `ItemIt`, which lists the items of a class, and `ClassIt`, which lists the alive classes, or, constructed with a class id, the classes of which `cls` is made. Finally, the structures `ItemT`, `ClassT`, `ItemNode` and `ClassNode` of the header are the internal records of these classes.

The program `manual/examples/08-union-find.cc` (the test `manual_08_union_find`) computes the components of a small graph with `UnionFind`, counting the edges that close a cycle. It also enumerates them with `UnionFindEnum`, and follows the minimum of a class of `HeapUnionFind` through a join, a decrease and a split.

8.3 The elevator

Push-relabel algorithms of maximum flow (Goldberg and Tarjan 1988; see chapter 12) keep a label on each node, i.e., a lower bound on its distance to the sink in the residual digraph, and a set of active nodes, those with a positive excess. At each step they take an active node of highest label (in the variant that we consider here), push its excess along the admissible arcs, and lift its label when no arc is admissible. Furthermore, the gap heuristic of Cherkassky and Goldberg 1997 moves to the top all the nodes above a level that has become empty, since none of them can reach the sink any longer. An elevator is the data structure that supports these operations, each in (amortized) constant time. Two of them are in `lemon/elevator.h`, `Elevator` and `LinkedElevator`, with the same interface; `Preflow` and `Circulation` take the type of theirs from their traits class (`Elevator` by default, another one by the named parameters `SetElevator` and `SetStandardElevator`), and the fractional matchings of chapter 17 use a `LinkedElevator`.

Both classes take as template parameters the type `GR` of the graph and the type `Item` of the labelled items (`GR::Node`, `GR::Arc` or `GR::Edge`), and the labels are the integers from 0 to `maxLevel()`. Its constructor `Elevator(graph, max_level)` sets the largest label, and `Elevator(graph)` takes it equal to the number of the items of the graph, as the flow algorithms do. An `Elevator` stores the items in one array, ordered by label, with the active items of each level before the inactive ones, and with two pointers per level, to its first item and to its last active item (Figure 8.2). Hence, activating, deactivating and lifting by one level are swaps of two places of the array, and `onLevel(l)`, `emptyLevel(l)`, `aboveLevel(l)` and `activesOnLevel(l)` are differences of pointers, computed in $O(1)$. A `LinkedElevator` keeps one doubly linked list per level instead; it needs no array of all the items, but it counts (somewhat slowly) the items of a level, or above a level, by walking the lists.

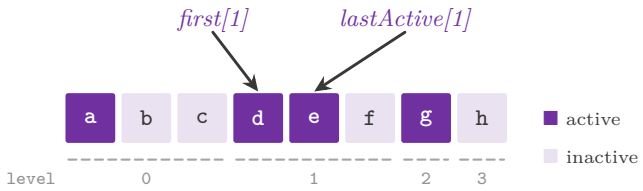


Figure 8.2. The array of an Elevator: the items in the order of their levels, the active items of each level first, and the two pointers of level 1.

The interface of the two classes has four groups of members. First, the initialization lists the items level by level: `initStart()` begins it, `initAddItem(i)` puts an item on the current level (level 0 at first), `initNewLevel()` passes to the next level, and `initFinish()` ends it, the items not listed being put on the top level, all inactive. Afterwards, the level of an item is read by `operator[](i)` and its state by `active(i)`, which `activate(i)` and `deactivate(i)` change (with the preconditions that the item is inactive, or active, before the call).

Note that the highest active item, given by `highestActive()` (or `INVALID`), has its level given by `highestActiveLevel()` (or −1); it is lifted by one level by `liftHighestActive()`, to a given higher level by `liftHighestActive(l)`, or to the top level, where it is deactivated, by `liftHighestActiveToTop()`. Three similar operations exist for the active item that `activeOn(l)` returns on a given level, namely `liftActiveOn(l)`, `liftActiveOn(`

`l, new_level)` and `liftActiveToTop(l)`, and `lift(i, new_level)` lifts any active item. Finally, `liftToTop(l)` moves all the items on level `l` and above to the top level and deactivates them (the gap heuristic), and `activeFree(l)` is true if a level has no active item. Also, `dirtyTopButOne(i)` moves an inactive item from the top level to the level below it without updating the structure, and the documentation reserves it to those who know what it is for. The costs of these operations follow from the invariant of the array.

Lemma 8.2. *After `initFinish()` and after each operation other than `dirtyTopButOne()`, the array of an `Elevator` holds the items in nondecreasing order of level. For each level l, its items occupy the places from first(l) to first$(l+1)-1$, the active ones before the others, the last active one at lastActive(l); and highest is the highest level with an active item (-1 if none).*

Proof. The initialization lists the items level by level and puts the others on the top level, all inactive. An activation swaps the item with the first inactive item of its level and increments *lastActive*, and a deactivation does the converse. A lift of an active item from level l to level $l' > l$ first moves it to the end of the active part of l. Then, for each level from l to $l'-1$, it moves the last item of that level (after its active part has been shifted by one place) to the first place of the next one, shifting the boundary *first* of each level crossed by one place to the left. The item lands in the first place of l', among the active ones. `liftToTop(l)` relabels all the items from level l on and moves all the boundaries of these levels to the first place of l. In each case *highest* is raised to a new active level, or lowered past the levels left without active items. □

Theorem 8.5. *In an `Elevator`, `activate()`, `deactivate()` (apart from the lowering of highest), the queries and `liftHighestActive()` take $O(1)$ time, a lift from level l to level l' takes $O(l'-l)$, and `liftToTop(l)` takes $O(k+L)$, where k is the number of the items moved and L the number of levels; the lowering of highest costs one step per level passed. In a `LinkedElevator`, the activations and the lifts take $O(1)$ time (apart from the lowering of highest), and `onLevel()`, `aboveLevel()` and `activesOnLevel()` are proportional to the number of the items they count.*

Proof. By Lemma 8.2, each of the first operations swaps two places or reads two boundaries, and a lift moves one item per level crossed. In a `LinkedElevator`, each level is a doubly linked list with the active items at its front. An activation or a lift relinks one item, and the counts walk the lists. □

abLEMON. `Elevator` allocates one place more than the items of the graph, so that it may be constructed on a graph without items; in LEMON 1.3.1, `Circulation` read past its storage on an empty digraph through it.

We remark that the constructor with a given largest label may take a label smaller than the number of the items, in which case the items not listed by the initialization stay on the largest level. In a `LinkedElevator` the items of the largest level are kept in no list, so that `onLevel()` and `aboveLevel()` count only those below it, as the flow algorithms expect. Also, `liftToTop(l)` of a `LinkedElevator` moves the items of the levels from $l+1$ up to the first empty one, which is what the gap heuristic needs (it calls it on an empty level l, and the levels above are then without gaps), while that of an `Elevator` moves all the items from level l on. The two classes define `Key` (the item) and `Value` (`int`), so that they can be read as maps of the levels.

abLEMON. In LEMON 1.3.1 the constructors with a largest label also took it as the number of the items, so that `Elevator(graph, max_level)` sized its storage and counted the items correctly only when the two were equal; and `LinkedElevator::lift(i, new_level)` updated the ends of the list of the new level, where it had to update those of the level that the item leaves, whenever the item was the first or the last of its list. The test `elevator_test` checks both.

8.4 Two-dimensional geometry

In `lemon/dim2.h`, the namespace `dim2` holds the plane vectors and boxes that the drawing function `graphToEps()` of chapter 21, the planar drawings of chapter 18 and the generators of geometric instances use. A `dim2::Point<T>` is a vector with the public coordinates `x` and `y` of type `T`. It is constructed from two coordinates (the default constructor value-initializes them) or converted from a `Point` of another coordinate type, and it has `size()` (always 2), `operator[]` (0 for `x`, 1 for `y`) and `normSquare()`. Its operators are the sum, the difference and the negation of vectors, the product by a scalar on either side and the division by a scalar, the scalar product `operator*` of two vectors, the comparisons `==` and `!=`, and the forms `+=`, `-=`, `*=` and `/=`. Free functions `makePoint(x, y)`, `rot90(p)`, `rot180(p)` and `rot270(p)` (the rotations in the positive direction) complete it, together with the stream operators, which write `(x,y)` and read the same form, with the parentheses and the comma optional.

A `dim2::Box<T>` is an axis-parallel rectangle, possibly empty. Its default constructor makes an empty box, while the box of a point, of two corners (bottom left and top right) or of four sides (left, bottom, right, top) is not empty. The calls `add(p)` and `add(box)` enlarge it to contain a point or another box, `operator&` intersects two

boxes (the result being empty if they do not overlap), and `inside(p)` is true if a point is in the closed box (never for an empty box); `empty()` and `clear()` read and reset the emptiness. Its corners and sides are read by `bottomLeft()`, `topRight()`, `bottomRight()`, `topLeft()`, `bottom()`, `top()`, `left()` and `right()`, and set by the members of the same names that take the new value; `width()` and `height()` give the sizes. Values of an empty box are undefined, and the setters require a box that is not empty; the stream operator `>>`, which reads `((l,b),(r,t))`, gives the box of the two corners, which is not empty (in LEMON 1.3.1 it set the corners by the setters, and therefore left an empty box empty; the test `dim_test` reads one). Finally, the maps `XMap` and `YMap` (read and write) and `ConstXMap`, `ConstYMap` and `NormSquareMap` (read only) turn a map whose values are points into a map of their coordinates or of their squared norm, and are made by `xMap()`, `yMap()` and `normSquareMap()`.

8.5 Radix sort

Both radix sorts of `lemon/radix_sort.h` sort a range of items by an integer key, given by a function object or a function `functor` applied to each item (the identity if it is omitted). Their time is linear in the number of items for a fixed size of the keys; hence, they are often worth trying in place of `std::sort` when the keys are integers of a few bytes. Its key may be of any signed or unsigned integer type, and its type is the one that `functor` returns.

A call `radixSort(first, last, functor)` runs a quicksort whose pivots are the powers of two. For a signed type, it first separates the items with a negative key from the others. Then it finds the highest bit b of the keys of each part, partitions the part on bit b by the swaps of the partition of quicksort, then each half on bit $b-1$, and so on (Algorithm 8.2). It is not stable, and it needs bidirectional iterators.

Algorithm 8.2 (Radix sort by bit partitions, `radixSort()`).

```
radixSort(R):                        R is a range, key(x) an integer
1  split R into R- (key < 0) and R+ (key >= 0) by swaps
2  for each part P in (R-, R+):
3     b := the highest bit in which the keys of P differ from 0 (from 1 in R-)
4     sort(P, b)

sort(P, b):
1  while b >= 0 and P has at least two items:
2     split P into P0 (bit b of the key is 0) and P1 (bit b is 1) by swaps
3     sort(P0, b - 1); P := P1; b := b - 1
```

Theorem 8.6. *`radixSort()` sorts the range in nondecreasing order of the keys in $O(n\,(1+\log_2 c))$ time and with $O(1+\log_2 c)$ extra space, where n is the number of items and c the largest absolute value of a key.*

Proof. In two's complement, the negative keys precede the nonnegative ones, and, within each of the two parts, the order of the keys is the order of the bit strings of their lowest $b+1$ bits, since all the higher bits are equal (to 1 in R^-, to 0 in R^+) by the choice of b in step 3. The procedure `sort(P, b)` sorts the items of P, whose keys agree on all the bits above b, by these bit strings: by induction on b, after the split the keys of P_0 precede those of P_1, and each part is sorted on the lower bits by the recursive call or by the next round of the loop. Each item takes part in one split per bit, from b down, and a split is linear in its range; hence, the time is $O(n)$ per bit, and there are at most $1+\log_2 c$ bits (one more for the negative keys). The recursion is made only on P_0, with one bit less, which bounds its depth. □ A call `stableRadixSort(first, last, functor)` sorts by the bytes of the keys, from the least significant to the most significant, with one counting pass per byte. Each pass counts the items of each of the 256 values of the byte, computes where the items of each value begin, and copies the items in that order into a second buffer, which keeps the relative order of the items with the same byte; on a signed type, the last pass puts the negative keys first. It allocates a buffer of $2n$ items, into which it copies the range and between whose two halves the passes alternate.

Theorem 8.7. *`stableRadixSort()` sorts the range in nondecreasing order of the keys, keeps the relative order of the items with equal keys, and takes $O(B\,(n+256))$ time, where B is the number of bytes of the type of the key (whatever the largest key).*

Proof. A counting pass on byte j is stable: the items of each value of the byte are copied in the order in which they are read. By induction on j, after the pass on byte j the items are sorted by their lowest $j+1$ bytes, read as an unsigned number, with ties in the original order: the pass orders them by byte j, and, being stable, keeps the order of the previous passes among the items with the same byte j. For a signed type, the last pass puts the

values 128 to 255 of the top byte, i.e., those of the negative keys in two's complement, before the values 0 to 127, which gives the order of the signed keys. Each pass reads the items twice and the 256 counters once. □

We remark that the buffer is raw memory, obtained by `allocate()` of a `std::allocator`, into which the items are copied by assignment and from which they are never destroyed. Hence, the function is right only for the items of a trivially copyable type (e.g., integers or plain structures), and `radixSort()`, which sorts in place by swaps, is the one to use for the others. Both functions have overloads for a plain function pointer, and both accept a lambda. The signed or unsigned version of each is chosen at compile time, from the type of the key, by the helpers `RadixSortSelector` and `StableRadixSortSelector` of the namespace `_radix_sort_bits`.

abLEMON. The type of the key is deduced with `decltype` from the call of `functor`, rather than read from its member `result_type`, which C++20 removed from the function objects of the standard library; a lambda or any other callable may therefore be passed, which LEMON 1.3.1 did not accept.

abLEMON. In LEMON 1.3.1, `radixSort()` built the mask of the highest bit as the `int` expression `1 << max_digit`, which overflowed for the keys of a 64-bit type that use more than 31 bits (and the sort then did not end), and shifted negative values to the left; the masks are now computed on the unsigned type of the keys, and the test `radix_sort_test` sorts keys of 64 bits, the extremes of the type included.

8.6 Random numbers

In `lemon/random.h`, abLEMON implements the Mersenne Twister of Matsumoto and Nishimura 1998, a generator of period $2^{19937} - 1$ that is equidistributed in 623 dimensions for 32-bit numbers. It comes in a 32-bit and a 64-bit version, which give different sequences from the same seed. A class template `_random_bits::Random<Word>` holds the interface, and three types instantiate it: `Random32` (the 32-bit version), `Random64` (the 64-bit one) and `Random`, whose word is `unsigned long`, i.e., 64 bits on Linux and macOS and 32 bits on Windows. Thus, a program that must give the same numbers on all platforms should use `Random32` or `Random64`. Its global object `rnd`, of type `Random`, is the generator that the library and most programs use; it is constructed with a fixed seed, and therefore gives the same sequence at each run until it is seeded again. Other names of the namespace `_random_bits` are the implementation: `RandomCore` holds the state and twists it, and `RealConversion`, `IntConversion`, `BoolConversion`, `Mapping`, `Masker`, `Shifting`, `ShiftMultiplier`, `Initializer` and `BoolProducer` turn words into numbers of a given type.

A generator is seeded by its constructor or by `seed()`, from one number or from a range of numbers, each converted to the word of the generator. Without arguments, `seed()` reads four words from `/dev/urandom` by `seedFromFile()` (except on Windows) and, if that fails, calls `seedFromTime()`, which seeds from the process id and the time of day; both return `true` on success. A copy, by construction or by assignment, copies the state, and the copy then gives the same sequence from that point on. However, the constructor from a number is a template on the type of the number, which also matches a generator of the other word size: `Random64 r(r32)` does not copy `r32` but fails to compile, since the state can only be copied between generators of the same type.

Uniform numbers are given by several members. The call `real<T>()` returns a number of type `T` (`double` by default) in $[0, 1)$, and `operator()()`, `operator()(b)` and `operator()(a, b)` a `double` in $[0, 1)$, $[0, b)$ and $[a, b)$. The call `integer(b)` and its alias `operator[](b)` return an integer in $\{0, \dots, b-1\}$, of the type of `b`, and `integer(a, b)` one in $\{a, \dots, b-1\}$. Also, `integer<T>()` and `uinteger<T>()` return an integer uniform over the whole range of `T`, or over its nonnegative part (`int` and `unsigned int` by default), and `boolean()` returns a fair bit, taken from a buffer of the bits of one word.

Several other distributions are available (cf. the test `test/random_test.cc`). The call `boolean(p)` is `true` with probability p, and `gauss()` and `gauss(mean, std_dev)` are normal, by the polar form of the method of Box and Muller 1958 given by Marsaglia and Bray 1964. The call `lognormal(n_mean, n_std_dev)` (or `lognormal(pair)`) returns the exponential of a normal variable of the given parameters, and `lognormalMD(mean, std_dev)` a lognormal variable of the given mean and standard deviation, whose parameters `lognormalParamsFromMD()` computes. Next come `exponential(lambda)` (of mean $1/\lambda$, with $\lambda = 1$ by default), `gamma(k)` for an integer shape (a sum of k exponentials), `gamma(k, theta)` for any positive shape (by rejection), `weibull(k, lambda)`, `pareto(k, x_min)` and `poisson(lambda)`; the last is the method of Knuth 1969. In the plane, `disc()` is uniform on the unit disc (by rejection from the square). Next, `gauss2()` gives the two normal coordinates of one step of the polar method, and `exponential2()` gives the point $(-\ln S/S)\,(V_1, V_2)$ of the same step, a rotationally symmetric distribution whose distance from the origin is $-\ln S/\sqrt{S}$ with S uniform on $(0, 1)$. The two lemmas below justify `gauss()` and `poisson()`, where $U_1, U_2, \dots$ are the independent uniform numbers of $[0, 1)$ that `real()` returns.

Lemma 8.3. *Let $V_1 = 2U_1 - 1$ and $V_2 = 2U_2 - 1$ be drawn again until $S = V_1^2 + V_2^2$ is in $(0, 1)$, and let $W = \sqrt{-2 \ln S / S}$. Then WV_1 and WV_2 are independent standard normal variables.*

Proof. The accepted point (V_1, V_2) is uniform on the punctured unit disc; in polar coordinates $(\sqrt{S} \cos \Theta, \sqrt{S} \sin \Theta)$, the angle Θ is uniform on $[0, 2\pi)$, the area πS of the disc inside the point is uniform, i.e., S is uniform on $(0, 1)$, and the two are independent. The output is $(R \cos \Theta, R \sin \Theta)$ with $R = \sqrt{-2 \ln S}$, and $P(R > r) = P(S < e^{-r^2/2}) = e^{-r^2/2}$. Hence, R has the density $re^{-r^2/2}$, and the output has the density $\frac{1}{2\pi} e^{-(x^2+y^2)/2}$, the product of two standard normal densities. □

Thus, the two coordinates of `gauss2()` are independent, although the documentation of the function states the contrary, and `gauss()` returns the first of them. The case $S = 0$, which `gauss()` and `gauss2()` do not exclude, has probability 2^{-2b}, where b is the number of random bits of the `double` that `real()` returns, and would give a NaN.

Lemma 8.4. *`poisson(lambda)` returns the least $k \geq 0$ such that $U_1 \cdots U_{k+1} < e^{-\lambda}$, which is a Poisson variable of mean λ, after $k + 1$ draws; its expected time is therefore $O(1 + \lambda)$.*

Proof. With $E_i = -\ln U_i$, which are independent exponential variables of mean 1, the condition reads $E_1 + \cdots + E_{k+1} > \lambda$; hence, k is the number of the points of a Poisson process of unit rate that fall in $[0, \lambda]$, which is a Poisson variable of mean λ. The loop draws one number per point and one more. □

abLEMON. The packages of abLEMON declare, next to the Boost Software License, the BSD-3-Clause notice of the Mersenne Twister that the header carries, and `rnd` is exported when the library is built as a DLL on Windows.

8.7 Counting and timing

In `lemon/counter.h`, a few classes help to count events in a program under development, e.g., the operations of an algorithm. A `Counter` has a title and an output stream (`std::cerr` by default); it is changed by `++`, `--`, `+=` and `-=`, read by its conversion to `int` and reset by `reset(c)`, and it prints its title and value when it is destroyed. A `Counter::SubCounter`, constructed on a parent counter with a title and a stream, counts in the same way, prints its value when it is destroyed, and adds it to the parent at that moment. A `Counter::NoSubCounter`, instead, prints no report and passes each change to its parent at once; thus, replacing one with the other turns the report of a phase on or off without touching the code that counts. Subcounters can have subcounters, and `NoCounter` has the interface of `Counter` but counts nothing; one can thus compile the counting of a program away by changing one type.

Time is measured by `lemon/time_measure.h`. A `TimeStamp` holds five times, i.e., the real time, the user and system times of the process and those of its children, which `stamp()` reads from the operating system. Its constructor `TimeStamp(NULL)` makes a stamp of the present moment, and the default constructor one of zeros. Stamps can be added, subtracted and scaled, `ellapsed()` gives the time passed since the stamp, and `realTime()`, `userTime()`, `systemTime()`, `cUserTime()` and `cSystemTime()` read the five values (the last two are not computed on Windows). Its stream operator prints `u: ...s, s: ...s, cu: ...s, cs: ...s, real: ...s`; after `TimeStamp::format(TimeStamp::SHORT)`, a setting global to all stamps, it prints the real time and its relative difference from the total processor time.

A `Timer` starts at its construction (unless it is constructed with `false`), and is controlled by `start()`, `stop()`, `halt()`, `reset()` and `restart()`. It counts the calls of `start()` and stops only after as many calls of `stop()`, which allows one to time a recursive function, while `halt()` stops it at once; `running()` returns the number of calls of `stop()` still needed. Its times are read by the same five members as those of a stamp, or by its conversion to `TimeStamp`, which printing it uses. A `TimeReport` is a `Timer` that prints a title and its times when it is destroyed (unless `active(false)` has been called), and `NoTimeReport` is the version that measures no time. The granularity of the clocks is typically 10 ms (as the documentation warns), which makes a single measure of a short function rather unreliable. For a short function, `runningTimeTest(f, min_time, num, full_time)` calls `f` until the number of calls reaches a power of two (1, 2, 4, ..., at most 2^{31}) at which the total real time exceeds `min_time` seconds (10 by default), and returns the average time of one call; the number of calls and the total time are written into the two optional pointers.

8.8 Colors

The colours that `graphToEps()` uses are those of `lemon/color.h`. A `Color` holds three components in $[0, 1]$, read and set by `red()`, `green()` and `blue()`, and set together by `set(r, g, b)`. Fifteen constants are defined

in the library: WHITE, BLACK, RED, GREEN, BLUE, YELLOW, MAGENTA, CYAN, GREY, DARK_RED, DARK_GREEN, DARK_BLUE, DARK_YELLOW, DARK_MAGENTA and DARK_CYAN. A Palette is a map from int to Color that cycles through a list of colours. Its constructor takes have_white (white first, false by default) and num (the number of colours, −1 by default), and fills the list with the 26 colours of its default list (27 with white), cut to num colours or repeated up to them. Then operator[](i) and set(i, c) read and write the colour of i modulo size(), and add(c) and resize(s) change the list. Finally, distantColor(c) returns the corner of the colour cube farthest from c, and distantBW(c) returns black for a light colour and white for a dark one, by the luminance of c.

8.9 Errors, assertions and the command line

All exceptions of the library derive from Exception (lemon/error.h), a class derived from std::exception, whose what() returns "lemon::Exception". An IoError is thrown when a file cannot be read or written; it carries a message and a file name, set by its constructors or by message() and file() and read by the members of the same names. A FormatError is thrown by the readers of chapter 21 when the input is malformed, and carries a line number too (line(), zero if unknown). The what() of both composes the message, the file and the line, e.g., lemon:FormatError: unknown section (in file 'g.lgf' at line 12). Other headers derive further exceptions from Exception, e.g., the ArgParserException described below.

In lemon/assert.h, the macro LEMON_ASSERT(exp, msg) checks the preconditions of the library, and the user may use it as well. If exp is false, it calls a handler with the file, the line, the function, the message and the text of the expression; the default handler, assert_fail_abort(), prints them on std::cerr and calls std::abort(). Assertions are enabled by default, and disabled by LEMON_DISABLE_ASSERTS or by the standard macro NDEBUG. Moreover, LEMON_ASSERT_ABORT selects the default handler explicitly, and LEMON_ASSERT_CUSTOM the function named by LEMON_CUSTOM_ASSERT_HANDLER, which must have the signature of assert_fail_abort(). Defining both modes, or one of them together with NDEBUG or LEMON_DISABLE_ASSERTS, is a compilation error, and the mode may be changed within a translation unit by redefining the macros and including the header again. Finally, LEMON_DEBUG(exp, msg) checks the internal invariants of the algorithms; it works as LEMON_ASSERT, but it is disabled unless LEMON_ENABLE_DEBUG is defined, since some of its checks are expensive.

ArgParser (lemon/arg_parser.h) parses the command line of the tools of chapter 22, and of any program. It is constructed on argc and argv, and the options are declared without their leading -. Calls of intOption(), doubleOption(), boolOption() and stringOption() declare an option with a help string, a default value and a flag that makes it mandatory, and refOption() declares one whose value is written directly into a variable of the program. In the same way, other(name, help) documents the arguments that are not options, synonym(syn, opt) adds another name of an option, and optionGroup(group, opt) puts options into a group, for which onlyOneGroup(group) and mandatoryGroup(group) require at most one, or at least one, of its options. All these calls return the parser, and can therefore be chained.

Then parse(), or its synonym run(), reads the arguments. An argument that starts with - (and is not - alone) must be a declared option; a bool option is set to true by its presence, and any other option takes the next argument as its value, which must be read in full as a number for a numeric option. Other arguments are collected in files(). Afterwards, given(name) is true if an option appeared, and operator[](name) returns its value through the proxy RefType, which converts to int, double, bool or std::string after checking the type of the option with LEMON_ASSERT (an integer option also converts to double); commandName() returns argv[0]. Options -help, --help and -h print a summary of the options. On a request of help, an unknown option, a missing or malformed value, or a violated condition on the mandatory options and groups, the parser prints a message and calls exit(1). If throwOnProblems() has been called, it throws instead an ArgParserException, whose reason() is HELP, UNKNOWN_OPT or INVALID_OPT. Its records GroupData and OtherArg of the header hold the groups and the help of the other arguments.

The program manual/examples/08-utilities.cc (the test manual_08_utilities) uses an elevator, the two radix sorts, two generators seeded alike and a box, a counter that writes into a string, a timer, and a parser of a command line given as an array, and checks the result of each.

8.10 Exercises

Exercise 8.1. Show that, if an erasure from an empty end of a Path moved all the arcs of the other vector instead of half of them, a sequence of q calls made of $q/2$ calls of addBack() followed by alternate calls of eraseFront() and eraseBack() would take $\Theta(q^2)$ time, and explain which step of the proof of Theorem 8.1 fails. (The test test/path_test.cc checks the four path classes.)

Exercise 8.2. Write a function that reverses a path of a digraph into a path of the adaptor `ReverseDigraph` of chapter 5, using `ListPath::spliceFront()`, and check the result with `checkPath()`; compare it with a copy by `pathCopy()` and the iterator `ArcIt`.

Exercise 8.3. Compute the connected components of a graph with `UnionFindEnum`, print the items of each class with `ClassIt` and `ItemIt`, and compare the classes with those of `connectedComponents()` of chapter 18. (The example `examples/08-union-find.cc` and the test `test/unionfind_test.cc` use the same calls.)

Exercise 8.4. Measure with `runningTimeTest()` the time of `radixSort()`, `stableRadixSort()` and `std::sort` on 10^6 random keys drawn by `rnd.integer<int>()`, and then on keys drawn in $\{0, \ldots, 255\}$; explain the two rankings by the costs of §8.5. (The test `test/radix_sort_test.cc` checks the two sorts.)

Exercise 8.5. Draw 10^5 numbers by `gamma(2.5, 2.0)` and by `pareto(3.0, 1.0)`, and compare their sample means with the means $k\theta$ and $kx_{\min}/(k-1)$ of the two distributions. (The test `test/random_test.cc` checks the generator.)

Exercise 8.6. Use a `Counter` with two subcounters to count the comparisons and the swaps of an insertion sort, and verify that the parent includes the counts of a `SubCounter` only after it is destroyed, and those of a `NoSubCounter` at once. (The test `test/counter_test.cc` exercises the counters.)

9. Graph search

We are concerned in this chapter with the visit of the nodes of a digraph $G = (N, A)$, with $n = |N|$ and $m = |A|$, that can be reached along the arcs from a given set $S \subseteq N$ of sources. The breadth-first search of `lemon/bfs.h` visits them in the order of their distance from S, i.e., of the least number of arcs of a path from a source. Instead, the depth-first search of `lemon/dfs.h` goes as deep as it can along each path before it backtracks. Both take $O(n + m)$ time, and both are the building blocks of many algorithms of the following chapters (e.g., connectivity, matchings and flows). Finally, the maximum cardinality search of `lemon/max_cardinality_search.h` is a third order of visit, driven by a priority queue; it recognizes chordal graphs, and it underlies the minimum cut algorithm of Nagamochi and Ibaraki of chapter 15. All three classes follow the pattern of the algorithm classes of chapter 3 (traits, named template parameters, `run()`, `init()` and `start()`, queries), which we recall in §9.1 for the first two. On an undirected graph, whose arcs are the two directions of its edges, all that follows applies to the arcs.

9.1 The common interface

`Bfs` and `Dfs` have the same shape. Their template parameters are the digraph type `GR` (`ListDigraph` by default) and a traits class `TR`, by default `BfsDefaultTraits<GR>` and `DfsDefaultTraits<GR>`. These traits give the type `Digraph` and the types of four maps on the nodes, each with a static function `create...()` that allocates it: `PredMap`, the arc by which each node was reached (by default a `NodeMap<Arc>`), `DistMap`, its distance or depth (a `NodeMap<int>`), `ReachedMap` (a `NodeMap<bool>`) and `ProcessedMap`. By default, the last one is a `NullMap`, i.e., a map that discards what is written into it, since most programs do not need it; the named parameter `SetStandardProcessedMap` makes it a `NodeMap<bool>`. The classes also define `Traits`, `Path` (the type of the path dumper returned by `path()`) and `Create`.

Named parameters `SetPredMap<T>`, `SetDistMap<T>`, `SetReachedMap<T>` and `SetProcessedMap<T>` set the type of a map to `T`. With them, the algorithm does not allocate the map: one of type `T` must be given by the setter of the same name (`predMap()`, `distMap()`, `reachedMap()` or `processedMap()`) before `init()`, or an assertion fails. With the default types, a map that is not given is allocated at `init()` and deleted by the destructor. A map that is given, on the other hand, stays the property of the caller, and keeps the results after the algorithm object is gone.

In the simplest use, one calls `run(s)`, which visits the nodes reachable from `s`, and then reads the results by the queries. For more control, `init()` allocates the maps and marks all the nodes unreached, `addSource(s)` adds a source (several may be added to a `Bfs`), and one of the functions `start()` performs the search. One can also run the search in steps of one node (`processNextNode()` of `Bfs`) or of one arc (`processNextArc()` of `Dfs`), while `emptyQueue()`, `queueSize()` and `nextNode()` or `nextArc()` give what is left to do. A search may be stopped when a target node is reached (`start(t)` and `run(s, t)`, which then returns whether `t` was reached) or when a condition given by a map is met. Finally, `run()` visits all the nodes, starting a new search from each node that the previous ones have not reached.

Its queries are `reached(v)`, `dist(v)`, `predArc(v)` and `predNode(v)` (`INVALID` for a source and for a node not reached), `distMap()` and `predMap()`. Moreover, `path(t)` returns a path dumper (cf. §8.1) of the path from a source to `t` along the arcs of the predecessor map. We remark that `dist(v)` is undefined for a node not reached, since `init()` does not write the distances. Each class also has a function-type interface, `bfs()` and `dfs()` (§9.4), and a version with a visitor, `BfsVisit` and `DfsVisit`, which calls the members of an object of the user at each event of the search in place of filling maps (§9.5).

9.2 Breadth-first search

Let $d(v)$ be the distance of v from S, i.e., the least number of arcs of a path from a node of S to v, with $d(v) = \infty$ if there is no such path. A breadth-first search computes $d(v)$ for all the nodes, together with a shortest path forest. It keeps a queue Q of the nodes that are reached but not yet processed, and processes them in the order in which they entered it. `Bfs` stores Q in a vector of n places, allocated by `init()`, with the index h of its first node and the index t of its first free place; it does not read the distance of a node to compute those of its successors, but counts the levels instead, by remembering in ℓ the place at which the nodes of the next level begin and in k the distance given to the nodes reached in the current step (Algorithm 9.1).

Algorithm 9.1 (Breadth-first search, `Bfs`).

```
init():
1  for each node v: pred(v) := INVALID; reached(v) := false; processed(v) := false
2  h := t := 0; k := 1
addSource(s):
3  if not reached(s): reached(s) := true; pred(s) := INVALID; d(s) := 0
4                     Q[t] := s; t := t + 1; l := t
processNextNode():
5  if h = l: k := k + 1; l := t                [the next level begins at h]
6  u := Q[h]; h := h + 1; processed(u) := true
7  for each arc a = uv out of u:
8     if not reached(v): reached(v) := true; pred(v) := a; d(v) := k
9                        Q[t] := v; t := t + 1
10 return u
start():   while h < t: processNextNode()
```

Theorem 9.1. *After `init()`, the calls `addSource(s)` for $s \in S$ and `start()`, the nodes reached are those with $d(v) < \infty$, `dist(v)` equals $d(v)$ for each of them, and the arcs `predArc(v)` form a forest in which the path from a source to v has $d(v)$ arcs. The search takes $O(n + m)$ time.*

Proof. Let D_j be the set of the nodes at distance j. We show by induction on $j \geq 0$ that, while the nodes of D_j are processed, Q holds, in its places from 0 on, the nodes of $D_0, \ldots, D_j$ level by level, each with `dist` equal to its distance, followed by the nodes of D_{j+1} reached so far, and that $k = j + 1$ and ℓ is the place after the last node of D_j. For $j = 0$, the sources fill the places $0, \ldots, t - 1$, and `init()` and step 4 give $k = 1$ and $\ell = t$, while step 5 does not fire before $h = \ell$. While h runs over D_j, step 8 gives $k = j + 1$ to each node first reached from a node of D_j; such a node v has $d(v) \leq j + 1$, and $d(v) \geq j + 1$ since all the nodes at distance at most j were already reached. Conversely, each node of D_{j+1} has an in-arc from a node of D_j, and is reached when the first such node is processed. Hence, when h reaches ℓ, the places from ℓ to $t - 1$ hold exactly D_{j+1}, and step 5 sets $k = j + 2$ and $\ell = t$, which is the claim for $j + 1$. A node with $d(v) = \infty$ has no path from S, and is never reached. The arc `pred(v)` goes from a node at distance $d(v) - 1$, which gives the forest and the length of its paths. Each node enters Q at most once, by the test of step 8, and the arcs out of each node are scanned once, when it is processed; hence, the time is $O(n + m)$. □

The proof needs the sources to be added before the search starts. In particular, `run()` adds the source of each new search to the queue of the previous one, without resetting k; hence, in abLEMON the distances that it gives to the nodes of the second and later searches are offset by the number of levels of the searches before (a node at distance 1 from the source of the second search may get 3), while the reached map, the predecessors and the paths are right. One who needs the distances of all the components should run one search per component, by `init()`, `addSource()` and `start()`.

Otherwise, the interface is the one of §9.1, and a call of `processNextNode()` processes the first node of the queue and returns it (the queue must not be empty). Its form `processNextNode(target, reach)` also sets `reach` to `true` if `target` is reached in the step, and `processNextNode(nm, rnode)` sets `rnode`, if it was `INVALID`, to the first node reached in the step whose value in the `bool` node map `nm` is true. Accordingly, `start(t)` processes nodes until `t` is reached, i.e., until it enters the queue (its distance and path being then final, by Theorem 9.1), and `start(nm)` processes them until a node with `nm` true is reached, which it returns (or `INVALID`). Thus, `run(s, t)` is `init()`, `addSource(s)` and `start(t)`, and returns `reached(t)`; we remark that a target that is itself a source is never "reached in a step", and `start(t)` then runs the whole search. Finally, `nextNode()` returns the first node of the queue (or `INVALID`), `queueSize()` the number of nodes in it, and `emptyQueue()` whether there are none.

Figure 9.1 shows, on the left, the tree and the distances of a breadth-first search. On the right, the tree of a

depth-first search of the same digraph is the subject of the next section.

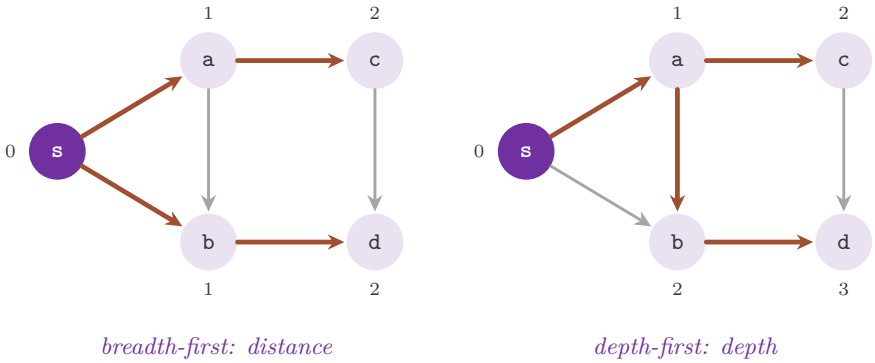


Figure 9.1. A breadth-first search (left) and a depth-first search (right) of the same digraph from s, the out-arcs of each node scanned from top to bottom: the arcs of the two trees, the distance of each node in the first and its depth in the tree in the second.

9.3 Depth-first search

The depth-first search advances from the last node reached along an arc not yet scanned, and backtracks to the node it came from when the node has no such arc left. `Dfs` implements it without recursion, by a stack K that holds, for each node of the current path from the source, the next out-arc to scan from it (Algorithm 9.2, where *top* is the index of the top of K, -1 for an empty stack). A node is processed, in the sense of the `ProcessedMap`, when the search backtracks from it. In `Dfs`, the value `dist(v)` is the depth of v in the tree of the search, i.e., the number of arcs of the path of the tree from the source, and in general it is not a distance in the digraph.

Algorithm 9.2 (Depth-first search with a stack of arcs, `Dfs`).

```
addSource(s):                                    [the stack must be empty]
1  if not reached(s): reached(s) := true; pred(s) := INVALID
2     if s has an out-arc: top := top + 1; K[top] := its first out-arc; d(s) := top
3     else: processed(s) := true; d(s) := 0
processNextArc():
4  a = uv := K[top]
5  if not reached(v): reached(v) := true; pred(v) := a
6                     top := top + 1; K[top] := the first out-arc of v (or INVALID)
7                     d(v) := top; x := v
8  else: K[top] := the next out-arc of u (or INVALID); x := u
9  while top >= 0 and K[top] = INVALID:
10    processed(x) := true; top := top - 1
11    if top >= 0: x := source(K[top]); K[top] := the next out-arc of x (or INVALID)
12 return a
start():   while top >= 0: processNextArc()
```

The properties for which the depth-first search is used (Tarjan 1972) follow from the invariant of the stack and from the fact that a node stays on the stack until all that it can reach is visited.

Lemma 9.1. *During the search from a source s, the sources of the arcs $K[0], \dots, K[top]$ (or, for an arc `INVALID` pushed at step 6, the node v of that step) are the nodes of the path of the tree from s to the node of the top, in this order; they are the nodes reached in this search and not processed, and the depth of the node of $K[i]$ is i.*

Proof. By induction on the steps: step 2 makes the path (s) with depth 0; step 6 extends it by the head v of the arc a out of the node of the top, which becomes the arc `pred(v)` of the tree, with depth *top*; steps 8 to 11 change only the arc of the top, or remove the top node, which is then marked processed. No other node is marked processed. □

Lemma 9.2. *Let v be reached when the set of the unreached nodes is U. Then each node w that is reachable from v by a path whose nodes after v are in U is reached while v is on the stack, i.e., before v is processed, and is a descendant of v in the tree.*

Proof. Let $v = x_0, x_1, \ldots, x_k = w$ be such a path, and suppose that some x_i is not reached while v is on the stack; let $i \geq 1$ be the least such index. Then x_{i-1} is reached while v is on the stack, and therefore, by Lemma 9.1, it is above v on the stack and processed before v. Before x_{i-1} is processed, all its out-arcs are scanned (steps 8 and 11 move to the next arc only after scanning one), the arc $x_{i-1}x_i$ among them; hence, x_i is reached before v is processed, and after v is reached, since it was in U, a contradiction. A node reached while v is on the stack is pushed above v, and its path of the tree passes through v by Lemma 9.1. □

Theorem 9.2. *(i) The search from a source s reaches exactly the nodes reachable from s through nodes not reached before it, and takes $O(1)$ time per node reached and per arc scanned, i.e., $O(n+m)$ for* ***`run()`****. (ii) An arc uv is scanned while v is on the stack only if G has a directed cycle through uv; conversely, if G has a directed cycle,* ***`run()`*** *scans such an arc. (iii) If G is acyclic, the reverse of the order in which* ***`run()`*** *processes the nodes is a topological order, i.e., the source of each arc precedes its target.*

Proof. (i) follows from Lemma 9.2 applied to s, and from the fact that each step either pushes a node, which happens once per node, or advances the arc of the top, which happens once per arc. For (ii), if v is on the stack when uv is scanned, then u is the top node, a descendant of v by Lemma 9.1, and the path of the tree from v to u and the arc uv form a cycle. Conversely, let C be a directed cycle and v its first node reached, and let uv be the arc of C entering v. By Lemma 9.2, u is reached while v is on the stack, and the arc uv is scanned before u is processed, i.e., while v is still on the stack. For (iii), let uv be an arc. It is scanned when u is the top node, before u is processed. If v is not reached at that moment, it is pushed above u and processed before u; if it is reached and not processed, it is on the stack (Lemma 9.1), and G has a cycle by (ii); otherwise it is already processed. Hence, v is processed before u, and u precedes v in the reverse order. □

The stack is a vector of n places, allocated by `init()`. Otherwise, the interface follows §9.1, with three differences. First, the unit of work is an arc: `processNextArc()` scans the arc on the top of the stack and returns it, and `nextArc()` returns the next arc to be scanned, as an `OutArcIt` (`INVALID` if the stack is empty), while `queueSize()` is the height of the stack. Second, `addSource()` requires an empty stack (checked by `LEMON_DEBUG`), since a source added to a search in progress would break the invariant of Lemma 9.1; this is why `run()` adds each new source only after the previous search has finished, and why the depths of `run()` are right in every search. Third, the stopping condition of `start(am)` is an arc map, not a node map: the search stops before scanning an arc a with `am[a]` true and returns it, or returns `INVALID` if the search ends first. Also, `start(t)` and `run(s, t)` stop as soon as `t` is reached (at once, if `t` is the source), and `run(s, t)` returns `reached(t)`.

9.4 Function-type interfaces

The functions `bfs(g)` and `dfs(g)` return a `BfsWizard` and a `DfsWizard` on the digraph `g`. Their members set the maps of the algorithm and the outputs, each returning a new wizard, and their `run()` runs a `Bfs` or a `Dfs` with them. Named function parameters `predMap(m)`, `distMap(m)`, `reachedMap(m)` and `processedMap(m)` pass the maps that the algorithm fills. Their types are deduced from the arguments, so that any writable map may be passed, e.g., a `LoggerBoolMap` (cf. chapter 6) that records the nodes in the order of processing. For a search between two nodes, `path(p)` gives a path into which the path to the target is copied, and `dist(d)` an `int` variable that receives its distance or depth (which, by the remark after Theorem 9.1, is undefined when the target is not reached).

Three runs are available: `run(s)`, `run(s, t)`, which returns whether `t` was reached and fills the path and the distance, and `run()`, which visits all the nodes (as `run()` of the classes, with the same distances). Maps not given are allocated by the algorithm for the duration of the run, and the path given to `path()` may be of any path type that a path dumper can be assigned to (by default a `Path`). The classes `BfsWizardBase` and `DfsWizardBase` hold the pointers to the parameters, and the traits `BfsWizardDefaultTraits` and `DfsWizardDefaultTraits` their default types; none of them is used directly. Two typical calls are the following.

```
bool found = bfs(g).path(p).dist(d).run(s, t);
dfs(g).processedMap(loggerBoolMap(std::back_inserter(order))).run();
```

9.5 Visitors

`BfsVisit<GR, VS, TR>` and `DfsVisit<GR, VS, TR>` perform the same searches as `Bfs` and `Dfs`. However, in place of filling a predecessor and a distance map, they call a member of a visitor object of type `VS` at each event of the search; the visitor is given to the constructor together with the digraph. They keep only a `ReachedMap` (a `NodeMap<bool>` by default, given by the traits `BfsVisitDefaultTraits` and `DfsVisitDefaultTraits`, and

set by `reachedMap()` or by the named parameter `SetReachedMap<T>`). Their execution control is that of `Bfs` and `Dfs` (`init()`, `addSource()`, `processNextNode()` or `processNextArc()`, `nextNode()` or `nextArc()`, `emptyQueue()`, `queueSize()`, the three `start()` and the three `run()`), and their only query is `reached(v)`. In the library, the visitors `BfsVisitor<GR>` and `DfsVisitor<GR>` define the events as members with an empty body, and a concept check of the members that a visitor must have. A visitor of the user may derive from them, and redefine only the members it needs.

`BfsVisit` calls `start(s)` for each source and `reach(v)` when a node is reached for the first time (a source included). It calls `process(u)` when a node is taken from the queue, `discover(a)` when an arc reaches a node not yet reached (followed by `reach()` of its target), and `examine(a)` when an arc is scanned whose target was already reached. `DfsVisit` calls `start(s)` and `stop(s)` when the search from the source `s` begins and ends, and `reach(v)`, `discover(a)` and `examine(a)` as `BfsVisit` does. It also calls `leave(v)` when the search backtracks from a node (the moment at which `Dfs` marks it processed), and `backtrack(a)` when the search returns along the arc `a` of the tree to its source. Thus, `reach()` and `leave()` delimit the time during which a node is on the stack, and an `examine(a)` whose target is on the stack reveals a directed cycle by Theorem 9.2 (ii). Note that `DfsVisit::nextArc()` returns an `Arc`, while `Dfs::nextArc()` returns an `OutArcIt`.

The program `manual/examples/09-search.cc` (the test `manual_09_search`) computes the distances of a breadth-first search, a shortest path by `bfs()` and a topological order by the order of processing of `dfs()` (Theorem 9.2 (iii)), and finds a directed cycle by a visitor of `DfsVisit` that keeps the nodes on the stack between `reach()` and `leave()`.

9.6 Maximum cardinality search

The maximum cardinality search orders the nodes of a graph so that each node, when it is chosen, has the largest number of neighbours among those already chosen. In abLEMON, `MaxCardinalitySearch<GR, CAP, TR>` extends it to capacities $c(a)$ on the arcs. The cardinality of an unprocessed node u is

$$\kappa(u) \;=\; \sum_{uv \in A,\ v \text{ processed}} c(uv), \tag{9.1}$$

and at each step the algorithm processes an unprocessed node of largest cardinality among those with at least one arc into the processed nodes (Algorithm 9.3), recording its cardinality at that moment. On a digraph the sum (9.1) is over the arcs from u to the processed nodes, since the algorithm scans the in-arcs of the node it processes. On an undirected graph it is over the edges that join u to processed nodes; with unit capacities, $\kappa(u)$ is then the number of processed neighbours of u. When the heap becomes empty, i.e., when no unprocessed node has an arc into a processed one, `run()` takes a new source among the nodes not reached.

Algorithm 9.3 (Maximum cardinality search, `MaxCardinalitySearch`).

```
init():            H := empty heap of maximum priority; no node processed
addSource(s, p):   if s was never in H: push s with priority p (0 by default)
processNextNode():
1  u := the node of largest priority in H; cardinality(u) := its priority
2  processed(u) := true; pop u
3  for each arc wu entering u:
4     if w was never in H: push w with priority c(wu)
5     else if w is in H: increase the priority of w by c(wu)
6  return u
start():           while H is not empty: processNextNode()
run():             init(); for each node s not reached: addSource(s); start()
```

Theorem 9.3. *In a search from a source added with priority 0, the priority of each node w in H is $\kappa(w)$ of (9.1), so that each node, when processed, has the largest cardinality among the unprocessed nodes that have an arc into the processed ones, and its `cardinality()` is its cardinality at that moment. The search takes $O(n+m)$ time with unit capacities and the default `BucketHeap`, and $O(m \log n)$ with the `BinHeap` of the other capacities.*

Proof. A node w enters H when the first node u with an arc wu is processed, with the priority $c(wu)$, and its priority grows by $c(wu')$ whenever another such node u' is processed (a parallel arc counting once for each copy); hence, it is $\kappa(w)$ at each moment, and the heap gives its maximum. Each node is pushed and popped once, and each arc causes at most one increase, when its head is processed. With a `BinHeap`, each operation takes $O(\log n)$. With unit capacities, the priorities are integers between 0 and m, and a `BucketHeap` of maximum

priority does each push and increase in $O(1)$, while the pointer to the largest bucket, which moves down over the empty buckets at the pops, has moved up at most once per increase; hence, the total is $O(n+m)$ (§7.4). □

With unit capacities, the order of the search recognizes the chordal graphs, i.e., those in which every cycle of four or more nodes has a chord. An order $v_1, \ldots, v_n$ of the nodes is a *perfect elimination order* if, for each i, the neighbours of v_i that follow it form a clique.

Theorem 9.4. *Let G be an undirected graph, and let $v_1, \ldots, v_n$ be the order in which* `run()` *with unit capacities processes its nodes. Then G is chordal if and only if, for each i, the neighbours of v_i among $v_1, \ldots, v_{i-1}$ form a clique, i.e., if and only if $v_n, \ldots, v_1$ is a perfect elimination order.*

Proof sketch. If the condition holds, let C be a chordless cycle of at least four nodes, and v_i its last node in the order: its two neighbours on C come before it and must be adjacent, which is a chord. Conversely, the key property of the order is that, if a precedes b, b precedes c, $ac \in E$ and $ab \notin E$, then some d before b has $db \in E$ and $dc \notin E$: indeed, when b was chosen, its count of processed neighbours was at least that of c, which includes a. Starting from a violation of the condition, i.e., two nonadjacent earlier neighbours of some v_i, and applying this property repeatedly to the extreme pair, one builds longer and longer chordless paths whose ends precede all their inner nodes, which in a chordal graph cannot go on, since a finite graph has no infinite such sequence. The complete argument is in Tarjan and Yannakakis 1984, section 2. □

Thus, chordality can be tested in linear time, by checking the condition of Theorem 9.4 on the order as in Tarjan and Yannakakis 1984 (the program of this section implements a simpler and slower version of the check, pair by pair). With general capacities, the order is a maximum adjacency order, which the algorithm of Nagamochi and Ibaraki (§15.3) needs.

Its template parameters are the graph type `GR`, the type `CAP` of the map of the capacities, by default `ConstMap< GR::Arc, Const<int, 1>>` (unit capacities), and the traits class `TR`, by default `MaxCardinalitySearchDefaultTraits<GR, CAP>`. Its traits give `Digraph`, the `CapacityMap` and its `Value`, the `CardinalityMap` (a `NodeMap` of `Value`), the `ProcessedMap` (a `NullMap` by default), the `HeapCrossRef` (a `NodeMap<int>`) and the `Heap`, each with its function `create...()`. The heap is a `BucketHeap` of maximum priority for the unit capacities, and a `BinHeap` with `std::greater`, i.e., again of maximum priority, otherwise; the internal templates `CapMapSelector`, `HeapSelector` and `Selector` of the traits make this choice and create the map of unit capacities. Named parameters `SetCapacityMap<T>`, `SetCardinalityMap<T>`, `SetProcessedMap<T>` and `SetHeap<H, CR>` change these types and require the map, or the heap, to be given by the corresponding setter; `SetStandardHeap<H, CR>` changes the type of the heap, which the algorithm then allocates with its cross reference map.

Its constructor takes the graph and the capacity map, or only the graph, in which case the traits create the map, i.e., the map of unit capacities by default, or a map of type `CAP` constructed on the graph (with the default values of its constructor) otherwise. Its setters are `capacityMap(m)`, `cardinalityMap(m)`, `processedMap(m)` and `heap(hp, cr)`, and the readers `capacityMap()`, `processedMap()`, `heap()`, `heapCrossRef()` and `cardinalityMap()` return the maps and the heap in use. Its execution control is made of `init()`, `addSource(s, capacity)`, `processNextNode()`, `nextNode()`, `emptyQueue()` and `emptySize()`, which, despite its name, is the number of nodes in the heap. Searches are run by `start()`, `run(s)`, `run()`, and `start(dest)` and `start(nm)`; the last two stop when `dest`, or a node with `nm` true, is on the top of the heap, and write its cardinality (and `true` into the `ProcessedMap`) without removing it from the heap, so that `processed()` stays false for it.

Its queries are `cardinality(v)`, i.e., the cardinality of v when it was processed (undefined for a node never processed), `currentCardinality(v)`, its priority in the heap (for a node reached but not processed), and `cardinalityMap()`. Also, `reached(v)` is true if v has ever been in the heap, and `processed(v)` if it has left it.

The program `manual/examples/09-max-cardinality.cc` (the test `manual_09_max_cardinality`) runs the search step by step on a chordal graph and on a chordless cycle of four nodes, records the order, and checks the condition of Theorem 9.4, which holds on the first graph and fails on the second, together with the cardinalities given by the algorithm.

9.7 Exercises

Exercise 9.1. Run `Bfs` from several sources added with `addSource()` and show that `dist(v)` is the distance from the nearest source; compare with a single search from a new node joined by an arc to each source. (The test `test/bfs_test.cc` exercises the interface of `Bfs`.)

Exercise 9.2. Use `Bfs` on an undirected graph to test whether it is bipartite, by checking that no edge joins two nodes at the same distance from the source of their component; compare the answer with `bipartite()` of

chapter 18, and explain why the distances must be taken from one search per component (cf. the remark after Theorem 9.1).

Exercise 9.3. Write a visitor of `DfsVisit` that numbers the nodes in preorder and in postorder, and use the two numbers to classify the arcs into tree, back, forward and cross arcs; check that a digraph is acyclic if and only if there is no back arc. (The program `manual/examples/09-search.cc` detects the back arcs; the test `test/dfs_test.cc` exercises `Dfs` and `DfsVisit`.)

Exercise 9.4. Stop a `Dfs` by `start(am)` at the first arc whose target is on the current path, with an arc map computed on the fly from the `ReachedMap` and the `ProcessedMap`; explain why a node map would not suffice for this, which is why `start()` of `Dfs` takes an arc map.

Exercise 9.5. Run `MaxCardinalitySearch` with the capacities of the edges of a graph, and verify that the cardinality of the last node processed, t, equals the capacity of a minimum cut between t and the node s processed just before it (the lemma behind the algorithm of Nagamochi and Ibaraki), by comparing it with `Preflow` of chapter 12. (The test `test/max_cardinality_search_test.cc` checks the cardinalities on a small digraph.)

10. Shortest paths

We are concerned in this chapter with the shortest paths of a digraph $G = (N, A)$, with $n = |N|$ nodes and $m = |A|$ arcs, whose arcs have lengths $\ell : A \to \mathbb{R}$. The library has five classes for them. `Dijkstra` (`lemon/dijkstra.h`) computes the shortest paths from a set of sources when the lengths are nonnegative, and `BidirDijkstra` and `AStar` (`lemon/bidir_dijkstra.h`, `lemon/astar.h`) compute a shortest path between two nodes, usually scanning much less of the digraph. `BellmanFord` (`lemon/bellman_ford.h`) accepts negative lengths and detects the negative cycles, and `Suurballe` (`lemon/suurballe.h`) finds k arc-disjoint paths of least total length between two nodes. We first state the problem and the optimality conditions that all of them rely on (§10.1), and then describe the classes in this order.

10.1 The problem and its optimality conditions

As in §8.1, a path is a sequence of arcs $P = (a_1, \dots, a_k)$, each starting where the previous one ends (so that a node may repeat), and we call it *simple* if no node repeats. Its length is $\ell(P) = \sum_{i=1}^{k} \ell(a_i)$, and, given a source s, the distance of a node v is

$$d(v) \;=\; \inf\{\, \ell(P) \;:\; P \text{ is a path from } s \text{ to } v \,\}, \tag{10.1}$$

with $d(v) = +\infty$ if v cannot be reached from s. If no cycle of negative length can be reached from s, removing the cycles of a path does not make it longer, and the infimum is the minimum over the (finitely many) simple paths; otherwise, $d(v) = -\infty$ for each node v reachable from such a cycle. With several sources S, each with an initial distance $d_0(s)$, the distance of v is the least value of $d_0(s) + \ell(P)$ over the sources s and the paths P from s to v, i.e., the distance (10.1) from a new source joined to each $s \in S$ by an arc of length $d_0(s)$. All the classes of this chapter maintain tentative distances d and a predecessor arc for each node, and stop when the following conditions hold.

Theorem 10.1. *Let d be finite on the nodes reachable from s, with $d(s) = 0$, and let $d(v)$ be the length of some path from s to v for each of them. Then d is the distance (10.1) if and only if*

$$d(v) \;\le\; d(u) + \ell(uv) \qquad \text{for each arc } uv \text{ with } u \text{ reachable.} \tag{10.2}$$

If, moreover, each node $v \neq s$ has a predecessor arc uv with $d(v) = d(u) + \ell(uv)$, and following the predecessor arcs back from each node leads to s, these arcs form a tree of shortest paths.

Proof. If (10.2) holds, summing it along any path P from s to v gives $d(v) \le d(s) + \ell(P) = \ell(P)$; since $d(v)$ is itself the length of a path, it is the minimum (10.1). Conversely, if d is the distance and is finite, appending the arc uv to a path of length $d(u)$ gives a path to v of length $d(u) + \ell(uv)$, which is at least $d(v)$. For the tree, the path of predecessor arcs from s to v has length $d(v)$, by summing the equalities along it, and is therefore a shortest path. Since it reaches s from each node, the arcs have no cycle and form a tree. □

Equivalently, (10.2) means that the *reduced lengths*

$$\ell_\pi(uv) \;=\; \ell(uv) + \pi(u) - \pi(v) \tag{10.3}$$

are nonnegative for the potential $\pi = d$. Since $\ell_\pi(P) = \ell(P) + \pi(u) - \pi(v)$ for any path P from u to v, a change of the lengths into reduced lengths changes the length of all the paths between two given nodes by the same amount, and keeps the shortest ones. We shall use this fact for A* (§10.3) and for Suurballe's algorithm (§10.5). Also, the shortest path problem is the minimum cost flow problem of chapter 13 in which one unit of flow is sent from s to each other node, and d is an optimal potential of its dual.

Table 10.1. The classes of this chapter, with the lengths they accept and the cost of a run on a digraph of n nodes and m arcs (k is the number of paths of `Suurballe`).

class	lengths	answers	time with the default heap
`Dijkstra`	≥ 0	all the distances from a set of sources	$O((n+m)\log n)$
`BidirDijkstra`	≥ 0	one path between two nodes	$O((n+m)\log n)$, usually much less
`AStar`	≥ 0	one path between two nodes, with a heuristic	$O((n+m)\log n)$ with a consistent heuristic
`BellmanFord`	any	all the distances, or a negative cycle	$O(nm)$
`Suurballe`	≥ 0	k arc-disjoint paths between two nodes	$O(k\,(n+m)\log n)$

10.2 Dijkstra's algorithm

In the algorithm of Dijkstra 1959, the reached nodes that are not yet processed are kept in a heap H, keyed by their tentative distance. At each step it removes a node of least tentative distance, declares its distance final, and relaxes its out-arcs, i.e., it decreases the tentative distance of each target v of an arc uv to $d(u)+\ell(uv)$ if this is smaller (Algorithm 10.1). In the class, the sum and the comparison are the members `plus()` and `less()` of an object of *operation traits*, and the initial distance of a source is `zero()` unless given; we write them $\oplus$ and $\prec$, and $\preceq$ for "not $\succ$".

Algorithm 10.1 (Dijkstra's algorithm, `Dijkstra`).

```
init():
1  for each node v: pred(v) := INVALID; processed(v) := false; v has never been in H
addSource(s, d0):
2  if s has never been in H: push s on H with priority d0
3  else if s is in H and d0 < priority(s): decrease it to d0; pred(s) := INVALID
processNextNode():
4  u := the node of least priority in H; d(u) := its priority; pop u; processed(u) := true
5  for each arc a = uv out of u:
6     if v was never in H: push v with priority d(u) + l(a); pred(v) := a
7     else if v is in H and d(u) + l(a) < priority(v):
8        decrease the priority of v to d(u) + l(a); pred(v) := a
9  return u
start():   while H is not empty: processNextNode()
```

In the theorem below, the value of a path $P=(a_1,\dots,a_k)$ from a source s is $d_0(s)\oplus\ell(a_1)\oplus\cdots\oplus\ell(a_k)$, computed from the left. The distance $d(v)$ is the least value of a path from a source to v; with the default operations it is the distance of §10.1.

Theorem 10.2. *Suppose that $\preceq$ is a total preorder (that of the heap too), and that, for each arc a and all the values $x\preceq y$, one has $x\preceq x\oplus\ell(a)$ and $x\oplus\ell(a)\preceq y\oplus\ell(a)$; with the default operations, this means that the lengths are nonnegative. Then each node processed by Algorithm 10.1 has its distance from the sources as its priority when it leaves the heap, and the predecessor arcs of the processed nodes form a forest of shortest paths. The algorithm performs at most n pushes and pops and at most m decreases of a priority (plus one per call of `addSource()`).*

Proof. Each priority is the value of a path (by steps 2, 6 and 8), and therefore not smaller than the distance of its node. By the two hypotheses, removing a cycle from a path does not increase its value, so that each distance is the value of a simple path. We show by induction on the order of processing that the priority $p(u)$ of the node u processed in step 4 is at most $d(u)$. Let P be a simple path of value $d(u)$ from a source s, and let y be its first node that is not processed before u (it exists, since u is not). If $y=s$, its priority is at most $d_0(s)$, by steps 2 and 3. If not, its predecessor x on P was processed with $p(x)=d(x)$, which is at most the value of the part of P up to x. When x was processed, steps 6 to 8 gave y a priority at most $p(x)\oplus\ell(xy)$, i.e., by the second hypothesis, at most the value of the part of P up to y. By the first hypothesis, the value does not decrease along the rest of P; hence, $p(y)\preceq d(u)$, and $p(u)\preceq p(y)$ since u leaves the heap first (or $u=y$). The predecessor of a processed node v is the arc xv that gave v its final priority, from a node x processed before v, with $d(v)=d(x)\oplus\ell(xv)$; hence, the path of predecessor arcs reaches a source, and its value is $d(v)$. Finally, each node enters the heap once, and each arc is relaxed once, from its source. □

The time therefore depends on the heap (cf. chapter 7). With the `BinHeap` of the default traits, all the operations take $O(\log n)$ and the search takes $O((n+m)\log n)$, while a `FibHeap` gives $O(m + n\log n)$ by its decreases in amortized $O(1)$ (Fredman and Tarjan 1987). Note that the Fibonacci bound is better only in theory for most digraphs of practical size, the binary heap (or the 4-ary `QuadHeap`) being usually faster; with small integer lengths, a `RadixHeap` or a `BucketHeap` is perhaps worth a try. For instance, $\oplus$ defined as the maximum (with nonnegative lengths) satisfies the hypotheses of Theorem 10.2, and gives the paths of least bottleneck, i.e., the paths whose longest arc is as short as possible. Note that the heap still orders the priorities by the comparison of its own type (`std::less` in the default traits), which must agree with `less()`.

Template parameters and traits. `Dijkstra<GR, LEN, TR>` takes the digraph type `GR` (`ListDigraph` by default), the type `LEN` of the map of the lengths (by default `GR::ArcMap<int>`), whose `Value` is the type of the distances, and the traits class `TR`, by default `DijkstraDefaultTraits<GR, LEN>`. These traits give `Digraph`, `LengthMap`, `Value`, `OperationTraits`, i.e., the operations on the distances (by default `DijkstraDefaultOperationTraits<Value>`, with the static `zero()`, `plus()` and `less()` that are the zero, the sum and the comparison of the type), and `HeapCrossRef` and `Heap`, i.e., a `NodeMap<int>` and a `BinHeap` with `std::less`. They also give the three maps `PredMap` (a `NodeMap<Arc>`), `DistMap` (a `NodeMap` of the distances) and `ProcessedMap` (a `NullMap`), each with a function `create...()`. Named parameters `SetPredMap<T>`, `SetDistMap<T>`, `SetProcessedMap<T>` and `SetStandardProcessedMap` change the maps as in §9.1. Moreover, `SetHeap<H, CR>` sets the types of the heap and of its cross reference map, both of which must then be given by `heap()`; `SetStandardHeap<H, CR>` sets them for a heap that the algorithm can allocate (one whose constructor takes the cross reference map, which takes the digraph). Finally, `SetOperationTraits<T>` replaces the operations. The class also defines `Traits`, `Path` (the type of the path dumper of `path()`) and `Create`.

> **abLEMON.** `Dijkstra` (and `BellmanFord`, §10.4) keep an object of their operation traits class, default constructed or given by `operationTraits(ops)`, and call its members `plus()` and `less()`; hence, the operations may depend on the instance, e.g., on a tolerance stored in the object. In LEMON 1.3.1 they were static functions of the traits class.

Interface. Its constructor takes the digraph and the length map, and its setters are `lengthMap(m)`, `operationTraits(ops)` (with the reader `operationTraits()`), `predMap(m)`, `distMap(m)`, `processedMap(m)` and `heap(hp, cr)`; the maps and the heap not given are allocated by `init()` and deleted by the destructor. Its execution control is made of `init()`, `addSource(s, dst)`, `processNextNode()`, `nextNode()`, `emptyQueue()` and `queueSize()`, the three forms of `start()`, and `run(s)` and `run(s, t)`. A call `addSource(s, dst)` adds a source with the initial distance `dst` (`zero()` by default), by steps 2 and 3 of Algorithm 10.1. Hence, a source added twice keeps the smaller of its two distances, and a node already processed is left as it is.

> **abLEMON.** In LEMON 1.3.1, `addSource(s, dst)` raised the priority of a source already in the heap with a smaller one to `dst`, and pushed again a node already processed; the test `dijkstra_test` adds sources twice.

Then `start()` processes the nodes until the heap is empty, while `start(t)` processes them until `t` is on the top of the heap, then writes the distance of `t`, marks it processed and pops it without relaxing its arcs. Also, `run(s, t)` does `init()`, `addSource(s)` and `start(t)`, and returns whether `t` was processed. Similarly, `start(nm)` stops when a node with `nm` true is on the top of the heap, writes its distance and marks it processed (in the `ProcessedMap`, while it stays in the heap), and returns it, or returns `INVALID` if the heap empties first. By Theorem 10.2, the distance of the node at which these searches stop is final.

Its queries are `dist(v)`, `predArc(v)`, `predNode(v)`, `path(t)` (a path dumper, cf. §8.1), `distMap()`, `predMap()`, `reached(v)` (whether v has been in the heap) and `processed(v)` (whether it has left it). The distance is written into the `DistMap` only when a node is processed; hence, `dist(v)` is undefined for a node that is reached but not processed, and for such a node, `currentDist(v)` returns its priority in the heap, i.e., an upper bound on its distance that may still decrease.

Function-type interface. Finally, `dijkstra(g, length)` returns a `DijkstraWizard`, whose named function parameters are `predMap(m)`, `distMap(m)`, `processedMap(m)`, `path(p)` and `dist(d)`, and whose runs are `run(s)` and `run(s, t)`. The latter returns whether `t` is reached and fills the path and the distance, as in §9.4; the base class `DijkstraWizardBase` holds the parameters, and `DijkstraWizardDefaultTraits` their default types.

The program `manual/examples/10-dijkstra.cc` (the test `manual_10_dijkstra`) runs Dijkstra's algorithm on a grid and checks the distances against (10.2). Then it uses two sources, the function-type interface and the operations of the least bottleneck, and compares the results with those of the two point-to-point searches of the next section.

10.3 Shortest paths between two nodes

When only the path from a source s to a target t is wanted, `Dijkstra::run(s, t)` stops when t is processed, but it still processes all the nodes closer to s than t. Both classes of this section usually scan far fewer nodes. Both have nonnegative lengths, and both answer any number of queries with one object, each by a call `run(s , t)` that returns whether t is reachable from s. Both give the result by `dist()` and `path()` (a `lemon::Path`, empty when $s = t$), with the sums and the comparisons of the type of the lengths. Their node maps are allocated once, by the constructor; each run resets only the nodes that the previous run touched, and its time is therefore proportional to the part of the digraph that it scans (times the logarithm of the size of the heap). Their heap is a `BinHeap` by default. The named parameter `SetHeap<H, CR>` replaces it by a heap of type `H` with a cross reference map of type `CR` (a `NodeMap<int>` by default), as in `SetHeap<FibHeap<int, IntNodeMap>>::Create`.

abLEMON. `BidirDijkstra` and `AStar` are new in abLEMON (from trac #249 and #376 of LEMON), and take the type of their heap from a traits class (`BidirDijkstraDefaultTraits`, `AStarDefaultTraits`) through `SetHeap`.

Bidirectional search. `BidirDijkstra<GR, LEN, TR>` runs two searches of Dijkstra (Pohl 1971), a forward one from s along the arcs and a backward one from t against them, with tentative distances d_f and d_b and heaps H_f and H_b. At each step it advances the search whose least priority is smaller (the forward one on a tie). Each arc uv scanned by one search whose other end has been seen by the other search gives a path from s to t, of length $d_f(u) + \ell(uv) + d_b(v)$, and the algorithm keeps the shortest one found, of length μ, with the arc *meet* that gave it. Writing $\min H$ for the least priority in H ($+\infty$ for an empty heap), it stops as soon as

$$\min H_f \;+\; \min H_b \;\geq\; \mu \tag{10.4}$$

(Algorithm 10.2, in which a node is *seen* by a search once it has entered its heap, and *done* once it has left it).

Algorithm 10.2 (Bidirectional Dijkstra, `BidirDijkstra::run(s, t)`).

```
1  if s = t: return true with the empty path
2  push s on Hf and t on Hb with priority 0; found := false
3  while Hf and Hb are not empty:
4     if found and min(Hf) + min(Hb) >= mu: stop
5     X := the forward search if min(Hf) <= min(Hb), else the backward one; Y := the other
6     u := pop the top of X
7     for each arc a of u (out-arcs if X is forward, in-arcs if backward), w its other end:
8        relax w in X with d_X(u) + l(a), as in Algorithm 10.1 (unless w is done in X)
9        if w has been seen by Y and (not found or d_X(u) + l(a) + d_Y(w) < mu):
10          found := true; mu := d_X(u) + l(a) + d_Y(w); meet := a
11 if not found: return false
12 path := the forward tree to the tail of meet, meet, the backward tree from its head
```

Theorem 10.3. *With nonnegative lengths, `BidirDijkstra::run(s, t)` returns `false` if and only if t is not reachable from s; otherwise, μ is the distance $\delta(s,t)$ from s to t, and the path built at step 12 has length μ. Each node is pushed and popped at most once by each search, and each arc relaxed at most once by each.*

Proof. Each search is a search of Dijkstra, interrupted at times. Hence, by Theorem 10.2, a node done by the forward search has d_f equal to its distance from s, and every node x with $\delta(s,x) < \min H_f$ is done by it (and similarly for the backward search and the distances to t). Each value of μ is the length of a path from s to t, and therefore $\mu \geq \delta(s,t)$ when it is defined. Suppose that t is reachable from s, and let P be a shortest path from s to t; at the end, either (10.4) holds or a heap is empty, and in both cases $\min H_f + \min H_b \geq \mu$, with $\mu = +\infty$ if nothing was found. The source s is done by the forward search, which takes the first step. If t is done by the forward search, the relaxation that gave it its final value $d_f(t) = \delta(s,t)$ found t seen by the backward search, with $d_b(t) = 0$, and set $\mu \leq \delta(s,t)$ at step 10. Otherwise, let u be the last node of P done by the forward search and v the next one. Since v is not done, $\delta(s,v) \geq \min H_f$, and hence $\delta(v,t) = \ell(P) - \delta(s,v) \leq \ell(P) - \min H_f$, which is smaller than $\min H_b$ if $\ell(P) < \mu$; in that case v is done by the backward search. When the later of u (forward) and v (backward) was processed, the arc uv was scanned with its other end done by the other search, which gave $\mu \leq \delta(s,u) + \ell(uv) + \delta(v,t) = \ell(P)$, a contradiction. Hence, $\mu \leq \ell(P)$ in all cases, a path is found, and $\mu = \delta(s,t)$. The path of step 12 is made of the arcs `pred` of the two searches, whose lengths add up to at most μ (the tentative distances can only have decreased since μ was recorded), and is a path from s to t; hence, its length is μ. The bounds follow from step 8, which ignores the nodes done. □

In practice, the two searches meet roughly halfway. On a digraph in which the number of nodes within distance r grows quickly with r (as in a road network, roughly as r^2), they scan far fewer nodes than one search from s.

Its template parameters are `GR`, `LEN` (by default `GR::ArcMap<int>`) and `TR` (by default `BidirDijkstraDefaultTraits<GR, LEN>`, which gives `HeapCrossRef` and `Heap`), and the backward search needs the in-arcs of the digraph. The class defines `Digraph`, `LengthMap`, `Value`, `Path`, `HeapCrossRef` and `Heap`, and its private record `Search` holds the distances, the predecessors, the cross reference map and the heap of one search.

Search guided by a heuristic. `AStar<GR, LEN, HM, TR>` runs a search of Dijkstra from s in which the priority of a node is its tentative distance $g(v)$ plus a heuristic value $h(v)$, an estimate of its distance $\delta(v,t)$ to t, given by a node map of type `HM` with the value type of the lengths (Hart, Nilsson and Raphael 1968). The search stops when t is removed from the heap (Algorithm 10.3). We call the heuristic admissible if

$$h(v) - h(t) \;\le\; \delta(v,t) \qquad \text{for each node } v, \tag{10.5}$$

which, for the usual heuristics with $h(t) = 0$, means that h never exceeds the distance to t, and consistent if

$$h(u) \;\le\; \ell(uv) + h(v) \qquad \text{for each arc } uv. \tag{10.6}$$

Only the differences of the values of h matter, since a constant added to h adds the same constant to all the priorities. A consistent heuristic is admissible (sum (10.6) along a shortest path from v to t), and with it the reduced lengths $\ell(uv) - h(u) + h(v)$ (i.e., (10.3) with $\pi = -h$) are nonnegative. With a heuristic that is everywhere zero, A* is Dijkstra's algorithm stopped at t; the closer h is to $\delta(\cdot,t)$, the fewer nodes it scans, and on a road network the straight-line distance to t is a typical choice.

Algorithm 10.3 (A*, `AStar::run(s, t)`).

```
1  g(s) := 0; push s on H with priority h(s); scanned := 0
2  while H is not empty:
3     u := pop the top of H; scanned := scanned + 1
4     if u = t: return true with the path of the predecessor arcs from s to t
5     for each arc a = uv out of u:
6        if v was never seen: g(v) := g(u) + l(a); pred(v) := a; push v with g(v) + h(v)
7        else if g(u) + l(a) < g(v):
8           g(v) := g(u) + l(a); pred(v) := a
9           decrease v to g(v) + h(v) if it is in H, else push it again
10 return false
```

Theorem 10.4. *Let the lengths be nonnegative. (i) With a consistent heuristic, Algorithm 10.3 is Dijkstra's algorithm on the reduced lengths of (10.6), no node is pushed again at step 9, and $g(t)$ is the distance from s to t when t is removed from the heap. (ii) With an admissible heuristic, the algorithm ends, and $g(t)$ is the distance from s to t when t is removed from the heap (and t is not reachable from s if it never is).*

Proof. (i) For each seen node, $g(v) + h(v) = g'(v) + h(s)$, where $g'(v)$ is the length of the same path under the reduced lengths $\ell'(uv) = \ell(uv) - h(u) + h(v)$. Hence, the priorities and the comparisons of step 7 are those of Dijkstra's algorithm on $\ell' \ge 0$, shifted by the constant $h(s)$, and Theorem 10.2 applies: a node leaves the heap with its final g', and no later relaxation improves it. The shortest paths for ℓ' and for ℓ coincide by (10.3). (ii) At each moment before t leaves the heap, let P be a shortest path from s to t and x its first node not yet removed from the heap with $g(x) = \delta(s,x)$. It exists, since s has $g(s) = 0$, and if a node of P was removed with its exact g, the next one received its exact g at that moment. Then x is in the heap with priority $\delta(s,x) + h(x) \le \delta(s,x) + \delta(x,t) + h(t) = \delta(s,t) + h(t)$ by (10.5). If t were removed with $g(t) > \delta(s,t)$, its priority $g(t) + h(t)$ would exceed that of x, a contradiction. For the termination, call $W(v)$ the path of s to v through which $g(v)$ was last set ($W(v) = W(u)$ followed by uv at steps 6 and 8). If $W(u)$ passed through v, its part up to v would be an earlier $W(v)$, of length at least the present $g(v)$, and the test of step 7 would fail. Hence, each $W(v)$ is a simple path, and $g(v)$, which strictly decreases at each new push of v, takes finitely many values. Finally, if t is reachable, the node x above exists while t has not left the heap, so the heap does not empty before. □

With an admissible heuristic that is not consistent, a node whose distance improves after its scan is pushed again (step 9) and scanned again, possibly many times. Its query `scanned()` returns the number of scans of the last run, a node scanned twice being counted twice. Note that the heuristic must be admissible for the target of each run; since the map is read through a reference, its values may be changed between two runs with different targets. We remark that `AStar` has no default for the type of the length map, that its heap is ordered by the priorities $g + h$, and that its traits `AStarDefaultTraits<GR, LEN>` give `HeapCrossRef` and `Heap`; the class defines `Digraph`, `LengthMap`, `HeuristicMap`, `Value`, `Path`, `HeapCrossRef` and `Heap`. Its constructor takes the digraph, the lengths and the heuristic.

10.4 The Bellman-Ford algorithm

In the algorithm of Bellman 1958 and Ford 1956, negative lengths are allowed. It proceeds by rounds, each of which relaxes the out-arcs of the nodes whose distance changed in the previous round, i.e., the active nodes (Algorithm 10.4, where ∞ is the value `infinity()` of the operation traits); with no negative cycle reachable from the sources, at most $n-1$ rounds change a distance. If a negative cycle is reachable from the sources, the rounds never end, and a cycle of negative length appears among the predecessor arcs.

Algorithm 10.4 (Bellman-Ford by rounds of the active nodes, `BellmanFord`).

```
init(value):
1  for each node v: pred(v) := INVALID; d(v) := value
2  active := all the nodes if value < infinity, else none
addSource(s, d0):
3  d(s) := d0; add s to active (if it is not there)
processNextWeakRound():
4  next := empty
5  for each node u in active, for each arc a = uv out of u:
6     if d(u) + l(a) < d(v): d(v) := d(u) + l(a); pred(v) := a; add v to next
7  active := next; return (active is empty)
processNextRound():  the same, with d(u) read at step 6 from a copy of the
                     distances of the active nodes made at step 4
start():          repeat processNextWeakRound() at most n - 1 times, until it returns true
checkedStart():   the same at most n times; return (active is empty)
limitedStart(k): repeat processNextRound() at most k times, until it returns true
```

`processNextWeakRound()` reads the current distance $d(u)$ at step 6, which may already have been decreased in the same round; `processNextRound()` reads it from the copy, which keeps the rounds exact in the sense of the following theorem. In it, $D_k(v)$ is the least of the values $d_0(s)+\ell(P)$ over the sources s and the paths P from s to v with at most k arcs (a path may repeat nodes, cf. §10.1), with $D_k(v)=\infty$ if there is none, after `init()` with the default value.

Theorem 10.5. *After k calls of `processNextRound()`, $d(v)=D_k(v)$ for each node v; after k calls of `processNextWeakRound()`, $D_\infty(v)\le d(v)\le D_k(v)$, where $D_\infty(v)=\inf_k D_k(v)$. If no negative cycle is reachable from the sources, the active set is empty after at most n rounds of either kind, and d is then the distance; if one is, the active set is never empty. Each round takes $O(n+m)$ time.*

Proof. The distances never increase, and each finite $d(v)$ is the value of a path, which gives $d(v)\ge D_\infty(v)$. For the exact rounds, we show by induction that, after round k, $d=D_k$ and the active nodes are those with $D_k(v)<D_{k-1}(v)$ (with $D_{-1}=\infty$, the sources being active before the first round). One has $D_k(v)=\min\{D_{k-1}(v),\ \min_{uv\in A} D_{k-1}(u)+\ell(uv)\}$; a node u that is not active at the start of round k has $D_{k-1}(u)=D_{k-2}(u)$, and its arcs were relaxed with this value in an earlier round. Hence, relaxing from the copied values of the active nodes gives exactly D_k, and the nodes whose value decreases are those put in the next active set. The weak rounds relax with values that are at most the copied ones, which gives $d\le D_k$ by the same induction. If no negative cycle is reachable, D_k is attained by simple paths for $k\ge n-1$, so that $D_{n-1}=D_\infty$ is the distance; after round $n-1$ the distances are final, and round n changes none, leaving the active set empty. If a negative cycle C is reachable, the active set is never empty: otherwise the last round would leave d finite on C and satisfying (10.2) on its arcs, and summing (10.2) along C would give $\ell(C)\ge 0$. Each round scans the arcs out of the active nodes, i.e., at most m arcs, and the active nodes are marked in a map. □

Hence, `start()` performs at most $n-1$ weak rounds and stops early when one changes no distance, which takes $O(nm)$ time in the worst case and much less in practice when the active nodes are few. `checkedStart()` performs at most n and returns `false` if some node is still active, which happens if and only if a negative cycle is reachable from the sources. On the other hand, `limitedStart(num)` performs at most `num` exact rounds, and computes the values D_{num} (the predecessor arcs, however, do not form such paths in general, as the documentation warns). Also, `run(s)` and `run(s, num)` are `init()` and `addSource(s)` followed by `start()` and `limitedStart(num)`. The predecessor arcs satisfy the following lemma, which `negativeCycle()` uses.

Lemma 10.1. *At any moment of the algorithm, every cycle formed by predecessor arcs has negative length.*

Proof. Let $C=(v_1,\dots,v_k)$ be such a cycle, with the arc v_iv_{i+1} as predecessor of v_{i+1} (indices modulo k), and let the predecessor of v_1 be the last of them to be set. Just before that, each v_{i+1} with $i\ne k$ has $d(v_{i+1})\ge d(v_i)+\ell(v_iv_{i+1})$. Indeed, equality held when its predecessor was set (with a value of $d(v_i)$ at least the

present one, in an exact round), $d(v_{i+1})$ has not changed since, since it changes only together with its predecessor, and $d(v_i)$ can only have decreased. The new predecessor of v_1 is set because $d'(v_k) + \ell(v_k v_1) < d(v_1)$, where $d'(v_k) \geq d(v_k)$ is the value used. Summing the k inequalities gives $d(v_1) > d(v_1) + \ell(C)$, i.e., $\ell(C) < 0$. □

Its other members are those of `Dijkstra`, with a few differences. Its template parameters `GR`, `LEN` and `TR` default to `ListDigraph`, `GR::ArcMap<int>` and `BellmanFordDefaultTraits<GR, LEN>`, whose types are `Digraph`, `LengthMap`, `Value`, `OperationTraits`, `PredMap` and `DistMap` (with their `create...()` functions), and the named parameters are `SetPredMap<T>`, `SetDistMap<T>` and `SetOperationTraits<T>`. Its default operation traits `BellmanFordDefaultOperationTraits<V>` have, besides `zero()`, `plus()` and `less()`, a static `infinity()`, the infinity of `V` if it has one and its largest value otherwise. In the latter case (the specialization for the types without infinity) `plus()` returns the largest value if one of its terms is that value, and an unreached node thus keeps its distance. The call `init(value)` gives all the nodes the distance `value` (`infinity()` by default). If `value` is finite, all the nodes are active; this computes the least lengths of the paths that may start anywhere, and is the usual way of looking for a negative cycle anywhere in the digraph (e.g., `init(0)` followed by `checkedStart()`). Then `addSource(s, dst)` sets the distance of `s` to `dst` (`zero()` by default) and makes it active.

Its queries are `dist(v)`, `predArc(v)`, `predNode(v)`, `path(t)`, `distMap()`, `predMap()` and `reached(v)`, which is true if the distance of v is not `infinity()`. Furthermore, the iterator `ActiveIt`, and the range `activeNodes()`, list the active nodes after the last round. Finally, `negativeCycle()` follows the predecessor arcs back from each active node in turn, marking the nodes it meets with the index of that node. It returns, as a `lemon::Path`, the first cycle of predecessor arcs that it closes, or an empty path if it closes none; it takes $O(n)$ time, and the cycle has negative length by Lemma 10.1. It may be called at any time, e.g., after each round. The function `bellmanFord(g, length)` returns a `BellmanFordWizard`, with the named function parameters `predMap(m)`, `distMap(m)`, `path(p)` and `dist(d)` (its base `BellmanFordWizardBase` and its traits `BellmanFordWizardDefaultTraits` hold them). Its runs are `run(s)` and `run(s, t)`, the latter running `run(s)` in full and then reading the path and the distance of `t`, and returning whether `t` is reached.

> **abLEMON.** `BellmanFord` keeps the maps it allocates in a `std::unique_ptr`, apart from the pointers through which it reads them, and a map given by the user is therefore never deleted; LEMON 1.3.1 could delete one. Like `Dijkstra`, it keeps an object of its operation traits, set by `operationTraits(ops)` and read by `operationTraits()`; the test `test/bellman_ford_test.cc` uses one whose comparison has a tolerance, for which an improvement within the tolerance is no improvement.

The program `manual/examples/10-bellman-ford.cc` (the test `manual_10_bellman_ford`) computes the distances with negative lengths and checks them against (10.2). It also computes the distances of the paths of at most two arcs by `run(s, 2)`, and finds a negative cycle by `checkedStart()` and `negativeCycle()`, whose length it checks.

10.5 Arc-disjoint paths of least total length

Given two nodes $s \neq t$ and an integer k, the problem is to find k paths from s to t that have no arc in common and the least total length, with nonnegative lengths. It is the minimum cost flow problem

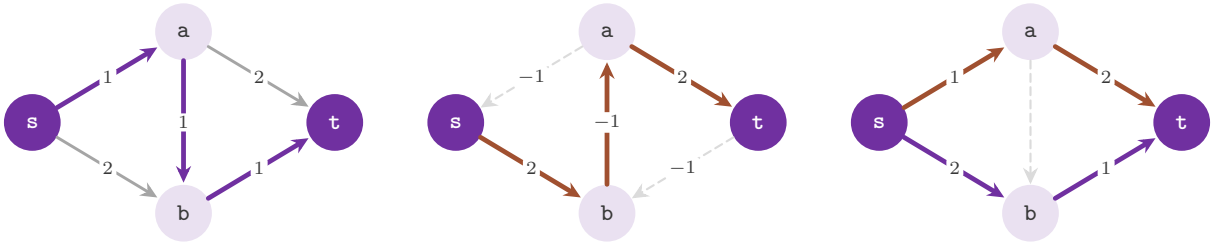
$$\min \Big\{ \sum_{a \in A} \ell(a)\, x_a \;:\; \sum_{a \in \delta^+(v)} x_a - \sum_{a \in \delta^-(v)} x_a = k\,[v = s] - k\,[v = t] \quad \forall v \in N, \quad x \in \{0,1\}^A \Big\}, \tag{10.7}$$

where $\delta^+(v)$ and $\delta^-(v)$ are the arcs out of and into v, and $[\cdot]$ is 1 if the condition holds and 0 otherwise. A feasible x is the union of k arc-disjoint paths (and possibly of cycles, which the decomposition below does not use). Note that the shortest path followed by a second shortest path that avoids its arcs is not optimal in general; in Figure 10.1, the shortest path s, a, b, t leaves no second path at all. Thus, the algorithm of Suurballe 1974, in the form of Suurballe and Tarjan 1984, is the successive shortest path algorithm for (10.7). At iteration i it finds a shortest path from s to t in the residual digraph G_x of the flow x of $i-1$ paths, which has

$$\text{an arc } uv \text{ of length } \ell(uv) \text{ for each } uv \text{ with } x_{uv} = 0, \qquad \text{an arc } vu \text{ of length } -\ell(uv) \text{ for each } uv \text{ with } x_{uv} = 1, \tag{10.8}$$

and sends one unit of flow along it, i.e., it sets x to 1 on its forward arcs and to 0 on the arcs whose reverse it uses (Algorithm 10.5).

This residual digraph has negative lengths, but with the potentials π kept by the algorithm the reduced lengths (10.3) of its arcs are nonnegative, and each path can therefore be found by Dijkstra's algorithm, stopped when t reaches the top of the heap. Let $\delta_\pi(v)$ be the reduced distance from s in G_x found by a search, $\delta_\pi(t)$ that of t,



first path, length 3 *residual: second path, length 3* *two paths, length 6*

Figure 10.1. Suurballe's algorithm on a digraph whose shortest path leaves no second arc-disjoint path: the first path, the second one in the residual digraph through the reverse of a-b (of length -1), and the two paths of least total length that result.

and P the path found. After the search, the potential of each node processed by it (i.e., removed from the heap before t) is increased by $\delta_\pi(v) - \delta_\pi(t)$, and the others are left as they are.

Algorithm 10.5 (Successive shortest paths with potentials, `Suurballe::findFlow(t, k)` after `init(s)`).

```
1  x := 0; pi := 0; i := 0
2  while i < k:
3     run Dijkstra from s on the residual digraph (10.8) with the reduced lengths (10.3),
      stopping when t is on the top of the heap
4     if t is not reached: stop
5     for each node v processed: pi(v) := pi(v) + dist(v) - dist(t)
6     push one unit along the path found: x := 1 on its forward arcs, 0 on the reversed ones
7     i := i + 1
8  return i
```

Lemma 10.2. *Suppose that the reduced lengths of all the arcs of G_x are nonnegative before an iteration of Algorithm 10.5 that finds a path P. Then, after it, the reduced lengths of all the arcs of the new residual digraph are nonnegative, and those of the arcs of P (and of their reverses) are zero.*

Proof. Let $\eta(v) = \min\{\delta_\pi(v), \delta_\pi(t)\}$. The update of step 5 is $\pi'(v) = \pi(v) + \eta(v) - \delta_\pi(t)$ for every node v: a processed node has $\delta_\pi(v) \le \delta_\pi(t)$, and a node not processed has $\delta_\pi(v) \ge \delta_\pi(t)$ (possibly $+\infty$), by Theorem 10.2. For an arc uv of G_x, one has $\delta_\pi(v) \le \delta_\pi(u) + \ell_\pi(uv)$ by (10.2) for the distances, and therefore $\eta(v) \le \eta(u) + \ell_\pi(uv)$, since $\ell_\pi(uv) \ge 0$; hence, $\ell_{\pi'}(uv) = \ell_\pi(uv) + \pi'(u) - \pi(u) - \pi'(v) + \pi(v) \ge 0$. Along P, whose nodes before t are processed and on which δ_π grows exactly by the reduced lengths, the same inequality is an equality, and $\ell_{\pi'} = 0$. The new residual digraph has the arcs of G_x not on P and the reverses of the arcs of P, whose reduced lengths are the opposite of zero. □

Theorem 10.6. *After i iterations, x is an optimal solution of (10.7) with $k = i$, and*

$$x_{uv} = 0 \;\Rightarrow\; \ell_\pi(uv) \ge 0, \qquad x_{uv} = 1 \;\Rightarrow\; \ell_\pi(uv) \le 0 \qquad \text{for each arc } uv. \tag{10.9}$$

If the loop stops at step 4, there are no $i+1$ arc-disjoint paths from s to t. The algorithm takes $O(k\,(n+m)\log n)$ time with the binary heap.

Proof. Conditions (10.9) say that the reduced lengths of the arcs of G_x are nonnegative, the arc vu of an arc uv with $x_{uv} = 1$ having reduced length $-\ell_\pi(uv)$. They hold at the start, with $x = 0$, $\pi = 0$ and $\ell \ge 0$, and are kept by Lemma 10.2. The flow x satisfies the constraints of (10.7) with $k = i$, since each iteration sends one unit along a path from s to t of G_x. For any other feasible x', summing (10.3) gives $\sum_a \ell(a)x'_a = \sum_a \ell_\pi(a)x'_a + i\,(\pi(t) - \pi(s))$, and the same for x. Hence, $\ell(x') - \ell(x) = \sum_a \ell_\pi(a)(x'_a - x_a) \ge 0$, since each term is nonnegative by (10.9) (where $x_a = 0$, $x'_a - x_a \ge 0$; where $x_a = 1$, $x'_a - x_a \le 0$). If the loop stops at step 4, let X be the set of the nodes reachable from s in G_x. Then $t \notin X$, and each arc out of X has $x = 1$ and each arc into X has $x = 0$, otherwise G_x would leave X. Hence, the arcs out of X are exactly i by the constraints of (10.7), and $i + 1$ arc-disjoint paths, each of which leaves X, cannot exist. Each iteration is one search of Dijkstra plus $O(n + m)$ work. □

The first search runs on the lengths, since all the potentials are zero, and the potentials of the nodes that it does not process stay zero. Then the paths are extracted from x by `findPaths()`, which follows from s the arcs of unit flow not yet used, scanning the arcs out of the current node from the first one. By the constraints of (10.7),

it can always leave a node other than t, and since each step uses up an arc it reaches t. It takes $O(km)$ time when x has no cycle (a path then visits no node twice), and $O(m\Delta)$ in general, where Δ is the largest out-degree, since it makes at most m steps. Also, the node-disjoint version of the problem is solved by the same class on the digraph of the adaptor `SplitNodes` of chapter 5, in which each node is split into an arc of capacity one.

Interface. `Suurballe<GR, LEN, TR>` takes the digraph type, the type of the length map (by default `GR::ArcMap<int>`) and the traits class, by default `SuurballeDefaultTraits<GR, LEN>`. These traits give `Digraph`, `LengthMap`, `Length` (the type of the lengths), `FlowMap` (an `ArcMap<int>`), `PotentialMap` (a `NodeMap` of the lengths), `Path` (a `lemon::Path`, or any path type with `addBack()`), and the `HeapCrossRef` and the `Heap` (a `BinHeap`) of the internal searches; the named parameters `SetFlowMap<T>`, `SetPotentialMap<T>`, `SetPath<T>` and `SetHeap<H, CR>` change them. Its constructor takes the digraph and the lengths, which must be nonnegative. The setters `flowMap(m)` and `potentialMap(m)` set the maps of the flow and of the potentials (allocated otherwise), and the searches are made by its private class `ResidualDijkstra`.

A call `run(s, t, k)` is `init(s)` followed by `start(t, k)`, with $k = 2$ by default, and returns the number of paths found, i.e., k if there are at least k arc-disjoint paths, and their largest number otherwise. In turn, `start(t, k)` is `findFlow(t, k)` (Algorithm 10.5) followed by `findPaths()`. If the paths from one source to many targets are needed, `fullInit(s)` replaces `init(s)`. It runs one complete search of Dijkstra from `s` on the lengths, which then serves as the first iteration of each subsequent `start(t, k)`, with π equal to the distances from s (for which Lemma 10.2 holds as well, since they satisfy (10.2) and are tight on the tree of the search). Each such call thus runs only $k - 1$ searches, and returns 0 when `t` cannot be reached from `s`, as it does after `init(s)`.

abLEMON. In LEMON 1.3.1, after `fullInit(s)`, `findFlow(t, k)` took the first path from the complete search without checking that `t` was reached, counted one path for a target that cannot be reached, and `findPaths()` then did not end; the test `suurballe_test` checks such a target.

Its queries are `totalLength()` (the length of the flow, computed in $O(m)$), `flow(a)` and `flowMap()`, `potential(v)` and `potentialMap()` (an optimal dual solution, i.e., potentials that satisfy (10.9) with the flow), `pathNum()` and `path(i)` for i from 0 to `pathNum()` -1.

The program `manual/examples/10-suurballe.cc` (the test `manual_10_suurballe`) solves the instance of Figure 10.1, checks that the two paths are arc-disjoint and that the flow and the potentials satisfy (10.9), and looks for a third path, which does not exist.

10.6 Exercises

Exercise 10.1. Show by an example that Dijkstra's algorithm may give a wrong distance when one arc has a negative length, although no cycle is negative, point out the step of the proof of Theorem 10.2 that fails, and check the example against `BellmanFord`. (The test `test/dijkstra_test.cc` exercises the interface of `Dijkstra`.)

Exercise 10.2. Compute the paths of least bottleneck of a digraph with the operation traits of the program `manual/examples/10-dijkstra.cc`, and verify that they satisfy the two hypotheses of Theorem 10.2; then show that the product of the lengths, with lengths in $(0, 1]$ and the comparison reversed, also does, and gives the paths of greatest reliability.

Exercise 10.3. Compare the number of nodes scanned by `Dijkstra::run(s, t)`, by `BidirDijkstra` and by `AStar` with the Manhattan distance on a grid of 100×100 nodes with random lengths at least 1, for random pairs of nodes. (The test `test/point_to_point_test.cc` checks the two point-to-point classes against `Dijkstra`, with heuristics that are zero, exact, half the exact one, and admissible but not consistent.)

Exercise 10.4. Find a negative cycle anywhere in a digraph by `init(0)` and `checkedStart()` of `BellmanFord`, and explain why `init(0)` is equivalent to adding a new source joined to each node by an arc of length zero. (The program `manual/examples/10-bellman-ford.cc` and the test `test/bellman_ford_test.cc` find negative cycles.)

Exercise 10.5. Use `Suurballe` on the digraph of `SplitNodes` to find two node-disjoint paths of least total length, and verify that they share no inner node. (The test `test/suurballe_test.cc` checks the flow, the potentials and the paths.)

Exercise 10.6. Show that, after i paths, the value $\pi(t) - \pi(s)$ of the potentials of Algorithm 10.5 is the length, in the residual digraph (10.8) of the previous flow, of the last path found, and check it by running the program `manual/examples/10-suurballe.cc` with $k = 1$ and $k = 2$.

11. Spanning trees and arborescences

We are concerned in this chapter with two problems of choosing a cheapest set of arcs that connects the nodes of a graph. The first one is the minimum spanning tree of an undirected graph, which `kruskal()` of `lemon/kruskal.h` solves by Kruskal's algorithm on a union-find structure (§8.2). Its directed counterpart, the second one, is the minimum cost arborescence of a digraph rooted at a given node; `MinCostArborescence` of `lemon/min_cost_arborescence.h` solves it by the algorithm of Chu, Liu and Edmonds, and also gives an optimal dual solution, which proves the optimality of the arborescence. The two problems look alike; however, the greedy choice that solves the first is not enough for the second, whose algorithm has to contract cycles (cf. Figure 11.2).

11.1 Minimum spanning trees

Let $G = (N, E)$ be an undirected graph with $n = |N|$ nodes, $m = |E|$ edges and costs $c : E \to \mathbb{R}$ on the edges, and let $N_1, \dots, N_p$ be the node sets of its connected components. A *spanning forest* of G is a set $T \subseteq E$ of edges that contains no cycle and connects the nodes of each N_i; it has $n - p$ edges, and it is a spanning tree when G is connected ($p = 1$). We consider the problem

$$\min \Big\{ c(T) = \sum_{e \in T} c(e) \ : \ T \text{ is a spanning forest of } G \Big\}, \tag{11.1}$$

which, on a connected graph, is the minimum spanning tree problem, and on any other graph is that of a minimum spanning tree of each component. For a spanning forest T and an edge $f \notin T$, the two ends of f lie in the same component, and we let $P_T(f)$ be the (unique) path of T between them; a loop f has $P_T(f) = \emptyset$. For an edge $e \in T$, the forest $T \setminus \{e\}$ splits the component of e into two node sets, and we let $C_T(e)$ be the set of the edges of G with one end in each of them (the *fundamental cut* of e). The two objects describe the same pairs of edges.

Lemma 11.1. *Let T be a spanning forest, $e \in T$ and $f \in E \setminus T$. Then $e \in P_T(f)$ if and only if $f \in C_T(e)$, and, in this case, $T' = (T \setminus \{e\}) \cup \{f\}$ is a spanning forest with $c(T') = c(T) - c(e) + c(f)$.*

Proof. Let U and W be the two node sets into which $T \setminus \{e\}$ splits the component of e. If f has one end in U and one in W, the path of T between its ends must cross from U to W, which only e does; hence, $e \in P_T(f)$. Conversely, if $e \in P_T(f)$, removing e cuts this path into a part in U and a part in W, and the two ends of f are therefore on opposite sides. In this case, $T \setminus \{e\}$ has one component more than T, and f joins two of its components (U and W); hence, T' has no cycle and the same components as T, i.e., it is a spanning forest, and its cost is obvious. □

Theorem 11.1. *A spanning forest T is optimal for (11.1) if and only if*

$$c(f) \ \geq \ c(e) \qquad \text{for each } f \notin T \text{ and each } e \in P_T(f), \tag{11.2}$$

or, equivalently, if and only if

$$c(e) \ \leq \ c(f) \qquad \text{for each } e \in T \text{ and each } f \in C_T(e). \tag{11.3}$$

Proof. By Lemma 11.1, (11.2) and (11.3) are the same set of inequalities, written once by the edges out of the forest and once by the edges in it. If (11.2) fails for some f and $e \in P_T(f)$, the forest T' of Lemma 11.1 costs $c(T) - c(e) + c(f) < c(T)$, and T is not optimal. Conversely, let T satisfy (11.3), and among the optimal forests let T^* be one with the largest $|T \cap T^*|$. If $T \neq T^*$, since both have $n - p$ edges, there is an edge $e \in T \setminus T^*$. Its ends are joined by the path $P_{T^*}(e)$, which, together with e, is a cycle; this cycle crosses the cut $C_T(e)$ once through e, and must therefore cross it at least once more, through an edge $f \neq e$ of $P_{T^*}(e)$. The only edge of

T in $C_T(e)$ is e; hence, $f \notin T$, and $c(e) \le c(f)$ by (11.3). By Lemma 11.1 applied to T^* and the pair $f \in T^*$, $e \notin T^*$ (indeed $f \in P_{T^*}(e)$), the forest $(T^* \setminus \{f\}) \cup \{e\}$ costs at most $c(T^*)$, and is therefore optimal, and it has one more edge in common with T than T^*, a contradiction. Thus, $T = T^*$ is optimal. □

Kruskal's algorithm (Kruskal 1956) builds a forest that satisfies (11.2) greedily. It scans the edges in nondecreasing order of cost, and keeps an edge if it joins two different components of the forest of the edges kept so far, i.e., if it closes no cycle. The components are the classes of a union-find structure on the nodes, and the test and the union are one call of `join()` (Algorithm 11.1, where *out* is the output of the function and *total* its value).

Algorithm 11.1 (Kruskal's algorithm, `kruskal()`).

```
1  if the costs are given as a map: list the pairs (e, c(e)) of all the edges
                                    and sort them by nondecreasing c(e)
2  make a union-find structure with one class {v} per node v; total := 0
3  for each pair (e, c(e)) in the order of the list, with e = uv:
4     if join(u, v):                       [u and v were in different classes]
5        out(e) := true; total := total + c(e)
6     else: out(e) := false
7  return total
```

Theorem 11.2. *If the list scanned by Algorithm 11.1 holds every edge of G in nondecreasing order of cost, the edges T with $out(e) = true$ form an optimal solution of (11.1), and the value returned is $c(T)$. For a list in any other order (or holding only some of the edges), T is a spanning forest of the graph of the edges of the list, i.e., one with no cycle and whose components are those of that graph.*

Proof. At each step, the classes of the union-find structure are the node sets of the components of the forest (N, T) kept so far: this is true at the start, and `join()` (Algorithm 8.1) unites the classes of u and v exactly when the edge uv is kept. An edge is kept only if its ends are in different components, and therefore closes no cycle; an edge is discarded only if its ends are already in the same component, which they remain. Hence, at the end, T has no cycle and connects the ends of each edge of the list, i.e., it is a spanning forest of the graph of the list. If the list is sorted and complete, let f be a discarded edge. When it was scanned, its ends were joined by a path of edges kept before it, which is $P_T(f)$ (a forest has one path between two nodes) and whose edges cost at most $c(f)$, since they come earlier in the list. Thus, (11.2) holds, and T is optimal by Theorem 11.1. The value is the sum of the costs of the kept edges, by step 5. □

Theorem 11.3. *Algorithm 11.1 takes $O(m \log m)$ time when the costs are given as a map, and $O(n + m\,\alpha(m, n))$ time when they are given as a sorted list of m pairs.*

Proof. Listing the pairs takes $O(m)$ and `std::sort` takes $O(m \log m)$. The rest makes n insertions and m calls of `join()`, each made of two finds and at most one union, which take $O(n + m\,\alpha(m, n))$ in all by Theorem 8.2. Since $\alpha(m, n) \le \log m$ for $m \ge 2$, the sort dominates in the first case. □

Note that the loop scans all the edges of the list even after the forest is complete, since the output receives a value for each of them; however, the work after the last useful edge is only one `join()` per edge. Of course, when the costs come already sorted (e.g., from a previous run), passing them as a list saves the sort, which is usually the dominating cost.

Interface. Function `kruskal(g, in, out)` accepts various types for its arguments, and returns the cost of the forest found, of the type of the costs. Its graph `g` may be undirected, with the costs on its edges, or directed, in which case the arcs are taken as edges (the union-find structure joins the target and the source of each arc) and their direction is disregarded. Its costs `in` may be given either (i) as a readable map on the edges (or arcs), whose values are the costs, or (ii) as an STL container of pairs `std::pair<GR::Edge, C>` (or `std::pair<GR::Arc, C>`), where `C` is the type of the costs. In case (i), the function collects the pairs of each item and its cost into a vector, and sorts it by cost with `std::sort`. In case (ii), the pairs must already be sorted by nondecreasing cost, since no sort is done (Theorem 11.2 gives what is computed otherwise), and an edge that is not in the container receives no value in `out`.

Its output `out` may also be given in two ways. It may be (i) a writable map on the edges with `bool` values, which receives `true` for the edges of the forest and `false` for the others, each edge of the list being written once. Alternatively, it may be (ii) an output iterator, e.g., `std::back_inserter(tree)`, into which the edges of the forest are copied in the order in which they are found; the function wraps it in a `LoggerBoolMap` (cf. chapter 6), which copies the keys set to `true` and ignores the others. A second overload takes `out` by constant reference, so that a temporary iterator may be passed. The choice among the forms is made at compile time by the helper templates of the namespace `_kruskal_bits`, which are not part of the interface: `KruskalValueSelector`, `Kru`

`skalInputSelector`, `KruskalOutputSelector`, `SequenceInputIndicator`, `MapInputIndicator`, `SequenceOutputIndicator`, `MapOutputIndicator`, `RemoveConst` and `PairComp`. They take an input for a sequence if its `value_type` has a `first_type`, and for a map if it has a type `Value`; an output is taken for a sequence (an iterator) if it has a `value_type`, and for a map if it has a `Value`.

Figure 11.1 shows the tree that the program of this section computes. Its edges, sorted by cost, are kept in the order 1, 2, 3, 5, 6, while the edges of cost 4, 7, 8 and 9 close cycles. Each of the latter costs at least as much as every edge of the path of the tree between its ends, as (11.2) requires.

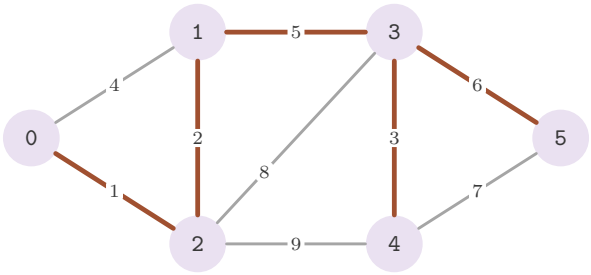


Figure 11.1. The minimum spanning tree found by Kruskal's algorithm, of total cost 17, with the cost of each edge.

The program `manual/examples/11-kruskal.cc` (the test `manual_11_kruskal`) computes the tree of Figure 11.1 twice, with a cost map in and a `bool` map out, and then with a sorted vector of pairs in and an output iterator out. It checks the cost, the number of edges and the condition (11.2), the latter by a breadth-first search of the path of the tree between the ends of each edge out of the tree, on the graph of the adaptor `FilterEdges` (chapter 5).

11.2 Minimum cost arborescences

Let $G = (N, A)$ be a digraph with costs $c : A \to \mathbb{R}$, and let $S \subseteq N$ be a nonempty set of *sources* (one root r in the simplest case), and $R \supseteq S$ the set of the nodes reachable from S. For $X \subseteq N$, we write $\delta^-(X)$ for the set of the arcs with their head in X and their tail out of X (so that a loop enters no set), and $\delta^-(v)$ for $\delta^-(\{v\})$. An *arborescence* of R rooted at S is a set B of arcs with both ends in R such that

$$|B \cap \delta^-(v)| = 1 \quad \text{for each } v \in R \setminus S, \qquad |B \cap \delta^-(s)| = 0 \quad \text{for each } s \in S, \tag{11.4}$$

and each node of R is reachable from S along the arcs of B. With one source r, it is a spanning tree of R whose arcs are directed away from r, and with several it is a union of such trees, one per source, that reaches all of R. We look for one of least cost $c(B) = \sum_{a \in B} c(a)$. Choosing the cheapest arc entering each node, which would mirror the greedy algorithm of the undirected case, may give cycles: in Figure 11.2, the cheapest arcs entering a and b form the cycle a, b, a. The optimality of an arborescence is proved instead by a family $\mathcal{F}$ of subsets of $R \setminus S$ with a value y_X on each set $X \in \mathcal{F}$ (a solution of the dual of the linear program of the problem, Edmonds 1967), through the *reduced costs*

$$c'(a) \;=\; c(a) \;-\; \sum_{X \in \mathcal{F} \,:\, a \in \delta^-(X)} y_X \qquad (a \in A). \tag{11.5}$$

Lemma 11.2. *Let $\mathcal{F}$ be a family of nonempty subsets of $R \setminus S$ and $y : \mathcal{F} \to \mathbb{R}$ be such that $y_X \geq 0$ for each $X \in \mathcal{F}$ with $|X| \geq 2$, and $c'(a) \geq 0$ for each arc a with its tail in R and its head in $R \setminus S$. Then each arborescence B' of R rooted at S has $c(B') \geq \sum_{X \in \mathcal{F}} y_X$, and an arborescence B attains this bound, and is therefore optimal, if $c'(a) = 0$ for each $a \in B$ and $|B \cap \delta^-(X)| = 1$ for each $X \in \mathcal{F}$.*

Proof. Summing (11.5) over the arcs of B', and exchanging the two sums, gives

$$c(B') \;=\; \sum_{a \in B'} c'(a) \;+\; \sum_{X \in \mathcal{F}} y_X \, |B' \cap \delta^-(X)|.$$

The first sum is nonnegative, since each arc of B' has its tail in R and, by (11.4), its head in $R \setminus S$. For a singleton $X = \{v\}$, one has $|B' \cap \delta^-(X)| = 1$ by (11.4), whatever the sign of y_X. Let X have $|X| \geq 2$: its nodes

are reachable from S along B', and $X \cap S = \emptyset$. Hence, the first node of X on such a path is entered by an arc of B' from out of X, i.e., $|B' \cap \delta^-(X)| \geq 1$, and $y_X|B' \cap \delta^-(X)| \geq y_X$ since $y_X \geq 0$. Thus, $c(B') \geq \sum_X y_X$, and under the two conditions of the statement both inequalities hold with equality for B. □

The algorithm of Chu and Liu 1965 and Edmonds 1967 builds such a pair (B, y) by contraction (Algorithm 11.2). Each node, and later each contracted set X, chooses the arc of least reduced cost that enters it; that reduced cost becomes the dual variable y_X, and it is therefore subtracted, by (11.5), from the reduced costs of all the arcs entering X, the chosen one falling to zero. A cycle of chosen arcs is contracted into a new set, which chooses in its turn, until a chosen arc comes from a node that is already connected to a source. Then the contractions are expanded, the arc that enters a contracted set from outside replacing the chosen arc of the node it enters, while the other chosen arcs of the set stay.

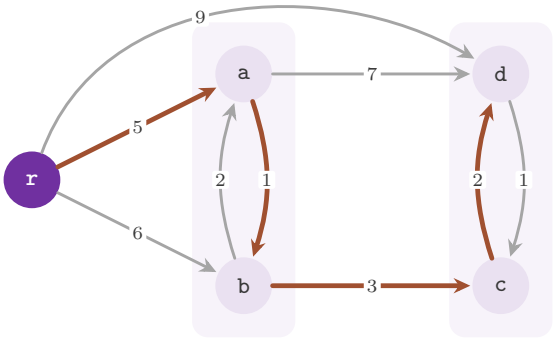


Figure 11.2. A minimum cost arborescence rooted at r: the cheapest arcs entering a and b, and those entering c and d, form two cycles, each contracted (the tinted sets) and then entered by one arc of the arborescence.

The implementation grows these choices along a *path* of sets, kept as a stack of *levels*. Each node is, at any moment, unreached, reached (i.e., in R and waiting to be processed), on the path, or processed. A level is a set X of nodes on the path, created with its dual variable y_X (the k-th variable created having index k). It is stored together with one arc $e_X(w)$ for each reached or processed node $w \notin X$ with arcs into X, the one of least reduced cost among them, and its reduced cost. The *chosen arc* a_X of X is the arc of least reduced cost among the $e_X(w)$, and $ord(a)$ is the index of the set of which a is the chosen arc (-1 for an arc chosen by none). In Algorithm 11.2, Q is the list of the reached nodes, in the order in which they were reached, and H is a heap of nodes.

Algorithm 11.2 (Minimum cost arborescence by contractions, `MinCostArborescence`).

```
init():         every node unreached, pred(v) := INVALID; no set, no dual variable;
                ord(a) := -1 and arborescence(a) := false for each arc a
addSource(s):   by a depth-first search from s, mark reached each unreached node
                that can be reached, and append it to Q; then mark s processed

processNextNode():
1  v := the last node of Q; remove it from Q
2  if v is reached:
3     a := prepare(v)
4     while source(a) is not processed:
5        u := source(a)
6        if u is on the path: a := contract(u)
7        else:                a := prepare(u)
8     finalize(a); empty the stack of the levels
9  return v

prepare(u):          [a new level X = {u}, at the top of the stack]
1  put u on the path
2  for each node w != u, reached, on the path or processed, with arcs into u:
      e_X(w) := an arc wu of least cost; its reduced cost := c(e_X(w))
3  a_X := the e_X(w) of least reduced cost; y_X := its reduced cost
4  subtract y_X from the reduced cost of each e_X(w); ord(a_X) := index of y_X
```

```
5  push the level X; return a_X

contract(u):          [the cycle closed by the chosen arc of the top level]
1  L := the level that contains u
2  pop L and all the levels above it; X := the union of their nodes
3  for each node w not in X with an arc stored in the popped levels:
      e_X(w) := the stored arc from w of least reduced cost
4  a_X := the e_X(w) of least reduced cost; y_X := its reduced cost
5  subtract y_X from the reduced cost of each e_X(w); ord(a_X) := index of y_X
6  push the level X; return a_X

finalize(a):          [expansion of the levels of the path]
1  t := target(a); pred(t) := a; insert t in H with key ord(a)
2  while H is not empty:
3     w := a node of least key in H; remove it from H; mark w processed
4     for each arc e = wz with ord(e) >= 0:
5        if z has never been in H: pred(z) := e; insert z in H with key ord(e)
6        else if z is in H and ord(e) < key(z): pred(z) := e; key(z) := ord(e)
7     arborescence(pred(w)) := true

start():  while Q is not empty: processNextNode()
run(s):   init(); addSource(s); start()
```

The steps are the private members `prepare()`, `contract()` and `finalize()` of the class. The levels are its records `StackLevel` (with the arcs $e_X(w)$ as records `CostArc` of an arc and its reduced cost), and the dual variables are its records `DualVariable`. The nodes on the path are stored in the order in which they were put there, so that the nodes of a level, and of each dual variable, are a range of that order (whose first element locates the level of a node in step 1 of `contract()`). Note that step 2 of `prepare()` keeps the cheapest of several parallel arcs from w, and skips the loops and the arcs from unreached nodes. The theory needs three facts on a call of `processNextNode()`; in Lemma 11.3, the reduced costs are those of (11.5) for the family $\mathcal{F}$ of all the sets created so far.

Lemma 11.3. *Suppose that all the calls of `addSource()` precede the first call of `processNextNode()`. Then the following hold at each point of the execution. (i) The sets of $\mathcal{F}$ form a laminar family (any two are disjoint or nested), and the levels on the stack are disjoint. (ii) For each level X on the stack and each node w with an arc stored in it, the stored reduced cost of $e_X(w)$ is its reduced cost (11.5), which is the least reduced cost of the arcs from w into X. (iii) Every arc with its tail in R and its head in $R \setminus S$ has $c'(a) \geq 0$, and $y_X \geq 0$ for each $X \in \mathcal{F}$ with $|X| \geq 2$. (iv) For each $X \in \mathcal{F}$, the chosen arcs of the sets of $\mathcal{F}$ strictly contained in X have both ends in X, have indices smaller than that of X, and connect the nodes of X strongly; the chosen arc a_X has $c'(a_X) = 0$, and enters no set of $\mathcal{F}$ other than X and the sets contained in X.*

Proof. We argue by induction on the steps, all statements holding trivially before the first set is created. A new set is either a singleton $\{u\}$ of a node u that was only reached, and therefore in no set (`prepare()`), or the union of the levels from L to the top (`contract()`). In both cases it is disjoint from the levels left on the stack and from the sets of the previous calls (whose nodes are processed), and contains the sets of the levels it replaces, which gives (i). For (ii), consider a level X and an arc e from $w \notin X$ into X. After X is created, the reduced cost of e changes only when a set is created that contains the head of e and not w; by (i), such a set contains X, and its creation pops X. Hence, the reduced costs stored at the creation of X (by step 4 of `prepare()`, or by step 5 of `contract()`) stay exact while X is on the stack. That they are computed right at the creation follows from (ii) for the popped levels. Indeed, an arc entering the union X of step 2 of `contract()` from $w \notin X$ enters only one of the popped levels, in which an arc from w of least reduced cost is stored; in `prepare()`, no set contains u before, and the reduced costs are the costs.

For (iii), the creation of X subtracts y_X, the least reduced cost of the arcs from the nodes of R out of X into X, from each of them. Hence, they stay nonnegative, and no other reduced cost changes. The arcs from the nodes that are not yet reached are excluded, but no node is reached after the first call of `processNextNode()`; hence, all the arcs from R are considered. Since the stored reduced costs are nonnegative after their level is created, the value y_X of a contracted set, which is one of them, is nonnegative.

For (iv), consider the stack $L_0, \ldots, L_t$ of the levels, from the bottom. The chosen arc of L_i, for $i < t$, comes from a node of L_{i+1}: indeed, L_{i+1} was pushed by `prepare()` on the tail of that arc, and the later contractions only

enlarge L_{i+1} (or pop L_i too). Thus, when `contract(u)` merges the levels $L_j, \dots, L_t$, with $u \in L_j$, the chosen arcs of $L_j, \dots, L_{t-1}$ go from each level to the one below it, and that of L_t goes from L_j to L_t. They form a cycle through the levels, all inside X, created before X. Together with the induction hypothesis for the levels, this shows that the chosen arcs of the sets strictly contained in X are inside X, have smaller indices, and connect X strongly. Finally, when a_X is chosen, its reduced cost becomes zero. Afterwards, by (i), a new set that contains the head of a_X contains X, and is created by a contraction that merges X with other levels, of which a_X is then an inside arc by what precedes. Hence, no later set is entered by a_X, and $c'(a_X)$ stays zero. □

Lemma 11.4. *When `finalize(a)` is called at step 8 of `processNextNode()`, it marks processed all the nodes on the path, and gives each of them an arc `pred` whose tail was processed before it. These arcs are such that exactly one of these arcs enters each set created in the same call of `processNextNode()`; all these arcs are chosen arcs.*

Proof. The arcs that `finalize()` follows have $ord \geq 0$, i.e., they are chosen arcs, whose heads are on the path (those of the previous calls having been removed from H for good, and the sources never entering it). With the notation of the proof of Lemma 11.3, a is the chosen arc of the top level L_t, from a processed node. From its head, the chosen arcs inside L_t reach all of L_t (Lemma 11.3 (iv)), that of L_{t-1} leads into L_{t-1}, and so on. Hence, all the nodes on the path enter H and are processed, each one with an arc `pred` whose tail is processed, and it remains to count the arcs entering a set X created in the call. An arc chosen for a set strictly contained in X has both ends in X (Lemma 11.3 (iv)). Hence, an arc with $ord \geq 0$ that enters X from out of X is the chosen arc of X or of a set containing X, created after X, and has an index at least that of X. Let x be the first node of X removed from H: its arc `pred` comes from out of X, since no node of X was processed before. Let z be another node of X, and consider the moment at which z is removed from H. Since x is processed and the chosen arcs inside X connect X strongly, there is such an arc from a processed node of X to a node z' of X not yet processed, and therefore in H, with a key smaller than the index of X (Lemma 11.3 (iv)). Since z has the least key in H, its key is also smaller than the index of X. Hence, the final `pred` of z, whose *ord* is that key, is not an arc entering X from outside, and exactly one arc `pred` enters X. □

Theorem 11.4. *After `init()`, the calls `addSource(s)` for the sources $s \in S$ and `start()`, the arcs `pred(v)` for $v \in R \setminus S$ form a minimum cost arborescence B of R rooted at S. The arborescence map is `true` on the arcs of B only, and the dual variables satisfy the conditions of Lemma 11.2 with B; hence, `dualValue()` $= \sum_X y_X = c(B) =$ `arborescenceCost()`.*

Proof. Each node of $R \setminus S$ is appended to Q when it is reached, and is processed either when it is taken from Q or, before, by `finalize()` in the call of another node (Lemma 11.4). A source is processed by `addSource()` and receives no arc, and no node out of R is touched. Each node $v \in R \setminus S$ gets its arc `pred(v)` when it is processed, from a node processed before, and keeps it. Hence, B satisfies (11.4), and the order of processing shows that it has no cycle and that each node of R is reachable from S along B, i.e., B is an arborescence of R rooted at S. Each set of $\mathcal{F}$ is entered by exactly one arc of B: by Lemma 11.4, by one arc `pred` given in its own call, and by no arc given in a later call, whose heads are disjoint from it. The arcs of B are chosen arcs, which have $c' = 0$ by Lemma 11.3 (iv), and Lemma 11.3 (iii) gives the remaining conditions of Lemma 11.2, which proves the optimality of B and the equality of the values. The map is set at step 7 of `finalize()`, once on each arc of B when its head is processed. □

The hypothesis that the sources are all given before the first node is processed is needed by Lemma 11.3 (iii): a node reached later may have arcs into sets created before it was reached, whose reduced costs were never reduced (and may be negative). Note also that the costs may have any sign: only the dual variables of the singletons may be negative, and Lemma 11.2 does not need them nonnegative, since each singleton of $R \setminus S$ is entered by one arc of every arborescence.

Theorem 11.5. *`init()` takes $O(n + m)$ time, all the calls of `addSource()` take $O(n + m)$ in all, and `start()` takes $O(n^2 + m)$ time, with the binary heap of the default traits or with any heap whose operations take $O(n)$ time.*

Proof. The first two bounds are clear: the search of `addSource()` marks each node at most once, and scans the arcs out of a node only when it marks it (or when it is a source). Each call of `prepare()` puts a new node on the path, and therefore happens at most once per node of R; it takes $O(|\delta^-(u)| + n)$ time, the second term for the at most n arcs stored. Each call of `contract()` pops at least two levels (the source of the chosen arc of the top level is out of it) and pushes one. Hence, in a call of `processNextNode()` that puts k nodes on the path, at most $k - 1$ contractions are made and at most $2k - 1$ levels are created, each holding at most n arcs and popped at most once; locating the level of u costs $O(k)$. Summing over all the calls, the levels cost $O(n^2)$ and the arcs entering the nodes $O(m)$. Finally, `finalize()` inserts each node of the path once in H and removes it once, and

scans the arcs out of it once; the decreases are at most as many as the chosen arcs, i.e., the dual variables, at most $2n$ in all. Hence, all the calls of `finalize()` take $O(n \log n + m)$ time with a binary heap, and $O(n^2 + m)$ with any heap of $O(n)$ operations. □

Faster implementations, in $O(m + n \log n)$, exist (Gabow et al. 1986), but the library does not use them. Note that the dual solution makes the result somewhat easier to check: the conditions of Lemma 11.2 can be verified arc by arc, and the sum of the y_X must equal the cost of the arborescence.

Interface. `MinCostArborescence<GR, CM, TR>` takes the digraph type `GR`, the type `CM` of the cost map (by default `GR::ArcMap<int>`), and the traits class `TR`, by default `MinCostArborescenceDefaultTraits<GR, CM>`. The cost map is read once for each arc at each node that the arc enters (in step 2 of `prepare()`), and once more by `arborescenceCost()`. The traits give the types `Digraph`, `CostMap`, `Value` (the type of the costs), `ArborescenceMap` and `PredMap`, with the functions `createArborescenceMap()` and `createPredMap()` that allocate the two maps. The `ArborescenceMap` (by default an `ArcMap<bool>`) receives `false` on each arc at `init()`, and then `true` once on each arc of the arborescence; the `PredMap` is a `NodeMap<Arc>`. They also give `HeapCrossRef` and `Heap`, i.e., a `NodeMap<int>` and a `BinHeap` on integer keys, which `finalize()` uses for H. The named parameters `SetArborescenceMap<T>` and `SetPredMap<T>` set the types of the two maps, which must then be given by `arborescenceMap(m)` and `predMap(m)` (their `create` functions assert). Similarly, `SetHeap<H, CR>` sets the type of the heap and of its cross reference map (by default a `NodeMap<int>`), which the algorithm constructs on the digraph with -1 as the value of each node. The class also defines `Create`, the type itself, which the named parameters produce.

abLEMON. `MinCostArborescence` takes the type of its heap from its traits class, with the named parameter `SetHeap`; LEMON 1.3.1 used a `BinHeap` that could not be changed.

Its constructor takes the digraph and the cost map, and the setters `arborescenceMap(m)` and `predMap(m)` return the object; the maps not given are allocated by `init()` and deleted by the destructor. Its execution control is made of `init()`, `addSource(s)`, `processNextNode()` (which returns the node taken from Q, and must not be called on an empty Q), `queueSize()`, `emptyQueue()`, `start()` and `run(s)`, which is `init()`, `addSource(s)` and `start()`. A call `addSource(s)` marks the nodes reachable from `s`, and may be repeated for several sources, before `start()` as Theorem 11.4 requires. Despite the word "queue" of the documentation, Q is a stack: the nodes are taken in the reverse of the order in which they were reached.

Its queries on the arborescence are `arborescence(a)`, which is true if the arc a is the arc `pred` of its target, `arborescenceMap()`, `pred(v)` (`INVALID` for a source and for an unreached node), `predMap()`, and `arborescenceCost()`, computed in $O(m)$ from the arcs `pred`. Also, `reached(v)` is true if v is reachable from a source, and `processed(v)` if it has been processed. Its queries on the dual solution are `dualNum()`, the number of the dual variables (some of which may have value zero), and `dualValue(k)` and `dualSize(k)`, the value and the number of nodes of the k-th one; `dualValue()` is the sum of all of them, in $O(n)$. An iterator `DualIt(mca, k)` lists the nodes of the k-th set, converts to `Node`, and is compared with `INVALID` to stop. Finally, the function `minCostArborescence(g, cost, source, arborescence)` runs the algorithm from one source, writes the arborescence into the given `bool` arc map, and returns its cost.

The program `manual/examples/11-arborescence.cc` (the test `manual_11_arborescence`) solves the instance of Figure 11.2, reads the dual solution through `DualIt`, checks the conditions of Lemma 11.2 arc by arc and the equality of the primal and dual values, and calls the function-type interface.

11.3 Exercises

Exercise 11.1. Prove that Kruskal's algorithm is also correct when the costs are scanned in nonincreasing order and an edge is discarded, instead of kept, if the edges not yet discarded still connect its ends (the reverse-delete algorithm), and check (11.3) on random graphs by the method of the program `manual/examples/11-kruskal.cc`. (The test `test/kruskal_test.cc` runs `kruskal()` with each form of input and output.)

Exercise 11.2. Compute a maximum spanning tree with `kruskal()`, through a map that negates the costs (e.g., `negMap()` of chapter 6), and explain by Theorem 11.1 why the greedy algorithm remains correct.

Exercise 11.3. Run `kruskal()` on a digraph and on the graph of the adaptor `Undirector` of the same digraph, and verify that the two trees have the same cost.

Exercise 11.4. Give a digraph on which choosing the cheapest arc entering each node, and then breaking each cycle at its most expensive arc, does not give a minimum cost arborescence, and check the answer of `MinCostAr`

`borescence` on it. (The test `test/min_cost_arborescence_test.cc` checks the dual solution as the program `manual/examples/11-arborescence.cc` does.)

Exercise 11.5. Use two sources with `addSource()` and verify that the result is the cheapest union of two arborescences rooted at them. Compare it with a single run from a new root joined to the two sources by arcs of cost zero, and explain the equality of the two costs by Lemma 11.2.

12. Maximum flows and circulations

We are concerned in this chapter with the classes of `lemon/preflow.h`, `lemon/edmonds_karp.h` and `lemon/circulation.h`. The first two compute a flow of maximum value between two nodes of a digraph, together with a cut that proves it maximum, and they share the interface of the concept in `lemon/concepts/max_flow.h`. The third one finds a flow within lower and upper bounds that meets given supplies, or a set of nodes that proves that no such flow exists. Section 12.1 states the problem and the duality on which the results of the chapter rest, sections 12.2 and 12.3 describe the two maximum flow algorithms, section 12.4 the circulation algorithm, and section 12.5 the tolerances and the concept that they share. We treat the minimum cost version of the circulation problem in chapter 13, and the minimum cuts of a graph with no source and no target in chapter 15. The results of the chapter are stated in exact arithmetic, i.e., for an integer value type or for data on which no rounding occurs; section 12.5 describes what changes in floating point.

12.1 The problem and its dual

Let $G = (V, A)$ be a digraph with n nodes and m arcs, let $cap : A \to \mathbf{R}_{\geq 0}$ be the capacities of the arcs, and let $s \neq t$ be two nodes, the source and the target. In the maximum flow problem one looks for a flow $f : A \to \mathbf{R}$ that solves

$$\begin{aligned} \max \quad & \sum_{sv \in A} f(sv) - \sum_{vs \in A} f(vs) \\ & \sum_{uv \in A} f(uv) = \sum_{vu \in A} f(vu) \qquad \forall u \in V \setminus \{s, t\} \\ & 0 \leq f(uv) \leq cap(uv) \qquad \forall uv \in A \end{aligned} \tag{12.1}$$

where the objective, i.e., the net flow out of s, is the *value* of the flow, and the equalities impose the conservation of the flow at the nodes other than s and t. A *cut* is a set $S \subset V$ with $s \in S$ and $t \notin S$, whose capacity is

$$cap(S) = \sum_{uv \in A,\ u \in S,\ v \notin S} cap(uv). \tag{12.2}$$

For a flow f, the *residual digraph* has an arc uv wherever $f(uv) < cap(uv)$ and an arc vu wherever $f(uv) > 0$, with the residual capacities

$$r_f(uv) = cap(uv) - f(uv), \qquad r_f(vu) = f(uv); \tag{12.3}$$

a path from s to t in it is an *augmenting path*, along which the flow may be increased by the smallest residual capacity of its arcs. We call a set of nodes *closed* (in the residual digraph of f) when no residual arc leaves it. All the results of this chapter rest on the following classical theorem.

Theorem 12.1. *(Max-flow min-cut.) The value of any feasible flow is at most the capacity of any cut. A feasible flow f is maximum if and only if its residual digraph has no path from s to t; then the set S of the nodes that s reaches in it is a cut with $cap(S)$ equal to the value of f, and therefore a minimum cut.*

Proof. Summing the conservation constraints over the nodes of a cut S, in which the terms of the arcs with both ends in S cancel, one finds that the value of f equals $\sum_{u \in S,\, v \notin S} f(uv) - \sum_{u \notin S,\, v \in S} f(uv)$, which is at most $cap(S)$ since $0 \leq f \leq cap$. If an augmenting path exists, increasing the flow along it by its smallest residual capacity, which is positive, gives a feasible flow of larger value, and f is not maximum. If none exists, $t \notin S$;

an arc uv with $u \in S$ and $v \notin S$ has $f(uv) = cap(uv)$, or uv would be a residual arc out of S, and an arc vu entering S has $f(vu) = 0$, or uv would be one. Hence, the value of f equals $cap(S)$, which by the first part bounds the value of every flow. □

Thus, the answer of the two maximum flow classes can be checked by the user. Both give, besides the flow and its value, a minimum cut through `minCut()` and `minCutMap()`, and the capacity (12.2) of that cut, computed on the data, must equal the value of the flow. Of course, the minimum cut need not be unique, and the two classes need not give the same one; the following lemma, which we use again for the certificates of chapter 14, describes where the minimum cuts lie.

Lemma 12.1. *Let f be a maximum flow, $S_{\min}$ the set of the nodes that s reaches in its residual digraph, and $S_{\max}$ the set of the nodes that do not reach t in it. A cut is minimum if and only if it is closed in the residual digraph of f; every minimum cut S has $S_{\min} \subseteq S \subseteq S_{\max}$, and both $S_{\min}$ and $S_{\max}$ are minimum cuts. Hence, the smallest and the largest minimum cuts do not depend on the maximum flow chosen.*

Proof. As in the proof of Theorem 12.1, the value of f equals $cap(S)$ for a cut S if and only if the arcs leaving S are saturated and those entering it are empty, i.e., if and only if S is closed. A closed set that contains s contains every node that s reaches, and its nodes reach no node outside it, t in particular; hence, $S_{\min} \subseteq S \subseteq S_{\max}$. The set $S_{\min}$ is closed by definition, and $t \notin S_{\min}$ by Theorem 12.1. The set $S_{\max}$ contains s (otherwise s would reach t) and does not contain t; it is closed, since a residual arc from $u \in S_{\max}$ to a node that reaches t would let u reach t. The last statement holds since the minimum cuts are defined by the data only. □

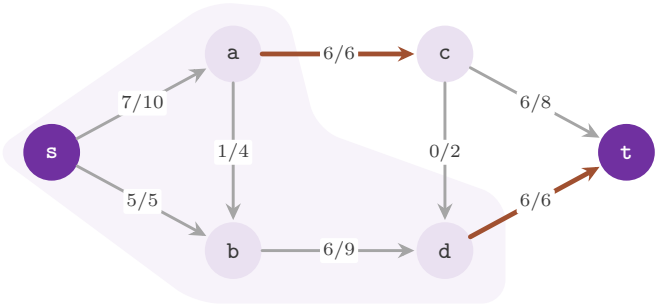


Figure 12.1. The network of the examples of sections 12.2 and 12.3: on each arc its flow in a maximum flow and its capacity; the source side of the minimum cut, {s, a, b, d}, of capacity 12, is on a tint, and the two arcs that leave it are saturated.

12.2 The push-relabel algorithm: Preflow

12.2.1 Preflows and labels

We begin with the class `Preflow`, which implements the push-relabel algorithm of Goldberg and Tarjan (Goldberg and Tarjan 1988; see also Cormen et al. 2001 and Ahuja, Magnanti and Orlin 1993, chapter 7). This algorithm does not keep a feasible flow during its course. It keeps instead a *preflow*, i.e., a map f within the bounds of (12.1) whose *excess* $e_f(u) = \sum_{vu \in A} f(vu) - \sum_{uv \in A} f(uv)$ is nonnegative at the nodes other than s. Together with it, it keeps integer *labels* $d(u) \in \{0, \dots, n\}$ (the levels of an `Elevator`, see chapter 8) such that

$$d(u) \le d(v) + 1 \qquad \text{for each arc } uv \text{ of the residual digraph.} \tag{12.4}$$

A node $u \neq t$ of positive excess and of label below n is *active*. A *push* along a residual arc uv with $d(v) < d(u)$ moves the smaller of $e_f(u)$ and $r_f(uv)$ from u to v; it is *saturating* if it empties the arc, and *emptying* if it empties u. Note that the class pushes towards any lower level, not only to the level just below, as the textbook rule does. A *relabel* of an active node raises $d(u)$ to one more than the least label of the heads of its residual arcs, or to n if that is larger or if no residual arc leaves u. The top level n marks the nodes that cannot reach t, which are never active; also, when a relabel leaves the old level of u empty, every node above that level is lifted to the top (the *gap* rule).

The implementation runs in two phases. In the first one the excess is pushed towards t. At its end the excess of t is the maximum flow value, and the nodes at the top level form the source side of a minimum cut; however, some excess may still be stranded at the nodes that cannot reach t. Hence, a second phase sends that excess

back to s, with labels computed from s in the same way, and it leaves a maximum flow. In both phases the active node of highest label is processed first; in the first phase, however, the header describes the rule as "a mixture of the highest label and the bound decrease heuristics". After each sweep of n selections of the highest active node, the code makes $20n$ selections of an active node of the current level, or of the nearest level below it that has one, without looking for the highest one again. It starts a new sweep when no level at or below the current one has an active node.

Algorithm 12.1 (Preflow, first phase).

```
1  f := 0; d(u) := BFS distance from u to t over the arcs of positive
   capacity, n if none, and d(s) := n
2  f(sv) := cap(sv) for the arcs sv of positive capacity with d(v) < n
3  repeat: (n selections of the highest active node, then 20n selections
           from the current level downwards, as described above)
4      u := the selected active node; if none: stop
5      for each residual arc uv (out-arcs, then in-arcs), while e(u) > 0:
6          if d(v) < d(u): push min{ e(u), r(uv) } along uv
7      if e(u) > 0:
8          L := d(u); d(u) := min{ n, 1 + min{ d(v) : uv residual } }
9          if level L is now empty: lift every node above L to level n
10 return e(t) as the value, { u : d(u) = n } as the source side of the cut
```

Steps 1 and 2 are `init()`, and steps 3 to 9 are `startFirstPhase()`. In step 8 the minimum runs over the residual arcs left after the pushes of step 6, which all go to nodes of level at least $d(u)$; hence, the new label exceeds the old one. The following two lemmas prove the first phase correct.

Lemma 12.2. *During `startFirstPhase()`, f is a preflow, $d(t) = 0$, $d(s) = n$, condition (12.4) holds for every residual arc, the labels never decrease, and the levels below n that hold a node are $0, 1, \ldots, h$ for some $h \leq n - 2$.*

Proof. After `init()` the labels below n are the distances of a breadth first search from t, which never exceed $n - 2$ since s is not among the nodes searched, and they form the levels $0, \ldots, h$ without gaps. Condition (12.4) holds for the arcs of positive capacity, along which $d(u) \leq d(v) + 1$; the only arcs carrying flow, those out of s saturated in step 2, give residual arcs into s, whose label is the largest. A residual arc out of a node of level n goes to a node of level n, since it is either an unsaturated arc out of s, whose head step 2 left at level n, or an arc out of a node that the search did not reach, whose head the search did not reach either. A push along uv with $d(v) < d(u)$ may add the residual arc vu, for which $d(v) \leq d(u) + 1$; it moves at most $e_f(u)$ and at most $r_f(uv)$, and f stays a preflow. A relabel sets $d(u) \leq d(v) + 1$ for every residual arc uv and raises $d(u)$, which keeps (12.4) on the arcs into u. When level L becomes empty, a residual arc out of a node u above L goes to a node v with $d(v) \geq d(u) - 1 \geq L$, hence above L, so that (12.4) still holds after those nodes are moved to n. A relabel moves u to at most $h + 1$, since some residual arc out of u reaches a level at most h unless the new label is n; if the old level of u becomes empty, the gap rule lifts u too. Hence, the occupied levels below n stay contiguous from 0, the level of t, which is never relabeled. Since at most $n - 1$ nodes lie below the top, $h \leq n - 2$: a relabel to level $n - 1$ would leave n levels occupied by $n - 1$ nodes, unless the old level became empty, and then the gap rule lifts u to the top. □

Lemma 12.3. *When `startFirstPhase()` ends, the set S of the nodes at level n is a cut, no residual arc of the preflow leaves it, and $e_f(t) = cap(S)$.*

Proof. The phase ends when no node is active, i.e., when every node below level n other than t has a zero excess. We have $s \in S$ and $t \notin S$ by Lemma 12.2. A residual arc from $u \in S$ to $v \notin S$ would need $n \leq d(v) + 1$, i.e., $d(v) = n - 1$, which Lemma 12.2 excludes. Hence, the arcs leaving S are saturated and those entering it are empty. The flow entering $V \setminus S$, which is then $cap(S)$, equals the sum of the excesses of its nodes, i.e., $e_f(t)$. □

The second phase, `startSecondPhase()`, keeps the nodes of $V \setminus S$ at level n and gives to those of S the distances of a breadth first search towards s over the residual arcs (level $n - 1$ to those that do not reach s); then it makes the same pushes and relabels from the active node of highest label, s taking the role of t.

Algorithm 12.2 (Preflow, second phase).

```
1  d(u) := n for u not in S; for u in S, d(u) := BFS distance from u to s
   in the residual digraph, n - 1 if none
2  while some node u != s, t with d(u) < n has e(u) > 0:
3      u := an active node of highest label
4      push from u along residual arcs uv with d(v) < d(u), while e(u) > 0
```

```
5       if e(u) > 0: relabel u as in Algorithm 12.1 (steps 8 and 9)
6   return f, a maximum flow
```

Theorem 12.2. *(Correctness of Preflow.) After* `init()` *and* `startFirstPhase()`, `flowValue()` *is the maximum flow value and* `minCut()` *describes a minimum cut S with $S_{\min} \subseteq S \subseteq S_{\max}$ (Lemma 12.1). After* `startSecondPhase()`, `flowMap()` *is a maximum flow and* `minCut()` *describes the same set S.*

Proof. Consider the second phase. A node of S with positive excess reaches s in the residual digraph. Indeed, if the preflow is decomposed into paths from s, cycles and paths from s that end at nodes with excess, the reverse of a path that ends at u is a residual path from u to s, which stays within S since no residual arc leaves S (Lemma 12.3). The arguments of Lemma 12.2, with s in place of t, show that (12.4) holds throughout the phase. Hence, the label of an active node is at most its residual distance to s, i.e., at most $|S| - 1 < n$, no active node reaches the top, and the branches that the code marks as a "calculation error" are never taken in exact arithmetic. No push crosses the boundary of S, since the nodes of $V \setminus S$ stay at level n and have no excess; hence, the arcs between S and $V \setminus S$ keep their flows. By Theorem 12.3 below the phase ends, with no active node, i.e., with a flow, whose value is the flow entering $V \setminus S$, i.e., $cap(S) = e_f(t)$ by Lemma 12.3. By Theorem 12.1 this flow is maximum and S is a minimum cut, closed in its residual digraph, and Lemma 12.1 gives $S_{\min} \subseteq S \subseteq S_{\max}$. After the first phase `minCut(u)` tests $d(u) = n$, and after the second phase $d(u) < n$, which holds again for the nodes of S and for no other. □

We remark that S need be neither the smallest nor the largest minimum cut: on the path $s \to a \to b \to t$ with unit capacities, e.g., the first phase leaves a and b below the top, so that $S = \{s\} = S_{\min}$, while $S_{\max} = \{s, a, b\}$. In general S depends on the order of the operations.

Theorem 12.3. *(Running time of Preflow.) In each phase there are at most n^2 relabels, at most $m(n+2)$ saturating pushes and at most $n^2 + nm(n+2)$ emptying pushes. Each selection of a node costs $O(\Delta)$ time, where Δ is the largest number of arcs incident to a node, since the code scans the arcs of the node from the first one; hence, each phase takes $O(n^2 m \Delta)$ time.*

Proof. Each label is at most n and only grows (Lemma 12.2), so that each node is relabeled at most n times. Between two saturating pushes along the same residual arc uv, a push along vu must occur. The first push needs $d(u) > d(v)$, the push along vu a label of v above the label of u at that time, and the second push a label of u above that of v; hence, $d(u)$ grows by at least 2. Hence, there are at most $n/2 + 1$ saturating pushes along each of the $2m$ residual arcs. Let Φ be the sum of the labels of the active nodes. An emptying push makes u inactive and may make active v, whose label is smaller, and therefore it decreases Φ by at least 1. A saturating push increases Φ by at most n, the relabels by the growth of the labels, which sums to at most n^2, and a lift to the top only makes nodes inactive. This gives the bound on the emptying pushes. A selection of a node ends with an emptying push or with a relabel, and it scans at most the arcs incident to the node. □

The header states the worst case running time $O(n^2\sqrt{m})$, which is the bound of the highest label rule implemented with a current arc for each node (Cheriyan and Maheshwari 1989; Ahuja, Magnanti and Orlin 1993, chapter 7). The code keeps no current arc, and its first phase is not the pure highest label rule; hence, the proof of that bound does not carry over to it, and Theorem 12.3 is the bound that we can prove for the class as it is. In practice the algorithm is usually the fastest maximum flow method of the library, and the documentation of the module recommends it "in most cases". We remark that the header warns against infinite or very large capacities (e.g., the largest value of the type), since the initial pushes of step 2 and the sums of the excesses at a node must fit in the value type.

12.2.2 The interface

`Preflow<GR, CAP, TR>` takes the type `GR` of the digraph, the type `CAP` of the capacity map (by default `GR::ArcMap< int >`), and a traits class `TR`, by default `PreflowDefaultTraits< GR , CAP >`. These traits define the types `Digraph` and `CapacityMap`, the value type `Value` (that of `CAP`), the type `FlowMap` of the flow map (`GR::ArcMap< Value >`), and the type `Elevator` of the labels (`Elevator< GR , GR::Node >`, whose alternative is `LinkedElevator`). They also define the static functions `createFlowMap(digraph)` and `createElevator(digraph, max_level)` that allocate the last two, and the type `Tolerance` (`Tolerance< Value >`) of the comparisons. The class repeats these types as its own typedefs, together with `Traits`, and its typedef `Create` is the class itself. One rarely sets the traits directly. Instead, the named template parameters `SetFlowMap< T >`, `SetElevator< T >` and `SetStandardElevator< T >` replace one type each, through their member `Create`. With `SetFlowMap` and `SetElevator` the object must be given by `flowMap()` or `elevator()` before `init()`, since the traits that they make cannot create it (they assert); `SetStandardElevator`, instead, makes a type that the class still allocates, from the digraph and the largest level.

The constructor `Preflow(digraph, capacity, source, target)` stores the data by reference; it does not check that the source differs from the target, as the algorithm requires. The setters `capacityMap()`, `flowMap()`, `source()`, `target()`, `elevator()` and `tolerance()` change the data, each returning the object, and `elevator()` and `tolerance()` without arguments return a const reference to the elevator and to the tolerance. A flow map or an elevator that is not given is allocated by `init()` and released by the destructor. The execution follows the pattern of chapter 3.

- `run()` is `init()`, `startFirstPhase()` and `startSecondPhase()`, and `runMinCut()` is `init()` and `startFirstPhase()`, which suffice for the value and for the minimum cut (Theorem 12.2).
- `init()` starts from the zero flow, and `init(flowMap)` from a given map, which must be at least a preflow: after copying the map, it returns `false` if the excess of a node other than s is negative beyond the tolerance; otherwise it computes the labels by a breadth first search from t over the residual arcs, saturates the residual arcs out of s (and empties the arcs into it) towards the nodes that reach t, and returns `true`. It does not check the bounds $0 \le f \le cap$, which the caller must ensure.
- `flowValue()` returns the excess of t, i.e., the maximum value already after the first phase, while `flow(arc)` and `flowMap()` give a maximum flow only after the second phase.
- `minCut(node)` returns `true` if a node is on the source side of the minimum cut found, and `minCutMap(map)` writes that side into a writable `bool` node map, in $O(n)$ time; both are meaningful after the first or the second phase (Theorem 12.2), and the header remarks that the two answers may differ slightly with inexact computation.

A typical call, with a `bool` node map `cut`, is

```
Preflow< ListDigraph > pf( g , cap , s , t );
pf.run();
pf.minCutMap( cut );
```

after which `pf.flowValue()` is the value. The program `manual/examples/12-max-flow.cc` (the test `manual_12_max_flow`) solves the network of Figure 12.1 with `Preflow` and `EdmondsKarp`, checks the value 12 against the capacity of the cut of each, and gives the maximum flow to a second `Preflow`, after the capacity of the arc (d, t) has grown from 6 to 10, whose first phase reaches the value 15.

abLEMON. With a floating-point `Value`, `init(flowMap)` judges the excess of a node relative to the flow through the node, rather than against the absolute epsilon of the tolerance. Hence, the maximum flow of another `Preflow` on large capacities (e.g., of the order of 10^9) is taken as a preflow, while LEMON 1.3.1 refused it because the rounding errors of the sums exceed that epsilon (trac #625). A program may then start from the maximum flow of a previous run after a change of the capacities, provided that the old flow is still a preflow for the new ones (e.g., after some capacities have grown).

12.3 Augmenting paths: EdmondsKarp

Next, the class `EdmondsKarp` implements the algorithm of Edmonds and Karp 1972, i.e., the augmenting path method of Ford and Fulkerson in which each augmenting path is a shortest one (in the number of arcs) of the residual digraph.

Algorithm 12.3 (Edmonds and Karp).

```
1  f := 0 (init()), or a given feasible flow (init(flowMap))
2  repeat:                                              (augment())
3      find by BFS from s a path P to t with the fewest arcs,
       along the residual arcs uv with r(uv) > 0
4      if there is none: stop, f is maximum
5      delta := min{ r(uv) : uv in P }
6      increase f along P by delta (decrease it on the backward arcs)
```

Steps 3 to 6 are one call of `augment()`, which returns `false` when no path is found; `start()` repeats it until then, and `run()` is `init()` followed by `start()`. The search of step 3 stops as soon as t is reached. In our view, the advantage of the class over `Preflow` lies in this control: between two calls of `augment()` the flow is a feasible flow, whose value `flowValue()` returns. One can then stop early, look at the intermediate flows, or interleave the augmentations with some other work, as a teaching or a debugging program may want.

Theorem 12.4. *(Edmonds and Karp.) Each call of* ***`augment()`*** *that returns* ***`true`*** *keeps f feasible and increases its value by $\delta > 0$; when it returns* ***`false`****, f is maximum and the nodes that the search reached form the cut $S_{\min}$*

of Lemma 12.1. From any feasible flow, at most nm calls return `true`*; hence,* `start()` *takes* $O(nm^2)$ *time.*

Proof. The first statement holds since δ is the smallest residual capacity of the path, and the second is Theorem 12.1, the search having visited every node that s reaches. Let $\delta_f(v)$ be the number of arcs of a shortest residual path from s to v. An augmentation along a shortest path P removes from the residual digraph at least one arc of P, and it may add the reverses of arcs of P, each of which goes from a node at distance $k+1$ to one at distance k. Such arcs create no shorter path; hence, $\delta_f(v)$ never decreases. An arc uv of P that an augmentation removes (a *critical* arc) has $\delta_f(v) = \delta_f(u) + 1$; it can be critical again only after an augmentation along a path through vu, which needs $\delta(u) = \delta(v) + 1$ at that time, so that $\delta(u)$ has grown by at least 2. Since a finite distance is below n, each of the $2m$ residual arcs is critical at most $n/2$ times. Hence, there are at most nm augmentations, each of which costs $O(m)$ time for the search and the update. □

The header gives the running time $O(nm^2)$, together with the advice to use `Preflow` when only the maximum flow is wanted. We remark that the bound does not depend on the capacities, and holds for real ones.

The template parameters are those of `Preflow`, with the traits `EdmondsKarpDefaultTraits< GR , CAP >` (the types `Digraph`, `CapacityMap`, `Value`, `FlowMap` with its creator `createFlowMap()`, and `Tolerance`) and the single named parameter `SetFlowMap< T >`; the class has the same typedefs and `Create`. The constructor `EdmondsKarp(digraph, capacity, source, target)` asserts that $s \neq t$ (the setters `source()` and `target()` do not); the setters `capacityMap()`, `flowMap()`, `source()`, `target()` and `tolerance()` return the object, `tolerance()` without arguments returns the tolerance, and the queries `flowValue()`, `flow(arc)`, `flowMap()`, `minCut(node)` and `minCutMap(map)` are as in `Preflow`. The cut is that of the last search, i.e., the set of the nodes reached from s when `augment()` returned `false`, which by Theorem 12.4 is $S_{\min}$. It is meaningful only after such a call: after `init()` only, or after a call that returned `true`, the predecessors of the search describe no cut. In the example of Figure 12.1 the minimum cut is unique, and the program of section 12.2.2 finds it with both classes. Finally, `init(flowMap)` starts from a given flow, and `checkedInit(flowMap)` is kept as another name of the same function.

abLEMON. `init(flowMap)` returns whether the map is a feasible flow, with the tolerance of the algorithm (conservation at the nodes other than s and t, and $0 \le f \le cap$ on the arcs), as `Preflow::init()` does; the flow is set in any case. In LEMON 1.3.1 it returned nothing, and only `checkedInit()` made the same checks.

12.4 Feasible circulations: Circulation

12.4.1 The problem and its certificate

Let $lower : A \to \mathbf{R}$ and $upper : A \to \mathbf{R} \cup \{\infty\}$, with $lower \le upper$, be bounds on the flow of the arcs, and let $sup : V \to \mathbf{R}$ be signed supplies (a node with $sup(u) > 0$ supplies $sup(u)$ units, one with $sup(u) < 0$ demands $-sup(u)$). In the *network circulation problem* of the class `Circulation` one looks for any f such that

$$\sum_{uv \in A} f(uv) - \sum_{vu \in A} f(vu) \ge sup(u) \quad \forall u \in V, \qquad lower(uv) \le f(uv) \le upper(uv) \quad \forall uv \in A. \tag{12.5}$$

The system can be feasible only if $\sum_{u \in V} sup(u) \le 0$, since its left-hand sides sum to zero; when the supplies sum to zero, the inequalities hold as equalities, and (12.5) defines the feasible flows with supplies and demands. One obtains the opposite inequalities (all the demands met, some supply left) by reversing the arcs and negating the supplies, as the header suggests, e.g., with the adaptor `ReverseDigraph` of chapter 5 and the map `NegMap` of chapter 6. As a certificate of infeasibility the class gives a *barrier*, i.e., a set $B \subseteq V$ whose supply exceeds what can leave it:

$$\sum_{uv \in A,\ u \in B,\ v \notin B} upper(uv) - \sum_{uv \in A,\ u \notin B,\ v \in B} lower(uv) < \sum_{v \in B} sup(v). \tag{12.6}$$

The theorem of Hoffman states that a barrier exists whenever (12.5) has no solution. Let $sup'(u) = sup(u) - \sum_{uv \in A} lower(uv) + \sum_{vu \in A} lower(vu)$ be the supplies net of the lower bounds, and let $x^+ = \max\{x, 0\}$ and $x^- = \max\{-x, 0\}$. The proof uses the network N that has the arcs of G, with the capacities $upper - lower$, a source σ with an arc to each node u of capacity $sup'(u)^+$, and an arc from each node u to a target τ of capacity $sup'(u)^-$. Let $P = \sum_{u \in V} sup'(u)^+$. A direct expansion of (12.2), in which the lower bounds of the arcs inside B cancel, gives for each $B \subseteq V$

$$P - cap(\{\sigma\} \cup B) = \sum_{v \in B} sup(v) - \sum_{uv \in A,\ u \in B,\ v \notin B} upper(uv) + \sum_{uv \in A,\ u \notin B,\ v \in B} lower(uv), \tag{12.7}$$

i.e., B is a barrier if and only if $cap(\{\sigma\} \cup B) < P$.

Theorem 12.5. *(Hoffman.) The system (12.5) has a solution if and only if no set $B \subseteq V$ satisfies (12.6).*

Proof. If f solves (12.5), summing its inequalities over the nodes of B gives $\sum_{v \in B} sup(v) \le \sum_{u \in B,\, v \notin B} f(uv) - \sum_{u \notin B,\, v \in B} f(uv)$, which is at most the left-hand side of (12.6). Conversely, suppose that a maximum flow x of N from σ to τ has value P, i.e., saturates every arc out of σ. At each node u the conservation in N gives $\sum_{uv \in A} x(uv) - \sum_{vu \in A} x(vu) = sup'(u)^+ - x(u\tau) \ge sup'(u)^+ - sup'(u)^- = sup'(u)$, and $f = x + lower$ solves (12.5). Hence, if (12.5) has no solution, the maximum flow value is below P, and by Theorem 12.1 some cut $\{\sigma\} \cup B$ of N has a capacity below P, which is finite (no arc of infinite upper bound leaves B). By (12.7), B satisfies (12.6). □

This is the circulation theorem of Hoffman 1960, which in the form with supplies is the feasibility theorem of Gale 1957.

12.4.2 The algorithm and its interface

`Circulation` is a push-relabel algorithm written directly for (12.5). In it, the excess of a node is $e(u) = sup(u)$ plus the flow into it less the flow out of it; hence, a node of positive excess is one whose inequality is violated. A residual arc is an arc uv with $f(uv) < upper(uv)$, or the reverse of an arc vu with $f(vu) > lower(vu)$, as in (12.3) with the bounds of (12.5). The flow starts at the lower bounds (`init()`) or at a greedy choice (`greedyInit()`), which sets each arc, in the order of the arcs, to the deficit $-e(v)$ of its head v at that moment, clipped to the bounds of the arc. All the nodes start at level 0, and the active nodes are those of positive excess. A comment in `init()` records that a global relabeling was tried and gave somewhat worse running times on random digraphs.

Algorithm 12.4 (Circulation).

```
1  set f by the greedy rule; e(u) := sup(u) + f(in(u)) - f(out(u))
2  d(u) := 0 for each node u
3  while some node has e(u) > 0:
4      u := an active node of highest level
5      push from u along residual arcs uv with d(v) < d(u), while e(u) > 0
6      if e(u) > 0:
7          if no residual arc leaves u: lift u to level n; L := n;
              return false
8          L := d(u); d(u) := 1 + min{ d(v) : uv residual }
9          if level L is now empty: return false
10 return true            (the barrier is { u : d(u) >= L } after false)
```

Steps 1 and 2 are `greedyInit()`, and steps 3 to 10 are `start()`.

Theorem 12.6. *(Correctness and running time of Circulation.) `start()` ends after at most $n(n-1)$ relabels, $O(nm)$ saturating pushes and $O(n^2m)$ emptying pushes, i.e., in $O(n^2m\Delta)$ time (Δ as in Theorem 12.3). If it returns `true`, the flow satisfies (12.5) and the barrier is empty; if it returns `false`, the barrier $\{u : d(u) \ge L\}$ satisfies (12.6).*

Proof. The flow stays within the bounds, since the greedy rule clips each value and a push moves at most the residual capacity, and (12.4) holds throughout by the arguments of Lemma 12.2, all the labels being 0 at the start. A push moves at most the excess of its tail; hence, a node whose excess is nonnegative never gets a negative one, and a node with a negative excess has never been active, so that it is still at level 0. The occupied levels are $0, \dots, h$: a relabel of u sets $d(u)$ to one more than the level of a node at a level at least $d(u)$, i.e., to at most $h+1$, and if the old level becomes empty the algorithm stops. As long as it does not stop, each level up to h holds a node, so that $h \le n-1$, and a relabel to n would need one node on each level, u the only node of its old one, which step 9 catches. Hence, while `start()` runs, the labels stay below n and each node is relabeled at most $n-1$ times; the counts of the pushes and the time follow as in the proof of Theorem 12.3. If `start()` returns `true`, every excess is at most zero, which is (12.5), and no node is at level n; hence, the barrier is empty. If it returns `false` in step 9, a residual arc out of a node u above L goes to a node v with $d(v) \ge d(u) - 1 \ge L$, and level L is empty; hence, no residual arc leaves $B = \{u : d(u) \ge L\}$. In step 7 the barrier is $\{u\}$, which no

residual arc leaves either, since no other node is at level n. In both cases the arcs leaving B are at their upper bounds and those entering it at their lower bounds, and therefore $\sum_{v \in B} e(v)$ is the right-hand side less the left-hand side of (12.6). Every node of B has a nonnegative excess (the nodes of negative excess are at level 0, and if $L = 0$ level 0 is empty), and u a positive one; hence, (12.6) holds. □

The header states no bound on the running time; Theorem 12.6 is the one that the code satisfies. The template parameters of the class are `GR`, the lower bound map `LM` (by default `GR::ArcMap< int >`), the upper bound map `UM` (by default `LM`), the supply map `SM` (by default `GR::NodeMap< UM::Value >`), whose value type is the `Value` of the algorithm, and the traits `CirculationDefaultTraits< GR , LM , UM , SM >`. These define the types `Digraph`, `LowerMap`, `UpperMap`, `SupplyMap`, `Value`, `FlowMap`, `Elevator` and `Tolerance` and the creators `creat eFlowMap()` and `createElevator()`, as those of `Preflow` do. Its named parameters `SetFlowMap< T >`, `SetE levator< T >` and `SetStandardElevator< T >` are those of `Preflow`, and its constructor `Circulation(gra ph, lower, upper, supply)` takes the three maps by reference, which `lowerMap()`, `upperMap()` and `supply Map()` change, while `flowMap()`, `elevator()` and `tolerance()` are as in `Preflow`. In a debug build, `init()` and `greedyInit()` check that no upper bound is below its lower bound. `run()` is `greedyInit()` followed by `s tart()`, and it returns whether a feasible circulation was found; in that case `flow(arc)` and `flowMap()` give it, and otherwise `barrier(node)` and `barrierMap(map)` give the barrier (which is empty after a feasible run). Two checkers verify an answer on the data: `checkFlow()` tests the bounds exactly and the inequalities of (12.5) up to the tolerance, and `checkBarrier()` tests (12.6) on the barrier found, with the tolerance. In the latter, an upper bound out of the set that is infinite (or so large that the sum would overflow) makes the test fail. The program `manual/examples/12-circulation.cc` (the test `manual_12_circulation`) solves a network of four nodes and then lowers the capacity of the arc (b, d) from 4 to 1, after which the node d, which demands 3 units, can receive at most 2; `run()` returns `false`, and the program checks (12.6) on the barrier.

abLEMON. On a digraph without nodes `Circulation` read the first of no items of its elevator in LEMON 1.3.1; now the elevator holds one item more than the digraph has, and the empty digraph has the empty circulation.

12.5 Tolerances and the common interface

All three classes compare the values of the flows through a `Tolerance< Value >` (see chapter 3), which is set and read by `tolerance()`. A residual capacity counts as positive only beyond the epsilon of the tolerance, and so does an excess; also, a push empties the node unless the residual capacity is less than the excess by more than the epsilon. For an integer `Value` the comparisons are exact, and the theorems of this chapter apply as stated. For a floating-point one the default epsilon of `Tolerance` (10^{-10} for `double`) is absolute, which usually suits capacities of moderate size, and a program with data of a rather different scale should set another one. The theorems then hold only up to the rounding errors; this is why the second phase of `Preflow` handles the cases that exact arithmetic excludes. Indeed, the only comparison that is relative to the data is the one of `Preflow:: init(flowMap)` (cf. section 12.2.2).

`Preflow` and `EdmondsKarp` meet the concept `concepts::MaxFlow< GR , CAP >` of `lemon/concepts/max_flo w.h`, against which a class is checked by `checkConcept` (see chapter 3). The concept requires the member `Cre ate` and the named parameter `SetFlowMap`, the constructor from the digraph, the capacities, the source and the target, the setters `capacityMap()`, `flowMap()`, `source()`, `target()` and `tolerance()`, the type `Toleran ce` and the getter `tolerance()`. It also requires the two `init()` functions and `run()`, and the queries `flowV alue()`, `flow()`, `flowMap()`, `minCut()` and `minCutMap()`; a function template written against these members accepts either class.

abLEMON. The concepts of the maximum flow, minimum cost flow and minimum mean cycle classes are in `lemon/concepts` (`MaxFlow`, `MinCostFlow` and `MinMeanCycle`), where the classes of the library are held to them; in LEMON 1.3.1 they were defined inside the tests.

12.6 Exercises

Exercise 12.1. Modify the program `manual/examples/12-max-flow.cc` so that it stops `EdmondsKarp` after its first augmenting path and prints the residual capacities (12.3) of the arcs. Then check by hand that the value of the flow and the number of paths agree with Algorithm 12.3. (The program counts the paths.)

Exercise 12.2. On the network of Figure 12.1, add an arc that makes the sets $S_{\min}$ and $S_{\max}$ of Lemma 12.1 differ, and compare `minCutMap()` of `Preflow` after `runMinCut()` with that of `EdmondsKarp`; by Theorems 12.2 and 12.4, the first lies between the two sets and the second is $S_{\min}$.

Exercise 12.3. Give `Preflow::init(flowMap)` a map that is not a preflow (e.g., one unit more out of a node than into it), and check that it returns `false`. Then repeat with capacities of the order of 10^9 in `double`, where the absolute epsilon of the tolerance would refuse a true preflow. (Solved by `checkInitPreflowLarge()` in `test/max_flow_test.cc`.)

Exercise 12.4. Write a function template that takes a class meeting `concepts::MaxFlow`, runs it, and returns the capacity (12.2) of its cut minus its value; instantiate it with both `Preflow` and `EdmondsKarp`. (The concept is checked in `test/max_flow_test.cc`.)

Exercise 12.5. Solve a problem with the opposite inequalities of (12.5), i.e., all the demands met and some supply left, by `Circulation` on a `ReverseDigraph` with a `NegMap` of the supplies, and verify the answer with `checkFlow()`. (The class is tested in `test/circulation_test.cc`.)

Exercise 12.6. Prove the identity (12.7) by expanding the capacity of the cut $\{\sigma\} \cup B$, and check it, together with (12.6), on the barrier of the second network of `manual/examples/12-circulation.cc`.

13. Minimum cost flows

We are concerned in this chapter with the four minimum cost flow algorithms of the library, `NetworkSimplex` (`lemon/network_simplex.h`), `CostScaling` (`lemon/cost_scaling.h`), `CapacityScaling` (`lemon/capacity_scaling.h`) and `CycleCanceling` (`lemon/cycle_canceling.h`), and with the concept `concepts::MinCostFlow` of `lemon/concepts/min_cost_flow.h` that they meet. We also describe two internal headers that a user meets through their effects, i.e., the default tolerances of `lemon/bits/mcf_tolerance.h`, and the starting flow and the bound on the infinite capacities of `lemon/bits/mcf_feasible_flow.h`. Section 13.1 states the problem in the forms that the library takes, together with its optimality conditions; section 13.2 describes the interface that the four classes share, and section 13.3 the way in which they handle floating-point data. Sections 13.4 to 13.8 describe the algorithms one by one, and section 13.9 gives some advice on the choice among them. In chapter 14 we treat the certificates that the four classes give when a problem has no optimal flow, and the classes that enumerate several flows. As in chapter 12, the results are stated in exact arithmetic, and section 13.3 describes what the tolerances change in floating point.

13.1 The problem, its forms and its dual

Let $G = (V, A)$ be a digraph with n nodes and m arcs, and let $lower : A \to \mathbf{R}$ and $upper : A \to \mathbf{R} \cup \{+\infty\}$, with $lower \le upper$, be bounds on the flow of the arcs. Let also $cost : A \to \mathbf{R}$ be the cost of a unit of flow on each arc, and $sup : V \to \mathbf{R}$ the signed supplies of the nodes (a supply if positive, a demand if negative). Following Ahuja, Magnanti and Orlin 1993 and the page `doc/min_cost_flow.dox` of the documentation, the library solves the problem in the form with "greater or equal" supply constraints (GEQ):

$$\begin{aligned} \min \quad & \sum_{uv \in A} cost(uv)\, f(uv) \\ & \sum_{uv \in A} f(uv) - \sum_{vu \in A} f(vu) \ge sup(u) \qquad \forall u \in V \\ & lower(uv) \le f(uv) \le upper(uv) \qquad \forall uv \in A. \end{aligned} \tag{13.1}$$

Since the left-hand sides sum to zero, (13.1) can be feasible only if $\sum_{u \in V} sup(u) \le 0$. Thus, the supplies must all leave their nodes, while some demand may be left unmet. When the supplies sum to zero, the constraints hold as equalities, and (13.1) is the usual equality form of the problem; a user who wants that form must therefore make the supplies sum to zero. In addition, `NetworkSimplex` takes the opposite form with "less or equal" constraints (LEQ),

$$\sum_{uv \in A} f(uv) - \sum_{vu \in A} f(vu) \le sup(u) \qquad \forall u \in V, \tag{13.2}$$

which needs $\sum_{u \in V} sup(u) \ge 0$, and in which the demands must be met while some supply may stay at its node. The other three classes take only the GEQ form; one can bring the LEQ form to it by reversing the arcs and negating the supplies, e.g., with `ReverseDigraph` (chapter 5) and `NegMap` (chapter 6).

In the dual problem the variables are the *potentials* $\pi : V \to \mathbf{R}$ of the nodes, and the *reduced cost* of an arc under them is

$$cost^{\pi}(uv) = cost(uv) + \pi(u) - \pi(v). \tag{13.3}$$

Hence, the potentials are the opposite of the prices of the textbook convention, in which $cost^\pi(uv) = cost(uv) - \pi(u) + \pi(v)$; we keep the sign of the library throughout. We write $N_f(u) = \sum_{uv \in A} f(uv) - \sum_{vu \in A} f(vu)$ for the net flow out of u. The *residual network* of a feasible flow f has an arc uv of residual capacity $upper(uv) - f(uv)$ and cost $cost(uv)$ wherever $f(uv) < upper(uv)$, and an arc vu of residual capacity $f(uv) - lower(uv)$ and cost $-cost(uv)$ wherever $f(uv) > lower(uv)$. The optimality conditions rest on an identity, which one obtains by expanding (13.3) and collecting the terms of each node: for any flow g and any potentials π,

$$\sum_{uv \in A} cost(uv)\, g(uv) = \sum_{uv \in A} cost^\pi(uv)\, g(uv) - \sum_{u \in V} \pi(u)\, N_g(u). \tag{13.4}$$

Theorem 13.1. *A feasible flow f of (13.1) is optimal if and only if there are potentials π such that, for each arc uv and each node u,*

$$\begin{aligned} &cost^\pi(uv) > 0 \Rightarrow f(uv) = lower(uv), \qquad cost^\pi(uv) < 0 \Rightarrow f(uv) = upper(uv), \\ &\pi(u) \le 0, \qquad N_f(u) \ne sup(u) \Rightarrow \pi(u) = 0. \end{aligned} \tag{13.5}$$

For the LEQ form the same holds with $\pi(u) \ge 0$ in place of $\pi(u) \le 0$. If the supplies sum to zero, the conditions on the signs of the potentials may be dropped.

Proof. Sufficiency. For a feasible flow g, (13.4) gives $\sum cost \cdot g = \sum cost^\pi g - \sum \pi(u) N_g(u)$. Each term of the first sum is at least $cost^\pi(uv)\, lower(uv)$ if $cost^\pi(uv) > 0$ and at least $cost^\pi(uv)\, upper(uv)$ if $cost^\pi(uv) < 0$, and each term $-\pi(u)N_g(u)$ is at least $-\pi(u) sup(u)$, since $\pi(u) \le 0$ and $N_g(u) \ge sup(u)$. The conditions (13.5) state that f attains all these lower bounds, so that no feasible flow costs less than f. Necessity. Add a node r of supply $-\sum_{u \in V} sup(u) \ge 0$ and, for each node u, an arc ru of cost 0 and infinite capacity. A flow f of (13.1) gives a flow of the equality form of this network, with $N_f(u) - sup(u)$ on the arc ru, and conversely; the two problems have the same optimal flows. If f is optimal, its residual network in the larger problem has no cycle of negative cost, since one unit (or the smallest residual capacity, if smaller) along such a cycle would decrease the cost. Hence, the cost $d(u)$ of a shortest residual path to u from an extra node joined to all the nodes by arcs of cost 0 is finite and nonpositive. Also, $d(v) \le d(u) + c$ for each residual arc uv of cost c, i.e., the reduced costs under d are nonnegative on the residual arcs. An arc with $f < upper$ thus has $cost^d \ge 0$, and one with $f > lower$ has $cost^d \le 0$, which are the conditions of (13.5) on the arcs. The arc ru is always residual, so that $d(u) \le d(r)$, and if it carries flow, i.e., if $N_f(u) \ne sup(u)$, its reverse is residual too, so that $d(u) = d(r)$; hence, $\pi = d - d(r)$ satisfies (13.5). For the LEQ form one adds arcs ur instead, and the signs change. When the supplies sum to zero, the last sum of (13.4) is the same for every feasible flow, and the argument holds for potentials of any sign. □

This is the duality theorem of linear programming for (13.1); see Ahuja, Magnanti and Orlin 1993, chapter 9. Its proof also gives the following characterization, due to Klein 1967, which underlies the cycle canceling methods of section 13.8 and the final potentials of cost scaling.

Theorem 13.2. *(Negative cycle optimality.) A feasible flow of the equality form is optimal if and only if its residual network has no directed cycle of negative cost. Then (13.5) holds for the potentials d, where $d(u) \le 0$ is the cost of a shortest residual path to u from an extra node joined to all the others by arcs of cost zero.*

Proof. A negative cycle allows a cheaper flow, as in the proof of Theorem 13.1. If there is none, the distances d are finite and give nonnegative reduced costs on the residual arcs, i.e., the conditions of (13.5) on the arcs, and $d \le 0$; by the sufficiency part of Theorem 13.1, in which the signs do not matter when the supplies sum to zero, f is optimal. □

Three outcomes are possible, and `run()` returns them as the constants of the nested enum `ProblemType` of each class. With `OPTIMAL` a flow and potentials meeting (13.5) were found; with `INFEASIBLE` no flow satisfies the constraints; and with `UNBOUNDED` the problem is feasible, while a directed cycle of negative cost, whose arcs have infinite upper bounds, lets the cost decrease without limit. An infinite upper bound is the constant `INF` of the class, i.e., the infinity of `Value` if the type has one and its largest value otherwise. The four algorithms only add and subtract residual capacities, supplies and bounds (and the bound B of section 13.5, which is integer on integer data); hence, on integer data they keep the flow integer, and the optimal flow that they return is integer. In particular, an integer problem that has an optimal flow has an integer one.

13.2 The common interface

All four classes take as template parameters the type `GR` of the digraph, the number type `V` of the flows, of the bounds and of the supplies (by default `int`), and the number type `C` of the costs and of the potentials (by default `V`). They name them `Value` and `Cost`, and both must be signed, as the constructors assert. `CostScaling` and `CapacityScaling` take a fourth parameter, a traits class, which we describe with them. The constructor takes the digraph, which must not change afterwards unless `reset()` is called; that of `NetworkSimplex` takes a second argument, `arc_mixing` (section 13.4). The parameters of the problem are then given by functions that copy the data at once, so that the maps need not outlive the call; each returns the object.

- `lowerMap(map)` sets the lower bounds, zero on the arcs if it is not called.
- `upperMap(map)` sets the upper bounds, `INF` on the arcs if it is not called.
- `costMap(map)` sets the costs, 1 on the arcs if it is not called.
- `supplyMap(map)` sets the supplies, zero if neither it nor `stSupply()` is called, and `stSupply(s, t, k)` gives k to s, $-k$ to t and zero to the other nodes.
- `supplyType(type)`, in `NetworkSimplex` only, sets the form, `GEQ` (the default) or `LEQ`, constants of the nested enum `SupplyType`.

Then `run()` solves the problem and returns its `ProblemType`, with arguments that differ from class to class (Table 13.1). After a run, the queries are `totalCost()`, i.e., the cost of the flow in $O(m)$ time, `flow(arc)` and `flowMap(map)`, which give the flow, and `potential(node)` and `potentialMap(map)`, which give the potentials. With a template argument, e.g., `totalCost< double >()`, the sum is made in another type, which helps when it may overflow `Cost`. We describe the certificates `infeasibleCut()`, `infeasibleCuts(k)`, `disjointInfeasibleCuts()`, `unboundedCycle()` and `unboundedCycles(k)`, which are meaningful after a run that returned `INFEASIBLE` or `UNBOUNDED`, in chapter 14, and the tolerances in section 13.3.

One may call `run()` again: the parameters are kept, and only those that changed have to be given anew. `resetParams()` sets the parameters back to their defaults (and the supply type to `GEQ`), while `reset()` also rebuilds the internal copy of the digraph, as is needed when the digraph itself was modified after the construction or after the last `reset()`. In any case a second run solves the problem from scratch, whatever changed. A typical use is

```
NetworkSimplex< ListDigraph > ns( g );
ns.upperMap( cap ).costMap( cost ).supplyMap( sup );
if( ns.run() == NetworkSimplex< ListDigraph >::OPTIMAL ) c = ns.totalCost();
```

and the other three classes are called in the same way.

Table 13.1. The four classes: the arguments of `run()` with their defaults, the worst case running time stated in each header (U is the largest supply or capacity, K the largest absolute value of a cost), the supply types, and the template parameters beyond `GR`, `V` and `C`.

Class	`run()`	Worst case	Forms	Traits
`NetworkSimplex`	`pivot_rule = BLOCK_SEARCH`	not stated	GEQ, LEQ	none
`CostScaling`	`method = PARTIAL_AUGMENT`, `factor = 16`	$O(n^2 m \log(nK))$	GEQ	`LargeCost`
`CapacityScaling`	`factor = 4`	$O(m \log U\,(n+m)\log n)$	GEQ	`Heap`
`CycleCanceling`	`method = CANCEL_AND_TIGHTEN`	$O(n^2 m^2 \log n)$	GEQ	none

All four meet the concept `concepts::MinCostFlow< GR , V , C >` of `lemon/concepts/min_cost_flow.h`. It requires the constructor from the digraph, `reset()`, `resetParams()`, the five setters of the parameters (without `supplyType()`), `run()` with its defaults, the queries above, the tolerance functions and types of section 13.3, and the single certificates `infeasibleCut()` and `unboundedCycle()`. A function template written against these members accepts any of the four, as does the function `optimal()` of the program `manual/examples/13-min-cost-flow.cc` (the test `manual_13_min_cost_flow`). This program solves a transshipment problem of seven nodes with the four classes, and it checks that their costs agree (36) and that the flow and the potentials of each meet (13.5). Then it runs `NetworkSimplex` again after a change of one cost (66); finally it gives two more units of supply to one node, which makes the problem infeasible in the GEQ form and leaves an optimal flow, with nonnegative potentials, in the LEQ form.

abLEMON. The four classes answer `OPTIMAL`, with the empty flow, on a digraph without nodes, which LEMON 1.3.1 reported as infeasible. They answer `UNBOUNDED` on the same problems, i.e., the feasible ones

with a cycle of negative cost made of arcs of infinite upper bound: in LEMON 1.3.1 `CostScaling` and `CycleCanceling` gave that answer on any arc of negative cost and infinite upper bound, `CapacityScaling` gave a wrong answer on feasible problems that its finite replacement of the infinite bounds made infeasible, and `NetworkSimplex` did not check feasibility first. The function `runWarm()`, which versions 1.4.0 to 1.5.0 added to re-optimize from the last basis, flow or potentials, was removed in 1.6.0: a re-optimization belongs to whoever knows what has changed, and a second `run()` solves the changed problem.

13.3 Tolerances and floating-point data

With integer types the data must be integer, and the comparisons are exact. With floating-point types the data need not be integer; however, a comparison of a residual capacity or of a reduced cost with an exact zero would make the algorithms pivot or push round-off for ever, or find a feasible problem infeasible. Instead, the four classes compare against two tolerances, one on the flows and one on the reduced costs, defined once in `lemon/bits/mcf_tolerance.h`. Unless the caller sets them, they are zero for an exact type and otherwise

$$\epsilon_f = 10^6\,\varepsilon\,\max\{1,\ |sup(u)|,\ |lower(uv)|,\ |upper(uv)| < \infty\}, \qquad \epsilon_c = 10^6\,\varepsilon\,\max\{1,\ |cost(uv)|\}, \tag{13.6}$$

where ε is the machine epsilon of the type (about $2.2 \cdot 10^{-16}$ for `double`), and the maxima run over the nodes and the arcs, with the infinite upper bounds (and the values equal to the largest of the type) left out. In other words, the tolerances are 10^6 units in the last place of the largest datum, or of 1 when the data are smaller. An excess, a deficit or a residual capacity within ϵ_f is taken as zero, and a reduced cost above $-\epsilon_c$ as nonnegative. Also, a sum of the supplies within ϵ_f of zero is taken as zero, since supplies such as 0.1, 0.2 and -0.3, which sum to zero in decimal, in binary sum to a few units in the last place of either sign.

A call of `tolerances(flow, cost)` sets the two tolerances, and a negative value, the default, leaves them to (13.6). In turn, `flowTolerance()` and `costTolerance()` return the tolerances in use, as a `Tolerance< Value >` and a `Tolerance< Cost >` (types named `FlowTolerance` and `CostTolerance` in each class) whose epsilon is zero for an exact type. We remark that `NetworkSimplex` computes them from its data at any time, while the other three return those that the last `run()` used, and zero before the first one. A tolerance of zero reproduces the behavior of LEMON 1.3.1, in which rounding errors may keep the algorithm going for ever; hence, one should set it only on data known to be exact in binary. The theorems of this chapter hold in floating point only up to these tolerances. The program `manual/examples/13-floating-point.cc` (the test `manual_13_floating_point`) solves with the four classes a problem whose supplies are 0.1, 0.2 and -0.3 and whose capacities and costs are fractional. It checks the optimal cost, 0.14, and the tolerances (13.6), which here are $10^6\varepsilon$ on the flows (the data being below 1) and $1.1 \cdot 10^6\varepsilon$ on the costs; then it sets other tolerances, which `flowTolerance()` returns.

abLEMON. The four algorithms solve problems with floating-point costs, capacities and supplies. In LEMON 1.3.1 `NetworkSimplex` compared reduced costs and residual flows with an exact zero, and typically pivoted for ever or reported infeasibility on such data, while the other three required integer data and answered `INFEASIBLE` on supplies that sum to zero in decimal but not in binary. The default tolerances have one definition for the four (version 1.9.0), those of `NetworkSimplex` on the flows counting the finite bounds too, and `tolerances()`, `flowTolerance()` and `costTolerance()` are new.

13.4 The network simplex method

13.4.1 Bases, pivots and potentials

`NetworkSimplex` implements the primal network simplex method (Dantzig 1963; Ahuja, Magnanti and Orlin 1993, chapter 11; Kelly and O'Neill 1991), i.e., the simplex method of linear programming specialized to (13.1), in which a basis is a spanning tree. After moving the lower bounds into the supplies, the class adds an artificial root node r and, for each node, an artificial arc between it and the root, oriented by the sign of its supply; the initial basis is the tree of these arcs, which carry the supplies. When the supplies do not sum to zero, a second set of artificial arcs, of cost zero and infinite capacity, lets the nodes keep the unmet demand (GEQ, arcs ru) or the unused supply (LEQ, arcs ur). The artificial arcs of the tree whose flow would leave a constraint violated (under GEQ those into the root, under LEQ and in the balanced case those out of it) have a large cost M, while the others cost zero. For an exact `Cost`, M is half of the largest value of the type plus one; otherwise $M = (1+K)\,n$, where $K = \max_{uv} |cost(uv)|$. Thus, the method is the "big-M" variant of the simplex method on an auxiliary problem of $n+1$ nodes in the equality form, which is always feasible, and the problem is infeasible exactly when some flow is left at the end on the artificial arcs that may carry none (Theorem 13.3).

Each nonbasic arc is at its lower or at its upper bound, and the potentials are those that give a zero reduced cost to the arcs of the tree; they are unique up to a constant, since the tree spans the nodes. A nonbasic arc e is *eligible* if its reduced cost (13.3) is negative at its lower bound, or positive at its upper bound. Sending one unit around the cycle that e closes with the tree, in the direction allowed by its bound, changes the cost by its reduced cost (taken with the opposite sign at the upper bound), since the tree arcs have a zero reduced cost; hence, an eligible arc is one along whose cycle the cost decreases. A *pivot* chooses an eligible arc by the pivot rule, and finds its cycle through the *join* node, i.e., the first common ancestor of its ends in the tree. Then it sends around the cycle the largest amount δ that the bounds allow, and it replaces by the entering arc the last arc of the cycle to reach a bound, the cycle being traversed in the direction of the flow from the join node (Figure 13.1). An amount of zero (a *degenerate* pivot) changes the basis but not the flow, while an unbounded amount (a cycle of arcs with infinite upper bounds and negative cost) means that the auxiliary problem is unbounded.

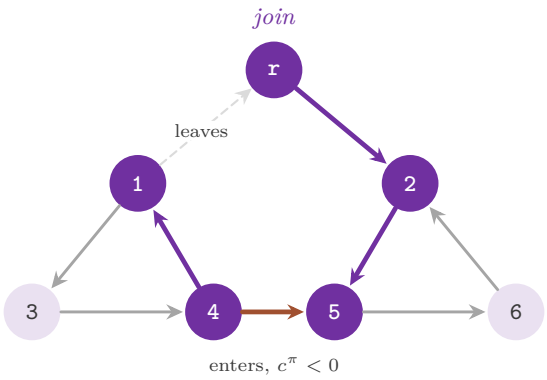


Figure 13.1. A pivot of the network simplex method: the tree of the basis hung from the root r (in the primary colour), the entering arc (4,5) of negative reduced cost, which closes a cycle with the tree path from 5 through the join node r back to 4, and the leaving arc (1,r), the arc of the cycle that reaches a bound.

Algorithm 13.1 (Network simplex).

```
1  if the sum of the supplies has the wrong sign for the supply type:
   return INFEASIBLE; move the lower bounds into the supplies
2  T := tree of the artificial arcs from and to the root; set the flows,
   the potentials and the thread of T
3  make the heuristic initial pivots (see below)
4  while the pivot rule finds an eligible arc e:
5      w := join node of the ends of e in T
6      delta := largest amount around the cycle of e and T;
       f := last arc of the cycle, from w in the direction of the flow,
       to reach a bound (f = e if e reaches its other bound first)
7      if delta is infinite: go to 12
8      send delta around the cycle; if f != e: T := T - f + e
9      update the potentials of the side that f splits off, or of the rest
10 if some artificial arc that may carry no flow carries some: INFEASIBLE
11 put the lower bounds back, shift the potentials; return OPTIMAL
12 solve the problem again with zero costs: if it is infeasible, return
   INFEASIBLE; otherwise solve it again with its costs: return UNBOUNDED
```

Step 6 is `findLeavingArc()`, which scans the two sides of the cycle with a strict and a weak comparison so as to keep the last blocking arc, and step 9 is `updatePotential()`. Step 10 checks the flow against ϵ_f, step 11 is the end of `start()`, and step 12 is `feasibleUnbounded()`. When the supplies sum to zero, step 11 shifts the potentials so that their largest value is at most zero under GEQ, and their smallest at least zero under LEQ, as (13.5) requires.

Lemma 13.1. *Let the subtree moved by a pivot be the part of the new tree hanging from the end u of the entering arc that was split off by the leaving arc, and let σ be the change of $\pi(u)$ that makes the entering arc's reduced cost zero. Adding σ to the potentials of the subtree, or subtracting σ from those of all the other nodes, gives potentials under which every arc of the new tree has a zero reduced cost.*

Proof. The arcs of the new tree are those of the old tree other than f, which have a zero reduced cost and

either both ends in the subtree or both ends outside it, together with the entering arc, which joins the two parts. Adding the same constant to the potentials of one part leaves the reduced costs of the arcs within each part unchanged, and σ makes the reduced cost of the entering arc zero; subtracting σ from the other part changes all the potentials by the constant $-\sigma$ with respect to the first choice. □

The code takes the second choice when the subtree holds more than half of the $n+1$ nodes, provided that the potential of the root stays within $M/2$ of zero for an exact `Cost` (within M otherwise). At the end, `alignPotential()` shifts all the potentials so that the one of the root is zero again. The next theorem gives the correctness of the answers; we assume, as holds by construction in floating point and for an exact `Cost` whenever nK is below half of its largest value, that $M > nK$.

Theorem 13.3. *(Correctness of NetworkSimplex.) Suppose that the pivots end. If no arc is eligible, then the problem is infeasible if and only if some artificial arc of cost M carries flow, and otherwise the flow and the shifted potentials satisfy (13.5) for the supply type. If a pivot has an infinite amount, its cycle is a cycle of negative cost made of arcs of G of infinite upper bound, and the answer of step 12 is right.*

Proof. Let c_0 be the costs with zero on the arcs of cost M, and c_1 the costs with one on those arcs and zero elsewhere, so that the cost of every arc is $c_0 + Mc_1$. By the linearity of the system that defines them, the tree potentials are $\pi = \pi_0 + My$, where π_0 and y are the tree potentials for c_0 and c_1 with the root at zero. Each tree path from the root contains at most one arc of cost M; hence, y takes the values -1, 0 and 1. The difference $\pi_0(u) - \pi_0(v)$ is the cost of the tree path between u and v, whose arcs of nonzero cost are arcs of G, at most $n-1$ of them; hence, the reduced cost of each arc under π_0 for c_0 is at most $nK < M$ in absolute value. Therefore, if the reduced cost of an arc under y for c_1 is nonzero, i.e., at least 1 in absolute value, it decides the sign of its reduced cost under π. If no arc is eligible, the nonbasic arcs at the lower bound have nonnegative reduced costs and those at the upper bound nonpositive ones, both under π and, by the remark above, under y for c_1, while the tree arcs have zero reduced costs under both. By Theorem 13.1 (equality form), the flow minimizes the total flow on the arcs of cost M in the auxiliary problem. The original problem is feasible if and only if that minimum is zero, since the arcs of cost zero take exactly the slack of the GEQ or LEQ constraints; in the balanced case the conservation at the root forces the remaining artificial arcs to zero as well. This proves the first part. If the minimum is zero, the flow is optimal for the auxiliary problem by Theorem 13.1, and its restriction to G, with the lower bounds put back, is optimal for the original one. Under GEQ with unbalanced supplies the arcs ru of cost zero have a nonnegative reduced cost, $\pi(r) - \pi(u) \geq 0$, which is zero when they carry flow, i.e., when the constraint of u is slack; with $\pi(r) = 0$ this is (13.5), and LEQ is symmetric, while in the balanced case the shift of step 11 gives the signs. Finally, an infinite amount means that every arc of the cycle is traversed forward and has an infinite capacity, and that its cost is negative, since the entering arc is eligible. A cycle through the root would traverse forward an arc out of the root and an arc into it. One of them has cost M (under GEQ the arcs into the root, under LEQ and in the balanced case those out of it), and the cycle would cost at least $M - nK > 0$. Hence, the cycle lies in G. If the problem is feasible, it is unbounded along this cycle; whether it is feasible is decided in step 12 by the same method on zero costs, which is never unbounded, and if it is, the run with the costs repeats the same pivots and ends unbounded again. □

The termination of the pivots rests on the rule of step 6. A tree is *strongly feasible* if a positive amount of flow can be sent from each node to the root along its tree path within the bounds. The rule that takes the last blocking arc of the cycle, traversed from the join node in the direction of the flow, is due to Cunningham 1976.

Theorem 13.4. *(Termination.) If the initial tree is strongly feasible, the tree stays strongly feasible after each pivot, no basis is repeated, and the method ends after finitely many pivots. The initial tree of the class is strongly feasible in the balanced case and under LEQ, and under GEQ when no node has a supply of zero after the lower bounds are moved into the supplies.*

Proof sketch. The first statement is the theorem of Cunningham 1976; see also Ahuja, Magnanti and Orlin 1993, §11.5. Its main step is that, after a pivot with the last blocking arc, every node of the new tree can again send a positive amount to the root. In a degenerate pivot the subtree that moves is then the one on the far side of the leaving arc, whose potentials all change in the same direction. Hence, a nondegenerate pivot strictly decreases the cost, a degenerate one strictly changes the sum of the potentials in a fixed direction, and the pair (cost, sum of potentials) never repeats. For the second statement, in the initial tree an arc ur carries $sup(u) \geq 0$ towards the root and has an infinite capacity, while an arc ru carries $-sup(u)$, which can be decreased if it is positive. In the balanced case and under LEQ the arcs ru are those of the nodes of negative supply; under GEQ they are those of the nodes of nonpositive supply, and an arc ru of a node of zero supply carries no flow that can be decreased. The heuristic initial pivots use the same rule. □

We remark that the header states no bound on the number of pivots, and that the pivot rules of the class are

not known to be polynomial; under GEQ with nodes of zero supply, Theorem 13.4 does not apply to the initial tree, and the class relies on the pivot rules only. Each pivot costs the search of the rule, plus time proportional to the length of the cycle and to the size of the smaller part of the tree. In practice the method is usually the fastest one of the library on graphs of up to some thousands of nodes and on dense graphs (cf. section 13.9).

In step 3 the initial pivots are a heuristic of the original code. With a single supply node and a single demand node, they are made on the arcs found by a search backwards from the demand node over the arcs whose capacity is at least the total supply; with more nodes, on the cheapest arc into each demand node (or, under GEQ with less supply than demand, the cheapest arc out of each supply node). Each of these arcs enters the basis if it is eligible. Also, the constructor takes the flag `arc_mixing`, `true` by default, which stores the arcs in an interleaved order, with a stride of $\max\{\lfloor m/n \rfloor, 3\}$, rather than in the order of the digraph. According to the header, this usually performs like the original order, but it makes the method more reliable and, in special cases, significantly faster.

abLEMON. The potentials are updated on the smaller side of the tree at each pivot (version 1.4.0). In floating point, the leaving arc goes to the bound it is nearer to, its flow being set to that bound, rather than to the upper bound for a flow of, e.g., 10^{-16} (version 1.9.0), which may change the sequence of pivots. A problem is answered `UNBOUNDED` only after a check of its feasibility (version 1.4.1).

13.4.2 Pivot rules

The argument of `run()` is the rule that chooses the entering arc, i.e., a constant of the nested enum `PivotRule`. The arcs are scanned in a wraparound fashion from where the last search stopped, and an arc is eligible when its reduced cost, signed by its state, is below $-\epsilon_c$. Let m' be the number of arcs searched, i.e., those of the digraph and, when the supplies do not sum to zero, also the second artificial arcs; the five rules are then as follows.

- `FIRST_ELIGIBLE` takes the next eligible arc after the last one taken.
- `BEST_ELIGIBLE` takes the most violating arc among all of them.
- `BLOCK_SEARCH`, the default, scans blocks of $b = \max\{\lfloor\sqrt{m'}\rfloor, 10\}$ arcs and stops at the end of the first block after which an eligible arc has been seen, taking the most violating arc scanned.
- `CANDIDATE_LIST` builds, in a major iteration, a list of at most $\ell = \max\{\lfloor 0.25\sqrt{m'}\rfloor, 10\}$ eligible arcs, and then takes the best arc of the list, dropping those no longer eligible, in at most $\max\{\lfloor 0.1\,\ell \rfloor, 3\}$ minor iterations.
- `ALTERING_LIST` keeps the $h = \max\{\lfloor 0.01\,b \rfloor, 3\}$ best candidates of the previous iteration that are still eligible, and extends them by blocks of b arcs until the list has more than h arcs after the first block, or any after a later one; it takes the best arc and keeps the next h best.

Each rule is a private class of `NetworkSimplex` (`FirstEligiblePivotRule`, `BestEligiblePivotRule`, `BlockSearchPivotRule`, `CandidateListPivotRule` and `AlteringListPivotRule`), which `run()` instantiates for the chosen rule, and whose constants are those given above. According to the header, `BLOCK_SEARCH` and `ALTERING_LIST` were the most efficient rules on the instances on which they were tested, and `BLOCK_SEARCH`, being simpler and somewhat more reliable, was made the default. Any rule takes an eligible arc whenever there is one, which is all that Theorems 13.3 and 13.4 need.

13.5 The starting flow and the infinite bounds

The other three classes work on a residual network, in which each arc of the digraph appears forward and backward. An extra root node is joined to the nodes by a pair of arcs, which under GEQ take the unmet demand, with no cost and a capacity of one more than the deficit of the supplies. Two functions of `lemon/bits/mcf_feasible_flow.h` prepare that network.

The first one, `feasibleFlow()`, finds the flow that `CostScaling` and `CycleCanceling` start from, i.e., a flow within the bounds that meets the supplies. It runs the push-relabel algorithm of `Circulation` (Algorithm 12.4 of chapter 12), with the active node of highest level first, all the nodes at level zero at the start, a push towards any lower level, and the first empty level as the proof that no such flow exists (Goldberg and Tarjan 1988; Cherkassky and Goldberg 1997). Its starting point is the greedy flow of `Circulation`, in which each arc takes the deficit of its head within its bounds. Theorem 12.6 applies to it word for word. Hence, it ends, and when it fails the node being lifted, together with the nodes above the empty level, form a set that no residual arc leaves and whose excess is positive; this set of positive violation proves the problem infeasible, and chapter 14 uses it as a certificate. In floating point, an excess or a residual capacity within ϵ_f counts as zero, and at the end the residual capacities within it are moved to the reverse arcs, lest the algorithms send round-off around.

The second function, `infiniteBound()`, gives the bound that replaces an infinite upper bound in `CostScaling` and `CycleCanceling` (on every arc of infinite upper bound) and in `CapacityScaling` (on the arcs of infinite upper bound and negative cost, which it saturates at the start), namely

$$B = 1 + \sum_{u \in V} |sup(u)| + \sum_{uv:\ upper(uv) < \infty} |upper(uv)| + \sum_{uv \in A} |lower(uv)| \tag{13.7}$$

beyond the lower bound of the arc. If this does not fit the sums of the flows over the k replaced arcs, the bound is cut to the largest value of `Value` divided by $k+2$. Whether a cycle of negative cost is made of arcs of infinite upper bound is decided beforehand by `negativeInfiniteCycle()`, a Bellman-Ford algorithm restricted to those arcs and run from a virtual node joined to all the nodes. It runs only if one of those arcs has a negative cost, and it reports a cycle if a distance still improves after n rounds; if such a cycle exists, a feasible problem is answered `UNBOUNDED`. We remark that its comparisons are exact, also in floating point: a cycle whose cost is zero in decimal but whose sum rounds below zero is then taken as negative, and a feasible problem with such a cycle is answered `UNBOUNDED`.

Theorem 13.5. *(The finite bound.) Suppose that (13.1) is feasible and that no cycle of negative cost is made of arcs of infinite upper bound. Replace the infinite upper bound of some arcs by their lower bound plus B, and let f^* and π^* be an optimal flow and optimal potentials of the problem so obtained. Then f^* and π^* are optimal for the original problem.*

Proof. The original problem is bounded, since an unbounded direction of the polyhedron of the flows would be a nonnegative combination of cycles of infinite capacity, one of them of negative cost; hence, it has an optimal basic solution f_0 (the polyhedron has vertices, the flows being bounded below). In a basic solution of the auxiliary problem of section 13.4 the nonbasic arcs are at a finite bound, an arc of infinite upper bound being nonbasic at its lower bound. The flow of a tree arc is then the supply of the side of the tree that it separates from the root, plus or minus the flows of the nonbasic arcs between the two sides. Hence, for each arc a, $|f_0(a)| \le \sum_u |sup(u)| + \sum_{b \ne a}(|lower(b)| + |upper(b)|)$, where the infinite upper bounds are left out, and $f_0(a) - lower(a) \le B - 1 < B$. Thus, f_0 is feasible for the problem with the bounds replaced, which has therefore the same optimal value, and f^* is optimal for the original problem. By the complementary slackness between π^* and f_0 in the replaced problem, an arc whose bound was replaced and whose reduced cost under π^* is negative would carry $lower + B$ in f_0, which is false; hence, $cost^{\pi^*} \ge 0$ on those arcs, and π^* with f^* satisfies (13.5) for the original bounds. □

13.6 Cost scaling

`CostScaling` implements the cost scaling algorithm of Goldberg and Tarjan (Goldberg and Tarjan 1990; Goldberg 1997; Bünnagel, Korte and Vygen 1998), a primal-dual method that one may regard as the generalization to costs of the push-relabel algorithm of `Preflow`. On the residual network with the root, a flow f (with excesses allowed) and potentials π are *ε-optimal* when

$$cost^{\pi}(uv) \ge -\varepsilon \qquad \text{for each arc } uv \text{ of the residual network of } f, \tag{13.8}$$

and $\varepsilon = 0$ gives back the conditions of Theorem 13.2. The class multiplies the costs by $(n+1)\alpha$, where α is the scaling factor, i.e., the argument `factor` of `run()` (16 by default, at least 2, as it asserts), and works on these scaled costs. It starts from ε equal to the largest scaled cost of a residual arc divided by α and from zero potentials, and it divides ε by α after each phase. In a phase it *refines* the flow. First it saturates the residual arcs of negative reduced cost, which makes the flow 0-optimal but creates excesses and deficits. Then it removes them by pushes along the *admissible* arcs, i.e., those of negative reduced cost, and by relabels, each of which lowers the potential of a node u by $c_u + \varepsilon$, where c_u is the least reduced cost of the residual arcs out of u.

Algorithm 13.2 (Cost scaling).

```
1  find a feasible flow f (section 13.5); if none: return INFEASIBLE
2  if a cycle of infinite capacity has negative cost: return UNBOUNDED
3  scale the costs by (n+1) alpha; eps := largest scaled cost / alpha;
   pi := 0
4  while eps >= eps_min:
5      from the second phase on: price refinement; if it proves the flow
       eps-optimal: skip to 9
```

```
6      saturate the residual arcs of negative reduced cost
7      while some node u has positive excess (FIFO order):
           push or augment along admissible arcs, or relabel u;
           every few relabels, make a global update of the potentials
8      (the flow is now eps-optimal and has no excess)
9      eps := eps / alpha, or eps_min if eps lies in (eps_min, alpha eps_min)
10 divide the potentials by (n+1) alpha; if some residual arc has a
   reduced cost below -eps_c, correct them by Bellman-Ford
11 shift the potentials so that the largest is zero; return OPTIMAL
```

The nested enum `Method` selects the operation of step 7. With `PUSH` the flow moves on one admissible arc at a time, with a look-ahead that avoids pushing into a node that would only send the flow back. With `AUGMENT` it moves along admissible paths from a node with excess to a node with deficit. With `PARTIAL_AUGMENT`, the default and, according to the header, the most efficient and reliable one, it moves along admissible paths of at most 4 arcs. Two heuristics of Goldberg 1997 are applied. The price refinement of step 5 computes potentials that may prove the current flow ε-optimal before any push. The global update of step 7 recomputes the potentials by a bucket-based search from the deficits, after a number of relabels equal to $n+1+s^2$ with `AUGMENT` and `PARTIAL_AUGMENT` and to twice that with `PUSH`, s being the number of nodes of positive supply net of the lower bounds. Both change only the potentials, and keep (13.8).

Lemma 13.2. *Step 6 makes the flow* 0*-optimal, and each push and each relabel of step 7 keeps it* ε*-optimal.*

Proof. After step 6 an arc of negative reduced cost is saturated, and its reverse, the only arc that step 6 adds, has a positive reduced cost. A push along an admissible arc uv may add the arc vu, whose reduced cost $-cost^{\pi}(uv)$ is positive. A relabel of u subtracts $c_u+\varepsilon$ from $\pi(u)$, where $c_u \geq -\varepsilon$ by (13.8); an arc out of u of reduced cost $c \geq c_u$ gets $c-c_u-\varepsilon \geq -\varepsilon$, and an arc into u gets $c+c_u+\varepsilon \geq c$. □

Lemma 13.3. *With an integer* `LargeCost`*, a feasible flow that is* 1*-optimal for the scaled costs is optimal; the final correction of step 10 gives potentials that, with the shift of step 11, satisfy (13.5).*

Proof. A cycle W of the residual network has at most $n+1$ arcs, and its scaled cost is the sum of its reduced costs, at least $-(n+1)$; its original cost is therefore at least $-1/\alpha > -1$, and it is an integer, hence nonnegative. By Theorem 13.2 (on the network with the root, in the equality form) the flow is optimal. The Bellman-Ford algorithm on the reduced costs of the residual arcs, from a virtual node, then finds no negative cycle, and adding its distances to the potentials makes every residual reduced cost nonnegative, as in Theorem 13.2. Under GEQ with unbalanced supplies, the arcs from the root to the nodes are always residual, since their capacity exceeds the unmet demand; hence, the root has the largest potential. Its arc to a node with unmet demand is residual in both directions, and that node has the same potential; after the shift of step 11 these are the conditions of (13.5) on the nodes. □

Theorem 13.6. *(Cost scaling.) On integer data* `CostScaling` *returns the right answer, and with* `OPTIMAL` *an optimal flow and potentials meeting (13.5). As stated in the header, its worst case running time is* $O(n^2 m \log(nK))$.

Proof sketch. The answers `INFEASIBLE` and `UNBOUNDED` follow from section 13.5 (the starting flow, Theorem 12.6, and `negativeInfiniteCycle()`), and the answer `OPTIMAL` from Lemmas 13.2 and 13.3 and from Theorem 13.5 for the replaced infinite bounds, provided that each refinement ends. The refinement ends by the analysis of Goldberg and Tarjan 1990: a node with excess has a residual path to a node with deficit, along which the reduced costs of an $\alpha\varepsilon$-optimal flow bound the drop of its potential. Hence, each node is relabeled $O(n)$ times per phase (for a fixed α), with $O(nm)$ saturating and $O(n^2 m)$ other pushes, and the refinement takes $O(n^2 m)$ time in its generic form. The number of phases is $O(\log_\alpha((n+1)\alpha K)) = O(\log(nK))$. See also Ahuja, Magnanti and Orlin 1993, §10.3. □

We remark that the bound is that of the generic method, whose operations the three methods of the class implement in different orders, and that the scaled costs $(n+1)\alpha\, cost$ must fit in `LargeCost`. For the internal computations the class uses the type `LargeCost` of the traits class `CostScalingDefaultTraits< GR , V , C >`, which also defines `Digraph`, `Value` and `Cost`: `LargeCost` is `long long` for an integer `Cost` (or `long` without `LEMON_HAVE_LONG_LONG`) and `double` otherwise. The named parameter `SetLargeCost< T >` changes it, provided that `Cost` is convertible to `T`, and the class names the traits `Traits` and the type `LargeCost`. The last value ε_{min} of the scaling is 1 for an integer `LargeCost`, which proves the flow optimal by Lemma 13.3, and otherwise $\epsilon_c\,(n+1)\,\alpha$, i.e., the tolerance on the reduced costs in the scaled units, which the final flow then meets. In practice the method is typically the most efficient one of the library on large sparse graphs (cf. section 13.9).

abLEMON. `CostScaling` finds its starting flow on its residual network (version 1.4.0), where LEMON 1.3.1 ran `Circulation` on the digraph, and it takes floating-point costs, capacities and supplies (version 1.6.0). The price refinement of LEMON 1.3.1 could write past the end of its buckets, which is fixed.

13.7 Capacity scaling

`CapacityScaling` implements the capacity scaling version of the successive shortest path algorithm (Edmonds and Karp 1972; Ahuja, Magnanti and Orlin 1993, chapter 10). It is a dual method: it keeps potentials under which the residual arcs of capacity at least Δ have nonnegative reduced costs (the flow is Δ-*optimal*), and it sends flow from the nodes with excess to the nodes with deficit along shortest paths. Under GEQ with unbalanced supplies the root starts with the excess $-\sum_u sup(u)$, which it may send to any node at no cost, i.e., it supplies the unmet demand.

Algorithm 13.3 (Capacity scaling).

```
1  saturate the arcs of negative cost (an infinite bound replaced by B,
   section 13.5); Delta := largest power of 2 (times the unit) not above
   the smallest of the largest excess, deficit and residual capacity
2  loop:
3      saturate the residual arcs with r >= Delta and reduced cost < -eps_c
4      while a node s has e(s) >= Delta and some node has e <= -Delta:
5          Dijkstra from s on the arcs with r >= Delta, until a node t
           with e(t) <= -Delta is settled
6          if none is reached: (at the last Delta) return INFEASIBLE,
           (otherwise) go on with the next node s
7          augment along the path by min{ e(s), -e(t), its capacities }
8          update the potentials of the settled nodes: pi += dist - dist(t)
9      if Delta = unit: return OPTIMAL (or UNBOUNDED, section 13.5)
10     Delta := Delta / factor, or unit if Delta <= factor unit
```

Here the unit is 1 for an exact `Value` and ϵ_f otherwise; hence, in floating point the last phase sends any amount above the tolerance. With a factor of at most 1, or when the initial Δ is already the unit, the scaling is disabled, and the method becomes the plain successive shortest path algorithm, which is usually the fastest choice when the supplies and the capacities are small. In the end the potentials are shifted so that the root has potential zero.

Lemma 13.4. *Steps 3, 7 and 8 keep the flow Δ-optimal, and after step 8 the arcs of the path found have a zero reduced cost.*

Proof. Step 3 saturates the arcs that violate the condition, whose reverses get a positive reduced cost. In step 5 the distances *dist* are computed on the nonnegative reduced costs; a settled node u has $dist(u) \le dist(t)$, and an unsettled node v has a true distance at least $dist(t)$. After step 8, an arc between two settled nodes has the reduced cost $cost^\pi(uv) + dist(u) - dist(v) \ge 0$, which is zero along the shortest path tree. An arc from a settled u to an unsettled v has $cost^\pi(uv) + dist(u) - dist(t) \ge 0$, since $dist(v) \le dist(u) + cost^\pi(uv)$ and $dist(v) \ge dist(t)$, and an arc into a settled node from an unsettled one does not decrease its reduced cost. The augmentation of step 7 moves at least Δ along arcs of zero reduced cost, whose reverses then have a zero reduced cost too. □

Theorem 13.7. *(Capacity scaling.) On integer data `CapacityScaling` returns the right answer, and with `OPTIMAL` an optimal flow and potentials meeting (13.5). As stated in the header, its worst case running time is $O(m \log U\,(n+m)\log n)$, under the assumption recalled below.*

Proof. In the last phase ($\Delta = 1$) every residual arc has a residual capacity of at least 1. If the phase ends with `OPTIMAL`, no node has an excess of 1 or more; since the excesses are integers and sum to zero, all of them are zero, and the flow is feasible. By Lemma 13.4 every residual arc then has a nonnegative reduced cost, so that the flow is optimal by Theorem 13.2 (with Theorem 13.5 for the arcs saturated at B), and the potentials meet (13.5) by the argument of Lemma 13.3. If the phase ends with `INFEASIBLE`, the set R of the nodes that the search reached from s is closed in the residual network and holds no node of negative excess, while $e(s) > 0$. If R does not hold the root, the excesses of R sum, as in the proof of Theorem 12.6, to the violation of R in (12.6), which is then positive. If it does, R holds every node, since the arcs from the root are residual; then no node has a deficit, which contradicts the zero sum of the excesses. For the running time, there are $\lceil \log U \rceil + 1$ phases, and each search takes $O((n+m)\log n)$ time with a heap. By the analysis of Ahuja, Magnanti and Orlin 1993, §10.2, a phase makes $O(m)$ augmentations, each of at least Δ, since the excesses left by the previous phase and the saturations of step 3 sum to $O((n+m)\Delta)$. □

We remark that this count of the augmentations assumes, as the model of that analysis does, that a node of excess at least Δ reaches a node of deficit at least Δ in the residual network of the arcs of capacity at least Δ. The code skips a node that does not, until a smaller Δ, which the count does not cover. In the searches the class uses the heap type `Heap` of the traits class `CapacityScalingDefaultTraits< GR , V , C >` (which also defines `Digraph`, `Value` and `Cost`), by default `QuadHeap< Cost , RangeMap< int > >`, whose fewer levels make the searches cheaper than on a binary heap (Johnson 1975). The named parameter `SetHeap< T >` replaces it by any heap that meets the concept of chapter 7 with the priority type `Cost` and the cross reference map `RangeMap< int >`, and the class names the traits `Traits` and the heap `Heap`.

abLEMON. The default heap is a 4-ary heap (version 1.4.0), the class compiles on value types narrower than `int`, and it takes floating-point data (version 1.6.0).

13.8 Cycle canceling

`CycleCanceling` implements three cycle canceling methods (Klein 1967; Goldberg and Tarjan 1989), which start from the feasible flow of section 13.5 and cancel cycles of negative cost of the residual network until none is left, i.e., until the flow is optimal by Theorem 13.2. The nested enum `Method` selects one of them.

- `SIMPLE_CYCLE_CANCELING` runs rounds of the algorithm of Bellman and Ford on the residual network, from a virtual node, with a limit on the number of rounds that starts at 2 and grows by a factor of 1.5 when no cycle is found; after each batch of rounds it cancels the node-disjoint cycles of the graph of the predecessors. It stops when a round improves no distance within the limit.
- `MINIMUM_MEAN_CYCLE_CANCELING` cancels a cycle of minimum mean cost at each step, found by Howard's algorithm with a limit of $\max\{n, 5\}$ iterations and, if the limit is reached, by the strongly polynomial algorithm of Hartmann and Orlin (chapter 16); it stops when the minimum mean is not below $-\epsilon_c$.
- `CANCEL_AND_TIGHTEN`, the default, which the header calls by far the most efficient and reliable, is the improved version of the previous method of Goldberg and Tarjan 1989.

In Cancel-and-Tighten an arc is admissible when its reduced cost is negative. A phase cancels, by a depth first search, the cycles of admissible arcs, until the admissible network is acyclic. Then it *tightens* the potentials along a topological order of that network, and it recomputes ε as the largest value of $-cost^\pi$ on the residual arcs. Every $\max\{\lfloor\sqrt{n+1}\rfloor, 5\}$-th phase, instead, sets ε to the opposite of the minimum cycle mean of the residual network, and recomputes the potentials from it by the algorithm of Bellman and Ford on the costs shifted by the mean.

Algorithm 13.4 (Cancel-and-Tighten).

```
1  find a feasible flow f (section 13.5); if none: return INFEASIBLE
2  if a cycle of infinite capacity has negative cost: return UNBOUNDED
3  pi := 0; eps := largest -cost(uv) over the residual arcs
4  while eps * (n+1) >= 1 (exact costs), or eps > eps_c (floating point):
5      cancel the cycles of admissible arcs (cost^pi < 0) by DFS
6      if this is not the k-th phase:
7          tighten pi along a topological order of the admissible arcs;
           eps := largest -cost^pi(uv) over the residual arcs
8      else:
9          eps := - (minimum cycle mean of the residual network);
           pi := potentials for eps, by Bellman-Ford
10 compute the final potentials by Bellman-Ford; return OPTIMAL
```

During the phases the potentials are `double`, fractional even with an exact `Cost`, and the test of admissibility uses a tolerance of 10^{-6} then, as in LEMON 1.3.1, and ϵ_c with a floating-point `Cost`. After the last phase of the two methods based on mean cycles, the potentials are computed by the algorithm of Bellman and Ford on the residual network (in `SIMPLE_CYCLE_CANCELING` its distances already give them). Its operation traits count an improvement only beyond ϵ_c, since in floating point the round-off of the sums can make a cycle of cost zero (e.g., an arc and its reverse) look negative for ever.

Theorem 13.8. *(Cycle canceling.) On integer data each of the three methods ends, and with `OPTIMAL` it returns an optimal flow and potentials meeting (13.5). The minimum mean cycle canceling makes $O(nm^2 \log n)$ cancellations and runs in $O(n^2m^3 \log n)$ time, as the header states, when each minimum mean cycle is found by the algorithm of Hartmann and Orlin; for Cancel-and-Tighten the header states $O(n^2m^2 \log n)$.*

Proof. Each cancellation sends a positive integer amount around a cycle of negative integer cost, and decreases the cost by at least 1. The problem with the bounds of section 13.5 is bounded, so that there are finitely many cancellations. `SIMPLE_CYCLE_CANCELING` stops only when a round of Bellman and Ford improves no distance, i.e., when the residual network has no negative cycle, and the minimum mean method stops when the minimum mean is nonnegative. Cancel-and-Tighten stops when every residual arc has $cost^\pi \geq -\varepsilon$ with $\varepsilon(n+1) < 1$; then each residual cycle, of at most $n+1$ arcs, has a cost above -1, hence nonnegative. In the three cases the flow is optimal by Theorem 13.2, and Theorem 13.5 covers the replaced bounds. The final distances of Bellman and Ford meet (13.5) by the argument of Lemma 13.3; the root has potential zero, since every node is reached from it by a residual arc of cost zero and every residual cycle through it has a nonnegative cost. For the running time of the minimum mean method, see Goldberg and Tarjan 1989 and Ahuja, Magnanti and Orlin 1993, §10.5: the opposite of the minimum mean never grows, and it decreases by a factor of at least $1 - 1/n$ every m cancellations, which bounds their number by $O(nm^2 \log n)$, each costing $O(nm)$ for the cycle. Cancel-and-Tighten makes $O(nm \log n)$ phases by the analysis of Goldberg and Tarjan, each of which takes $O(nm)$ for its cancellations. □

We remark that, with an exact `Cost`, the tolerance of 10^{-6} on admissibility exceeds $1/(n+1)$ when the digraph has more than about 10^6 nodes; then a cycle whose mean is negative but above -10^{-6} is never admissible, and the termination of Cancel-and-Tighten is no longer guaranteed by the argument above. The class has no traits parameter, and the header warns that, in practice, the method is typically orders of magnitude slower than the scaling algorithms and `NetworkSimplex`.

abLEMON. `CycleCanceling` finds its starting flow on its residual network (version 1.4.0), takes floating-point data (version 1.6.0), and compares with zero only through the tolerances, its inner minimum mean cycle searches included (version 1.9.0).

13.9 Choosing an algorithm

In the documentation of the module, which follows the experimental study of Király and Kovács 2012, `NetworkSimplex` and `CostScaling` are recommended as the most efficient classes in general. `NetworkSimplex` is usually the fastest on relatively small graphs (up to several thousands of nodes) and on dense graphs, while `CostScaling` is typically more efficient on large graphs (e.g., hundreds of thousands of nodes or more), especially sparse ones. Other classes may be faster in special cases; e.g., when the total supply or the capacities are rather small, `CapacityScaling` without scaling is usually the fastest. In our view, `CycleCanceling` is mostly of interest for its strongly polynomial bound and for teaching. We remark that these are statements about typical instances, and that the ranking on a given family of problems is best found by running the four classes on it, which their common interface makes easy (cf. section 13.2). Also, note that only `NetworkSimplex` takes the LEQ form directly.

13.10 Exercises

Exercise 13.1. Verify the identity (13.4), and use it to show directly that a flow of (13.1) with a cycle of negative cost in its residual network is not optimal. Then check (13.5) on the four answers of `manual/examples/13-min-cost-flow.cc`, which does it in its function `optimal()`.

Exercise 13.2. Transform a problem in the LEQ form (13.2) into the GEQ form with `ReverseDigraph` and `NegMap`, solve it with `CostScaling`, and compare the cost with that of `NetworkSimplex` with `supplyType(LEQ)`. (The supply types are checked by `checkSupplyTypes()` in `test/min_cost_flow_test.cc`.)

Exercise 13.3. Solve the problem of `manual/examples/13-floating-point.cc` with `tolerances(0, 0)` on supplies that are exact in binary (e.g., 0.25, 0.5 and -0.75), and then on 0.1, 0.2 and -0.3 with the default tolerances; explain the difference by (13.6). (The tolerances are checked by `checkTolerances()` and `checkToleranceGetters()` in `test/min_cost_flow_test.cc`.)

Exercise 13.4. Time the five pivot rules of `NetworkSimplex` on random instances of a few thousand nodes, on an idle machine and with repeated runs, and compare the default with `BEST_ELIGIBLE`; repeat with `arc_mixing` set to `false`.

Exercise 13.5. Run `CostScaling` with the three methods and with the factors 2, 16 and 64, and `CapacityScaling` with the factors 1 and 4, on the same instances, and check that the costs agree. (The battery `test/min_cost_flow_battery_test.cc` runs every algorithm and method on limit cases and random instances, with integer and floating-point data.)

Exercise 13.6. Compute the bound B of (13.7) for the problem of `manual/examples/13-min-cost-flow.cc` with all the upper bounds infinite, and verify that an optimal flow of that problem sends less than B on each arc, as Theorem 13.5 asserts.

14. Certificates and the k best flows

We now turn to what the library gives besides one optimal flow. When a minimum cost flow problem has no optimal flow, the four algorithms of chapter 13 give a certificate of the answer, i.e., a set of nodes that proves it infeasible or a cycle that proves it unbounded, which a program can check on the data; they can also give several such certificates, in a definite order. When the problem has an optimal flow, the class `KBestFlows` of `lemon/k_best_flows.h` gives the k integer flows of least cost, and the class `KBestExtremeFlows` of `lemon/k_best_extreme_flows.h` gives the k vertices of least cost of the polyhedron of the flows, together with distinct optimal potentials. Section 14.1 treats the certificates of infeasibility and section 14.2 those of unboundedness, both implemented in `lemon/bits/mcf_certificate.h`; sections 14.3 and 14.4 describe the two classes of the k best flows, and section 14.5 the alternative optimal potentials. We keep the notation of section 13.1, i.e., the GEQ form (13.1), the reduced costs (13.3), the optimality conditions (13.5) and the residual network of a flow, and, as there, we state the results in exact arithmetic.

abLEMON. The content of this chapter is new with respect to LEMON 1.3.1, which gave no certificate and no other flow than an optimal one: `infeasibleCut()` and `unboundedCycle()` appeared in version 1.6.0, `infeasibleCuts()`, `disjointInfeasibleCuts()`, `unboundedCycles()`, `KBestFlows` and `KBestExtremeFlows` in version 1.7.0, the alternative optimal potentials in version 1.8.0, and their method `PIVOTS`, the cuts of `NetworkSimplex` taken from its last tree, the ties of `infeasibleCuts()` and the single run of Howard's algorithm in `unboundedCycles()` in version 1.9.0.

14.1 Certificates of infeasibility

14.1.1 Violated sets

For a set $N' \subseteq V$ of nodes, the *violation* of N' under GEQ is its supply less what can leave it,

$$viol(N') = \sum_{u \in N'} sup(u) - \sum_{uv \in A,\ u \in N',\ v \notin N'} upper(uv) + \sum_{uv \in A,\ u \notin N',\ v \in N'} lower(uv), \tag{14.1}$$

where no arc out of N' may have an infinite upper bound (such a set proves no infeasibility). Under LEQ the same holds with $-sup$ in place of sup and the arcs taken backwards, i.e., the demand of N' less what can enter it. A set of positive violation proves (13.1) infeasible, since its supply cannot leave it, and the converse holds too. As in section 12.4, let sup' be the supplies net of the lower bounds and P the sum of their positive values. Let also N be the network that joins a source σ to each node u with $sup'(u) > 0$ by an arc of capacity $sup'(u)$, each node u with $sup'(u) < 0$ to a target τ by an arc of capacity $-sup'(u)$, and that gives each arc uv the capacity $upper(uv) - lower(uv)$.

Theorem 14.1. *(Gale, Hoffman.) Problem (13.1) is infeasible if and only if some set N' has $viol(N') > 0$. For each cut $\{\sigma\} \cup N'$ of N, $viol(N') = P - cap(\{\sigma\} \cup N')$; hence, the largest violation is P less the value of a maximum flow of N, and the source side of each minimum cut, without σ, is a set of largest violation.*

Proof. The constraints of (13.1) are those of (12.5), and (14.1) is the right-hand side less the left-hand side of (12.6). The first statement is therefore Theorem 12.5, and the identity is (12.7). By Theorem 12.1, the smallest capacity of a cut of N is the value of a maximum flow, which gives the rest. □

This is the feasibility theorem of Gale 1957 and Hoffman 1960. Under LEQ the same holds on the reversed network with the negated supplies.

14.1.2 The interface

After a `run()` that returned `INFEASIBLE`, the four classes give three kinds of certificates, as sets of nodes of the digraph (vectors of `Node`); after any other answer they are empty.

- `infeasibleCut()` returns one set N' with $viol(N') > 0$.
- `infeasibleCuts(k)` returns the k sets of largest violation, in the order of their violations, at most k and only those that prove the problem infeasible.
- `disjointInfeasibleCuts()` returns sets of positive violation that are pairwise disjoint, with no arc between any two of them.

Each set is checked on the data, by (14.1), before it is returned. In floating point the check requires the violation to exceed both the tolerance on the flows and the round-off of the sum, i.e., the default epsilon of `Tolerance` times one plus the sum of the absolute values of the terms of (14.1); when it does not, no set is returned. Hence, a set that only round-off makes violated is never given as a certificate. `NetworkSimplex` gives its sets under the supply type, i.e., by (14.1) with the arcs reversed and the supplies negated under LEQ, and the other three classes use the GEQ form. In all of them the sets are computed and checked on the data as they are when the function is called, which must therefore be those of the last `run()` (the supply type included).

If it proves the answer, the set of `infeasibleCut()` is the one that the algorithm has at hand when it finds no feasible flow, at no further cost. It is the set of all the nodes when the supplies sum to the wrong sign (positive under GEQ, negative under LEQ). Otherwise, it depends on the class, as follows.

- `NetworkSimplex` takes it from its last tree. By the proof of Theorem 13.3, the tree potentials y under the costs of the first phase (one on the artificial arcs that should carry no flow, zero on the others) are an optimal solution of the dual of the first phase; so is the indicator of each level set of them, i.e., of the nodes whose potential is at least, or at most, a value, and the first level set that proves the answer is kept.
- `CostScaling` and `CycleCanceling` take the nodes above the level that the push-relabel algorithm of their starting flow leaves empty, together with the node it was lifting, from which no residual arc leaves (section 13.5); by Theorem 12.6 this set has a positive violation.
- `CapacityScaling` takes the nodes, other than the root, that its last search for a shortest augmenting path reached from a node with excess without finding a deficit; by the proof of Theorem 13.7, no residual arc leaves this set, and it has a positive violation.

If that set does not prove the answer (which may happen, although rarely, in floating point), the class falls back on a set of largest violation, by one maximum flow of `Preflow` on the network of Theorem 14.1. Thus, `infeasibleCut()` costs nothing beyond the run in exact arithmetic, and one maximum flow at most; we remark that one can therefore call it without concern even on "large" problems.

In turn, the sets of `disjointInfeasibleCuts()` are the weakly connected components of the set of `infeasibleCut()` whose violation is positive, in the order of their violations. Since no arc joins two components, the violation of the set is the sum of those of its components; hence, the violations of the sets given sum to at least that of the set. Note that they depend on the set that the class has at hand: in the example of section 14.1.4, `NetworkSimplex` gives a set of violation 4, whose two components are violated, while the other three classes give a set of violation 2, which is a single component.

14.1.3 The k most violated sets

We find the sets of `infeasibleCuts(k)` by the partition of Lawler 1972, applied to the cuts of the network of Theorem 14.1 as in Hamacher, Picard and Queyranne 1984. Here the space of the sets is split into *parts*, in each of which some nodes are fixed in the set and some out of it. The best set of a part is found by a maximum flow in which a node fixed in the set is joined to the source, and a node fixed out of it to the target, by an extra capacity of $2P + 3$, which no minimum cut may contain. Its violation bounds from above those of all the sets of the part. Once the best set of a part is given, the part is split by its free nodes $u_1, u_2, \ldots$ in turn: the j-th child keeps $u_1, \ldots, u_{j-1}$ as they are in the set, and takes u_j out of the set if the set has it, in otherwise.

Algorithm 14.1 (The k most violated sets).

```
1  root part: no node fixed; solve it by one maximum flow (Theorem 14.1)
2  give the sets that tie for the largest violation, from the residual
   digraph of that flow (the closed sets, Lemma 14.1), up to k of them
3  while fewer than k sets are given:
4      p := the part of largest bound (its own violation, or that of its
           parent if it is not solved yet; a solved part first on ties)
```

```
5       if p is not solved: solve it, from the maximum flow of its parent;
            if its best set violates a fix or is not violated, drop p;
            continue
6       give the best set of p unless it was already given; remove p
7       for each free node u_j of p, j = 1, 2, ...:
8           add the child that fixes u_1 .. u_{j-1} as in the set of p and
            u_j opposite to it, with the bound of p, not solved
```

Two devices keep the number of maximum flows rather small. First, a child waits in the list with the violation of its parent as a bound, and its best set is computed only when it is the most promising part; hence, a maximum flow is made only for the parts that might hold one of the k sets, rather than for each child of each set given. Second, the maximum flow of a part is a preflow of each of its children, whose capacities are only larger (one more fixed node adds capacity on one arc from the source or to the target); hence, `Preflow::init(flowMap)` starts each child from the flow of its parent (chapter 12). The ties of step 2 come from Lemma 12.1, as Picard and Queyranne 1980 observed; the partition then runs as usual, and a set that it finds again is not given twice.

Lemma 14.1. *(Closed sets.) Let x be a maximum flow of N. The minimum cuts of N are the sets that contain σ, do not contain τ, and are closed in the residual digraph of x, i.e., the unions of the closure of σ with the closures of strongly connected components of that digraph that do not reach τ. The function `closedSets()` lists them breadth first, from the closure of σ, by adding the closure of one component at a time.*

Proof. The first statement is Lemma 12.1. The closure of a node (the set of the nodes it reaches) is closed, and so is a union of closed sets; conversely, a closed set contains the closure of each of its nodes, and hence equals the union of the closures of its components. A closed set holds τ if and only if one of its components reaches τ, which the function excludes. □

Lemma 14.2. *A part that contains a set of positive violation has, as its best set, a minimum cut that respects its fixes, and that set has the largest violation in the part; a part whose minimum cut violates a fix contains no set of positive violation.*

Proof. A cut $\{\sigma\} \cup N'$ that respects the fixes cuts none of the extra capacities, and its capacity is $P - viol(N')$ by Theorem 14.1; it is below P if $viol(N') > 0$. A cut that violates a fix cuts an extra capacity of $2P + 3 > P$, and so does not attain the minimum when some set of the part has a positive violation. The same holds for the arcs of infinite capacity, which the network replaces by $P + 1$: a cut that contains one has a capacity above P. □

Theorem 14.2. *(The k most violated sets.) `infeasibleCuts(k)` returns the $\min\{k, q\}$ sets of largest violation, where q is the number of sets of positive violation, distinct and in a nonincreasing order of violation. It makes at most $1 + n\,s$ maximum flows, where s is the number of parts whose best set was given; i.e., at most n maximum flows for each set, as the header states.*

Proof. We show by induction that the parts in the list partition the sets of positive violation not yet given (the ties of step 2 aside, which are the sets of largest violation by Lemma 14.1 and are given first). This holds for the root part. When a part p with best set S is removed, its children are pairwise disjoint (the j-th one differs from the earlier ones on u_j), and they are contained in p. A set of p other than S differs from S on some free node, the first of which places it in exactly one child; hence, the children partition $p \setminus \{S\}$. A dropped part contains no set of positive violation by Lemma 14.2. The bound of each part is at least the largest violation of its sets, since a child's sets are sets of its parent. Hence, when the part of largest bound is solved and its best set is violated, that set has a violation at least that of every set left, and the sets are given in a nonincreasing order. Each part is solved at most once, and each part whose best set is given adds at most n children, which proves the count. □

14.1.4 An example

Figure 14.1 shows a problem of five nodes, in which the node a must send 4 units on an arc of capacity 2, and d must send 3 units on one of capacity 1, while b, with no supply, can only feed a. The sets $\{a\}$, $\{a, b\}$ and $\{d\}$ have violation 2, and the sets $\{a, d\}$ and $\{a, b, d\}$ have violation 4, the largest; hence, the four best sets of `infeasibleCuts(4)` have the violations 4, 4, 2 and 2.

The program `manual/examples/14-certificates.cc` (the test `manual_14_certificates`) solves the problem of Figure 14.1, and the unbounded problem of section 14.2, with the four classes, and it checks each certificate on the data. For `infeasibleCuts()` it checks positive violations (14.1) in a nonincreasing order and distinct sets, and for `disjointInfeasibleCuts()` pairwise disjoint sets with no arc between them; in the unbounded case it checks closed cycles of negative cost made of arcs of infinite capacity, pairwise arc-disjoint.

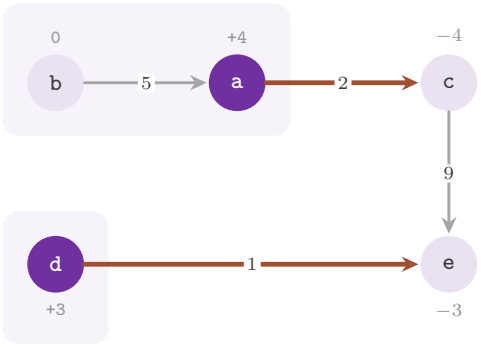


Figure 14.1. An infeasible problem: the supply of each node next to it, the capacity on each arc; the sets {a, b} and {d}, each of violation 2, are disjoint and joined by no arc, and their union, of violation 4, is a set of largest violation.

14.2 Certificates of unboundedness

A run returns `UNBOUNDED` only on a feasible problem with a directed cycle whose arcs have infinite upper bounds and whose cost is negative; along such a cycle the flow may grow without limit while the cost decreases (section 13.1). After such a run the four classes give two kinds of certificates, both empty after any other answer.

- `unboundedCycle()` returns one such cycle, as the vector of its arcs in the direction of the flow.
- `unboundedCycles(k)` returns at most k such cycles, pairwise arc-disjoint, in the order of their mean costs.

In `NetworkSimplex` the cycle is the one of the pivot that it could not perform, i.e., its entering arc and the path of the tree between the two ends of it, unless that path goes through an artificial arc, which by the proof of Theorem 13.3 does not happen in exact arithmetic when $M > nK$. In that case, and always in the other three classes, the cycle is found by the algorithm of Bellman and Ford on the arcs of infinite upper bound, run from a virtual node joined to all the others. After each round the graph of the predecessors is searched for a cycle, which is then returned (Lemma 14.3), and the search gives up, returning no cycle, after $4n + 4$ rounds or when a round improves no distance. In floating point the cycle is returned only if its cost is negative beyond the round-off.

Lemma 14.3. *(Tarjan.) If the graph of the predecessors of the algorithm of Bellman and Ford contains a cycle, the cost of that cycle is negative.*

Proof. Let C be the cycle, and consider the moment just before the last of its arcs, $p(v)v$, was set, by a strict improvement $d(v) > d(p(v)) + c(p(v)v)$. For each other arc $p(w)w$ of C, $d(w)$ was set to $d(p(w)) + c(p(w)w)$ when the arc was set, and $d(p(w))$ can only have decreased since; hence, $d(w) \geq d(p(w)) + c(p(w)w)$. Summing over the arcs of C, in which each node appears once as a head and once as a tail, the distances cancel, and $0 > \sum_{a \in C} c(a)$. □

One might think of the cycle of least total cost as the natural choice for several cycles, but it is NP-hard to find, since it would decide whether a digraph is Hamiltonian (cf. Exercise 14.3). Instead, the cycles of `unboundedCycles(k)` are those of minimum mean cost, which are the most negative per arc. Howard's algorithm (class `HowardMmc`, chapter 16), run on the arcs of infinite upper bound, ends on each strongly connected component with a policy whose cycle has the minimum mean of the component (Dasdan 2004); the class gives these cycles through `componentCycleNum()` and `componentCycle()`.

Algorithm 14.2 (Arc-disjoint negative cycles).

```
1  C := empty; T := the arcs of infinite upper bound
2  while |C| < k and T is not empty:
3      run Howard's algorithm on the arcs of T
4      for each strongly connected component, take its cycle of minimum
       mean if that mean is negative beyond the round-off
5      if none: stop
6      add them to C by increasing mean, up to k; remove their arcs from T
7  sort C by mean; return C
```

Lemma 14.4. *The cycles of Algorithm 14.2 are pairwise arc-disjoint, each of negative cost, and the first one has the minimum mean of all the cycles of infinite upper bound.*

Proof. The cycles of one run lie in distinct strongly connected components, and are therefore node-disjoint, and those of later runs use only the arcs not yet taken. The minimum cycle mean of a digraph is the least of those of its components, and the first run is made on all the arcs of infinite upper bound. A negative mean means a negative cost. □

A cycle of a later run may have a smaller mean than some of an earlier one (never than the first one), which is why step 7 sorts them. Since one run of Howard's algorithm gives the cycles of all the components, the algorithm runs again only when fewer than k cycles were found.

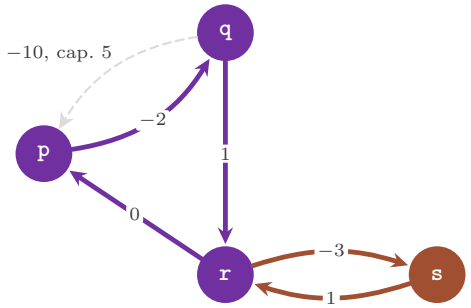


Figure 14.2. An unbounded problem: the cost on each arc; the arcs of infinite capacity are drawn full, and the arc (q,p), of cost -10 but capacity 5, dashed. The cycle (r,s,r), of mean -1, is the one of minimum mean; the cycle (p,q,r,p), of mean -1/3, is found by the second run on the arcs left.

In the problem of Figure 14.2, the four nodes form one strongly connected component of the arcs of infinite capacity, whose cycle of minimum mean is (r, s, r), of cost -2 and mean -1. A second run on the arcs left finds (p, q, r, p), of cost -1 and mean $-1/3$, and `unboundedCycles(3)` gives these two cycles, as the program of section 14.1.4 checks. The arc (q, p) closes with (p, q) a cycle of cost -12, but its capacity is finite; hence, it bounds the flow on that cycle, and it never appears in a certificate.

14.3 The k best integer flows: KBestFlows

14.3.1 The algorithm

We now describe the class `KBestFlows`, which finds the k integer flows of least cost of (13.1), in the order of their costs: first an optimal flow, then the other optimal flows, if any, and then the suboptimal ones. The constraints are brought to the equality form by an extra node, the root, with an arc of no cost and no upper bound into each node, whose flow is the slack $N_f(u) - sup(u)$ of the node (i.e., the unmet demand). Its algorithm is the one of Hamacher 1995, which rests on the following property of the integer flows.

Theorem 14.3. *(Hamacher.) Let f be a flow of least cost in a set of integer flows defined by integer bounds on the arcs. Then a second best flow of the set, if there is one, is obtained from f by sending one unit along a cycle of least cost of the residual network of f, where an arc together with its own reverse does not count as a cycle.*

Proof. Let g be a second best flow, i.e., one of least cost among the flows of the set other than f. The difference $g - f$ is an integer circulation that, on each arc, goes forward only where $g > f$ and backward only where $g < f$. It decomposes into unit cycles $C_1, \ldots, C_p$ of the residual network of f, each conformal to it; hence, no cycle uses an arc and its own reverse, and $f + \chi(C_j)$ is a flow of the set for each j, χ denoting the unit flow along the cycle. Each C_j has a nonnegative cost, since f is a best flow, and $f + \chi(C_j) \neq f$, so that its cost is at least that of g. Hence, $cost(g) = cost(f) + \sum_j cost(C_j) \geq cost(f) + cost(C_1) = cost(f + \chi(C_1)) \geq cost(g)$, and $f + \chi(C_1)$ is a second best flow; a cycle of least cost costs at most $cost(C_1)$, and one unit along it gives a flow of the set different from f. □

Then the set of all the integer flows is split as in the partition of Lawler 1972, on an arc where the best flow and the second best flow differ, i.e., the arc a of the first residual arc of the cycle. On one side the flow of a stays on the side of its value in the best flow, which remains the best of that side; on the other it lies on the side of the value of the second best flow, which becomes the best of that side. Hence, the next flow of the sequence is the cheapest of the second best flows of the sides.

Algorithm 14.3 (K best integer flows).

```
1  f_1 := an optimal flow (NetworkSimplex, or initFlow()); if none: return
```

```
2  S := { (bounds of the problem, f_1) }; compute the second best of it
3  for i = 2, ..., k:
4      (B, f) := the set of S whose second best flow g is the cheapest;
       if no set has one: stop
5      f_i := g; let a be the arc of the first residual arc of the cycle
6      if g(a) > f(a): B' := B with lower(a) := f(a) + 1;
                       B  := B with upper(a) := f(a)
       else:           B' := B with upper(a) := f(a) - 1;
                       B  := B with lower(a) := f(a)
7      add (B', g) to S; compute the second best flows of (B, f), (B', g)
```

Theorem 14.4. *(KBestFlows.)* ***run(k)*** *gives the* $\min\{k, q\}$ *integer flows of least cost of (13.1) in a nondecreasing order of cost, where* q *is the number of integer flows, provided that the first flow is optimal.*

Proof. We show by induction that the sets of S partition the integer flows not yet given, and that the given flow of each set is its best flow. This holds at the start. When the set B with best flow f and second best flow g is split, B (with the new bound on a) and B' are disjoint and their union is B, since the flow of a is an integer. Also, f stays in B and is still its best flow, while g is in B' and is its best flow, since $B' \subseteq B \setminus \{f\}$ and g is a best flow of $B \setminus \{f\}$ by Theorem 14.3. The flows not yet given are those of the sets other than their best flows, and the cheapest of them is the cheapest of the second best flows. Hence, each flow given is a cheapest one among those not yet given. □

In `secondBest()`, the second best flow of a set is found as follows. First, potentials of the residual network of the best flow of the set are computed by the algorithm of Bellman and Ford, from a virtual node joined to all the others; since the best flow of the set leaves no cycle of negative cost, the reduced costs are nonnegative (Theorem 13.2). Then, for each node v, the algorithm of Dijkstra computes the shortest paths from v on the reduced costs, and for each residual arc (u, v) the cycle of least cost through it is the arc and a shortest path from v back to u. A shortest path from v to u is simple, so that the cycle is simple and contains the reverse of (u, v) only if the path is that reverse and nothing else; then one more search is made without the reverse arc. Hence, as stated in the header, the k-th flow takes $O(nm \log n)$ time after the $(k-1)$-th when few such searches are made, and $O(m^2 \log n)$ at worst, with one search for each residual arc. Of course, the sequence does not depend on the algorithm that found the first flow, but for the order of the flows of equal cost.

14.3.2 The interface

`KBestFlows<GR, V, C>` takes the type of the digraph, the number type `V` of the flows, of the bounds and of the supplies (by default `int`), and the number type `C` of the costs (by default `V`), which it names `Digraph`, `Value` and `Cost`. Its constructor takes the digraph, whose nodes and arcs it numbers at once; the digraph must not change afterwards. Until they are given, the bounds are 0 and infinite, the costs 1 and the supplies 0. Its setters, each returning the object, are the following ones.

- `lowerMap(map)`, `upperMap(map)`, `costMap(map)` and `supplyMap(map)` copy the data as in chapter 13, an upper bound that is infinite or the largest value of `Value` meaning no bound.
- `initFlow(map)` gives an optimal flow to start from, which must be optimal for the data given, and after which no minimum cost flow algorithm is run.
- `tolerances(flow, cost)` sets the tolerances of section 13.3, which are given to the `NetworkSimplex` that finds the optimal flow, a negative value leaving them to the data; `flowTolerance()` and `costTolerance( )` return those in use, as a `Tolerance< Value >` and a `Tolerance< Cost >`.

Then `run(k)` finds the flows and returns their number, which is less than k if the problem has fewer integer flows, and zero if $k \leq 0$ or the problem has no optimal flow (it is infeasible or unbounded). Its queries are `flowNum()` , the number of flows of the last run, `totalCost(i)`, `flow(i, arc)` and `flowMap(i, map)`, for the i-th flow, where the first one is the flow 0. Both the bounds and the supplies must be integer, also with a floating-point `V` (whose values are then integer ones), since the flows differ from each other by whole units. Also, an infinite upper bound is allowed only if no cycle of negative cost is made of arcs of infinite upper bound, or the problem has no optimal flow. We remark that the comparisons of the search for the second best flows are exact on such data.

14.4 The k best extreme flows: KBestExtremeFlows

14.4.1 Vertices, bases and the ranking of Murty

Next, the class `KBestExtremeFlows` finds the k vertices of least cost of the polyhedron of the flows of (13.1), in the order of their costs: first an optimal vertex, then the other optimal ones, if any, and then those whose cost exceeds the optimum by at most a given gap. As the header remarks, these are the flows that a Lagrangian or a Benders decomposition may want several of, since each vertex gives a different (sub)gradient, while a flow inside a face gives a convex combination of those of its vertices. The class works, as `KBestFlows`, on the network with the root and its arcs into the nodes, in the equality form. A flow is a vertex if and only if the arcs strictly within their bounds (the *free* arcs) form a forest (Lemma 14.5); a *basis* of a vertex is a spanning tree that contains its free arcs, the other arcs being at a bound. A vertex is *degenerate* when it has more than one basis, i.e., when its free arcs do not span the nodes.

Lemma 14.5. *A feasible flow is a vertex of the polyhedron of the flows if and only if its free arcs form a forest.*

Proof. If the free arcs contain a cycle, sending a small amount $\pm\delta$ around it gives two feasible flows whose midpoint is the flow, which is then no vertex. If they form a forest, fixing every other arc at its bound leaves a system of conservation equations on the arcs of a forest, which has at most one solution (a leaf of the forest determines the flow of its arc, and so on). Hence, the flow is the only feasible flow on which those bounds are tight, i.e., a vertex. □

Its algorithm is the ranking of the extreme points of Murty 1968, which rests on the fact that the next vertex in the order of cost is adjacent to one of the vertices already given. Adjacency means here one pivot of the network simplex method away from one of their bases, i.e., along the cycle that a nonbasic arc closes with the tree, saturated. Candidates wait in a priority queue by their costs, and the bases of a degenerate vertex, which the pivots of zero amount reach, are all expanded.

Algorithm 14.4 (Ranking of the vertices).

```
1  x_1 := an optimal flow, made a vertex; T_1 := a basis of it
2  Q := { (cost(x_1), the basis T_1) }
3  while Q is not empty and fewer than k vertices are given:
4      take from Q the candidate of least cost; stop if beyond the gap
5      make its basis (one pivot from the expanded basis it came from)
6      skip it if the basis was already expanded
7      if basisLimit() bases were expanded: exact := false; stop
8      give its vertex if it is new
9      if its vertex already used up vertexBasisLimit(): exact := false;
       skip it
10     for each nonbasic residual arc r of the basis:
11         c := the cost after the pivot of r (if its cycle is bounded)
12         put (c, r) in Q unless c exceeds the gap (the degenerate pivots
           only if the vertex has not used up its limit)
13 if not exact: sort the vertices given by cost
```

A candidate is kept in the queue only as its cost, with the index of the expanded basis and of the arc of its pivot; the basis it leads to is made only when it leaves the queue. Thus, the memory is $O(n+m)$ for each basis expanded. Each pivot takes $O(n)$ time for its cycle, and each basis $O(n(n+m))$ for all its pivots, as the documentation of `basisLimit()` puts it.

As in `KBestFlows`, the first vertex is the optimal flow that `NetworkSimplex` finds, or the one given by `initFlow()`, which need not be a vertex. It is made a vertex by `purify()`: while the free arcs close a cycle, the cycle is saturated in a direction in which it is bounded, the direction in which the cost does not grow being tried first.

Lemma 14.6. *`purify()` ends after at most $n+m$ saturations with a vertex whose cost is not larger than that of the flow it started from, and equal to it if that flow is optimal.*

Proof. A cycle of free arcs is bounded in at least one direction, since in each direction the arcs traversed backward have a finite residual capacity, and all the arcs traversed forward in one direction are traversed backward in the other. Saturating it leaves the other arcs as they are and takes at least one of its arcs to a bound, so that the number of free arcs decreases; at the end the free arcs form a forest, and the flow is a vertex by Lemma 14.5. The direction in which the cost does not grow is bounded unless the cycle is one of infinite capacity and negative cost, which a feasible problem with an optimal flow does not have, or of zero cost, in which case the

other direction costs nothing either. At an optimal flow the free arcs have a zero reduced cost by (13.5), so that a cycle of them costs zero. □

Lemma 14.7. *At a nondegenerate vertex x with basis T, the edges of the polyhedron at x are the directions of the cycles that the nonbasic arcs close with T, each in the direction allowed by the bound of its arc, and the adjacent vertices are the vertices reached by saturating those of them that are bounded.*

Proof. Every arc of T is free at x. A direction d is feasible at x if and only if it is a circulation that moves each nonbasic arc only away from its bound. Such a circulation is determined by its values on the nonbasic arcs (the tree arcs follow), and it is therefore the nonnegative combination $\sum_e d(e)\,\chi(C_e)$ of the cycles C_e, oriented as allowed, with coefficients the moves of the nonbasic arcs. These cycles are linearly independent, each containing a nonbasic arc that the others do not; hence, the cone of the feasible directions is simplicial, and its extreme rays, the edges at x, are the directions $\chi(C_e)$. Moving along a bounded one up to the first bound reached gives the adjacent vertex. □

Theorem 14.5. *(Murty.) Suppose that the vertices that Algorithm 14.4 expands are nondegenerate and that no limit acts. Then it gives the vertices of the polyhedron of the flows, up to k and within the gap, in a nondecreasing order of cost, the first one optimal.*

Proof. Let X be the set of the vertices given so far, which contains an optimal one, and let w be a vertex of least cost among those not in X. If some edge at w leads to a vertex of smaller cost, that vertex is in X (by the choice of w), and w is adjacent to X. Otherwise no edge at w decreases the cost, and no unbounded edge does either, since the problem has an optimal flow; as the cone of the feasible directions at w is generated by its edges, w is optimal. The optimal vertices are those of the optimal face, whose graph of vertices and edges is connected; since X contains an optimal vertex, some optimal vertex outside X is adjacent to an optimal vertex of X, and its cost is that of w. In both cases the cheapest vertex adjacent to X and outside X has the cost of w, and by Lemma 14.7 it is among the candidates of the queue, which gives the next vertex in the order of cost. □

We remark that, at a degenerate vertex, the edges are the pivots of some of its bases, and the algorithm expands every basis that the degenerate pivots of its leaving rule reach from the first one. We do not prove that these are all the bases of the vertex; the tests compare the ranking with an enumeration of the vertices on small problems (Exercise 14.5).

14.4.2 Degeneracy and the limits on the bases

A degenerate vertex may have very many bases, and network problems are often highly degenerate. For instance, a vertex of a shortest path problem, i.e., a path whose other basic arcs carry no flow, has as many bases as there are spanning trees containing the path, and the ranking expands all those it meets. Two limits bound this work, and `exact()` returns whether a run stayed within them.

- `basisLimit(limit)` sets the largest number of bases a run expands, by default 1000. When it is reached the run stops; the flows given are still distinct vertices in the order of their costs, but they may not be the k best ones.
- `vertexBasisLimit(limit)` sets the largest number of bases of one vertex that a run expands, by default no limit (a value of 0). A basis counts only if one of its pivots leads to another vertex, so that the bases whose pivots are all degenerate do not take the place of one that is not. With a positive limit, the bases of a vertex beyond it are not expanded, and the degenerate pivots of its last basis are not taken. The flows given are then distinct vertices in the order of their costs, the first one optimal and the others adjacent to the bases expanded, but they may not be the k best ones.

When a limit has acted, a vertex reached only through a basis left out may have come after one of larger cost; hence, the flows are sorted by their costs at the end (step 13). Thus, `exact()` returns `false` when the last run stopped at `basisLimit()`, or left out a basis or a pivot for `vertexBasisLimit()`.

14.4.3 The cycles of least reduced cost

With `method(CYCLES)` the class enumerates no basis, and thus avoids the expansion of the many bases of a degenerate vertex. Starting from the optimal vertex and from potentials optimal for it, it first finds the other optimal vertices. A residual arc of zero reduced cost closes a cycle of zero reduced cost if and only if its ends are in the same strongly connected component of the open residual arcs of zero reduced cost, and the cycle is then found from its head along those arcs. Each such cycle, saturated and made a vertex, gives an optimal vertex, from which the same is done again. Then, from each optimal vertex and for each arc at one of its bounds, the cycle of least reduced cost through the arc, saturated and made a vertex, gives a vertex of larger cost. Only

the arcs at a bound are tried, since the free arcs of a vertex make no cycle; they are tried in the order of their reduced costs, which bound from below those of their cycles, until one reaches the reduced cost of the worst cycle kept.

Algorithm 14.5 (Vertices by the cycles of least reduced cost).

```
1  x := an optimal vertex; pi := optimal potentials (Bellman-Ford)
2  O := (x); for each vertex y of O, in turn, while fewer than k:
3      for each arc r at a bound of y with reduced cost 0 whose ends lie
       in one strongly connected component of the zero arcs:
4          close r by a path of zero reduced cost (Dijkstra on zero arcs)
5          saturate the cycle, make it a vertex z; if new, give z, add to O
6  K := empty (kept candidates, at most 2 (k - given))
7  for each vertex y of O, for each arc r at a bound of y,
       by increasing reduced cost while it is below the worst kept:
8      c := rc(r) + shortest path from head(r) to tail(r) (Dijkstra,
            stopped at the end or at the bound)
9      if c > 0: saturate the cycle, make it a vertex z; keep (c, z) if
            its cost exceeds the optimum and is within the gap
10 give the vertices of K in the order of their costs, up to k
```

Lemma 14.8. *An open residual arc r of zero reduced cost lies on a cycle of open residual arcs of zero reduced cost if and only if its ends are in the same strongly connected component of those arcs. Saturating such a cycle at an optimal flow, and then purifying, gives an optimal vertex.*

Proof. The arc r goes from its tail to its head, and a cycle through it exists if and only if the head reaches the tail along such arcs, i.e., if and only if the two ends are in one component. A cycle of zero reduced cost has a zero cost (the potentials cancel around it), so that saturating it keeps the cost, and the flow stays optimal; by Lemma 14.6 purifying keeps it optimal and makes it a vertex. □

With this method, too, the flows are distinct vertices in the order of their costs, the optimal ones first, but those of larger cost are only some of the vertices one cycle away from an optimal one; hence, `exact()` is always `false` with this method. Each arc tried costs one search of $O(m \log n)$ time; the header states $O(nm \log n)$ time for each optimal vertex, while with one search for each arc at a bound the bound is $O(m^2 \log n)$ at worst. The method needs $O(km + pn)$ memory for the p trees of shortest paths made, as the header states.

abLEMON. Since version 1.8.1 the method `CYCLES` tries only the arcs at a bound of a vertex, finds the cycles of zero reduced cost through the strongly connected components of the arcs of zero reduced cost, and tries the other cycles by increasing reduced cost of their arc, each path being looked for by a Dijkstra's algorithm stopped at its end or at the bound. The vertices of larger cost kept are those of the cycles of least reduced cost, while in version 1.7.0 they were those of least cost.

14.4.4 The interface

`KBestExtremeFlows<GR, V, C>` has the template parameters and typedefs, the constructor, the setters `lowerMap()`, `upperMap()`, `costMap()`, `supplyMap()` and `tolerances()`, and the queries `flowTolerance()`, `costTolerance()`, `flowNum()`, `totalCost(i)`, `flow(i, arc)` and `flowMap(i, map)` of `KBestFlows`. Its `initFlow(map)` takes a flow that must be optimal for the data given but need not be a vertex. Its tolerances are given to the `NetworkSimplex` that finds the optimal flow, and they are also used in its own comparisons: a flow within the tolerance on the flows of a bound is at the bound, and a reduced cost within the tolerance on the costs of zero is zero. The method is set by `method(m)`, with the constants `RANKING` (the default) and `CYCLES` of the nested enum `Method`, and the limits by `basisLimit()` and `vertexBasisLimit()`. Then `run(k, gap)` finds at most k vertices, a negative `gap`, the default, setting no limit on the excess of their cost over the optimum. It returns the number of vertices found, which is less than k if the problem has fewer vertices within the gap, and zero if $k \leq 0$ or the problem has no optimal flow; `exact()` returns whether the run was exact.

The program `manual/examples/14-k-best.cc` (the test `manual_14_k_best`) solves a problem of four nodes and five arcs, small enough to enumerate all its integer flows. It has 5 integer flows, of costs 5, 6, 6, 7 and 8, of which 4 are vertices (the flow of cost 7, the midpoint of the vertices of costs 6 and 8, has four free arcs, which make a cycle). The program checks that `KBestFlows` gives the 5 costs in order and that `RANKING` gives the 4 costs of the vertices with `exact()` true; then `CYCLES`, asked for 3 vertices, gives the costs 5, 6 and 6, and finally the program checks the potentials of section 14.5 against (13.5).

14.5 Alternative optimal potentials

By Theorem 13.1, potentials are optimal if and only if, together with an optimal flow f, they satisfy (13.5); for the network with the root, in which the arcs of the root are open, this means that the reduced cost of each open residual arc of f is nonnegative. These potentials form a polyhedron (up to a constant, which the class fixes by making the potential of the root zero), which has, in general, many vertices: an optimal flow that is degenerate in the primal has several optimal potentials. A program may want several of them, e.g., distinct dual solutions for a sensitivity analysis, or different cuts in a decomposition (somewhat as for the extreme flows of section 14.4).

Lemma 14.9. *The set of the optimal potentials, with the potential of the root fixed to zero, is the set of the potentials under which every open residual arc of one optimal flow has a nonnegative reduced cost; it is the same for every optimal flow. A potential of this set is a vertex of it if the arcs of zero reduced cost contain a spanning tree.*

Proof. The first statement is Theorem 13.1 on the network with the root, in the equality form. The set does not depend on the optimal flow, since by the sufficiency part of that proof a potential satisfying (13.5) with one optimal flow makes every optimal flow attain the same lower bound, i.e., satisfies (13.5) with each of them. If the tight arcs contain a spanning tree, the equations $cost^\pi = 0$ on the tree, with the root fixed, have a unique solution. Hence, π is the only point of the set at which those inequalities are tight. □

`runPotentials(k)` finds at most k distinct optimal potentials, and returns their number (zero if $k \leq 0$ or the problem has no optimal flow); `potentialNum()` and `potential(i, node)` read them, and the first ones are the potentials 0. Two potentials that differ by a constant are the same, and two within the tolerance on the costs of each other are the same too. One sets the method by `potentialMethod(m)`, with the constants `PATHS` and `PIVOTS` of the nested enum `PotentialMethod`.

With `PATHS`, the default, the potentials are computed from potentials π optimal for the optimal vertex of section 14.4, found by the algorithm of Bellman and Ford. They are $\pi + d_s$ and $\pi - d'_s$, for each node s in turn, where d_s are the distances from s and d'_s the distances to s in the residual network of the flow on the reduced costs (one Dijkstra's algorithm each), the nodes not reached taking the largest distance found.

Lemma 14.10. *The potentials $\pi + d_s$ and $\pi - d'_s$ are optimal, and each is a vertex of the polyhedron of the optimal potentials when its search reaches every node.*

Proof. For $\pi + d_s$, let D be the largest distance found. An open residual arc uv with u reached has v reached and $d_s(v) \leq d_s(u) + cost^\pi(uv)$, so that its new reduced cost is nonnegative; if u is not reached, the new reduced cost is $cost^\pi(uv) + D - d_s(v) \geq 0$ when v is reached, and $cost^\pi(uv)$ when it is not. For $\pi - d'_s$ the argument is symmetric: an arc from a node that does not reach s to one that does cannot be open. When every node is reached, the arcs of the tree of the shortest paths have a zero reduced cost, and Lemma 14.9 applies. □

With `PIVOTS` the potentials are the vertices reached from that of the optimal vertex by the pivots of the dual simplex method, breadth first. In a tree of arcs of zero reduced cost, an arc whose opposite residual arc is closed (an arc at a bound, i.e., a degenerate arc of the primal) leaves. Then the side of its tail is raised until an arc into it reaches a zero reduced cost and enters, and the potentials change if the step is not zero.

Algorithm 14.6 (Optimal potentials by dual pivots).

```
1  pi := optimal potentials for the optimal vertex (Bellman-Ford)
2  while the tree of zero reduced cost from the root misses some nodes:
       raise the nodes it misses by the least reduced cost of an open arc
       entering them
3  P := (pi); for each pi' of P, in turn, while fewer than k:
4      build the tree of zero reduced cost under pi'
5      for each tree arc r whose opposite residual arc is closed:
6          raise the side of the tail of r by the least reduced cost of
           an open arc entering it
7          if the step is positive: add the new potentials to P if new
```

Lemma 14.11. *Each raise of Algorithm 14.6 keeps the potentials optimal. After step 2 the potentials are a vertex of the polyhedron of the optimal potentials, and so is each potential added in step 7.*

Proof. Raising the potentials of a set Q by δ decreases by δ the reduced costs of the arcs entering Q, increases those of the arcs leaving it, and leaves the others unchanged; with δ the least reduced cost of an open arc entering Q, every open arc keeps a nonnegative reduced cost (Lemma 14.9). In step 2 the arc that reaches the least reduced cost becomes tight and joins the tree, which therefore grows. Since the arcs from the root to the nodes

are always open, some open arc enters any set of missed nodes; at the end the tight arcs span the nodes, which by Lemma 14.9 gives a vertex. In step 6 the arc r goes from Q to the other side and becomes looser, which is allowed since its opposite arc, which would enter Q with a zero reduced cost, is closed; the entering arc replaces r, and the tight arcs again contain a spanning tree. □

According to the header, each vertex costs $O(nm)$ time with this method; more precisely, it costs one search for the tight tree and one raise for each of its arcs, each of $O(n + m)$ time.

abLEMON. `runPotentials()` appeared in version 1.8.0 with the method `PATHS`, and `potentialMethod(PIVOTS)` in version 1.9.0.

14.6 Exercises

Exercise 14.1. Prove the identity of Theorem 14.1 by expanding the capacity of the cut $\{\sigma\} \cup N'$, and use it to compute by hand the largest violation in the problem of Figure 14.1. (The program `manual/examples/14-certificates.cc` checks the violations of the sets it receives.)

Exercise 14.2. Enumerate all the subsets of the nodes of a random infeasible problem with eight nodes, compute their violations by (14.1), and compare the k largest with `infeasibleCuts(k)` of the four classes. (Solved by the battery `test/min_cost_flow_battery_test.cc`, which checks the k best sets against the enumeration of all the subsets.)

Exercise 14.3. Show that the cycle of least total cost among the cycles of negative cost decides whether a digraph has a Hamiltonian cycle (give each arc the cost -1 and look for a cycle of cost $-n$), and explain why Algorithm 14.2 uses the mean cost instead. (The certificates are checked by `checkCertificates()` in `test/min_cost_flow_test.cc`.)

Exercise 14.4. Change the capacities of the example of section 14.4.4 and check, by the enumeration of the program, that `KBestFlows` still gives all the integer flows in order. (Solved by `checkAgainstEnumeration()` in `test/k_best_flows_test.cc`.)

Exercise 14.5. Build a shortest path problem on a complete digraph of six nodes, as a minimum cost flow problem with one unit of supply, and run `KBestExtremeFlows` with `basisLimit(10)`; observe `exact()`, and then raise the limit or set `vertexBasisLimit(1)`. (The extreme flows are checked against an enumeration by `checkExtremeAgainstEnumeration()`, and the limit cases by `checkLimitCases()`, in `test/k_best_flows_test.cc`.)

Exercise 14.6. Verify that the potentials of `runPotentials()` satisfy (13.5) with any optimal flow, not only with the one they were computed from (Lemma 14.9), and that both methods give optimal potentials (Lemmas 14.10 and 14.11). (Solved by `checkPotentials()` in `test/k_best_flows_test.cc`, and in part by `manual/examples/14-k-best.cc`.)

15. Minimum cuts

We are concerned in this chapter with the cuts of minimum capacity of a graph taken as a whole, i.e., with the nonempty proper subsets of the nodes whose boundary has the least capacity, when no source and no sink are fixed in advance. Between two given nodes, a minimum cut is the dual of a maximum flow, and we refer to chapter 12 for it. Three classes of abLEMON solve these problems, all in the group Minimum Cut Algorithms: `HaoOrlin` (`lemon/hao_orlin.h`) for a digraph, `NagamochiIbaraki` (`lemon/nagamochi_ibaraki.h`) for an undirected graph, and `GomoryHu` (`lemon/gomory_hu.h`), which builds a tree from which the minimum cut between any two nodes of an undirected graph can be read. Section 15.1 states the problems and the facts on which the algorithms rest, sections 15.2 to 15.4 describe the algorithms, with their correctness and their complexity, and the classes that implement them, and section 15.5 compares them.

15.1 The problems

Let $D = (V, A)$ be a digraph with $n = |V| \geq 2$ nodes, $m = |A|$ arcs and nonnegative capacities $c : A \to \mathbb{R}_+$. For $X \subseteq V$ we write $\delta^+(X)$ and $\delta^-(X)$ for the arcs that leave and that enter X, and $c(F)$ for the sum of the capacities of a set F of arcs. A *cut* is a set X with $\emptyset \neq X \subsetneq V$, and the *minimum cut* of D is the value

$$\lambda(D) \;=\; \min\{\, c(\delta^+(X)) \;:\; \emptyset \neq X \subsetneq V \,\} \tag{15.1}$$

together with a set that attains it, which we call a minimum cut as well. For two distinct nodes s and t, a set X with $s \in X$ and $t \notin X$ is an s-t cut, and the minimum s-t cut $\lambda(s,t)$ is the least capacity $c(\delta^+(X))$ of such a set. By the max-flow min-cut theorem (Theorem 12.1) it equals the value of a maximum flow from s to t. For an undirected graph $G = (V, E)$ with capacities on the edges, $\delta(X)$ is the set of the edges with one end in X and the other outside, and $\lambda(G)$ and $\lambda(s,t)$ are defined as above with $c(\delta(X))$ in place of $c(\delta^+(X))$. In this case the cut function is symmetric, i.e., $c(\delta(X)) = c(\delta(V \setminus X))$, and hence $\lambda(s,t) = \lambda(t,s)$. With unit capacities $\lambda(G)$ is the edge connectivity of G, i.e., the least number of edges whose removal disconnects it; the documentation of the library mentions this use first, as a test of the "reliability" of a network.

Our first fact shows that $n-1$ minimum s-t cuts in each direction are enough to find $\lambda(D)$, and it is the starting point of the algorithm of Hao and Orlin.

Theorem 15.1. *Let $r \in V$ be any node. Then*

$$\lambda(D) \;=\; \min\Big\{\, \min_{t \neq r} \lambda(r,t)\,,\; \min_{t \neq r} \lambda(t,r) \,\Big\}. \tag{15.2}$$

Proof. A cut X that contains r is an r-t cut for any node $t \notin X$, and hence $c(\delta^+(X)) \geq \lambda(r,t)$; a cut that does not contain r is a t-r cut for any node $t \in X$, and hence $c(\delta^+(X)) \geq \lambda(t,r)$. Conversely, each s-t cut is a cut, and therefore each term of the right-hand side of (15.2) is at least $\lambda(D)$. □

For an undirected graph the two halves of (15.2) coincide, and the second can be dropped; we use this in section 15.2.2. The second fact, due to Gomory and Hu (1961), is that the $n(n-1)/2$ values $\lambda(s,t)$ of an undirected graph take at most $n-1$ distinct values, and that one tree stores them together with a minimum cut for each pair. It rests on the following inequality.

Lemma 15.1. *Let $G = (V, E)$ be an undirected graph with nonnegative capacities, and let $x_0, x_1, \ldots, x_k$ be nodes of G with $x_0 \neq x_k$. Then $\lambda(x_0, x_k) \geq \min_{1 \leq i \leq k} \lambda(x_{i-1}, x_i)$, where a term with $x_{i-1} = x_i$ is taken as $+\infty$.*

Proof. Let X be a minimum x_0-x_k cut. Since $x_0 \in X$ and $x_k \notin X$, there is an index i such that $x_{i-1} \in X$ and $x_i \notin X$; then X is an x_{i-1}-x_i cut, and $\lambda(x_0, x_k) = c(\delta(X)) \geq \lambda(x_{i-1}, x_i)$. □

Theorem 15.2. *Let $G = (V, E)$ be an undirected graph with nonnegative capacities. There is a tree $T = (V, F)$, with a weight $w(f)$ on each edge $f \in F$, such that for any two distinct nodes s and t the value $\lambda(s, t)$ is the least weight $w(f)$ of an edge f on the path of T from s to t, and the two components of $T - f$ are the two sides of a minimum s-t cut of G.*

Proof. Algorithm 15.3 of section 15.4 constructs, by Theorem 15.7, a tree T and weights w such that, for every edge $f = v\,p(v)$ of T, the component of $T - f$ that contains v is a minimum v-$p(v)$ cut of G of capacity $w(f)$. Let $s = x_0, x_1, \ldots, x_k = t$ be the path of T from s to t, and let f be an edge of least weight on it. By Lemma 15.1 and since each $w(x_{i-1}x_i)$ equals $\lambda(x_{i-1}, x_i)$, one has $\lambda(s, t) \geq w(f)$. Conversely, the component of $T - f$ that contains one end of f contains exactly one of s and t, since f lies on their path. Hence, it is an s-t cut (or a t-s cut) of capacity $w(f)$, which proves $\lambda(s, t) \leq w(f)$ and that this component is a minimum cut between s and t. □

Such a tree is called a *Gomory-Hu tree*, and it is usually not unique. We remark that its edges need not be edges of G, and that even when they are (as in Figure 15.2) their weights are usually not the capacities of G, which only bound them from below, since an edge lies in every cut that separates its ends. Also, $\lambda(G)$ is the weight of the lightest edge of T, and the at most $n - 1$ distinct weights of T are all the values $\lambda(s, t)$.

15.2 The Hao-Orlin algorithm

15.2.1 The method

Theorem 15.1 requires $2(n - 1)$ maximum flows, each of which costs $O(n^2\sqrt{m})$ with the highest-label preflow push-relabel method of Goldberg and Tarjan (1988) (see chapter 12). Hao and Orlin (1994) observed that the minimum cuts from a node r to all the other nodes can be found in the time of one such computation. To this aim the sinks are taken one after the other, and a node that has been the sink becomes a source; the preflow and most labels of one computation then remain valid at the start of the next one.

We recall the terms of chapter 12. A *preflow* is a map $f : A \to \mathbb{R}$ with $0 \leq f \leq c$ whose *excess* $e(v) = f(\delta^-(v)) - f(\delta^+(v))$ is nonnegative at every node that is not a source. The *residual digraph* D_f has an arc (u, v) for each arc $(u, v) \in A$ with $f(u, v) < c(u, v)$ (a forward arc, of residual capacity $c(u, v) - f(u, v)$), and also an arc (v, u) for each arc $(u, v) \in A$ with $f(u, v) > 0$ (a backward arc, of residual capacity $f(u, v)$). To *push* θ along a residual arc means to increase the flow on its arc by θ if it is forward, and to decrease it if it is backward, so that θ units of excess move from its tail to its head. To *saturate* it means to push its whole residual capacity. The node set is split, at any moment, into the sources S, the *awake* nodes W, among which lies the current sink t, and a stack of *dormant* sets $R_1, \ldots, R_q$ (R_q on top). Each awake node v has a *label* $d(v)$, a nonnegative integer, and the labels are *valid* if $d(u) \leq d(w) + 1$ for every residual arc (u, w) with $u, w \in W$. Instead of relabelling the nodes from which the current sink can no longer be reached, the algorithm puts them aside in a new dormant set, which it brings back when W has been exhausted. An awake node other than t with a positive excess is *active*.

Algorithm 15.1 (Hao-Orlin, the cuts that contain r).

```
 1  S <- {r}; f <- 0; lambda <- +inf; saturate every arc (r, v) with
    c(r, v) > 0
 2  for each node u not in S and not yet reached, in the order of the
    node iterator: search the nodes that reach u by arcs of positive
    capacity, avoiding r and the nodes already reached, and label them
    by their distance to u; the first search makes W, every later one a
    dormant set put under the previous ones; t <- the node from which
    the first search started
 3  loop
 4    while W - {t} has an active node, take an active v of highest label:
 5      push from v along the residual arcs (v, w), w in W, d(w) = d(v) - 1
 6      if e(v) > 0 still:
 7        if v is alone in its label: the nodes of W of label >= d(v)
          become a new dormant set on top of the stack (a gap)
 8        else if v has no residual arc into W: {v} becomes a new dormant
          set on top of the stack
 9        else d(v) <- 1 + min{ d(w) : (v, w) residual, w in W }
10    if e(t) < lambda then lambda <- e(t); X <- V - W
```

```
11    S <- S + {t}; W <- W - {t}; saturate the residual arcs out of t
12    if W is empty: stop if the stack is empty, else W <- the dormant
      set on top of the stack, which leaves the stack
13    t <- a node of W of lowest label
```

Step 4 is the highest-label rule of the generic method, and steps 7 to 9 replace its relabelling. In `findMinCutOut()` of `lemon/hao_orlin.h` the labels are positions in a list of buckets, i.e., of lists of nodes of equal label (`_first`, `_last`, `_next`, `_prev`). The awake set and the dormant sets are lists of buckets (`_sets`, whose last element is W), and the active nodes come first in their bucket. Step 9 moves v into the bucket just above the lowest one that it can reach, which it creates if there is none. Step 10 compares the new cut with the best one by an exact `<`, while the tests that an arc is residual and that an excess is left go through the tolerance (section 15.2.2). The invariants of the algorithm are stated by the next lemma.

Lemma 15.2. *At the beginning of each iteration of the loop of step 4, and at step 10, (i) f is a preflow whose excess is nonnegative outside S; (ii) no residual arc goes from S to a node outside S, and no residual arc goes from a dormant set R_i to W or to a dormant set R_j with $j > i$; (iii) the labels of W are valid, and they form a set of consecutive integers.*

Proof. We first check the three properties after step 2. The pushes of step 1 make a preflow with $e(r) \leq 0$ and saturate the arcs out of r, which is (i) and the first part of (ii). A node u' of a later search with an arc (u', w) of positive capacity to a node w found by an earlier search would have been reached by that search, since w reaches the start of it. As f is still zero outside the arcs out of r, this is the second part of (ii), with the dormant sets stacked so that the later searches lie below. In each search an arc (u', w) of positive capacity gives $d(u') \leq d(w) + 1$, since the search reaches u' at most one layer after w, and the layers are consecutive, which is (iii).

We now check that each step preserves them. A push of step 5 moves excess from v to w, both awake, and it creates at most the residual arc (w, v), with $d(w) = d(v) - 1 \leq d(v) + 1$; hence (i) to (iii) hold. At step 7 the pushes of step 5 have saturated every residual arc from v to a node of label $d(v) - 1$, and v is the only node of label $d(v)$. Validity then shows that no residual arc goes from a node of label at least $d(v)$ to a node of W of smaller label. Hence, the new dormant set has no residual arc into what remains of W, and the older dormant sets have none into it, since they had none into W. At step 8 the same holds for $\{v\}$ by the test itself, and the label of v stays occupied by the other nodes of its bucket. At step 9, the minimum is taken over nodes of label at least $d(v)$ (those of label $d(v) - 1$ are no longer reachable), and the label of v increases. Validity is kept for the arcs out of v by the choice of the minimum and for the arcs into v since the label grows. The new label is at most one more than the largest label of W, while the old one stays occupied, since v is not alone in it. At step 11 the saturation removes every residual arc out of t and adds excess only to nodes outside S, while the residual arcs that it creates enter t, which is now in S. This keeps (i) and (ii), and removing t from the lowest label keeps (iii). At step 12, W is empty, and the set R_q on top of the stack receives no residual arc from the sets below it nor from S. Its labels, which no step has changed while it was dormant, are valid and consecutive, since no residual arc between two of its nodes has changed either (pushes stay inside W, and the saturations of step 11 only touch arcs incident to t). □

The value of a cut is read at step 10 from the excess of the sink, and the next lemma shows that it is the best one among the cuts that contain the current sources and leave out the current sink.

Lemma 15.3. *At step 10, let $X = V \setminus W$. Then $c(\delta^+(X)) = e(t)$, $S \subseteq X$ and $t \notin X$, and every set Y with $S \subseteq Y$ and $t \notin Y$ satisfies $c(\delta^+(Y)) \geq e(t)$.*

Proof. The sources are not awake, and t is, which gives $S \subseteq X$ and $t \notin X$. Since no node of W other than t is active at step 10, the excess of W is $e(t)$, and therefore $e(t) = f(\delta^+(X)) - f(\delta^-(X))$. By Lemma 15.2(ii) no residual arc goes from X to W, i.e., every arc of $\delta^+(X)$ has $f = c$ and every arc of $\delta^-(X)$ has $f = 0$; hence, $e(t) = c(\delta^+(X))$. Now let Y contain S and not t. Summing the excesses over Y gives $f(\delta^+(Y)) - f(\delta^-(Y)) = -\sum_{v \in Y} e(v) = \sum_{v \notin Y} e(v)$, since the excesses of all the nodes add up to zero. Every node outside Y is outside S, and therefore has a nonnegative excess by Lemma 15.2(i), and t is one of them; hence, $c(\delta^+(Y)) \geq f(\delta^+(Y)) \geq f(\delta^+(Y)) - f(\delta^-(Y)) \geq e(t)$. □

Theorem 15.3. *Algorithm 15.1 terminates, and at the end $\lambda = \min\{c(\delta^+(X)) : r \in X \subsetneq V\}$ and X is a set that attains this minimum.*

Proof. We first show that the loop of step 4 ends for a fixed sink. While the sink does not change, the labels are positions above the bucket of t, and they never decrease: step 9 increases a label, and steps 7 and 8 only remove nodes from W. By Lemma 15.2(iii) they are less than $|W| \leq n$, and hence steps 7 to 9 occur at most n^2 times.

Between two of them, every iteration empties the active node v of highest label h and makes active only nodes of label $h-1$. Hence, the pair formed by the highest label of an active node and the number of active nodes of that label decreases lexicographically, and it can do so at most n^2 times in a row. Each iteration scans the arcs of v once, and the loop therefore ends. Then every node other than r becomes the sink exactly once, since step 11 moves the sink to S, step 12 refills W from the stack, and the algorithm stops when W and the stack are empty, i.e., when $S = V$. Let $t_1, \dots, t_{n-1}$ be the sinks in this order and $S_k = \{r, t_1, \dots, t_{k-1}\}$ the sources while t_k is the sink. Each set stored at step 10 contains r and not the sink, and its capacity is the stored value by Lemma 15.3. Conversely, let X contain r, with $X \neq V$, and let k be the least index with $t_k \notin X$; then $S_k \subseteq X$, and Lemma 15.3 gives $c(\delta^+(X)) \geq e(t_k) \geq \lambda$. □

For the cuts that do not contain r one runs the same algorithm on the reverse digraph (the searches of step 2 follow the arcs forward, the pushes go backwards along them), and the stored set is then the awake set W of step 10, which contains the sink but not r. By Theorem 15.3 applied to the reverse digraph, its value is $\min\{c(\delta^+(X)) : r \notin X \neq \emptyset\}$, and by Theorem 15.1 the better of the two phases is $\lambda(D)$.

Theorem 15.4. *Each phase of Algorithm 15.1, as implemented in* `lemon/hao_orlin.h`*, runs in* $O(n^2\sqrt{m})$ *time.*

Proof sketch. The argument of Hao and Orlin (1994) bounds the work of a whole phase by that of a single run of the push-relabel method. With labels counted from a fixed origin instead of from the bucket of the current sink, a node is relabelled $O(n)$ times in the phase, and there are $O(nm)$ saturating pushes, while the scans of the arcs cost $O(nm)$ in all. The creation of the dormant sets and their return to W are paid by the relabellings and by the $n-1$ changes of the sink. The pushes that do not saturate their arc are counted by the potential argument of Cheriyan and Maheshwari for the highest-label rule (given for the maximum flow problem in Ahuja, Magnanti and Orlin 1993, §7.8). It bounds them by $O(n^2\sqrt{m})$, and this term dominates. □

The header also states the bound $O(nm)$ for unit capacities, and Hao and Orlin (1994) give the bound $O(nm \log(n^2/m))$ for a version with dynamic trees, which is not the one implemented.

15.2.2 The class HaoOrlin

The class is declared as `template< typename GR , typename CAP = typename GR::template ArcMap< int > , typename TOL = Tolerance< typename CAP::Value > > class HaoOrlin`, where `GR` is the digraph type, `CAP` a readable arc map of capacities of any numeric type, and `TOL` the tolerance of the comparisons; there are no named template parameters, and no traits class. Its public types are `Digraph`, `CapacityMap` and `Tolerance` (the value type `CAP::Value`, which `minCutValue()` returns, has no public name in the class). Its constructor `HaoOrlin(graph, capacity, tolerance = Tolerance())` stores references to the digraph and to the capacities, and `tolerance(tol)` and `tolerance()` set and read the tolerance afterwards. An execution starts with `init()`, which takes the first node of the node iterator of the digraph as r, or with `init(source)`, which takes the given one; both allocate the structures of the algorithm and set the stored value to the largest value of the capacity type. Then `calculateOut()` runs the phase of Algorithm 15.1 (the cuts with r inside) and `calculateIn()` the other one (the cuts with r outside), in either order, and one may also call only one of them. Also, `run()` and `run(s)` call `init()` (respectively `init(s)`), `calculateOut()` and `calculateIn()`.

Each phase replaces the stored cut only if it finds one of strictly smaller value. Hence, by Theorem 15.3, after `calculateOut()` the stored cut is a minimum one among the sets that contain r, and after both phases it is a minimum cut of the digraph (Theorem 15.1). The results are read by `minCutValue()`, which returns the value, and by `minCutMap(cutMap)`, which also returns it after writing into a writable `bool` node map the characteristic vector of the set X of (15.1), i.e., of the side whose leaving arcs have the least capacity. The digraph must have at least two nodes, since a cut is a nonempty proper subset of them, and `init()` asserts it with `LEMON_ASSERT` (which checks nothing when `NDEBUG` or `LEMON_DISABLE_ASSERTS` is defined).

In this class the tolerance decides which arcs exist for the algorithm and which nodes are active. An arc whose capacity (or residual capacity) is not positive for the tolerance is ignored by the initial searches and by the pushes, a push empties the node when the residual capacity is not less than its excess, and a node keeps being processed while its excess is nonzero for the tolerance. We recall that `Tolerance< CAP::Value >` is exact by default on integer types, while on floating-point ones it compares with the absolute epsilon of the library (i.e., $a < b$ is $a + \varepsilon < b$) (see chapter 3). With an inexact tolerance Lemma 15.3 holds only up to the excesses that are left on the nodes as zero.

abLEMON. The constructor and `tolerance()` are as in LEMON 1.3.1. In abLEMON, however, the test that a node still has an excess goes through the tolerance (`nonZero()`) instead of a comparison with an exact zero, so that on floating-point capacities a residue of rounding no longer keeps a node active. Furthermore, `init()`

states and asserts the precondition of at least two nodes, on fewer of which LEMON 1.3.1 read an empty list.

On an undirected graph one may run `HaoOrlin` on the digraph `Undirector<GR>` of chapter 5, with the capacities of the original edges. By Theorem 15.1 and the symmetry of the cut function, `init()` followed by `calculateOut()` is then enough (the test `test/hao_orlin_test.cc` runs both phases on an `Undirector`). The digraph of Figure 15.1 has, as its minimum cut with node 0 inside, the set $\{0, 1, 2\}$, of value 5, whereas its minimum cut is $X = \{3, 4, 5\}$, of value 4, which only the second phase finds. The program `manual/examples/15-hao-orlin.cc` (the test `manual_15_hao_orlin`) runs the two phases one at a time from node 0 on this digraph, and then `run()`. It checks each value against the enumeration of the $2^6 - 2$ cuts.

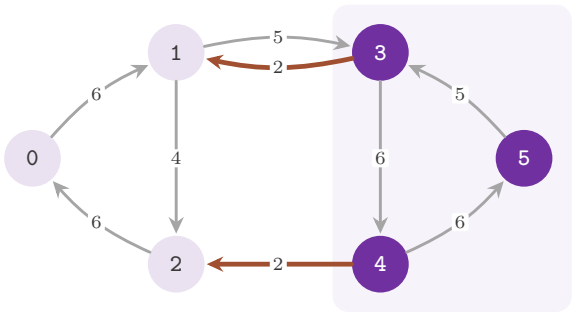


Figure 15.1. A digraph with the capacity of each arc on its plate; its minimum cut is $X = \{3, 4, 5\}$, whose two leaving arcs carry 4 units in all.

15.3 The Nagamochi-Ibaraki algorithm

15.3.1 Maximum adjacency orderings

On an undirected graph, Nagamochi and Ibaraki (1992) find $\lambda(G)$ without any flow, by orderings of the nodes that certify a lower bound on $\lambda(u, v)$ for many pairs at once. We assume in this section that G has no loops and no parallel edges (the algorithm merges the parallel edges, and a loop is in no cut), and for a node u and a set $B \subseteq V$ we write $c(u, B)$ for the capacity of the edges between u and the nodes of B. An ordering $v_1, \ldots, v_n$ of the nodes is a *maximum adjacency ordering* if v_1 is arbitrary and each v_j, $j \geq 2$, maximizes the adjacency $c(v, V_{j-1})$ among the nodes $v \notin V_{j-1}$, where $V_k = \{v_1, \ldots, v_k\}$. For an edge $e = v_i v_j$ with $i < j$ we write $q(e) = c(v_j, V_i)$; this is the adjacency of v_j just after v_i has been ordered, i.e., the key of v_j in the priority queue when e is scanned. Given a cut X, we call v_k, $k \geq 2$, *active* for X if v_{k-1} and v_k lie on different sides of X, and we write $\delta_k(X)$ for the edges of $\delta(X)$ with both ends in V_k. The next two lemmas follow the argument of Stoer and Wagner (1997).

Lemma 15.4. *In a maximum adjacency ordering, every node v_k active for a cut X satisfies $c(v_k, V_{k-1}) \leq c(\delta_k(X))$.*

Proof. By induction on the active nodes, in their order. For the first active node v_k, the nodes $v_1, \ldots, v_{k-1}$ lie on the same side, and v_k on the other; hence, every edge between v_k and V_{k-1} is in $\delta_k(X)$. Let now v_k be active and let v_a, $a < k$, be the previous active node. When v_a was ordered, v_k was not yet, and therefore $c(v_k, V_{a-1}) \leq c(v_a, V_{a-1}) \leq c(\delta_a(X))$, by the maximality of the ordering and by the induction hypothesis. The nodes $v_a, \ldots, v_{k-1}$ lie on the same side (there is no active node between a and k), and v_k on the other, since it is active; hence, the edges between v_k and $V_{k-1} \setminus V_{a-1}$ are in $\delta_k(X)$, and they are not in $\delta_a(X)$, which only has edges inside V_a. Adding up, $c(v_k, V_{k-1}) = c(v_k, V_{a-1}) + c(v_k, V_{k-1} \setminus V_{a-1}) \leq c(\delta_k(X))$. □

Lemma 15.5. *In a maximum adjacency ordering, every edge $e = v_i v_j$ with $i < j$ satisfies $\lambda(v_i, v_j) \geq q(e)$. In particular, the edge between v_n and the node of V_{n-1} ordered last among its neighbours has $q(e) = c(\delta(v_n))$.*

Proof. Let X be a cut that separates v_i from v_j, and let $v_a, \ldots, v_i$ be the longest run of consecutive nodes of the ordering that ends at v_i and lies on the side of v_i. If $a = 1$, all of V_i lies on the side opposite to v_j, and $q(e) = c(v_j, V_i) \leq c(\delta(X))$. Otherwise v_a is active, and v_j was not yet ordered when v_a was; hence, by the maximality of the ordering and by Lemma 15.4, $c(v_j, V_{a-1}) \leq c(v_a, V_{a-1}) \leq c(\delta_a(X))$. The edges between v_j and $v_a, \ldots, v_i$ are in $\delta(X)$ and not in $\delta_a(X)$, since $v_j \notin V_a$; adding up, $q(e) = c(v_j, V_{a-1}) + c(v_j, V_i \setminus V_{a-1}) \leq c(\delta(X))$. Since X is arbitrary, $\lambda(v_i, v_j) \geq q(e)$. For the second part, the key of v_n after the last of its neighbours has been ordered is $c(v_n, V_{n-1})$, which is $c(\delta(v_n))$. □

It follows that an edge with $q(e)$ not smaller than the value μ of a cut already found can be contracted: a cut that separates its ends has capacity at least $\lambda(v_i, v_j) \geq q(e) \geq \mu$, and it is not better than the one at hand. Each phase of the algorithm computes one ordering and takes as μ the least capacity of a prefix V_k, $1 \leq k < n$, of the ordering (these are cuts, and their capacities are computed while the ordering is built, by $c(\delta(V_k)) = c(\delta(V_{k-1})) + c(\delta(v_k)) - 2c(v_k, V_{k-1})$). Then it contracts the edges with $q(e) \geq \mu$, i.e., it merges their two ends into one node, merges the parallel edges so created by adding their capacities, and drops the loops.

Algorithm 15.2 (Nagamochi-Ibaraki).

```
1  skip the loops and merge the parallel edges;
   lambda <- min_v c(delta(v)); X <- that {v}
2  if lambda = 0: stop
3  repeat
4    build a maximum adjacency ordering v_1, ..., v_n' of the current
     nodes from the first one, with q(e) for every edge e
5    if it does not reach every node: X <- the original nodes of
     the nodes reached; lambda <- 0; stop
6    mu <- min{ c(delta(V_k)) : 1 <= k < n' }, attained at V_k
7    if mu < lambda: lambda <- mu; X <- the original nodes of V_k
8    contract the edges e with q(e) >= mu
9  until a single node is left
```

In the class, steps 1 and 2 are `init()`, and steps 4 to 9 are one call of `processNextPhase()`. In `lemon/nagamochi_ibaraki.h` the keys of the ordering are kept in a heap that maximizes them, and the value $q(e)$ is stored on the edge (`cut`) when it is scanned. The original nodes merged into a node are kept in a linked list (`_next_rep`), which steps 5 and 7 follow to write the cut. Step 5 detects a disconnected graph, whose cut of value 0 is the connected component of the first node. Step 8 scans the edges of each current node, and the edges of a node merged into it are appended to its list, so that they are scanned in the same step. An edge built by merging two parallel edges keeps the value q of the edge that was already incident to the node, which may leave it uncontracted until the next phase. In abLEMON the comparisons of steps 1, 2 and 7 go through the tolerance, while those of steps 6 and 8 are exact.

Theorem 15.5. *With exact comparisons, Algorithm 15.2 applied to a graph with at least two nodes terminates after at most $n-1$ phases, and at the end X is a minimum cut of G and $\lambda = \lambda(G)$.*

Proof. Each cut of the current graph, expanded to the original nodes merged into its nodes, is a cut of G of the same capacity, since the merged parallel edges carry the sum of the capacities and the dropped loops join nodes of the same side. We show that after each step the stored X is a cut of G of capacity λ. We also show that either $\lambda = \lambda(G)$ or a minimum cut of G is still a cut of the current graph, i.e., separates no two original nodes merged into one node. This holds after step 1, where nothing has been merged, and at the stop of step 2, since no cut has a negative capacity. If step 5 stops, the nodes reached form a union of connected components of the current graph, i.e., a cut of value 0, which is minimum. After step 7 one has $\lambda \leq \mu$; at step 8, by Lemma 15.5, every cut of the current graph that separates the ends of a contracted edge has capacity at least $q(e) \geq \mu \geq \lambda$. Hence, if a minimum cut of G of value less than λ was a cut of the current graph, it separates the ends of no contracted edge and remains a cut of the contracted graph; otherwise $\lambda = \lambda(G)$. When a single node is left, no cut of G is a cut of the current graph, and therefore $\lambda = \lambda(G)$. Finally, at step 6 the prefix $V_{n'-1}$ has $c(\delta(V_{n'-1})) = c(\delta(v_{n'}))$, and hence $\mu \leq c(\delta(v_{n'}))$. By Lemma 15.5 the last edge scanned into $v_{n'}$ has $q(e) = c(\delta(v_{n'})) \geq \mu$; since step 8 scans every current edge, at least one edge is contracted in each phase, which merges two nodes. □

Theorem 15.6. *`init()` runs in $O(n+m)$ time, and a phase in $O(m \log n)$ time with the default binary heap and in $O(m + n \log n)$ time with a Fibonacci heap; hence, `run()` takes $O(nm \log n)$ and $O(nm + n^2 \log n)$ time, respectively.*

Proof. Step 1 scans each edge from its two ends once, and the parallel edges are found by remembering, for each neighbour of the node being scanned, the last edge built towards it (`curr_arc`). A phase inserts and removes each current node once in the heap, and increases the key of a node once for each edge scanned, i.e., at most m times. With a binary heap each of these operations costs $O(\log n)$, and with a Fibonacci heap the n removals cost $O(\log n)$ amortized each and the increases $O(1)$ amortized each (chapter 7). Steps 5 to 7 cost $O(n)$, and step 8 handles each edge of a merged node once, in $O(m)$. By Theorem 15.5 there are at most $n-1$ phases. □

The header adds that with unit capacities a bucket heap gives $O(nm)$. We remark, however, that the merged edges no longer have unit capacities. Also, the test of the library runs only the binary heap and the Fibonacci heap.

15.3.2 The class NagamochiIbaraki

The class is declared as `template< typename GR , typename CM = typename GR::template EdgeMap< int > , typename TR = NagamochiIbarakiDefaultTraits< GR , CM > > class NagamochiIbaraki`, where `GR` is an undirected graph type, `CM` a readable edge map of capacities, and `TR` the traits class. Its public types are `Traits`, `Graph`, `CapacityMap`, `Value` (the value type of the capacities), `Heap` and `HeapCrossRef`, and `Create`, the class itself, as in every class with named template parameters (chapter 3). The default traits `NagamochiIbarakiDefaultTraits< GR , CM >` define `Value`, `Graph`, `CapacityMap`, the cross reference `HeapCrossRef` (i.e., `GR::NodeMap< int >`), the heap `BinHeap< Value , HeapCrossRef , std::greater< Value > >`, a binary heap that maximizes the keys, and the functions that allocate them: `createCapacityMap(graph)`, which fails with an assertion (the capacities must be given to the constructor), `createHeapCrossRef(graph)` and `createHeap(crossref)`.

There are two constructors: `NagamochiIbaraki(graph, capacity)` takes the graph and the capacities, while `NagamochiIbaraki(graph)` is meant for the named template parameter `SetUnitCapacity`, whose traits allocate a constant map of value 1 on the edges; e.g., `NagamochiIbaraki< G >::SetUnitCapacity::Create ni(g)` computes the edge connectivity of `g`. Furthermore, the class declares the named parameters `SetHeap< H , CR >` and `SetStandardHeap< H , CR >`, where `H` is a heap that maximizes the keys and `CR` its cross reference, a node map of `int` by default. With `SetStandardHeap` the algorithm constructs the cross reference on the graph and the heap on the cross reference, while with `SetHeap` they must be handed to the algorithm by `heap(hp, cr)` before `init()` (otherwise `init()` fails with an assertion). The function `heap(hp, cr)` takes objects of the types `Heap` and `HeapCrossRef` of the class, and it may also be used with the other traits, in which case the class uses the given objects instead of allocating its own. For instance, a Fibonacci heap of `std::greater< Value >` gives the bounds of the second case of Theorem 15.6.

> **abLEMON.** In LEMON 1.3.1 the two named parameters did not compile, since the function that allocates the structures passes the graph to the `createHeapCrossRef()` of their traits, which took an `int`, and their default cross reference was a `RangeMap< int >`; the test `nagamochi_ibaraki_test` now runs both with a Fibonacci heap.

In the execution, `init()` builds the internal copy of the graph and makes steps 1 and 2. Then `processNextPhase()` makes one phase and returns `true` when the algorithm has finished (and again at every later call), `start()` calls it until then, and `run()` is `init()` followed by `start()`; `tolerance(tol)` and `tolerance()` set and read the `Tolerance< Value >` of the comparisons. As for the results, `minCutValue()` returns the value of the best cut found so far, which after `init()` is already the capacity of a cut (the lightest star of a node) and at the end is $\lambda(G)$ by Theorem 15.5. Similarly, `minCutMap(cutMap)` writes the characteristic vector of one side of that cut into a writable `bool` node map, and returns its value. The graph must have at least two nodes, a precondition that the header does not state: on one node the stored set is the whole node set, of value 0. Also, the header warns that the value type must hold the cut values of the graph, since their sums may otherwise overflow, and notes that the capacities are supposed to be integer. With floating-point capacities the result rests on the tolerance and on the exact comparisons of steps 6 and 8, and the tests of the library cover only integer capacities. On the graph of Figure 15.2 the algorithm finds the value 3 (section 15.4.2 names the program that runs it phase by phase).

> **abLEMON.** `tolerance()` is new in abLEMON. LEMON 1.3.1 compared the cut values exactly, whereas abLEMON compares through the tolerance the stars of step 1, the value 0 that ends step 2, and the update of step 7.

15.4 Gomory-Hu trees

15.4.1 The construction

Gomory and Hu's original construction (Gomory and Hu 1961) computes $n - 1$ maximum flows on graphs obtained by contracting parts of G. Gusfield (1990) showed that a tree with the properties of Theorem 15.2 can be obtained by $n - 1$ maximum flows on the original graph, if the cuts are kept in a rooted tree that is rearranged after each flow. This second construction is the one of `lemon/gomory_hu.h`; there each node v other than the root r has a parent $p(v)$ and a weight $w(v)$, the weight of the edge $v\,p(v)$ of the tree, and the nodes are processed in the order of the node iterator.

Algorithm 15.3 (Gomory-Hu tree, the construction of Gusfield).

```
1  r <- the first node; p(v) <- r for every v != r
2  for each node v != r, in the order of the node iterator:
```

```
3    compute a minimum cut X between v and p(v), v in X, and its
     value f, by Preflow on G
4    w(v) <- f
5    for each u != v with u in X and p(u) = p(v): p(u) <- v
6    if p(p(v)) is defined and lies in X:
7      u <- p(v); p(v) <- p(u); p(u) <- v; w(v) <- w(u); w(u) <- f
8  number the nodes so that every node comes after its parent
```

Step 3 is a call of `Preflow::runMinCut()` with the tolerance of the class, and X is the source side given by `minCut()`. Step 7 makes v the parent of its former parent when the cut puts its grandparent on its side: v takes the place of $p(v)$ under the grandparent, with the weight that $p(v)$ had, and $p(v)$ hangs from v with the weight f. Step 8 computes the numbering by which the queries climb the two paths of the tree towards their common ancestor. The correctness of the construction rests on the fact that minimum cuts can be chosen so that they do not cross. We state it for the cut function $\kappa(Y) = c(\delta(Y))$ of G. We recall that it is *submodular*, i.e., $\kappa(Y) + \kappa(Z) \geq \kappa(Y \cap Z) + \kappa(Y \cup Z)$ for any two sets Y and Z (an edge counted on the right is counted at least as many times on the left).

Lemma 15.6. *Let X be a minimum s-t cut of G, and let $u \neq v$ be two nodes of X. Then there is a minimum cut between u and v contained in X.*

Proof. Let Y be a minimum u-v cut. If $t \in Y$, we replace Y by $V \setminus Y$, which by the symmetry of κ is a minimum v-u cut, and in both cases $t \notin Y$ and Y contains one of u and v but not the other. Then $X \cap Y$ separates u from v and is contained in X, while $X \cup Y$ contains s and not t, and therefore $\kappa(X \cup Y) \geq \kappa(X)$. By submodularity, $\kappa(X \cap Y) \leq \kappa(X) + \kappa(Y) - \kappa(X \cup Y) \leq \kappa(Y)$, i.e., $X \cap Y$ is a minimum cut between u and v. □

Theorem 15.7. *At the end of Algorithm 15.3, for every node $v \neq r$ the nodes of the subtree of v in the tree of the parents form a minimum v-$p(v)$ cut of G, of capacity $w(v)$; hence, by the proof of Theorem 15.2, the tree with the edges $v\,p(v)$ and the weights $w(v)$ is a Gomory-Hu tree.*

Proof sketch. This is the theorem of Gusfield (1990) on his cut-tree method, and the proof is an induction on the iterations of step 2. After the iteration of v, every processed node u has $w(u) = \lambda(u, p(u))$, and the cuts that the processed nodes have produced can be taken pairwise non-crossing. The induction step rests on Lemma 15.6, which shows that the minimum cut of step 3 can be taken so that it does not cross those of the nodes processed before. Steps 5 and 7 then rearrange the tree so that each subtree is the side of the cut of its root. The details, and the proof that the arbitrary minimum cut returned by the maximum flow is enough, are in Gusfield (1990); the Gomory-Hu theorem, with the uncrossing of Lemma 15.6, is also in Schrijver (2003), §15.4. □

The queries of the class climb the tree with the numbering of step 8, which we call $o(v)$ (it is 0 for the root, and larger for a node than for its parent), and the next lemma shows that the climb meets the common ancestor.

Lemma 15.7. *Consider the procedure that starts from two nodes s and t and, while the two current nodes differ, replaces the one of larger number o by its parent. It stops at the nearest common ancestor a of s and t, after $h(s) + h(t) - 2h(a)$ steps, where h is the depth in the tree, and the edges that it climbs form the path of the tree from s to t.*

Proof. If the current nodes $x \neq y$ have $o(x) < o(y)$, then y is not an ancestor of x, since an ancestor has a smaller number. Hence, the nearest common ancestor of x and y is a proper ancestor of y, and it does not change when y is replaced by its parent. Each step decreases $h(x) + h(y)$ by one, and the procedure stops when $x = y$, i.e., at the common ancestor; the edges climbed are those from s and from t up to it. □

Theorem 15.8. *`run()` takes the time of $n - 1$ calls of `Preflow::runMinCut()` plus $O(n^2)$, i.e., $O(n^3\sqrt{m})$. Let h be the height of the tree. Then `minCutValue(s, t)` takes $O(h)$ time, and `minCutMap(s, t, cutMap)` and the constructors of the two iterators take $O(nh)$ time, which is $O(n^2)$ in the worst case.*

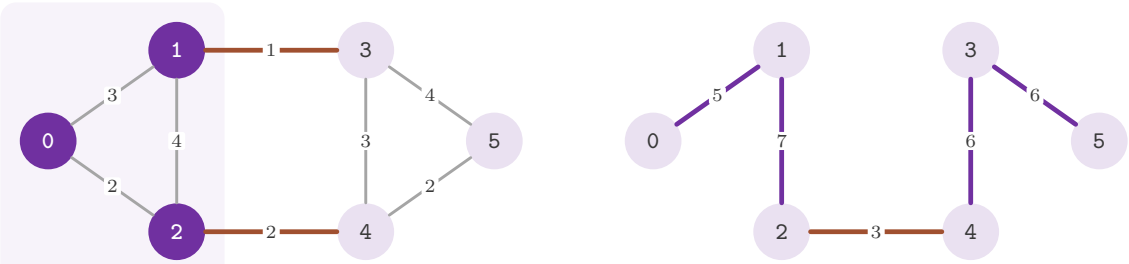
Proof. Steps 5 to 7 scan the nodes once per iteration, in $O(n)$, and step 8 climbs from each node to the first numbered ancestor and numbers the nodes on the way, in $O(n)$ in all. Each maximum flow costs $O(n^2\sqrt{m})$ by the bound of the highest-label preflow algorithm (Ahuja, Magnanti and Orlin 1993, §7.8). By Lemma 15.7 `minCutValue()` makes at most $2h$ steps. Then `minCutMap()` finds the lightest edge of the path in the same way, and writes the side of each node by climbing from it to the root or to the lower end of that edge. Since the nodes met on the way are not marked, each climb costs up to h, and the n climbs up to nh. □

15.4.2 The class GomoryHu

The class is declared as `template< typename GR , typename CAP = typename GR::template EdgeMap< int > > class GomoryHu`, where `GR` is an undirected graph type and `CAP` a readable edge map of capacities (the constructor checks the `ReadMap` concept on it). Its public types are `Graph`, `Capacity` and `Value`, and it has no traits class. Its constructor `GomoryHu(graph, capacity)` stores the references, `tolerance(tol)` and `tolerance()` set and read the `Tolerance< Value >` given to the maximum flows, and `run()` executes Algorithm 15.3 (there is no public `init()`). One reads the tree by `predNode(v)`, the parent of v (`INVALID` for the root), and by `predValue(v)`, the weight of the edge between v and its parent (undefined for the root, where it is in fact the largest value of `Value`). Also, `rootDist(v)` returns the number $o(v)$ of step 8: 0 for the root, and for any other node a number larger than that of its parent, distinct for distinct nodes, i.e., an order of the tree from the root, not the distance from it.

A minimum cut between two nodes is read by `minCutValue(s, t)`, which climbs the two paths of the tree to their common ancestor and returns the least weight met (Lemma 15.7 and Theorem 15.2). In the same way `minCutMap(s, t, cutMap)` finds the lightest edge of the path, writes `true` on the side of s and `false` on the other side into a readable and writable `bool` node map, and returns the value. For $s = t$, `minCutValue()` returns the largest value of `Value`, whereas `minCutMap()` (and hence the iterators below) must not be called, since it then writes a map at the node `INVALID`. A cut can also be listed by two iterators. `MinCutNodeIt(gomory, s, t, side = true)` lists the nodes of the side of s (or those of the other side if `side` is `false`). Similarly, `MinCutEdgeIt(gomory, s, t, side = true)` lists the edges of the cut, each once, as the arc that leaves the side of s (or enters it if `side` is `false`); the latter converts to both `Arc` and `Edge`. Each constructor calls `minCutMap()` on a node map that the iterator holds, at the cost of Theorem 15.8, and the postfix increments return a `Node` (respectively an `Arc`) instead of an iterator. Since a minimum cut need not be unique, `MinCutNodeIt(gomory, s, t, true)` and `MinCutNodeIt(gomory, t, s, false)` may list different nodes, whereas `MinCutNodeIt(gomory, s, t, true)` and `MinCutNodeIt(gomory, s, t, false)` together list each node once. The graph must have at least one node, since `run()` writes the maps of the class at the first node, which on an empty graph is `INVALID`.

abLEMON. `tolerance()` is new in abLEMON. LEMON 1.3.1 gave its maximum flows the default tolerance of the value type, while abLEMON gives them the one set on the class.



G, $\lambda(G) = 3$ *Gomory-Hu tree T*

Figure 15.2. Left, an undirected graph G and its minimum cut $X = \{0, 1, 2\}$, of value 3, with the two cut edges; right, the Gomory-Hu tree computed by `GomoryHu`, whose lightest edge (between 2 and 4) gives the same cut.

On the graph of Figure 15.2 the tree is the path $0, 1, 2, 4, 3, 5$ with the weights $5, 7, 3, 6, 6$; hence, e.g., $\lambda(0, 5) = 3$ and $\lambda(1, 2) = 7$. The program `manual/examples/15-undirected-cuts.cc` (the test `manual_15_undirected_cuts`) computes the minimum cut of this graph by `NagamochiIbaraki`, one phase at a time, and then its Gomory-Hu tree. It checks the value of the cut against the enumeration of all the cuts. For each of the 15 pairs of nodes, it also checks the value read from the tree against a maximum flow, the sides given by `minCutMap()`, and the sum of the capacities listed by `MinCutEdgeIt`.

15.5 Choosing an algorithm

For a digraph `HaoOrlin` is the only class of the chapter, and it is arguably also the natural choice when one needs the best cut on a given side of a node (the stored cut after `calculateOut()`). For the minimum cut of an undirected graph, `NagamochiIbaraki` uses no flow, and its phases may be far fewer than $n - 1$, since all the edges with $q(e) \geq \mu$ are contracted at once. On the other hand, `HaoOrlin` with the first phase only, on the `Undirector`, has the bound of a single maximum flow (Theorem 15.4), against the $O(nm \log n)$ of Theorem 15.6.

Which of the two is faster typically depends on the instance, and neither the headers nor the works cited here give a rule that settles it in general. `GomoryHu` costs $n - 1$ maximum flows. It pays off when the cuts of "many" pairs of nodes are needed, since each value is then read from the tree in $O(h)$ time, whereas for the cut of one pair (or of a "few" pairs) a maximum flow of chapter 12 is cheaper.

15.6 Exercises

Exercise 15.1. Check Theorem 15.1 on the digraph of Figure 15.1 by computing $\lambda(0, t)$ and $\lambda(t, 0)$ with `Preflow` for every $t \neq 0$ and comparing the least of them with `HaoOrlin::minCutValue()` (the program `manual/examples/15-hao-orlin.cc` gives the value 4).

Exercise 15.2. Show that, on the digraph `Undirector<GR>` of an undirected graph, the value stored after `calculateOut()` is already $\lambda(G)$, and verify it on the graph of `test/hao_orlin_test.cc`, whose test runs both phases.

Exercise 15.3. Compute the edge connectivity of the graph of Figure 15.2 with `NagamochiIbaraki< SmartGraph >::SetUnitCapacity::Create`, and check the cut it gives by counting its edges (the fourth block of `test/nagamochi_ibaraki_test.cc` does the same on another graph).

Exercise 15.4. Print the value `minCutValue()` of `NagamochiIbaraki` on the graph of Figure 15.2 after `init()` and after each call of `processNextPhase()`. Explain why the sequence is nonincreasing, and why its first value is the lightest star of a node (5, at node 0).

Exercise 15.5. Using only `predNode()` and `predValue()` of `GomoryHu`, count the distinct values among the $\lambda(s, t)$ of a graph, and deduce from Theorem 15.2 that there are at most $n - 1$ of them; check the values against `minCutValue(s, t)` for each pair, as `test/gomory_hu_test.cc` does against `Preflow`.

Exercise 15.6. Show that the sum of the capacities listed by `MinCutEdgeIt(gomory, s, t)` equals `minCutValue(s, t)`, and that the nodes listed by `MinCutNodeIt` with `side` true and false partition V (both facts are checked by `test/gomory_hu_test.cc`, the first also by `manual/examples/15-undirected-cuts.cc`).

16. Minimum mean cycles

We are concerned in this chapter with the directed cycles of least mean cost of a digraph, i.e., with the cycles whose cost divided by their number of arcs is minimum. abLEMON has three algorithms for this problem, in the group Minimum Mean Cycle Algorithms: Karp's algorithm (`KarpMmc`, in `lemon/karp_mmc.h`), its variant with early termination by Hartmann and Orlin (`HartmannOrlinMmc`, in `lemon/hartmann_orlin_mmc.h`), and Howard's policy iteration (`HowardMmc`, in `lemon/howard_mmc.h`). All three have the same interface, stated as the concept `concepts::MinMeanCycle` of `lemon/concepts/min_mean_cycle.h`. Inside the library, the minimum mean cycle is used by the cycle-canceling algorithm of chapter 13 and by the certificates of unboundedness of chapter 14. Section 16.1 states the problem and its optimality conditions, and sections 16.2 to 16.4 describe the three algorithms with their proofs; section 16.5 describes the interface they share, and section 16.6 has the exercises.

16.1 The problem

Let $D = (V, A)$ be a digraph with n nodes, m arcs and a cost $c(a) \in \mathbb{R}$ on each arc a. A *walk* is a sequence of arcs in which each arc starts where the previous one ends, and it is *closed* if the last arc ends where the first one starts; a directed cycle is a closed walk that visits no node twice. A closed walk W has $|W|$ arcs (counted with their repetitions) and cost $c(W) = \sum_{a \in W} c(a)$, and its *mean cost* is $\mu(W) = c(W)/|W|$. The minimum cycle mean of D is

$$\mu^*(D) \;=\; \min\{\, \mu(C) \;:\; C \text{ a directed cycle of } D \,\}. \tag{16.1}$$

A digraph without directed cycles has no minimum cycle mean, and the algorithms report this case. This problem is closely related to conservative costs, i.e., to costs under which no directed cycle has a negative cost (cf. Ahuja, Magnanti and Orlin 1993, chapter 5). We first collect the elementary facts that the proofs of this chapter use.

Lemma 16.1. *(i) A closed walk W is the union of directed cycles $C_1, \dots, C_p$ (possibly with repetitions), and $\min_i \mu(C_i) \le \mu(W) \le \max_i \mu(C_i)$; in particular, $\mu(W) \ge \mu^*(D)$, and if $\mu(W) = \mu^*(D)$ then each C_i is a minimum mean cycle. (ii) Adding a constant α to the cost of all the arcs adds α to the mean of each closed walk, and hence it leaves the minimum mean cycles unchanged. (iii) A directed cycle lies in one strongly connected component of D, and therefore $\mu^*(D)$ is the least of the minimum cycle means of the components that have a cycle.*

Proof. (i) If W visits no node twice (except at its end), it is a cycle. Otherwise, we take two visits of the same node with no repeated node between them: the arcs between them form a cycle C_1, and removing them leaves a shorter closed walk, to which we apply induction. Then $c(W) = \sum_i c(C_i)$ and $|W| = \sum_i |C_i|$, and $\mu(W)$ is the average of the $\mu(C_i)$ with weights $|C_i|/|W|$, which lies between their minimum and their maximum. If it equals $\mu^*(D)$, which is a lower bound on each $\mu(C_i)$, all of them equal it. (ii) $c(W) + \alpha|W|$ divided by $|W|$ is $\mu(W) + \alpha$. (iii) The nodes of a cycle are mutually reachable. □

The algorithms of this chapter use (iii) as it stands. They find the strongly connected components first, by `stronglyConnectedComponents()` of chapter 18, solve the problem on each component that has a cycle (i.e., on each component of at least two nodes, and on each node with a loop), and keep the best cycle. The optimality condition is the following one.

Theorem 16.1. *Let $\mu \in \mathbb{R}$. Each directed cycle of D has mean cost at least μ if and only if there are potentials $\pi : V \to \mathbb{R}$ such that*

$$c(u,v) - \mu + \pi(u) - \pi(v) \;\geq\; 0 \qquad \text{for every arc } (u,v) \in A. \tag{16.2}$$

Proof. If (16.2) holds, summing it along a cycle C cancels the potentials and gives $c(C) - \mu|C| \geq 0$, i.e., $\mu(C) \geq \mu$. Conversely, if no cycle has mean less than μ, the costs $c' = c - \mu$ give no cycle a negative cost. We add a node r and an arc of cost 0 from r to each node, and we let $\pi(v)$ be the least c'-cost of a walk from r to v. This value is finite, since removing the cycles of a walk (Lemma 16.1 (i)) does not increase its cost, and the least is therefore taken over the finitely many paths. Since a walk to u followed by (u,v) is a walk to v, one has $\pi(v) \leq \pi(u) + c(u,v) - \mu$, which is (16.2). □

Hence, $\mu^*(D)$ is the largest μ for which the costs $c - \mu$ are conservative, i.e., the negative of the smallest amount that, added uniformly to the costs, makes them conservative (as the documentation of the group puts it). Furthermore, a closed walk W together with potentials that satisfy (16.2) for $\mu = \mu(W)$ certifies, by Theorem 16.1 and Lemma 16.1 (i), that $\mu(W) = \mu^*(D)$ and that the cycles of W are minimum mean cycles. Hartmann and Orlin check a certificate of this kind to stop early, and Howard's algorithm stops when its distances (section 16.4) are such a certificate.

16.2 Karp's algorithm

Let D be strongly connected and have a cycle, and let s be any node. For $k = 0, \ldots, n$ and $v \in V$, let $D_k(v)$ be the least cost of a walk from s to v with exactly k arcs ($+\infty$ if there is none). Then $D_0(s) = 0$, $D_0(v) = +\infty$ for $v \neq s$, and

$$D_k(v) \;=\; \min\{\, D_{k-1}(u) + c(u,v) \;:\; (u,v) \in A \,\} \qquad k = 1, \ldots, n, \tag{16.3}$$

since the last arc of a walk of k arcs to v enters v from a node reached by a walk of $k-1$ arcs. Each node has an arc leaving it inside D (it lies on a cycle), and hence walks of any length start at s; in particular, $D_n(v) < \infty$ for some v. We use the following characterization, due to Karp (1978).

Theorem 16.2. *If D is strongly connected and has a cycle, then*

$$\mu^*(D) \;=\; \min_{v \in V:\, D_n(v) < \infty} \; \max_{0 \leq k < n:\, D_k(v) < \infty} \frac{D_n(v) - D_k(v)}{n - k}. \tag{16.4}$$

Furthermore, if v^* attains the minimum, a walk of n arcs from s to v^* of cost $D_n(v^*)$ visits some node twice, and the arcs between two visits of a node with no repeated node between them form a cycle of mean $\mu^*(D)$.

Proof. Let $\mu^* = \mu^*(D)$. Subtracting μ^* from the costs changes $D_k(v)$ into $D_k(v) - k\mu^*$, and hence each ratio of (16.4) into the ratio minus μ^*, while it leaves the walks that attain the values and (by Lemma 16.1 (ii)) the minimum mean cycles unchanged. We may therefore assume $\mu^* = 0$, i.e., that no cycle is negative and some cycle C has cost 0, and we must show that the right-hand side of (16.4) is 0. Let $\delta(v)$ be the least cost of a walk from s to v of any length; as in the proof of Theorem 16.1, it is attained by a path, which has fewer than n arcs, and hence $\delta(v) = \min_{0 \leq k < n} D_k(v)$. If $D_n(v) < \infty$, then $D_n(v) \geq \delta(v)$, and the inner maximum at v, which is at least $(D_n(v) - \delta(v))/(n-k)$ for the k that attains $\delta(v)$, is nonnegative.

To see that some v gives 0, let w be a node of C and P a path from s to w of cost $\delta(w)$, and, for two nodes x, y of C, let $c_C(x,y)$ be the cost of the arcs of C from x to y. Since $\delta(y) \leq \delta(x) + c_C(x,y)$, $\delta(x) \leq \delta(y) + c_C(y,x)$ and $c_C(x,y) + c_C(y,x) = c(C) = 0$, one has $\delta(y) = \delta(w) + c_C(w,y)$ for each node y of C. We follow P and then C until the walk has n arcs (which is possible, since $|P| < n$), and we let v be its last node. The cost of this walk is $\delta(w) + c_C(w,v) = \delta(v)$, which gives $D_n(v) \leq \delta(v)$, and hence $D_n(v) = \delta(v)$. The inner maximum at v is then at most 0, since $D_k(v) \geq \delta(v) = D_n(v)$ for all k, and therefore it is 0.

For the second claim, the minimizer v^* has inner maximum 0, which gives, by the same two inequalities, $D_n(v^*) = \delta(v^*)$. A walk W of n arcs has $n+1$ visits of nodes, and hence some node repeats. Removing from W its cycles, one at a time, as in Lemma 16.1 (i), leaves a path from s to v^* of cost at least $\delta(v^*)$, and the removed cycles have nonnegative costs. Since the total is $c(W) = \delta(v^*)$, each removed cycle has cost $0 = \mu^*$, and the cycle between the two visits of the statement may be taken as the first one removed. □

Karp's algorithm computes the table (16.3) round by round and then evaluates (16.4). In `KarpMmc` this is done on each strongly connected component that has a cycle, with s its first node and n its number of nodes. In the

last step, the walk back reads, at each node v and round k, the arc $\mathrm{pred}_k(v)$ through which $D_k(v)$ was last improved.

Algorithm 16.1 (Karp, on a strongly connected component of n nodes).

```
1  s <- the first node; D_0(s) <- 0; D_j(v) <- +inf for all other (v, j)
2  R <- { s }                              (the nodes with D_{k-1} < +inf)
3  for k = 1, ..., n:
4    for each node u of R (of the component, once R has all n nodes):
5      for each arc (u, v) of the component:
6        if D_{k-1}(u) + c(u, v) < D_k(v):
7          D_k(v) <- D_{k-1}(u) + c(u, v); pred_k(v) <- (u, v)
8    if R has fewer than n nodes: R <- { v : D_k(v) < +inf }
9  for each node v with D_n(v) < +inf:
10   M(v) <- max{ (D_n(v) - D_k(v)) / (n - k) : 0 <= k < n, D_k(v) < +inf }
11 mu <- min_v M(v), attained at v*                (findCycleMean ends here)
12 (findCycle) walk back from v* along pred_n, pred_{n-1}, ... until a
   node repeats; the arcs between its two visits form the cycle
```

Steps 4 to 8 are `processNextBuildRound()` while R has fewer than n nodes, and `processNextFullRound()` afterwards; the comparison of step 6 goes through the tolerance. In steps 10 and 11 the ratios are compared without division, by cross-multiplication in the type `LargeCost`. The first node (in the order of the component) that attains the minimum is kept. Across the components, the best cycle is replaced only by a strictly better one. As the next lemma shows, the full rounds add no cost to an infinite value.

Lemma 16.2. *Steps 3 to 8 of Algorithm 16.1 compute the values (16.3). Furthermore, at each round k the set R is the set of the nodes with $D_{k-1} < \infty$, and once it has all the n nodes, $D_j(v) < \infty$ for each v and each $j \geq k-1$.*

Proof. The minimum (16.3) is over the arcs (u, v) with $D_{k-1}(u) < \infty$, which are those that step 4 scans while $R = \{u : D_{k-1}(u) < \infty\}$. The build round collects these nodes as they first get a finite value (a node is added to the next R when its value for round k stops being infinite). If $D_{k-1}(u) < \infty$ for all the nodes u, then $D_k(v) < \infty$ for all the nodes v, since each node of the component has an arc entering it from the component, and the claim follows by induction on k. □

Theorem 16.3. *`KarpMmc::findCycleMean()` returns `false` if and only if the digraph has no directed cycle. Otherwise the mean of the pair it keeps, i.e., `cycleCost()` divided by `cycleSize()`, is $\mu^*(D)$, and `findCycle()` then stores a minimum mean cycle in the path.*

Proof. A digraph has a cycle if and only if some strongly connected component has two nodes or a loop, and these are the components that `initComponent()` does not skip. On each of them Lemma 16.2 gives the table (16.3), and Theorem 16.2 shows that steps 9 to 11 compute the minimum cycle mean of the component, at the node v^*; by Lemma 16.1 (iii) the best of the components gives $\mu^*(D)$. Step 12 follows $\mathrm{pred}_n(v^*)$, pred_{n-1}, and so on, which retrace a walk of n arcs of cost $D_n(v^*)$. It stops at the first repeated node, and by the second part of Theorem 16.2 its cycle has mean $\mu^*(D)$. □

We remark that until `findCycle()` has been called, `cycleCost()` and `cycleSize()` return the numerator $D_n(v^*) - D_k(v^*)$ and the denominator $n - k$ of the ratio of (16.4) at v^*. Their quotient is $\mu^*(D)$, but they need not be the cost and the length of a cycle; after `findCycle()`, they are those of the cycle found in step 12.

Theorem 16.4. *Algorithm 16.1 runs in $O(nm)$ time and $O(n^2 + m)$ space on a digraph of n nodes and m arcs.*

Proof. The strongly connected components, with their lists of arcs, cost $O(n+m)$. A component of n_i nodes has $m_i \geq n_i$ arcs, since each node has a leaving arc, and each of its n_i rounds costs $O(n_i + m_i)$; steps 9 to 11 cost $O(n_i^2)$ and step 12 costs $O(n_i)$. The sum over the components is $O(nm)$. The table of a node of the component i has $n_i + 1$ entries, which gives $\sum_i n_i(n_i+1) = O(n^2)$, plus $O(m)$ for the lists of arcs. □

16.3 The Hartmann-Orlin algorithm

Hartmann and Orlin (1993) observed that Karp's table often contains a minimum mean cycle long before round n, and that Theorem 16.1 allows one to recognize it. At some rounds k the algorithm follows backwards the walks that end at round k, takes the closed walk W of least mean among those that these walks close, and computes the potentials

$$\pi(v) \;=\; \min_{0\le j\le k:\, D_j(v)<\infty} \big(\, |W|\, D_j(v) - j\, c(W) \,\big). \tag{16.5}$$

Thus, $\pi(v)$ is $|W|$ times the least cost under $c-\mu(W)$ of a walk from s to v with at most k arcs ($+\infty$ if there is no such walk). If these potentials satisfy (16.2) for $\mu=\mu(W)$, scaled by $|W|$, then W and its cycles are optimal by Theorem 16.1, and the algorithm stops.

Algorithm 16.2 (Hartmann-Orlin, on a strongly connected component of n nodes).

```
1  as steps 1 to 8 of Algorithm 16.1; after the rounds k = 4, 6, 9, 13,
   ... (each the previous one times 3/2, rounded down) and after k = n:
2    for each node v with D_k(v) < +inf, walk back from v along
     pred_k, pred_{k-1}, ..., pred_1; each time the walk meets again a
     node u, met last at round j' > j, it closes the closed walk of
     cost D_j'(u) - D_j(u) and of j' - j arcs
3    W <- the closed walk of least mean among those closed in step 2,
     with its node u and its round j'
4    if a closed walk was found and k < n:
5      compute pi by (16.5)
6      if |W| c(u, v) - c(W) >= pi(v) - pi(u) for every arc (u, v) of
       the component: stop with W
7    if k = n: stop with W
8  (findCycle) walk back from u along pred_j', pred_{j'-1}, ... until a
   node repeats; the arcs between its two visits form the cycle
```

This schedule of the checks is the one of `processRounds()` in `lemon/hartmann_orlin_mmc.h`, and steps 2 to 7 are `checkTermination()`; the comparisons of steps 5 and 6 go through the tolerance, and step 6 scans the arcs inside the component, the only ones whose ends have potentials. Across the components, the closed walk of least mean is kept (the first one, in case of ties), and `findCycle()` performs step 8 on it.

abLEMON. In LEMON 1.3.1 step 6 scanned the arcs of the whole digraph, whose ends outside the component have no potentials computed, and hence the test could fail more often and delay the stop.

Theorem 16.5. *Assume exact arithmetic, with $+\infty$ larger than all finite values and $+\infty - x = +\infty$ for x finite. On a strongly connected component with a cycle, Algorithm 16.2 stops with a closed walk W of mean μ^* (the minimum cycle mean of the component), and the cycle of step 8 is a minimum mean cycle.*

Proof. The walk back from a node v at round k retraces a walk of k arcs of cost $D_k(v)$, whose prefix of j arcs, which ends at a node u, has cost $D_j(u)$. The arcs between two visits of u at rounds $j<j'$ are therefore a closed walk of cost $D_{j'}(u)-D_j(u)$ and of $j'-j$ arcs, and by Lemma 16.1 (i) its mean is at least μ^*. If the algorithm stops in step 6, all the potentials are finite: otherwise, since $\pi(s)=0$ and the component is strongly connected, some arc (u,v) has $\pi(u)<\infty=\pi(v)$, and it violates the test. Hence $\pi/|W|$ satisfies (16.2) for $\mu=\mu(W)$ on the arcs of the component, and $\mu(W)\le\mu^*$ by Theorem 16.1 (applied to the component). If the algorithm stops in step 7, at $k=n$, the walk back from the node v^* of Theorem 16.2 closes, at its first repeated node, a cycle of mean μ^*, and the least mean of step 3 is again at most μ^*. In both cases $\mu(W)=\mu^*$. The walk back of step 8 starts where the walk of step 2 met u at round j', and it meets a node twice at the latest when it reaches u again at round j. Hence its cycle consists of arcs of W, and it is one of the cycles of a decomposition of W as in Lemma 16.1 (i), each of which has mean μ^* by that lemma. □

This assumption holds when `LargeCost` is a floating-point type, whose infinity the code uses, but not when it is an integer type (the default for integer costs), where $+\infty$ is represented by the largest value. In that case, at a check made before all the nodes are reached by walks of at most k arcs, the difference $\pi(v)-\pi(u)$ of step 6 with $\pi(v)$ infinite and $\pi(u)<0$ overflows. The violated test may then pass, and the algorithm may stop with a closed walk that is not optimal; the checks made when all the potentials are finite, and the stop at $k=n$, are not affected.

Theorem 16.6. *Algorithm 16.2 runs in $O(nm)$ time and $O(n^2+m)$ space.*

Proof. As in Theorem 16.4, the rounds and the table cost $O(nm)$ and $O(n^2+m)$. On a component of n_i nodes and $m_i\ge n_i$ arcs, the check at round k walks back k arcs from at most n_i nodes, computes (16.5) in $O(n_ik)$ and scans $O(m_i)$ arcs. The rounds of the checks grow geometrically, with ratio 3/2, up to n_i, plus the check at n_i; hence their sum is $O(n_i)$ and their number $O(\log n_i)$. The checks therefore cost $O(n_i^2+m_i\log n_i)$, which is $O(n_im_i)$. □

In the worst case the checks add to the work of Karp's algorithm, and the bounds are the same; according to the header, the early termination makes the algorithm significantly faster on some instances, but slower on others. Unlike those of `KarpMmc`, the values of `cycleCost()` and `cycleSize()` after `findCycleMean()` alone are the cost and the number of arcs of the closed walk W, which may visit a node more than twice. After `findCycle()`, they are those of the cycle of step 8, of the same mean.

16.4 Howard's policy iteration

Howard's algorithm comes from the policy iteration of Markov decision processes (Howard 1960); its use for the minimum mean cycle is analysed in Cochet-Terrasson et al. (1998) and in Dasdan and Gupta (1998). A *policy* p chooses one leaving arc $p(u)$ for each node u of a strongly connected component, and the arcs chosen form the *policy graph*, in which every node has out-degree 1. Following the policy from any node one meets a node twice, and hence each node leads to a cycle of the policy graph. Each iteration evaluates the policy by the cycle C of least mean among them, with $\lambda = \mu(C)$, and computes the *distances* d towards C under the costs $w(u,v) = |C|\,c(u,v) - c(C) = |C|\,(c(u,v) - \lambda)$, which are scaled by $|C|$ to keep integer costs integer; then it improves the policy by these distances.

Algorithm 16.3 (Howard, on a strongly connected component).

```
1  for each node u: p(u) <- an arc (u, v) of least cost
2  loop
3    find the cycles of the policy graph; C <- the one of least mean,
     v0 <- the first node of C met
4    d(v0) <- 0; backwards from v0 along the policy arcs (u, v),
     breadth-first: d(u) <- d(v) + w(u, v)
5    backwards, breadth-first, from the nodes reached, along any arc
     (u, v) with u not reached: p(u) <- (u, v); d(u) <- d(v) + w(u, v)
6    improved <- false
7    for each arc (u, v): if d(v) + w(u, v) < d(u):
8      d(u) <- d(v) + w(u, v); p(u) <- (u, v); improved <- true
9    if not improved: return C
```

Steps 1, 3 and 4 to 9 are `buildPolicyGraph()`, `findPolicyCycle()` and `computeNodeDistances()` of `lemon/howard_mmc.h`, and each pass of the loop is one iteration. In step 3 the nodes are taken in the order of the component, each one followed along the policy until a node already met, and C is replaced only by a cycle of strictly smaller mean (compared by cross-multiplication). The test of step 7 goes through the tolerance, and the updates of step 8 are made in place, i.e., an arc scanned later reads the distances already decreased; step 1 compares the costs exactly.

Lemma 16.3. *After step 5 of an iteration, all the nodes of the component have a distance, the policy graph consists of the cycle C and of arcs that lead to it, and $d(u) = d(v) + w(u,v)$ for each policy arc $(u,v) = p(u)$, that of v_0 included.*

Proof. Step 4 reaches the nodes whose policy leads to v_0, among which are those of C, and step 5 reaches all the others, since in a strongly connected component each node has a path to the nodes reached. Each node u gets its distance from the arc $p(u)$ that reaches it, which leads to an already reached node, and hence to C. The equality holds by construction for $u \neq v_0$, and for v_0 because the sum of w along C is $|C|c(C) - |C|c(C) = 0$. □

Theorem 16.7. *Let C be the cycle of an iteration and C' the cycle of the next one. Then $\mu(C') \leq \mu(C)$. Furthermore, if step 7 improves no distance (and the comparisons are exact), C is a minimum mean cycle of the component.*

Proof. Let d' and p' be the distances and the policy at the end of step 8; we claim that $d'(u) \geq d'(v) + w(u,v)$ for $(u,v) = p'(u)$ and all the nodes u. If u is never improved, then $p'(u) = p(u)$ and, by Lemma 16.3, $d'(u) = d(u) = d(v) + w(u,v) \geq d'(v) + w(u,v)$, since distances only decrease. If u is improved, the last improvement set $d'(u)$ to $d(v) + w(u,v)$ for the current value $d(v)$, which is at least $d'(v)$. The cycle C' is a cycle of p', and summing the claim along it gives $0 \geq \sum_{a \in C'} w(a) = |C|\,c(C') - |C'|\,c(C)$, i.e., $\mu(C') \leq \mu(C)$. If no distance is improved, then $d(u) \leq d(v) + w(u,v)$ for each arc (u,v) of the component; the potentials $\pi = -d/|C|$ then satisfy (16.2) for $\mu = \lambda$, and C is optimal by Theorem 16.1. □

Termination needs more care, since an iteration may leave the mean unchanged. In the literature, the result is the following one, for the variant of policy iteration in which a policy arc is replaced only by a strictly better one

and the root of the distances is kept from one iteration to the next whenever it is still on a cycle of least mean.

Theorem 16.8. *In that variant, no policy is met twice; hence policy iteration terminates in fewer iterations than the number* $\prod_u \deg^+(u)$ *of the policies, and each iteration costs* $O(n+m)$ *time on a component of* n *nodes and* m *arcs.*

Proof (sketch). An iteration costs as Algorithm 16.3: step 3 follows each policy arc a bounded number of times, steps 4 and 5 are two breadth-first searches, and step 7 scans each arc once. As for the termination, by Theorem 16.7 the mean never increases, and hence the iterations in which it decreases cannot return to a previous policy. At an iteration in which the mean does not change, keeping the root and the policy arcs that are not strictly improved makes the vector of the distances decrease lexicographically, by a maximum principle for the transient part of the policy graph. The complete proof, for the max-plus formulation that contains this problem, is Theorem 4.5 of Cochet-Terrasson et al. (1998), with the rules of their Algorithms 4.3 and 4.4; it adapts the proof of Denardo and Fox (1968) for Markov renewal programs. □

We remark that `HowardMmc` replaces a policy arc only when the distance decreases by more than the tolerance, as the variant requires, but it takes as root v_0 the first node met of the cycle of least mean, without preferring the root of the previous iteration. Theorem 16.7 bounds the number of the iterations in which the mean decreases by the number of the distinct cycle means; however, we know of no bound for the code as written on the iterations in which the mean stays the same (no instance of cycling is known to us, and the limit on the iterations described below bounds them in any case). As the header remarks, the best known bound on the number of iterations is exponential, while the documentation of the group reports that in practice Howard's algorithm is by far the most efficient of the three, in agreement with the experiments of Dasdan (2004).

One can also limit the iterations, and then the algorithm returns a cycle that is usually good, but not necessarily optimal. Indeed, `findCycleMean(limit)` takes the largest number of iterations (by default the largest `int`, i.e., no limit), counted over all the components together, and it checks the limit before each iteration. It returns a value of the enumeration `TerminationCause`: `NO_CYCLE` (0) if the digraph has no directed cycle, `OPTIMAL` (1) if the minimum mean was found, and `ITERATION_LIMIT` (2) if the limit was reached first. In the last case the cycle kept is the best among the policy cycles of the components processed until then, whose mean is an upper bound on $\mu^*(D)$. The components after the one where the limit was reached are not processed at all. If the limit stops the first component that has a cycle before its first iteration (e.g., with a limit of 0), no cycle is kept, and `findCycle()` returns `false`. Since `NO_CYCLE` is 0, the result converts to `false` only when there is no cycle, and `run()` keeps the meaning of a `bool`.

Howard's algorithm ends on each component with a policy whose cycle is a minimum mean cycle of that component, and `HowardMmc` keeps these cycles, one for each component that has a cycle. The number of such components is `componentCycleNum()`, and for $0 \le i <$ `componentCycleNum()`, in the order of the components, `componentCycleCost(i)` and `componentCycleSize(i)` give the cost and the number of arcs of the i-th cycle, while `componentCycle(i, path)`, a template on the path type, appends it to any path with `addBack()`. These cycles are pairwise node-disjoint, since the components are. They are of minimum mean in their components only if `findCycleMean()` was not stopped by its limit; otherwise they are those of the components processed until then, the last one possibly not optimal (a component where the limit stopped the search before its first policy cycle keeps none). The min-cost flow algorithms of chapter 13 take from one run of `HowardMmc` the disjoint negative cycles that their function `unboundedCycles(k)` returns (see chapter 14).

> **abLEMON.** `componentCycleNum()`, `componentCycleCost()`, `componentCycleSize()` and `componentCycle()` are new in abLEMON (version 1.9.0), and `unboundedCycles(k)` of the four minimum cost flow algorithms reads them; a run stopped by the limit before the first policy cycle of a component no longer keeps the cycle of the previous component a second time. The iteration limit of `findCycleMean()` was already in LEMON 1.3.

16.5 The common interface

Each of the three classes is a template `KarpMmc< GR , CM , TR >` (and likewise `HartmannOrlinMmc` and `HowardMmc`), where `GR` is the digraph type, `CM` a readable arc map of costs, by default `GR::ArcMap< int >`, and `TR` the traits class, by default `KarpMmcDefaultTraits< GR , CM >` (respectively `HartmannOrlinMmcDefaultTraits` and `HowardMmcDefaultTraits`). Their default traits define `Digraph`, `CostMap`, `Cost` (the value type of the costs), `LargeCost` (the type of the internal computations), `Tolerance`, which is `lemon::Tolerance< LargeCost >`, and `Path`, the type of the cycle found, which is `lemon::Path< Digraph >`. Each class repeats these typedefs and adds `Traits`, the traits class itself. By default, `LargeCost` is `long long` when `Cost` is an integer type (`long` if the compiler has no `long long`) and `double` otherwise, and `Cost` must be convertible to it. Also, `Path` must meet the concept of chapter 8 and have `addFront()` for `KarpMmc` and `HartmannOrlinMmc`, `addBack(`

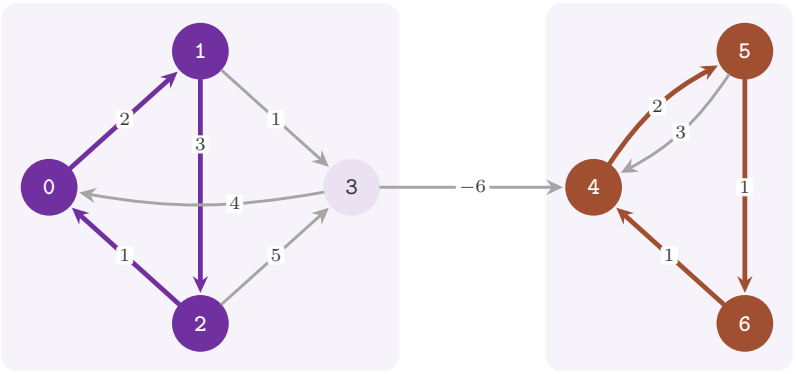


Figure 16.1. A digraph with two strongly connected components (the tinted regions) and the cost of each arc on its plate: the cycle 0, 1, 2 of mean 2 is the best of the first component, and the cycle 4, 5, 6 of mean 4/3 the minimum mean cycle of the digraph.

) for `HowardMmc`, which are (with `clear()`) the only functions the algorithms call on it. Two named template parameters change the traits. `SetLargeCost< T >` sets `LargeCost` to `T` and `Tolerance` to `lemon::Tolerance< T >`, and `SetPath< T >` sets `Path`; e.g., `HowardMmc< G , CM >::SetPath< ListPath< G > >::Create` stores the cycle in a `ListPath`.

Each class is constructed from the digraph and the costs, and `cycle(path)` makes the algorithm store the cycle in an external path (otherwise the first run allocates one, which the destructor deletes). Then `tolerance(tol )` and `tolerance()` set and read the tolerance; the first two functions return the object, and hence calls may be chained, as in `mmc.cycle(p).run()`. To execute the algorithm one calls `run()`, which is `findCycleMean( ) && findCycle()` and returns `true` if the digraph has a directed cycle (for `HowardMmc`, with no limit on the iterations); it may be called again after the digraph or the costs have changed. In more detail, `findCycleMean( )` computes the minimum mean (it returns a `bool` in `KarpMmc` and `HartmannOrlinMmc`, a `TerminationCause` in `HowardMmc`), and `findCycle()` builds the cycle from its data, appending the arcs to the path, and returns `fals e` if no cycle was found. Note that `findCycleMean()` clears the path, and `findCycle()` does not; hence, `findC ycle()` should be called once after each `findCycleMean()`.

We read the results by `cycleCost()`, a `Cost` converted from the `LargeCost` value kept by the algorithm, and `cy cleSize()`, an `int` (before a cycle is found, they are 0 and 1). Also, `cycleMean()` is the `double` quotient of the two, computed from the `LargeCost` value, and `cycle()` is a constant reference to the path. In all three classes, the tolerance is used where the algorithms compare costs of walks or distances, namely in step 6 of Algorithm 16.1, in steps 5 and 6 of Algorithm 16.2 and in step 7 of Algorithm 16.3. The comparisons of the means of two cycles or closed walks, on the other hand, are exact cross-multiplications in `LargeCost`, which must therefore hold the products of a cost of a walk by a number of arcs; with integer costs, this is the reason why `LargeCost` is `long long` by default.

In `lemon/concepts/min_mean_cycle.h`, the concept `concepts::MinMeanCycle< GR , Cost >` checks this interface, on the class obtained with `SetPath< ListPath< GR > >` and `SetLargeCost< Cost >`. The test `test/ min_mean_cycle_test.cc` checks the three classes against it, with `int` and `float` costs, and checks that `Large Cost` is `long long` for `int` costs and `double` for `float` costs; then it solves four cost functions of a digraph of 7 nodes with the three classes.

abLEMON. The concept `concepts::MinMeanCycle` is in `lemon/concepts/min_mean_cycle.h` since abLEMON 1.9.0, where the classes added to the library are held to it; in LEMON 1.3.1 the same checks were written in the test.

The program `manual/examples/16-min-mean-cycle.cc` (the test `manual_16_min_mean_cycle`) solves the problem of Figure 16.1 with the three classes, and checks that they agree on the mean 4/3 and that each cycle is a closed walk with the cost and the length reported. It certifies the mean by Theorem 16.1, since `BellmanFord` of chapter 10, started with all the distances equal to 0, finds no negative cycle under the costs $|C|\,c(a) - c(C)$, and it checks that `HowardMmc` gives the two cycles of the components, of means 2 and 4/3, and, with a limit of one iteration, a cycle whose mean is not less than the optimal one.

In our view, the choice among the three is usually simple: `HowardMmc` is the fastest in practice and the only one with the cycles of the components and the limit on the iterations, while `KarpMmc` and `HartmannOrlinMmc` have the polynomial bound $O(nm)$ (Theorems 16.4 and 16.6), which may matter when the running time must be

bounded a priori.

16.6 Exercises

Exercise 16.1. Verify Theorem 16.1 on the digraph of Figure 16.1 by computing with `BellmanFord` the potentials of (16.2) for $\mu = 4/3$, scaled by 3 (the program `manual/examples/16-min-mean-cycle.cc` checks only that no negative cycle exists).

Exercise 16.2. Compute by hand the table $D_k(v)$ of (16.3) for the component $\{4, 5, 6\}$ of Figure 16.1, with $s = 4$, and evaluate (16.4); then compare `cycleCost()` and `cycleSize()` of `KarpMmc` after `findCycleMean()` and after `findCycle()`.

Exercise 16.3. Run the three classes on the four cost functions of `test/min_mean_cycle_test.cc`, and check that they find the cycles that the test expects (the test does it with `checkMmcAlg()`).

Exercise 16.4. For the cost function `len1` of `test/min_mean_cycle_test.cc`, find the least limit for which `HowardMmc::findCycleMean(limit)` returns `OPTIMAL` (the test checks that 2 iterations are not enough and 4 are).

Exercise 16.5. Use `componentCycleNum()` and `componentCycle()` of `HowardMmc` to list, for a digraph with several strongly connected components, the node-disjoint cycles of minimum mean of the components, and keep those of negative mean, as `unboundedCycles(k)` does (the program of section 16.5 lists the two cycles of Figure 16.1).

Exercise 16.6. Check Lemma 16.1 (ii) with any of the three classes on the digraph of Figure 16.1, by adding the same constant to all the costs.

Exercise 16.7. Build a digraph on which Algorithm 16.2 checks its certificate at round 4 while some node has no walk of at most 4 arcs from s, e.g., a cycle of 10 nodes through s with a loop at s whose mean is not the least. Then compare the cycles found by `HartmannOrlinMmc` with `int` costs and with `SetLargeCost< double >` (see the remark after Theorem 16.5).

17. Matchings

We are concerned in this chapter with finding, in an undirected graph $G = (V, E)$ with n nodes and m edges, a set of edges no two of which share an end, i.e., a *matching*, which is as large as possible, or of largest total weight, or of least total cost among those that cover a prescribed set of nodes. abLEMON solves these problems with three headers. The first, `bipartite_matching.h`, holds the algorithms for bipartite graphs (`HopcroftKarp` and `MinCostAssignment`). The second, `matching.h`, holds the blossom algorithms of Edmonds for general graphs (`MaxMatching`, `MaxWeightedMatching` and `MaxWeightedPerfectMatching`). The third, `fractional_matching.h`, holds the algorithms for the relaxation in which an edge may be taken by halves (`MaxFractionalMatching`, `MaxWeightedFractionalMatching` and `MaxWeightedPerfectFractionalMatching`), which `matching.h` also uses to start its weighted algorithms. Each class gives, besides the matching, a dual solution that proves it optimal, and we describe the two together, since one usually checks a result against its dual (a "proof" that the user can verify in linear time, or nearly so). The structure of the chapter is as follows. Section 17.1 states the problems and the duality results on which the algorithms rest; sections 17.2 and 17.3 treat the bipartite case, sections 17.4 and 17.5 the general one, section 17.6 the fractional relaxations, and section 17.7 gives the exercises.

17.1 Problems, duality and the choice of an algorithm

Let $\delta(u)$ denote the set of the edges incident to the node u, $\gamma(X)$ the set of the edges with both ends in $X \subseteq V$, and $N(X)$ the set of the nodes adjacent to at least one node of X. A matching M *covers* the ends of its edges, and it is *perfect* when it covers all the nodes. The cardinality of a maximum matching is denoted by $\nu(G)$. With a weight w_e on each edge, the maximum weight matching problem is the integer program

$$\max\Big\{\sum_{e\in E} w_e x_e \;:\; \sum_{e\in\delta(u)} x_e \le 1 \quad \forall u \in V, \quad x \in \{0,1\}^E\Big\}, \tag{17.1}$$

of which the maximum cardinality matching problem is the case $w \equiv 1$ and the maximum weight perfect matching problem the case with equations in place of the inequalities (a minimum cost perfect matching is then found by negating the costs). The classes of this chapter ignore the loops, except where a flag of the fractional classes says otherwise (cf. section 17.6). An *alternating path* with respect to M is a path whose edges alternate between M and $E \setminus M$. It is *augmenting* when it has at least one edge and neither of its ends is covered by M. Exchanging the edges of M and of the path along an augmenting path P gives the matching $M \oplus P$ (where $\oplus$ is the symmetric difference) with one more edge, and the algorithms of the chapter that compute an integer matching all rest on the converse.

Theorem 17.1 (Berge). *A matching M of G is maximum if and only if no path is augmenting with respect to M.*

Proof. If P is augmenting, $M \oplus P$ is a larger matching. Conversely, let M^* be a matching with $|M^*| > |M|$. Each node has at most one edge of M and one of M^* in $M \oplus M^*$; hence, the subgraph $(V, M \oplus M^*)$ is made of node-disjoint paths and cycles whose edges alternate between the two matchings. A cycle has as many edges of M as of M^*, and so has a path with an even number of edges; since $|M^* \setminus M| > |M \setminus M^*|$, some path has one more edge of M^* than of M, i.e., it starts and ends with an edge of M^*. Its ends are not covered by M (an edge of M at an end would continue the path), and therefore the path is augmenting with respect to M. □

The result is due to Berge 1957 (cf. also Schrijver 2003, ch. 24). In a bipartite graph, whose nodes are split into the *red* nodes R and the *blue* nodes B with each edge joining a red node to a blue one (the `BpGraph` structures of chapter 4), the dual object of a matching is a *vertex cover*, i.e., a set of nodes that contains at least one end of each edge. In a general graph the dual object is rather a set of nodes A, called a *barrier*, whose removal leaves many components with an odd number of nodes. We denote by $o(G - A)$ the number of the odd components of $G - A$. The two weak duality bounds are the following.

Lemma 17.1. *(i) For a matching M and a vertex cover C of any graph, $|M| \le |C|$. (ii) For a matching M and a set $A \subseteq V$, at least $o(G-A) - |A|$ nodes are left uncovered by M; hence,*

$$\nu(G) \;\le\; \tfrac{1}{2}\big(n + |A| - o(G-A)\big) \qquad \forall\, A \subseteq V. \tag{17.2}$$

Proof. (i) Each edge of M has an end in C, and distinct edges of M have distinct ends. (ii) In an odd component K of $G-A$ the edges of M with both ends in K cover an even number of its nodes; hence, at least one node of K is either uncovered or matched to a node of A. At most $|A|$ components are of the second kind, and therefore at least $o(G-A) - |A|$ nodes are uncovered, i.e., $2|M| \le n - o(G-A) + |A|$. □

In a bipartite graph the first bound is tight. We give the proof here, since `HopcroftKarp` computes the cover by its construction (cf. section 17.2).

Theorem 17.2 (König). *In a bipartite graph the maximum cardinality of a matching equals the minimum cardinality of a vertex cover. More precisely, if M is a maximum matching and Z is the set of the nodes reachable from the uncovered red nodes by alternating paths that start with an edge not in M, then $C = (R \setminus Z) \cup (B \cap Z)$ is a vertex cover with $|C| = |M|$.*

Proof. Let rb be an edge with $r \in R$ and $b \in B$, and suppose that $r \in Z$ and $b \notin Z$. If $rb \notin M$, the alternating path that reaches r ends at r with an edge of M or is empty (r uncovered), and it continues along rb to b, against $b \notin Z$. If $rb \in M$, then r is covered, hence not a start, and it is reached only through its matching edge, i.e., from b, again against $b \notin Z$. Thus, each edge has its red end in $R \setminus Z$ or its blue end in $B \cap Z$, and C is a cover. Each blue node of Z is covered by M (otherwise the path that reaches it would be augmenting, against Theorem 17.1), and each red node outside Z is covered (the uncovered red nodes are in Z); furthermore, no edge of M has both ends in C, since the mate of a blue node of Z is in Z. Hence $|C| \le |M|$, and equality follows from Lemma 17.1. □

The result is due to König 1931 (cf. also Schrijver 2003, ch. 16). In a general graph a minimum vertex cover may be twice as large as a maximum matching (as in a triangle), and it is (17.2) that becomes tight.

Theorem 17.3 (Tutte–Berge formula). *For every graph,*

$$\nu(G) \;=\; \min_{A \subseteq V} \; \tfrac{1}{2}\big(n + |A| - o(G-A)\big). \tag{17.3}$$

Proof. The inequality $\le$ is (17.2). For the converse, Theorem 17.10 of section 17.4 shows that, at the end of `MaxMatching` on G, the set A of the nodes with status `ODD` satisfies (17.2) with equality for the matching found. □

The formula is due to Tutte 1947 and Berge. The barrier that `MaxMatching` returns is moreover the canonical one of the following structure theorem, which we do not prove (its proof takes several pages).

Theorem 17.4 (Gallai–Edmonds decomposition). *Let D be the set of the nodes left uncovered by at least one maximum matching, A the set of the nodes not in D with a neighbour in D, and $C = V \setminus (D \cup A)$. Then A attains the minimum in (17.3). Each component of the subgraph induced by D is factor-critical (i.e., it has a perfect matching after the removal of any one of its nodes). The subgraph induced by C has a perfect matching, and each maximum matching matches the nodes of A to nodes of D in distinct components.*

Proof (sketch). One runs Edmonds' search of section 17.4 to its end and shows that the nodes left `EVEN` are those of D (a node is `EVEN` exactly when an alternating path of even length joins it to an uncovered root, and exchanging along that path gives a maximum matching that leaves it uncovered), that the `ODD` nodes are its neighbours, and that the blossoms of the `EVEN` nodes are factor-critical. The full proof is in Schrijver 2003, ch. 24, after Gallai 1964 and Edmonds 1965a. □

For the weighted problems the inequalities of (17.1) do not describe the convex hull of the matchings, since the point with $x_e = \frac{1}{2}$ on the edges of a triangle satisfies them and is not a combination of matchings. Let $\mathcal{O}$ be the family of the subsets of V of odd cardinality at least 3; Edmonds showed that the *odd set inequalities*

$$\sum_{e \in \gamma(S)} x_e \;\le\; \frac{|S|-1}{2} \qquad \forall\, S \in \mathcal{O} \tag{17.4}$$

are the missing ones, and the weighted classes of `matching.h` work on his description. Its linear programming dual has a variable y_u for each node and a variable $z_S \ge 0$ for each odd set:

$$\min \Big\{ \sum_{u \in V} y_u + \sum_{S \in \mathcal{O}} \frac{|S|-1}{2}\, z_S \;:\; y_u + y_v + \sum_{S \in \mathcal{O}:\, uv \in \gamma(S)} z_S \;\ge\; w_{uv} \;\; \forall\, uv \in E \Big\}, \tag{17.5}$$

with $y \geq 0$ for the version with inequalities and y free for the perfect version. The *slack* of the edge uv is $y_u + y_v + \sum_{S:\, uv \in \gamma(S)} z_S - w_{uv}$, and the edge is *tight* when its slack is 0.

Lemma 17.2. *Let M be a matching (a perfect one, in the perfect version) and (y, z) a feasible solution of (17.5) such that (i) each edge of M is tight, (ii) each node with $y_u > 0$ is covered by M (in the version with inequalities), and (iii) each set S with $z_S > 0$ contains $\frac{|S|-1}{2}$ edges of M. Then M is a maximum weight matching (perfect matching), and (y, z) is an optimal solution of (17.5).*

Proof. Summing the equations (i) over the edges of M gives $w(M) = \sum_{uv \in M}(y_u + y_v) + \sum_S z_S |M \cap \gamma(S)|$, which by (ii) and (iii) is the objective of (17.5). For any matching M' (perfect, in that version), the constraints of (17.5), $y \geq 0$ (or M' perfect) and $|M' \cap \gamma(S)| \leq \frac{|S|-1}{2}$ give

$$w(M') \;\leq\; \sum_{uv \in M'} (y_u + y_v) + \sum_S z_S |M' \cap \gamma(S)| \;\leq\; \sum_u y_u + \sum_S \frac{|S|-1}{2} z_S \;=\; w(M).$$

The same chain applied to a feasible point of the relaxation proves the optimality of (y, z). □

Theorem 17.5 (Edmonds). *The convex hull of the incidence vectors of the matchings of G is the set Q of the $x \geq 0$ that satisfy $\sum_{e \in \delta(u)} x_e \leq 1$ for each $u \in V$ and (17.4); that of the perfect matchings is obtained by turning the node inequalities into equations.*

Proof. Let P be the convex hull; clearly $P \subseteq Q$, and Q is a polytope. For any w, the algorithm of section 17.5 finds a matching M and a feasible (y, z) that satisfy the conditions of Lemma 17.2 (Theorem 17.12); since the dual objective bounds $w^\top x$ from above on Q (the chain of the proof of Lemma 17.2), $\max_P w^\top x \geq w(M) \geq \max_Q w^\top x$. Two polytopes with the same maximum of each linear function coincide. The perfect version is identical, with Theorem 17.12 for `MaxWeightedPerfectMatching`. □

The theorem is in Edmonds 1965b; cf. also Schrijver 2003, ch. 25 and 26. Dropping (17.4) gives the *fractional matching* problem, whose dual is (17.5) without the z variables. Its optimum bounds that of (17.1) from above, and a triangle with unit weights shows that the bound may be strict ($\frac{3}{2}$ against 1). The algorithms of section 17.6 always return an optimal fractional matching made of a matching, with value 1 on its edges, and of node-disjoint odd cycles, with value $\frac{1}{2}$ on theirs (Theorems 17.14 and 17.16). Indeed, each vertex of the fractional matching polytope has this form (Schrijver 2003, ch. 30).

As for the choice among the classes, a bipartite graph is best handled by `bipartite_matching.h`, i.e., by `HopcroftKarp` for the cardinality, in $O((n+m)\sqrt{n})$ time, and by `MinCostAssignment` for a matching of least cost that covers all the red nodes, in $O(n_R\,(n + m \log m))$ time with $n_R = |R|$. A general graph needs the blossoms (which make the algorithms somewhat more involved), i.e., `MaxMatching` for the cardinality and `MaxWeightedMatching` or `MaxWeightedPerfectMatching` for the weights, the latter two in $O(nm \log n)$ time. In our view, the fractional classes are mostly of interest when the relaxation is itself the object (e.g., as a bound), and as the starting point of the weighted blossom algorithms, which call them by default. Of course, one can also write an assignment problem as a minimum cost flow problem (chapter 13), and a bipartite cardinality matching as a maximum flow problem (chapter 12). This requires building the network, which may be rather wasteful when the graph is "large". The classes of this chapter work on the graph itself, and they give the dual objects described above directly.

17.2 Maximum cardinality matching in bipartite graphs

`HopcroftKarp< BPGR >` finds a maximum matching of a bipartite graph of type `BPGR`, together with the minimum vertex cover of Theorem 17.2, by the algorithm of Hopcroft and Karp 1973. The algorithm grows the matching in *phases*. In each phase a breadth-first search from the uncovered red nodes labels each red node r with the number $d(r)$ of matched edges on a shortest alternating path that reaches it from an uncovered red node, and it stops at the first layer L from which an uncovered blue node is adjacent. An augmenting path then has $2L + 1$ edges at least. Then a depth-first search from each uncovered red node follows these labels, one layer at a time, until it meets an uncovered blue node from layer L, and the matching is augmented along the path found.

Algorithm 17.1 (Hopcroft–Karp).

```
1  M <- greedy matching (greedyInit) or empty matching (init)
2  loop
3     for each red r do d(r) <- 0 if r is uncovered, d(r) <- inf otherwise
4     breadth-first search from the uncovered red nodes: take x from the
```

```
       queue; if d(x) >= L then stop; for each edge xb, if b is uncovered
       then L <- d(x), else, if b is matched to y and d(y) = inf, then
       d(y) <- d(x) + 1 and y enters the queue (L = inf at the start)
 5     if L = inf then exit the loop
 6     for each red node s, in the order of RedNodeIt, if s is uncovered do
 7        depth-first search from s: at a red x, scan the edges xb from the
           one x stopped at; if b is uncovered and d(x) = L, augment M along
           the path found and stop; if b is matched to y with
           d(y) = d(x)+1 <= L, go down to y
 8        a red node all of whose remaining edges fail gets d <- inf, and
           its parent moves to its next edge
 9  Z <- nodes reached by alternating paths from the uncovered red nodes
10  cover <- (red nodes not in Z) + (blue nodes in Z)
```

Steps 3 to 5 are `bfs()`, steps 7 and 8 `dfs()`, and steps 9 and 10 `findCover()`, which is called once at the end of `start()`. The depth-first search is written without recursion, and each red node keeps in the map `_current` the edge it has reached, which is reset only at the start of a phase. The certificate does not depend on the phases, since step 10 builds it from the final matching alone, as in the proof of Theorem 17.2. The bound of the method rests on the following property of shortest augmenting paths, in which $|P|$ is the number of edges of a path P.

Lemma 17.3. *(i) Let P be a shortest augmenting path with respect to a matching M of any graph, and P' an augmenting path with respect to $M \oplus P$. Then $|P'| \geq |P| + 2|P \cap P'|$, where $P \cap P'$ is the set of their common edges. (ii) Let M_0 be a matching, and $P_1, \ldots, P_k$ paths such that each P_i is a shortest augmenting path with respect to M_{i-1}, where $M_i = M_{i-1} \oplus P_i$. Then $|P_1| \leq \cdots \leq |P_k|$, and two of the paths with the same number of edges are node-disjoint.*

Proof. (i) The matching $N = M \oplus P \oplus P'$ has $|M| + 2$ edges, and $M \oplus N = P \oplus P'$. As in the proof of Theorem 17.1, $M \oplus N$ contains at least two node-disjoint paths augmenting with respect to M, each with at least $|P|$ edges; hence $|P| + |P'| - 2|P \cap P'| = |P \oplus P'| \geq 2|P|$. (ii) The monotonicity follows from (i). Let $i < j$ be such that $|P_i| = |P_j|$ and P_i, P_j share a node, with $j - i$ least. Then P_j is node-disjoint from $P_{i+1}, \ldots, P_{j-1}$ (their lengths are between the two), and therefore it is augmenting with respect to M_i as it is with respect to M_{j-1}. By (i), P_i and P_j share no edge. However, a common node v is covered in M_i by an edge of P_i, and it is an inner node of P_j, which then contains that edge, a contradiction. □

Lemma 17.4. *In a phase with $L < \infty$, each path along which step 7 augments has $2L+1$ edges and is a shortest augmenting path with respect to the current matching, and these paths are node-disjoint. At the end of the phase, any augmenting path has more than $2L + 1$ edges.*

Proof. Step 4 is a breadth-first search on the red nodes, in which a red node y is reached from x through a non-matching edge xb and the matching edge by; hence, for each red node r with $d(r) \leq L$, $d(r)$ is the number of matched edges of a shortest alternating path from an uncovered red node to r, and $2L + 1$ is the length of a shortest augmenting path at the start of the phase. A path found by step 7 visits red nodes with $d = 0, 1, \ldots, L$ and ends with an edge to an uncovered blue node; hence, it has $2L + 1$ edges, and, by Lemma 17.3(ii) and by induction on the paths of the phase, it is a shortest augmenting path for the matching it augments, and it is node-disjoint from the previous ones. Suppose now that, at the end of the phase, Q is an augmenting path with $2L + 1$ edges, with red nodes $r_0, \ldots, r_L$ (in this order, r_0 uncovered) and blue nodes $b_0, \ldots, b_L$ (b_L uncovered). By Lemma 17.3(ii), applied to the paths of the phase followed by Q, no node of Q is on a path of the phase; hence, Q was augmenting at the start of the phase, with the same matching edges $b_j r_{j+1}$ throughout. The search of step 4 gives $d(r_0) = 0$ and $d(r_{j+1}) \leq d(r_j) + 1$, and $d(r_L) \geq L$, since the tree path to r_L followed by $r_L b_L$ is augmenting; hence $d(r_j) = j$ for each j. Furthermore, the pointer of r_L never passes the edge $r_L b_L$, which is accepted whenever it is scanned, and, by backward induction, the pointer of r_j never passes $r_j b_j$, which it leaves only when r_{j+1} fails. Thus no r_j fails. Then the search of step 7 from r_0, which is uncovered when its turn comes, either augments, covering r_0, or fails, and both contradict what we have shown. □

Theorem 17.6. *After `start()`, the matching is maximum, and `cover()` describes a vertex cover with `matchingSize()` nodes, i.e., a minimum one.*

Proof. The loop ends when step 4 meets no uncovered blue node, i.e., when no alternating path from an uncovered red node reaches an uncovered blue node. Every augmenting path of a bipartite graph has an uncovered red end, and therefore the matching is maximum by Theorem 17.1. The cover is the set C of Theorem 17.2. □

Theorem 17.7. *`start()` performs at most $2\sqrt{n} + 2$ phases and runs in $O((n + m)\sqrt{n})$ time; `greedyInit()` and `init()` run in $O(n + m)$ time.*

Proof. A phase costs $O(n+m)$. The breadth-first search scans each edge once. In the depth-first searches a red node is pushed on the stack only along the edge its parent points to, and that pointer moves forward when the node fails. It stays only along a path just augmented, and those paths are node-disjoint (Lemma 17.4), and therefore each pointer moves at most once per edge and the augmentations cost $O(n)$ in total. Also `findCover()` is a breadth-first search. By Lemma 17.4, after k phases each augmenting path has at least $2k+1$ edges, i.e., $2k+2$ nodes. Let M be the matching after $k = \lceil \sqrt{n} \rceil$ phases and M^* a maximum one: as in the proof of Theorem 17.1, $M \oplus M^*$ contains $|M^*| - |M|$ node-disjoint augmenting paths, each with more than $2\sqrt{n}$ nodes, and therefore $|M^*| - |M| < \sqrt{n}/2$. Each further phase but the last augments at least once, and the number of phases is at most $\lceil \sqrt{n} \rceil + \sqrt{n}/2 + 1 \le 2\sqrt{n} + 2$. □

The class states the bound $O(m\sqrt{n})$, which is the same when the graph has no isolated nodes. The bound is usually far from attained in practice. This class follows the pattern of the algorithm classes of chapter 3, although it has no named template parameters, since it uses no auxiliary structure that one could replace. Its types are `BpGraph` (i.e., `BPGR`) and `MatchingMap`, i.e., `BpGraph::NodeMap< Edge >`, and its constructor `HopcroftKarp( const BpGraph & graph )` builds the maps of the matching, of the layers and of the cover. Execution starts with `init()`, which empties the matching, or with `greedyInit()`, which also matches each red node, in the order of `RedNodeIt`, to the first uncovered blue node among its neighbours (in the order of `IncEdgeIt`). Then `start()` augments the matching until it is maximum and computes the cover, and `run()` is simply `greedyInit()` followed by `start()`. Among the queries, `matchingSize()`, `matching( const Edge & )` (whether the edge is in the matching), `matching( const Node & )` (the matching edge at the node, or `INVALID`), `matchingMap()` (a const reference to the map of the matching) and `mate( const Node & )` give the primal solution. Finally, `cover( const Node & )` returns whether the node is in the minimum vertex cover. It is meaningful only after `start()` or `run()`, and before them it is `false` on every node.

abLEMON. Both `HopcroftKarp` and `MinCostAssignment` are new in abLEMON (version 1.5.0, LEMON trac ticket #168). LEMON 1.3.1 has bipartite graphs but no matching algorithm written for them. Its general matching classes give no vertex cover and no assignment potentials.

17.3 Minimum cost assignment

We now give a cost c_e to each edge of a bipartite graph, which may have any number of blue nodes and need not be complete. In this setting `MinCostAssignment< BPGR, CM >` solves

$$\min \Big\{ \sum_{e \in E} c_e x_e \;:\; \sum_{e \in \delta(r)} x_e = 1 \;\; \forall r \in R, \quad \sum_{e \in \delta(b)} x_e \le 1 \;\; \forall b \in B, \quad x \in \{0,1\}^E \Big\}, \tag{17.6}$$

i.e., it finds a matching of least cost among those that cover all the red nodes, if one exists. Its dual solution is a *potential* π on the nodes with

$$\bar{c}_e \;=\; c_e + \pi(r) - \pi(b) \;\ge\; 0 \qquad \forall\, e = rb \in E, \tag{17.7}$$

with equality on the edges of the matching, and with the blue nodes left uncovered all at the largest potential $K = \max_{b \in B} \pi(b)$ of the blue nodes; $\bar{c}_e$ is the *reduced cost* of e. Setting $\alpha_r = K - \pi(r)$ and $\beta_b = \pi(b) - K \le 0$ gives a feasible solution of the linear programming dual of the relaxation of (17.6), i.e., $\max\{\sum_r \alpha_r + \sum_b \beta_b : \alpha_r + \beta_b \le c_{rb}, \beta \le 0\}$, whose value equals the cost of the matching (each matched blue node b_r contributes $\pi(b_r) - K$, each uncovered one 0). Thus, the optimal cost is $\sum_{r \in R} \big(\pi(b_r) - \pi(r)\big)$, where b_r is the mate of r.

The method is that of the successive shortest augmenting paths, with the potentials of Edmonds and Karp 1972 and Tomizawa 1971. Here the red nodes are matched one at a time, each along a shortest path of the residual graph, in which an edge not in the matching is traversed from its red end to its blue end at cost c_e and an edge of the matching from its blue end to its red end at cost $-c_e$. The reduced cost of the second traversal is $-\bar{c}_e$. Each path is found by Dijkstra's algorithm (Dijkstra 1959) on the reduced costs, which (17.7) keeps nonnegative.

Algorithm 17.2 (Successive shortest paths for the assignment).

```
1  M <- empty; pi(v) <- 0 for each node v
2  for each red r do if r has no edge then return false, else
       pi(r) <- - (least cost of an edge of r)
3  for each red node s, in the order of RedNodeIt do
4     Dijkstra from s, d(s) = 0, on the reduced costs: red x -> blue b
       along xb not in M at d(x) + c(xb) + pi(x) - pi(b); matched blue b
       -> its mate y at d(b) - (c(yb) + pi(y) - pi(b)); stop when an
```

```
       uncovered blue node t is taken from the heap
5     if no uncovered blue node is taken then return false
6     for each node v do pi(v) += d(v) if v was taken from the heap with
       d(v) < d(t), and pi(v) += d(t) otherwise
7     augment M along the path from s to t
8  return true
```

Step 4 and the rest of each iteration are the private function `augment()`. The heap of step 4 is a `std::priority_queue` of pairs (distance, node id) with lazy deletion: a node reached again at a shorter distance is pushed again, and the stale entries are skipped when popped. The correctness rests on the following invariant.

Lemma 17.5. *Before each search of step 4, (17.7) holds, the edges of M have zero reduced cost, and all the uncovered blue nodes have the same potential, the largest of the blue nodes.*

Proof. After step 2, $\bar{c}_{rb} = c_{rb} - \min_{e\in\delta(r)} c_e \geq 0$, M is empty and all the blue potentials are 0. Assume the claim before a search, let $d(v)$ be the distance from s in the residual graph with the reduced costs (which are nonnegative, and therefore the distances of the nodes taken from the heap are exact), with $d(v) = \infty$ for the nodes not reached, and let $\pi' = \pi + \min\{d, d(t)\}$, which is the update of step 6 (a node not taken has $d \geq d(t)$). For a residual arc from x to y with reduced cost $\bar{c} \geq 0$, the new reduced cost is $\bar{c} + \min\{d(x), d(t)\} - \min\{d(y), d(t)\}$, which is nonnegative since $d(y) \leq d(x) + \bar{c}$. On the path from s to t, whose nodes have $d \leq d(t)$ and satisfy $d(y) = d(x) + \bar{c}$, the new reduced costs are zero, and the augmentation of step 7 reverses only arcs of zero reduced cost; hence, (17.7) holds after the iteration, with equality on M. The search stops at the first uncovered blue node taken from the heap; hence, each other uncovered blue node gets the increase $d(t)$, which no other blue node exceeds, and t becomes covered. □

Theorem 17.8. ***`run()`*** *returns* ***`true`*** *if and only if a matching that covers all the red nodes exists. In that case the matching is an optimal solution of (17.6), and* ***`potential()`*** *gives the dual solution (17.7).*

Proof. A red node without edges cannot be covered. If the search from s meets no uncovered blue node, let X be the set of the nodes it reaches: every blue node of X is matched to a red node of X (the only residual arc out of a matched blue node goes to its mate), each blue neighbour of a red node of X is in X, and s is the only uncovered red node of X; hence, the $|X \cap B| + 1$ red nodes of X have only $|X \cap B|$ blue neighbours, and no matching covers them. Otherwise each iteration covers one more red node and keeps the others covered, and at the end Lemma 17.5 and the computation after (17.7) give a feasible dual solution of the relaxation with the value of the matching, which is therefore optimal. □

Theorem 17.9. ***`run()`*** *takes $O(n_R\,(n + m\log m))$ time, i.e., $O(n_R\,(n + m\log n))$ on a graph without parallel edges.*

Proof. Step 2 takes $O(n + m)$ time. In a search each red node taken from the heap relaxes its edges once and each blue node one edge; hence, at most $m + n + 1$ pairs enter the heap, each at a cost of $O(\log m)$. Step 6 takes $O(n)$ time, and step 7 and the reset of the marks take time proportional to the nodes reached. There are at most n_R searches. □

The class states the bound $O(n_R\, m \log n)$, which omits the term $n_R\, n$ of the updates of step 6 (the two differ only when m is smaller than n, i.e., when many blue nodes are isolated). Note that the comparisons are exact (the class has no tolerance), which is usually adequate for integer costs but may be somewhat fragile with floating-point ones, and that the cost type must be a signed arithmetic type. The interface is reduced to what the method needs. Its types are `BpGraph`, `CostMap` (i.e., `CM`, by default `BpGraph::EdgeMap< int >`), `Value` (the type of the costs) and `MatchingMap` (as in `HopcroftKarp`), and its constructor is `MinCostAssignment( const BpGraph & graph , const CostMap & cost )`. Its only execution function is `run()`, which returns `true` if a matching covering all the red nodes exists and `false` otherwise (in which case the matching is left as the method had grown it, and the potentials have no meaning). As in `HopcroftKarp`, `totalCost()`, `matching( const Edge & )`, `matching( const Node & )` and `matchingMap()` give the primal solution, while `potential( const Node & )` gives the dual solution (17.7).

The program `manual/examples/17-bipartite.cc` (the test `manual_17_bipartite`) solves a "small" instance with both classes of the header: it checks the cover of Theorem 17.2, the conditions (17.7) and the value of the dual solution, and it compares the optimal cost with an enumeration of all the assignments.

17.4 Maximum cardinality matching in general graphs

We now turn to general graphs, in which an alternating search tree may close an odd cycle, and the search then has to treat the cycle as a single node. This is the *blossom* of Edmonds 1965a, and `MaxMatching< GR >` implements his alternating forest algorithm on an undirected graph of type `GR`. Each node has a status, `EVEN`, `ODD`, `MATCHED` or `UNMATCHED`. At the start the covered nodes are `MATCHED` and the others `UNMATCHED`. A search tree is grown from an `UNMATCHED` root, which becomes `EVEN`. A `MATCHED` node reached from an `EVEN` node becomes `ODD`, and its mate `EVEN`. An edge between two `EVEN` nodes of the same tree in distinct blossoms closes a blossom, i.e., an odd cycle of blossoms of the tree, which is merged into one blossom whose nodes are all `EVEN`. An edge from an `EVEN` node to an `UNMATCHED` node gives an augmenting path. Internally, the blossoms are kept by a `UnionFindEnum` and the trees by an `ExtendFindEnum` (see chapter 8), and each node of a blossom keeps an *ear*, i.e., the arc along which an even alternating path inside the blossom leaves it towards the base of the blossom (the node through which the blossom is attached to its parent in the tree).

Algorithm 17.3 (Edmonds' alternating forest, sparse variant).

```
1  M <- empty, greedy or given matching; covered nodes MATCHED, the
       others UNMATCHED
2  for each node r, in the order of NodeIt, while more than 1 nodes are
       UNMATCHED (1 = 0, or 1 = 1 without the decomposition) do
3      if r is UNMATCHED then r becomes the EVEN root of a new tree; Q <- (r)
4      while Q is not empty do take u from Q; for each arc uv do
5          if v is EVEN and in another blossom then shrink the blossom closed
            by uv; its ODD nodes become EVEN and enter Q
6          if v is MATCHED then extend: v becomes ODD, its mate EVEN (into Q)
7          if v is UNMATCHED then augment along uv and the tree path of u; the
            nodes of the tree become MATCHED; go to the next root
8          (if v is ODD, or EVEN in the blossom of u, nothing happens)
9  (a tree that did not augment keeps its statuses)
```

Step 5 is `shrinkOnEdge()`, step 6 `extendOnArc()` and step 7 `augmentOnArc()`, and the loop of steps 4 to 8 is `processSparse()`. A tree that ends without augmenting is called *Hungarian*, and the analysis rests on what it leaves behind.

Lemma 17.6. *When the search from a root ends without augmenting, every neighbour of an `EVEN` node of its tree is an `ODD` node (of this or of an earlier tree) or an `EVEN` node of the same blossom. The statuses of the nodes of a Hungarian tree do not change afterwards, and each of its blossoms has an odd number of nodes and induces a connected subgraph.*

Proof. Each node that becomes `EVEN` enters the queue (as the root, as the mate of a new `ODD` node, or by a shrink), and all its arcs are scanned. A scanned arc to an `EVEN` node of another blossom is shrunk, to a `MATCHED` node extends the tree, and to an `UNMATCHED` node augments, and one to an `ODD` node is left alone. Hence, at the end of a search without augmentation, an `EVEN` node has no `MATCHED` or `UNMATCHED` neighbour and no `EVEN` neighbour in another blossom of its tree. It has none in an earlier Hungarian tree either, since such a neighbour would have a neighbour outside its own tree, against the claim for that tree. A later search reaches only `MATCHED`, `UNMATCHED` and `EVEN` nodes of its own tree from its `EVEN` nodes, and the nodes of a Hungarian tree are neither of these; hence, it never touches them. Finally, a blossom is either a single node or the union of an odd number of blossoms joined in a cycle by edges, each with an odd number of nodes, which gives the parity and the connection. □

Theorem 17.10. *At the end of `startSparse()` or `startDense()` the matching is maximum. With the decomposition, the set A of the `ODD` nodes attains the minimum in (17.3), the subgraphs induced by the blossoms of the `EVEN` nodes are the odd components of $G - A$ that contain them, and the `MATCHED` nodes are matched among themselves.*

Proof. At the end, each uncovered node is the root of a Hungarian tree, except at most one `UNMATCHED` node u that was never searched from (without the decomposition). In a tree, each node but the root is covered, an `ODD` node is matched to an `EVEN` node of the tree, and the number of the `EVEN` blossoms exceeds by one the number of the `ODD` nodes (an extension adds one of each, and a shrink of $2k+1$ blossoms removes k `ODD` nodes and $2k$ `EVEN` blossoms). The mate of a `MATCHED` node is `MATCHED`. By Lemma 17.6, an `EVEN` blossom has no neighbour outside A, and therefore it is an odd component of $G - A$. Also u, which has no `EVEN` neighbour by the same lemma, lies in a component of $G - A$ made of u and of `MATCHED` nodes matched among themselves, i.e., in an odd one. If h is the number of the Hungarian trees, the number of the uncovered nodes is h (or $h+1$, counting

u) and $o(G-A) \geq h + |A|$ (or $h + 1 + |A|$); hence $n - 2|M| \leq o(G-A) - |A|$, and by Lemma 17.1 both the matching and A are optimal. □

Hence, with the decomposition the statuses give the barrier that proves the matching maximum by (17.3). By Theorem 17.4 and its proof they are, moreover, the canonical decomposition, the `EVEN` nodes forming D, the `ODD` nodes A and the `MATCHED` nodes C. Without the decomposition, the loop of step 2 stops when at most one node is left `UNMATCHED`, i.e., uncovered and never searched from. The matching is still maximum, but u has no status of the decomposition, and the statuses do not form it. According to the header, this may save much time, in particular on a dense graph with an odd number of nodes, where the last search, from a node that remains uncovered, usually explores most of the graph and finds no augmenting path.

The variant `startDense()` runs the same search, in `processDense()`, with the heuristic of postponing the shrinks. It first extends the tree from all the nodes in the queue, augmenting as soon as it can. Only when no extension or augmentation is left does it scan the arcs of the queued nodes again for the shrinks, extending (and possibly augmenting) again from the nodes that each shrink makes `EVEN`. The header reports this to be faster on "dense" graphs (presumably a matter of constants, since the worst case is the same). Lemma 17.6 and Theorem 17.10 hold for it with the same proofs, since each `EVEN` node still has all its arcs scanned for the three cases. The header states no bound on the running time. The following one, for this implementation, is not the best known for the problem, which is $O(m\sqrt{n})$ (Galil 1986 surveys the implementations of the blossom algorithm).

Theorem 17.11. *`startSparse()` and `startDense()` run in $O(n\,m\,\alpha(n) + n^3 \log n)$ time, where α is the inverse of Ackermann's function.*

Proof (sketch). There is at most one search per node. In a search, each node becomes `EVEN` at most once and is queued once, and its arcs are scanned once (twice in the dense variant), each scan costing a `find()` of the union-find structure, i.e., $O(\alpha(n))$ amortized (it joins by size and compresses the paths). A search makes at most $n/2$ extensions and at most $n/2$ shrinks, since a shrink merges at least three blossoms of the tree into one. A shrink first finds the nearest common ancestor of the two blossoms by walking up from both alternately and storing the visited bases in two `std::set`s, at a cost of $O(n \log n)$. Then it redirects the ears along the two paths, walking inside each blossom of the paths from the node where the path enters it to its base, which costs $O(n)$ in total, since these walks visit distinct nodes. An augmentation, and the reset of the statuses of the tree at its end, cost $O(n)$. Hence, a search costs $O(m\,\alpha(n) + n^2 \log n)$. □

Besides the constructor `MaxMatching( const Graph & graph )`, the class has three functions that set the initial matching. These are `init()` (the empty one), `greedyInit()` (each node, in the order of `NodeIt`, matched to its first uncovered neighbour in the order of `OutArcIt`, loops excluded) and `matchingInit( const MatchingMap & matching )`, a template on the type of the map, which takes a `bool` edge map and returns `false` if two edges with value `true` share a node (in which case the matching is left half built, and the function must be called again before a start). Two functions, `startSparse( bool decomposition = true )` and `startDense( bool decomposition = true )`, run the two variants of Algorithm 17.3, after one of the three initializations, and `run( bool decomposition = true )` calls `greedyInit()` and `startSparse()` if $m < 2n$, and `init()` and `startDense()` otherwise. The types of the class are `Graph` (i.e., `GR`), `MatchingMap` (i.e., `Graph::NodeMap< Arc >`) and `StatusMap` (i.e., `Graph::NodeMap< Status >`). Of the queries, `matchingSize()` counts the covered nodes (in $O(n)$ time), and `matching( const Edge & )`, `matching( const Node & )` (the matching arc at the node, directed away from it, or `INVALID`), `matchingMap()` and `mate( const Node & )` complete the primal solution. Lastly, `status( const Node & )` and `statusMap()` give the values of the enum `Status`, i.e., `EVEN` $= 1$ (also called `D`), `MATCHED` $= 0$ (also `C`), `ODD` $= -1$ (also `A`) and `UNMATCHED` $= -2$, and `barrier( const Node & )` is `true` on the `ODD` nodes.

The program `manual/examples/17-edmonds.cc` (the test `manual_17_edmonds`) computes the matching of Figure 17.1, counts the components of the subgraph induced by the `EVEN` nodes with a `FilterNodes` adaptor (chapter 5) and `countConnectedComponents()` (chapter 18), checks the equality of (17.3) for the barrier found, and runs the algorithm once more without the decomposition.

abLEMON. The parameter `decomposition` of `startSparse()`, `startDense()` and `run()` is not in LEMON 1.3.1: it was added to the repository of LEMON in 2018, after its last release, and abLEMON is the first release to distribute it. With the default `true`, a program written for LEMON 1.3.1 behaves as before.

17.5 Maximum weight matchings in general graphs

`MaxWeightedMatching< GR, WM >` solves (17.1), and `MaxWeightedPerfectMatching< GR, WM >` its perfect version, by the primal-dual blossom algorithm of Edmonds 1965a. Throughout, the algorithm keeps a matching,

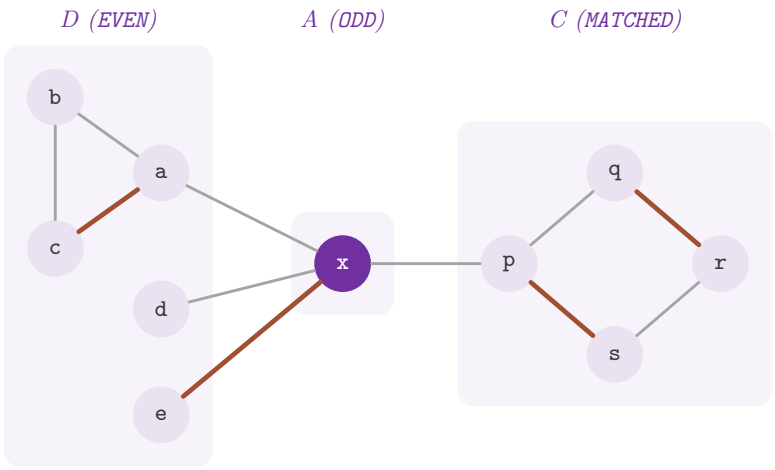


Figure 17.1. The graph of the example below, with the maximum matching found by MaxMatching (the edges in the colour of the added objects) and its Gallai–Edmonds decomposition: the EVEN nodes (D) induce the factor-critical components {a, b, c}, {d} and {e}, the barrier A is {x}, and the MATCHED nodes (C) induce a square with a perfect matching. By (17.2), at least $3 - 1 = 2$ nodes are uncovered by any matching.

a family of nested blossoms and a dual solution of (17.5) with $z_S > 0$ only on blossoms, and it changes the duals until all the nodes are covered or, in the version with inequalities, until the uncovered nodes have $y_u = 0$. Its weight map `WM` defaults to `GR::EdgeMap< int >`, and loops are ignored. Both classes use binary heaps (`BinHeap`) for the kinds of dual change, four in `MaxWeightedMatching` and three in the perfect version, and a `HeapUnionFind` for the nested blossoms, which gives the bound $O(nm \log n)$ stated by the header, i.e., that of Galil, Micali and Gabow 1986. In the following, a node or a blossom is *in the forest* when it belongs to an alternating tree rooted at an uncovered blossom, `EVEN` or `ODD` as in section 17.4. The others are `MATCHED`, and they include the nodes that left the forest uncovered with $y_u = 0$.

Algorithm 17.4 (Primal-dual blossom algorithm).

```
1  init: y(u) <- max over the edges e at u of w(e)/2, and at least 0 in
      the version with inequalities; z <- 0; M empty; or fractionalInit:
      y and M from the optimal fractional matching of section 17.6, each
      of its odd cycles a blossom with z = 0 (the default of run())
2  each uncovered blossom is the EVEN root of a tree
3  while some blossom of the forest is uncovered do
4     d1 <- least y(u) over the EVEN nodes (not in the perfect version)
      d2 <- least slack of an edge from an EVEN node to a MATCHED blossom
      d3 <- half the least slack of an edge between two EVEN blossoms
      d4 <- half the least z(B) of a nontrivial ODD blossom B
5     d <- least of them; if d is infinite then return false (perfect
       version only)
6     move the duals by d: y down on EVEN nodes and up on ODD ones,
       z(B) up by 2d on EVEN blossoms and down by 2d on ODD ones
7     d1 -> exchange M along the even path from the root to the node
            whose y is now 0, which is left uncovered, and dissolve the
            tree (all its blossoms become MATCHED)
      d2 -> extend the tree, or augment if the blossom reached is
            uncovered (it left the forest at a d1)
      d3 -> same tree, shrink a new blossom; distinct trees, augment
            along the edge and the two tree paths, and dissolve both
      d4 -> expand the blossom, keeping in the tree its even path
8  extract the matching and the duals, the nested blossoms flattened
```

In the code, the quantities of step 4 are kept in the heaps `_delta1` to `_delta4`, and the dual changes of step 6 are applied lazily through a running sum. Thus, a step costs a few heap operations instead of a pass over the graph. When two quantities tie, the code prefers d_3, then d_1, d_2 and d_4 in this order (in the perfect version, d_3, d_2, d_4). If the weights are of an integer type, each dual value is multiplied by the constant `dualScale`, which is 4 (and 1

for a floating-point type), and thus the halvings of steps 1 and 4 stay exact for integer weights, as in the analysis of Galil 1986. By default the algorithm starts from the optimal fractional matching (cf. section 17.6), solved by `MaxWeightedFractionalMatching` or `MaxWeightedPerfectFractionalMatching` with loops excluded. The header calls this initialization the "jumpstart" heuristic, and it may change which of several optimal matchings is found, but not the optimal value. In the perfect version, when no perfect fractional matching exists (e.g., when two pendant edges end at the same node), `fractionalInit()` records the failure and `start()` returns `false` at once.

Lemma 17.7. *Throughout Algorithm 17.4, (i) (y, z) is feasible for (17.5) (with $y \geq 0$ in the version with inequalities), (ii) the edges of M, of the trees and of the cycles of the blossoms are tight, (iii) $z_B > 0$ only on blossoms, each of which holds $\frac{|B|-1}{2}$ edges of M, and (iv) a node not covered by M and outside the forest has $y_u = 0$.*

Proof (sketch). After `init()`, $y_u + y_v \geq w_{uv}$ by the choice of y, and (ii) to (iv) are empty; after `fractionalInit()` they follow from the optimality conditions of the fractional problem (Theorem 17.16), each odd half-integral cycle being a tight cycle with $z = 0$. A dual change by d keeps the tree edges tight (their ends are one `EVEN` and one `ODD`), leaves unchanged the slack of the edges inside a blossom and of the edges between two `MATCHED` blossoms, changes by $-d$ the slack of an edge from an `EVEN` node to a `MATCHED` one and by $-2d$ that of an edge between two `EVEN` blossoms, and keeps $y \geq 0$ and $z \geq 0$ by the definitions of d_1 and d_4. The four operations of step 7 use only tight edges, and a blossom contracted by a shrink has $\frac{|B|-1}{2}$ matched edges by construction. The full argument is in Schrijver 2003, ch. 26 and Galil 1986. □

Theorem 17.12. *At the end of `start()` of `MaxWeightedMatching`, the matching is a maximum weight matching, and the dual values returned are `dualScale` times an optimal solution of (17.5); `MaxWeightedPerfectMatching::start()` returns `false` if and only if the graph has no perfect matching, and otherwise gives a maximum weight perfect matching with an optimal dual solution.*

Proof. The loop ends when no blossom of the forest is uncovered. Then the conditions of Lemma 17.2 hold by Lemma 17.7 (an uncovered node is outside the forest, with $y_u = 0$), and the matching and the duals are optimal. In the perfect version, if all the d are infinite, the dual change of step 6 can be made with any $d > 0$ without losing feasibility, and it decreases the objective of (17.5) by d for each tree (the `EVEN` blossoms of a tree are one more than its `ODD` blossoms, and a blossom B counts $|B|$ nodes against the $\frac{|B|-1}{2}$ of its z_B); hence, (17.5) is unbounded, and by linear programming duality and Theorem 17.5 there is no perfect matching. Conversely, if the loop ends, the matching found is perfect. The failure of `fractionalInit()` is the same certificate for the relaxation without (17.4) (Theorem 17.16). □

Theorem 17.13. *`start()` of both classes runs in $O(nm \log n)$ time.*

Proof (sketch). Call a *stage* the steps between two decreases of the number of the uncovered blossoms of the forest, which happen at each d_1 and at each augmentation. There are at most n stages. In a stage, each d_2 adds a blossom to the forest, each d_3 that shrinks reduces the number of its blossoms, and each d_4 expands a blossom formed before the stage, and hence a stage has $O(n)$ steps; with the heaps and the `HeapUnionFind` of the blossoms, the total work of a stage is $O(m \log n)$, since the heaps of the slacks are updated only when a blossom enters the forest, changes status or is split, and the `HeapUnionFind` makes the split of a blossom cost $O(\log n)$ per node. The complete count is in Galil, Micali and Gabow 1986. □

The two classes have the same members, except for the return type of the execution functions and for `matchingSize()`, which only `MaxWeightedMatching` has. Their types are `Graph`, `WeightMap`, `Value` (the type of the weights) and `MatchingMap` (i.e., `Graph::NodeMap< Arc >`), and the constant `dualScale` is public. Their constructor takes `( const Graph & graph , const WeightMap & weight )`. Either `init()` or `fractionalInit()` sets the initial solution. Then `start()` runs the algorithm, and `run()` is `fractionalInit()` followed by `start()`. In `MaxWeightedPerfectMatching`, both `start()` and `run()` return `false` when the graph has no perfect matching (and the queries then have no meaning). The functions `matchingWeight()`, `matching( const Edge & )`, `matching( const Node & )`, `matchingMap()` and `mate( const Node & )` give the primal solution, as in `MaxMatching`, and `MaxWeightedMatching` also has `matchingSize()`. For the dual one, `dualValue()` returns the value of the scaled dual solution, i.e., $\sum_u y_u + \sum_B \lfloor |B|/2 \rfloor z_B$, and `nodeValue( const Node & )` the scaled y_u. Furthermore, `blossomNum()` returns the number of the nontrivial blossoms of the final family (whose values may be 0), `blossomSize( int k )` and `blossomValue( int k )` (the scaled z_B) describe the k-th of them, and the nested class `BlossomIt`, built as `BlossomIt( algorithm , k )`, lists its nodes. A blossom comes before the blossoms nested in it, whose nodes form contiguous parts of its own list.

The program `manual/examples/17-weighted.cc` (the test `manual_17_weighted`) computes a maximum weight matching of a triangle with two pendant edges, checks the conditions of Lemma 17.2 on each edge (the slack

computed from `nodeValue()` and from the blossoms that contain both ends, which `BlossomIt` lists) and the equality of the primal and dual values, and observes that the graph has no perfect matching. It ends with the fractional classes of the next section.

abLEMON. In LEMON 1.3.1 the extraction of the nested blossoms at the end of `start()` (step 8 of Algorithm 17.4) was recursive, with one level per level of nesting, and it could overflow the stack on large graphs (LEMON trac ticket #638). In abLEMON it uses an explicit stack, a change made in the repository of LEMON in 2021 and never released by it. Furthermore, the node records of the two classes initialize all their fields, some of which LEMON 1.3.1 copied while still indeterminate. The results and the interface do not change.

17.6 Fractional matchings

We finally describe the three classes of `fractional_matching.h`, which solve the relaxation of section 17.1 without the odd set inequalities (17.4). They represent a fractional matching by a node map of arcs, the *pointers*, in which each covered node points along an arc to a neighbour. An edge has value $\frac{1}{2}$ for each of its ends that points along it; thus, the two ends of an edge with value 1 point to each other, and the nodes of a half-valued odd cycle point to their successors on the cycle. Since the values are multiples of $\frac{1}{2}$, the primal solution is reported multiplied by the constant `primalScale`, which is 2. Thus, `matching( const Edge & )` returns an `int` in $\{0, 1, 2\}$ (the number of ends of the edge that point along it), and `matchingSize()` returns the number of the nodes that point somewhere, i.e., twice the value of the fractional matching. All three constructors take a flag `allow_loops`, `true` by default, which decides whether a loop may be taken, as a cycle of length one whose node points to itself. The loop then counts twice in the degree of its node and has value $\frac{1}{2}$, although `matching( )` returns 2 for it, since both its ends are that node.

`MaxFractionalMatching< GR, TR >` maximizes the cardinality. The relaxation is equivalent to a bipartite matching problem on the *double cover* of G, in which each node u has a copy u' on the left and a copy u'' on the right, and each edge uv joins u' to v'' and v' to u'': a pointer from u to v is the edge $u'v''$, and the value of a fractional matching is half the number of the edges of the corresponding matching of the double cover. This problem is solved by a push-relabel method in the spirit of Goldberg and Tarjan 1988, in which the right copies without an incoming pointer are *active* and pull a pointer from a node on a lower level. The levels $\ell(v) \in \{0, \dots, n\}$ are kept by an `Elevator` (chapter 8) whose top level is n. We write $\mathrm{in}(v)$ for the number of the nodes that point to v, and $t(u)$ for the node that u points to.

Algorithm 17.5 (Push-relabel for the maximum fractional matching).

```
1  each node u points to its first neighbour in the order of OutArcIt
      (loops skipped unless allowed), or to nothing; all nodes on level
      0; the nodes v with in(v) = 0 are active
2  while some node below the top is active do
      x <- an active node on the highest level l
3     if a neighbour u of x has l(t(u)) < l then (the first such u)
         in(t(u)) -= 1, and t(u) becomes active if in(t(u)) = 0;
         u points to x; in(x) <- 1; x becomes inactive
4     else lift x to 1 + the least l(t(u)) over its neighbours u, or to
         the top if this is not below the top; if level l is left
         empty, lift all the nodes above it to the top
5  a node u with in(t(u)) > 1 drops its pointer, until every in(v) <= 1
6  if postprocess then turn the alternating paths and the even cycles of
      pointers into matching edges, leaving odd cycles of halves
```

Steps 1 to 5 are `init()` and `start()`, and step 6 is the private function `postprocessing()`. The perfect variant `startPerfect()` returns `false` as soon as a node would be lifted to the top or a level is left empty. It skips step 5. The dual object, which we also call a *barrier*, is a set S of nodes with $|S| - |N(S)|$ as large as possible.

Lemma 17.8. *For every fractional matching x and every $S \subseteq V$, $2\sum_e x_e \le n - |S| + |N(S)|$ (with a loop at u making $u \in N(\{u\})$).*

Proof. In the double cover, the right copies of the nodes of S are adjacent only to the left copies of the nodes of $N(S)$; hence, the value that x, doubled, gives to the right copies of S is at most $|N(S)|$, while the others take at most 1 each. □

Lemma 17.9. *During `start()`: (i) the occupied levels below the top are $0, 1, \dots, k$ for some k; (ii) for each node x below the top and each neighbour u of x, $\ell(x) \le \ell(t(u)) + 1$; (iii) a node with $\mathrm{in}(v) \ge 2$ is on level 0; (iv)*

for each node x on the top and each neighbour u of x, $t(u)$ is on the top.

Proof. All four hold after step 1, where every node with a neighbour points to one. A push of step 3 redirects u from $t(u)$ to x with $\ell(t(u)) < \ell(x)$, which keeps (ii); u has no neighbour on the top, since by (iv) $t(u)$ would be on the top; $\mathrm{in}(x)$ becomes 1 and no in-degree grows beyond it, which keeps (iii) and (iv). A lift of step 4 raises x to $1 + \min_u \ell(t(u)) \geq \ell(x) + 1$, which keeps (ii) for x, and raising a level can only help (ii) for the other nodes. Also x, being active, has $\mathrm{in}(x) = 0$, and the nodes that a gap lifts are above level $l \geq 0$, which keeps (iii). The new level of x is at most one above the occupied levels, by (ii) and (i). If level l is left empty, all the levels above it are lifted, and (i) holds again. For (iv), let x be lifted to the top by the first case: all the $t(u)$ of its neighbours are on level $n-1$ or on the top, and by (i) a node on level $n-1$ means that each level below the top holds exactly one node, and level l of x is left empty and the gap lifts that node too. For a node w that a gap at level l lifts, and a neighbour u of w, (ii) gives $\ell(t(u)) \geq \ell(w) - 1 \geq l$, and $\ell(t(u)) \neq l$, so that $t(u)$ is lifted as well. □

Theorem 17.14. *`start()` terminates with a maximum fractional matching, made (with `postprocess`) of matching edges and of half-valued odd cycles, and the set S of the nodes for which `barrier()` is `true` (the nodes on the top) satisfies*

$$n - \texttt{matchingSize()} \;=\; |S| - |N(S)| \;=\; \max_{X \subseteq V} \big(|X| - |N(X)|\big). \tag{17.8}$$

`startPerfect()` returns `false` only if the graph has no perfect fractional matching, and otherwise it gives one.

Proof. Termination follows from Theorem 17.15. At the end, no node below the top is active, i.e., each node v with $\mathrm{in}(v) = 0$ is on the top. By Lemma 17.9(iv), each node of $N(S)$ points into S, and each pointer into S comes from $N(S)$. By (iii) the nodes of S have in-degree at most 1, so that step 5 removes no pointer into S. Hence, S holds $|N(S)|$ nodes with an incoming pointer and all those without one, and after step 5 the number of the pointers, i.e., `matchingSize()`, is $n - (|S| - |N(S)|)$. After step 5 each node has at most one pointer in and one out, and therefore the pointers give a fractional matching. By Lemma 17.8 it is maximum, and so is S. The pointers form node-disjoint paths and cycles. Step 6 replaces a path with k nodes, which has $k-1$ pointers, by $\lfloor k/2 \rfloor$ matching edges, i.e., by $2\lfloor k/2 \rfloor \geq k-1$ pointers, with equality for k odd, and an even cycle by matching edges with as many pointers. By the optimality no path has an even number of nodes, and therefore the value is kept. For `startPerfect()`, the steps are those of `start()` until it returns. It returns `false` when an active node reaches the top, directly or by a gap, and such a node keeps $\mathrm{in} = 0$ to the end of `start()`; hence, by what we have shown, the maximum fractional matching leaves a right copy uncovered. If it does not return `false`, each node has an incoming pointer at the end, and the pointers form a perfect fractional matching. □

After a failed `startPerfect()`, `barrier()` returns the nodes on or above the level found empty, or those on the top level and on the level below it when the failure was a lift to the top. The test `test/fractional_matching_test.cc` checks that this set has $|S| > |N(S)|$ on its instances. With `postprocess` set to `false`, the result may also contain even cycles and paths of half-valued edges, with the same value.

Theorem 17.15. *Steps 2 to 4 perform at most n^2 pushes and at most n^2 lifts, and `run()` takes $O(n^2\Delta + n + m)$ time, where Δ is the largest degree of a node.*

Proof. Each node is lifted at most n times, since its level grows at each lift and is at most n. Let Φ be the sum of the levels of the active nodes below the top: a push decreases Φ by at least 1 (it removes x and adds at most $t(u)$, on a lower level), a lift to the top or a gap does not increase it, and the other lifts increase it by at most n^2 in total; since $\Phi \geq 0$ and $\Phi = 0$ at the start, there are at most n^2 pushes. A push or a lift of x scans at most the $\deg(x) \leq \Delta$ arcs into x, and the operations of the elevator take constant amortized time (a node reaches the top once). Steps 1, 5 and 6 take $O(n+m)$ time. □

The header states no bound; push-relabel methods are often much faster in practice than their bound. We have modeled the interface on the push-relabel classes of chapter 12. The traits class `MaxFractionalMatchingDefaultTraits< GR >` defines `Graph`, `MatchingMap` (`GR::NodeMap< GR::Arc >`) and `Elevator` (`LinkedElevator< GR, GR::Node >`) with their factory functions `createMatchingMap()` and `createElevator()`, and the named template parameters `SetMatchingMap< T >` (a map that the caller passes with `matchingMap()`), `SetElevator< T >` (an elevator that the caller passes with `elevator()`) and `SetStandardElevator< T >` (an elevator that the algorithm allocates) replace them, each through its nested type `Create`. The class has the types `Traits`, `Graph`, `MatchingMap` and `Elevator`, and its constructor is `( const Graph & graph , bool allow_loops = true )`. Two functions, `matchingMap( MatchingMap & )` and `elevator( Elevator & )`, set external structures before `init()`, and `elevator()` returns the one in use. Execution is controlled by `init()`, `start( bool postprocess = true )` and `startPerfect( bool postprocess = true )` (which returns whether a perfect fractional matching exists), or by their shortcuts `run( bool postprocess = true )` and `runPerfect( bool postprocess = true )`. Its queries are `matchingSize()`, `matchingMap()`, `matching( const Edge & )`, `matching( con`

`st Node & )` (the pointer of the node, i.e., on a half-valued cycle its arc to the successor) and `barrier( const Node & )`.

`MaxWeightedFractionalMatching< GR, WM >` and `MaxWeightedPerfectFractionalMatching< GR, WM >` solve the weighted relaxation, without and with the perfection constraint, by the primal-dual method of Algorithm 17.4 applied to the problem without (17.4). An edge that closes an odd cycle in a tree is not shrunk into a blossom: the cycle is kept as a half-valued cycle of the solution, and the tree is dissolved. There are no z variables, and hence no fourth kind of dual change, and the `EVEN` and `ODD` objects are single nodes.

Algorithm 17.6 (Primal-dual method for the weighted fractional matching).

```
1  y(u) <- max over the edges e at u (loops only if allowed) of w(e)/2,
       and at least 0 without the perfection constraint; no pointers;
       each node the EVEN root of a tree
2  while some tree is left do
3      d1 <- least y(u) over the EVEN nodes (not in the perfect version)
       d2 <- least slack of an edge from an EVEN node to a node out of
             the forest
       d3 <- half the least slack of an edge between two EVEN nodes
4      d <- least of them; if d is infinite then return false (perfect
        version only)
5      move the duals by d: y down on EVEN nodes and up on ODD ones
6      d1 -> exchange the pointers along the even path from the root to
             the node whose y is now 0, which is left uncovered, and
             dissolve the tree
       d2 -> if the node reached is uncovered, augment; if it is on a
             half-valued odd cycle, augment into the cycle, which breaks
             into matching edges; else extend the tree
       d3 -> distinct trees, augment; same tree, keep the odd cycle that
             the edge closes, with the even path from the root to it
             turned into matching edges, and dissolve the tree
```

Theorem 17.16. *At the end of* `start()` *the pointers form an optimal fractional matching, made of matching edges and of half-valued odd cycles, and the node values are* `dualScale` *times an optimal solution of the dual of the relaxation. In the perfect class,* `start()` *returns* `false` *if and only if no perfect fractional matching exists. Both classes run in* $O(nm \log n)$ *time.*

Proof (sketch). The invariants of Lemma 17.7 hold without z, with "blossom" replaced by "half-valued odd cycle", whose edges are tight. At the end each node with $y_u > 0$ has value 1 (a matching edge or two halves of a cycle), each pointer is along a tight edge, and the complementary slackness conditions of the relaxation hold, as in Lemma 17.2. In the perfect class the unboundedness argument of Theorem 17.12 applies. Each of the at most n stages (between two decreases of the number of the trees, at each d_1, at each augmentation and at each closed cycle) has $O(n)$ steps, and the heaps make the work of a stage $O(m \log n)$, as in Theorem 17.13. □

Their constructor is `( const Graph & graph , const WeightMap & weight , bool allow_loops = true )`, their types are `Graph`, `WeightMap`, `Value` and `MatchingMap`, and their constants `primalScale` and `dualScale` are public. Their execution functions are `init()`, `start()` and `run()`, which in the perfect class return `false` when no perfect fractional matching exists. Their queries are `matchingWeight()` (the weight scaled by `primalScale`), `matchingSize()`, `matching( const Edge & )`, `matching( const Node & )`, `matchingMap()`, `dualValue()` and `nodeValue( const Node & )`, with the duals scaled by `dualScale` as in section 17.5. At the optimum, `primalScale` times `dualValue()` equals `dualScale` times `matchingWeight()`. The program of section 17.5 checks this on a triangle, whose perfect fractional matching has weight $\frac{15}{2}$, while the triangle has no perfect matching at all, and it checks (17.8) for the barrier of `MaxFractionalMatching`.

17.7 Exercises

Exercise 17.1. Prove that the set returned by `cover()` after `HopcroftKarp::run()` is a vertex cover with `matchingSize()` nodes, using only the fact that the final matching admits no augmenting path (cf. Theorem 17.2). Then check it on random bipartite graphs against `MaxMatching` run on the same graph. (The test `test/bipartite_matching_test.cc` does the latter. The program `manual/examples/17-bipartite.cc` checks the cover on one instance.)

Exercise 17.2. Show that the potentials returned by `MinCostAssignment` give a feasible solution of the linear programming dual of (17.6) with the value of the matching, and use them to write a check that needs no other solver. Then compare `totalCost()` with the optimum of the same problem written as a minimum cost flow problem for `NetworkSimplex` (chapter 13). (The test `test/bipartite_matching_test.cc` makes this comparison and checks the conditions (17.7).)

Exercise 17.3. Modify the graph of Figure 17.1 so that its barrier has two nodes, predict the statuses of the nodes from Theorem 17.4, and verify the prediction and the equality of (17.3) with the program `manual/examples/17-edmonds.cc`.

Exercise 17.4. On a complete graph with an odd number of nodes, measure the running time of `MaxMatching::run()` with and without the decomposition, and explain the difference from step 2 of Algorithm 17.3. (The test `test/matching_test.cc` checks that the two give matchings of the same size.)

Exercise 17.5. Find a minimum cost perfect matching of a general graph with `MaxWeightedPerfectMatching`, by negating the costs, and verify its optimality from `nodeValue()`, `blossomValue()` and `BlossomIt` as the function `reducedWeight()` of `manual/examples/17-weighted.cc` does, with the sign of y free. (The test `test/matching_test.cc` checks the dual solutions of both weighted classes in this way.)

Exercise 17.6. On a graph made of node-disjoint triangles with unit weights, compare the values of `MaxWeightedFractionalMatching` and `MaxWeightedMatching`, and find the odd set inequalities (17.4) that close the gap. Then compare the running time of `MaxWeightedMatching` started by `init()` and by `fractionalInit()`. (The test `test/fractional_matching_test.cc` checks the primal and dual solutions of the fractional classes.)

Exercise 17.7. Prove that, after a failed `startPerfect()` of `MaxFractionalMatching` caused by a gap at level l, the nodes on the top satisfy $|S| > |N(S)|$, by the argument of Lemma 17.9 and Theorem 17.14. (The test `test/fractional_matching_test.cc` checks the barrier after each failure on its instances.)

18. Connectivity, planarity, isomorphism

This chapter collects the algorithms that decide a structural property of a graph, or compute the object that certifies it, instead of optimizing a weight. They are the components of a graph with the nodes, edges and arcs that separate them, the tests of acyclicity and the topological order (`lemon/connectivity.h`), the Euler tours (`lemon/euler.h`), the planarity test with its embedding, its straight-line drawing and the colorings of a planar graph (`lemon/planarity.h`), and the search for an isomorphic copy of a graph inside another (`lemon/vf2.h`, `lemon/vf2pp.h`). All of them read only the structure of the graph (and, for the isomorphism, two node labelings), and most of them run in time linear in its size (which, on the graphs met in practice, makes them rather cheap compared with the optimization algorithms of the previous chapters). We remark three exceptions: the planarity test, which sorts the children of each node of a search tree and therefore costs $O(n \log n)$ in the worst case, the five coloring, which is quadratic in the worst case, and the isomorphism search, which solves an NP-complete problem and may take exponential time. We write $G = (V, E)$ for an undirected graph and $D = (V, A)$ for a digraph, with $n = |V|$ and $m = |E|$ or $m = |A|$, and $\deg(v)$ for the number of edges incident to v (a loop counted twice). We use the iterators, the maps and the graph structures of chapter 3 and chapter 4, while the depth-first and breadth-first searches (DFS and BFS), on which most functions are built, are those of chapter 9.

18.1 Connected components, acyclicity and simple graphs

Two nodes of an undirected graph are *connected* if a path joins them. This is an equivalence relation, whose classes are the *connected components*, and a graph is connected if it has one component (by convention, the graph without nodes is connected and has zero components, as the documentation of each function states). Three functions compute the components, i.e., `bool connected(g)`, `int countConnectedComponents(g)` and `int connectedComponents(g, compMap)`. Of these, `connected()` runs one depth-first search from the first node and checks that it has reached all the nodes, while `countConnectedComponents()` starts a breadth-first search from each node not yet reached and counts the searches. In the same way, `connectedComponents()` writes in the writable node map `compMap` the index of the component of each node, from 0 to the number of components minus one, and returns that number; the nodes of one component receive their index consecutively, in the order in which the search reaches them, and each value is written once.

Theorem 18.1. *A search (depth-first or breadth-first) of an undirected graph started from a node s reaches the nodes of the connected component of s. Hence, `connected()`, `countConnectedComponents()` and `connectedComponents()` return what their documentation states, and each of them costs $O(n + m)$.*

Proof. A node is reached only through an edge from a node already reached; hence, by induction on the order of discovery, each reached node is joined to s by a path. Conversely, let P be a path from s to a node t that is not reached, and let uv be the first edge of P with u reached and v not: the search processes u, and therefore examines the edge uv and reaches v, a contradiction. A search started from a node not yet reached thus collects one new component, which gives the count and the consecutive numbering. Each search examines each edge of its component twice (once from each end) and each node once, and the initialization of the maps of the search costs $O(n)$. Hence, the total is $O(n + m)$. □

A graph is *acyclic* (a forest) if it contains no cycle, and it is a *tree* if, furthermore, it is connected. The function `acyclic(g)` runs a depth-first search from each unreached node. It returns `false` at the first arc that leads to a reached node without being the opposite of the arc by which the search entered the current node; similarly, `tree(g)` makes the same test from the first node only and then checks that all the nodes have been reached (thus, the empty graph is a tree). The proof rests on a property of the depth-first search of an undirected graph, which

we use throughout the chapter. We call *tree edges* the edges by which the search discovers a node, *non-tree edges* the others, and we say that u is an *ancestor* of v (and v a *descendant* of u) if u lies on the path of tree edges from the root to v.

Lemma 18.1. *In a depth-first search of an undirected graph, every non-tree edge joins a node to one of its ancestors.*

Proof. Let uv be a non-tree edge with u discovered first. The search leaves u only after examining all the edges of u, including uv; at that moment v is already reached (otherwise uv would become a tree edge), and, since v was not reached when u was discovered, it was reached while u was active, i.e., inside the subtree of u. Hence, u is an ancestor of v. □

Theorem 18.2. *The graph has a cycle if and only if* `acyclic()` *meets an arc uv with v already reached and uv different from the opposite of the arc that entered u. Hence,* `acyclic()` *and* `tree()` *are correct, and both cost $O(n + m)$.*

Proof. Such an arc belongs to a non-tree edge, since the only tree edge at u whose other end is reached is the one that entered u, and its arc from u is the opposite of the arc that entered u. By Lemma 18.1 that edge, together with the path of tree edges between its two ends, forms a cycle (of length one for a loop, and of length two for an edge parallel to a tree edge). Conversely, a cycle cannot consist of tree edges only, so it has a non-tree edge, and the first of its two arcs that the search examines leads to a reached node (otherwise the edge would become a tree edge) and is not the opposite of the arc that entered its source. Each arc is examined at most once. □

A graph is *bipartite* if its nodes can be split into two sets with no edge inside either of them. One decides it with `bipartite(g)`, while `bool bipartitePartitions(g, partMap)` also writes the two sides into the writable `bool` node map `partMap` (`true` on the side of the first node of each component). Both run a breadth-first search that puts each discovered node on the side opposite to that of its parent. They stop with `false` after processing a node that has an edge to a node of its own side; in that case `partMap` has been written only on the nodes reached so far, and its values are of no use.

Theorem 18.3. *A graph is bipartite if and only if it has no cycle of odd length, and* `bipartite()` *and* `bipartitePartitions()` *return* `true` *if and only if the graph is bipartite, in which case* `partMap` *is a bipartition. Both cost $O(n + m)$.*

Proof. In a bipartite graph every closed walk alternates between the two sides and has even length. The side that the search gives to a node is the parity of its depth in the tree of the search, since each discovered node takes the side opposite to that of its parent. If no edge joins two nodes of the same side, the two sides are a bipartition. Otherwise, let xy be such an edge: the paths of tree edges from x and from y to their lowest common ancestor z have lengths of the same parity, and together with xy they form a closed walk of odd length. Such a walk contains a cycle of odd length (if it repeats a node, it splits into two shorter closed walks, one of which is odd), and the graph is not bipartite. Indeed, the cost is that of the searches of Theorem 18.1. □

Finally, three functions test whether a graph is simple, and they apply to digraphs and to undirected graphs alike. Here `loopFree(g)` returns `true` if no arc has the same source and target, `parallelFree(g)` if no two arcs that leave a node enter the same node, and `simpleGraph(g)` if both conditions hold. The latter two stamp, in a node map, the targets of the arcs leaving a node with the index of that node, and they report a target found already stamped; hence, each of the three costs $O(n + m)$ and needs no sorting. We remark that on an undirected graph the two arcs of a loop leave its node and enter it, so that `parallelFree()` returns `false` on an undirected graph with a loop even if no two edges are parallel (on a digraph a loop is one arc, and does not count). Several algorithms of this chapter (cf. §18.5, §18.6 and §18.7) assume a simple graph, which `simpleGraph()` checks (a graph read from a file, e.g., may well have parallel edges).

18.2 Biconnectivity

A connected graph may be disconnected by the removal of one node or of one edge, and the parts that survive such removals measure how "reliable" its connections are (e.g., those of a communication network). A *cut node* (or articulation node) is a node whose removal increases the number of connected components, and a *cut edge* (or bridge) is an edge with the same property. Two edges are in the same *bi-node-connected component*, or *block*, if they are equal or lie on a common cycle; this is an equivalence relation on the edges, whose classes are separated by the cut nodes. Two nodes are in the same *bi-edge-connected component* if two edge-disjoint paths join them; this is an equivalence relation on the nodes, whose classes are separated by the bridges. For instance, the graph

of Figure 18.1 has four blocks (the triangles A and B, which share the cut node 2, the bridge C and the triangle D), three cut nodes and two bi-edge-connected components.

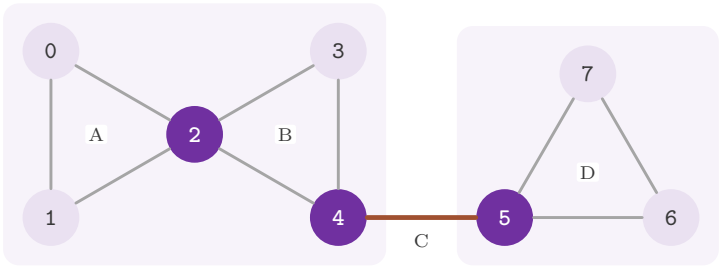


Figure 18.1. A connected graph with four blocks A, B, C and D. The cut nodes 2, 4 and 5 are in the primary colour, the bridge 4-5 in the colour of the added items, and the two bi-edge-connected components, which the bridge separates, are the shaded regions.

The header provides, for the nodes and for the edges, the same kinds of function as for the connected components. For the blocks they are `bool biNodeConnected(g)`, `int countBiNodeConnectedComponents(g)`, `int biNodeConnectedComponents(g, compMap)` and `int biNodeConnectedCutNodes(g, cutMap)`. For the bi-edge-connected components they are `bool biEdgeConnected(g)`, `int countBiEdgeConnectedComponents(g)`, `int biEdgeConnectedComponents(g, compMap)` and `int biEdgeConnectedCutEdges(g, cutMap)`. Note that the map of the blocks is an edge map, since a block is a set of edges and a cut node belongs to several blocks, whereas the map of the bi-edge-connected components is a node map; both are numbered from 0 as in §18.1, consecutively within a component. Those that find the cut nodes and the cut edges set `cutMap` to `true` on them (once each) and return their number. They leave the other items untouched, and the caller must therefore initialize the map to `false` when it reads the other items too.

abLEMON. `biNodeConnectedCutNodes()` keeps the nodes it has counted in a map of its own and only writes `cutMap`, as the concept check of its argument (a writable map) requires. In LEMON 1.3.1 it read `cutMap` in order not to count a node twice, so that a node already `true` on entry was neither counted nor reported; the test `connectivity_test` passes a map that is `true` everywhere.

At the boundary we follow the conventions of the header. A graph with zero or one node is bi-node-connected, two isolated nodes are not, and two nodes joined by an edge are. In general, `biNodeConnected()` returns `false` if the graph has an isolated node together with some other node, and otherwise whether it has at most one block (an isolated node is in no block, since a block is a set of edges). On the other hand, `biEdgeConnected()` returns whether there is at most one bi-edge-connected component; hence, the empty graph is bi-edge-connected. A loop forms a block of its own (which may be surprising), and `biNodeConnectedCutNodes()` reports the node that carries it as a cut node, also when its removal does not increase the number of components (e.g., a single node with a loop).

All these functions rely on one depth-first search, with the low points of Hopcroft and Tarjan 1973 and Tarjan 1972. We denote by $\mathrm{num}(v)$ the index of v in the order of discovery, by T_v the subtree of the search rooted at v, and by $\mathrm{low}(v)$ the least of $\mathrm{num}(v)$ and of the indices $\mathrm{num}(x)$ of the nodes x joined by a non-tree edge to a node of T_v.

Lemma 18.2. *In a depth-first search of a connected graph, the root is a cut node if and only if it has at least two children in the tree, and any other node u is a cut node if and only if it has a child w with $\mathrm{low}(w) \geq \mathrm{num}(u)$. The tree edge uw, with u the parent of w, is a bridge if and only if $\mathrm{low}'(w) > \mathrm{num}(u)$, where low' is computed like* low *but without the edge uw itself; a non-tree edge is never a bridge.*

Proof. By Lemma 18.1, an edge with one end in T_w and the other outside T_w is either the tree edge uw or a non-tree edge to an ancestor of w, i.e., to u or to an ancestor of u. If u is not the root and $\mathrm{low}(w) \geq \mathrm{num}(u)$, no edge leaves T_w except towards u, and the removal of u separates T_w from the parent of u. If, instead, $\mathrm{low}(w) < \mathrm{num}(u)$ for each child w of u, each T_w keeps an edge to a proper ancestor of u after the removal of u, and the nodes outside T_u stay connected by the tree edges; hence, u is not a cut node. For the root, Lemma 18.1 implies that no edge joins the subtrees of two different children, so the removal of the root separates them when there are two or more, and leaves the connected T_w of a single child w. Finally, the removal of the tree edge uw disconnects the graph if and only if no other edge leaves T_w, i.e., if and only if $\mathrm{low}'(w) > \mathrm{num}(u)$. A non-tree edge lies on the cycle that it closes with the tree path between its ends, and its removal disconnects nothing. □

In the library, the visitor of the cut nodes applies the lemma as Algorithm 18.1 shows, in which pred(v) is the parent of v in the tree.

Algorithm 18.1 (Cut nodes by the low points).

```
1  for each node r not yet reached: root := r; rootCut := false; DFS(r)
2  DFS(v): num(v) := low(v) := next index
3     for each arc vw:
4        if w is not reached: pred(w) := v; DFS(w)
5           low(v) := min(low(v), low(w))
6           if num(v) <= low(w):
7              if v != root or rootCut: mark v as a cut node, once
8              else rootCut := true
9        else if w = v: mark v as a cut node, once        (a loop)
10       else if w != pred(v): low(v) := min(low(v), num(w))
```

Steps 2, 4 and 10 are the visitor functions `reach()`, `discover()` and `examine()` of `DfsVisit`, and steps 5 to 8 its `backtrack()`; the flag "once" is the private map of the note above. Step 7 marks the root at the second of its children that satisfy the condition of step 6, the flag `rootCut` recording the first one. Step 10 skips all the edges to the parent, including the parallel ones, which does not change the outcome of step 6.

Theorem 18.4. *Algorithm 18.1 marks exactly the cut nodes of a loopless graph (and, in addition, the nodes that carry a loop), and it costs $O(n+m)$. The count of the blocks, which grows by one at each loop and at each tree edge vw that satisfies the condition of step 6, is the number of blocks, and `biNodeConnectedComponents()` gives the same index to two edges if and only if they lie in the same block.*

Proof. When DFS(w) returns, low(w) is the least of num(w), of the values num(x) of step 10 over the arcs leaving w and of the low points of its children. By induction on the height of T_w, this is the low point of the definition (the arcs of step 10 that lead to descendants have num(x) > num(w) and do not count, and the skipped edges to the parent only add num(pred(w)), which leaves the test of step 6 unchanged). Since each child of the root satisfies step 6, steps 6 to 8 apply Lemma 18.2 in each component. For the blocks, call a tree edge vw that satisfies step 6 a *closing* edge. Removing v separates T_w (or, at the root, the subtree of each child), so no cycle contains both an edge of $T_w \cup \{vw\}$ and an edge outside it. Conversely, the edges of $T_w \cup \{vw\}$ that are not inside the subtree of a lower closing edge, together with the non-tree edges from those nodes to v or below, are pairwise on common cycles (each of them closes a cycle through the tree path, and these cycles overlap along tree edges that are not closing). Hence, each closing edge determines one block. The function that numbers them keeps on a stack the edges discovered since the block was opened and pops them when its closing edge is backtracked. The search examines each arc once, and each edge enters and leaves the stack once. □

The full proof of the block structure is in Hopcroft and Tarjan 1973 and Tarjan 1972 (see also Cormen et al. 2001, Problem 22-2), whose arguments the proof above follows. In contrast, the functions on the bi-edge-connected components store the arc by which each node was entered and skip only its opposite, since a second edge parallel to a tree edge does give a path back and prevents that edge from being a bridge. They update low(v) at a non-tree arc vx with the current low point of x rather than with num(x), which, as the following proof shows, does not change the test.

Theorem 18.5. *In the search of the bi-edge-connected components, a node $v \neq$ root satisfies* low(v) = num(v) *when it is left if and only if the tree edge that entered it is a bridge. Hence, `biEdgeConnectedCutEdges()` marks the bridges, `biEdgeConnectedComponents()` numbers exactly the bi-edge-connected components (the nodes still on a stack when such a v, or a root, is left), and all four functions cost $O(n+m)$.*

Proof. At a non-tree arc vx, the current low point of x is at most num(x), and it is less than num(v) only if x is outside T_v or some node of T_x, already examined, has an edge leaving T_v. Hence, low(v) < num(v) holds when v is left if and only if an edge other than the one that entered v leaves T_v, which by Lemma 18.2 is the negation of the bridge condition. The removal of all the bridges leaves connected graphs without bridges, whose node sets are the sets popped from the stack. In a connected graph without bridges any two nodes are joined by two edge-disjoint paths, by the edge version of Menger's theorem (Diestel 2017, Theorem 3.3.6), while two nodes separated by a bridge are not. Each arc is examined once and each node is pushed and popped once. □

18.3 Strong components and topological order

Two nodes of a digraph are *strongly connected* if directed paths join them in both directions. This is an equivalence relation, whose classes are the *strongly connected components*, and the digraph that has one node per component, and an arc from one component to another whenever some arc of D goes so, is acyclic (we call it the *condensation* of D). The library offers `bool stronglyConnected(d)`, `int countStronglyConnectedComponents(d)`, `int stronglyConnectedComponents(d, compMap)` and `int stronglyConnectedCutArcs(d, cutMap)`, with the conventions of §18.1, i.e., the empty digraph is strongly connected and has zero components. Of these, `stronglyConnected()` runs a depth-first search from the first node on D and another on its reverse (the adaptor `ReverseDigraph` of chapter 5), and it checks that both searches reach all the nodes (which is correct by Theorem 18.1 applied to directed paths: a node is reached from s if and only if a directed path joins s to it). Instead, the other three functions use the algorithm usually attributed to Kosaraju and published by Sharir 1981.

Algorithm 18.2 (Strongly connected components by two depth-first searches).

```
1  run a DFS of D from each node not yet reached, and list the nodes
   in the order in which the search leaves them
2  k := 0
3  for each node v in the reverse of that list:
4     if v is not reached by the second search:
5        run a DFS of the reverse of D from v; give index k to the nodes
         it reaches; k := k + 1
6  return k
```

For a set C of nodes we denote by $f(C)$ the largest position, in the list of step 1, of a node of C.

Lemma 18.3. *If an arc of D goes from a strongly connected component C to another one C', then $f(C) > f(C')$.*

Proof. Let x be the first node of $C \cup C'$ that the search of step 1 reaches. If $x \in C$, each node of $C \cup C'$ is reachable from x by a directed path of nodes not yet reached, and the search leaves all of them before leaving x; hence, $f(C)$ is the position of x, larger than any position in C'. If $x \in C'$, the search of x reaches all of C' and no node of C (a directed path from C' to C would merge the two components), and it leaves all of C' before any node of C is reached. □

Theorem 18.6. *Each search of step 5 gives its index to exactly one strongly connected component, and no arc of D goes from a component of higher index to one of lower index. Hence, `countStronglyConnectedComponents()` and `stronglyConnectedComponents()` are correct, the numbering is a topological order of the condensation, the arcs that `stronglyConnectedCutArcs()` marks are the arcs whose ends lie in different components, and each of the four functions costs $O(n + m)$.*

Proof. Let v start a search of step 5, and let C be its component. All of C is reachable from v in the reverse digraph, and none of it was reached by an earlier search, since an earlier search would have started from a node of larger position and, by induction, collected its own component only. An arc of the reverse digraph that leaves C corresponds to an arc of D that enters C from a component C'', and by Lemma 18.3 $f(C'') > f(C)$; hence C'' contains a node of larger position than v, and by induction it was collected by an earlier search. Thus the search from v reaches C and nothing else. The components are numbered in decreasing order of f, so an arc from C to C', for which $f(C) > f(C')$, goes from a lower index to a higher one. The cut arcs are found in the second pass: when an arc of the reverse digraph is examined, its other end is either in the current component or in one already numbered, and the arc is marked if the two indices differ. Each pass examines each arc once. □

The function `stronglyConnectedCutArcs()` sets its arcs to `true` in `cutMap`, leaves the other arcs untouched (the map must start at `false`), and returns their number. The proof is the one of Cormen et al. 2001, §22.5 (Lemma 22.14 and Theorem 22.16), with the list of step 1 in place of the finishing times.

A digraph without directed cycles is a *DAG*, and a *topological order* of a DAG is a numbering of its nodes in which each arc goes from a lower to a higher number. Three functions deal with them, i.e., `bool dag(d)`, `void topologicalSort(d, order)` and `bool checkedTopologicalSort(d, order)`. The first one runs a depth-first search from each unreached node and returns `false` at the first arc whose target has been reached but not yet left (a *back arc*; a loop is one). Next, `topologicalSort()` writes in the writable node map `order` the values from $n - 1$ down to 0, in the order in which a depth-first search leaves the nodes, each value once. It does not check that the digraph is a DAG, which is its precondition. By contrast, `checkedTopologicalSort()` does check it, and it needs a map that can also be read: it first sets all the values to -1, and it returns `false` at the first arc that leads to a node reached and still at -1 (i.e., not yet left), leaving the map partly written.

Theorem 18.7. *A digraph has a directed cycle if and only if a depth-first search of it meets a back arc. Hence,* `dag()` *and* `checkedTopologicalSort()` *return* `false` *if and only if the digraph has a directed cycle; on a DAG,* `topologicalSort()` *and* `checkedTopologicalSort()` *give* `order[u] < order[v]` *for each arc* uv*. All three cost* $O(n + m)$*.*

Proof. A back arc uv goes to a node v that is still active, i.e., to an ancestor of u in the tree, and it closes a directed cycle with the tree path from v to u. Conversely, let C be a directed cycle and v its first node reached; all the other nodes of C are reachable from v through unreached nodes, and they become descendants of v, so that the arc of C that enters v is examined while v is active, and it is a back arc. On a DAG there is no back arc, so when an arc uv is examined, v is either unreached (and then it becomes a descendant of u and is left before u) or already left. In both cases v is left before u, and since the values are given in decreasing order of leaving, $\text{order}(u) < \text{order}(v)$. □

One usually runs these functions, or the strongly connected components, before an algorithm that assumes a DAG or a strongly connected digraph; for instance, the minimum mean cycle algorithms of chapter 16 call `stronglyConnectedComponents()` and then work on one component at a time. The program `manual/examples/18-components.cc` (the test `manual_18_components`) builds the graph of Figure 18.1 and checks the numbers of its components, blocks, cut nodes and bridges. It then checks that the numbering of the strongly connected components of a digraph with three of them is a topological order of the condensation, and sorts a "small" DAG.

18.4 Euler tours

An *Euler tour* of a graph is a closed walk that traverses each edge once (for a digraph, each arc in its direction). We recall its characterization, which is the oldest theorem of graph theory, and we prove it after the algorithm, whose analysis gives the sufficiency.

Theorem 18.8. *A graph whose edges lie in one connected component has an Euler tour if and only if each node has even degree. A digraph whose arcs lie in one connected component of the underlying undirected graph has an Euler tour if and only if each node has as many entering arcs as leaving ones.*

The proof of sufficiency, due to Hierholzer and Wiener 1873, is also the algorithm: from any node, a walk that never reuses an arc can stop only at its starting node, and a closed walk that misses some arc can be extended by splicing into it a closed walk from one of its nodes. In abLEMON the tour is given by an iterator, `DiEulerIt< GR >` for a digraph and `EulerIt< GR >` for an undirected graph. It is constructed from the graph and, optionally, from the starting node (by default `INVALID`, which stands for the first node that has a leaving arc), and it is advanced with `++` until it compares equal to `INVALID` (`operator==` and `operator!=` with `INVALID` test whether the tour is over). `DiEulerIt` converts to `GR::Arc`, while `EulerIt` converts to `GR::Arc` and to `GR::Edge`, and it gives arcs in order to show in which direction each edge is traversed; after the end, both convert to `INVALID`. As for the other iterators of the library, the postfix `++` returns the current arc instead of the iterator. A digraph can also be given to `EulerIt` through the adaptor `Undirector`, and an undirected graph to `DiEulerIt`, which then treats its arcs as those of a symmetric digraph.

Algorithm 18.3 (Euler tour, as `DiEulerIt` produces it).

```
1  for each node v: next(v) := first arc leaving v
2  L := empty list; s := start
3  while next(s) != INVALID: append next(s) to L; advance next(s);
                             s := target of the arc appended
4  (operator Arc) the current arc is the first of L
5  (operator++) s := target of the first arc of L; remove it from L;
6     while next(s) != INVALID: insert next(s) at the front of the rest
      of L, after the arcs inserted before; advance next(s);
      s := target of that arc
```

The constructor walks greedily from the start (step 3), and each increment splices, right after the arc just given, the closed walk that starts where that arc ends and uses the arcs not yet taken (step 6). `EulerIt` works in the same way with an edge map that marks the edges already used, since each edge appears as an arc leaving each of its two ends, and it skips the arcs of the marked edges when it advances next(v). We call a digraph *balanced* if each node has as many entering arcs as leaving ones, and an undirected graph balanced if each degree is even.

Lemma 18.4. *In a balanced graph, a walk from* s *that uses each arc (edge) at most once and stops when its current node has no unused arc stops at* s*, and the unused arcs form a balanced graph again.*

Proof. Each time the walk enters a node $x \neq s$, it has used one more arc entering x than arcs leaving it (for an undirected graph, an odd number of the edges of x), so by the balance an unused arc leaves x and the walk does not stop there. The used arcs form a closed walk, which enters and leaves each node equally often; their removal keeps the balance. □

Theorem 18.9. *If the graph is balanced, its arcs (edges) lie in one connected component of the underlying undirected graph, and the start has a leaving arc, the iterators give an Euler tour that starts at the start. In all cases they cost $O(n + m)$ for the whole walk.*

Proof. Let W be the sequence of the arcs already given followed by L. By Lemma 18.4, after the constructor W is a closed walk from the start that repeats no arc. At an increment the first arc of L, say uv, is given, and step 6 is the walk of Lemma 18.4 from v on the unused arcs, which are balanced; it is closed at v, and its insertion right after uv keeps W a closed walk that repeats no arc. Furthermore, step 3 or step 6 stops at a node whose pointer is `INVALID`, which is the node where the walk started; hence, when the iteration ends, each node that W visits has no unused leaving arc. Suppose that an arc is unused at the end; since the arcs lie in one connected component, some node t visited by W is joined by an unused arc to a node not visited. The arc cannot leave t, so it enters t; but W is closed and contains all the arcs leaving t, hence as many arcs entering t as the balance allows, i.e., all of them, a contradiction. The pointers next(v) only advance and each arc enters L once, while the initialization of the pointers (and of the edge map of `EulerIt`) costs $O(n)$. □

Proof of Theorem 18.8. A closed walk enters each node as often as it leaves it (and, undirected, uses an even number of its edge ends), so the condition is necessary. Theorem 18.9 shows that it suffices. □

If the graph has no Euler tour, the iterators still give a walk (possibly a rather short one), which is not closed or does not contain all the edges, and from which one can usually tell little. One should therefore check the property first, with `bool eulerian(g)`, which works on digraphs and on undirected graphs (the two versions are selected through the undirected tag of the graph type). On an undirected graph it returns `true` if each node has even degree (`countIncEdges()`, which counts a loop twice) and the graph is connected; on a digraph, if each node has as many entering as leaving arcs and the underlying undirected graph (the adaptor `Undirector`) is connected. Both checks cost $O(n + m)$. We remark that `eulerian()` requires that all the nodes be connected, whereas Theorem 18.8 requires it of the edges only; thus, a graph with an Euler tour and an isolated node is not Eulerian for the function, as its documentation warns. The program `manual/examples/18-euler.cc` (the test `manual_18_euler`) follows the tour of the complete graph on five nodes by `EulerIt` and that of two directed triangles sharing a node by `DiEulerIt`. It checks each to be a closed walk through every edge or arc once, and checks that an isolated node makes `eulerian()` false while the tour is still there.

18.5 Planarity, embedding and drawing

We recall that a graph is *planar* if it can be drawn in the plane with no two edges crossing. Up to deformation, such a drawing is described by its *combinatorial embedding*, i.e., by the cyclic order of the edges around each node (e.g., counterclockwise in the drawing). In turn, the *faces* of the drawing are the orbits of the map that takes an arc uv to the arc that follows vu around v, i.e., the walks along the boundaries of the regions into which the drawing cuts the plane. We first state the counting facts that the algorithms of this section and of §18.6 use.

Lemma 18.5. *A connected plane graph with n nodes, m edges and f faces satisfies $n - m + f = 2$ (Euler's formula). Hence, a planar simple graph with $n \geq 3$ nodes has $m \leq 3n - 6$ edges, and every planar simple graph has a node of degree at most five.*

Proof. We argue by induction on m. A tree has $m = n - 1$ and one face. If the graph is not a tree, an edge e of a cycle has, by the Jordan curve theorem, different faces on its two sides, and its removal leaves a connected plane graph with one edge and one face less (see Diestel 2017, Theorem 4.2.9). For the bound we may add edges until the graph is connected, which keeps it planar and simple. In a simple graph with $n \geq 3$ nodes each face is bounded by a walk of at least three edges, and each edge lies twice on these walks; hence $2m \geq 3f$, and Euler's formula gives $m \leq 3n - 6$. Finally, the degrees sum to $2m \leq 6n - 12 < 6n$, so some node has degree at most five (with fewer than three nodes, each degree is at most one). □

Furthermore, the obstacles to planarity are known.

Theorem 18.10. *A graph is planar if and only if it contains no subdivision of K_5 (the complete graph on five nodes) and no subdivision of $K_{3,3}$ (the complete bipartite graph with three nodes on each side).*

Proof sketch. Neither graph is planar: K_5 has $10 > 3 \cdot 5 - 6$ edges (Lemma 18.5), and $K_{3,3}$, being bipartite, has faces of at least four edges, hence $2m \geq 4f$ and $m \leq 2n - 4 = 8 < 9$. A subdivision of a nonplanar graph is

nonplanar, and so is a graph that contains it. The converse, i.e., that a nonplanar graph contains one of the two subdivisions, is the theorem of Kuratowski 1930. Its proof, by a minimal counterexample that is shown to be 3-connected and then embedded around a contracted edge, is given in Diestel 2017, §4.4, Theorem 4.4.6. □

A subdivision of either graph inside a nonplanar graph, called a *Kuratowski subdivision*, certifies that the graph is not planar, as an embedding certifies that it is. We now describe the implementation of the algorithm of Boyer and Myrvold 2004, which computes either an embedding or a Kuratowski subdivision, and which is usually regarded as one of the simplest linear planarity tests. The function `bool checkPlanarity(g)` runs a simplified version of it that builds neither certificate, and the class `PlanarEmbedding< Graph >` runs the full one. Both expect an undirected simple graph (a precondition of the constructor), which `simpleGraph()` of §18.1 checks. Once the class is constructed from the graph, `run(bool kuratowski = true)` returns whether the graph is planar. If it is, `next(a)` gives the arc that follows the arc `a` in the cyclic order of the arcs leaving its source, and `embeddingMap()` returns the whole order as a `const` reference to an arc map of arcs (the type `EmbeddingMap`, i.e., `Graph::ArcMap< Arc >`). If it is not, and `run()` was called with `true`, `kuratowski(e)` returns whether the edge `e` belongs to the subdivision found, whose nodes of degree three or four are those of a $K_{3,3}$ or of a K_5 (with `false`, or before `run()`, it returns `false` on every edge).

Algorithm 18.4 (Planarity by edge addition, outline of `PlanarEmbedding::run()`).

```
1  run a DFS; record the order of discovery, the tree edges, the low points
   and, for each node, the least ancestor joined to it by a back edge;
   sort the children of each node by their low points (radixSort)
2  for each node v in reverse order of discovery:
3     embed each tree edge from v to a child as a separate biconnected
      component, rooted at a virtual copy of v
4     for each back edge from a descendant w to v: walk up from w and mark
      the components that the edge will cross (walkUp)
5     for each marked root: walk down along the external faces, embedding
      the back edges to v and merging the components on the way, flipping
      a component when its orientation requires it (walkDown)
6     if a back edge to v is left unembedded:
         if kuratowski: isolate a Kuratowski subdivision
         return false
7  merge the remaining components and store the cyclic orders; return true
```

Theorem 18.11. *`PlanarEmbedding::run()` and `checkPlanarity()` return `true` if and only if the simple graph is planar; in that case `embeddingMap()` is a combinatorial embedding of a plane drawing, and otherwise, after `run(true)`, the edges marked by `kuratowski()` form a Kuratowski subdivision. The sort of step 1 costs $O(n \log n)$ in all, and the rest of the method $O(n + m)$, which is $O(n)$ on a planar simple graph.*

Proof sketch. The invariant, proved by Boyer and Myrvold 2004, is that after processing v the tree, together with the back edges to the nodes processed so far, is embedded with each node that still has a back edge to an unprocessed ancestor on the external face of its component. The walk-down of step 5 embeds each back edge to v that some embedding with this property can accommodate, and it fails only when none can; at that point the obstruction it meets is completed, along the external faces and the tree paths, to a subdivision of $K_{3,3}$ or of K_5. For the cost, the children of a node are sorted by `radixSort()` of chapter 8, which costs $O(k \log n)$ for k children with low points below n, and the $n - 1$ children of all the nodes give $O(n \log n)$ (the paper sorts by buckets in linear time). The flips of step 5 are recorded lazily, with one flag per component, and each step of the walks is charged to an edge embedded or to a face boundary shortened, which keeps the rest linear. On a nonplanar graph the method stops at the first node at which a back edge is left unembedded. □

We remark that the documentation of `PlanarEmbedding` states $O(n)$, which holds for the rest of the method but not for the sort of step 1.

The class `PlanarDrawing< Graph >` computes a straight-line drawing on an integer grid by the method of Schnyder 1990. Its coordinates have the type `dim2::Point< int >` (the type `Point`, stored in the node map type `PointMap`), and its interface is the constructor from a simple graph and the two functions `run()` and `run(embedding)`. Of these, the former computes an embedding with `PlanarEmbedding` and returns `false` if the graph is not planar; the latter, a template on the type of the map, draws according to a given combinatorial embedding (e.g., `embeddingMap()` of a `PlanarEmbedding`) and has no return value. Then the coordinates are read with `operator[](node)` or, as a node map, with `coords()`. On a graph with $m = 3n - 6$, `run(embedding)` draws the graph itself; on any other graph it copies the graph into an edge set and adds edges, consistently with

the embedding, to make it connected, then bi-node-connected and then maximal planar, draws the triangulation and keeps the coordinates of the nodes. A graph with fewer than three nodes is drawn on a vertical line, with $x = 0$ and $y = 0, 1$.

Theorem 18.12. *For a planar simple graph with $n \geq 3$ nodes and a combinatorial embedding of a plane drawing,* `PlanarDrawing::run(embedding)` *places the nodes at distinct points of the grid $[0, n-2] \times [0, n-2]$ so that the straight segments of the edges do not cross.*

Proof sketch. A maximal planar graph (a triangulation, with $m = 3n - 6$) has a *realizer*, i.e., a partition of its inner edges into three trees, rooted at the three nodes of the outer face, such that around each inner node the edges of the three trees appear in a fixed cyclic pattern. The three paths from a node to the three roots split the triangle into three regions, and the numbers of nodes in two of them (less the nodes on their boundary paths) are integer coordinates in $[0, n-2]$, for which each face is a non-degenerate triangle with the orientation of the embedding. This is the main theorem of Schnyder 1990; the edges added to reach a triangulation are dropped afterwards, which removes segments and creates no crossing. The header states a time of $O(n)$. □

The grid is somewhat small (of side $n - 2$), which is often convenient for a picture, although some angles between edges may be quite sharp. Figure 18.2 shows the drawing of the cube that the program of the next section computes.

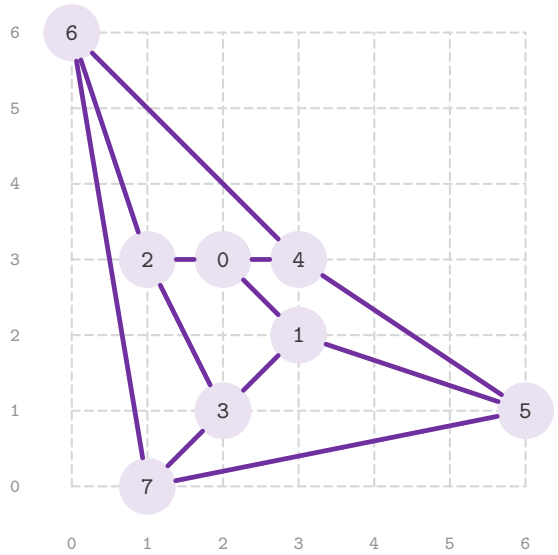


Figure 18.2. The straight-line drawing of the cube computed by `PlanarDrawing` in the program 18-planar.cc: the eight nodes on the grid $[0, 6] \times [0, 6]$, at the coordinates that the program prints, and no two edges crossing.

abLEMON. In LEMON 1.3.1, `PlanarDrawing::run()` without arguments passed the `PlanarEmbedding` object, instead of its embedding map, to `run(embedding)`, and therefore did not compile once instantiated. abLEMON has the correction made in the repository of LEMON after that release (trac #610), as well as the drawing of the graphs with fewer than three nodes (trac #611), on which the algorithm failed.

18.6 Coloring planar graphs

We call *coloring* of a graph an assignment of a color to each node, with different colors at the ends of each edge. As is well known, every planar graph can be colored with four colors, by the theorem of Appel and Haken 1977, whose proof yields a quadratic algorithm that the header judges of no practical use. Deciding whether a planar graph can be colored with three colors is NP-complete (Garey, Johnson and Stockmeyer 1976), and two colors, when they suffice, are found by `bipartitePartitions()` of §18.1. The class `PlanarColoring< Graph >` computes colorings with six and with five colors, both from the order in which the nodes are removed from the graph when one always takes a node of least degree among the remaining ones (a *smallest-last* order, cf. Matula and Beck 1983).

Lemma 18.6. *In the smallest-last order of a planar simple graph, each node has at most five neighbors that are removed after it.*

Proof. When a node is removed it has the least degree in the remaining graph, which is planar and simple, and therefore it has at most five neighbors left, by Lemma 18.5. □

Hence, one can give the colors greedily in the reverse of that order, which is what Algorithm 18.5 does; there $k = 6$ for `runSixColoring()` and $k = 5$ for `runFiveColoring()`.

Algorithm 18.5 (Coloring of a planar graph with six or five colors).

```
1  repeatedly remove a node of least degree in the remaining graph
   (a bucket heap keyed by the degree), and record the order
2  for each node v in the reverse of that order:
3     give v the least color in {0,..,k-1} not used by its colored
      neighbors
4     if there is none (k = 6): return false
5     if there is none (k = 5): take the first four colored neighbors
      u1, u2, u3, u4 of v in the cyclic order of the embedding;
6        if the Kempe chain of u1 (the component of the colors of u1, u3
         that contains u1) does not contain u3: swap its two colors,
         and give v the old color of u1
7        else swap the colors in the Kempe chain of u2 (colors of u2 and
         u4), and give v the old color of u2
```

Theorem 18.13. *`runSixColoring()` costs $O(n + m)$; on a planar simple graph it returns `true` with a coloring by at most six colors, and when it returns `false` the graph is not planar (although `true` does not prove that it is).*

Proof. By Lemma 18.6, when v is colored at most five of its neighbors are colored, so step 3 succeeds with six colors on a planar simple graph, and the contrapositive gives the second claim. A node is colored differently from the neighbors colored before it, and the neighbors colored after it avoid its color, so the coloring is proper. The heap receives n insertions, n deletions of the minimum and at most $2m$ decreases, each of which lowers a key by one. Hence the minimum of the bucket heap moves back by at most one per decrease, and its forward moves are bounded by these plus the largest degree, which gives $O(n + m)$ with the $O(n + m)$ of step 2. □

Theorem 18.14. *Given a planar simple graph and a combinatorial embedding of a plane drawing of it, `runFiveColoring(embedding)` computes a coloring with at most five colors in $O(n(n + m)) = O(n^2)$ time.*

Proof. Step 3 fails only when the colored neighbors of v, at most five by Lemma 18.6, use all five colors; then they are five, $u_1, \dots, u_5$ in the cyclic order around v, with distinct colors $c_1, \dots, c_5$. Note that a swap of the two colors of a Kempe chain, i.e., of a connected component of the subgraph induced by the colored nodes of those two colors, keeps the coloring proper. If the chain of u_1 does not contain u_3, the swap gives u_1 the color c_3 and leaves u_3 (and the neighbors u_2, u_4, u_5, of other colors) unchanged, so that c_1 becomes free at v. Otherwise a path of colors c_1, c_3 joins u_1 to u_3, and with v it forms a cycle of the plane drawing. Since u_2 and u_4 lie between u_1 and u_3 on the two sides of the cyclic order, the Jordan curve theorem puts them on different sides of the cycle, and a path of colors c_2, c_4 from u_2, which meets no node of the cycle, cannot reach u_4. Hence the swap of the chain of u_2 frees c_2 at v. This is the argument of Kempe 1879, as Heawood 1890 restricted it to five colors (see also Diestel 2017, Proposition 5.1.2). In fact, each recoloring is a breadth-first search of an adaptor `FilterNodes` of the graph, which costs $O(n + m)$, and it happens at most once per node; the rest is as in Theorem 18.13. □

As for its interface, `PlanarColoring` has the constructor from a simple graph and three functions to compute a coloring. Of these, `runSixColoring()` is the one of Theorem 18.13; `runFiveColoring(embedding)`, a template on the type of the map, needs a combinatorial embedding and has no return value; and `runFiveColoring()` computes the embedding with `PlanarEmbedding`, returns `false` on a nonplanar graph, and costs in addition the time of Theorem 18.11. One reads the results with `colorIndex(node)`, between 0 and 4 or 5, and with `color(node)`, a `Color` of a palette of six (red, green, blue, yellow, magenta and cyan, in the order of the indices). As maps, `colorIndexMap()` returns a const reference to the node map of the indices (the type `IndexMap`), and `colorMap()` the map of the colors (the type `ColorMap`, a `ComposeMap` of the palette and of the indices). Both `PlanarDrawing` and `PlanarColoring` also export the item and iterator types of the graph (`Node`, `Arc`, `NodeIt`, and so on).

abLEMON. `PlanarColoring::colorIndexMap()` returns a const reference to the map of the color indices; in LEMON 1.3.1 it returned a copy of it, made in $O(n)$ at each call.

The program `manual/examples/18-planar.cc` (the test `manual_18_planar`) computes the embedding of the cube and counts its faces against Euler's formula. It then draws the cube on the grid, checks that no two edges

cross, colors it with six and with five colors, and finds the Kuratowski subdivision of $K_{3,3}$, which coincides with $K_{3,3}$.

18.7 Subgraph isomorphism: VF2 and VF2++

Given two undirected graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$, with $n_1 = |V_1|$ and $n_2 = |V_2|$, a *mapping* (or embedding) is an injective function $f : V_1 \to V_2$. In the *Subgraph Isomorphism Problem* one looks for a mapping with $f(u)f(v) \in E_2$ whenever $uv \in E_1$, and in the *Induced Subgraph Isomorphism Problem* one requires, furthermore, that $f(u)f(v) \notin E_2$ whenever $uv \notin E_1$. In the *Graph Isomorphism Problem* the induced mapping must also be bijective. When the nodes carry labels $l_1 : V_1 \to L$ and $l_2 : V_2 \to L$, one also requires that $l_1(u) = l_2(f(u))$ for each $u \in V_1$. An enumeration `MappingType` (in the namespace `lemon`, from `lemon/bits/vf2_internals.h`) selects the problem, with the values `SUBGRAPH` (the default), `INDUCED` and `ISOMORPH`. According to the header, `ISOMORPH` is equivalent to `INDUCED` when the two graphs have the same number of nodes, and to `SUBGRAPH` when they also have the same number of edges, but it may be faster than both, since its cutting rules are stronger. We remark that the classes do not compare n_1 with n_2. With $n_1 < n_2$, `ISOMORPH` gives the induced mappings that preserve the degrees, i.e., the isomorphisms of G_1 onto a union of connected components of G_2 (e.g., four mappings of one edge into two disjoint edges), so that the caller who wants the isomorphisms checks `countNodes()` first. Both classes assume simple graphs: a pair of parallel edges in G_2 makes a candidate appear twice, and the same mapping is then reported twice.

Of course, the subgraph isomorphism problem is NP-complete, since it contains the clique problem. Hence, the two algorithms of this section are backtracking searches that may take exponential time, and their speed in practice depends mostly (and often decisively) on the order in which they consider the nodes of G_1 and on the rules with which they cut the search tree. We now give the search common to both, in which $\mathrm{conn}(w)$, for a node w of G_2, is -1 if w is the image of a node already mapped and otherwise the number of its neighbors that are such images.

Algorithm 18.6 (VF2 and VF2++ search, `find()`).

```
1  order the nodes of G1 as v_0, .., v_{n1-1} (VF2: breadth first from
   each unreached node; VF2++: see the text)
2  depth := 0; all the nodes of G1 unmapped
3  while depth >= 0:
4     if depth = n1: depth -= 1; return true (the mapping is in m)
5     u := v_depth; take the next candidate w for u, after the one tried
      last (whose pair is undone first): if u has a mapped neighbor, the
      neighbors w of the image of the first of them with conn(w) > 0;
      else the nodes w of G2 with conn(w) >= 0 (SUBGRAPH) or
      conn(w) = 0 (INDUCED, ISOMORPH), in the order of NodeIt
6     if there is none: unmap u; depth -= 1; continue
7     if feasible(u, w): map u to w, update conn; depth += 1
8  return false
```

The test `feasible(u, w)` checks the label (or the equivalence of the nodes), and it checks that each mapped neighbor of u has its image among the neighbors of w and, for `INDUCED` and `ISOMORPH`, that each mapped neighbor of w is such an image. It then looks ahead. With the numbers $r^{\mathrm{io}}(u)$ of the unmapped neighbors of u adjacent to the mapped part and $r^{\mathrm{new}}(u)$ of the others (precomputed for u along the order), and with the same numbers for w, read from conn, it requires $r^{\mathrm{io}}(u) \le r^{\mathrm{io}}(w)$, also $r^{\mathrm{new}}(u) \le r^{\mathrm{new}}(w)$ for `INDUCED`, and both equalities for `ISOMORPH`. We call *admissible* the mappings of the type (with the labels), where for `ISOMORPH` this means induced and preserving the degrees.

Theorem 18.15. *On simple graphs, the successive calls of `find()` return each admissible mapping exactly once, and then `false`.*

Proof. The search is a depth-first enumeration of the partial mappings of $v_0, \dots, v_{k-1}$. Step 7 maps v_k only to a w for which the partial mapping stays consistent on the mapped nodes (labels, edges, and for the induced types non-edges); at depth n_1 the mapping is therefore admissible for `SUBGRAPH` and `INDUCED`, and for `ISOMORPH` the equalities of the look-ahead give $\deg(w) = \deg(u)$ at each step. Conversely, let f be admissible and assume that its restriction to $v_0, \dots, v_{k-1}$ has been built. If v_k has a mapped neighbor x, then $f(v_k)$ is an unmapped neighbor of $f(x)$ with $\mathrm{conn} > 0$; otherwise $f(v_k)$ is unmapped and, for the induced types, adjacent to no image. In both cases it is a candidate of step 5. It passes the consistency test, and it passes the look-ahead, since f maps

the unmapped neighbors of v_k injectively into those of $f(v_k)$, keeping (for the induced types) their adjacency to the mapped part, and onto them for `ISOMORPH`, where the degrees are equal. Hence the search reaches f. Each candidate is tried once at its depth, since the candidates are scanned by an iterator whose position is kept between calls (the incident edge of step 5, stored per depth, or the node that follows the current image in the order of `NodeIt`), and since in a simple graph the neighbors of a node are distinct. A call returns at depth n_1 and the next one resumes from the last candidate of v_{n_1-1}. □

Theorem 18.16. *Let Δ_1 and Δ_2 be the largest degrees of G_1 and of G_2. A complete enumeration by `find()` examines at most $P = \sum_{k=0}^{n_1} n_2(n_2-1)\cdots(n_2-k+1)$ partial mappings and costs $O(P\, n_2(\Delta_1+\Delta_2))$, i.e., exponential in n_1 in the worst case; the constructor of `Vf2` costs $O(n_1+m_1+n_2)$, that of `Vf2pp` $O(n_1^2+m_1+n_2+K)$, with K the largest label plus one.*

Proof. The partial mappings of depth k are injections of k nodes, at most $n_2(n_2-1)\cdots(n_2-k+1)$ of them. At each of them step 5 scans at most n_2 candidates (the nodes of G_2, or the neighbors of one node). The test of step 7 and the update of conn visit the edges of u and of w a constant number of times (VF2++ also visits the lists of labels of u, which are not longer than its degree). The order of `Vf2` is one breadth-first search, the precomputed numbers take one pass over the edges, and the map conn is initialized in $O(n_2)$. The order of `Vf2pp` chooses the root of each tree by a scan of the nodes and sorts each level of the tree by selection, which costs $O(n_1^2)$ in all, while its counts of the labels are vectors indexed by the label. □

The class `Vf2< G1, G2, M, NEQ >` implements the algorithm of Cordella et al. 2004. Its first two template parameters are the type of the graph to be embedded (`G1`, by default `ListGraph`) and that of the graph it is embedded into (`G2`, by default `ListGraph`). The other two are the type of the read-write node map of G_1 that stores the mapping (`M`, by default `G1::NodeMap< G2::Node >`) and a binary functor on a node of G_1 and one of G_2 that decides whether the first may be mapped to the second (`NEQ`, by default a functor that always returns `true`). Its constructor `Vf2(g1, g2, m, neq = NEQ())` computes the order of step 1 and the numbers of the look-ahead, and sets all the values of `m` to `INVALID`. Then `mappingType()` returns the type and `mappingType(t)` sets it, and `find()` searches for the next mapping, writes it into `m` and returns `true`, or returns `false` when none is left (and then `m` is `INVALID` again everywhere). The destructor deletes the map only when a wizard allocated it (see below).

The class `Vf2pp< G1, G2, M, M1, M2 >` implements the algorithm of Jüttner and Madarasi 2018, which differs from VF2 in two respects. First, its labels are integer node maps (`M1` and `M2`, by default `G1::NodeMap< int >` and `G2::NodeMap< int >`), and there is no functor of node equivalence. The header asks the labels to be $0, 1, \ldots, K-1$ for K distinct labels, since the algorithm keeps vectors indexed by the label, of length the largest label plus one (any nonnegative values work, at the price of longer vectors). Second, the order of step 1 is chosen to "fail early". Each breadth-first tree starts from an unordered node whose label is the "rarest" among the nodes of G_2 (among those, one of largest degree). Within each level of the tree the nodes are taken by most neighbors already ordered, then by largest degree, then by rarest label; the look-ahead of `feasible()` compares the numbers r^{io} and r^{new} label by label. According to the experiments of Jüttner and Madarasi 2018, these two changes typically reduce the search considerably, especially on labeled graphs; we have not measured the two classes against each other, and on "easy" instances the difference may well be small. Its constructor is `Vf2pp(g1, g2, m, labels1, labels2)`, and `mappingType()`, `find()` and the destructor are as for `Vf2`.

Both algorithms also have a function-type interface, `vf2(g1, g2)` and `vf2pp(g1, g2)`, which return objects of the classes `Vf2Wizard` and `Vf2ppWizard` (built on the base classes `Vf2WizardBase` and `Vf2ppWizardBase`, which hold the graphs and the parameters). Their named parameters are `mapping(m)`, the map that receives the result (by default, a map is allocated and deleted inside), and `nodeLabels(l1, l2)`. The latter takes two node maps whose values `vf2()` compares with `==` and which `vf2pp()` takes as integer labels as above (without them, `vf2()` may map any node to any node, and `vf2pp()` gives all the nodes the label 0). The others are `nodeEq(f)`, the functor (for `vf2()` only), and `mappingType(t)`, `induced()` and `iso()`. Such an expression ends with `run()`, which returns whether a mapping exists and leaves it in the map given by `mapping()`, with `count()`, which returns the number of mappings, or with `getPtrToVf2Object()` (respectively `getPtrToVf2ppObject()`). This last function returns a pointer to a new algorithm object, on which one calls `find()` repeatedly and which the caller deletes; the object then deletes the maps that the wizard allocated, if any. We remark that both classes work on undirected graphs only (they iterate on the incident edges), and that the count of the mappings onto a graph with symmetries includes its automorphisms (e.g., the cycle on five nodes has 10 automorphisms). Of course, on the same input the two algorithms give the same set of mappings, possibly in another order; `Vf2` remains the one to use when a functor of node equivalence is more "natural" than integer labels.

abLEMON. `Vf2` and `Vf2pp` were added to the repository of LEMON after the release of 1.3.1 (trac #597) and were never in a release of LEMON, so that abLEMON is the first release that contains them. Furthermore,

> abLEMON reads the connection counter of the image of a neighbor only when the neighbor is mapped, where the code indexed a node map with `INVALID` (trac #672). Also, `getPtrToVf2Object()` and `getPtrToVf2ppObject()` compile, the wizards being friends of the classes whose flags of deallocation they set (trac #682); and `lemon/vf2pp.h` includes `lemon/list_graph.h`, whose `ListGraph` its default arguments name, so that each of the two headers is self-contained.

The program `manual/examples/18-isomorphism.cc` (the test `manual_18_isomorphism`) counts the automorphisms of the cycle on five nodes and the embeddings of a path of three nodes into the cycle on four nodes and into K_4, as subgraphs and as induced subgraphs (into K_4 there are 24 of the former and none of the latter, since any two nodes of K_4 are adjacent). It then repeats the counts with labels, and enumerates the mappings one by one with `find()`, by the class and through the wizard, checking each of them.

18.8 Exercises

Exercise 18.1. Add the edge 1-6 to the graph of Figure 18.1, and predict by Lemma 18.2 the blocks, the cut nodes and the bridges of the new graph. Check the prediction with the functions of §18.2, starting from the program `manual/examples/18-components.cc` (the test `test/connectivity_test.cc` checks the same functions on other graphs).

Exercise 18.2. Build the condensation of a digraph from the map of `stronglyConnectedComponents()`, with one node per component and one arc per cut arc of `stronglyConnectedCutArcs()`. Verify with `dag()` and `checkedTopologicalSort()` that it is acyclic and that the indices of the components are a topological order of it, as Theorem 18.6 states (the program `manual/examples/18-components.cc` checks the second property on the original digraph).

Exercise 18.3. A connected graph with two nodes of odd degree, s and t, has an Euler path from s to t. Obtain it with `EulerIt` by adding a new edge ts, starting the tour at s and dropping the new edge from the walk, and check the result as the program `manual/examples/18-euler.cc` checks a tour (the test `test/euler_test.cc` checks the iterators on several graphs).

Exercise 18.4. Show that the faces of an embedding of a planar graph with c connected components, counted as in the program `manual/examples/18-planar.cc` (one orbit per face of each component), satisfy $f+n-c=m+1$, and verify it on a graph with two components; the test `test/planarity_test.cc` checks this identity in `checkEmbedding()`.

Exercise 18.5. Run `PlanarEmbedding` on K_5 and on the Petersen graph, and check that the edges marked by `kuratowski()` form a subdivision of K_5 (five nodes of degree four) or of $K_{3,3}$ (six nodes of degree three), the other nodes having degree zero or two in it; the test `test/planarity_test.cc` makes this check in `checkKuratowski()`.

Exercise 18.6. Count with `vf2(g, g).iso().count()` and with `vf2pp(g, g).iso().count()` the automorphisms of the cube and of the Petersen graph (48 and 120), and compare the number of calls of `find()` with and without labels that distinguish one node. The program `manual/examples/18-isomorphism.cc` does the same on the cycle on five nodes, and the test `test/vf2_test.cc` checks the mappings found by both classes.

Exercise 18.7. Show that, for a pattern G_1 that is a complete graph, the counts of the `SUBGRAPH` and of the `INDUCED` mappings into any G_2 coincide, and that, for a pattern without edges, the `INDUCED` mappings correspond to the ordered independent sets of G_2 of that size. Check both claims on small graphs, starting from the program `manual/examples/18-isomorphism.cc`.

Exercise 18.8. Run `vf2(g1, g2).iso().count()` with G_1 a single edge and G_2 two disjoint edges, and explain the result 4 by the remark of §18.7 on the type `ISOMORPH`; then show that, when $n_1 = n_2$, the admissible mappings of `ISOMORPH` are the isomorphisms.

19. Travelling salesman and maximum clique

This chapter describes two families of algorithms for NP-hard problems, for which the library mostly offers heuristics. The first family consists of the five tour construction and improvement methods for the symmetric Travelling Salesman Problem (TSP) of `nearest_neighbor_tsp.h`, `greedy_tsp.h`, `insertion_tsp.h`, `christofides_tsp.h` and `opt2_tsp.h`, which share one interface (§19.1 to §19.4). The second consists of the two algorithms for the maximum clique problem of `grosso_locatelli_pullan_mc.h` and `tomita_seki_mc.h`, an iterated local search and an exact branch and bound (§19.5 to §19.7). None of these classes takes a tolerance or uses the named template parameters of chapter 3: the TSP classes are parameterized by the type of the cost map, and the clique classes by the type of the graph.

19.1 The symmetric travelling salesman problem

We are concerned with an undirected complete graph $K_n = (V, E)$, $|V| = n$, whose edges carry a cost $c : E \to \mathbb{R}$, and with the problem of finding a Hamiltonian cycle (a *tour*) of least cost, i.e.,

$$c^* = \min \Big\{ \sum_{i=0}^{n-1} c(v_{\pi(i)}, v_{\pi(i+1 \bmod n)}) \ : \ \pi \text{ a permutation of } \{0, \dots, n-1\} \Big\}, \tag{19.1}$$

where $v_0, \dots, v_{n-1}$ are the nodes and $c(u, v)$ is the cost of the edge $\{u, v\}$. For a set F of edges (a tour, a tree, a matching) we write $c(F) = \sum_{e \in F} c(e)$. The cost is *metric* when it satisfies the triangle inequality $c(u, w) \le c(u, v) + c(v, w)$ for all distinct u, v, w. The documentation of the group states that the TSP classes are intended for metric costs, and that on other costs they may give worse tours; this is a statement on quality, since no class checks the inequality and each of them returns a tour whatever the costs are. Of course, (19.1) is NP-hard, and without the triangle inequality it cannot be approximated within any constant factor unless P = NP. With the triangle inequality, the best guarantee among the classes of the library is the factor 3/2 of Christofides' algorithm (Theorem 19.5).

All five classes run on a `FullGraph` (chapter 4), the complete undirected graph in which `edge(u, v)` finds the edge between two nodes in constant time, and they have the same interface, which we describe once. The class template takes one parameter `CM`, the type of the cost map, i.e., a readable edge map of `FullGraph`; it is exported as `CostMap`, and its value type as `Cost`. The constructor takes the graph and the cost map and keeps references to them, which must therefore live as long as the object.

A call of `run()` computes a tour and returns its cost, which `tourCost()` gives again afterwards. Then `tourNodes()` returns a constant reference to the `std::vector< FullGraph::Node >` of the nodes in the order of the tour, `tourNodes(out)` copies this sequence through an output iterator (e.g., `nodes.begin()` of a vector of the right size, or a `std::back_inserter`), while `tour(path)` clears a path structure (chapter 8) and fills it with the arcs of the tour, the last of which goes from the last node of the sequence back to the first. The returned cost is recomputed at the end as the sum of the n edges of the sequence, and hence it agrees with `tourNodes()` up to the order of the floating-point sums. The query functions require that `run()` has been called. The degenerate graphs are handled alike by all the classes: with no node the tour is empty and costs 0, with one node it consists of that node, costs 0 and gives a path of no arc, and with two nodes it visits both, costs twice their edge and gives a path of two opposite arcs.

Two facts on metric costs are used by several proofs of this chapter. We remark that a metric cost on at least three nodes is nonnegative, since adding $c(u, v) \le c(u, w) + c(w, v)$ and $c(u, w) \le c(u, v) + c(v, w)$ gives

$0 \le 2c(v, w)$.

Lemma 19.1. *If $c \ge 0$, every tour costs at least $c(T)$, where T is a minimum cost spanning tree of K_n; in particular $c(T) \le c^*$.*

Proof. Removing one edge e from a tour H leaves a Hamiltonian path, which is a spanning tree, and hence $c(T) \le c(H) - c(e) \le c(H)$. □

Lemma 19.2. *Let c be metric, and let $W = (w_0, w_1, \dots, w_k = w_0)$ be a closed walk of K_n that visits every node. The tour that lists the nodes in the order of their first occurrence in W (a* shortcut *of W) costs at most the sum of the costs of the edges of W.*

Proof. Let $u_0, \dots, u_{n-1}$ be the nodes in the order of their first occurrence, and let j_i be the position of the first occurrence of u_i, so that $j_0 < j_1 < \cdots < j_{n-1}$; set $j_n = k$ and $u_n = u_0$. By induction on the length of the subwalk, the triangle inequality gives $c(u_i, u_{i+1}) \le \sum_{t=j_i}^{j_{i+1}-1} c(w_t, w_{t+1})$, i.e., the direct edge costs at most the subwalk from w_{j_i} to $w_{j_{i+1}}$, and the subwalks for $i = 0, \dots, n-1$ partition the edges of W. Summing over i gives the claim. □

19.2 Tour construction heuristics

We now describe the four classes that build a tour from scratch, ordered by the effort they spend. In the notation of (19.1), T is the current partial tour or path, $d(w, T) = \min_{t \in T} c(w, t)$ is the distance of a node w from it, and

$$\delta(u, v; w) = c(u, w) + c(w, v) - c(u, v) \tag{19.2}$$

is the increase of cost when w is inserted between the consecutive nodes u and v of T.

19.2.1 Nearest neighbor

Our first class, `NearestNeighborTsp< CM >`, implements the nearest neighbor heuristic in the variant that grows a path at both of its ends. It starts from an edge of minimum cost, found by `mapMin()` over all the edges; at each step it appends to the end whose nearest unvisited node is nearer (the second end on a tie), until the path contains all the nodes. The closing edge then turns the path into a tour, which `tourNodes()` lists from the first end of the path.

Algorithm 19.1 (Nearest neighbor, two-ended).

```
1  {n1, n2} := an edge of minimum cost; P := (n1, n2); e1 := e2 := INVALID
2  while |P| < n:
3      if e1 = INVALID: e1 := a cheapest edge from n1 to an unvisited node
4      if e2 = INVALID: e2 := a cheapest edge from n2 to an unvisited node
5      if c(e1) < c(e2):
6          w := the other end of e1; prepend w to P; n1 := w; e1 := INVALID
7          if w is an end of e2: e2 := INVALID
8      else:
9          w := the other end of e2; append w to P; n2 := w; e2 := INVALID
10         if w is an end of e1: e1 := INVALID
11 close P with the edge {n2, n1}
```

Theorem 19.1. *Algorithm 19.1 returns a tour, and it runs in $O(n^2)$ time.*

Proof. We show that at line 5 each valid e_i is a cheapest edge from n_i to an unvisited node. This holds when e_i has just been computed; afterwards the set of the unvisited nodes only shrinks, and a cheapest edge into a set stays cheapest into a subset that still contains its other end, which is the case unless that end has been visited, i.e., unless e_i was used (line 6 or 9) or its end was reached from the other end of P (line 7 or 10), and in both cases e_i is reset. Hence each iteration appends to P a node not yet in it, P is a simple path, and after $n-2$ iterations it contains all the nodes, so that line 11 closes a tour. Line 1 scans the $n(n-1)/2$ edges once, and each iteration scans at most two stars of $n-1$ edges; the total is $O(n^2)$. □

In practice the tours are usually "reasonable", but on special instances they may be very bad. For metric costs, Rosenkrantz, Stearns and Lewis 1977 proved that a nearest neighbor tour costs at most $(\lceil \log_2 n \rceil + 1)/2$ times c^*, and gave instances on which the ratio exceeds $\log_2(n+1)/3 + 4/9$. Their analysis, however, concerns the classical one-ended variant, which differs from the one of the library, and we do not claim it for Algorithm 19.1.

19.2.2 Greedy edge selection

Next, `GreedyTsp< CM >` builds a tour by the greedy matching heuristic. It sorts all the edges by nondecreasing cost and scans them; an edge is kept when both of its ends have degree at most one among the edges already kept and it does not close a cycle, which a union-find structure on the nodes (chapter 8) detects. After $n-1$ edges have been kept they form a Hamiltonian path, whose two ends of degree one are joined to close the tour; `tourNodes()` lists it from the node of id 0.

Algorithm 19.2 (Greedy).

```
1  sort the edges by nondecreasing cost (std::sort)
2  F := {}; each node is a singleton of the union-find structure
3  for each edge {u, v} in that order, until |F| = n - 1:
4      if deg_F(u) <= 1 and deg_F(v) <= 1 and join(u, v) merges two sets:
5          F := F + {u, v}
6  join the two nodes of degree one of F
```

Theorem 19.2. *For $n \geq 2$, the scan of Algorithm 19.2 reaches $|F| = n-1$ before the end of the list, F is then a Hamiltonian path, and the algorithm returns a tour in $O(n^2 \log n)$ time.*

Proof. By lines 4 and 5, F is at all times a forest of maximum degree two, i.e., a set of node-disjoint paths covering V (an isolated node being a path of length zero). An edge rejected at line 4 stays rejectable afterwards, since the degrees only grow and two nodes in the same set of the union-find structure stay in the same set. Suppose that $|F| < n-1$ after some edge has been scanned; then F has at least two paths, and an edge e that joins an end of one to an end of another satisfies the test of line 4. This edge has not been kept, since its ends are in different sets, and it has not been rejected, since it would still be rejectable; hence it comes later in the list, and the scan does not run past the end before $|F| = n-1$. A forest of $n-1$ edges on n nodes is a spanning tree, and with maximum degree two it is a Hamiltonian path, whose two ends (the only nodes of degree one) are joined by line 6. Line 1 sorts the $n(n-1)/2$ edges in $O(n^2 \log n)$ time, which dominates the scan, whose $O(n^2)$ operations of union-find cost $O(n^2\,\alpha(n))$ (see chapter 8). □

As for the nearest neighbor method, the tours are usually short but may be very bad in special cases; the header gives no guarantee on their cost.

19.2.3 Insertion

In `InsertionTsp< CM >` one maintains a subtour T, initially of two nodes, and inserts the other nodes one at a time, each at the position that minimizes (19.2). The node that enters next is decided by one of the four selection rules of the enumeration `SelectionRule`, passed to `run(rule)`; the default is `FARTHEST`.

Rule	Initial subtour	Node selected at each step	Time
`NEAREST`	ends of an edge of minimum cost	an unvisited w of least $d(w,T)$	$O(n^2)$
`FARTHEST` (default)	ends of an edge of maximum cost	an unvisited w of largest $d(w,T)$	$O(n^2)$
`CHEAPEST`	ends of an edge of minimum cost	the unvisited w and position of least $\delta(u,v;w)$	$O(n^3)$
`RANDOM`	ends of an edge of minimum cost	an unvisited node drawn uniformly by the global `rnd`	$O(n^2)$

Table 19.1. The selection rules of `InsertionTsp`, as `init()` and the selection functors of the header implement them.

The subtour is kept as a vector $(t_0, \ldots, t_{k-1})$, in which position p stands for the pair (t_{p-1}, t_p) of consecutive nodes and position 0 for the pair (t_{k-1}, t_0); inserting w at position p places it before t_p.

Algorithm 19.3 (Insertion).

```
1  T := the two ends of the initial edge of the rule (Table 19.1)
2  for each unvisited x: d(x) := d(x, T)                    (NEAREST, FARTHEST)
                         or (cost(x), pos(x)) := a best insertion of x into T
                                                            (CHEAPEST)
3  repeat n - 2 times:
4      select w by the rule; remove it from the unvisited nodes
5      insert w at the position p of least delta(t_{p-1}, t_p; w) of T
```

```
          (for CHEAPEST, at pos(w))
6       for each unvisited x:
          d(x) := min{ d(x), c(w, x) }                    (NEAREST, FARTHEST)
          or, for CHEAPEST, compare (cost(x), pos(x)) with the two new
          pairs around w; if neither is better and pos(x) was the pair just
          broken, search the best insertion of x again; shift pos(x) if it
          follows w
7   recompute the cost of T
```

In the code, the rules are private functors passed to `start()`: `ComparingSelection` (with `std::less` for `NEAREST` and `std::greater` for `FARTHEST`), `CheapestSelection` and `RandomSelection` select the node, and `DefaultInsertion` inserts it at its best position (line 5), while `CheapestInsertion` only adds the cost that `CheapestSelection` has already computed, the insertion being done by the latter. In line 6, a new pair is taken over the stored one also on a tie (the comparison is `<=`), and the first of the two new pairs on a tie between them.

Theorem 19.3. *For every rule, Algorithm 19.3 returns a tour. Its running time is $O(n^2)$ for `NEAREST`, `FARTHEST` and `RANDOM`, and $O(n^3)$ for `CHEAPEST`.*

Proof. Each step moves one unvisited node into the vector of T, and after $n-2$ steps T lists every node once. For `NEAREST` and `FARTHEST`, the invariant $d(x) = d(x,T)$ holds after line 2 and is kept by line 6, since $d(x, T \cup \{w\}) = \min\{d(x,T), c(w,x)\}$. For `CHEAPEST`, the invariant is that $(\mathrm{cost}(x), \mathrm{pos}(x))$ is a best insertion of x into the current T. After the insertion of w between t_{p-1} and t_p, the pairs of T are the old ones except (t_{p-1}, t_p), which is replaced by the two pairs around w. Hence, the best insertion of x is the better of the stored one and of the two new ones, unless the stored one was the broken pair, in which case it is searched again, and in all cases the positions after w shift by one. For the time, lines 4 to 6 cost $O(n)$ per step for the first three rules (one scan of the unvisited nodes, one scan of the positions, and the insertion into the vector), and line 2 costs $O(n)$; this gives $O(n^2)$. For `CHEAPEST`, line 2 costs $O(n^2)$ and line 6 costs $O(1)$ per node, or $O(n)$ when the search is repeated, i.e., $O(n^2)$ per step and $O(n^3)$ in all. □

The header remarks that `CHEAPEST`, in most cases, is almost as fast as the other rules, since the search of line 6 is repeated only for the nodes whose best pair has just been broken. As for `RANDOM`, it draws from the global generator `rnd` (chapter 8), which one may seed before `run()` to make the tour reproducible. The header describes `FARTHEST` as typically the most effective rule, and `RANDOM` as usually slightly worse but less sensitive to the instance.

For metric costs, the nearest and the cheapest insertion give tours of cost at most twice c^*; this is a result of Rosenkrantz, Stearns and Lewis 1977, which we prove for the variants of the library through a property of the cuts of K_n. For $S \subset V$, $\emptyset \neq S \neq V$, let $\mu(S)$ be the least cost of an edge with one end in S and the other in $V \setminus S$.

Lemma 19.3. *Let $S_1 \subset S_2 \subset \cdots \subset S_{n-1}$ be subsets of V with $|S_k| = k$, and let T be a minimum cost spanning tree of K_n. Then $\sum_{k=1}^{n-1} \mu(S_k) \leq c(T)$.*

Proof. Fix a real θ, let G_θ be the graph on V of the edges of cost at most θ, and let r_θ be its number of connected components. If $\mu(S_k) > \theta$, no edge of G_θ leaves S_k, and therefore S_k is a union of components of G_θ. The sets S_k with this property form a strictly increasing chain of nonempty proper subsets of V, each the union of a number of components between 1 and $r_\theta - 1$ that grows strictly along the chain, and hence there are at most $r_\theta - 1$ of them. Next, the edges of T of cost at most θ form a forest of G_θ, and we claim that it is spanning, i.e., that it has $n - r_\theta$ edges. Otherwise, a component A of this forest is a proper part of a component of G_θ, and some edge e of cost at most θ leaves A. The cycle that e closes in T leaves A through another edge f of T, which has cost larger than θ since A is a component of the forest, and $T - f + e$ would be a spanning tree cheaper than T. Thus T has exactly $r_\theta - 1$ edges of cost larger than θ, and we have shown that, for every θ, the number of the k with $\mu(S_k) > \theta$ is at most the number of the edges of T of cost larger than θ. Let $a_1 \geq \cdots \geq a_{n-1}$ be the values $\mu(S_k)$ and $b_1 \geq \cdots \geq b_{n-1}$ the costs of the edges of T, both sorted; for each j and each $\theta < a_j$, at least j values a exceed θ, hence so do at least j values b, i.e., $b_j > \theta$. This gives $b_j \geq a_j$ for every j, and the claim follows by summing. □

Theorem 19.4. *If c is metric and $n \geq 3$, the tours of `InsertionTsp` with the rules `NEAREST` and `CHEAPEST` cost at most $2c(T) \leq 2c^*$, where T is a minimum cost spanning tree.*

Proof. Let $\{n_1, n_2\}$ be the initial edge, of minimum cost, let $S_1 = \{n_1\}$, and let S_k $(2 \leq k \leq n-1)$ be the subtour with k nodes, so that the sets S_k satisfy the hypothesis of Lemma 19.3. The initial subtour costs $2c(n_1, n_2) = 2\mu(S_1)$, since the edge of minimum cost of K_n is a cheapest edge leaving $\{n_1\}$. At the step that

extends S_k, $k \geq 2$, let $\{t, x\}$ be an edge of cost $\mu(S_k)$ with $t \in S_k$, and let t' be a neighbor of t in the subtour. Inserting x between t and t' costs $\delta(t, t'; x) = c(t, x) + c(x, t') - c(t, t') \leq 2c(t, x) = 2\mu(S_k)$, by the triangle inequality $c(x, t') \leq c(x, t) + c(t, t')$. With `NEAREST` the selected node w has $d(w, S_k) = \mu(S_k)$, and the same argument applied to w and to its nearest node of S_k shows that its best insertion, which line 5 takes, costs at most $2\mu(S_k)$; with `CHEAPEST` the step takes the least insertion cost over all the unvisited nodes and positions (Theorem 19.3), which is at most $\delta(t, t'; x)$. In both cases the increase is at most $2\mu(S_k)$, and the cost of the tour is at most $2\sum_{k=1}^{n-1} \mu(S_k) \leq 2c(T)$ by Lemma 19.3, while $c(T) \leq c^*$ by Lemma 19.1, the costs being nonnegative. □

The proof does not apply to `FARTHEST` and `RANDOM`, for which the inserted node is not near the subtour. For an arbitrary order of insertion only a factor logarithmic in n is known (Rosenkrantz, Stearns and Lewis 1977), although in practice the farthest insertion is often (but not always) the best of the four rules.

19.2.4 Christofides' algorithm

Finally, `ChristofidesTsp< CM >` implements the heuristic of Christofides 1976. It combines a minimum cost spanning tree with a minimum cost perfect matching on the nodes of odd degree of the tree (Figure 19.1), and it is the only class of the family with a guaranteed approximation factor.

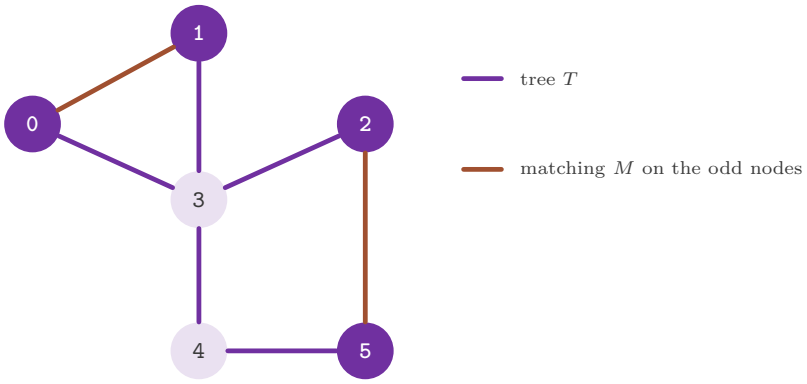


Figure 19.1. Christofides' algorithm: the minimum spanning tree T (primary), its nodes of odd degree (primary), and the minimum cost perfect matching M on them (the added colour). Each node has even degree in $T \cup M$, whose Euler tour, without the nodes already visited, is the tour.

Algorithm 19.4 (Christofides).

```
1  T := minimum cost spanning tree of K_n                    (kruskal)
2  O := nodes of odd degree in T
3  M := minimum cost perfect matching of the complete graph on O
       (MaxWeightedPerfectMatching on the costs -c, on a SmartGraph)
4  H := the multigraph T + M on a SmartGraph
5  walk the Euler tour of H (EulerIt); append each node when first reached
```

In the code, the tree is computed by `kruskal()` (chapter 11), and the closed walk by `EulerIt` (chapter 18). The matching is computed by `MaxWeightedPerfectMatching` (chapter 17) on a `SmartGraph` with one node per node of O and with the negated costs as weights; for this reason, the header warns that `CM::Value` must be a signed number type. This graph has two parallel edges for each pair of nodes of O, one per order of the pair, which doubles its edges but does not change the minimum cost of a perfect matching. Line 5 skips the nodes already visited, i.e., it shortcuts the closed walk as in Lemma 19.2.

Lemma 19.4. *For $n \geq 3$, $|O|$ is even and positive, the multigraph H of line 4 is connected with all degrees even, and line 5 produces a tour.*

Proof. The sum of the degrees of T is $2(n-1)$, and hence the number of the nodes of odd degree is even; it is positive, since a tree with at least two nodes has leaves. Then K_n restricted to O is complete with an even number of nodes and has a perfect matching M, each node of O gains one edge of M, and so every degree of $H = T + M$ is even; H is connected since it contains T. By Euler's theorem (chapter 18), H has a closed walk that uses each of its edges once, and this walk visits every node; listing the nodes at their first occurrence gives each node once, i.e., a tour. □

Theorem 19.5. *If c is metric and $n \geq 3$, the tour returned by Algorithm 19.4 costs at most $c(T) + c(M) \leq \frac{3}{2}c^*$.*

Proof. By Lemma 19.2, the tour costs at most the closed walk of line 5, i.e., $c(T) + c(M)$. By Lemma 19.1, $c(T) \leq c^*$. Let H^* be an optimal tour, and let C_O be the cycle on O obtained from H^* by the shortcut of Lemma 19.2 (the nodes of O in the order of H^*), so that $c(C_O) \leq c^*$. Since $|O|$ is even, the edges of C_O taken alternately form two perfect matchings M_1 and M_2 of O (equal to each other when $|O| = 2$), with $c(M_1) + c(M_2) = c(C_O) \leq c^*$, and therefore $c(M) \leq \min\{c(M_1), c(M_2)\} \leq c^*/2$. □

Lemma 19.5. *If c is metric and $n \geq 3$, then $c(M) \leq c(T)$, and hence the tour of Algorithm 19.4 costs at most $2c(T)$.*

Proof. We show that T contains $|O|/2$ edge-disjoint paths whose ends pair up the nodes of O. Start from a node a of odd degree in the current set of edges, initially T, and walk along edges not yet used; since T has no cycle, the walk is a path, and it can only stop at a node whose last edge it has just used, i.e., a node $b \neq a$ of odd degree. Removing the edges of the path leaves the parity of the inner nodes unchanged and makes a and b even; repeating, one pairs all the nodes of O. By the triangle inequality, the edge between the ends of each path costs at most the path, and these $|O|/2$ edges form a perfect matching of O of cost at most $c(T)$, since the paths are edge-disjoint and the costs are nonnegative. As M is a minimum cost perfect matching, $c(M) \leq c(T)$, and the second claim follows from the first part of the proof of Theorem 19.5. □

Theorem 19.6. *Algorithm 19.4 runs in $O(n^3 \log n)$ time, the bound of the header.*

Proof. Line 1 runs Kruskal's algorithm on the $n(n-1)/2$ edges in $O(n^2 \log n)$ time. Line 3 builds a graph with $|O| \leq n$ nodes and $|O|(|O|-1)$ edges and runs on it the weighted perfect matching algorithm of chapter 17, whose $O(nm \log n)$ bound gives $O(n^3 \log n)$. Lines 4 and 5 handle a multigraph of at most $n - 1 + n/2$ edges in linear time. □

In practice, the tours are usually much better than the bound of Theorem 19.5: in the example of §19.4 the tour costs less than 1.3 times the spanning tree.

19.3 Improvement by 2-opt

We now turn to `Opt2Tsp< CM >`, which implements the 2-opt local search of Croes 1958. Given a tour, it looks for two of its edges $\{u, u^+\}$ and $\{v, v^+\}$ that share no node, with u^+ and v^+ the successors of u and v, such that

$$c(u, u^+) + c(v, v^+) > c(u, v) + c(u^+, v^+), \tag{19.3}$$

and replaces them by $\{u, v\}$ and $\{u^+, v^+\}$ (a *2-opt move*), which reverses the path from u^+ to v (Figure 19.2). The search stops when no pair satisfies (19.3), i.e., at a *2-optimal* tour.

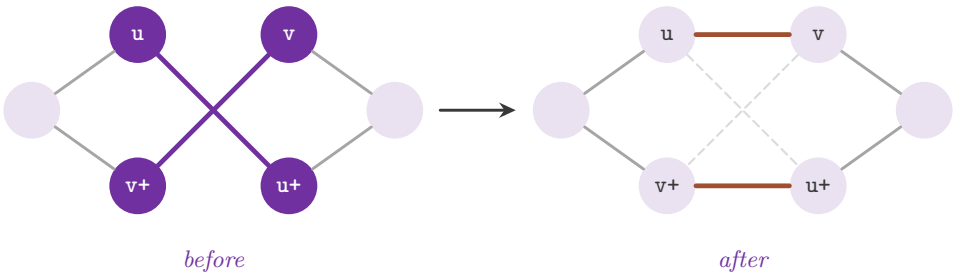


Figure 19.2. A 2-opt move: the two edges of the tour that cross (primary) are replaced by the two that do not (the added colour), and the path between u^+ and v is traversed in the opposite direction.

There are three ways to give the starting tour. A call of `run()` starts from the order of the node ids; `run(path)` starts from a path structure, which must be a valid path of n arcs (cf. `checkPath()` in chapter 8); and `run(nodes)` starts from a `std::vector< Node >` that holds all the nodes in the order of the tour. The typical use is the improvement of the tour of a construction heuristic, e.g., by passing to `run()` the `tourNodes()` of an `InsertionTsp`. The tour is kept as a list of two neighbors per node, without an orientation, which the private iterator `PathListIt` follows by remembering the node it comes from; hence, a move rewires four entries and costs $O(1)$, with no explicit reversal of a path.

Lemma 19.6. *A 2-opt move turns a tour into a tour, whose cost is the old one minus the difference between the two sides of (19.3).*

Proof. Removing the two edges, which share no node, splits the tour into the path from u^+ to v and the path from v^+ to u, which together cover every node. The edge $\{u, v\}$ joins the end v of the first to the end u of the second, and $\{u^+, v^+\}$ joins their other ends, which closes a single cycle through all the nodes. The cost changes by $c(u, v) + c(u^+, v^+) - c(u, u^+) - c(v, v^+)$. □

Algorithm 19.5 (2-opt, first improvement).

```
1  restart:
2  for u := node 0, then along the tour, while u+ and u++ are not node 0:
3      for v := u++, then along the tour, while v is not node 0 and not
               the predecessor of u:
4          if (19.3) holds for (u, u+) and (v, v+):
5              replace {u,u+}, {v,v+} by {u,v}, {u+,v+}   (checkOpt2())
6              goto restart
7  list the tour from node 0 and recompute its cost
```

Theorem 19.7. *Algorithm 19.5 terminates with a 2-optimal tour. A pass that finds no move evaluates (19.3) on each of the $n(n-3)/2$ pairs of edges of the tour that share no node, and the whole run costs $O(n^2(k+1))$ time, where k is the number of moves; for integer costs, $k \le c(H_0) - c^*$, where H_0 is the starting tour.*

Proof. Let the tour be $p_0 = 0, p_1, \dots, p_{n-1}$ in the order of the iterators, and let the edge a be $\{p_a, p_{a+1}\}$ (indices modulo n). Line 2 takes $u = p_a$ for $a = 0, \dots, n-3$, and line 3 takes $v = p_b$ for $b = a+2, \dots, n-1$, except $b = n-1$ when $a = 0$, since then $v^+ = p_0 = u$. These are exactly the pairs $a < b$ of edges that share no node, $n(n-3)/2$ in all. Hence a pass ends at line 7 only if no such pair satisfies (19.3), i.e., the tour is 2-optimal. By Lemma 19.6 each move gives a tour and, by (19.3), decreases its cost strictly. For a floating-point `Cost`, the comparison is made on the two computed sums, and since rounding is monotone, a computed sum on the left larger than the one on the right implies the same inequality between the exact sums, so that the exact cost of the tour (the sum of the stored costs of its edges) still decreases strictly. No tour can therefore occur twice, and the number of tours is finite; hence the algorithm terminates. Each pass before a move costs at most the $O(n^2)$ of a full pass, and so does the last one. For integer costs each move decreases the cost by at least 1, and no tour costs less than c^*. □

The number of moves is not polynomially bounded in general, even for points of the plane (Englert, Röglin and Vöcking 2014), and the header describes the method as rather slow but effective. Note that (19.3) is a strict comparison without tolerance; for floating-point costs, a move whose gain is below the rounding of the four costs may or may not be taken. For points of the plane, 2-optimality has a geometric meaning, which explains the quality of the tours in practice and gives the check of the example of §19.4.

Lemma 19.7. *Let the nodes be points of the plane, no three of them collinear, with the Euclidean distances as costs. (i) If the segments of two edges $\{u, u^+\}$ and $\{v, v^+\}$ of a tour cross, they satisfy (19.3), and therefore a 2-optimal tour has no crossing edges. (ii) If the points are in convex position, the only tour without crossing edges is the boundary of their convex hull, which is therefore both the optimal tour and the tour returned by `Opt2Tsp`, whatever its starting tour.*

Proof. (i) Let the two segments cross at the point x. By the triangle inequality of the plane, $c(u, v) \le |ux| + |xv|$ and $c(u^+, v^+) \le |u^+x| + |xv^+|$, and both inequalities are strict, since x lies on the segment from u to v only if u, x, v, and hence u, u^+, v, are collinear. Summing, and since $|ux| + |xu^+| = c(u, u^+)$ and $|vx| + |xv^+| = c(v, v^+)$, one obtains (19.3). (ii) Let $\{p, q\}$ be an edge of a tour that is not a side of the convex hull. The line through p and q then leaves points on both of its sides, and the rest of the tour, a path from q to p through all the other points, has an edge $\{a, b\}$ with a and b on opposite sides. In convex position, the segment ab meets the line inside the convex hull, whose intersection with the line is the segment pq; hence $\{a, b\}$ crosses $\{p, q\}$. A tour without crossings therefore uses only sides of the hull, i.e., it is its boundary. An optimal tour is 2-optimal, since a move would decrease its cost, and so is the tour of `Opt2Tsp` (Theorem 19.7); by (i) neither has crossings, and both are the boundary. □

19.4 Choosing a heuristic, and an example

We summarize the trade-off as the documentation of the group does. The fastest methods are `NearestNeighborTsp`, `GreedyTsp` and `InsertionTsp`, and the insertion is usually quite effective; `ChristofidesTsp` is somewhat

slower, but it has the best guarantee. Finally, `Opt2Tsp` is the slowest, it usually gives the best tours in practice, and it can improve the tour of any other method.

On random Euclidean instances, the farthest insertion followed by 2-opt is a sensible default. The run of the program described below on 40 points of the unit square illustrates this (on one instance, and therefore only as an illustration). The nearest neighbor and the greedy tours cost 5.58 and 5.33, the four insertion rules give 5.37 (nearest), 4.76 (farthest), 5.20 (cheapest) and 4.93 (random), and Christofides' tour costs 4.90, against 3.81 for the spanning tree. Also, 2-opt brings the farthest insertion tour to 4.76, and the other three to between 4.83 and 5.21.

The program `manual/examples/19-tsp.cc` (the test `manual_19_tsp`) checks these properties on two instances. On twelve points of a circle, numbered out of their angular order, it checks that every tour costs at least the perimeter of the regular polygon. It also checks that Christofides' tour costs at most 3/2 of it (Theorem 19.5), and that the 2-opt tour from the order of the ids costs the perimeter (Lemma 19.7). On the 40 random points, it checks that each tour visits each node once and costs the returned value, that it costs at least the minimum spanning tree (Lemma 19.1), that Christofides' tour costs at most twice the tree (Lemma 19.5), and that 2-opt does not worsen the tour it starts from.

19.5 The maximum clique problem

Given an undirected graph $G = (V, E)$, $n = |V|$, a *clique* is a set $C \subseteq V$ of pairwise adjacent nodes. In the maximum clique problem one looks for a clique of largest cardinality $\omega(G)$, i.e.,

$$\omega(G) = \max\{|C| \ : \ C \subseteq V, \ \{u, v\} \in E \text{ for all distinct } u, v \in C\}. \tag{19.4}$$

Since the problem is NP-hard and hard to approximate, the library offers two algorithms with different aims. `GrossoLocatelliPullanMc` is a heuristic that quickly finds a "large" clique, without a proof of its maximality. `TomitaSekiMc` is an exact branch and bound that proves the clique it finds maximum, in a time that may grow exponentially with the size of the graph, unless a limit on its search tree stops it first. Both copy the graph into an $n \times n$ adjacency matrix of `char`, indexed by the order of the node iterator, which costs $O(n^2)$ memory and $O(n^2 + m)$ time before the search starts, m being the number of edges. Both also ignore loops (a loop does not make a node adjacent to itself) and parallel edges.

> **abLEMON.** `TomitaSekiMc` is new in abLEMON (version 1.5.0). In LEMON 1.3.1, `GrossoLocatelliPullanMc` ran forever on a graph with a loop, which made a node adjacent to itself and kept it among the candidates of the clique it already belonged to; abLEMON ignores loops when it builds the adjacency matrix.

19.6 The iterated local search of Grosso, Locatelli and Pullan

Our heuristic class, `GrossoLocatelliPullanMc< GR >`, implements the iterated local search of Grosso, Locatelli and Pullan 2008 on an undirected graph of type `GR`. It keeps a current clique C and, for each node w, the number $\delta(w)$ of nodes of C not adjacent to w, where a node of C counts itself (the matrix has no loops). Thus, $\delta(w) = 0$ holds for the nodes that can be added to C, and $\delta(w) = 1$, with $w \notin C$, for the nodes that can replace one node of C. An *iteration* perturbs C and improves it by a local search with a tabu set; the largest clique found over all the iterations is the result.

Algorithm 19.6 (One iteration of `start()`).

```
1  restart := restart + 1
   R := { w : delta(w) >= 4 }; r := uniform in R, or in V if R is empty
2  remove from C the nodes not adjacent to r; add r to C          (restart)
3  tabu := {}; max_swap := |C|
4  while select < max_select:
5      select := select + 1
6      if a non-tabu w has delta(w) = 0: add w to C
           (and max_swap := |C| while tabu is still empty)
7      elif a non-tabu w not in C has delta(w) = 1:
           add w, remove the one node v of C not adjacent to w, tabu += v
           max_swap := max_swap - 1; leave the loop if max_swap <= 0
8      elif some w has delta(w) = 0 (tabu allowed): add w to C
```

```
9      else leave the loop
10 if |C| > |best|: best := C; return SIZE_LIMIT if |best| >= max_size
11 update the penalties of the rule
```

The iterations are repeated while `select < max_select` and `restart < max_restart`, where `max_restart`, `max_select` and `max_size` are the three limits described below; at the end, `run()` returns `ITERATION_LIMIT` if `restart` has reached its limit, and `STEP_LIMIT` otherwise. The 4 of line 1 is a constant of the class (the restart based on δ is always on), and the update of δ after each addition or removal is one pass over a row of the matrix, in `addCliqueNode()` and `delCliqueNode()`.

Lemma 19.8. *At all times, C and `best` are cliques of G and $\delta(w) = |\{x \in C : x = w \text{ or } \{x, w\} \notin E\}|$ for every node w.*

Proof. Initially C and `best` are empty and $\delta = 0$; every change of C is made by `addCliqueNode()` and `delCliqueNode()`, which increase or decrease by one the δ of the nodes not adjacent to the node added or removed, the node itself included, and this keeps the formula. Line 2 leaves in C only nodes adjacent to r and then adds r, so that C stays a clique. Lines 6 and 8 add a node w with $\delta(w) = 0$, i.e., $w \notin C$ and adjacent to every node of C. At line 7, $w \notin C$ and $\delta(w) = 1$, so that exactly one node v of C is not adjacent to w, and $C - v + w$ is a clique. Finally, `best` is a copy of C. □

Theorem 19.8. *Each iteration of Algorithm 19.6 performs at most $2n$ steps, i.e., passes through line 5, and costs $O(n^2)$ time; hence `run()` performs at most $\min\{I, S\}$ iterations, where I is the iteration limit and S the step limit, and costs $O(n^2 + m + n^2 \min\{I, S\})$ time.*

Proof. Each step but the last of the iteration performs a move. An addition (lines 6 and 8) increases $|C|$, and a swap (line 7) keeps it, so that there are at most $n - 1$ additions after the restart, which leaves $|C| \geq 1$. Before the first swap, `max_swap` is at most $|C| \leq n$, and after it the tabu set is no longer empty and `max_swap` is never reset; since each swap decreases it and the loop is left when it reaches 0, there are at most n swaps. Adding the last step, which leaves the loop or reaches the step limit, an iteration has at most $2n$ steps. Each step costs $O(n)$, i.e., a scan of the nodes by the rule and one or two passes over a row of the matrix, and so do lines 1, 2, 10 and 11. Each iteration increases `restart` by one and `select` by at least one, which gives the bound on the iterations, and the initialization of `init()` costs $O(n^2 + m)$. □

Thus every run ends, although with both limits disabled the bound of Theorem 19.8 is the largest `int`, and the header warns that one should set a reasonable limit on the iterations or on the steps. The node selected among the eligible ones at lines 6, 7 and 8 depends on the rule passed to `run(rule)`, of the enumeration `SelectionRule`, and each selection scans the nodes in cyclic order from a starting index drawn uniformly at random. With `RANDOM` it is the first eligible node in this order; with `DEGREE_BASED` it is an eligible node of maximum degree, ties going to the first one in the order; with `PENALTY_BASED`, the default, it is an eligible node of minimum penalty, ties going to the first one, as in Grosso, Locatelli and Pullan 2008. The three rules are the private classes `RandomSelectionRule`, `DegreeBasedSelectionRule` and `PenaltyBasedSelectionRule`, of which `start()` is a template, and each of them provides the three selections and the `update()` of line 11, empty for the first two.

The penalties start at zero at each `run()` and are updated at line 11 as in DLS-MC (Pullan and Hoos 2006). Those of the nodes of the current clique increase by one, and every p_d iterations all the positive ones decrease by one, where the penalty delay p_d is set by `penaltyDelay(pd)` (2 by default, and a value smaller than 1 is taken as 1). Larger values keep the nodes of the recent cliques out of the next ones for longer, and the best value depends on the graph (DLS-MC uses values from 1 to about 50). The smallest value makes the rule degenerate, as follows.

Lemma 19.9. *With $p_d = 1$, all the penalties stay at zero, and `run(PENALTY_BASED)` and `run(RANDOM)`, started with generators in the same state, give the same clique and the same termination cause.*

Proof. With $p_d = 1$, every update first increases the penalties of the nodes of C and then decreases every positive penalty; by induction on the updates, the penalties are zero before each update, and hence also after it. With all the penalties equal, the strict comparison of the rule selects the first eligible node in the cyclic order, which is the node that `RANDOM` selects. Both rules draw one random starting index per selection, in the same sequence of calls, and the restarts draw from the same generator. By induction on the steps, the two runs pass through the same states. □

abLEMON. In LEMON 1.3.1 the penalties were never updated (the function `update()` of the rule was empty), and `PENALTY_BASED`, the default rule, selected the same node as `RANDOM`; `penaltyDelay()` is new. Also, the constructor without a seed now uses the global generator `rnd`, as its documentation says, where it copied it, so that two objects built one after the other drew the same sequence.

Three limits control the search; each is set by a function that returns a reference to the object, and read by the function of the same name without argument. The function `iterationLimit(int)` sets the number of iterations `max_restart`, 1000 by default; `stepLimit(int)` sets the number of steps `max_select` over all the iterations, by default -1; and `sizeLimit(int)` sets the clique size `max_size` at which the search stops, by default -1, which stands for the number of nodes. A negative iteration or step limit means no limit (the largest `int`), and with a limit of zero no iteration is made, so that the clique is empty and the cause is `ITERATION_LIMIT` (or `STEP_LIMIT` for a step limit of zero). The size limit is tested only when an iteration has found a larger clique (line 10), and therefore a size limit of 0 stops the search after the first iteration. The function `run(rule)` returns the cause of the termination, of the enumeration `TerminationCause`: `ITERATION_LIMIT`, `STEP_LIMIT` or `SIZE_LIMIT`. The last one is also the answer on a graph with no node (clique of size 0) and on a graph with one node (clique of size 1).

The random choices use a generator of type `Random` (chapter 8), which depends on the constructor. `GrossoLocatelliPullanMc(graph)` draws from the global generator `rnd`, which it advances, so that two objects built one after the other draw different sequences; `GrossoLocatelliPullanMc(graph, seed)` draws from a generator of its own seeded with `seed`, and `GrossoLocatelliPullanMc(graph, random)` from a copy of the given one. Since the object may refer to a generator it owns, it cannot be copied (its copy constructor and assignment are private). The results are read by `cliqueSize()` and by `cliqueMap(map)`, which writes `true` on the nodes of the clique and `false` elsewhere, in a writable node map with `bool` (or convertible) values. Also, the iterator `CliqueNodeIt`, constructed from the algorithm object, lists the nodes of the clique in the order of the node iterator; it converts to `Node`, compares with `INVALID`, and its postfix increment returns a `Node`, not an iterator.

19.7 The exact algorithm of Tomita and Seki

For the exact solution, `TomitaSekiMc< GR >` implements the branch and bound MCQ of Tomita and Seki 2003 on an undirected graph of type `GR`, exported as `Graph`. A node of its search tree is a clique C together with the set P of the candidates, i.e., nodes adjacent to all the nodes of C. The bound that prunes the tree is the number of colors of a proper coloring of the subgraph induced by P, which no clique of P can exceed.

Lemma 19.10. *If the nodes of P are partitioned into k independent sets, a clique contained in P has at most k nodes; hence, no clique that extends C by nodes of P is larger than $|C| + k$.*

Proof. Two nodes of a clique are adjacent, and two nodes of an independent set are not; a clique therefore has at most one node in each of the k sets. □

In `colorSort()`, the coloring is computed greedily: each candidate, in the order of P, goes into the first color class that contains none of its neighbors, or into a new class. The candidates are then listed by color, in the vector `order`, and `color[k]` is the number of the class of `order[k]`.

Lemma 19.11. *The classes of `colorSort()` are independent sets, and the prefix `order[0..k]` is contained in the first `color[k]` classes; hence, by Lemma 19.10, no clique extending C by nodes of `order[0..k]` has more than $|C|+$`color[k]` nodes. The function costs $O(|P|^2)$ time.*

Proof. A node enters a class only if it has no neighbor there, and the classes are disjoint, so that each is an independent set. Since `order` lists the classes in increasing order and `color[k]` is the class of `order[k]`, every node of the prefix belongs to a class of number at most `color[k]`. Each candidate is compared with at most all the nodes already placed, i.e., with $O(|P|)$ nodes. □

Algorithm 19.7 (Tomita and Seki, `expand(P)`).

```
1  order, color := colorSort(P)
2  for k from |P| - 1 down to 0:
3      if |C| + color[k] <= |best|: return                    (Lemma 19.11)
4      if the tree limit is reached: stopped := true; return
5      count a node of the search tree; v := order[k]; C := C + v
6      P' := { order[j] : j < k, order[j] adjacent to v }
7      if P' is empty: best := C if |C| > |best|
8      else expand(P')
9      C := C - v; return if stopped
```

At the root, C is empty and the candidates are all the nodes, sorted by nonincreasing degree as Tomita and Seki order them; the degree counts distinct neighbors, the sort is stable, and ties keep the order of the node iterator. The function `treeLimit(long long)` sets the largest number of nodes of the search tree, i.e., of the

cliques that line 5 extends, after which `run()` stops; a negative value, the default (-1), means no limit, and a limit of 0 stops the search before its first node.

Theorem 19.9. *With no tree limit, `run()` returns `true` and `cliqueSize()` is $\omega(G)$. With a tree limit, the clique found is a clique of G, and if `run()` returns `true` it is maximum.*

Proof. By induction on line 5, C is a clique and every node of P is adjacent to all of C, since P' keeps the candidates adjacent to v; hence `best`, a copy of C, is a clique. We show by induction on $|P|$ that, if `expand(P)` is not stopped, then on return `best` is at least as large as every clique K with $K \supseteq C$ and $K \setminus C \subseteq P$. Let $v =$`order[k]` be the node of $K \setminus C$ of largest index (if $K = C$, the clique $C + v$ of any $v \in P$ is larger and is covered by what follows, since `expand()` is called only with P not empty). The loop stops before reaching k only at line 3 for some $k' \geq k$, where $|K| \leq |C|+$`color[k']`$\leq |$`best`$|$ by Lemma 19.11, since $K \setminus C$ lies in `order[0..k']`. Otherwise the branch of v is entered, $K \setminus (C \cup \{v\}) \subseteq P'$, and either P' is empty, in which case $K = C + v$ is recorded at line 7, or `expand(P')` covers K by the induction hypothesis. Applying the claim at the root, with $C = \emptyset$ and $P = V$, gives `cliqueSize()` $= \omega(G)$ whenever the search is never stopped, and `run()` returns `true` exactly in this case. □

Theorem 19.10. *The nodes of the search tree are distinct cliques of G, and hence their number `treeSize()` is at most the number of nonempty cliques of G, less than 2^n. The run costs $O(n^2 + m)$ time before the search and $O(n^2)$ time per node of the tree, with $O(n^2)$ memory.*

Proof. A node of the tree is the clique $C + v$ of line 5. Two nodes with the same clique K would branch on its last node in the order of the parent call, which shows by induction that they have the same parent call and the same v, i.e., that they are the same node. Line 1 costs $O(|P|^2) = O(n^2)$ by Lemma 19.11, and line 6 costs $O(|P|)$ per iteration of the loop, i.e., $O(n^2)$ per call. The matrix takes $O(n^2)$ memory, and the recursion, of depth at most $\omega(G)$, keeps $O(n)$ integers per level. □

In the worst case the time is exponential, as for any exact algorithm for (19.4) known so far, and the size of the tree depends strongly on the density of the graph. The function `run()` returns `true` when the clique found is proved maximum, and `false` when the limit stopped the search first; in this case the clique is the largest one found so far. The results are read by `cliqueSize()`, by `treeSize()` (the number of nodes of the search tree of the last run) and by `cliqueMap(map)`, with the same contract as in §19.6. On sparse graphs the coloring bound prunes almost all the branches: in the program described below, a planted clique of 9 nodes in a random graph of 60 nodes and density 0.2 is proved maximum with a search tree of 9 nodes. On dense random graphs of a few hundred nodes, instead, the tree may already be very large.

The program `manual/examples/19-clique.cc` (the test `manual_19_clique`) compares the exact algorithm with the enumeration of all the subsets on five random graphs of 14 nodes. On the graph with the planted clique, it checks that the heuristic, seeded and stopped at the maximum size, finds a clique no larger than the maximum one, and that the rules `RANDOM` and `PENALTY_BASED` with the same seed give the same clique when the penalty delay is 1 (Lemma 19.9); finally, it stops an exact search after 5 nodes of its tree.

19.8 Exercises

Exercise 19.1. Check on random Euclidean instances that each tour returned by the five TSP classes costs at least the minimum spanning tree (Lemma 19.1), and that the tour of `ChristofidesTsp` costs at most twice it (Lemma 19.5); `manual/examples/19-tsp.cc` does it for 40 points.

Exercise 19.2. Give five points of the plane, not in convex position, and a 2-optimal tour on them that is not optimal; deduce that Lemma 19.7 (ii) needs the convex position, and check it with `Opt2Tsp::run(nodes)` started from that tour (`manual/examples/19-tsp.cc` checks the convex case on twelve points of a circle).

Exercise 19.3. Build a metric instance on which `NearestNeighborTsp` returns a tour much more expensive than `InsertionTsp` with the `FARTHEST` rule, e.g., points on a line with exponentially growing gaps, and measure the ratio as n grows.

Exercise 19.4. Give a non-metric instance on which the shortcuts of Algorithm 19.4 increase the cost of the Euler tour, and compare the resulting tour with the bound of Theorem 19.5 (`test/tsp_test.cc` checks only metric instances).

Exercise 19.5. Count the evaluations of (19.3) made by `Opt2Tsp` from the order of the node ids and from the tour of `InsertionTsp`, on random Euclidean instances of growing size, and relate them to the $n(n-3)/2$ evaluations of a pass (Theorem 19.7).

Exercise 19.6. Verify on random graphs that `TomitaSekiMc` finds a clique of the size given by the enumeration of all the subsets, and that a tree limit of 1 stops the search after one node (`test/tomita_seki_mc_test.cc` and `manual/examples/19-clique.cc`).

Exercise 19.7. Run `GrossoLocatelliPullanMc` with the rules `RANDOM` and `PENALTY_BASED` and the same seed on the same graph, observe that they return the same clique when the penalty delay is 1 (Lemma 19.9); then compare the sizes of the cliques found by `PENALTY_BASED` with penalty delays from 2 to 50 (`manual/examples/19-clique.cc` checks the first part; `test/max_clique_test.cc` checks the termination causes of the three rules).

Exercise 19.8. Plot the size of the search tree of `TomitaSekiMc` (`treeSize()`) on random graphs of 100 nodes as the density grows from 0.1 to 0.9, and find where the coloring bound stops being effective.

20. The LP and MIP interface

This chapter describes the interface through which a program builds a Linear Program (LP) or a Mixed Integer Program (MIP) and has it solved by an external solver. The interface is declared in `lp_base.h`, the default solvers are chosen in `lp.h`, the skeletons of `lp_skeleton.h` do nothing but compile, and `glpk.h`, `cplex.h`, `clp.h`, `cbc.h` and `soplex.h` connect the interface to GLPK, CPLEX, Clp, Cbc and SoPlex. Unlike the rest of the library, these classes are not templates: they are compiled into the library, each backend only when its solver is found at configuration time (chapter 2). A program written against the base classes compiles with any backend; it also compiles with none, although then only the skeletons are available.

20.1 The structure of the interface

We are concerned with problems of the form

$$\min \text{ (or } \max) \;\; c^\top x + c_0 \quad \text{s.t.} \quad l^r \le Ax \le u^r, \quad l^c \le x \le u^c, \quad x_j \in \mathbb{Z} \;\; (j \in J), \tag{20.1}$$

where $x \in \mathbb{R}^n$ are the columns, A is an $m \times n$ matrix whose row i has the bounds $l^r_i \in \mathbb{R} \cup \{-\infty\}$ and $u^r_i \in \mathbb{R} \cup \{+\infty\}$, and each column j has the bounds l^c_j and u^c_j alike. Also, $c \in \mathbb{R}^n$ is the vector of the objective coefficients, c_0 is a constant, and J is the set of the integer columns, which is empty for an LP. A row with $l^r_i = u^r_i$ is an equation, and one with both bounds finite is a *range* constraint. The interface describes (20.1) with three classes (Figure 20.1): `LpBase` holds the model, i.e., the columns, the rows, the coefficients, the bounds and the objective; `LpSolver` adds the solution of an LP, with its dual values and its basis; `MipSolver` adds the integrality of the columns and the solution of a MIP. Both `LpSolver` and `MipSolver` derive virtually from `LpBase`, and a backend derives from one of them. Hence, a function that takes an `LpSolver &` works with any LP backend, and one that takes an `LpBase &` builds a model for both kinds of solvers.

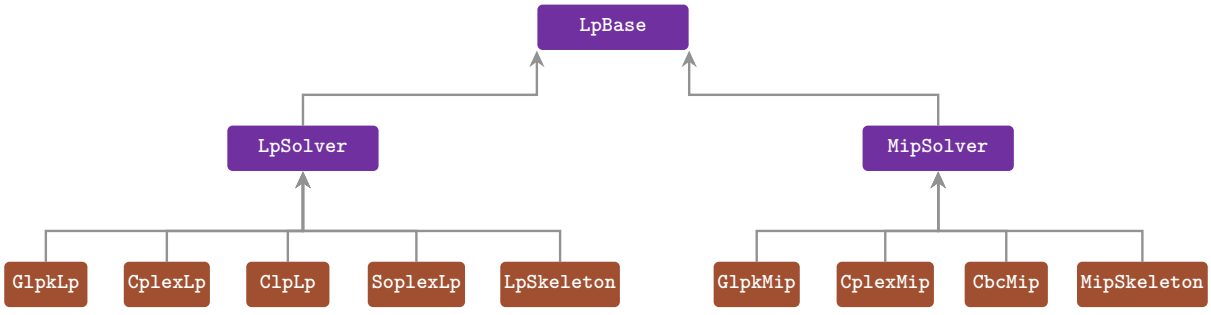


Figure 20.1. The classes of the interface: the model is held by `LpBase` (primary), the solving and the solution by `LpSolver` and `MipSolver` (primary), and each backend (the added colour) derives from one of the two. The common bases `GlpkBase`, `CplexBase` and `SkeletonSolverBase` are not drawn.

All the numbers of the interface are of the type `LpBase::Value`, which is `double`. The constants `LpBase::INF` and `LpBase::NaN` are the infinity and the quiet NaN of `double`; an infinite bound is written as `INF` or `-INF`, and NaN marks an unset bound of a constraint (§20.2). The enumerations of `LpBase` are `Sense` (`MIN`, `MAX`), `MessageLevel` (`MESSAGE_NOTHING`, `MESSAGE_ERROR`, `MESSAGE_WARNING`, `MESSAGE_NORMAL`, `MESSAGE_VERBOSE`) and `SolveExitStatus`. The last is the value returned by `solve()`: it is `SOLVED` (0) when the solver has found an optimal solution or proved the problem infeasible or unbounded, and `UNSOLVED` (1) in any other case, e.g., when

a limit of the solver stopped it. Whether a solved problem is optimal, infeasible or unbounded is then read from the *problem type* (§20.4 and §20.5).

Then `lp.h` defines the default solvers. When at least one backend is found at configuration time, the macro `LEMON_HAVE_LP` (and, for GLPK, CPLEX and Cbc, `LEMON_HAVE_MIP`) is defined in `lemon/config.h`, together with `LEMON_HAVE_GLPK`, `LEMON_HAVE_CPLEX`, `LEMON_HAVE_CLP`, `LEMON_HAVE_CBC` and `LEMON_HAVE_SOPLEX` for the backends found. In this case `Lp` is the default LP solver, and `Mip` the default MIP solver, chosen by the macros `LEMON_DEFAULT_LP` (one of `LEMON_GLPK_`, `LEMON_CPLEX_`, `LEMON_SOPLEX_`, `LEMON_CLP_`) and `LEMON_DEFAULT_MIP` (one of `LEMON_GLPK_`, `LEMON_CPLEX_`, `LEMON_CBC_`), which `lp.h` maps to the classes of the backends and whose headers it includes. Usually the CMake options `LEMON_DEFAULT_LP` and `LEMON_DEFAULT_MIP` choose them; when they are not given, or they name a solver that was not found, the first solver found is taken in the order CPLEX, Clp (with Cbc for the MIP), GLPK, SoPlex. SoPlex is an LP solver only, and therefore `Mip` is not defined when it is the only backend; neither `Lp` nor `Mip` is defined when no backend is found. A program that must compile in all cases guards their use with `#ifdef LEMON_HAVE_LP` and `#ifdef LEMON_HAVE_MIP`, as `test/lp_test.cc` and `test/mip_test.cc` do, and as the program of §20.8 does.

20.2 Columns, rows and expressions

A column (a variable) is referred to by an object of the class `LpBase::Col`, and a row (a constraint) by an object of `LpBase::Row`. These handles behave as the items of a graph (chapter 3): they are copied freely, they compare with `==`, `!=` and `<` (an artificial order, for the associative containers), a default constructed one is undefined, and one constructed from `INVALID` is invalid. The static functions `id(col)`, `colFromId(id)`, `id(row)` and `rowFromId(id)` convert between a handle and its integer id. The iterators `ColIt` and `RowIt`, constructed from the solver, list the columns and the rows in the order of their addition, and the non-constant functions `cols()` and `rows()` return ranges for the loops of C++11. The handles stay valid when other columns or rows are added or erased, since `LpBase` keeps, for the columns and for the rows, its own index (the class `VarIndex` of `bits/solver_bits.h`) between the ids of the handles and the indices of the solver.

Lemma 20.1. *The id of a handle does not change while its column or row exists, and the index translates it into the current index of the solver in $O(1)$ time. Adding a column or a row costs $O(1)$ amortized time in the index; erasing one costs $O(1)$, plus $O(k)$ for the backends whose solver renumbers the following k columns or rows (GLPK, Clp and CPLEX); SoPlex moves the last column or row into the place of the erased one, in $O(1)$. The id of an erased column or row is reused by the next one added.*

Proof. The index keeps a vector of records, one per id, linked in a list in the order of addition (the order of `ColIt` and `RowIt`) and holding the index of the solver, and a vector `cross` from the indices of the solver to the ids. Adding takes the first record of a list of free ids, or a new one at the end of the vector, and sets both directions in $O(1)$ (amortized, for the growth of the vectors). Erasing unlinks the record and puts it at the head of the free list, which is why its id is reused next. When the solver shifts down the indices that follow the erased one, `shiftIndices()` moves the entries of `cross` and decreases the indices stored in their records, in time proportional to their number, and when it moves the last one into the hole, `relocateIndex()` updates one entry. The id of the other handles is never changed. □

Hence, a handle of an erased column must not be used afterwards, since it may refer to the column added next. A linear expression of the columns with a constant term is an object of the class `LpBase::Expr`, whose `Key` is `Col` and whose `Value` is `double`. It stores the nonzero coefficients in a `std::map` from the column id to the value, and the constant apart, so that an access to a coefficient costs $O(\log k)$ for an expression of k terms. A number and a column convert implicitly to an `Expr`, and the binary operators `+`, `-`, `*` and `/` by a number, with their compound forms, combine expressions (there is no unary minus, and one writes `-1 * x`). For instance, `2 * x + 3 * (y - x) + 4` is the expression $-x + 3y + 4$. The coefficient of a column is read with `e[c]` (zero when absent) and changed through the reference returned by the non-constant `e[c]`, and the constant term is accessed by the unary `*e`. Also, `clear()` sets everything to zero, and `simplify(eps)` removes the coefficients whose absolute value is at most `eps` (by default 0) and sets the constant to zero under the same condition. The iterators `CoeffIt` and `ConstCoeffIt` list the terms in the order of the ids, converting to the column of the term and dereferencing to its coefficient. Note that `set(c, v)` removes the term when `v` is zero, and otherwise gives the column the coefficient `v`, as `e[c] = v` does.

abLEMON. In LEMON 1.3.1, `Expr::set()` and `DualExpr::set()` called `std::map::insert`, and therefore left an existing coefficient unchanged; the test `lp_test` checks that they replace it.

A constraint is an object of the class `LpBase::Constr`, made of an expression and of a lower and an upper bound, read and written by `expr()`, `lowerBound()` and `upperBound()`; `lowerBounded()` and `upperBounded`

`()` tell whether a bound is set and finite, and `clear()` empties the expression and unsets both bounds. A bound equal to NaN is unset, which is the state of both bounds of a default constructed constraint and of one constructed from an expression alone. The comparison operators between expressions and numbers build constraints, as Table 20.1 shows.

Written	Lower bound	Expression	Upper bound
`e <= s`	unset	`e`	`s`
`s <= e`	`s`	`e`	unset
`e <= f`	0	`f - e`	unset
`e == s`	`s`	`e`	`s`
`e == f`	0	`f - e`	0
`(s <= e) <= t`	`s`	`e`	`t`

Table 20.1. The constraints built by the comparison operators, `e` and `f` expressions (or columns) and `s` and `t` numbers; `>=` gives the mirrored cases, and `s == e` is taken as `e == f` with `f` the constant `s`.

A second bound can be added to a constraint by comparing it with a number, as in the last row of Table 20.1, but only on the side that is still unset. This is checked at run time by `LEMON_ASSERT` (when assertions are enabled, chapter 8), and hence `(e <= 5) <= 6` compiles but fails the assertion. Since C++ evaluates `s <= e <= t` as `(s <= e) <= t`, the form `-3 <= v <= 4` builds the range constraint intended. We remark that a chain of two expressions and a number is valid too: `x <= y <= 5` is `(x <= y) <= 5`, i.e., the range constraint $0 \le y - x \le 5$, which passes the assertion (the header documentation states the opposite, cf. the end of §20.7).

Similarly, `LpBase::DualExpr` is the analogous expression of the rows, i.e., a column of the matrix, or a linear combination of the dual variables; its `Key` is `Row`, it has the same members as `Expr` except the constant term, and the same operators. Rows convert to it implicitly, and it is used to set or read a whole column (§20.3) and to evaluate a combination of the dual values (§20.4).

20.3 Building and modifying a problem

We now describe the members of `LpBase` that build and change the model. All of them work through the virtual functions of the backend, and therefore their cost is the one of the solver, besides the $O(1)$ of the index (Lemma 20.1) and the $O(k \log k)$ of an expression of k terms. A column is added by `addCol()`, which gives an empty column with the bounds $-\infty$ and $+\infty$ in all the backends of the library, and by `addCol(e, o)`, which also sets its coefficients from the `DualExpr e` and its objective coefficient to `o` (0 by default). The function `addColSet(t)` fills a container with new columns and returns their number. The argument `t` can be a container of `Col` (e.g., a `std::vector`), an associative container whose mapped type is `Col` (e.g., a `std::map< K , Col >`), or a writable map of the library with values of type `Col`, e.g., a `ListDigraph::ArcMap< LpBase::Col >`, which gives one column per arc. The rows are added in the same ways: `addRow()` gives an empty row, `addRowSet(t)` fills a container, and `addRow(l, e, u)` and `addRow(c)` add a row from an expression with two bounds or from a constraint.

When an expression with constant term c_e enters a row, the constant is moved to the bounds, i.e., the row $l \le a^\top x + c_e \le u$ is stored as $l - c_e \le a^\top x \le u - c_e$. In `addRow(c)` and `row(r, c)` from a constraint, an unset or infinite bound becomes $\mp\infty$ first, while `addRow(l, e, u)` and `row(r, l, e, u)` pass the given bounds as they are (and so pass a NaN to the solver). The functions that take an expression (`addRow()`, `row()`, `addCol(e, o)` and `col()`) first remove its zero coefficients, which they do in the expression of the caller, although they take it by constant reference.

Once created, a column is changed by `col(c, e)`, which sets its coefficients, and read back as a `DualExpr` by `col(c)`; the analogous `row(r, l, e, u)` and `row(r, c)` set a row, and `row(r)` returns its coefficients as an `Expr`. A single coefficient is set and read by `coeff(r, c, v)` and `coeff(r, c)`, and `erase(c)` and `erase(r)` remove a column or a row. The bounds are set and read by `colLowerBound(c, v)`, `colUpperBound(c, v)`, `colBounds(c, l, u)`, `rowLowerBound(r, v)` and `rowUpperBound(r, v)`, with the getters `colLowerBound(c)`, `colUpperBound(c)`, `rowLowerBound(r)` and `rowUpperBound(r)`; the three column setters also accept a container or a map of columns, as `addColSet()` does, to give the same bounds to all of them. Names are given by `colName(c, s)` and `rowName(r, s)`, read by `colName(c)` and `rowName(r)`, and searched by `colByName(s)` and `rowByName(s)`, which return `INVALID` when no item has the name.

The objective is set by `obj(e)`, which replaces all the coefficients and whose constant term is kept by `LpBase` itself and added to the objective value of the solution, and read by `obj()`; `objCoeff(c, v)` and `objCoeff(c)`

set and read one coefficient. The direction is set by `sense(MIN)` or `sense(MAX)`, or by the shorthands `min()` and `max()`, and read by `sense()`. Finally, `clear()` empties the model and the index of the columns and of the rows, `messageLevel(level)` sets how much the solver prints (all the backends start at `MESSAGE_NOTHING`), and `solverName()` returns the name of the backend. We remark that `clear()` does not reset the constant term of the objective, which `primal()` and `solValue()` keep adding until the next `obj(e)`. Also, `write(file, format )` writes the model to a file, by default in the format `"MPS"`; a format that the backend does not support throws `LpBase::UnsupportedFormatError`, an `Exception` whose `what()` names the format (Table 20.2).

20.4 Solving an LP and reading the solution

For an LP, i.e., (20.1) with $J = \emptyset$, the class `LpSolver` adds the solution. We recall the optimality conditions that its dual values satisfy, for a minimization (for a maximization, one minimizes $-c^\top x$). For $y \in \mathbb{R}^m$, let $d = c - A^\top y$ be the vector of the reduced costs, and let

$$\varphi(y) = \sum_{i=1}^{m} \min_{l_i^r \le s \le u_i^r} y_i s \; + \; \sum_{j=1}^{n} \min_{l_j^c \le t \le u_j^c} d_j t, \tag{20.2}$$

where a minimum is $-\infty$ when it is unbounded, e.g., when $y_i > 0$ and $l_i^r = -\infty$. The *dual problem* is $\max\{\varphi(y) : y \in \mathbb{R}^m\}$, and a y with $\varphi(y) > -\infty$ is dual feasible.

Lemma 20.2. *For every x feasible for (20.1) with $J = \emptyset$ and every y, $c^\top x \ge \varphi(y)$, with equality if and only if*

$$y_i > 0 \Rightarrow (Ax)_i = l_i^r, \quad y_i < 0 \Rightarrow (Ax)_i = u_i^r, \quad d_j > 0 \Rightarrow x_j = l_j^c, \quad d_j < 0 \Rightarrow x_j = u_j^c. \tag{20.3}$$

Proof. One has $c^\top x = y^\top A x + d^\top x = \sum_i y_i (Ax)_i + \sum_j d_j x_j$. Since $(Ax)_i \in [l_i^r, u_i^r]$ and $x_j \in [l_j^c, u_j^c]$, each term is at least the corresponding minimum of (20.2), and the sum is at least $\varphi(y)$. Equality holds if and only if each term attains its minimum; the minimum of $y_i s$ over the interval is attained at the lower end when $y_i > 0$, at the upper end when $y_i < 0$, and anywhere when $y_i = 0$, and likewise for $d_j t$, which gives (20.3). □

Theorem 20.1. *A feasible x is optimal for (20.1) with $J = \emptyset$ if and only if there is a y that satisfies (20.3), and then y is optimal for the dual problem.*

Proof. If y satisfies (20.3), then $c^\top x = \varphi(y) \le c^\top x'$ for every feasible x' by Lemma 20.2, and $\varphi(y') \le c^\top x = \varphi(y)$ for every y'. Conversely, if x is optimal, the strong duality theorem of linear programming, applied to (20.1) written with the inequalities $Ax \ge l^r$, $-Ax \ge -u^r$, $x \ge l^c$ and $-x \ge -u^c$ (the infinite ones dropped), gives nonnegative multipliers of the four groups whose dual objective equals $c^\top x$. The differences of the multipliers of the two rows of each pair form a y with $\varphi(y) \ge c^\top x$, and hence $\varphi(y) = c^\top x$, which gives (20.3) by Lemma 20.2. The strong duality theorem is proved, e.g., in Bertsimas and Tsitsiklis 1997, §4.3, Theorem 4.4, and in Dantzig 1963. □

In the classical interpretation of the multipliers, the dual value y_i of a row is the rate of change of the optimal value with respect to the bound that is active, and by (20.3) its sign indicates which bound it is. The interface passes on the dual values of the solver unchanged, and hence their sign convention, in particular for a maximization, is the one of the solver.

A call of `solve()` solves the problem and returns the `SolveExitStatus` of §20.1. The *problem type* of the primal and of the dual problem is then read by `primalType()` and `dualType()`, of the enumeration `LpSolver::ProblemType`: `UNDEFINED` (no feasible solution found, although one may exist), `INFEASIBLE`, `FEASIBLE` (a feasible solution found, its optimality not proved), `OPTIMAL` and `UNBOUNDED`. The values of a solution are read by `primal(c)` for a column, `primal(e)` for an expression (the scalar product with the primal solution plus the constant term), `dual(r)` for a row and `dual(e)` for a `DualExpr`; all of them require a solved problem. As for the objective value, `primal()` includes the constant term of the objective; according to the documentation, it is `INF` or `-INF` for an infeasible or unbounded problem, depending on the direction, and NaN when no primal solution is found. Also, the basis is read by `colStatus(c)` and `rowStatus(r)`, of the enumeration `VarStatus`: `BASIC`, `FREE` (nonbasic and free), `LOWER` and `UPPER` (nonbasic at a bound) and `FIXED` (nonbasic and fixed).

When the problem has no optimal solution, the rays certify it, in the backends that provide them (the header warns that some solvers do not compute them). Let φ_0 be the function (20.2) with c replaced by 0, i.e., the dual problem with its right-hand sides set to 0.

Lemma 20.3. *(i) Let r satisfy the bounds of (20.1) with each finite bound replaced by 0 and the infinite ones kept, i.e., $(Ar)_i \geq 0$ if l_i^r is finite, $(Ar)_i \leq 0$ if u_i^r is finite, and likewise for r_j, and let $c^\top r < 0$ (a* primal ray*). Then the dual problem has no feasible y, and if (20.1) has a feasible x, it is unbounded. (ii) Let w satisfy $\varphi_0(w) > 0$ (a* dual ray*). Then (20.1) is infeasible.*

Proof. (i) Let $\varphi(y) > -\infty$. If $y_i > 0$ then l_i^r is finite, since otherwise the i-th minimum of (20.2) is $-\infty$, and so $(Ar)_i \geq 0$; if $y_i < 0$, then u_i^r is finite and $(Ar)_i \leq 0$. Hence $y_i(Ar)_i \geq 0$ for every i, and in the same way $d_j r_j \geq 0$ for every j. Then $c^\top r = y^\top Ar + d^\top r \geq 0$, a contradiction. If x is feasible, so is $x + tr$ for every $t \geq 0$, since a finite bound that x satisfies stays satisfied along r and the infinite ones impose nothing, and $c^\top(x + tr) \to -\infty$. (ii) If x were feasible, Lemma 20.2 with $c = 0$ would give $0 = 0^\top x \geq \varphi_0(w) > 0$, which is impossible. □

In the interface, `primalRay(c)` gives the component of a primal ray on the column c, with a negative objective value for a minimization and a positive one for a maximization. Similarly, `dualRay(r)` gives the component of a dual ray on the row r; the first requires that the dual problem be infeasible, and the second that the primal problem be infeasible. Finally, `newSolver()` returns a new empty solver of the same type, and `cloneSolver()` a copy of the problem; both return a pointer that the caller deletes.

20.5 Mixed integer programs

In `MipSolver` one describes (20.1) with integer columns. The type of a column is set by `colType(c, type)` and read by `colType(c)`, of the enumeration `MipSolver::ColTypes`: `REAL` (the default) or `INTEGER`; a binary column is an integer one with the bounds 0 and 1. As for an LP, `solve()` returns `SOLVED` or `UNSOLVED`, and `type()` returns the problem type, of the enumeration `MipSolver::ProblemType`, with the same five values as `LpSolver::ProblemType`; `UNBOUNDED` means that the MIP, or at least its continuous relaxation, is unbounded. The solution is read by `sol(c)`, `sol(e)` and `solValue()`, the last one including the constant term of the objective. A MIP solver gives no dual values and no basis, and its `newSolver()` and `cloneSolver()` return a `MipSolver *`.

The integrality of a column whose bounds are not integer deserves a remark (cf. Exercise 20.5). The integer values of such a column are those of $[\lceil l_j^c \rceil, \lfloor u_j^c \rfloor]$, and the problem is infeasible when this interval is empty. GLPK refuses such a column, and Cbc does not round its bounds; hence, `GlpkMip` and `CbcMip` round the bounds of each integer column inwards before the solve, up to an integer tolerance $\epsilon = 10^{-6}$, as in Algorithm 20.1, which is the one of `GlpkMip::_solve()`.

Algorithm 20.1 (The solve of `GlpkMip`).

```
1  solve the continuous relaxation by the dual simplex method
   (on a bad, singular or ill-conditioned basis, build an advanced basis by
   glp_adv_basis() and solve once more; return UNSOLVED if this fails)
2  if the relaxation is not optimal: return SOLVED (type: of the relaxation)
3  for each integer column j, until one without integer values is found:
4      l' := ceil(l_j - eps) if l_j is finite; u' := floor(u_j + eps) if u_j is
         finite
5      if both bounds are finite and l' > u': the problem is INFEASIBLE;
         skip the search
6      else give the column the bounds [l', u'] (fixed if l' = u')
7  run the branch and cut of GLPK (glp_intopt)
8  give back the bounds given by the user; return SOLVED, or UNSOLVED
   if glp_intopt failed
```

Theorem 20.2. *Let $Z_j = \{z \in \mathbb{Z} : l_j^c - \epsilon \leq z \leq u_j^c + \epsilon\}$ for each integer column j. Algorithm 20.1 solves the MIP whose integer columns range over the sets Z_j, i.e., (20.1) with each bound of an integer column relaxed by ϵ; it declares the problem infeasible, without calling the branch and cut, exactly when some Z_j is empty, and it leaves the bounds of the model as they were given. The rounding costs $O(n)$ calls of GLPK.*

Proof. For an integer z and a real l, $z \geq l - \epsilon$ holds if and only if $z \geq \lceil l - \epsilon \rceil$, and $z \leq u + \epsilon$ if and only if $z \leq \lfloor u + \epsilon \rfloor$. Hence the integers of $[l', u']$ are Z_j, and the rounded MIP has the same feasible set as the relaxed one. When both bounds are finite, Z_j is empty if and only if $l' > u'$, and then the MIP has no feasible solution, which line 5 records (the type is then `INFEASIBLE`); a column with an infinite bound has a nonempty Z_j. Line 8 restores the bounds saved before they were changed, and each column is examined once. □

`CbcMip` rounds the bounds in the same way in the copy of the problem that it gives to Cbc, which leaves the bounds of the model unchanged, and a column without integer values makes the initial solve of Cbc infeasible.

The test `test/mip_test.cc` checks both that the bounds stay as they were given and that a column without integer values makes the problem infeasible.

abLEMON. The rounding of the bounds of the integer columns in `GlpkMip` and `CbcMip` is new in abLEMON (version 1.4.1): GLPK 5.0 refused such a column, which left the problem unsolved, and Cbc gave a solution beyond the bound. Also, `CbcMip` no longer reports a proven infeasible problem as feasible (trac #626, version 1.4.0).

20.6 The backends

Each backend wraps one solver and adds a few functions of its own. All of them have a default constructor and a copy constructor, which copies the problem. Table 20.2 summarizes them, and the paragraphs below give the details.

Class	Solver	`solve()`	Other solves	Formats of `write()`
`GlpkLp`	GLPK	primal simplex	`solvePrimal()`, `solveDual()`	`"MPS"`, `"LP"`
`GlpkMip`	GLPK	Algorithm 20.1		`"MPS"`, `"LP"`
`CplexLp`	CPLEX	`CPXlpopt`	`solvePrimal()`, `solveDual()`, `solveBarrier()`	`"MPS"`, `"LP"`, `"SOL"`
`CplexMip`	CPLEX	branch and cut of CPLEX		`"MPS"`, `"LP"`, `"SOL"`
`ClpLp`	Clp	primal simplex	`solvePrimal()`, `solveDual()`, `solveBarrier()`	`"LP"`
`CbcMip`	Cbc	branch and cut of Cbc		none
`SoplexLp`	SoPlex	simplex of SoPlex		none

Table 20.2. The backends, the method of `solve()`, the other solve functions, and the formats accepted by `write()`; any other format throws `UnsupportedFormatError`.

GLPK. `GlpkLp` and `GlpkMip` share the base `GlpkBase`, whose `lpx()` returns the pointer to the problem object of GLPK (as a `_solver_bits::VoidPtr`, which converts to any pointer type), and whose `lpxRow(r)` and `lpxCol(c)` give the indices that GLPK uses. Each solve function of `GlpkLp` runs the simplex method of GLPK; when it reports a bad, singular or ill-conditioned basis, it builds an advanced basis by `glp_adv_basis()` and runs once more, and it returns `UNSOLVED` if this fails too. The function `presolver(bool)` turns the presolver of GLPK on or off, and it is off by default. Also, a static member of `GlpkBase` of the private type `FreeEnvHelper` frees the environment of GLPK (`glp_free_env()`) when the program ends.

CPLEX. `CplexLp` and `CplexMip` share the base `CplexBase`, whose `cplexEnv()` and `cplexLp()` return the environment and the problem objects of CPLEX, and whose `env()` returns the `CplexEnv` in use. A `CplexEnv` wraps the environment of CPLEX with a reference count protected by a lock, so that several problems can share one environment (and one license); it is copied and assigned by sharing the environment. The constructors of `CplexLp` and `CplexMip` that take a `CplexEnv` use the given one, and the others open a new one. When the license check fails, the constructor of `CplexEnv` throws `CplexEnv::LicenseError`, whose `what()` gives the message of CPLEX. In `CplexLp`, `solve()` calls `CPXlpopt`, which lets CPLEX choose the method, and the three other functions force the primal simplex, the dual simplex and the barrier method.

Clp and Cbc. `ClpLp` wraps a `ClpSimplex` object of Clp; `solve()` runs the primal simplex method, and `clpRow(r)` and `clpCol(c)` give the indices of Clp. Note that `ClpLp` writes only the format `"LP"`, and therefore `write(file)` with the default `"MPS"` throws. `CbcMip` keeps the model in a `CoinModel` of Cbc, and at each solve it copies the model into a new `OsiClpSolverInterface` (with the rounding of §20.5) and solves the continuous relaxation. When this is optimal, it runs the branch and cut of Cbc with a fixed set of cut generators (probing, Gomory, knapsack cover, odd hole, clique, mixed integer rounding and flow cover) and of primal heuristics (rounding, local search, greedy cover, feasibility pump and RINS).

SoPlex. `SoplexLp` wraps the class `SoPlex` of SoPlex. Its message level `MESSAGE_NOTHING` maps to the verbosity of errors of SoPlex, which is (apparently) the lowest one SoPlex offers. Its primal ray is the one of SoPlex, and its dual ray is the Farkas proof of the infeasibility of the primal problem, i.e., a w of Lemma 20.3 (ii).

abLEMON. `SoplexLp` works with SoPlex 5 and later, through the API of its class `SoPlex`; in LEMON 1.3.1 it did not compile with these versions, and it gave the Farkas proof of the dual as its primal ray (version 1.5.0). SoPlex is found through its CMake configuration.

20.7 The skeletons

The header `lp_skeleton.h` declares `LpSkeleton` and `MipSkeleton`, which derive from `LpSolver` and `MipSolver` and from the common base `SkeletonSolverBase`. They implement the virtual functions of the interface by doing (almost) no work. The coefficients, the bounds and the names are not stored, the getters return 0 (or `MIN` for the sense, an empty name, and `-1`, i.e., `INVALID`, for a search by name), and `solve()` returns `SOLVED`. The problem types are `UNDEFINED`, the basis statuses are `BASIC`, `write()` writes nothing and throws nothing, and `newSolver()` and `cloneSolver()` return a null pointer. Their names are `"LpSkeleton"` and `"MipSkeleton"`. The indices of the solver are counted from 0, and `clear()` starts them from 0 again. On a skeleton, however, all that `LpBase` keeps works as with a real solver: the index of the columns and of the rows, and hence `ColIt`, `RowIt`, `cols()`, `rows()` and the handles, and the classes `Expr`, `DualExpr` and `Constr` with their operators, which do not involve the solver at all.

We remark that `MipSkeleton` does not implement the two functions `_setColType()` and `_getColType()` that `MipSolver` declares pure virtual, and therefore, in this version, it is an abstract class, which cannot be instantiated. A program that needs a MIP model without a solver can only build it on an `LpSkeleton` through `LpBase &`.

abLEMON. `clear()` of the skeletons starts the indices of the columns and of the rows from 0 again, as the constructor does; in LEMON 1.3.1 they started from 1 after a `clear()`.

We see two uses of the skeletons. First, a program can build and check a model, e.g., count its columns and rows, without any solver; `test/lp_test.cc` does so in all configurations, and so does the program of §20.8. Second, they are the starting point for a new backend. Such a backend must implement the pure virtual functions declared in `lp_base.h`: `_addCol()`, `_addRow()`, `_eraseCol()`, `_eraseRow()`, the functions of the names (`_getColName()`, `_setColName()`, `_colByName()` and those of the rows), of the coefficients (`_setRowCoeffs()`, `_getRowCoeffs()`, `_setColCoeffs()`, `_getColCoeffs()`, `_setCoeff()`, `_getCoeff()`), of the bounds, of the objective (`_setObjCoeffs()`, `_getObjCoeffs()`, `_setObjCoeff()`, `_getObjCoeff()`) and of the sense, `_clear()`, `_solverName()` and `_messageLevel()`. An LP backend also implements `_solve()`, the getters of the solution, of the rays, of the basis and of the problem types, `newSolver()` and `cloneSolver()`; a MIP backend implements `_solve()`, `_setColType()`, `_getColType()`, `_getType()`, `_getSol()`, `_getSolValue()`, `newSolver()` and `cloneSolver()`. The coefficients reach the backend through two protected iterators of `LpBase`: `ExprIterator` lists the terms of an expression as pairs of a solver index and a value, and `InsertIterator` stores the pairs returned by the solver into an expression, translating the indices. The function `_addRow(l, b, e, u)`, which adds a row with its coefficients and bounds in one call, has a default implementation by `_addRow()` and the setters, and `_write()` one that throws `UnsupportedFormatError`. The virtual functions `_addColId()`, `_addRowId()`, `_eraseColId()` and `_eraseRowId()` update the index, and a backend whose solver renumbers its columns or rows on an erasure overrides the last two to call `shiftIndices()` (or, as `SoplexLp` does, `relocateIndex()`). The header documentation of `Constr` states that `x[1] <= x[2] <= 5` fails an assertion, which it does not (cf. §20.2).

20.8 An example

The program `manual/examples/20-lp.cc` (the test `manual_20_lp`) writes the maximum flow problem on a digraph $G = (N, A)$ with capacities u, a source s and a sink t as the LP

$$\max \sum_{a \in \delta^+(s)} x_a - \sum_{a \in \delta^-(s)} x_a \quad \text{s.t.} \quad \sum_{a \in \delta^-(v)} x_a = \sum_{a \in \delta^+(v)} x_a \;\; (v \in N \setminus \{s,t\}), \quad 0 \le x_a \le u_a \;\; (a \in A). \tag{20.4}$$

Here $\delta^+(v)$ and $\delta^-(v)$ are the arcs leaving and entering v. The model is built in a function that takes any `LpSolver`, with one column per arc obtained by `addColSet()` on an arc map and one row per inner node. It builds the model on an `LpSkeleton`, on which it checks the number of columns and rows and the algebra of the expressions and constraints of Table 20.1, including a coefficient set twice by `Expr::set()`. When the library has an LP backend, the same function builds the model on the default solver `Lp`, which solves it, and the program compares the optimal value with the one of `Preflow` (chapter 12). Without a backend this part is left out by `#ifdef LEMON_HAVE_LP`, and the program still compiles and checks the rest.

20.9 Exercises

Exercise 20.1. Write the shortest path problem from s to t as an LP with one column per arc, solve it with the default solver, and compare its value with the distance computed by `Dijkstra` (chapter 10). Then read the

dual values of the rows and check that they are distances from s (up to a constant and to the sign convention of the solver), as Theorem 20.1 predicts.

Exercise 20.2. Build an infeasible LP, e.g., by adding to the model of (20.4) the row $\sum_{a\in\delta^+(s)} x_a \geq 1 + \sum_a u_a$, and verify that `primalType()` is `INFEASIBLE` and that the dual ray, when the backend provides it, satisfies Lemma 20.3 (ii) (`test/lp_test.cc` checks the problem types of a small LP).

Exercise 20.3. Show with a short program that `Expr::set(c, v)` and `e[c] = v` give the same expression, also when `c` already has a coefficient, and that `simplify(eps)` also zeroes the constant term when it is at most `eps` in absolute value (the program `manual/examples/20-lp.cc` checks the first part).

Exercise 20.4. Write the maximum weight matching problem of a small bipartite graph as a MIP with binary columns, solve it with `Mip`, and compare the result with `MinCostAssignment` of chapter 17 (on the negated weights). Then relax the integrality and observe that the LP gives the same value, as the total unimodularity of the bipartite incidence matrix implies.

Exercise 20.5. Give an integer column the bounds 0.5 and 0.7, and check that `GlpkMip` and `CbcMip` report the problem as `INFEASIBLE` and leave the bounds of the model unchanged (Theorem 20.2 and `test/mip_test.cc`). Then give it the bounds $1 + 10^{-7}$ and 1.5, and explain why the solution may take the value 1, below the bound given.

Exercise 20.6. Write a function that builds the model of (20.4) on an `LpBase &`, use it with an `LpSkeleton`, and count its columns and rows through `ColIt` and `RowIt`. Then erase a column, add a new one, and check that the new one takes the id of the erased one (Lemma 20.1; `manual/examples/20-lp.cc` counts the columns and rows on the skeleton).

21. Input and output

This chapter describes how graphs and the data attached to them are read from and written to files. The format of the library is the LEMON Graph Format (LGF), read by the classes of `lgf_reader.h` and written by those of `lgf_writer.h` (§21.1 to §21.4). Besides it, `dimacs.h` reads the formats of the first DIMACS implementation challenge and writes one of them (§21.5), `nauty_reader.h` reads the graph6 format of the nauty package (§21.6), and `graph_to_eps.h` draws a graph in Encapsulated PostScript (§21.7). A malformed input makes the readers and the writers throw `FormatError`, and a file that cannot be opened makes them throw `IoError` (chapter 8); both derive from `Exception`.

21.1 The LEMON Graph Format

An LGF file is a text file made of *sections*, each opened by a header line whose first character is `@`, followed by the type of the section and by an optional name, the *caption*, which distinguishes sections of the same type. A line whose first non-blank character is `#` is a comment, and the empty lines are ignored; whatever precedes the first header is skipped. The standard types are `@nodes`, `@arcs` (with its synonym `@edges`) and `@attributes` for digraphs and graphs, and `@red_nodes` and `@blue_nodes` for the two node sets of a bipartite graph. A file may contain other sections, whose content is free (§21.4).

Standard sections are made of *tokens* separated by white space. A token is either plain, i.e., a sequence of characters other than white space, or quoted, i.e., enclosed in double quotes, in which case it may contain white space. In both kinds a backslash introduces an escape sequence of C: `\\`, `\"`, `\'`, `\?`, `\a`, `\b`, `\f`, `\n`, `\r`, `\t`, `\v`, a character in octal (one to three digits) or in hexadecimal (`\x` with one or two digits). Moreover, the node and arc sections are *column oriented*: their first line gives the names of the maps, and each following line, up to the next header, gives the values of these maps for one item. A short file shows the structure:

```
@nodes
label  name
0      "the source"
1      a
2      t
@arcs
          label  capacity
0    1    0      16
0    2    1      12
1    2    2      4
@attributes
source  0
target  2
```

In a node section, the map named `label` plays a special role, since its values identify the nodes in the rest of the file; it is needed whenever the arcs or the attributes refer to the nodes. In an arc section, each line starts with the labels of the source and of the target node, followed by the values of the maps; the map `label` is optional here, and it identifies the arcs for the attributes. An arc section without maps must contain a single `-` in its first line (in a node section, a `-` would be read as the name of a map). For an undirected graph, the section `@edges` (or `@arcs`) describes the edges, and two columns whose names are the name of a map prefixed with `+` and `-` hold the values of an arc map on the two arcs of each edge. Each line of the section `@attributes` consists of two tokens, a name and a value. The value may be the label of a node or of an edge, and the label of an edge prefixed with `+` or `-` refers to the forward or the backward arc of that edge. For a bipartite graph, the sections `@`

`red_nodes` and `@blue_nodes` replace `@nodes`, each with its own maps; a map present in both sections can be read as a node map.

21.2 Reading

We now describe `DigraphReader< DGR >`, which reads an LGF file into a digraph of type `DGR` (exported as `Digraph`) by *batch processing*. One creates a reader, adds to it the rules that specify which maps and attributes are wanted and where they go, and executes them by `run()`, which reads the file once. The functions that add the rules return a reference to the reader, and are therefore chained:

```
DigraphReader< ListDigraph >( g , std::cin )
  .arcMap( "capacity" , cap ).node( "source" , s ).run();
```

Its three constructors take the digraph and an input stream (by default `std::cin`), a file name as a `std::string` or a file name as a `const char *`; the last two open the file and throw `IoError` when they cannot. With the same three kinds of source, `digraphReader()` returns a reader, which lets one write the chain without naming the type (the reader returned is moved, not copied, and a reader whose stream has been taken over by another one may not be run).

abLEMON. The readers and the writers take over the stream of the object returned by `digraphReader()` and the like through a constructor that takes it by rvalue reference, which C++23 requires; in LEMON 1.3.1 it took a non-constant lvalue reference, and the LGF classes did not compile under C++23 (version 1.4.0).

We now list the reading rules. A rule `nodeMap(caption, map)` reads the column `caption` of the node section into `map`, a writable node map, and `arcMap(caption, map)` does the same for the arc section. One column can be read into several maps. The value of each token is converted to the value type of the map by `operator>>` of a `std::istringstream`, and the token must be read whole, i.e., a token that cannot be converted, or that leaves characters behind, throws `FormatError`. A token read into a `std::string` is taken as it is. The overloads `nodeMap(caption, map, converter)` and `arcMap(caption, map, converter)` take instead a functor from `std::string` to the value type. The rules `attribute(caption, value)` and `attribute(caption, value, converter)` read an attribute into a variable, and `node(caption, n)` and `arc(caption, a)` read an attribute whose value is a label into a node or an arc handle.

By default the reader uses the first section of each type. The functions `nodes(caption)`, `arcs(caption)` and `attributes(caption)` select instead the section of that type with the given caption. With `useNodes(map)` the nodes are taken as already existing, and they are not created. Then `map` gives the label of each existing node (its values are turned into strings by `operator<<`, or by a converter in the overload `useNodes(map, converter)`), and the node section only identifies the nodes by their labels, reading the other maps into them. Similarly, `useArcs(map)` and `useArcs(map, converter)` reuse the existing arcs. These functions allow a *multipass* reading, e.g., to read a node set and one arc section into a digraph in a first pass, and a second arc section into an arc set of chapter 4 in a second pass. The functions `skipNodes()` and `skipArcs()` leave a section out, i.e., the nodes are then not created, and the arcs can only be read if `useNodes()` gives the labels.

Algorithm 21.1 (The reading of an LGF file by `run()`).

```
1  read lines up to the first header (skipSection)
2  while a header "@type caption" has been read:
3      if type = nodes, no node section read yet, and the caption matches:
4          read the map names; for each line up to the next header: read one
           token per map; create a node (or find the node of its label, with
           useNodes); store its label; apply the node rules (readNodes)
5      elif type is arcs or edges, likewise, the first two tokens of a line
           being the labels of the source and of the target (readArcs)
6      elif type = attributes, likewise: each line is a name and a value,
           given to all the rules of that name (readAttributes)
7      else skip the section
8  throw FormatError if the node or the arc section was not read (unless
   skipped), or if attribute rules exist and no attribute section was read
```

Note that the checks are rather strict. A map named by a rule and absent from the header, a column missing in a line, an extra token at the end of a line, a label not found, a map name repeated in a header and an attribute repeated in its section all throw `FormatError`; so does an attribute required by a rule and absent from the section. In all cases, the error carries the line number and, when the reader was opened on a file, the file name,

and its `what()` gives both. A label repeated in a node or an arc section, however, is not detected: the item read first keeps it, and the later references go to that item.

Theorem 21.1. *Algorithm 21.1 reads each line of the stream once, and on success it has created one node per line of the selected node section and one arc per line of the selected arc section, in the order of the lines, with the values of the rules set on them. With L characters in the input, n nodes, m arcs and k map columns, it performs $O(L)$ character operations, $O((n+m)\log(n+m))$ comparisons of labels, and $n+m$ times k conversions at most.*

Proof. Every line is obtained by `readLine()`, which advances the stream, and a header line is pushed back only into the buffer of the current line, never into the stream; the outer loop handles one header per iteration, and each of `readNodes()`, `readArcs()`, `readAttributes()` and `skipSection()` consumes the lines up to the next header. A section with a type already read, or with a caption that does not match, falls into the last branch and is skipped. In `readNodes()` each line yields one node, created in the order of the lines (or looked up by its label when `useNodes()` is given), and its tokens are passed to the storages of the rules; `readArcs()` does the same, looking up the source and the target by their labels. The labels are kept in a `std::map` from the token to the item, so that each insertion and each look-up costs $O(\log(n+m))$ comparisons. Each token is converted once for each rule that reads its column. □

For an undirected graph one uses `GraphReader< GR >` (exported as `Graph`), which reads it in the same way, with the rules `nodeMap()`, `edgeMap()`, `attribute()`, `node()` and `edge()`, the selection functions `nodes()`, `edges()` and `attributes()`, and `useNodes()`, `useEdges()`, `skipNodes()` and `skipEdges()`. Its rule `arcMap(caption, map)` reads the two columns `+caption` and `-caption` into the values of an arc map on the forward and the backward arc of each edge, and `arc(caption, a)` reads an attribute whose value is an edge label with a sign. Similarly, `BpGraphReader< BGR >` reads a bipartite graph from the sections `@red_nodes`, `@blue_nodes` and `@edges`, all three of which must be present. It has the members of `GraphReader`, its `nodes(caption)` selects both node sections, and it adds the rules `redNodeMap()`, `blueNodeMap()`, `redNode()` and `blueNode()`, while its `nodeMap()` reads a map that both node sections contain. The functions `graphReader()` and `bpGraphReader()` return readers as `digraphReader()` does. In the code, each rule is an object of the namespace `_reader_bits`, i.e., a `MapStorage` (or a `GraphArcMapStorage`, for the two columns of an arc map) for a map, and a `ValueStorage` for an attribute, both converting by a `DefaultConverter` unless a converter is given. The label of an attribute is resolved by a `MapLookUpConverter`, a `DoubleMapLookUpConverter` (for a node of a bipartite graph, searched among the red and the blue labels) or a `GraphArcLookUpConverter` (for a signed edge label).

21.3 Writing

Symmetrically, `DigraphWriter< DGR >` writes a digraph in the same batch style: the rules `nodeMap(caption, map)`, `arcMap(caption, map)`, `attribute(caption, value)`, `node(caption, n)` and `arc(caption, a)`, executed by `run()`. Its constructors take the digraph and an output stream (by default `std::cout`) or a file name, and throw `IoError` when the file cannot be opened; `digraphWriter()` returns a writer in the same three ways. Each value of a map or of an attribute is converted to a token by `operator<<` of a `std::ostringstream`, or by a functor from the value type to `std::string` given as the third argument of the rule. A token that is empty, starts with `@`, or contains white space or one of the characters `\`, `"`, `'`, `\a` and `\b`, is written quoted. Its characters are written with the escapes `\\`, `\"`, `\a`, `\b`, `\f`, `\n`, `\r`, `\t`, `\v` and the octal code of any other character below 32, and the tokens that need no quotes are written as they are. We remark that the default conversion writes a `double` with the default precision of the streams, i.e., 6 significant digits, so that a file written with it does not give back the same values when it is read; a converter that writes 17 significant digits (as the program of §21.8 does) avoids this.

Lemma 21.1. *Let s be a string of characters of codes 32 to 126 and of the characters `\a`, `\b`, `\f`, `\n`, `\r`, `\t` and `\v`, which does not start with `#`. Then the token that the writer writes for s is read back by the reader as s.*

Proof. If the writer does not quote s, then s is not empty, does not start with `@` or `"`, and contains no white space and no backslash; the reader reads a plain token up to the next white space, i.e., s, and finds no escape in it. If the writer quotes s, then each `\` and `"` of s is written as a two-character escape that the reader maps back, each of the seven control characters as its letter escape, which the reader maps back, and every other character of s as it is. The reader stops at the first `"` that is not escaped, which is the closing one. In both cases the token is followed by white space or by the end of the line, so that the next token does not absorb its last character. The condition on `#` is needed only for the first token of a line, since a line whose first non-blank character is `#` is a comment. □

The conditions of Lemma 21.1 are not formalities, since three cases fall outside it in this version. A label (or the

first column) that starts with # is written plain, and its line is then read as a comment. A character below 32 other than those listed is written in octal without padding to three digits, so that a following digit from 0 to 7 is read as part of the escape. Finally, a byte above 127, e.g., of a UTF-8 text, is written in a quoted token as the octal code of a negative number, which is read back as different bytes. Also, the reader accepts any letter after \x as a hexadecimal digit (\xg is the character 16).

Each item gets a label from the writer. If a map is written under the name label, its values are the labels, and the items are written in the order of these values (an order that operator< must define, otherwise FormatError is thrown at run time). Otherwise the writer adds a column label with the ids of the items, and writes them in the order of the ids. Hence, the arc section always has the label column, and therefore it never needs the - of §21.1. The functions nodes(caption), arcs(caption) and attributes(caption) give a caption to the sections, skipNodes() and skipArcs() leave a section out (its labels are still computed, for the sections that follow). Also, ostream() returns the stream of the writer. These allow writing two arc sections in two passes on the same stream: the second writer skips the nodes and appends its arc section to the output of the first.

Theorem 21.2. *run() of DigraphWriter writes each node and each arc once, in the order of their labels (or of their ids), with distinct labels when the ids are used, in $O((n+m)\log(n+m))$ comparisons plus one conversion per value written.*

Proof. The writer collects the items of each section in a vector and sorts it, by the comparator MapLess (or GraphArcMapLess) on the label map when it is given, and by the ids otherwise. The ids are distinct, and hence so are the labels it adds. The labels of the nodes are then kept in a std::map to write the source and the target of each arc and the attributes that refer to items, with $O(\log n)$ comparisons per look-up. □

Note that the writer keeps the maps and the values of the rules by reference, and reads them only in run(). Thus, they must exist until then. A temporary passed to a rule lives until the end of the full expression that contains it; this is enough in a chain that ends with run(), but not when the rules and run() are separated.

abLEMON. The documentation of the writers states that the values of their rules are kept by reference; in LEMON 1.3.1 the test of the format passed a temporary arc to a writer, which read it after its end (trac #683, version 1.4.0).

For an undirected graph, GraphWriter< GR > is used, with edgeMap() and edge() for the edges and an arcMap() that writes the columns +caption and -caption; the rule arc(caption, a) writes the label of the edge of a with the sign of its direction, and nodes(), edges(), attributes(), skipNodes(), skipEdges() and ostream() work as for digraphs. Similarly, BpGraphWriter< BGR > (exporting the type as BpGraph) writes the sections @red_nodes, @blue_nodes and @edges, with the rules redNodeMap(), blueNodeMap(), redNode() and blueNode(); its nodeMap() writes the map in both node sections, and its nodes(caption) gives the caption to both. The functions graphWriter() and bpGraphWriter() return writers. Their rules are objects of the namespace _writer_bits with the same names as those of the readers (MapStorage, GraphArcMapStorage, ValueStorage, MapLookUpConverter, DoubleMapLookUpConverter and GraphArcLookUpConverter), and the items are sorted by their labels through the comparators MapLess and GraphArcMapLess.

21.4 Extra sections and the contents of a file

Sections of other types are read by SectionReader and written by SectionWriter, which are constructed, like the other readers and writers, from a stream or a file name, or returned by sectionReader() and sectionWriter(). In a SectionReader, sectionLines(type, functor) gives each line of the section @type to a functor that takes a std::string, after the empty lines and the comments have been removed and the leading white space trimmed. Instead, sectionStream(type, functor) calls once a functor that takes the std::istream & and an int & line number, which it reads from and updates. A section is read at most once (a second section of the same type throws FormatError), and run() throws FormatError when a section with a rule is missing. In a SectionWriter, sectionLines(type, generator) writes the strings returned by a functor without arguments until it returns an empty string, while sectionStream(type, functor) lets a functor write to the stream, its last line ending with a newline. The two kinds of rules are stored as objects of the classes LineSection and StreamSection of the namespaces _reader_bits and _writer_bits, derived from a class Section. These classes act on the extra sections only, and they can be run on the same file as the graph readers, which skip the sections they do not know.

Finally, LgfContents reads a file without knowing its contents and records its structure, for general tools that must guess what a file holds; it is constructed from a stream or a file name. After run(), nodeSectionNum(), nodeSection(i) and nodeMapNames(i) give the number of node sections, the caption of the i-th one and the

names of its maps; `arcSectionNum()`, `arcSection(i)` and `arcMapNames(i)` do the same for the arc and edge sections, with the synonyms `edgeSectionNum()`, `edgeSection(i)` and `edgeMapNames(i)`. Also, `attributeSectionNum()`, `attributeSectionNames(i)` and `attributes(i)` describe the attribute sections, and `extraSectionNum()` and `extraSection(i)` give the types of the other sections.

21.5 The DIMACS formats

We now turn to the formats of the first DIMACS implementation challenge (Johnson and McGeoch 1993). They are line oriented: a line starting with `c` is a comment, the line `p type n m` gives the type of the problem and the numbers of nodes and arcs, a line starting with `n` describes a node, and a line starting with `a` an arc, the nodes being numbered from 1. A call of `dimacsType(is)` reads the comments and the problem line, and returns a `DimacsDescriptor`, whose fields are the `type` (`NONE`, `MIN`, `MAX`, `SP` or `MAT`, of the enumeration `DimacsDescriptor::Type`), the `nodeNum`, the `edgeNum` and the `lineShift`, the number of lines it has read. An unknown type, a missing problem line or another line before it throws `FormatError`. Each reader takes the descriptor as its last, optional, argument. Without it (i.e., with the default descriptor of type `NONE`), the reader calls `dimacsType()` itself, and with it, the reader continues on the stream where `dimacsType()` stopped. A reader clears the graph, creates the n nodes in order, and throws `FormatError` when the type of the file is not the one it reads.

Function	Type	Lines read	Data written
`readDimacsMin(is, g, lower, capacity, cost, supply, infty, desc)`	`min`	`n id supply`, `a u v low cap cost`	lower bounds, capacities, costs, supplies (0 when absent)
`readDimacsMax(is, g, capacity, s, t, infty, desc)`	`max`	`n id s`, `n id t`, `a u v cap`	capacities, source, target
`readDimacsSp(is, g, length, s, desc)`	`sp`	`n id`, `a u v len`	lengths, source
`readDimacsCap(is, g, capacity, infty, desc)`	`max` or `sp`	`a u v cap`	capacities (or lengths)
`readDimacsMat(is, g, desc)`	`mat`	`a u v`	the arcs (or edges)

Table 21.1. The DIMACS readers of `dimacs.h`; the problem line and the comments are read as described in the text.

An infinite capacity is written in the DIMACS files in two ways, and the readers translate both into the value `infty`. In a `min` file, an arc whose capacity is smaller than its lower bound gets the capacity `infty`, and in a `max` file (also through `readDimacsCap()`) an arc with a negative capacity gets it, while the lengths of an `sp` file are kept as they are. When `infty` is 0, its default, the readers take `std::numeric_limits< Capacity >::infinity()` if the type has an infinity, and its maximum value otherwise. The source and the target of `readDimacsMax()` and `readDimacsSp()` are `INVALID` when the file has no `n` line for them. As for `readDimacsMat()`, it adds an edge for each `a` line to an undirected graph, and an arc to a digraph, and it reads only the `a` lines. A line that starts with another letter, e.g., the `e` lines that some matching and clique files use, is skipped one character at a time, which is harmless for lines of numbers but misreads a later `a`, `c` or `n` on the same line. Conversely, `writeDimacsMat(os, g, comment)` writes a digraph in the format `mat`, i.e., an optional comment line, the problem line, and one `a` line per arc; the nodes are numbered from 1 in the order of the node iterator.

Theorem 21.3. *On a file whose node numbers lie between 1 and the n of its problem line, each reader of Table 21.1 builds a graph with n nodes, the node of number k being the k-th created, and one arc (or edge) per `a` line, in the order of the lines, in $O(L + n)$ time for L characters.*

Proof. The reader creates the n nodes in a vector indexed by their numbers, and then reads the stream once. Each `a` line adds one arc between the nodes of its two numbers and sets its data, each `n` line sets a supply or a source or target, each `c` line is skipped by `getline()`, and each step consumes at least one character. □

The readers do not check the node numbers, and a number outside $[1, n]$ indexes the vector of the nodes out of its bounds; the input must therefore be trusted.

abLEMON. `dimacsType()` is inline, and hence two translation units that include `dimacs.h` link together, which they did not in LEMON 1.3.1 (version 1.4.0).

21.6 The graph6 format of nauty

Our next format comes from the program `geng` of the nauty package (McKay and Piperno 2014), which generates all the non-isomorphic undirected graphs of a class with a given number of nodes, writing one graph per line in the *graph6* format. A call of `readNautyGraph(graph, is)` reads one line of the stream (by default `std::cin`) and builds the graph it describes, after clearing `graph`. It returns the stream, and it can thus be used as the condition of a loop that reads a whole file. In graph6, each character carries six bits, its code minus 63, and the upper triangle of the adjacency matrix is listed column by column. Thus, the pairs $\{i, j\}$, $i < j$, in the order $(0,1), (0,2), (1,2), (0,3), \ldots$, padded with zeros to a multiple of six bits.

Algorithm 21.2 (The decoding of a graph6 line).

```
1  skip the header ">>graph6<<" (10 characters) if the line starts with '>'
2  c := next character - 63
3  if c < 63: n := c
4  else: n := the 18 bits of the next three characters (minus 63 each)
5  if the line is shorter than its header, its n or its ceil(n(n-1)/12)
      characters of edges: leave the graph empty, fail the stream, return
6  create n nodes
7  for j := 1 to n - 1, for i := 0 to j - 1:
8      take the next bit, from the most significant of each character;
       if it is 1, add the edge {i, j}
```

Lemma 21.2. *On a line of the graph6 format with $n < 258048$ nodes, Algorithm 21.2 builds the graph of the line, with its edges in the order of the pairs, in $O(n^2)$ time; on a line too short it reads no character past its end.*

Proof. The pairs $\{i, j\}$ with $i < j$ are taken in the order of j and, for each j, of i, which is the order of the format. The k-th bit of the sequence is the bit $5 - (k \bmod 6)$ of the character $\lfloor k/6 \rfloor$ after the header and the size; hence the edge $\{i, j\}$ is added exactly when its bit is 1. Line 5 compares the length of the line with the 1 or 4 characters of the size and with the $\lceil n(n-1)/12 \rceil$ characters of the $n(n-1)/2$ bits before reading them. The loop takes $n(n-1)/2$ steps. □

For instance, the line `D~{` describes the complete graph on 5 nodes: `D` gives $n = 68 - 63 = 5$, and `~{` give the bits 111111 and 1111 of the 10 pairs, followed by two bits of padding. A line too short for its header, its number of nodes or its edges leaves the graph empty and sets the `failbit` of the stream. The form of the format for $n \geq 258048$, with 36 bits after two characters `~`, is not read, and on it the size is misread. The header suggests to pipe the output of `geng` into the program, since the files are large.

abLEMON. In LEMON 1.3.1 a line too short was read past its end; the test `nauty_reader_test` reads complete and truncated lines.

21.7 Drawing a graph in EPS

A call of `graphToEps(g, os)` draws a graph or a digraph as an Encapsulated PostScript picture on a stream (by default `std::cout`); the overloads `graphToEps(g, file_name)`, with a `std::string` or a `const char *`, write to a file, which they open and which `run()` closes, and throw `IoError` when they cannot open it. The function returns an object of the class `GraphToEps< T >`, where `T` is the traits class `DefaultGraphToEpsTraits < GR >` that holds the parameters. The member functions of this class set the parameters of the picture by the technique of the named parameters (chapter 3), and its `run()` writes the picture. Nothing is written without `run()`, which therefore must close the chain (and without which the file opened by the overloads is never closed):

```
graphToEps( g , "flow.eps" ).coords( coords ).arcWidths( cap )
  .autoArcWidthScale().drawArrows().title( "a flow problem" ).run();
```

The maps of the picture are given by functions that return a new object with the type of the map, and they are kept by reference. The object on which such a function is called is left inert, i.e., its `run()` writes nothing, so that the chain must continue on the returned object. First, `coords(map)` gives the position of each node, a node map with values of type `dim2::Point` (chapter 8), without which all the nodes are drawn at the point $(1, 1)$. Then `nodeSizes(map)`, `nodeShapes(map)`, `nodeColors(map)`, `nodeTexts(map)`, `nodeTextColors(map)` and `nodePsTexts(map)` set the size, the shape (of the enumeration `NodeShapes`: `CIRCLE`, `SQUARE`, `DIAMOND`, `MALE`, `FEMALE`, i.e., 0 to 4), the color, a text written on the node (of any type that `operator<<` writes), its color, and a block of PostScript code drawn at the node, which `nodePsTextsPreamble(str)` may precede with a block of definitions. Also, `arcWidths(map)` and `arcColors(map)` set the width and the color of the arcs. Colors are

objects of the class `Color` of `color.h`, and a `Palette` composed with an integer map (`composeMap()`, chapter 6) gives distinct colors.

Other parameters are set by functions that return the same object. Usually one scales the sizes by `nodeScale(d)` (0.01 by default) and `arcWidthScale(d)` (0.003). Both are relative to the diagonal of the bounding box of the nodes, unless `absoluteNodeSizes()` and `absoluteArcWidths()` make them absolute, and `autoNodeScale()` and `autoArcWidthScale()` rescale them so that the largest size equals the scale. Similarly, `scale(d)` scales the picture (1 by default); by default, the prescaling divides the scale by 10 while a side of the bounding box exceeds 1000 points, and then multiplies it by 10 while a side is below 100, so that a picture whose sides differ by more than a factor of 10 may keep a side above 1000; `preScale(false)` turns this off. Also, `scaleToA4()` fits the picture to an A4 page, `border(b)` and `border(x, y)` set the margin (10 points by default), and `negateY()` flips the vertical axis. As for the arcs, `drawArrows()` draws the arrowheads, whose size `arrowLength(d)` (1 by default) and `arrowWidth(d)` (0.3) set, and `enableParallel()` draws parallel arcs as distinct curves at the distance `parArcDist(d)` (1 by default). Moreover, `hideNodes()` and `hideArcs()` leave the nodes or the arcs out, `nodeTextSize(d)` sets the size of the texts (1 by default), and `distantColorNodeTexts()` and `distantBWNodeTexts()` color each text so that it stands out on its node. The functions `undirected()` and `directed()` choose whether the edges of an undirected graph are drawn once (the default for undirected graphs) or as two arcs, and `title(t)` and `copyright(t)` write the DSC fields `%%Title` and `%%Copyright` of the header, which also gets the creation date. Also, `edgeWidths()`, `edgeColors()`, `edgeWidthScale()`, `autoEdgeWidthScale()`, `absoluteEdgeWidths()`, `parEdgeDist()` and `hideEdges()` are aliases of the functions on the arcs. On a graph with no node the bounding box is a single point, and the prescaling, which cannot reach a side of 100 points, writes a meaningless `%%BoundingBox`. The demo `graph_to_eps_demo` (chapter 22) uses most of these parameters.

21.8 Examples

The program `manual/examples/21-lgf.cc` (the test `manual_21_lgf`) writes a network with integer capacities, floating-point costs, a name for each node (one of which contains a blank and is therefore quoted) and the source and the target as attributes, and reads it back into another digraph. It writes the costs twice, once with the default conversion and once with a converter that prints 17 significant digits, and checks that only the second gives back the same values; then it checks that a rule for a map absent from the file throws `FormatError`.

The program `manual/examples/21-formats.cc` (the test `manual_21_formats`) reads a maximum flow instance in DIMACS format from a string, with an arc of infinite capacity, and checks the maximum flow found by `Preflow`. Then it writes the digraph in the format `mat` and reads it back as an undirected graph, decodes two graph6 lines, and draws the digraph in EPS on a string, checking the header of the picture.

21.9 Exercises

Exercise 21.1. Write a digraph with two arc maps into an LGF file in two passes, each with its own arc section and caption, and read it back into a digraph and an arc set of chapter 4 with `useNodes()` and `arcs(caption)` (the multipass reading is exercised by `test/lgf_reader_writer_test.cc`).

Exercise 21.2. Write a `double` map with the default conversion and read it back; measure the largest relative error, and explain it by the precision of the streams (`manual/examples/21-lgf.cc`).

Exercise 21.3. Write an undirected graph with an arc map by `GraphWriter`, inspect the columns `+caption` and `-caption`, and read the map back by `GraphReader::arcMap()`; then write an arc as an attribute and read it back by `arc()` (`test/lgf_test.cc`).

Exercise 21.4. Use `SectionWriter` and `SectionReader` to store a list of paths, one per line as a sequence of arc labels, in an extra section of a file that also contains the digraph, and read both back.

Exercise 21.5. Convert a `min` file of DIMACS into LGF as the tool `dimacs-to-lgf` of chapter 22 does, and check that `NetworkSimplex` (chapter 13) finds the same optimal cost on both.

Exercise 21.6. Count the non-isomorphic connected planar graphs with 7 nodes from the output of `geng -c 7`, read by `readNautyGraph()` and tested by `PlanarityChecking` (chapter 18), and compare the count with the known value 646.

Exercise 21.7. Write a node map of strings whose values contain a blank and one of the cases excluded by Lemma 21.1 (a control character followed by a digit, or a UTF-8 letter), and observe what is read back.

Exercise 21.8. Draw a spanning tree of a random geometric graph by `graphToEps()`, with the tree arcs wider and in another color than the others, and compare the bounding box with and without `preScale()` (`manual/examples/21-formats.cc` draws a small digraph).

22. The tools

This chapter describes the programs distributed with the library: the three command-line tools of `tools/`, i.e., `dimacs-solver`, `dimacs-to-lgf` and `lgf-gen`, the script `lemon-0.x-to-1.x.sh`, and the three demos of `demo/`. These tools are small applications of the classes of the previous chapters, and their options are read from their sources; we also give, for each of them, the classes it uses, so that one can reproduce its work in a program (as the program of §22.6 does for two of them).

22.1 Building and calling the tools

By default CMake builds the tools, when the option `LEMON_BUILD_TOOLS` is on, and installs them into the directory of the executables (`bin` under the prefix, by `GNUInstallDirs`); the script `lemon-0.x-to-1.x.sh` is installed with them, except on Windows. Similarly, the demos are built when `LEMON_BUILD_DEMOS` is on, also by default, but they are not installed. Both can also be built with GNU make, through the makefiles of `tools/` and `demo/`, without a configuration step (chapter 2).

All three tools parse their command line with `ArgParser` (chapter 8), and hence they share its conventions. An option is a word preceded by a single dash, e.g., `-q` or `-infcap 100`, the options of a group marked as exclusive may not be given together, and `-h`, `-help` and `--help` print the list of the options with their descriptions and exit. The words that are not options are the file arguments; in `dimacs-solver` and `dimacs-to-lgf` they are an optional input file and an optional output file, which default to the standard input and output, and more than two of them is an error. Since the input file is read once and the problem line comes first, both tools also read from a pipe.

22.2 dimacs-solver

Our first tool, `dimacs-solver`, reads a problem in one of the DIMACS formats of §21.5 and solves it with the algorithm of the library for that type, reporting the times of the reading, of the construction of the algorithm object, and of the run, and it writes the solution to the output file, or to the standard output when no output file is given. The reports go to the standard error, so that the standard output holds the solution alone.

Option	Meaning	Default
`-q`	print no report	off
`-int`	`int` for capacities, costs and supplies	on
`-long`	`long long` (only where the compiler has it)	off
`-double`	`double`	off
`-ldouble`	`long double`	off
`-infcap v`	the value of an infinite capacity (§21.5)	`0`, i.e., the infinity or the maximum of the type

Table 22.1. The options of `dimacs-solver`; the four options of the type form an exclusive group.

At the start, the tool reads the problem line by `dimacsType()`, and, unless `-q` is given, writes the type of the problem and the numbers of nodes and arcs. A file that cannot be opened, of unknown type, or a value of `-infcap` that cannot be read in the chosen type, ends the program with a message and the exit status 1. Then it builds a `SmartDigraph` (or a `SmartGraph` for the type `mat`) and proceeds by type, as follows. For a `sp` file, it reads the digraph and the lengths by `readDimacsSp()`, computes the shortest paths from the source by `Dijkstra` (chapter 10), and writes a line `d v d(v)` for each node `v` reached. For a `max` file, it reads the capacities by `readDimacsMax()`, computes a maximum flow by `Preflow` (chapter 12), and writes its value as `s value` and a

line `f u v x` for each arc (u, v) with a nonzero flow x. For a `mat` file, it reads the graph by `readDimacsMat()`, computes a maximum matching by `MaxMatching` (chapter 17), and writes its cardinality as `s k` and a line `m u v` for each matched edge. The nodes are numbered from 1, as in the input file.

For a `min` file, the tool reads the problem by `readDimacsMin()` and solves it by `NetworkSimplex` (chapter 13), as in Algorithm 22.1. It writes the cost as `s cost` and a line `f u v x` for each arc with a nonzero flow, or `s INFEASIBLE` or `s UNBOUNDED`; the cost is computed in a wider type (`long long` when the values are `int`, the type of the values otherwise). Its report says whether a feasible flow was found and gives the cost.

Algorithm 22.1 (The minimum cost flow problem in `dimacs-solver`).

```
1  read lower, capacity, cost, supply (readDimacsMin, infinite capacity =
   the value of -infcap)
2  sigma := sum of the supplies
3  eps := the data tolerance of NetworkSimplex for the largest magnitude
          of the supplies, of the lower bounds and of the finite capacities
4  if |sigma| <= eps: sigma := 0
5  if sigma > 0: supply type LEQ, else GEQ (the default of NetworkSimplex)
6  run NetworkSimplex; report "found" if the result is OPTIMAL, "not found"
   otherwise, and the total cost; write the cost and the flows (or the cause)
   to the output
```

The choice of line 5 rests on the following property of the flows, in which $b(v)$ is the supply of the node v (negative for a demand) and $\sigma = \sum_v b(v)$.

Lemma 22.1. *For any flow x on the arcs, the excesses $e(v) = \sum_{a \in \delta^+(v)} x_a - \sum_{a \in \delta^-(v)} x_a$ sum to zero. Hence, the constraints $e(v) = b(v)$ have no solution when $\sigma \neq 0$; the constraints $e(v) \leq b(v)$ (type `LEQ`) can have one only if $\sigma \geq 0$, and $e(v) \geq b(v)$ (type `GEQ`) only if $\sigma \leq 0$; when $\sigma = 0$, both are equivalent to the equations.*

Proof. Each arc contributes $+x_a$ to the excess of its tail and $-x_a$ to that of its head, which gives $\sum_v e(v) = 0$. If $e \leq b$, then $0 = \sum_v e(v) \leq \sigma$, and symmetrically for $\geq$. If $\sigma = 0$ and $e \leq b$, then $\sum_v (b(v) - e(v)) = 0$ with nonnegative terms, and hence $e = b$; symmetrically for $\geq$. □

Thus, with a total supply larger than the total demand the problem can only be feasible if some supply is left unused, which the type `LEQ` allows; with a smaller one, it needs the type `GEQ`, under which some demand stays unsatisfied. Lines 3 and 4 treat as zero a sum that only the rounding of fractional supplies keeps away from zero, with the tolerance that `NetworkSimplex` itself uses to balance the supplies (chapter 13). We remark that the report of line 6 calls an unbounded problem "not found" and still prints a total cost for it, which is not meaningful; the output file, instead, says `s UNBOUNDED`.

abLEMON. `dimacs-solver` takes as zero a sum of the supplies within the tolerance of `NetworkSimplex`; in LEMON 1.3.1 it compared the sum with zero exactly, and a sum of 10^{-12} left by fractional supplies of a `double` instance made it choose and announce the type `LEQ`. Also, in LEMON 1.3.1 it wrote no solution (the output file, opened when given, stayed empty), wrote the first lines of its report to the standard output, and ended with an uncaught exception on a file of unknown type.

22.3 dimacs-to-lgf

Next, `dimacs-to-lgf` converts a DIMACS file into an LGF file (§21.1), and it has no option besides the input and the output file. It reads the problem by `dimacsType()` and by the reader of its type, all values being of type `double`, into a `SmartDigraph`, and it writes the digraph by `DigraphWriter` with the maps and attributes of Table 22.2; the nodes are labeled by their ids, i.e., by the numbers of the DIMACS file minus one.

Type	Node maps	Arc maps	Attributes
`min`	`supply`	`lower`, `capacity`, `cost`	`problem` = `min`
`max`		`capacity`	`source`, `target`, `problem` = `max`
`sp`		`capacity` (the lengths)	`source`, `problem` = `sp`
`mat`			`problem` = `mat`

Table 22.2. The contents of the LGF file written by `dimacs-to-lgf`, by type of the DIMACS file.

Two remarks are in order. First, the infinite capacities of a `min` or `max` file are written as the infinity of `double`, i.e., as the token `inf`; whether a program can read this token back depends on the `operator>>` of its standard library, and it may need a converter (§21.2). Second, the values are written with the default conversion, i.e., with 6 significant digits (§21.3), which is exact for the integer data of the usual DIMACS instances below 10^6 but not for larger or fractional values. Unlike `dimacs-solver`, the tool catches no exception: a file that cannot be opened, a file of unknown type (on which `dimacsType()` throws `FormatError`), or a `max` or `sp` file without its source or target (which the writer cannot label) ends it with an uncaught exception.

22.4 lgf-gen

Finally, `lgf-gen` generates random plane graphs, i.e., graphs whose nodes are points of the plane, and writes them as LGF files with the coordinates of the nodes, optionally with an EPS drawing. It is a convenient source of test instances for the geometric and graph algorithms of the library.

Option	Meaning	Default
`-n N`	number of nodes	100
`-disc`, `-square`, `-gauss`	the nodes uniformly in the unit disc, uniformly in the square $[-1,1]^2$, or by a two-dimensional Gaussian distribution	`-disc`
`-cities k`	the nodes in k clusters, with random centres and sizes	1
`-area a`	the scale of the clusters (their full relative area)	1
`-2con`, `-tree`, `-tsp`, `-tsp2`, `-dela`	the edges: a planar graph built by short disjoint paths, a minimum spanning tree, a TSP tour, a tree-based TSP tour, the Delaunay triangulation	no edges
`-g d`	the girth parameter of `-2con`	10
`-dir`	write a digraph, each edge as two opposite arcs	off
`-eps`	also write `prefix.eps`	off
`-nonodes`	draw only the edges in the EPS file	off
`-rand`, `-seed s`	seed the generator with the time, or with `s`	the default seed

Table 22.3. The options of `lgf-gen`; the distributions, the algorithms and the two ways of seeding form three exclusive groups. The only file argument is the prefix of the output files, `lgf-gen-out` by default.

Thus, the nodes are placed first. With k clusters, each cluster s receives a random weight w_s, uniform in $[0,1)$, and a share of the N nodes proportional to it; its centre is a random point of the unit disc (the origin when $k=1$), and its points are drawn by the chosen distribution and scaled by $a\sqrt{w_s/\sum_r w_r}$. Then the edges are built by the chosen algorithm, the lengths being the Euclidean distances, as follows.

Algorithm 22.2 (The edges of `lgf-gen`).

```
-dela : the Delaunay triangulation of the points (delaunay() of the tool)
-tree : the Delaunay triangulation; keep the spanning tree that
        kruskal() finds on the squared lengths
-tsp  : the cycle of the nodes in the order of creation, improved by
        2-opt moves of gain larger than 1e-8 until none applies
-tsp2 : the tree of -tree; scan the pairs of its nodes of odd degree by
        increasing distance, and join a pair when both ends are still odd;
        shortcut the Euler tour of the result to a tour; improve it by 2-opt
-2con : for each pair of nodes by increasing length: add the edge if it
        crosses no edge already added and the two nodes do not have two
        edge-disjoint paths of total length at most d (Suurballe, unit
        lengths); then, in reverse order of addition, remove each edge
        whose ends keep two such paths without it
```

The algorithm `-2con` takes d from `-g`, and `Suurballe` is the algorithm of chapter 10. At the end, the tool prints the number of nodes and edges and the total length of the edges, and, for `-tsp` and `-tsp2`, the number of 2-opt moves; these reports go to the standard output. It writes `prefix.lgf`, by `GraphWriter`, or by `DigraphW`

`riter` with `-dir`; the only node maps are `coordinates_x` and `coordinates_y`, the coordinates multiplied by 600. With `-eps`, it also writes `prefix.eps` by `graphToEps()` (§21.7), scaled to an A4 page, without prescaling, with the node scale 0.005 and the arc width scale 0.001. The help of the tool calls the graph of `-2con` "two connected"; what the procedure guarantees is the following.

Lemma 22.2. *(i) The graph of `-2con` has no two crossing edges. (ii) In `-tsp2`, every node has even degree after the pairing. (iii) The 2-opt phases of `-tsp` and `-tsp2` terminate after at most $\ell_0/10^{-8}$ moves, where ℓ_0 is the length of the starting tour.*

Proof. (i) An edge is added only if it crosses no edge already present, and the second phase only removes edges. (ii) An edge is added only between two nodes that are odd at that moment, which makes both even; hence a node that becomes even stays even. If two nodes a and b were odd at the end, they would have been odd throughout the scan, and the pair $\{a, b\}$, which is scanned, would have been joined. (iii) Each move decreases the length of the tour by more than 10^{-8}, and the length stays nonnegative. □

The `-2con` graph is not necessarily 2-connected: an edge is left out when its ends already have two edge-disjoint short paths, and a later removal may destroy one of them. Its construction scans the $N(N-1)/2$ pairs, and each of them costs a scan of the edges for the crossings and possibly a run of `Suurballe`, so that it is by far the slowest of the five algorithms.

22.5 lemon-0.x-to-1.x.sh and the demos

We now turn to `lemon-0.x-to-1.x.sh`, which helps to port a program written for the old series 0.x of LEMON: it rewrites in place each file given as an argument by a long sequence of `sed` substitutions, and with no argument, or with `-h`, `-help` or `--help`, it prints its usage. The substitutions rename the graph concepts of the old series: a graph becomes a digraph, an undirected graph a graph, an edge an arc and an undirected edge an edge, and the `A`- and `B`-nodes of the bipartite graphs become red and blue nodes. They also rename the headers (e.g., `digraph_adaptor.h` becomes `adaptors.h` and `topology.h` becomes `connectivity.h`), the named template parameters `Def*` into `Set*`, the adaptors into their current names (e.g., `SubDigraphAdaptor` becomes `SubDigraph` and `ResDigraphAdaptor` becomes `ResidualDigraph`), several maps (e.g., `InvertableMap` becomes `CrossRefMap`), and the LP classes (`LpGlpk` becomes `GlpkLp`, and so on). Since the substitutions are textual, the result must be read and compiled; in particular, the words *graph* and *edge* in the comments are renamed too.

Each of the three demos of `demo/` is a short program that shows the use of one tool of the library. First, `arg_parser_demo` declares options of all the kinds of `ArgParser` (a mandatory integer `-n`, doubles with and without a reference, a synonym, a string, switches, an exclusive and mandatory group of three options and the file arguments), parses the command line with `throwOnProblems()`, and prints what it found. Then `graph_to_eps_demo` builds a small digraph and writes seven EPS files, `graph_to_eps_demo_out_1_pure.eps` to `graph_to_eps_demo_out_7_colors.eps`, which show the plain drawing, the sizes, shapes, colors and widths given by maps, the arrowheads, the parallel arcs, the fit to an A4 page and, on a second graph, the colors of a `Palette` with the texts of the nodes in distant colors. Finally, `lgf_demo` reads the digraph, the capacities and the source and target of the file `digraph.lgf` of the directory of the demos, which it must be started from, and writes them back on the standard output.

22.6 An example

The program `manual/examples/22-tools.cc` (the test `manual_22_tools`) does, on a small minimum cost flow problem, what two of the tools do. It declares the options of `dimacs-solver` and parses a given command line with them, reads a DIMACS `min` file from a string with an arc of infinite capacity, chooses the supply type by the sign of the sum of the supplies as in Algorithm 22.1 (the supplies are integer, so that the tolerance of line 4 plays no role), and solves the problem by `NetworkSimplex`. Then it writes the problem as `dimacs-to-lgf` writes it, reads the LGF text back, and checks the costs and the attribute `problem`.

22.7 Exercises

Exercise 22.1. Generate a Delaunay triangulation with `lgf-gen -dela -n 1000`, read it by `GraphReader` with the coordinates, and check that its number of edges is at most $3N - 6$, as for any simple plane graph (and that `PlanarityChecking` of chapter 18 accepts it).

Exercise 22.2. Generate a TSP tour with `lgf-gen -tsp` and with `lgf-gen -tsp2` on the same seed, and compare their lengths with the tours of the classes of chapter 19 on the same points (read by `GraphReader`, with the costs given by the coordinates).

Exercise 22.3. Write a `min` file whose supplies sum to a positive value, and check that `dimacs-solver` announces the type `LEQ` and finds the same cost as a program that solves the problem by `NetworkSimplex` with this supply type (`manual/examples/22-tools.cc`).

Exercise 22.4. Convert a `max` file with an arc of negative capacity by `dimacs-to-lgf`, and read the result back by `DigraphReader` with a converter that maps the token `inf` to a large finite capacity; check that `Preflow` gives the same value as `dimacs-solver` on the original file.

Exercise 22.5. Look for a seed on which `lgf-gen -2con` gives a graph that is not 2-connected, as the remark after Lemma 22.2 allows, testing the graphs with `biNodeConnected()` of chapter 18.

Exercise 22.6. Run `arg_parser_demo` without the mandatory option `-n`, with two options of the group `-gra`, `-grb`, `-grc`, and with none of them, and explain each message from the declarations of the demo.

A. Writing a graph, a map or a heap

We are concerned in this appendix with the classes that a user may add to the library. These are typically a graph structure that computes its items from some private data (an implicit graph, or a view of another data structure), a map that computes its values on demand, an observer that follows the changes of a graph, and a heap. Such a class is accepted by the algorithms as soon as it conforms to the concept of its kind; also, most of the work is done by the classes of `lemon/bits/`, which turn a small base class into a complete graph. Section A.1 states the concepts as contracts, and section A.2 describes the base class of a graph and the extenders that complete it. Section A.3 deals with the alteration notifier through which a graph informs its maps, sections A.4 and A.5 with the maps and the heaps, and section A.6 with the checking of a new class. The program `manual/examples/A-cycle-digraph.cc` (the test `manual_A_cycle_digraph`) writes a digraph, a map and an observer in the way described here, and runs `Dijkstra` on the first two.

A.1 The concepts as contracts

A concept class of `lemon/concepts/` (cf. chapter 3) lists what a generic algorithm may use, and hence what a new class must provide. For a digraph this is `concepts::Digraph`, i.e., the base, idable, iterable and mappable components of chapter 4, while an undirected graph conforms to `concepts::Graph` and a bipartite one to `concepts::BpGraph`. A structure that can grow, shrink or be cleared also conforms to the extendable, erasable and clearable components. Each of them requires the alterable one, since a structure that changes must notify its maps. However, the concepts state only the syntax; some semantic requirements are written only in their documentation, and the algorithms rely on them as well.

- The identifiers of the items of one kind are distinct and immutable, and they lie between 0 and `maxNodeId()` (or the other maxima). The maximum should not be much larger than the number of items, since the standard maps allocate one slot for each identifier up to it.
- Each iterator visits each item exactly once and then becomes `INVALID`. In an undirected graph `ArcIt` visits each of the two arcs of an edge once, and `InArcIt` and `OutArcIt` visit the arcs of the incident edges.
- The ends `u(e)` and `v(e)` do not change while the edge exists, and `direct(e, true)` goes from `u(e)` to `v(e)`.

For a map the contracts are `concepts::ReadMap< K , V >`, `concepts::WriteMap< K , V >`, `concepts::ReadWriteMap< K , V >` and `concepts::ReferenceMap< K , V , R , CR >` of `lemon/concepts/maps.h` (chapter 6), and for a heap `concepts::Heap< PR , IM , CMP >` of `lemon/concepts/heap.h` (section A.5). Each concept class has a nested template `Constraints`, whose function `constraints()` is the executable form of the contract that `checkConcept` instantiates (section A.6).

A.2 The base class and the extenders

Most of the interface of a graph does not depend on how the graph is stored, or depends on it only somewhat. Indeed, the iterators can be written once from functions such as `first(Node &)` and `next(Node &)`, the standard maps need only the identifiers and a notifier, and the ranges, `oppositeNode()`, `baseNode()` and `runningNode()` follow from the iterators. Hence, the header `lemon/bits/graph_extender.h` defines three class templates, `DigraphExtender< Base >`, `GraphExtender< Base >` and `BpGraphExtender< Base >`, which derive from a base class written by the user and add this common part. All the structures of chapter 4 are made in this way (e.g., `ListDigraph` derives from `DigraphExtender< ListDigraphBase >`). So are the arc and edge sets, by `ArcSetExtender` and `EdgeSetExtender` of `lemon/bits/edge_set_extender.h`, and the adaptors of chapter 5, by `DigraphAdaptorExtender` and `GraphAdaptorExtender` of `lemon/bits/graph_adaptor_extender.h`.

The base of a digraph. A base class for `DigraphExtender` defines the following members, which are those of `BaseDigraphComponent`, `IDableDigraphComponent` and the low-level part of `IterableDigraphComponent`.

- The handle types `Node` and `Arc`, with default and `Invalid` constructors, `==`, `!=` and `<`.
- The functions `source(a)` and `target(a)`.
- The functions `first(Node &)`, `next(Node &)`, `first(Arc &)` and `next(Arc &)`, which enumerate the items and set the handle to the invalid value after the last one. Also, `firstOut(Arc &, Node)`, `nextOut(Arc &)`, `firstIn(Arc &, Node)` and `nextIn(Arc &)` enumerate the arcs at a node.
- The functions `id(Node)`, `id(Arc)`, `nodeFromId(int)`, `arcFromId(int)`, `maxNodeId()` and `maxArcId()`.

If the structure can change, the base also has `addNode()`, `addArc(s, t)`, `erase(Node)`, `erase(Arc)` and `clear()`, which modify the storage only, since the extender adds the notifications. A static structure has instead `build(g, nodeRef, arcRef)` and the tag `BuildTag`. Optionally, the tags `NodeNumTag` and `ArcNumTag`, with `nodeNum()` and `arcNum()`, make the counting of chapter 4 take $O(1)$ time, and `FindArcTag`, with `findArc(s, t, prev)`, replaces the scan of `findArc()`.

The extender. On top of the functions above, `DigraphExtender< Base >` defines `NodeIt`, `ArcIt`, `OutArcIt` and `InArcIt` (Algorithm A.1), with the ranges `nodes()`, `arcs()`, `outArcs(v)` and `inArcs(v)`. It also defines `baseNode()`, `runningNode()`, `oppositeNode()`, `maxId()` and `fromId()`, and one `AlterationNotifier` for the nodes and one for the arcs, returned by `notifier(Node())` and `notifier(Arc())`. Its map templates `NodeMap< V >` and `ArcMap< V >` are `DefaultMap`s with the iterators of `MapExtender` (cf. chapter 6). Finally, its functions `addNode()`, `addArc()`, `erase()`, `clear()` and `build()` call those of the base and notify the observers in the order of section A.3; in particular, `erase(Node)` erases the arcs at the node first, one by one, and the base need not do it, while its destructor notifies the observers of a `clear()`.

For an undirected graph, `GraphExtender< Base >` requires from the base the type `Edge` as well, with `u(e)`, `v(e)`, `direction(a)`, `direct(e, bool)`, `first(Edge &)`, `next(Edge &)`, `firstInc(Edge &, bool &, Node)`, `nextInc(Edge &, bool &)`, `id(Edge)`, `edgeFromId()` and `maxEdgeId()`. In return it defines the tag `UndirectedTag`, `EdgeIt`, `IncEdgeIt`, `edges()`, `incEdges(v)`, `oppositeArc()`, `direct(e, node)`, a third notifier for the edges and `EdgeMap< V >`, and its `addEdge()` notifies the new edge and then its two arcs. In the same way, `BpGraphExtender< Base >` adds the red and the blue nodes, with their iterators, notifiers and maps.

Algorithm A.1 (The arc iterators that DigraphExtender derives from the base).

```
ArcIt(g):          g.first(a)
++a:               g.next(a)
OutArcIt(g, v):    g.firstOut(a, v)
++a:               g.nextOut(a)
InArcIt(g, v):     g.firstIn(a, v)
++a:               g.nextIn(a)
a != INVALID:      Arc::operator!= of the base, against Arc(INVALID)
```

Lemma A.1. *If the enumerations `first()` and `next()` (and `firstOut()`, `nextOut()`, `firstIn()`, `nextIn()` at a node v) of the base visit each item of their set once and then set the handle to the value of `Arc(INVALID)` (or `Node(INVALID)`), then the iterators of Algorithm A.1 satisfy the contract of section A.1 on the same sets, and each step of an iterator costs one step of the base.*

Proof. An iterator is a handle of the base, which its constructor sets by `first()` (or `firstOut()`, `firstIn()`) and its `operator++` advances by `next()` (or `nextOut()`, `nextIn()`); hence, the sequence of the values of the iterator is the enumeration of the base, and the comparison with `INVALID`, which converts to the handle of the base, becomes true after the last item. □

The digraph of the program of this appendix, the directed cycle on n nodes, is made in this way (Figure A.1): its base class `CycleDigraphBase` stores only n and computes the arc i as going from node i to node $(i+1) \bmod n$, `CycleDigraph` derives from `DigraphExtender< CycleDigraphBase >`, and its `resize(n)` notifies the maps that all the items were erased and that the new ones were built, as `FullDigraph::resize()` does. The program checks the digraph against `concepts::Digraph` and runs `Dijkstra` on it, with no change to the algorithm.

A.3 The alteration notifier

A graph informs the objects that depend on its items of each change through one notifier for each kind of item. This is an object of the class `AlterationNotifier< Container , Item >` of `lemon/bits/alteration_notifier.h`, which keeps a list of observers, i.e., of objects of classes derived from its nested class `ObserverBase`. An

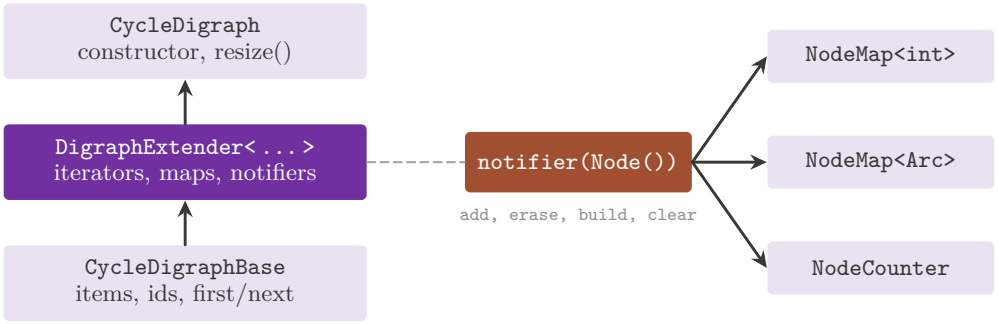


Figure A.1. The layers of the digraph of the program: the base class computes the items, DigraphExtender adds the iterators, the maps and the notifiers, and the notifier of the nodes informs the observers attached to it (the node maps among them).

observer is attached to a notifier by the protected constructor `ObserverBase( notifier )` or by `attach( notifier )`, and it is detached by `detach()` or by its destructor; `notifier()` and `attached()` give its notifier and its state (these functions are protected as well, i.e., for the derived class), and a copy of an observer is attached to the same notifier. An observer overrides six pure virtual functions, which the notifier calls.

- `add(item)` and `add(items)`, after one item or a `std::vector` of items has been added to the container.
- `erase(item)` and `erase(items)`, before one item or several items are erased from it.
- `build()`, after the container has been built from empty, and `clear()`, before all its items are erased.

A graph calls the functions `add()`, `erase()`, `build()` and `clear()` of its notifier in a fixed order with respect to its storage. An item is added to the storage first and notified afterwards, so that the observer can read it (e.g., its identifier), while an item to be erased is notified first and removed afterwards. The notifier also lets an observer enumerate the current items by `first(item)` and `next(item)`, and read their identifiers by `id(item)` and the bound on them by `maxId()`. We remark that an observer must not trigger another alteration of the same notifier, nor detach another observer, from within these functions.

Exceptions. The calls of `add()` and `build()` go from the observer attached first to the one attached last. If an observer throws, those already notified receive `erase()` (or `clear()`) for the same items, and the exception is propagated (Algorithm A.2). The calls of `erase()` and `clear()` go from the last attached to the first, and they must not throw, with one exception (the `erase()` of a single item is `noexcept`, so that any other exception there terminates the program). An observer may throw `ImmediateDetach`, after which the notifier detaches it and goes on with the others. The `Snapshot` of a list graph uses it when an item older than the snapshot is erased, and so do the `Smart` arc and edge sets when an erased node has arcs in them (chapter 4).

Algorithm A.2 (Notifying an addition, with rollback, and an erasure).

```
add(x):
 1  for each observer o, from the first attached to the last:
 2    try o.add(x)
 3    on an exception:  for each observer o' notified before o:
 4                        o'.erase(x)
 5                      rethrow the exception
erase(x):
 6  for each observer o, from the last attached to the first:
 7    try o.erase(x)
 8    on ImmediateDetach:  detach o and continue
```

Lemma A.2. *Let the notifier have k observers, and let the `erase(x)` of each observer undo its `add(x)`. If `add(x)` of Algorithm A.2 throws, then every observer has either received nothing or received `add(x)` followed by `erase(x)`, and the exception reaches the caller; `add(x)` and `erase(x)` make at most $2k$ and k calls of the observers, and after `erase(x)` exactly the observers that threw `ImmediateDetach` are detached.*

Proof. The observers are kept in a list in which each new one is put at the front, and `add(x)` visits it from the back, i.e., in the order of attachment. When the observer o throws, those visited before it are those that received `add(x)`, and lines 3 and 4 send each of them `erase(x)`, which by hypothesis undoes its `add(x)`; o itself and the later ones received nothing. Each observer is called at most once on line 2 and at most once on line 4. The loop

of `erase(x)` visits each observer once from the front; one that throws `ImmediateDetach` is unlinked from the list and marked as not attached before the loop goes on to its successor, and the others stay. □

Lemma A.2 concerns the observers only. The extender of section A.2 adds the item to the storage of the base before it notifies it, and it does not remove the item when the notification throws; hence, a graph whose `addArc ()` fails in one of its maps (e.g., for lack of memory) has its maps rolled back, but the arc remains in its storage, with no slot in the maps, and the graph should not be used further.

Thread safety and limits. Attaching and detaching go through a lock, the class `bits::Lock` of `lemon/bits/lock.h`. It is a mutex of POSIX threads or of Windows when the library is configured with `LEMON_THREADING` set to `Pthread` or `Win32` (cf. chapter 2), which defines `LEMON_USE_PTHREAD` or `LEMON_USE_WIN32_THREADS` in `lemon/config.h`, and an empty class otherwise; thus, maps may be created and destroyed on one graph from several threads, while the notifications are not locked. The header also remarks that the notifications are not always complete. A reversal of an arc by `reverseArc()` is not notified (the observers see neither an erasure nor an addition), nor are the other changes of the ends of an arc or the splitting of a node of chapter 4; therefore, the degree maps `InDegMap` and `OutDegMap` of chapter 6, which observe the arcs, may become wrong after them. Also, a filter adaptor of chapter 5 cannot notify the change of its filter map at all.

The program of this appendix includes `NodeCounter`, an observer of the nodes of a `ListDigraph` that keeps their number by `build()`, `add()` and `erase()`, and checks it against `countNodes()` after an addition and an erasure.

A.4 Writing a map

A map that an algorithm only reads is a class with the types `Key` and `Value` and a constant `operator[]( const Key & )`, which returns a `Value` (or a reference to one). A map that the algorithm writes also has `set( const Key & , const Value & )`. No other member is needed; in fact, the map need not store its values and need not know the graph, unless it computes from it. It is passed by reference to the algorithm, which keeps a pointer to it (and therefore requires it to outlive the algorithm). For instance, the class `LengthByIndex` of the program of this appendix gives each arc of `CycleDigraph` the length $1 + (i \bmod 3)$ from the identifier i of the arc, and `Dijkstra< CycleDigraph , LengthByIndex >` reads it in place of an arc map. A map may derive from `MapBase < K , V >` of `lemon/maps.h`, which only defines the two types. Also, a reference map defines the types `Reference` and `ConstReference` and the tag `typedef True ReferenceMapTag`, by which `MapTraits` (cf. chapter 3) recognizes it.

A map that stores a value for each item of a graph, and must therefore follow its changes, is an observer of the notifier of that kind of item: its `add()` and `erase()` make room for the new items and release the erased ones. The standard maps of the extenders, `VectorMap` and `ArrayMap` (described in chapter 6), are written in this way, and so are the degree maps of `lemon/maps.h`, while `CrossRefMap` and `RangeIdMap` derive from a standard map and extend its observer functions. A new graph structure obtains the standard maps from its extender; it usually needs a specific map only for a different storage (e.g., a map of the arcs of an implicit graph that stores values only for some of them).

A.5 Writing a heap

The algorithms that keep a priority queue of nodes (`Dijkstra`, `BidirDijkstra`, `AStar`, `MinCostArborescence` and several others of Part III) take the type of their heap from their traits. A user may replace it by the named template parameter `SetHeap< H , CR >` (cf. chapter 3), with a heap that conforms to `concepts::Heap< PR , IM , CMP >`. The parameters of the concept are the type `PR` of the priorities, the type `IM` of a reference map from the items to `int` (i.e., one whose `operator[]` gives a reference, through which the heap writes), and the comparison functor `CMP`, by default `std::less< PR >`. Through the map `IM`, called the cross reference map, the heap finds the place of an item in $O(1)$ time, and its key type `IM::Key` is the type of the items. The concept requires the following members, which one can compare with those of the heaps of the library.

- The types `Item`, `Prio`, `ItemIntMap` and `State`, the last one an enumeration with `IN_HEAP` $= 0$, `PRE_HEAP` $= -1$ and `POST_HEAP` $= -2$.
- The explicit constructors `Heap( IM & map )` and `Heap( IM & map , const CMP & comp )`, and a copy constructor (the `Constraints` of the concept copy a heap).
- The functions `size()`, `empty()` and `clear()`, the last of which empties the heap without touching the cross reference map.

- The functions `push(i, p)` for an item not in the heap, `top()` and `prio()` for the item of least priority and its priority, `pop()`, which removes that item, and `erase(i)` for an item in the heap.
- The query `heap[i]`, the priority of an item in the heap, and the functions `set(i, p)`, which inserts the item or changes its priority, and `decrease(i, p)` and `increase(i, p)`, whose precondition is that the new priority is not larger (resp. smaller) than the old one.
- The functions `state(i)`, which returns `PRE_HEAP` for an item that has never been in the heap, `IN_HEAP` for one in it and `POST_HEAP` for one removed, and `state(i, st)`, which sets the state of an item that is not in the heap.

The cross reference map is owned by the caller and shared with the heap. It must give `PRE_HEAP` for each item that may be pushed, and the heap typically writes into it the position of each item in its storage (a nonnegative value) and `POST_HEAP` after its removal. For instance, `Dijkstra::init()` calls `clear()` on its heap and then sets the cross reference of each node to `PRE_HEAP`, in $O(n)$ time. The implementations of the library, their costs, and the test of all of them against the concept (`test/heap_test.cc`) are given in chapter 7.

A.6 Checking a new class

A new class is checked at compile time by `checkConcept< Concept , T >()` of `lemon/concept_check.h`, as the program of this appendix does for `CycleDigraph` (against `concepts::Digraph`) and for `LengthByIndex` (against `concepts::ReadMap< CycleDigraph::Arc , int >`). The call instantiates the function `constraints()` of the concept on the class, and a missing type or function (or one with an incompatible signature) gives a compilation error at that point. It has no cost at run time, and it is empty when `NDEBUG` is defined (in which case no check is made). A structure that offers more than the concept is also checked against the components it adds, e.g., `checkConcept< concepts::ExtendableDigraphComponent<> , G >()` and the erasable, clearable and alterable components, as `test/digraph_test.cc` does for the structures of the library.

Since the concepts check the syntax only, the semantic requirements of section A.1 are checked by running the class. In the tests, the header `test/graph_test.h` has function templates that do it for any graph type, and a test of a new structure may call them. Thus, `checkGraphNodeList(g, n)` and `checkGraphArcList(g, m)` count the items visited by the iterators, `checkGraphOutArcList(g, v, d)` and `checkGraphInArcList(g, v, d)` the arcs at a node, and `checkGraphIncEdgeArcLists(g, v, d)` the incidences of an undirected graph. Also, `checkNodeIds(g)` and `checkArcIds(g)` verify that the identifiers are distinct and that `nodeFromId()` inverts them, `checkArcDirections(g)` that the arcs of each edge are directed consistently, and `checkGraphNodeMap(g)` and `checkGraphArcMap(g)` that a map has a value for each item. Finally, an algorithm run on the new structure should give the results known in closed form or computed on a copy of it into a `ListDigraph` (by `digraphCopy()` of chapter 4); the program of this appendix, e.g., checks the distances of `Dijkstra` against their closed form, and a node map against a `resize()`.

Y. Glossary

The terms defined in the chapters, each with the section that defines it.

2-optimal tour: a tour that no 2-opt move, i.e., the replacement of two of its edges by the two other edges that reconnect it, makes cheaper (§19.3).

active item: in an elevator, an item marked as active, e.g., a node with a positive excess in a push-relabel algorithm; in the Bellman-Ford algorithm, a node whose distance changed in the last round (§8.3, §10.4).

admissible arc: in a push-relabel or cost scaling algorithm, a residual arc along which flow may be pushed: one whose head is on a lower level than its tail (maximum flow), or one of negative reduced cost (cost scaling) (§12.2, §13.6).

admissible heuristic: a heuristic h of A* with $h(v) - h(t) \leq \delta(v, t)$ for each node v, i.e., one that never overestimates the distance of a node to the target when $h(t) = 0$ (§10.3).

alteration notifier: the object of a graph, one for each kind of item, that informs the attached observers (the maps among them) of each item added or erased, and of the graph being built or cleared (§A.3).

alternating path: a path whose edges are alternately in a matching and out of it; it is augmenting when neither of its ends is covered by the matching (§17.1).

amortized cost: a bound on the total cost of any sequence of operations from the empty structure, divided among its operations, e.g., by means of a potential function (§7.1).

arborescence: a set of arcs in which each node reachable from the root, the root excepted, has one entering arc and can be reached from the root (§11.2).

arc set: a digraph (or, as an edge set, a graph) that has its own arcs on the nodes of another graph, such as ListArcSet and SmartEdgeSet (§4.7).

archetype: a concept class whose members are declared with empty bodies, so that a template that compiles with it uses only what the concept guarantees (§3.5).

assignment: a matching of a bipartite graph that covers every red node; MinCostAssignment finds one of least cost (§17.3).

augmenting path: a directed path from the source to the sink in the residual digraph of a flow; sending along it its smallest residual capacity increases the value of the flow (§5.4).

awake set: in the Hao-Orlin algorithm, the nodes among which the preflow is pushed towards the current sink, as opposed to the sources and to the dormant sets (§15.2).

back arc: in a depth-first search of a digraph, an arc whose target has been reached and not yet left; a digraph is a DAG if and only if no search meets one (§18.3).

barrier (circulation): a set of nodes whose supply exceeds what the upper bounds of the arcs leaving it, less the lower bounds of the arcs entering it, can carry, which proves a circulation problem infeasible (§12.4).

barrier (matching): a set of nodes whose removal leaves more odd components than it has nodes, which bounds from below the number of nodes uncovered by any matching; for fractional matchings, a set S with $|S| - |N(S)|$ as large as possible (§17.1).

base node: the node on which an incidence iterator was constructed; the other end of its current arc or edge is the running node (§3.3).

basis: in the network simplex method, a spanning tree of the arcs together with the bound at which each other arc lies; a basis of a vertex of the polyhedron of the flows is a spanning tree containing its free arcs (§13.4, §14.4).

bi-edge-connected component: a class of the nodes of an undirected graph joined by two edge-disjoint paths; the components are separated by the bridges (§18.2).

bind arc: the arc of a SplitNodes adaptor from the in-node to the out-node of a node of the digraph, which carries the data of the node (§5.5).

binomial heap: a heap made of a list of heap-ordered binomial trees of distinct degrees, like the binary digits of the number of items (§7.3).

bipartite graph: a graph whose nodes split into two sets with no edge inside either of them, i.e., a graph without cycles of odd length (§18.1).

block: a bi-node-connected component of an undirected graph, i.e., a class of the edges that lie on a common cycle; the blocks are separated by the cut nodes (§18.2).

blossom: an odd cycle closed by an alternating search tree, which Edmonds' algorithms shrink into a single node and expand later (§17.4).

breadth-first search: the visit of the nodes reachable from the sources in the order of their distance, i.e., of the least number of arcs of a path from a source (§9.2).

bridge: an edge whose removal increases the number of connected components of a graph, also called a cut edge (§18.2).

bucket heap: a heap of non-negative integer priorities that keeps a list of the items of each priority in a vector indexed by the priority (§7.4).

caption: the optional name that follows the type in the header line of an LGF section, by which a reader selects one of several sections of the same type (§21.1).

cascading cut: in a Fibonacci heap, the cut of a node that has lost a second child since it became a child, repeated upwards (§7.3).

clique: a set of pairwise adjacent nodes of an undirected graph (§19.5).

closed set (residual digraph): a set of nodes that no residual arc of a flow leaves; for a maximum flow, the minimum cuts are the closed sets that contain the source and not the target (§12.1, §14.1).

combinatorial embedding: the cyclic order of the edges around each node of a planar graph, which describes a plane drawing up to deformation (§18.5).

concept: the list of the types and functions that a class of a kind (graph, map, heap) must provide, written as a class of lemon/concepts/ and checked by checkConcept (§3.5).

condensation: the acyclic digraph with one node per strongly connected component of a digraph and an arc between two components whenever an arc of the digraph joins them (§18.3).

consistent heuristic: a heuristic h of A* with $h(u) \leq \ell(uv) + h(v)$ on each arc, with which each node is scanned at most once (§10.3).

core number: the largest k such that a node belongs to a subgraph of minimum degree k; the largest core number is the degeneracy of the graph (§6.6).

critical arc: an arc of an augmenting path whose residual capacity equals the bottleneck, and which the augmentation removes from the residual digraph (§5.4).

cross reference map: the map from the items to integers that a heap shares with its user, through which it finds the place of an item and records its state (PRE_HEAP, IN_HEAP, POST_HEAP) (§A.5).

cross-reference map: the map from the items to int through which a heap finds an item in its storage, holding its position while the item is in the heap and its state otherwise (§7.1).

cut: a set of nodes containing the source and not the target; its capacity is the sum of the capacities of the arcs that leave it (§12.1).

cut node: a node whose removal increases the number of connected components of a graph, also called an articulation node (§18.2).

d-ary heap: a heap stored as a complete tree in which each node has at most d children, laid out level by level in a vector (§7.2).

DAG: a directed acyclic graph, i.e., a digraph without directed cycles (§18.3).

degeneracy: the largest least degree of a subgraph of an undirected graph, i.e., the largest core number of its nodes (§6.6).

degenerate vertex: a vertex of the polyhedron of the flows whose free arcs do not span the nodes, and which therefore has more than one basis (§14.4).

depth-first search: the visit that advances from the last node reached along an arc not yet scanned and backtracks when there is none, a node being processed (finished) when the search backtracks from it (§9.3).

dormant set: in the Hao-Orlin algorithm, a set of nodes put aside, without relabelling them, because the current sink can no longer be reached from them in the residual digraph; the sets are woken up in the reverse order of their creation (§15.2).

dual ray: a solution of the dual LP with its right-hand sides (the objective coefficients of the primal) set to 0 and a positive objective value, which proves the primal problem infeasible (§20.4). (Replaces the entry of `Y-glossary.md`, which sets the primal bounds to 0 instead.)

dual scale: the constant (4 for integer weights, 1 otherwise) by which the weighted matching classes multiply their dual values, so that the halvings of the algorithm stay exact (§17.5).

edge connectivity: the least number of edges whose removal disconnects an undirected graph, i.e., its minimum cut with unit capacities (§15.1).

elevator: the data structure of the labels of the push-relabel algorithms, which keeps the items by level with the active ones first and gives the highest active item in constant time (§8.3).

eligible arc: in the network simplex method, a nonbasic arc whose reduced cost is negative at its lower bound or positive at its upper bound, which may enter the basis (§13.4).

epsilon-optimality: the property of a flow and of potentials under which no residual arc has a reduced cost below the opposite of a given epsilon (§13.6).

Euler tour: a closed walk that traverses each edge of a graph (each arc of a digraph, in its direction) once (§18.4).

extender: a class template of lemon/bits/ (DigraphExtender, GraphExtender, BpGraphExtender) that completes a base class with the iterators, the maps, the notifiers and the notifying modifications (§A.2).

extreme flow: a vertex of the polyhedron of the flows of a minimum cost flow problem, i.e., a feasible flow whose arcs strictly within their bounds form a forest (§14.4).

factor-critical graph: a graph that has a perfect matching after the removal of any one of its nodes (§17.1).

Fibonacci heap: a heap made of heap-ordered trees whose roots are linked only at the removal of the minimum, with cascading cuts at the decreases (§7.3).

filter: a bool map that decides which nodes, arcs or edges a filtering adaptor shows (§5.2).

forward arc: the arc of an edge that goes from u(e) to v(e), for which direction() is true; the other arc of the edge is the backward one (§3.2).

fractional matching: a solution of the linear relaxation of the matching problem without the odd set inequalities, whose vertices take the values 0, 1/2 and 1 (§17.1).

function-type interface: a function (e.g., dijkstra()) that returns a wizard object, on which the optional outputs of an algorithm are given by name before a single run (§3.6).

fundamental cut: for an edge e of a spanning forest T, the set of the edges with one end in each of the two node sets into which $T \setminus \{e\}$ splits the component of e (§11.1).

Gallai–Edmonds decomposition: the partition of the nodes into those left uncovered by some maximum matching (D), their other neighbours (A) and the rest (C), which MaxMatching returns as the status of each node (§17.1).

gap heuristic: the lift to the top level of all the items above a level that has become empty, which can no longer reach the sink (§8.3).

GEQ and LEQ forms: the minimum cost flow problem with supply constraints "greater or equal" (all the supplies leave their nodes, some demand may be unmet) or "less or equal" (all the demands are met, some supply may stay) (§13.1).

Gomory-Hu tree: a tree on the nodes of an undirected graph, with a weight on each edge, such that the minimum cut between two nodes is the lightest edge of the path between them in the tree, and the components left by removing that edge are the two sides of the cut (§15.1).

graph adaptor: a class that presents a digraph or a graph as another one (filtered, reversed, without orientation, residual, split) by translating each query, without copying it (§5.1).

graph6: the format of the nauty package that encodes an undirected graph in one line of printable characters, six bits of the upper triangle of the adjacency matrix per character (§21.6).

handle: a small value object (Node, Arc, Edge, RedNode, BlueNode) that identifies an item of a graph, meaningful only together with its graph (§3.2).

heap order: the invariant of a heap-ordered tree that no item has a smaller priority than its parent (§7.2).

identifier: the integer that a graph gives to an item, unique among the items of its kind and immutable while the item exists, used as the index of the maps (§3.2).

implicit graph: a graph structure that stores only its sizes and computes its items from their indices, such as FullGraph, GridGraph and HypercubeGraph (§4.5).

inherent orientation: the orientation of an edge from u(e) to v(e), given by the order of the end nodes at its creation (from red to blue in a bipartite graph) (§3.2).

INVALID: the only object of the type Invalid, to which each handle and iterator converts, and which marks the end of an iteration or the absence of an item (§3.2).

iterable map: a graph map that also lists, in time proportional to their number, the items that have a given value (§6.6).

Kempe chain: a connected component of the subgraph induced by the nodes of two given colors, whose colors can be swapped without spoiling a coloring (§18.6).

Kuratowski subdivision: a subdivision of K_5 or of $K_{3,3}$ contained in a graph, which certifies that the graph is not planar (§18.5).

label: in an LGF file, the value of the map named `label`, which identifies a node or an arc in the rest of the file (§21.1).

laminar family: a family of sets any two of which are disjoint or nested, e.g., the sets contracted by the algorithm of the minimum cost arborescence (§11.2).

Lawler's partition: the splitting of a set of solutions, once its best one is known, into parts in which the elements of that solution are fixed in turn, used to find the k best solutions in order (§14.1, §14.3).

LGF: the LEMON Graph Format, a column-oriented text format made of sections (`@nodes`, `@arcs`, `@edges`, `@attributes`, `@red_nodes`, `@blue_nodes` and free extra sections) (§21.1).

low point: in a depth-first search, the least index of discovery of a node reachable from a given node by going down the tree and then along at most one non-tree edge (§18.2).

map adaptor: a map whose values are computed at each read from one or two other maps, a constant or a functor, which it keeps by reference (§6.4).

matching: a set of edges no two of which share an end; it is perfect when it covers every node (§17.1).

maximum adjacency ordering: an ordering of the nodes of an undirected graph in which each node maximizes the capacity of its edges to the nodes before it; the orderings of the Nagamochi-Ibaraki algorithm (§15.3).

maximum cardinality search: the order of the nodes in which each node, when chosen, has the largest total capacity of arcs to the nodes chosen before it (§9.6).

mean cost (of a cycle): the cost of a directed cycle divided by its number of arcs (§16.1).

metric cost: a cost on the edges of a complete graph that satisfies the triangle inequality $c(u, w) \leq c(u, v)+c(v, w)$ (§19.1).

minimum cut (global): a nonempty proper subset of the nodes whose leaving arcs (or boundary edges, in an undirected graph) have the least total capacity, with no source and no sink fixed (§15.1).

named template parameter: a nested template of an algorithm class (e.g., SetDistMap< M >) whose member Create is the class with one type of its traits replaced (§3.6).

negative cycle: a directed cycle of negative total length, whose presence makes the distances of the nodes reachable from it $-\infty$ (§10.4).

observer: an object of a class derived from AlterationNotifier::ObserverBase, attached to a notifier, whose functions add, erase, build and clear are called at each change of the graph (§A.3).

odd set inequalities: the constraints that a set S of odd cardinality holds at most $(|S| - 1)/2$ edges of a matching, which with the degree constraints describe the matching polytope (§17.1).

operation traits: the class of the operations (`zero()`, `plus()`, `less()` and, for Bellman-Ford, `infinity()`) by which `Dijkstra` and `BellmanFord` compute and compare the distances (§10.2).

pairing heap: a heap made of one heap-ordered tree, whose subtrees are joined by the two-pass pairing at the removal of the root (§7.3).

path compression: the rule of a find in a union-find structure that makes each item met on the way to the root point to the root (§8.2).

path dumper: an object that lists the arcs of a path, possibly from the last to the first (when its `RevPathTag` is `True`), returned by the `path()` queries of the search algorithms and meant to be assigned to a path class (§8.1).

perfect elimination order: an order of the nodes of a graph in which the neighbours of each node that follow it form a clique; a graph has one if and only if it is chordal (§9.6).

planar graph: a graph that can be drawn in the plane with no two edges crossing (§18.5).

policy graph: in Howard's algorithm, the subgraph made of one chosen leaving arc for each node, in which each weakly connected part contains one cycle (§16.4).

potentials: the dual variables of a minimum cost flow problem, one per node; the reduced cost of an arc is its cost plus the potential of its tail minus that of its head (§13.1).

preflow: a map within the bounds of the arcs whose excess, the flow into a node less the flow out of it, is nonnegative at every node other than the source (§12.2).

primal ray: a solution of the primal LP with its finite bounds set to 0 and an improving objective value, which proves the dual problem infeasible (§20.4).

primal scale: the constant 2 by which the fractional matching classes multiply their primal values, so that the half-valued edges are reported as integers (§17.6).

priority queue: the abstract data type of a set of items with priorities, with the item of least priority, its removal and the change of a priority as operations; the heaps implement it (§7.1).

problem type: the outcome of a solved LP or MIP (`UNDEFINED`, `INFEASIBLE`, `FEASIBLE`, `OPTIMAL` or `UNBOUNDED`), read after `solve()` by `primalType()`, `dualType()` or `type()` (§20.4).

radix heap: a heap of integer priorities, monotone in the minimum, whose items are kept in boxes of widths 1, 1, 2, 4, ... from the current minimum (§7.4).

range constraint: a row of a linear program with both a finite lower and a finite upper bound, written in the interface as `(s <= e) <= t` (§20.1).

range identifier: the identifier in $[0, n)$ that a RangeIdMap gives to each of the n items of a kind, kept contiguous under erasures (§6.5).

realizer: a partition of the inner edges of a triangulation into three trees rooted at the nodes of the outer face, from which Schnyder's method computes the coordinates of a straight-line drawing (§18.5).

reduced length: the length $\ell(uv) + \pi(u) - \pi(v)$ of an arc under a potential π, which changes the length of all the paths between two nodes by the same amount (§10.1).

reference map: a read-write map whose operator[] returns a reference to the value, recognized by its tag ReferenceMapTag (§6.1).

relaxation: the update of the tentative distance of the target of an arc to the distance of its source plus the length of the arc, when this is smaller (§10.2).

residual digraph: for a flow on a digraph, the digraph of the forward arcs below their capacity and of the backward arcs of positive flow, each with its residual capacity (§5.4).

residual network: for a flow, the network of the arcs along which flow may still be sent, forward up to the upper bound and backward down to the lower bound, the backward arcs with the opposite cost (§12.1, §13.1).

shortcut: of a closed walk that visits every node of a complete graph, the tour that lists the nodes in the order of their first occurrence in the walk (§19.1).

smallest-last order: the order in which the nodes are removed from a graph by always taking a node of least degree among the remaining ones (§18.6).

smallest-last ordering: the order of the nodes obtained by removing, one at a time, a node of least degree in the graph that remains, read backwards (§6.6).

snapshot: a saved state of a list or smart graph, to which restore() brings the graph back by erasing the items added after it (§4.2).

spanning forest: a set of edges with no cycle that connects the nodes of each connected component of a graph; a spanning tree when the graph is connected (§11.1).

spanning tree: a set of $n - 1$ edges of a connected graph with no cycle, which therefore connects all the nodes (§11.1).

splay tree: the self-adjusting binary search tree of Sleator and Tarjan, used by DynArcLookUp, whose operations take logarithmic amortized time (§4.8).

static graph: a graph structure built in one call and not modifiable afterwards, which stores the arcs out of each node contiguously (StaticDigraph, StaticGraph, CompactDigraph) (§4.4).

strongly connected component: a class of the nodes of a digraph joined by directed paths in both directions (§18.3).

strongly feasible tree: in the network simplex method, a spanning tree of a basis along which a positive amount of flow can be sent from each node to the root within the bounds; the rule of the last blocking arc keeps it so and prevents cycling (§13.4).

subgraph isomorphism: an injective map of the nodes of a graph into those of another that takes each edge to an edge; it is induced if it also takes each non-edge to a non-edge (§18.7).

successive shortest paths: the algorithm of minimum cost flow that sends flow along shortest paths of the residual digraph, keeping the reduced lengths nonnegative by potentials, used by `Suurballe` (§10.5).

tolerance: the object of class Tolerance< T > that compares numbers allowing for an absolute error up to its epsilon, exactly for the exact types (§3.7).

topological order: an order of the nodes of an acyclic digraph in which the source of each arc precedes its target, e.g., the reverse of the finishing order of a depth-first search (§9.3).

tour: a Hamiltonian cycle of a complete graph, i.e., a cycle that visits each node once; the solution of the travelling salesman problem (§19.1).

traits class: the class, a template parameter of an algorithm, that defines the types the algorithm uses (heap, maps, operations) and creates their objects (§3.6).

unbounded cycle: a directed cycle of negative cost whose arcs have infinite upper bounds, which proves a feasible minimum cost flow problem unbounded (§14.2).

union-find structure: a partition of a set of items into classes under the union of two classes and the find of the representative of the class of an item, stored as a forest with union by size and path compression (§8.2).

vertex cover: a set of nodes containing at least one end of every edge; in a bipartite graph its least cardinality equals that of a maximum matching (§17.1).

violation: of a set of nodes, its supply less the upper bounds of the arcs leaving it plus the lower bounds of the arcs entering it; a positive violation proves a minimum cost flow problem infeasible (§14.1).

visitor: an object whose members a search with a visitor interface (`BfsVisit`, `DfsVisit`) calls at each event of the search (§9.5).

wizard: the object returned by a function-type interface (e.g., `bfs()`, `dijkstra()`), on which the maps and the outputs are given by named function parameters before `run()` (§9.4).

Z. Bibliography

The works that the headers of the library cite, those that the README and NEWS name, and those that the chapters cite, in the alphabetical order of their first authors. The anchor of each entry is the key under which the chapters cite it.

Ravindra K. Ahuja, Kurt Mehlhorn, James B. Orlin and Robert E. Tarjan. Faster algorithms for the shortest path problem. Journal of the ACM, 37(2):213–223, 1990. doi:10.1145/77600.77615

Ravindra K. Ahuja, Thomas L. Magnanti and James B. Orlin. Network Flows: Theory, Algorithms, and Applications. Prentice-Hall, 1993.

K. Appel and W. Haken. Every planar map is four colorable. Part I: Discharging. Illinois Journal of Mathematics, 21(3):429–490, 1977. doi:10.1215/ijm/1256049011

Richard Bellman. On a routing problem. Quarterly of Applied Mathematics, 16(1):87–90, 1958. doi:10.1090/qam/102435

Claude Berge. Two theorems in graph theory. Proceedings of the National Academy of Sciences of the United States of America, 43(9):842–844, 1957. doi:10.1073/pnas.43.9.842

D. Bertsimas and J. N. Tsitsiklis. Introduction to Linear Optimization. Athena Scientific, Belmont, MA, 1997.

George E. P. Box and Mervin E. Muller. A note on the generation of random normal deviates. The Annals of Mathematical Statistics, 29(2):610–611, 1958. doi:10.1214/aoms/1177706645

J. M. Boyer and W. J. Myrvold. On the cutting edge: simplified O(n) planarity by edge addition. Journal of Graph Algorithms and Applications, 8(3):241–273, 2004. doi:10.7155/jgaa.00091

Ursula Bünnagel, Bernhard Korte and Jens Vygen. Efficient implementation of the Goldberg-Tarjan minimum-cost flow algorithm. Optimization Methods and Software, 10(2):157–174, 1998. doi:10.1080/10556789808805709

Cbc, the COIN-OR Branch and Cut solver. https://github.com/coin-or/Cbc

Joseph Cheriyan and S. N. Maheshwari. Analysis of preflow push algorithms for maximum network flow. SIAM Journal on Computing, 18(6):1057–1086, 1989. doi:10.1137/0218072

Boris V. Cherkassky and Andrew V. Goldberg. On implementing the push-relabel method for the maximum flow problem. Algorithmica, 19(4):390–410, 1997. doi:10.1007/PL00009180

N. Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. Report 388, Graduate School of Industrial Administration, Carnegie Mellon University, 1976. Reprinted in Operations Research Forum, 3(1):20, 2022. doi:10.1007/s43069-021-00101-z

Yoeng-Jin Chu and Tseng-Hong Liu. On the shortest arborescence of a directed graph. Scientia Sinica, 14:1396–1400, 1965.

Clp, the COIN-OR Linear Programming solver. https://github.com/coin-or/Clp

Jean Cochet-Terrasson, Guy Cohen, Stéphane Gaubert, Michael McGettrick and Jean-Pierre Quadrat. Numerical computation of spectral elements in max-plus algebra. IFAC Proceedings Volumes, 31(18):667–674, 1998. doi:10.1016/S1474-6670(17)42067-2

L. P. Cordella, P. Foggia, C. Sansone and M. Vento. A (sub)graph isomorphism algorithm for matching large graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence, 26(10):1367–1372, 2004. doi:10.1109/TPAMI.2004.75

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest and Clifford Stein. Introduction to Algorithms, 2nd edition. The MIT Press, 2001.

G. A. Croes. A method for solving traveling-salesman problems. Operations Research, 6(6):791–812, 1958. doi:10.1287/opre.6.6.791

William H. Cunningham. A network simplex method. Mathematical Programming, 11(1):105–116, 1976. doi:10.1007/BF01580379

George B. Dantzig. Linear Programming and Extensions. Princeton University Press, 1963.

Ali Dasdan. Experimental analysis of the fastest optimum cycle ratio and mean algorithms. ACM Transactions on Design Automation of Electronic Systems, 9(4):385–418, 2004. doi:10.1145/1027084.1027085

Ali Dasdan and Rajesh K. Gupta. Faster maximum and minimum mean cycle algorithms for system-performance analysis. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, 17(10):889–899, 1998. doi:10.1109/43.728912

Eric V. Denardo and Bennett L. Fox. Multichain Markov renewal programs. SIAM Journal on Applied Mathematics, 16(3):468–487, 1968. doi:10.1137/0116038

B. Dezső, A. Jüttner and P. Kovács. LEMON – an open source C++ graph template library. Electronic Notes in Theoretical Computer Science, 264(5):23–45, 2011. doi:10.1016/j.entcs.2011.06.003

Robert B. Dial. Algorithm 360: shortest-path forest with topological ordering. Communications of the ACM, 12(11):632–633, 1969. doi:10.1145/363269.363610

Reinhard Diestel. Graph Theory, 5th edition. Graduate Texts in Mathematics 173, Springer, 2017. doi:10.1007/978-3-662-53622-3

Edsger W. Dijkstra. A note on two problems in connexion with graphs. Numerische Mathematik, 1(1):269–271, 1959. doi:10.1007/BF01386390

Jack Edmonds. Maximum matching and a polyhedron with 0,1-vertices. Journal of Research of the National Bureau of Standards, Section B, 69B(1–2):125–130, 1965. doi:10.6028/jres.069B.013

Jack Edmonds. Paths, trees, and flowers. Canadian Journal of Mathematics, 17:449–467, 1965. doi:10.4153/CJM-1965-045-4

Jack Edmonds. Optimum branchings. Journal of Research of the National Bureau of Standards, 71B(4):233–240, 1967. doi:10.6028/jres.071B.032

Jack Edmonds and Richard M. Karp. Theoretical improvements in algorithmic efficiency for network flow problems. Journal of the ACM, 19(2):248–264, 1972. doi:10.1145/321694.321699

M. Englert, H. Röglin, and B. Vöcking. Worst case and probabilistic analysis of the 2-Opt algorithm for the TSP. Algorithmica, 68(1):190–264, 2014. doi:10.1007/s00453-013-9801-4

Lester R. Ford and Delbert R. Fulkerson. Maximal flow through a network. Canadian Journal of Mathematics, 8:399–404, 1956. doi:10.4153/CJM-1956-045-5

Lester R. Ford, Jr. Network flow theory. Paper P-923, The RAND Corporation, Santa Monica, 1956.

Michael L. Fredman. On the efficiency of pairing heaps and related data structures. Journal of the ACM, 46(4):473–501, 1999. doi:10.1145/320211.320214

Michael L. Fredman and Robert E. Tarjan. Fibonacci heaps and their uses in improved network optimization algorithms. Journal of the ACM, 34(3):596–615, 1987. doi:10.1145/28869.28874

Michael L. Fredman, Robert Sedgewick, Daniel D. Sleator and Robert E. Tarjan. The pairing heap: a new form of self-adjusting heap. Algorithmica, 1(1):111–129, 1986. doi:10.1007/BF01840439

Harold N. Gabow, Zvi Galil, Thomas Spencer and Robert E. Tarjan. Efficient algorithms for finding minimum spanning trees in undirected and directed graphs. Combinatorica, 6(2):109–122, 1986. doi:10.1007/BF02579168

David Gale. A theorem on flows in networks. Pacific Journal of Mathematics, 7(2):1073–1082, 1957. doi:10.2140/pjm.1957.7.1073

Zvi Galil. Efficient algorithms for finding maximum matching in graphs. ACM Computing Surveys, 18(1):23–38, 1986. doi:10.1145/6462.6502

Zvi Galil, Silvio Micali and Harold Gabow. An O(EV log V) algorithm for finding a maximal weighted matching in general graphs. SIAM Journal on Computing, 15(1):120–130, 1986. doi:10.1137/0215009

Tibor Gallai. Maximale Systeme unabhängiger Kanten. A Magyar Tudományos Akadémia Matematikai Kutató Intézetének Közleményei, 9:401–413, 1964.

M. R. Garey, D. S. Johnson and L. Stockmeyer. Some simplified NP-complete graph problems. Theoretical Computer Science, 1(3):237–267, 1976. doi:10.1016/0304-3975(76)90059-1

GLPK, the GNU Linear Programming Kit. https://www.gnu.org/software/glpk/

Andrew V. Goldberg. An efficient implementation of a scaling minimum-cost flow algorithm. Journal of Algorithms, 22(1):1–29, 1997. doi:10.1006/jagm.1995.0805

Andrew V. Goldberg and Robert E. Tarjan. A new approach to the maximum-flow problem. Journal of the ACM, 35(4):921–940, 1988. doi:10.1145/48014.61051

Andrew V. Goldberg and Robert E. Tarjan. Finding minimum-cost circulations by canceling negative cycles. Journal of the ACM, 36(4):873–886, 1989. doi:10.1145/76359.76368

Andrew V. Goldberg and Robert E. Tarjan. Finding minimum-cost circulations by successive approximation. Mathematics of Operations Research, 15(3):430–466, 1990. doi:10.1287/moor.15.3.430

Ralph E. Gomory and T. C. Hu. Multi-terminal network flows. Journal of the Society for Industrial and Applied Mathematics, 9(4):551–570, 1961. doi:10.1137/0109047

A. Grosso, M. Locatelli, and W. Pullan. Simple ingredients leading to very efficient heuristics for the maximum clique problem. Journal of Heuristics, 14(6):587–612, 2008. doi:10.1007/s10732-007-9055-x

Dan Gusfield. Very simple methods for all pairs network flow analysis. SIAM Journal on Computing, 19(1):143–155, 1990. doi:10.1137/0219009

Horst W. Hamacher. A note on K best network flows. Annals of Operations Research, 57(1):65–72, 1995. doi:10.1007/BF02099691

Horst W. Hamacher, Jean-Claude Picard and Maurice Queyranne. On finding the K best cuts in a network. Operations Research Letters, 2(6):303–305, 1984. doi:10.1016/0167-6377(84)90083-X

J. Hao and James B. Orlin. A faster algorithm for finding the minimum cut in a directed graph. Journal of Algorithms, 17(3):424–446, 1994. doi:10.1006/jagm.1994.1043

Peter E. Hart, Nils J. Nilsson and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. IEEE Transactions on Systems Science and Cybernetics, 4(2):100–107, 1968. doi:10.1109/TSSC.1968.300136

Mark Hartmann and James B. Orlin. Finding minimum cost to time ratio cycles with small integral transit times. Networks, 23(6):567–574, 1993. doi:10.1002/net.3230230607

P. J. Heawood. Map-colour theorem. The Quarterly Journal of Pure and Applied Mathematics, 24:332–338, 1890.

C. Hierholzer and C. Wiener. Ueber die Möglichkeit, einen Linienzug ohne Wiederholung und ohne Unterbrechung zu umfahren. Mathematische Annalen, 6(1):30–32, 1873. doi:10.1007/BF01442866

Alan J. Hoffman. Some recent applications of the theory of linear inequalities to extremal combinatorial analysis. In Combinatorial Analysis, Proceedings of Symposia in Applied Mathematics, 10:113–127, 1960. doi:10.1090/psapm/010/0114759

J. Hopcroft and R. Tarjan. Algorithm 447: efficient algorithms for graph manipulation. Communications of the ACM, 16(6):372–378, 1973. doi:10.1145/362248.362272

John E. Hopcroft and Richard M. Karp. An n^{5/2} algorithm for maximum matchings in bipartite graphs. SIAM Journal on Computing, 2(4):225–231, 1973. doi:10.1137/0202019

Ronald A. Howard. Dynamic Programming and Markov Processes. MIT Press, 1960.

D. S. Johnson and C. C. McGeoch (editors). Network Flows and Matching: First DIMACS Implementation Challenge. DIMACS Series in Discrete Mathematics and Theoretical Computer Science, volume 12. American Mathematical Society, 1993. doi:10.1090/dimacs/012

Donald B. Johnson. Priority queues with update and finding minimum spanning trees. Information Processing Letters, 4(3):53–57, 1975. doi:10.1016/0020-0190(75)90001-0

A. Jüttner and P. Madarasi. VF2++: an improved subgraph isomorphism algorithm. Discrete Applied Mathematics, 242:69–81, 2018. doi:10.1016/j.dam.2018.02.018

Richard M. Karp. A characterization of the minimum cycle mean in a digraph. Discrete Mathematics, 23(3):309–311, 1978. doi:10.1016/0012-365X(78)90011-0

Damian J. Kelly and Garrett M. O'Neill. The Minimum Cost Flow Problem and The Network Simplex Method. Master's thesis, University College, Dublin, 1991.

A. B. Kempe. On the geographical problem of the four colours. American Journal of Mathematics, 2(3):193–200, 1879. doi:10.2307/2369235

Zoltán Király and Péter Kovács. Efficient implementations of minimum-cost flow algorithms. Acta Universitatis Sapientiae, Informatica, 4:67–118, 2012.

Morton Klein. A primal method for minimal cost flows with applications to the assignment and transportation problems. Management Science, 14(3):205–220, 1967. doi:10.1287/mnsc.14.3.205

Donald E. Knuth. The Art of Computer Programming, Volume 2: Seminumerical Algorithms. Addison-Wesley, 1969.

Dénes König. Gráfok és mátrixok. Matematikai és Fizikai Lapok, 38:116–119, 1931.

Joseph B. Kruskal. On the shortest spanning subtree of a graph and the traveling salesman problem. Proceedings of the American Mathematical Society, 7(1):48–50, 1956. doi:10.1090/S0002-9939-1956-0078686-7

C. Kuratowski. Sur le problème des courbes gauches en topologie. Fundamenta Mathematicae, 15:271–283, 1930. doi:10.4064/fm-15-1-271-283

Eugene L. Lawler. A procedure for computing the K best solutions to discrete optimization problems and its application to the shortest path problem. Management Science, 18(7):401–405, 1972. doi:10.1287/mnsc.18.7.401

George Marsaglia and Thomas A. Bray. A convenient method for generating normal variables. SIAM Review, 6(3):260–264, 1964. doi:10.1137/1006063

Makoto Matsumoto and Takuji Nishimura. Mersenne twister: a 623-dimensionally equidistributed uniform pseudo-random number generator. ACM Transactions on Modeling and Computer Simulation, 8(1):3–30, 1998. doi:10.1145/272991.272995

David W. Matula and Leland L. Beck. Smallest-last ordering and clustering and graph coloring algorithms. Journal of the ACM, 30(3):417–427, 1983. doi:10.1145/2402.322385

B. D. McKay and A. Piperno. Practical graph isomorphism, II. Journal of Symbolic Computation, 60:94–112, 2014. doi:10.1016/j.jsc.2013.09.003

Karl Menger. Zur allgemeinen Kurventheorie. Fundamenta Mathematicae, 10:96–115, 1927. doi:10.4064/fm-10-1-96-115

Katta G. Murty. Solving the fixed charge problem by ranking the extreme points. Operations Research, 16(2):268–279, 1968. doi:10.1287/opre.16.2.268

Hiroshi Nagamochi and Toshihide Ibaraki. Computing edge-connectivity in multigraphs and capacitated graphs. SIAM Journal on Discrete Mathematics, 5(1):54–66, 1992. doi:10.1137/0405004

Seth Pettie. Towards a final analysis of pairing heaps. In 46th Annual IEEE Symposium on Foundations of Computer Science (FOCS 2005), pages 174–183, 2005. doi:10.1109/SFCS.2005.75

Jean-Claude Picard and Maurice Queyranne. On the structure of all minimum cuts in a network and applications. Mathematical Programming Study, 13:8–16, 1980. doi:10.1007/BFb0120902

Ira Pohl. Bi-directional search. Machine Intelligence, 6:127–140, 1971.

Wayne Pullan and Holger H. Hoos. Dynamic local search for the maximum clique problem. Journal of Artificial Intelligence Research, 25:159–185, 2006. doi:10.1613/jair.1815

D. J. Rosenkrantz, R. E. Stearns, and P. M. Lewis II. An analysis of several heuristics for the traveling salesman problem. SIAM Journal on Computing, 6(3):563–581, 1977. doi:10.1137/0206041

W. Schnyder. Embedding planar graphs on the grid. Proceedings of the First Annual ACM-SIAM Symposium on Discrete Algorithms (SODA '90), 138–148, 1990.

Alexander Schrijver. Combinatorial Optimization: Polyhedra and Efficiency. Springer-Verlag, 2003.

M. Sharir. A strong-connectivity algorithm and its applications in data flow analysis. Computers & Mathematics with Applications, 7(1):67–72, 1981. doi:10.1016/0898-1221(81)90008-0

D. D. Sleator and R. E. Tarjan. Self-adjusting binary search trees. Journal of the ACM, 32(3):652–686, 1985. doi:10.1145/3828.3835

SoPlex, the Sequential object-oriented simPlex. https://soplex.zib.de/

Mechthild Stoer and Frank Wagner. A simple min-cut algorithm. Journal of the ACM, 44(4):585–591, 1997. doi:10.1145/263867.263872

IBM ILOG CPLEX Optimization Studio. https://www.ibm.com/products/ilog-cplex-optimization-studio

John W. Suurballe. Disjoint paths in a network. Networks, 4(2):125–145, 1974. doi:10.1002/net.3230040204

John W. Suurballe and Robert E. Tarjan. A quick method for finding shortest pairs of disjoint paths. Networks, 14(2):325–336, 1984. doi:10.1002/net.3230140209

Robert E. Tarjan. Depth-first search and linear graph algorithms. SIAM Journal on Computing, 1(2):146–160, 1972. doi:10.1137/0201010

Robert E. Tarjan. Efficiency of a good but not linear set union algorithm. Journal of the ACM, 22(2):215–225, 1975. doi:10.1145/321879.321884

Robert E. Tarjan. Data Structures and Network Algorithms. SIAM, 1983. doi:10.1137/1.9781611970265

Robert E. Tarjan and Jan van Leeuwen. Worst-case analysis of set union algorithms. Journal of the ACM, 31(2):245–281, 1984. doi:10.1145/62.2160

Robert E. Tarjan and Mihalis Yannakakis. Simple linear-time algorithms to test chordality of graphs, test acyclicity of hypergraphs, and selectively reduce acyclic hypergraphs. SIAM Journal on Computing, 13(3):566–579, 1984. doi:10.1137/0213035

E. Tomita and T. Seki. An efficient branch-and-bound algorithm for finding a maximum clique. In Discrete Mathematics and Theoretical Computer Science (DMTCS 2003), Lecture Notes in Computer Science, volume 2731, pages 278–289. Springer, 2003. doi:10.1007/3-540-45066-1_22

N. Tomizawa. On some techniques useful for solution of transportation network problems. Networks, 1(2):173–194, 1971. doi:10.1002/net.3230010206

William T. Tutte. The factorization of linear graphs. Journal of the London Mathematical Society, s1–22(2):107–111, 1947. doi:10.1112/jlms/s1-22.2.107

Jean Vuillemin. A data structure for manipulating priority queues. Communications of the ACM, 21(4):309–315, 1978. doi:10.1145/359460.359478

J. W. J. Williams. Algorithm 232: Heapsort. Communications of the ACM, 7(6):347–348, 1964. doi:10.1145/512274.3734138